Programming Language Semantics
A Rewriting Approach

Grigore Roșu

University of Illinois at Urbana-Champaign

Contents

1 Introduction **7**

2 Background and Preliminaries **15**

2.1 Mathematical Background . . . 16
2.1.1 Functions . . . 16
2.1.2 Partial Functions . . . 17
2.1.3 Context-Free Grammars and the Mixfix Algebraic Notation . . . 18
2.1.4 Basic Term Rewriting . . . 19
2.1.5 On Logics and Logic Deduction . . . 20
2.2 Basic Computability Elements . . . 22
2.2.1 Turing Machines . . . 22
2.2.2 Universal Machines, the Halting Problem, and Decision Problems . . . 24
2.2.3 The Arithmetic Hierarchy . . . 25
2.2.4 Notes . . . 26
2.2.5 Exercises . . . 26
2.3 Propositional Logic . . . 27
2.3.1 Syntax and Semantics . . . 27
2.3.2 Proof System and Soundness . . . 28
2.3.3 Consistency and Model Existence . . . 29
2.3.4 Completeness and Compactness . . . 30
2.3.5 Notes . . . 30
2.3.6 Exercises . . . 31
2.4 Equational Logic . . . 32
2.4.1 Signatures, Algebras, Equations, Satisfaction . . . 32
2.4.2 Proof System and Soundness . . . 32
2.4.3 Computation as Equational Deduction . . . 33
2.4.4 Initial Algebra . . . 36
2.4.5 Completeness . . . 37
2.4.6 Defining Data-Types Equationally . . . 42
2.4.7 Notes . . . 49
2.4.8 Exercises . . . 49
2.5 Rewrite logic . . . 50
2.5.1 Equations versus Rewrite Rules . . . 50
2.5.2 Proof System and Soundness . . . 55
2.5.3 Computation as Rewrite Logic Deduction . . . 58

2.5.4 Completeness . . . 61
2.5.5 Rewrite Logic as a Universal Logical Formalism . . . 61
2.5.6 Maude: A High Performance Rewrite Logic System . . . 62
2.5.7 Exercises . . . 82
2.5.8 Notes . . . 84
2.6 Complete Partial Orders and the Fixed-Point Theorem . . . 85
2.6.1 Posets, Upper Bounds and Least Upper Bounds . . . 85
2.6.2 Complete Partial Orders . . . 86
2.6.3 Monotone and Continuous Functions . . . 87
2.6.4 The Fixed-Point Theorem . . . 89
2.6.5 Equational Framework for Fixed-Points . . . 90
2.6.6 Exercises . . . 92
2.6.7 Notes . . . 92
2.7 Category Theory . . . 95

3 Executable Semantics Approaches 101
3.1 IMP: A Simple Imperative Language . . . 109
3.1.1 IMP Syntax . . . 109
3.1.2 IMP State . . . 114
3.1.3 Notes . . . 115
3.2 Big-Step Structural Operational Semantics (Big-Step SOS) . . . 117
3.2.1 IMP Configurations for Big-Step SOS . . . 118
3.2.2 The Big-Step SOS Rules of IMP . . . 119
3.2.3 Big-Step SOS in Rewrite Logic . . . 123
3.2.4 Defining a Type System for IMP Using Big-Step SOS . . . 130
3.2.5 Notes . . . 135
3.2.6 Exercises . . . 137
3.3 Small-Step Structural Operational Semantics (Small-Step SOS) . . . 139
3.3.1 IMP Configurations for Small-Step SOS . . . 140
3.3.2 The Small-Step SOS Rules of IMP . . . 141
3.3.3 Small-Step SOS in Rewrite Logic . . . 147
3.3.4 Notes . . . 153
3.3.5 Exercises . . . 154
3.4 Denotational Semantics . . . 157
3.4.1 The Denotational Semantics of IMP . . . 162
3.4.2 Denotational Semantics in Equational/Rewrite Logic . . . 162
3.4.3 Notes . . . 165
3.4.4 Exercises . . . 168
3.5 IMP++: IMP Extended with Several Features . . . 171
3.5.1 Adding Variable Increment . . . 172
3.5.2 Adding Input/Output . . . 176
3.5.3 Adding Abrupt Termination . . . 181
3.5.4 Adding Dynamic Threads . . . 188
3.5.5 Adding Local Variables . . . 193
3.5.6 Putting Them All Together: First Attempt . . . 196
3.5.7 Putting Them All Together: Second Attempt . . . 204

3.5.8 Notes . . . 204
3.5.9 Exercises . . . 205
3.6 Modular Structural Operational Semantics (MSOS) . . . 213
3.6.1 The MSOS of IMP . . . 218
3.6.2 The MSOS of IMP++ . . . 218
3.6.3 MSOS in Rewrite Logic . . . 224
3.6.4 Notes . . . 230
3.6.5 Exercises . . . 230
3.7 Reduction Semantics with Evaluation Contexts . . . 233
3.7.1 The Reduction Semantics with Evaluation Contexts of IMP . . . 236
3.7.2 The Reduction Semantics with Evaluation Contexts of IMP++ . . . 239
3.7.3 Reduction Semantics with Evaluation Contexts in Rewrite Logic . . . 242
3.7.4 Notes . . . 263
3.7.5 Exercises . . . 264
3.8 The Chemical Abstract Machine (CHAM) . . . 269
3.8.1 The CHAM of IMP . . . 272
3.8.2 The CHAM of IMP++ . . . 278
3.8.3 CHAM in Rewrite Logic . . . 281
3.8.4 Notes . . . 291
3.8.5 Exercises . . . 293
3.9 Advanced Denotational Semantics: Continuations, Monads and Powerdomains . . . 295
3.10 Discussion: Advantages, Disadvantages and Comparisons . . . 296
3.11 Existing Approaches and Their Limitations for Programming Language Semantics . . . 296
3.11.1 Structural Operational Semantics (SOS) Frameworks . . . 296
3.11.2 Rewriting Frameworks . . . 297
3.11.3 *old limitations stuff about big-step and small-step SOS* . . . 299
3.11.4 *old stuff about MSOS* . . . 303
3.11.5 *old stuff about context reduction* . . . 303
3.11.6 *other old stuff about context reduction* . . . 305
3.11.7 *older stuff about cham* . . . 308
3.12 K . . . 309
3.13 Notes . . . 310

4 Idealized Programming Languages, Features, and Models of Computation 311
4.1 Combinatory Logic . . . 312
4.2 Turing Machines . . . 317
4.2.1 Notes . . . 317
4.3 Post Correspondence . . . 318
4.4 Counter Machines . . . 319
4.5 Lambda Calculus . . . 320
4.5.1 Syntax . . . 320
4.5.2 Free and Bound Variables . . . 321
4.5.3 Substitution . . . 321
4.5.4 Alpha Conversion . . . 322
4.5.5 Beta Equivalence and Beta Reduction . . . 322
4.5.6 Lambda Calculus as a Programming Language . . . 324

4.5.7 Fixed-Points and Recursion . . . 328
4.5.8 Eta Equivalence and Extensionality . . . 331
4.5.9 Combinatory Logic . . . 332
4.5.10 De Bruijn Nameless Representation . . . 336
4.5.11 Notes . . . 337
4.5.12 Exercises . . . 337
4.6 Simply-Typed Lambda Calculs . . . 339
4.6.1 Syntax . . . 339
4.6.2 Terms and Typye Checking . . . 339
4.6.3 Equational Rules . . . 341
4.6.4 Models . . . 343
4.6.5 Full Type Frame . . . 344
4.6.6 Completeness . . . 346
4.6.7 Cartesian Closed Categories as Simply-Typed Lambda-Calculus Models . . . 347
4.7 Recursion and PCF . . . 353
4.7.1 Transitional semantics . . . 361
4.7.2 Natural Semantics . . . 363
4.7.3 Denotational Semantics of PCF . . . 364
4.8 Polymorphims . . . 366

A Maude Stuff **395**
A.1 Defining "Builtin" Modules for Maude Language Definitions . . . 395

---

take into account Peter's comments on both my lecture notes and the previous draft on K

use either rewrite logic or rewriting logic everywhere

Chapter 1

Introduction

This paper is an informal introduction to K, a rewriting-based language definitional framework. By saying that K is a "rewriting-based" framework, in contrast to a "reduction-based" one, we mean that in K rules can be applied concurrently and unrestricted by context, following the basic intuitions and operational/semantical strengths of Meseguer's rewrite logic [42]. K was first introduced by the author in an algebraic form in the lecture notes of a programming language design course at the University of Illinois at Urbana-Champaign in Fall 2003 [64], as a means to define executable concurrent languages in rewrite logic using the Maude executable specification system [17]. A more formal, detailed algebraic description of K can be found in [66].

The major purpose of this report is to explain K using a non-algebraic, intuitive and conventional notation based on context-free grammars enriched with structural equivalences and rewrite rules, as opposed to algebraic signatures, algebraic specifications and rewrite logic theories. The reason for doing so is that we strongly believe that K can also be useful to the working programming language or calculi designer who may not like, may not be familiar with, or simply may not want to use the algebraic notation. An additional purpose for stripping K of its algebraic armor is to reflect the important fact that K is more than a technique to implement interpreters in Maude; it is a language definitional style, like structural operational semantics (SOS) [60, 61], modular SOS (MSOS) [53], reduction semantics with evaluation contexts (context reduction) [25], or the chemical abstract machine (Cham) [9].

Figure 1 shows definitions of a simple imperative language and of some λ-calculi using both context reduction and K. For a fair comparison on the imperative language, we assumed that both definitions initiate the evaluation of a program p by wrapping it into double square brackets, $[\![p]\!]$; then each definition takes that and initiates the configuration in its own way. The syntax of the simple imperative language is:

$$
\begin{array}{rcl}
\mathit{Var} & ::= & \text{standard identifiers} \\
\mathit{Val} & ::= & \mathit{Int} \\
\mathit{AExp} & ::= & \mathit{Var} \mid \mathit{Val} \mid \mathit{AExp} + \mathit{AExp} \\
\mathit{BExp} & ::= & \mathsf{true} \mid \mathsf{false} \mid \mathit{AExp} \leq \mathit{AExp} \mid \mathit{BExp}\ \mathsf{and}\ \mathit{BExp} \mid \mathsf{not}\ \mathit{BExp} \\
\mathit{Stmt} & ::= & \mathit{Var} := \mathit{AExp} \mid \mathit{Stmt}; \mathit{Stmt} \mid \mathsf{if}\ \mathit{BExp}\ \mathsf{then}\ \mathit{Stmt}\ \mathsf{else}\ \mathit{Stmt} \mid \\
 & & \mathsf{while}\ \mathit{BExp}\ \mathsf{do}\ \mathit{Stmt} \mid \mathsf{halt}\ \mathit{AExp} \\
\mathit{Pgm} & ::= & \mathit{AExp} \mid \mathit{Stmt}; \mathit{AExp}
\end{array}
$$

To make it more interesting, we assumed that $_ + _$ is strict but evaluates its arguments non-deterministically (i.e., it may interleave execution steps in its argument expressions), that $_ \leq _$ is strict from left to right (it first evaluates its first argument completely then its second), and that $_\mathsf{and}_$ shortcuts (it evaluates its first argument and it continues to evaluate the second only if the first is true). Only arithmetic expressions can be

Context reduction	K
A simple imperative language	
Configuration and initialization $Config ::= Int \mid [\![Pgm]\!] \mid [\![Pgm, State]\!]$ $[\![p]\!] \to [\![p, \emptyset]\!]$ $[\![v, \sigma]\!] \to v$ $Cxt ::= \square \mid [\![Cxt, State]\!]$	$KResult ::= Val$ $K ::= KResult \mid \mathbf{List}_{\curvearrowright}\{K\}$ $Config ::= Int \mid [\![K]\!] \mid \langle K \rangle_{\mathsf{k}} \mid \langle State \rangle_{\mathsf{state}} \mid \langle \mathbf{Set}\{Config\} \rangle_{\top}$ $[\![p]\!] = \langle \langle p \rangle_{\mathsf{k}}\ \langle \emptyset \rangle_{\mathsf{state}} \rangle_{\top}$ $\langle \cdots \langle v \rangle_{\mathsf{k}} \cdots \rangle_{\top} = v$
Variable lookup $[\![c, \sigma]\!][x] \to [\![c, \sigma]\!][\sigma[x]]$	$\langle \underset{\sigma[x]}{\underline{x}} \cdots \rangle_{\mathsf{k}}\ \langle \sigma \rangle_{\mathsf{state}}$
Operators	
$Cxt ::= \ldots \mid \mathsf{not}\ Cxt$ $\mathsf{not}\ b \to \mathsf{not}_{Bool}\ b$	$\mathsf{not}\ BExp$ [*strict extends* $\mathsf{not}_{Bool_}$]
$Cxt ::= \ldots \mid Cxt\ \mathsf{and}\ BExp$ $\mathsf{true}\ \mathsf{and}\ b \to b$ $\mathsf{false}\ \mathsf{and}\ b \to \mathsf{false}$	$BExp\ \mathsf{and}\ BExp$ [*strict*(1)] $\mathsf{true}\ \mathsf{and}\ b \to b$ $\mathsf{false}\ \mathsf{and}\ b \to \mathsf{false}$
Statements $Stmt ::= \ldots \mid \mathsf{skip}$ $Cxt ::= \ldots \mid Var := Cxt$ $[\![c, \sigma]\!][x := v] \to [\![c, \sigma[v/x]]\!][\mathsf{skip}]$	$Var := AExp$ [*strict*(2)] $\langle \underset{\cdot}{\underline{x := v}} \cdots \rangle_{\mathsf{k}}\ \langle \underset{\sigma[v/x]}{\underline{\sigma}} \rangle_{\mathsf{state}}$
$Cxt ::= \ldots \mid Cxt; Stmt$ $\mathsf{skip}; s \to s$	$k_1; k_2 \rightleftharpoons k_1 \curvearrowright k_2$
$\mathsf{while}\ b\ \mathsf{do}\ s \to$	$\langle \mathsf{while}\ b\ \mathsf{do}\ s \cdots \rangle_{\mathsf{k}} =$
$Cxt ::= \ldots \mid \mathsf{halt}\ Cxt$ $c[\mathsf{halt}\ i] \to i$	$\mathsf{halt}\ AExp$ [*strict*] $\langle \mathsf{halt}\ i \cdots \rangle_{\mathsf{k}} \to \langle i \rangle_{\mathsf{k}}$
Programs $Cxt ::= \ldots \mid Cxt; AExp$ $\mathsf{skip}; a \to a$	
λ-calculus	
$Cxt ::= \lambda Var.Cxt \mid Cxt\ Exp \mid Exp\ Cxt$ $(\lambda x.e)\ e' \to e[e'/x]$	$(\lambda x.e)\ e' \to e[e'/x]$
Call-by-value λ-calculus	
$Val ::= \lambda Var.Exp$ $Cxt ::= Cxt\ Exp \mid Exp\ Cxt$ $(\lambda x.e)\ v \to e[v/x]$	$Val ::= \lambda Var.Exp$ $Exp\ Exp$ [*strict*] $\langle (\lambda x.e)\ v \cdots \rangle_{\mathsf{k}} \to \langle e[v/x] \cdots \rangle_{\mathsf{k}}$
Call-by-name λ-calculus	
$Cxt ::= Cxt\ Exp$ $(\lambda x.e)\ e' \to e[e'/x]$	$Exp\ Exp$ [*strict*(1)] $\langle (\lambda x.e)\ e' \cdots \rangle_{\mathsf{k}} \to \langle e[e'/x] \cdots \rangle_{\mathsf{k}}$

Figure 1.1: A simple imperative language and λ-calculi defined using evaluation contexts and K ($p \in Pgm$, $v \in Val$, $c \in Cxt$, $x \in Var$, $\sigma \in State$, $i, i_1, i_2 \in Int$, $b \in Bool$, $s, s_1, s_2 \in Stmt$, $a \in AExp$; $e, e' \in Exp$)

assigned, so in particular variables can take only integer values. Programs consist of a statement followed by an arithmetic expression, so they can only evaluate to an integer value. Halt also halts programs only with integer values. Booleans can be introduced in programs only as results of comparisons and can be used in conditionals and loops. This language is admittedly trivial and limited, but it serves our purpose of introducing K by comparison with context reduction. This language will be non-trivially modified and extended in Section **??**. Note that even though we had to rewrite the syntactic constructs in the K definition in Figure 1 in order to augment them with strictness attributes[1], the K definition was still more compact than the context reduction one. For the sake of completeness, in both definitions we also included declarations of default items that appear in almost any language definition and so deserve to be implicit to the framework, such as the $\square$ in context reduction, and the configuration constructs $\langle K \rangle_{\mathsf{k}}$ and $\langle \mathbf{Set\{}\mathit{Config}\mathbf{\}} \rangle_{\top}$, respectively, in K. In general, K definitions tend to be more compact than their equivalent context reduction definitions; that holds true especially for more complex definitions of languages. As discussed in Section **??**, context reduction definitions can be automatically converted into equivalent K definitions. However, K has several other advantages over context reduction that may make one consider using K directly at its full strength, instead of adopting a methodological fragment of it.

Since K makes use only of standard, context-insensitive term rewriting modulo equations, it can be executed on rewrite engines like Maude almost *as is*. Nevertheless, the relationship between K and Maude is like that between SOS, or any of the above-mentioned styles, and Maude (or ML, or Scheme, or Haskell, etc.): Maude can be used as a means to execute and analyze K language definitions. The interested reader is referred to [74] for a detailed discussion on how the various language definitional styles can be faithfully embedded in rewrite logic and then executed using systems like Maude, or even more conventional programming languages. The basic idea of these faithful embeddings is that a language definition in any of these styles, say $\mathcal{L}$, can be regarded as a rewrite logic theory, say $R_{\mathcal{L}}$, in such a way that there is a *one-to-one computational equivalence* between reductions using $\mathcal{L}$ and rewrites using $R_{\mathcal{L}}$. Note that this is significantly stronger than encoding, or implementing, a framework into another framework: *$R_{\mathcal{L}}$ is $\mathcal{L}$, not an encoding of it*, the only difference between the two being insignificant notational/syntactic conventions. This is totally different from encoding $\mathcal{L}$ on a Turing machine or in a λ-calculus, for example, because such encodings would not preserve the intended computational granularity of $\mathcal{L}$'s reductions (if correct, they would only preserve the relational behavior of $\mathcal{L}$: whatever is reduced with $\mathcal{L}$ can also be reduced, in any number of steps, with its encodings).

One can naturally ask the following question then:

> *What is the need for yet another language definitional framework that can be embedded in rewrite logic, K in this case, if rewriting logic is already so powerful?*

Unfortunately, in spite of its definitional strength as a computational logic framework, rewrite logic does not give, and does not intend to give, the programming language designer any recipe on *how* to define a language. It essentially only suggests the following: however one wants to formally define a programming language or calculus, one can probably also do it in rewrite logic following the same intuitions and style. Therefore, rewrite logic can be regarded as a *meta-framework* that supports definitions of programming languages and calculi among many other things, providing the language designer with a means to execute and formally analyze languages in a generic way, but only *after* the language is already defined. Additionally, as discussed in Section **??** The following important question remains largely open to the working programming language designer, and not only:

> *Is there any language definitional framework that, at the same time,*

[1] In standalone K definitions, the language syntax is annotated with K attributes when the syntax is defined, not as part of the semantics as we did in Figure 1.

1. *Gives a strong intuition, even precise recipes, on* how *to define a language?*
2. *Same for language-related definitions, such as type checkers, type inferences, abstract interpreters, safety policy or domain-specific checkers, etc.?*
3. *Can define arbitrarily complex language features, including, obviously,* all *those found in existing languages, capturing also their intended computational granularity?*
4. *Is modular, that is, adding new features to a language does not require modifying existing definitions of unrelated features? Modularity is crucial for scalability and reuse.*
5. *Is generic, that is, not tied to any particular programming language or paradigm?*
6. *Supports naturally non-determinism* and *concurrency?*
7. *Is executable, so one can* test *language or formal analyzer definitions, as if one already had an interpreter or a compiler for one's language? Efficient executability of language definitions may even eliminate the need for interpreters or compilers.*
8. *Has state-exploration capabilities, including exhaustive behavior analysis (e.g., finite-state model-checking), when one's language is non-deterministic or/and concurrent?*
9. *Has a formal semantics, so one can also carry out proofs about programs? A fix-point, or an initial model, semantics is necessary if proofs by induction are desired.*

We believe that the list above contains a *minimal* set of desirable features that an ideal language definitional framework should have. Unfortunately, the current practice is to take the above desired features one at a time, temporarily or permanently declaring the others as something else. We next list how, in our view, current practices and language definitional styles fail to satisfy the above-mentioned requirements.

1. To formally capture one's intuition about a language feature to be defined, one may use a big-step or a small-step SOS reduction semantics, with or without evaluation contexts, typically on paper, without any machine support. Sometimes this so-called "formal" process is pushed to extreme in what regards its informality, in the sense that one can see definitions of some language features using one definitional style and of other features using another definitional style, without ever proving that the two definitional styles can co-exist in the claimed form for the particular language under consideration. For example, one may use a big-step SOS to give semantics to a code-self-generation extension of Java, while using a small-step SOS to define the concurrency semantics of Java. However, once one has concurrency and shared memory, one cannot have a big-step SOS definition of almost anything. An ideal language definitional framework should provide a uniform, compact and rigorous way to modularly define various language features, avoiding the need to define different language features following different styles.

2. To define a type system or a (domain-specific or not) safety policy for a language, one may follow a big-step-like definitional style, or even simply provide an algorithm to serve as a formal definition. While this appears to be, and in many cases indeed is acceptable, there can be a significant *formal gap* between the actual language semantic definition and its type system or safety policy regarded as mathematical objects, because in order to carry out proofs relating the two one needs one common formal ground. In practice, one typically ends up encoding the two in yet another framework, claimed to be "richer", and then carry out the proofs within that framework. But how can one make sure that the encodings are correct? Do they serve as alternative definitions for that sole purpose? An ideal definitional framework should have all the benefits of the richer framework, at no additional notational

or logical complexity, yet capturing the complete meaning of the defined constructs. In other words, in an ideal framework one should define a language as a mathematical object, say $\mathcal{L}$, and a type system or other abstract interpretation of it as another mathematical object over the same formalism, say $\mathcal{L}'$, and then carry out proofs relating $\mathcal{L}$ and $\mathcal{L}'$ using the provided proof system of the definitional framework. $\mathcal{L}$, $\mathcal{L}'$, as well as other related definitions, should be readable and easy to understand enough so that one does not feel the drive to give alternative, more intuitive definitions using a more informal notation or framework.

3. Some popular language definitional frameworks are incapable of defining even existing language features. In our view, the fact that a particular language feature is supported in some existing language serves as the strongest argument that that feature may be desirable, so an ideal language definitional framework must simply support it; in other words, one cannot argue against the usefulness of that feature just because one's favorite definitional framework does not support it. For example, since in standard SOS definitions (not including reduction semantics with evaluation contexts) the control flow information of a program is captured within the structure of the proof derivation, and since proof derivations are not first class objects in these formalisms, it makes it very hard, virtually almost impossible in these formalisms to define complex control intensive language constructs like, e.g., call-with-current-continuation (callcc). Another important example showing that conventional definitional frameworks fail to properly support existing common language features is concurrency: most frameworks enforce an interleaving semantics, which may not necessarily always be desired. Concurrency is further discussed in item 5 below. Some frameworks provide a "well-chosen" set of constructs, shown to be theoretically sufficient to define any computable function or algorithm, and then propose *encodings* of other language features into the set of basic ones; examples in this category are Turing machines or the plethora of (typed or untyped) λ-calculi, or π-calculi, etc. While these basic constructs yield interesting and meaningful idealized programming languages, using them to encode other language features is, in our view, inappropriate. Indeed, encodings hide the intended *computational granularity* of the defined language constructs; for example, a variable lookup intended to be a one-step operation in one's language should take precisely one step in an ideal framework (not hundreds/thousands of steps as in a Turing machine or lambda calculus encoding, not even two steps: first get location, then get value).

4. As Mosses pointed out in [53], SOS is non-modular. Even in the original notes on SOS [60, 61], Plotkin had to modify the definition of simple arithmetic expressions three times as his initial language evolved [53]. Even more frustratingly, to add an innocent abrupt termination statement to a language defined using SOS, say a halt, one would need to more than double the total number of rules: each language construct needs to be allowed to propagate the halting signal potentially generated by its arguments. Also, as one needs to add more items into configurations to support new language features, in SOS one needs to change again every rule to include the new items; note that there are no less than $7 + n * 10$ configuration items, where n is the number of threads, in the configuration of KOOL (which is a comparatively simple language) as shown in Figure CHANGETHIS. It can easily become very annoying and error prone to modify a large portion of unrelated existing definitions when adding a new feature. A language designer may be unwilling to add a new feature or improve the definition of an existing one, just because of the large number of required changes. Informal writing conventions are sometimes adopted to circumvent the non-modularity of SOS. For example, in the definition of Standard ML [49], Milner and his collaborators propose a "store convention" to avoid having to mention the store in every rule, and an "exception convention" to avoid having to double the number of rules for

the sole purpose of supporting exceptions. As rightfully noticed by Mosses [53], such conventions are not only adhoc and language specific, but may also lead to erroneous definitions. Mosses' Modular SOS [53] (MSOS) brings modularity to SOS in a formal and elegant way, by grouping the non-syntactic configuration items into transition labels, and allowing rules to mention only those items of interest from each label. As discussed in Section 3.6, MSOS still inherits all the remaining limitations of SOS.

5. A non-generic framework, i.e., one building upon a particular programming language or paradigm, may be hard or impossible to use at its full strength when defining a language that crosses the boundaries of the underlying language or paradigm. For example, a framework enforcing object or thread communication via explicit send and receive messages may require artificial encodings of languages that opt for a different communication approach (e.g., shared memory), while a framework enforcing static typing of programs in the defined language may be inconvenient for defining dynamically typed or untyped languages. In general, a framework providing and enforcing particular ways to define certain types of language features would lack genericity. Within an ideal framework, one can and should develop and adopt methodologies for defining certain types of languages or language features, but these should not be enforced. This genericity requirement is derived from the observation that today's programming languages are so diverse and based on orthogonal, sometimes even conflicting paradigms, that, regardless of how much we believe in the superiority of a particular language paradigm, be it object-oriented, functional or logical, a commitment to any existing paradigm would significantly diminish the strengths of a language definitional framework.

6. By inherently enforcing an interleaving semantics for concurrency, existing reduction semantics definitions (including ones based on evaluation contexts) can only capture a projection of concurrency (when one's goal is to define a truly concurrent language), namely its resulting non-determinism. Proponents of existing reduction semantics approaches may argue that the resulting non-deterministic behavior of a concurrent system is all what matters, while proponents of true concurrency may argue that a framework which does not support naturally concurrent actions, i.e., actions that take place *at the same time*, is not a framework for concurrency. We do not intend to discuss the important and debatable distinctions between non-determinism and interleaving vs. true concurrency here. The fact that there are language designers who desire an interleaving semantics while others who desire a true concurrency semantics for their language is strong evidence that an ideal language definitional framework should simply support both, preferably with no additional settings of the framework, but rather via particular definitional methodologies within the framework.

7. Most existing language definitional frameworks are, or until relatively recently were, lacking tool support for executability. Without the capability to execute language definitions, it is virtually impossible to debug or develop large and complex language definitions in a reasonable period of time. The common practice today is still to have a paper definition of a language using one's desired formalism, and *then* to implement an interpreter for the defined language following in principle the paper definition. This approach, besides the inconvenience of having to define the language twice, guarantees little to nothing about the appropriateness of the formal, paper definition. Compare this approach to an approach where there is *no gap* between the formal definition and its implementation as an interpreter. While any definition is by definition correct, one gets significantly more confidence in the appropriateness of a language definition, and is less reluctant to change it, when one is able to run it *as is* on tens or hundreds of programs. Recently, executability engines have been proposed both for MSOS (the MSOS tool, implemented by Braga and collaborators in Maude [16]) and for reduction semantics with evaluation contexts (the PLT Redex tool, implemented by Findler and his collaborators in Scheme [37]).

A framework providing *efficient* support for executability of formal language definitions may eliminate entirely the need to implement interpreters, or type checkers or type inferencers, for a language, because one can use directly the formal definition for that purpose.

8. While executability of language definitions is indispensable when designing non-trivial languages, one needs richer tool support when the language is concurrent. Indeed, it may be that one's definition is appropriate for particular thread or process interleavings (e.g., when blocks are executed atomically), but that it has unexpected behaviors for other interleavings. Moreover, somewhat related to the desired computational granularity of language constructs mentioned in item 3 above, one may wish to exhaustively investigate all possible interleavings or executions of a particular concurrent program, to make sure that no undesired behaviors are present and no desired behaviors are excluded. Moreover, when the state space of the analyzed program is large, manual analysis of behaviors may not be feasible; therefore, model-checking and/or safety property analysis are also desirable as intrinsic components of an ideal language definitional framework.

9. To prove properties about programs in a defined programming language, or properties about the programming language itself, as also mentioned in item 2 above, the current practice is to encode/redefine the language semantics in a "richer" framework, such as a theorem prover, and then carry out the desired proofs there. Redefining the semantics of a fully fledged programming language in a different formalism is a highly nontrivial, error prone and tedious task, possibly taking months; automated translations may be possible when the original definition of the language is itself formal, though one would need to validate the translator. In addition to the *formal gap* mentioned in item 2 due to the translation itself, this process of redefining the language is simply inconvenient. An ideal language definitional framework should allow one to have, for each language, *one definition serving all purposes*, including all those mentioned above. Most likely, proofs about programs or programming languages will need induction. In order for induction to be a valid proof principle, either it needs to be hardwired as one or more proof rules in the framework when the framework is proof-theoretical, or the framework must posses a fixed-point or an initial model semantics when it is model-theoretical.

Peter: reuse; Chucky: proof modularity

There are additional desirable, yet of a more subjective nature and thus harder to quantify, requirements of an ideal language definitional framework. For example, it should be simple and easy to understand, teach and use by mainstream enthusiastic language designers, not only by language experts—in particular, an ideal framework should not require its users to have advanced concepts of category theory, logics, or type theory, in order to use it. Also, it should have good data representation capabilities and should allow proofs of theorems about programming languages that are easy to comprehend. Additionally, a framework providing support for parsing programs directly in the desired language syntax may be desirable to one requiring the implementation of an additional, external to the definitional setting, parser.

The nine requirements above are nevertheless ambitious. Some proponents of existing language definitional frameworks may argue that their favorite framework has these properties; however, a careful analysis of existing language definitional frameworks reveals that they actually fail to satisfy some, sometimes many, of these ideal features (we discuss several such frameworks and their limitations in Chapter 3). Others may argue that their favorite framework has some of the properties above, the "important ones", declaring the other properties either "not interesting" or "something else". For example, one may say that what is important in one's framework is to get a dynamic semantics of a language, but its (model-theoretical)

algebraic denotational semantics, proving properties about programs, model checking, etc., are "something else" and therefore are allowed to need a different "encoding" of the language. Our position is that an ideal language definitional framework should not compromise any of the nine requirements above. Whether K satisfies all the requirements above or not is, and probably should always be, open. What we can mention with regards to this aspect, though, is that K was motivated and stimulated by the observation that the existing language definitional frameworks fail to fully satisfy these minimal requirements; consequently, K's design and development was conducted aiming *explicitly* to fulfill all nine requirements discussed above.

K's actual merit is in addressing the first four requirements in our list above, namely it proposes a scalable, modular and comprehensive formal approach to define programming languages and language-related features and analyses. While the first four requirements are specific to programming languages, the remaining five, namely genericity, support for concurrency, executability, formal analysis and formal semantics, transcend the domain of programming languages. Indeed, the latter are desired features of system specification formalisms in general, an area of great interest to many researchers and, consequently, with a variety of approaches, methods, techniques and supporting tools. K's current approach is to avoid proposing a new general purpose system specification formalism, with a new semantics and with tool support developed from scratch (in other words, to avoid "reinventing the wheel"), but instead to build upon the vast existing theoretical and tool infrastructure of rewrite logic [42, 17]. This decision more than paid off. For example, we can carry out proofs about language definitions and corresponding type systems using the existing proof machinery of rewrite logic (see Section CHANGETHIS). Also, our definition of a polymorphic type inferencer in K using Maude (see Section CHANGETHIS) significantly outperforms the current type inferencer of SML and slightly that of Haskell, while slightly under-performs that of OCAML. Experiments performed with the K definition of Java 1.4 using Maude's builtin model checker, that translates into a model-checker for Java *for free*, outperforms state-of-the art model checkers developed specifically for Java, such as Java PathFinder [23].

In this report, syntax is defined using only conventional and familiar context-free grammars, extended with lists and sets in a natural way. K is formally defined avoiding the algebraic notation, but its intuitions are borrowed from rewrite logic, an inherently algebraic meta-framework.

We generate all these structural equations automatically from strictness attributes in our implementation (see Section **??***).*

---

structure of the paper, machine data (fsl), what versions of maude, ocaml, haskell, sml

---

---

have a section on what this book is not about:

- advanced parsing; we assume advanced parsing is done outside and an AST is obtained; nevertheless, we still allow human readable, mixfix syntactic notation, etc.

---

say that concurrency is nothing special, all defined languagees are concurrent

---

Chapter 2

Background and Preliminaries

say that we use `tt` *font for code, including integers and program variables, and that we use italic or mathematical font for meta-variables ranging over the various syntactic categories.*

add Computability, Degrees (Π^0_2, etc.)

add Initial algebra, induction, consistency, Π^0_2 results, etc.

say that the various mathematical domains defined here will be used as a basis for the various language semantics in Chaper 3.

powerset of a set

Equational logic, or the logic of equality, is one of the most basic logical formalisms. Term rewriting is a standard computational model supported by many systems and languages. rewrite logic, not to be confused with term rewriting, organizes term rewriting modulo equations as a logic with a complete proof system and initial model semantics. Maude is an efficient rewrite logic engine.

2.1 Mathematical Background

lists, sets, multisets, $|A|$, ∞, 0^ω, 0^*

Lists, sets, multisets, and functions (partial or total) are basic mathematical structures that are indispensable when defining programming language semantics. We assume the reader familiar with elementary mathematical notions, the role of this section being to introduce our notational conventions.

2.1.1 Functions

Discuss Currying; needed in Section 4.5.

A function, or map, uniquely relates elements in its source set, also called its domain, to elements in its target set, also called its co-domain.

Definition 1. *$[A \to B]$ is the set of **functions**, or **maps**, defined from A to B. To state that f is a function from A to B, we may write either $f \in [A \to B]$ or the more common $f : A \to B$. If $f : A \to B$ then A is called the **domain** or the **source** of f and B is called the **co-domain** or the **target** of f. If $a \in A$ then $f(a) \in B$ denotes the unique element in B that f relates to a.*

Unlike partial functions which are discussed below, functions are defined on all the elements in their source, reason for which we also call their source domain.

Definition 2. *If $f : A \to B$, $a \in A$ and $b \in B$, then we let $f[b/a] : A \to B$ denote the function that* ***updates*** *f* ***in*** *a* ***to*** *b, defined formally as follows:*

$$f[b/a](a') = \begin{cases} f(a') & \text{when } a' \neq a, \\ b & \text{when } a' = a. \end{cases}$$

Exercise 1. *With the notation in Definition 2, could it be the case that $f[b/a] = f$? If $a, a' \in A$ and $b, b' \in B$, is it always the case that $f[b/a][b'/a'] = f[b'/a'][b/a]$? What if $b = b'$?*

Exercise 2. *Show that if A is empty then the set $[A \to B]$ contains precisely one function, regardless of whether B is empty or not.*

Definition 3. *By abuse of notation, we let $\emptyset$ also denote the unique function in $[\emptyset \to B]$.*

Exercise 3. *Show that $[A \to B]$ contains only one function if and only if $A = \emptyset$ or $|B| = 1$.*

Exercise 4. *Show that the set $[A \to \emptyset]$ is empty when $A \neq \emptyset$.*

Exercise 5. *Show that if A and B are finite, then $[A \to B]$ is also finite and $|[A \to B]| = |B|^{|A|}$.*

2.1.2 Partial Functions

---

make sure that the variable lookup and update operations are also partial functions, so be in synch with the notations/terminology when we define the state of IMP in Section 3.1.2; not clear whether this should be here or rather in Section 2.4.6

A partial function, or partial map, uniquely relates a subset of elements in its source set, called its domain, to elements in its target set.

Definition 4. *$[A \rightharpoonup B]$ is the set of* ***partial functions****, or* ***partial maps****, from A to B. We may write either $f \in [A \rightharpoonup B]$ or $f : A \rightharpoonup B$ when stating that f is a partial function from A to B. If $f : A \rightharpoonup B$ then A is the* ***source*** *of f and B is the* ***target*** *of f. Like for functions, if $a \in A$ then we let $f(a) \in B$ be the unique element in B that f relates to a, in case such an element exists; if such an $f(a) \in B$ does not exists for a given $a \in A$, then we say that f is* ***undefined*** *in a, or that $f(a)$ is* ***undefined****. $Dom(f) \stackrel{def}{=} \{a \in A \mid f(a) \text{ is defined}\}$ is the* ***definitional domain****, or just the* ***domain****, of f. If $Dom(f) = A$ then f is a function like in Definition 1 and we may call it a* ***total*** *(partial) function. If $Dom(f)$ is finite then f is called a* ***partial finite-domain function****. We let $[A \rightharpoonup B]^{finite}$ denote the set of partial finite-domain functions in $[A \rightharpoonup B]$.*

Therefore, ordinary functions are special partial functions, namely ones defined on all the elements in their sources. The function update operation in Definition 2 can be naturally extended to partial functions:

Definition 5. *If $f : A \rightharpoonup B$, $a \in A$ and $b \in B$, then we let $f[b/a] : A \to B$ denote the partial function that* ***updates*** *f* ***in*** *a* ***to*** *b, defined formally as follows:*

$$f[b/a](a') = \begin{cases} f(a') & \text{when } a' \in Dom(f) \text{ and } a' \neq a, \\ b & \text{when } a' = a, \\ \textit{undefined} & \text{when } a' \notin Dom(f) \cup \{a\} \end{cases}$$

If one assumes that $f(a')$ can either be an element in B or the special element *undefined*, the definition above can be stated more compactly, like Definition 2:

$$f[b/a](a') = \begin{cases} f(a') & \textit{when } a' \neq a, \\ b & \textit{when } a' = a. \end{cases}$$

Exercise 6. *With the notation in Definition 5, show that* $Dom(f[b/a]) = Dom(f) \cup \{a\}$. *Under what conditions* $f[b/a] = f$*?*

Exercise 7. *Show that* $[\emptyset \rightharpoonup B] = [\emptyset \to B]$ *for any set* B.

The exercise above says, in particular, that the function $\emptyset : \emptyset \rightharpoonup B$ is total.

Definition 6. *For any* A *and* B*, we let* $\perp : A \rightharpoonup B$ *denote the* ***undefined everywhere partial function*** *from* A *to* B*, that is, the partial function with* $\perp(a)$ *undefined for any* $a \in A$.

Therefore, unlike $[A \to B]$, the set $[A \rightharpoonup B]$ always contains at least one element, the undefined everywhere partial function $\perp : A \rightharpoonup B$.

Exercise 8. *Show that the partial functions* $\emptyset : \emptyset \rightharpoonup B$ *and* $\perp : \emptyset \rightharpoonup B$ *are equal.*

Exercise 9. *Show that* $[A \rightharpoonup B]$ *contains only one element if and only if* $A = \emptyset$ *or* $B = \emptyset$ *or* $|B| = 1$.

Exercise 10. *If* A *and* B *are finite then* $[A \rightharpoonup B]$ *is also finite; if that is the case, can you express* $|[A \rightharpoonup B]|$ *as a closed expression in terms of* $|A|$ *and* $|B|$*?*

2.1.3 Context-Free Grammars and the Mixfix Algebraic Notation

BNF (Bachus-Naur Form) or CFG (Context Free Grammar) notations to describe the syntax of programming languages are very common. We also used BNF to define the syntax of our simple language in the lectures on SOS. From here on, we can interchangeably use the mix-fix notation instead of BNF or CFG, so it is important to understand the relationship between the mix-fix notation and BNF and CFG:

Exercise 11. *Argue that the mix-fix notation is equivalent to BNF and CFG. Find an interesting simple example language and express its syntax using the mix-fix notation, BNF, and production-based CFG (e.g.,* $A \to AB \mid a$ *and* $B \to BA \mid b$*).*

---

we may write $a \in AExp$ *instead of* $a \in \mathcal{L}(AExp)$

---

introduce the notations

---

introduce "algebraic CFG": extend CFG with these.

---

say that we allow the use of parentheses for disambiguation.

---

the following is also something to take care of:

---

e.g., **List{**S**}** defines comma-separated lists of elements of type S, and could be expressed with the CFG rule **List{**S**}** ::= $\cdot \mid S(,S)^*$, with the additional understanding that '_, _' is associative and $\cdot$ satisfies the unit axioms.

---

say that from a parsing perspective sets and multisets are no different than lists; also, say that in RWL set/multiset axioms are important since rewriting happens modulo those, while K has builtin support for these; in K rules adapt modulo their axioms and not the world.

---

say that if t is a term using the CFG notation, we usually let $\bar{t}$ denote the algebraic version of t; in case t has any "parameter", as needed for example in deduction rules, then that parameter becomes of variable in $\bar{t}$ of appropriate sort. See, for example, the translations of the various semantic approaches to RL and their correctness proofs for what we need to cover here.

More precisely, one sort is added for each syntactic category, or non-terminal in the grammar, and one operation $\gamma_{_} : NT(\gamma) \to S$ is added for each production S ::= γ in the grammar, where $\gamma_{_}$ replaces each non-terminal in γ by an underscore and $NT(\gamma)$ is the product of the sorts corresponding to the non-terminals in γ, in the order of their appearance. For example, the production *Stmt* ::= `if` *BExp* `then` *Stmt* `else` *Stmt* is associated the algebraic operation

$$\texttt{if_then_else_} : BExp \times Stmt \times Stmt \to Stmt.$$

Applying this syntax algebrization technique to the entire syntax of IMP in Figure 3.1, we obtain the algebraic signature in Figure 3.2.

Notes

- mixfix notation as equivalent to CFG

- BNF is an acronym for "Backus Naur Form". John Backus and Peter Naur introduced for the first time a formal notation to describe the syntax of a given language (This was for the description of the ALGOL 60 programming language, see [Naur 60]). To be precise, most of BNF was introduced by Backus in a report presented at an earlier UNESCO conference on ALGOL 58. Few read the report, but when Peter Naur read it he was surprised at some of the differences he found between his and Backus's interpretation of ALGOL 58. He decided that for the successor to ALGOL, all participants of the first design had come to recognize some weaknesses, should be given in a similar form so that all participants should be aware of what they were agreeing to. He made a few modifications that are almost universally used and drew up on his own the BNF for ALGOL 60 at the meeting where it was designed. Depending on how you attribute presenting it to the world, it was either by Backus in 59 or Naur in 60. (For more details on this period of programming languages history, see the introduction to Backus's Turing award article in Communications of the ACM, Vol. 21, No. 8, august 1978. This note was suggested by William B. Clodius from Los Alamos Natl. Lab).

2.1.4 Basic Term Rewriting

---

define confluence

---

define orthogonality and state/cite the result saying that orthogonality implies confluence, etc.

A term rewrite system (TRS) consists of a set of uninterpreted operations, possibly over different syntactic categories, or sorts, and a set of rewrite rules of the form $l \rightarrow r$, where l and r are terms possibly containing variables with possibly multiple occurrences. Term rewriting is a method of computation that works by progressively changing (rewriting) a term, using a TRS. A rule can apply to the entire term being rewritten or to a subterm of the term. First, a match within the current term is found. This is done by finding a substitution, θ, from variables to terms such that the left-hand-side of the rule, l, matches part or all of the current term when the variables in l are replaced according to the substitution. The matched subterm is then replaced by the result of applying the substitution to the right-hand-side of the rule, r. Thus, the part of the current term matching $\theta(l)$ is replaced by $\theta(r)$. The rewriting process continues as long as it is possible to find a subterm, rule, and substitution such that $\theta(l)$ matches the subterm. When no matching subterms are found, the rewriting process terminates, with the final term, a normal form, being the result of the computation. One of the distinctive features of term rewriting is that rules apply in a *context insensitive* manner; in other words, the larger context in which a rule is matched does not influence the applicability of the rewrite rule. Rewriting, like other methods of computation, can continue forever. The following is a rewrite definition of a factorial language construct in a hypothetical calculator language:

$$
\begin{aligned}
&Exp ::= Nat \mid Exp * Exp \mid Exp - Exp \mid \ldots \mid Exp\,! \\
&0! \rightarrow 1 \\
&n! \rightarrow n * (n-1)!, \quad \text{when } n \geq 1
\end{aligned}
$$

There are a large number of term rewriting engines, including ASF [82], Elan [12], Maude [17], OBJ [28], Stratego [85], and others, some of which are capable of executing several million rewrites per second. Rewriting is also a fundamental part of many existing functional languages and theorem provers. Term rewriting and rewrite logic definitions of languages or language features, if executed on efficient rewrite engines, can be as efficient or even outperform implementations of interpreters or other tools in conventional programming languages. This point can be quickly reflected even on the simple calculator language above. For example, in our experiments executing the definition above, the factorial of 50,000, a number of 213,237 digits, was calculated in 18.620 seconds by Maude and in 19.280 and 16.770 seconds by the programming languages ML and Scheme, respectively. While for this particular example one may say that the quality of large integers libraries plays the major role, we obtained similar results for many other experiments.

... some of them mentioned in ... (wait until numbers are added for the various operational semantics)

one other comparison with ML and Scheme is in Section 2.5, where we discuss permutation generation

2.1.5 On Logics and Logic Deduction

...

more general than here: search for all the places in the book where this section is referred to and make sure that everything needed is included here.

proof systems, side conditions of proof rules (make text below make sense)

The conditions which make a rule applicable, such as, for example, "where i is the sum of i_1 and i_2" in rule (Add), are called *side conditions*. Side conditions typically constrain variables over recursively enumerable domains (e.g., i, i_1 and i_2 range over integer numbers in rule (Add). Therefore, each SOS rule comprises a *recursively enumerable* collection of concrete instance rules. One could also move the side-conditions into the actual condition of the rule (above the line), but many rules that are unconditional would then become conditional and thus harder to type and read.

add background here in order for the next to make sense

An SOS definition is a collection of *parametric rules* of the form:

$$\frac{\mathit{transition}_1,\ \mathit{transition}_2,\ \ldots, \mathit{transition}_k}{\mathit{transition}}$$

The transitions above the line are called the *condition* of the rule, and the transition below the line is called the *conclusion* of the rule. The intuition here is that *transition* is possible whenever $\mathit{transition}_1$, $\mathit{transition}_2$, ..., $\mathit{transition}_k$ are possible. We may also say that *transition* is *derivable*, or can be *inferred*, from $\mathit{transition}_1$, $\mathit{transition}_2$, ..., $\mathit{transition}_k$. This reflects the fact that an SOS definition can (and should) be viewed as a logical system, where one can deduce possible behaviors of programs.

Structural induction on proof/derivation trees

For Section 2.3:

introduce the notation $\Gamma \vdash \varphi$

axioms vs. proof rules; former a special case of the latter

axiom / proof rule schemata

$\vdash \varphi$ *is a shorthand for* $\emptyset \vdash \varphi$*;* φ *called a* theorem *in the corresponding logic/proof system.*

general properties about entailment, such as $\vdash \varphi$ *implies* $\Gamma \vdash \varphi$*, etc. (obviously, in a more general form).*

Proposition 1. *If* $\Gamma \vdash \varphi$ *then there is some finite subset* Γ_0 *of* Γ *such that* $\Gamma_0 \vdash \varphi$*.*

2.2 Basic Computability Elements

In this section we recall very basic concepts of computability theory needed for other results in the book, and introduce our notation for these. This section is by no means intended to serve as a substitute for much thorough presentations found in dedicated computability textbooks (some mentioned in Section 2.2.4).

2.2.1 Turing Machines

Turing machines are abstract computational models used to formally capture the informal notion of a *computing system*. The *Church-Turing thesis* postulates that any computing system, any algorithm, or any program in any programming language running on any computer, can be equivalently simulated by a Turing machine. Having a formal definition of computability allows us to rigorously investigate and understand what can and what cannot be done using computing devices, regardless of what languages are used to program them. Intuitively, by a computing device we understand a piece of machinery that carries out tasks by successively applying sequences of instructions and using, in principle, unlimited memory; such sequences of instructions are today called *programs*, or *procedures*, or *algorithms*.

Turing machines are used for their theoretical value; they are not meant to be physically built. A Turing machine is a finite state device with infinite memory. The memory is very primitively organized, as one or more infinite *tapes* of cells that are sequentially accessible through *heads* that can move to the left or to the right cell only. Each cell can hold a bounded piece of data, typically a Boolean, or bit, value. The tape is also used as the input/output of the machine. The computational steps carried out by a Turing machine are also very primitive: in a state, depending on the value in the current cell, a Turing machine can only rewrite the current cell on the tape and/or move the head to the left or to the right. Therefore, a Turing machine does not have the direct capability to perform random memory access, but it can be shown that it can simulate it.

There are many equivalent definitions of Turing machines in the literature. We prefer one with a tape that is infinite at both ends and describe it next (interestingly, an almost identical machine was proposed by Emil Post independently from Alan Turing also in 1936; see Section 2.2.4). Consider a mechanical device which has associated with it a tape of infinite length in both directions, partitioned in spaces of equal size, called *cells*, which are able to hold either a 0 or an 1 and are rewritable. The device examines exactly one cell at any time, and can, potentially nondeterministically, perform any of the following four operations (or *commands*):

1. Write a 1 in the current cell;
2. Write a 0 in the current cell;
3. Shift one cell to the right;
4. Shift one cell to the left.

The device performs one operation per unit time, called a *step*. We next give a formal definition.

Definition 7. *A **(deterministic) Turing machine** $\mathcal{M}$ is a 6-tuple (Q, B, q_s, q_h, C, M), where:*

- *Q is a finite set of **internal states**;*
- *$q_s \in Q$ is the **starting state** of $\mathcal{M}$;*
- *$q_h \in Q$ is the **halting state** of $\mathcal{M}$;*
- *B is the set of **symbols** of $\mathcal{M}$; we assume without loss of generality that $B = \{0, 1\}$;*
- *$C = B \cup \{\rightarrow, \leftarrow\}$ is the set of **commands** of $\mathcal{M}$;*
- *$M : (Q - \{q_h\}) \times B \rightarrow Q \times C$ is a total function, the **transition function** of $\mathcal{M}$.*

We assume that the tape contains only 0's before the machine starts performing.

Our definition above is for *deterministic* Turing machines. One can also define nondeterministic Turing machines by changing the transition function M into a relation. Non-deterministic Turing machines have the same computational power as the deterministic Turing machines (i.e., they compute/decide the same classes of problems; computational speed or size of the machine is not a concern here), so, for our purpose in this section, we limit ourselves to deterministic machines.

A *configuration* of a Turing machine is a 4-tuple consisting of an internal state, a current cell, and two infinite strings (notice that the two infinite strings contain only 0's starting with a certain cell), standing for the cells on the left and for the cells on the right of the current cell, respectively. We let $(q, L\underline{b}R)$ denote the configuration in which the machine is in state q, with current cell b, left tape L and right tape R. For convenience, we write the left tape L backwards, that is, its head is at its right end; for example, Lb appends a b to the left tape L. Given a configuration $(q, L\underline{b}R)$, the content of the tape is LbR, which is infinite at both ends. We also let $(q, L\underline{b}R) \rightarrow_{\mathcal{M}} (q', L'\underline{b'}R')$ denote a configuration transition under one of the four commands. Given a configuration in which the internal state is q and the examined cell contains b, and if $M(q, b) = (q', c)$, then exactly one of the following configuration transitions can take place:

$(q, L\underline{b}R) \rightarrow_{\mathcal{M}} (q', L\underline{c}R)$ if $c = 0$ or $c = 1$;

$(q, L\underline{b}b'R) \rightarrow_{\mathcal{M}} (q', Lb\underline{b'}R)$ if $c = \rightarrow$;

$(q, Lb'\underline{b}R) \rightarrow_{\mathcal{M}} (q', L\underline{b'}bR)$ if $c = \leftarrow$.

Since our Turing machines are deterministic (M is a function), the relation $\rightarrow_{\mathcal{M}}$ on configurations is also deterministic. The machine starts performing in the internal state q_s. If there is no input, the initial configuration on which the Turing machine is run is $(q_s, 0^\omega\underline{0}0^\omega)$, where 0^ω is the infinite string of zeros. If the Turing machine is intended to be run on a specific input, say $x = b_1b_2\cdots b_n$, its initial configuration is $(q_s, 0^\omega\underline{0}b_1b_2\cdots b_n0^\omega)$. We let $\rightarrow^\star_{\mathcal{M}}$ denote the transitive and reflexive closure of the binary relation on configurations $\rightarrow_{\mathcal{M}}$ above.

A Turing machine $\mathcal{M}$ terminates when it reaches its halting state:

Definition 8. *Turing machine $\mathcal{M}$* ***terminates on input*** *$b_1b_2\cdots b_n$ iff*

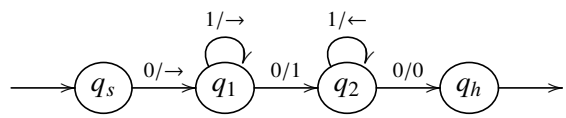
$$(q_s, 0^\omega\underline{0}b_1b_2\cdots b_n0^\omega) \rightarrow^\star_{\mathcal{M}} (q_h, L\underline{b}R)$$

for some $b \in B$ and some tape instances L and R. A set $S \subseteq B^\star$ is ***recursively enumerable (r.e.,)*** *or* ***semi-decidable****, respectively* ***co-recursively enumerable (co-r.e.)*** *or* ***co-semi-decidable****, iff there is some Turing machine $\mathcal{M}$ which terminates on precisely all inputs $b_1b_2\cdots b_n \in S$, respectively on precisely all inputs $b_1b_2\cdots b_n \notin S$, and is* ***recursive*** *or* ***decidable*** *iff it is both r.e. and co-r.e.*

Note that a Turing machine as we defined it cannot get stuck in any state but q_h, because the mapping M is defined everywhere except in q_h. Therefore, for any given input, a Turing machine carries out a determined succession of steps, which may or may not terminate. A Turing machine can be regarded as an idealized, low-level programming language, which can be used for computations by placing a certain input on the tape and letting it run; if it terminates, the result of the computation can be found on the tape. Since our Turing machines have only symbols 0 and 1, one has to use them ingeniously to encode more complex inputs, such as natural numbers. There are many different ways to do this. A simple tape representation of natural numbers is to represent a number n by n consecutive cells containing the bit 1. This works, however, only when n is strictly larger than 0. Another representation, which also accommodates $n = 0$, is as a sequence of $n + 1$ cells containing 1. With this latter representation, one can then define Turing machines corresponding

States and transition function (graphical representation to the right):

$Q = \{q_s, q_h, q_1, q_2\}$
$M(q_s, 0) = (q_1, \rightarrow)$
$M(q_s, 1) = \text{anything}$
$M(q_1, 0) = (q_2, 1)$
$M(q_1, 1) = (q_1, \rightarrow)$
$M(q_2, 0) = (q_h, 0)$
$M(q_2, 1) = (q_2, \leftarrow)$



Sample computation:

$(q_s, 0^\omega \underline{0}1110^\omega) \rightarrow_{\mathcal{M}} (q_1, 0^\omega \underline{1}110^\omega) \rightarrow_{\mathcal{M}} (q_1, 0^\omega 1\underline{1}10^\omega) \rightarrow_{\mathcal{M}} (q_1, 0^\omega 11\underline{1}0^\omega) \rightarrow_{\mathcal{M}}$
$(q_1, 0^\omega 111\underline{0}0^\omega) \rightarrow_{\mathcal{M}} (q_2, 0^\omega 111\underline{1}0^\omega) \rightarrow_{\mathcal{M}} (q_2, 0^\omega 11\underline{1}10^\omega) \rightarrow_{\mathcal{M}} (q_2, 0^\omega 1\underline{1}110^\omega) \rightarrow_{\mathcal{M}}$
$(q_2, 0^\omega \underline{1}1110^\omega) \rightarrow_{\mathcal{M}} (q_2, 0^\omega \underline{0}11110^\omega) \rightarrow_{\mathcal{M}} (q_h, 0^\omega \underline{0}11110^\omega)$

Figure 2.1: Turing machine $\mathcal{M}$ computing the successor function, and sample computation

to functions that take any natural numbers as input and produce any natural numbers as output. For example, Figure 2.1 shows a Turing machine computing the successor function. Cells containing 0 can then be used as number separators, when more natural numbers are needed. For example, a Turing machine computing a binary operation on natural numbers would run on configurations $(q_s, 0^\omega \underline{0}1^{m+1}01^{n+1}0^\omega)$ and would halt on configurations $(q_h, 0^\omega \underline{0}1^{k+1}0^\omega)$, where m, n, k are natural numbers.

One can similarly have tape encodings of rational numbers; for example, one can encode the number m/n as m followed by n with two 0 cells in-between (and keep the one-0-cell-convention for argument separation). Real numbers are not obviously representable, though. A Turing machine is said to *compute* a real number r iff it can finitely approximate r (for example using a rational number) with any desired precision; one way to formalize this is as follows: Turing machine $\mathcal{M}_r$ computes r iff when run on input natural number p, it halts with result rational number m/n such that $|r - m/n| < 1/10^p$. If a real number can be computed by a Turing machine then it is called *Turing computable*. Many real numbers, e.g., π, e, $\sqrt{2}$, etc., are Turing computable.

2.2.2 Universal Machines, the Halting Problem, and Decision Problems

Since Turing machines have finite descriptions, they can be encoded themselves as natural numbers. Therefore, we can refer to "the kth Turing machine", where k is a natural number, the same way we can refer to the ith input to a Turing machine. A *universal Turing machine* is a Turing machine that can simulate an arbitrary Turing machine on arbitrary input. The universal machine essentially achieves this by reading both the description of the machine to be simulated as well as the input thereof from its own tape. There are various constructions of universal Turing machines in the literature, which we do not repeat here. We only notice that we can construct such a universal machine $\mathcal{U}$ which terminates precisely on all inputs of the form 1^k01^i where Turing machine k terminates on input i. This immediately implies that the language $\{1^k01^i \mid \text{Turing machine } k \text{ terminates on input } i\}$ is recursively enumerable. However, the undecidability of the famous *halting problem* (does a given Turing machine terminates on a given input?) implies that this language is not recursive; more specifically, it is not co-recursively enumerable.

Since the elements of many mathematical domains can be encoded as words in $B^\star$, the terminology in Definition 8 is also used for *decision problems* over such domains. For example, the decision problem of

whether a graph given as input has a cycle or not is recursive; in other words, the set of cyclic graphs (under some appropriate encoding in $B^\star$) is recursive/decidable. Since there is a bijective correspondence between elements in $B^\star$ and natural numbers, and also between tuples of natural numbers and natural numbers, decision problems are often regarded as one- or multi-argument relations or predicates over natural numbers. For example, a subset $R \subseteq B^\star$ can be regarded as a predicate, say of three natural number arguments, where $R(i, j, k)$ for $i, j, k \in Nat$ indicates that the encoding of (i, j, k) belongs to R.

While the haling problem is typically an excellent vehicle to formally state and prove that certain problems are undecidable, so they cannot be solved by computers no matter how powerful they are or what programming languages are used, its reflective nature makes the halting problem sometimes hard to use in practice. The *Post correspondence problem (PCP)* is an another canonical undecidable problem, which is sometimes easier to use to show that other problems are undecidable. The PCP can be stated as follows: given a set of (domino-style) tiles each containing a top and a bottom string of 0/1 bits, is it possible to find a sequence of possibly repeating such tiles so that the concatenated top strings equal the concatenated bottom strings? For example, if the given tiles are the following:

$$1: \begin{array}{|c|}\hline \circ \\ \hline \bullet\circ\circ \\ \hline\end{array} \quad 2: \begin{array}{|c|}\hline \circ\bullet \\ \hline \circ\circ \\ \hline\end{array} \quad 3: \begin{array}{|c|}\hline \bullet\bullet\circ \\ \hline \bullet\bullet \\ \hline\end{array}$$

then the answer to the PCP problem is positive, because the sequence of tiles 3 2 3 1 yields the same concatenated strings at the top and at the bottom.

2.2.3 The Arithmetic Hierarchy

The *arithmetical hierarchy* defines classes of problems of increasing difficulty, called *degrees*, as follows:

$$\Sigma_0^0 = \Pi_0^0 = \{R \mid R \text{ recursive}\}$$

$$\Sigma_{n+1}^0 = \{P \mid \exists Q \in \Pi_n^0,\ \forall i(P(i) \leftrightarrow \exists j Q(i, j))\}$$

$$\Pi_{n+1}^0 = \{P \mid \exists Q \in \Sigma_n^0,\ \forall i(P(i) \leftrightarrow \forall j Q(i, j))\}$$

For example, the Σ_1^0 degree consists of the predicates P over natural numbers for which there is some recursive predicate R such that for any $i \in Nat$, $P(i)$ holds iff $R(i, j)$ holds for some $j \in Nat$. It can be shown that Σ_1^0 contains precisely the recursively enumerable predicates. Similarly, Π_1^0 consists of the predicates P for which there is some recursive predicate R such that for any $i \in Nat$, $P(i)$ holds iff $R(i, j)$ holds for all $j \in Nat$, which is precisely the set of co-recursively enumerable predicates. An important degree is also Π_2^0, which consists of the predicates P over natural numbers for which there is some recursive predicate R such that for any $i \in Nat$, $P(i)$ holds iff for any $j \in Nat$ there is some $k \in Nat$ such that $R(i, j, k)$ holds. A prototypical Π_2^0 problem is the following: giving a Turing machine $\mathcal{M}$, does it terminate on all inputs? The complexity of the problem stays in the fact that there are infinitely many (but enumerable) inputs, so one can never be "done" with testing them; moreover, even for a given input, one does not know when to stop running the machine and reject the input. However, if one is given for each input accepted by the machine a run of the machine, then one can simply check the run and declare the input indeed accepted. Therefore, $\mathcal{M}$ terminates on all inputs iff for any input there exists some accepting run of $\mathcal{M}$, which makes it a Π_2^0 problem because checking whether a given run on a given Turing machine with a given input accepts the input is decidable. Moreover, it can be shown that if we pick $\mathcal{M}$ to be the universal Turing machine $\mathcal{U}$ discussed above, then the following problem, which we refer to as Totality from here on, is in fact Π_2^0-complete:

INPUT: A natural number k;
OUTPUT: Does $\mathcal{U}$ terminate on all inputs 1^k01^i for all $i \geq 0$?

For any n, both degrees Σ_n^0 and Π_n^0 are properly included in both the immediately above degrees Σ_{n+1}^0 and Π_{n+1}^0. There are extensions which define degrees Σ_n^1, Π_n^2, etc., but we do not discuss them here.

2.2.4 Notes

The abstract computational model that we call the "Turing machine" today was originally called "the computer" when proposed by Alan Turing in 1936-37 [80]. In the original article, Turing imagined not a machine, but a person (the computer) who executes a series of deterministic mechanical rules "in a desultory manner". In his 1948 essay "Intelligent Machinery", Turing calls the machine he proposed the *logical computing machine*, a name which has not been adopted, everybody preferring to call it the *Turing machine*.

It is insightful to understand the scientific context in which Turing proposed his machine. In the 1930s, there were several approaches attempting to address Hilbert's tenth question of 1900, the *Entscheidungsproblem* (the *decision problem*). Partial answers have been given by Kurt Gödel in 1930 (and published in 1931), under the form of his famous *incompleteness theorems*, and in 1934, under the form of his *recursion theory*. Alonzo Church is given the credit for being the first to effectively show that the Entscheidungsproblem was indeed *undecidable*, introducing also λ-calculus (discussed in Section 4.5). Church published his paper on 15 April 1936, about one month before Turing submitted his paper on 28 May 1936. In his paper, Turing also proved the equivalence of his machine to Church's λ-calculus. Interestingly, Turing's paper was submitted only a few months before Emil Post, another great logician, submitted a paper independently proposing an almost identical computational model on 7 October 1936 [62]. The major difference between Turing's and Post's machines is that the former uses a tape which is infinite in only one direction, while the latter works with a tape which is infinite at both ends, like the "Turing machine" that we used in this section. We actually took our definitions in this section from Rogers' book [67], which we recommend the reader for more details in addition to comprehensive textbooks such as Sipser [76] and Hopcroft, Motwani and Ullman [33]. To remember the fact that Post and Turing independently invented an almost identical computational model of utmost importance, several computer scientists call it the "Post-Turing machine".

Even though Turing was not the first to propose what we call today a Turing-complete model of computation, many believe that his result was stronger than Church's, in that his computational model was more direct, easier to understand, and based on first, low-level computational principles. For example, it is typically easy to implement Turing Machines in any programming language, which is not necessarily the case for other Turing-complete models, such as, for example, the λ-calculus. As seen in Section 4.5, λ-calculus relies on a non-trivial notion of substitution, which comes with the infamous *variable capture* problem.

2.2.5 Exercises

Exercise 12. *Define Turing machines corresponding to the addition, multiplication, and power operations on natural numbers. For example, the initial configuration of the Turing machine computing addition with 3 and 7 as input is* $(q_s, 0^\omega \underline{0}11110111111110^\omega)$*, and its final configuration is* $(q_h, 0^\omega \underline{0}111111111110^\omega)$*. We here assumed that n is encoded as* $n+1$ *cells containing 1.*

Exercise 13. *Show that there are real numbers which are not Turing computable.*
(Hint: The set of Turing machines is recursively enumerable.)

2.3 Propositional Logic

In this section we discuss propositional logic, often referred to as propositional *calculus*. Its sentences are called *propositions* and are constructed from propositional variables, truth values, and the usual propositional connectives. Its models are simple maps assigning truth values to propositional variables. We present a common proof system for propositional calculus and prove its soundness and completeness.

2.3.1 Syntax and Semantics

Propositional formulae, or simply *propositions*, are constructed using the following formal syntax:

$$\begin{aligned} PVar &::= \text{ propositional variables, assumed to be a countable set} \\ \varphi &::= PVar \mid \bot \mid \varphi \to \varphi \end{aligned}$$

We typically use Latin letters p, q, r, etc., with or without subscripts and superscripts, to refer to propositional variables in *PVar*, and use Greek letters φ, ϕ, ψ, etc., with or without subscripts or superscripts, to refer to propositions. The formula $\bot$ is read *false*, and $\to$ is read *implies*. It is customary to extend the syntax of formulae with additional *syntactic sugar* constructs, such as the following common ones:

$$\varphi ::= \cdots \mid \top \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \leftrightarrow \varphi$$

These can be (iteratively) desugared into the basic constructs as follows:

$$\begin{array}{lrcl} (\textit{true}) & \top & \equiv & \neg\bot \\ (\textit{negation}) & \neg\varphi & \equiv & \varphi \to \bot \\ (\textit{and}) & \varphi_1 \wedge \varphi_2 & \equiv & \neg(\varphi_1 \to \neg\varphi_2) \\ (\textit{or}) & \varphi_1 \vee \varphi_2 & \equiv & \neg\varphi_1 \to \varphi_2 \\ (\textit{equivalent}) & \varphi_1 \leftrightarrow \varphi_2 & \equiv & (\varphi_1 \to \varphi_2) \wedge (\varphi_2 \to \varphi_1) \end{array}$$

We took $\bot$ and $\to$ as basic constructs here, but there are other formulations of propositional logic where other constructs are taken as basic and then desugar $\bot$ and $\to$ into those (e.g., $\varphi_1 \to \varphi_2 \equiv \neg\varphi_1 \vee \varphi_2$, etc.).

A propositional *model*, also called a *truth assignment*, is a map assigning a truth value to each propositional variable. Models and propositions yield a *satisfaction relation*:

Definition 9. *A **propositional model** is a map $\rho : PVar \to \{true, false\}$. The **satisfaction relation** $\models$ between models and propositions is defined inductively as follows:*

$$\begin{array}{l} \rho \models p \quad \textit{iff } \rho(p) = \textit{true} \\ \rho \models \bot \quad \textit{never holds, that is, } \rho \not\models \bot \\ \rho \models \varphi_1 \to \varphi_2 \quad \textit{iff } \rho \models \varphi_1 \textit{ implies } \rho \models \varphi_2 \end{array}$$

*The satisfaction relation extends to sets of propositions: if Γ is a set of propositions then $\rho \models \Gamma$ iff $\rho \models \varphi$ for each $\varphi \in \Gamma$. If Γ is a set of propositions and φ is a proposition, then Γ **semantically entails** φ, written $\Gamma \models \varphi$, iff $\rho \models \Gamma$ implies $\rho \models \varphi$ for each model ρ. A proposition φ is **valid**, or is a **tautology**, written $\models \varphi$, iff $\emptyset \models \varphi$, or equivalently, iff $\rho \models \varphi$ for any model ρ.*

The following results will help us prove the soundness of our proof system for propositional logic:

Proposition 2. *The following hold for any propositions φ_1, φ_2 and φ_3:*

- $\models \varphi_1 \to (\varphi_2 \to \varphi_1)$;
- $\models (\varphi_1 \to (\varphi_2 \to \varphi_3)) \to ((\varphi_1 \to \varphi_2) \to (\varphi_1 \to \varphi_3))$;
- $\models (\neg\varphi_1 \to \neg\varphi_2) \to (\varphi_2 \to \varphi_1)$;
- *If* $\models \varphi_1$ *and* $\models \varphi_1 \to \varphi_2$ *then* $\models \varphi_2$.

Proof. Left as an exercise to the reader (Exercise 14). □

2.3.2 Proof System and Soundness

Consider the following proof system based on the properties proved in Proposition 2:

(*Axiom*$_1$)	$\varphi_1 \to (\varphi_2 \to \varphi_1)$
(*Axiom*$_2$)	$(\varphi_1 \to (\varphi_2 \to \varphi_3)) \to ((\varphi_1 \to \varphi_2) \to (\varphi_1 \to \varphi_3))$
(*Axiom*$_3$)	$(\neg\varphi_1 \to \neg\varphi_2) \to (\varphi_2 \to \varphi_1)$
(*Modus Ponens*)	$\dfrac{\varphi_1 \qquad \varphi_1 \to \varphi_2}{\varphi_2}$

As explained in Section 2.1.5, the above are in fact axiom and proof rule *schemata*, that is, each stands for infinitely many instances of the metavariables φ_1, φ_2 and φ_3.

We let Γ, Γ', Γ_0, etc., range over sets of propositions. Following the general notation and terminology in Section 2.1.5, $\Gamma \vdash \varphi$ denotes that φ can be derived from the set of propositions Γ using the proof system above. Also, $\vdash \varphi$ is a shorthand for $\emptyset \vdash \varphi$, and such a proposition φ is called a *theorem* of propositional logic.

Theorem 1. ***(Soundness)*** $\Gamma \vdash \varphi$ *implies* $\Gamma \models \varphi$. *In particular,* $\vdash \varphi$ *implies* $\models \varphi$ *(when* $\Gamma = \emptyset$*).*

Proof. Follows easily by structural induction on the proof derivation tree of $\Gamma \vdash \varphi$, using Proposition 2. □

Proposition 3. $\varphi \to \varphi$, $\top$, $\varphi \to \top$, *and* $\bot \to \varphi$ *are all theorems of propositional logic.*

Proof. For $\vdash \varphi \to \varphi$, use an *Axiom*$_2$ instance with $\varphi_1 \mapsto \varphi$ and $\varphi_2 \mapsto (\varphi \to (\varphi \to \varphi))$ and $\varphi_3 \mapsto \varphi$, followed by two applications of *Modus Ponens* with two different instances of *Axiom*$_1$. Since $\top \equiv \bot \to \bot$, $\vdash \top$ can be derived similarly to how $\vdash \varphi \to \varphi$ was derived above. $\vdash \varphi \to \top$ follows by *Modus Ponens* from $\vdash \top$ and an *Axiom*$_1$ instance with $\varphi_1 \mapsto \top$ and $\varphi_2 \mapsto \varphi$. Finally, $\vdash \bot \to \varphi$ follows by *Modus Ponens* from $\vdash \neg\varphi \to \top$ (which can be derived similarly to $\vdash \varphi \to \top$) and an instance of *Axiom*$_3$ with $\varphi_1 \mapsto \varphi$ and $\varphi_2 \mapsto \bot$. □

Theorem 2. ***(Deduction)*** $\Gamma \vdash \varphi_1 \to \varphi_2$ *iff* $\Gamma \cup \{\varphi_1\} \vdash \varphi_2$.

Proof. If $\Gamma \vdash \varphi_1 \to \varphi_2$ then $\Gamma \cup \{\varphi_1\} \vdash \varphi_2$ follows easily by *Modus Ponens*. For the other implication, we show by structural induction on the proof derivation tree that for any derived formula φ in the derivation tree of $\Gamma \cup \{\varphi_1\} \vdash \varphi_2$ we can also derive $\Gamma \vdash \varphi_1 \to \varphi$. The result will then follow by taking φ to be φ_2.

If φ is an axiom (instance) or a formula in Γ, then we can use *Axiom*$_1$ instantiated with $\varphi_1 \mapsto \varphi$ and $\varphi_2 \mapsto \varphi_1$, followed by *Modus Ponens* instantiated with $\varphi_1 \mapsto \varphi$ and $\varphi_2 \mapsto (\varphi_1 \to \varphi)$, and derive $\Gamma \vdash \varphi_1 \to \varphi$.

If φ is φ_1 then $\Gamma \vdash \varphi_1 \to \varphi$ by Proposition 3.

If φ is derived using *Modus Ponens* from previously derived formulae ψ and $\psi \to \varphi$, then $\Gamma \vdash \varphi_1 \to \psi$ and $\Gamma \vdash \varphi_1 \to (\psi \to \varphi)$ are derivable (the induction hypothesis), so we can also derive $\Gamma \vdash \varphi_1 \to \varphi$ by *Axiom*$_2$ with instance $\varphi_1 \mapsto \varphi_1$ and $\varphi_2 \mapsto \psi$ and $\varphi_3 \mapsto \varphi$, using *Modus Ponens* twice. □

Proposition 4. *$\vdash \varphi \to \neg\neg\varphi$ and $\vdash \neg\neg\varphi \to \varphi$.*

Proof. $\{\varphi, \varphi \to \bot\} \vdash \bot$ by *Modus Ponens*. Then the Deduction Theorem 2 applied twice yields $\vdash \varphi \to \neg\neg\varphi$. We can similarly show $\vdash \neg\varphi \to \neg\neg\neg\varphi$, which together with $Axiom_3$ instantiated with $\varphi_1 \mapsto \varphi$ and $\varphi_2 \mapsto \neg\neg\varphi$, yields $\vdash \neg\neg\varphi \to \varphi$ by *Modus Ponens*. □

Corollary 1. *$\Gamma \vdash \varphi$ iff $\Gamma \vdash \neg\neg\varphi$ iff $\Gamma \cup \{\neg\varphi\} \vdash \bot$.*

Proof. Deduction Theorem 2 implies $\Gamma \cup \{\neg\varphi\} \vdash \bot$ iff $\Gamma \vdash \neg\varphi \to \bot$. The rest follows by Proposition 4. □

2.3.3 Consistency and Model Existence

Consistency is a crucial instrument for the proof of the Completeness Theorem in Section 2.3.4. A set of propositions is consistent iff it cannot be used to derive $\bot$. The main result of this section states that a set of propositions is consistent iff it admits a model.

Definition 10. *Γ is **inconsistent** iff $\Gamma \vdash \bot$, and is **consistent** iff $\Gamma \nvdash \bot$. Moreover, Γ is **maximally consistent** iff (1) Γ is consistent and (2) $\Gamma \subseteq \Gamma'$ and Γ' consistent imply $\Gamma = \Gamma'$.*

Proposition 5. *Γ is inconsistent iff $\Gamma \vdash \varphi$ for any proposition φ.*

Proof. If $\Gamma \vdash \varphi$ for any proposition φ, then also $\Gamma \vdash \bot$, so Γ inconsistent. If $\Gamma \vdash \bot$ then since $\vdash \bot \to \varphi$ for any φ (by Proposition 3), it follows by *Modus Ponens* that $\Gamma \vdash \varphi$ for any proposition φ. □

Proposition 6. *Suppose that Γ is consistent and φ is any proposition. Then:*

1. *$\Gamma \cup \{\varphi\}$ is consistent, or $\Gamma \cup \{\neg\varphi\}$ is consistent, or both;*
2. *If Γ is maximally consistent, then either $\varphi \in \Gamma$ or $\neg\varphi \in \Gamma$.*

Proof. For the first part, if both $\Gamma \cup \{\varphi\}$ and $\Gamma \cup \{\neg\varphi\}$ are inconsistent, then $\Gamma \vdash \neg\varphi$ and $\Gamma \vdash \neg\varphi \to \bot$ by Deduction Theorem 2, so *Modus Ponens* implies Γ inconsistent. The second is immediate from the first. □

Corollary 2. *Suppose that Γ is maximally consistent and φ is any proposition. Then $\Gamma \vdash \varphi$ iff $\varphi \in \Gamma$.*

Proof. If $\varphi \in \Gamma$ then obviously $\Gamma \vdash \varphi$. Conversely, suppose $\Gamma \vdash \varphi$. If $\varphi \notin \Gamma$ then $\neg\varphi \in \Gamma$ by Proposition 6, which together with $\Gamma \vdash \varphi$ yield $\Gamma \vdash \bot$ by *Modus Ponens*, which contradicts the consistency of Γ. □

Proposition 7. *If Γ is maximally consistent, then $\varphi_1 \to \varphi_2 \in \Gamma$ iff $\varphi_1 \in \Gamma$ implies $\varphi_2 \in \Gamma$.*

Proof. If $\varphi_1 \to \varphi_2 \in \Gamma$ and $\varphi_1 \in \Gamma$, then $\varphi_2 \in \Gamma$ by *Modus Ponens* and Corollary 2. Conversely, suppose that $\varphi_1 \in \Gamma$ implies $\varphi_2 \in \Gamma$. We can distinguish two cases: (1) $\varphi_2 \in \Gamma$; and (2) $\varphi_1 \notin \Gamma$, that is, $\neg\varphi_1 \in \Gamma$ (by Proposition 6). In the first case, $\varphi_1 \to \varphi_2 \in \Gamma$ by *Modus Ponens* with an instance of $Axiom_1$ with $\varphi_1 \mapsto \varphi_2$ and $\varphi_2 \mapsto \varphi_1$, and by Corollary 2. In the second case, $\varphi_1 \to \varphi_2 \in \Gamma$ by *Modus Ponens* with the theorem $\neg\varphi_1 \to (\varphi_1 \to \varphi_2)$ and by Corollary 2 again. That $\neg\varphi_1 \to (\varphi_1 \to \varphi_2)$ is a theorem follows easily by Deduction Theorem 2 and $\vdash \bot \to \varphi_2$ (which follows by Proposition 5). □

Theorem 3. ***(Lindenbaum)*** *Γ consistent implies there exists some Γ' maximally consistent such that $\Gamma \subseteq \Gamma'$.*

Proof. Since the set of propositional variables *PVar* is countable, the entire (infinite) set of propositions is countable. Let us enumerate them in some arbitrary but fixed order, say φ_1, φ_2, ..., and let us build the (infinite) sequence of consistent sets of propositions $\Gamma_0 \subseteq \Gamma_1 \subseteq \Gamma_2 \subseteq \ldots$ as follows: $\Gamma_0 = \Gamma$ and for each $n > 0$, Γ_n is one of $\Gamma_{n-1} \cup \{\varphi\}$ or $\Gamma_{n-1} \cup \{\neg\varphi\}$ which is consistent; if both are consistent, then we arbitrarily pick one. Since Γ_0 is consistent, by Proposition 6 it inductively follows that each of Γ_n is consistent.

Let Γ' be $\bigcup_{n\geq 0} \Gamma_n$. We claim that Γ' is consistent. Indeed, if $\Gamma' \vdash \bot$ then there is a finite proof derivation of $\bot$ from Γ', involving only a finite number of propositions in Γ' (by Proposition 1). In other words, there is some $n \geq 0$ such that all those propositions are in Γ_n, which implies $\Gamma_n \vdash \bot$, which contradicts Γ_n consistent. We claim that Γ' is also maximal. Indeed, suppose that there is some φ such that $\varphi \notin \Gamma'$ and $\Gamma' \cup \{\varphi\}$ is consistent. Since we enumerated all propositions, there exists an $n \geq 0$ such that $\varphi = \varphi_n$. Since $\varphi_n \notin \Gamma'$, it follows that $\varphi_n \notin \Gamma_n$, so it must be that $\neg\varphi_n \in \Gamma_n$, that is, $\neg\varphi \in \Gamma'$ (since $\varphi = \varphi_n$ and $\Gamma_n \subseteq \Gamma'$), which contradicts $\Gamma' \cup \{\varphi\}$ consistent. □

Theorem 4. ***(Model Existence)*** *Γ consistent iff $\rho \models \Gamma$ for some model ρ.*

Proof. If $\rho \models \Gamma$ for some model ρ, then Γ must be consistent: otherwise, if $\Gamma \vdash \bot$, then by Soundness Theorem 1 we get $\Gamma \models \bot$, which leads to a contradiction, namely $\rho \models \bot$.

Conversely, suppose that Γ is consistent. Let Γ' be a maximally consistent set with $\Gamma \subseteq \Gamma'$, which exists thanks to Lindenbaum Theorem 3. Let us define a model $\rho : PVar \to \{true, false\}$ as follows: for each $p \in PVar$, $\rho(p) = true$ iff $p \in \Gamma'$. Proposition 6 implies that ρ is indeed well-defined.

To show $\rho \models \Gamma$, we show by structural induction on φ the stronger result "$\rho \models \varphi$ iff $\varphi \in \Gamma'$". The cases $\varphi \in PVar$ and $\varphi \equiv \bot$ are trivial. If $\varphi \equiv \varphi_1 \to \varphi_2$ then $\varphi \in \Gamma'$ iff (by Proposition 7) $\varphi_1 \in \Gamma'$ implies $\varphi_2 \in \Gamma'$, iff (by the induction hypothesis) $\rho \models \varphi_1$ implies $\rho \models \varphi_2$, iff $\rho \models \varphi$. □

2.3.4 Completeness and Compactness

The Completeness Theorem 5 below states that the three axiom schemata together with *Modus Ponens* are in fact capable of deriving any tautology of propositional calculus. We prefer to present the Compactness Theorem 6 as an immediate corollary of the Soundness Theorem 1 and the Completeness Theorem 5, although the reader should note that in other places it is presented as a corollary of the Model Existence Theorem 4.

Theorem 5. ***(Completeness)*** *$\Gamma \models \varphi$ implies $\Gamma \vdash \varphi$. In particular, $\models \varphi$ implies $\vdash \varphi$ (when $\Gamma = \emptyset$).*

Proof. Suppose that $\Gamma \models \varphi$ and $\Gamma \nvdash \varphi$. Then $\Gamma \cup \{\neg\varphi\}$ is consistent due to Corollary 1, so Theorem 4 implies that $\rho \models \Gamma$ and $\rho \not\models \varphi$ for some model ρ, which contradicts $\Gamma \models \varphi$. □

Theorem 6. ***(Compactness)*** *If $\Gamma \models \varphi$ then there is a finite subset Γ_0 of Γ such that $\Gamma_0 \models \varphi$.*

Proof. If $\Gamma \models \varphi$ then $\Gamma \vdash \varphi$ by the Completeness Theorem (Theorem 5). By Proposition 1 there is some finite subset Γ_0 of Γ such as $\Gamma_0 \vdash \varphi$. Finally, by the Soundness Theorem (Theorem 1) it follows that $\Gamma_0 \models \varphi$. □

2.3.5 Notes

Propositional logic evolved over a period of more than 2000 years from general advances in philosophy, mathematics and logic, where a basic calculus with propositions and truth values could simply not be avoided. For example, the Greek Stoics were well-known for their use of rigorous logic in the 3rd century BC. Although the origins of propositional logic as we know it today can be and have been pushed back to the 17th century to Gottfried Wilhelm Leibniz, and even to the 12th century to Peter Abelard, it is generally accepted now

that the fathers of propositional logic are George Boole (1815-1864) and Augustus De Morgan (1806-1871). Jan Łukasiewicz (1878-1956), nevertheless, has made many important contributions to propositional logic, including the proof system that we presented in this section. There are many other proof systems for propositional logics in the literature, several proposed by Łukasiewicz himself, but their soundness and completeness can be shown following a similar pattern. The general proof style based on a model existence theorem that we followed in this section, which also applies to more complex logics, is due to Leon Albert Henkin (1921-2006). The Lindenbaum theorem, which is critical for the proof of the model existence theorem, was attributed to Adolf Lindenbaum (1904-1941) by Alfred Tarski (1901-1983).

2.3.6 Exercises

The following exercises complete the material on propositional logic in Sections 2.3.1, 2.3.2, and 2.3.3.

Exercise 14. *Prove Proposition 2.*

Exercise 15. *Using only the basic notions in Section 2.3.1, prove that* $\Gamma \models \varphi_1 \to \varphi_2$ *iff* $\Gamma \cup \{\varphi_1\} \models \varphi_2$.

Exercise 16. *Using only the basic notions in Section 2.3.1, prove that the theorems listed in Proposition 3 are all tautologies.*

Exercise 17. *Give complete proof derivations for the propositional logic theorems in Proposition 3.*

Exercise 18. *Prove* $\vdash \neg\varphi_1 \to (\varphi_1 \to \varphi_2)$*, both directly using the proof system and indirectly using the Deduction Theorem (Theorem 2). This theorem is used in Proposition 7, which can be consulted for a hint.*

Exercise 19. *Prove* $\vdash (\varphi_1 \to \varphi_2) \to ((\varphi_2 \to \varphi_3) \to (\varphi_1 \to \varphi_3))$ *and* $\vdash (\varphi_1 \to \varphi_2) \to (\neg\varphi_2 \to \neg\varphi_1)$.

Exercise 20. *If* Γ *is maximally consistent then:*

- $\varphi_1 \wedge \varphi_2 \in \Gamma$ *iff* $(\varphi_1 \in \Gamma$ *and* $\varphi_2 \in \Gamma)$*; and*
- $\varphi_1 \vee \varphi_2 \in \Gamma$ *iff* $(\varphi_1 \in \Gamma$ *or* $\varphi_2 \in \Gamma)$*.*
- $\varphi_1 \leftrightarrow \varphi_2 \in \Gamma$ *iff* $(\varphi_1 \in \Gamma$ *iff* $\varphi_2 \in \Gamma)$*.*

Should we move after the RL section and add one or two ACI rewriting decision procedures for propositional logic? Similar question for FOL, once added.

2.4 Equational Logic

define lists, sets, bags

say that we allow Boolean terms b in conditions of equations and rules, as syntactic sugar for $b = true$

add the Π^0_2 result for initial model satisfaction here: grab material from the corresponding directory under "Current Work".

add a subsection on Σ-algebras

say this once and for all in this section when discussing the property of the initial algebra, and make sure that it does not repeat in the rest of the book: "we let θ also denote the homomorphic extension $T_\Sigma(\mathcal{X}) \to T_\Sigma$ of $\theta : \mathcal{X} \to T_\Sigma$ to terms with variables in $\mathcal{X}$"

use calligraphic font as above for sets, as we use X for one variable in Maude specs.

algebraic specification, equational theory, equational specification

...

2.4.1 Signatures, Algebras, Equations, Satisfaction

...

2.4.2 Proof System and Soundness

...

sorts:
$Cell$, $Tape$, $Configuration$
operations:
$0, 1 \ : \ \rightarrow Cell$
$zeros \ : \rightarrow Tape$
$_ : _ \ : \ Cell \times Tape \rightarrow Tape$
$q \ : \ Tape \times Tape \rightarrow Configuration$ — one such operation for each $q \in Q$
generic equation:
$zeros = 0 : zeros$
specific equations:
$q(L, b{:}R) = q'(L, b'{:}R)$ — one equation for each $q, q' \in Q$, $b, b' \in Cell$ with $M(q, b) = (q', b')$
$q(L, b{:}R) = q'(b{:}L, R)$ — one equation for each $q, q' \in Q$, $b \in Cell$ with $M(q, b) = (q', \rightarrow)$
$q(B{:}L, b{:}R) = q'(L, B : b{:}R)$ — one equation for each $q, q' \in Q$, $b \in Cell$ with $M(q, b) = (q', \leftarrow)$

Figure 2.2: Lazy equational logic representation $\mathcal{E}^{lazy}_{\mathcal{M}}$ of Turing machine $\mathcal{M}$

2.4.3 Computation as Equational Deduction

Here we discuss simple equational logic encodings of Turing machines (see Section 2.2.1 for general Turing machine notions). The idea is to associate an equational theory to any Turing machine, so that an input is accepted by the Turing machine if and only if an equation corresponding to that input can be proved from the equational theory of the Turing machine, using conventional equational deduction. Moreover, as seen in Section 2.5.3, the resulting equational theories can be executed as rewrite theories by rewrite engines, thus yielding actual Turing machine interpreters. We present two encodings, both based on intuitions from lazy data-structures, specifically stream data-structures. The first is simpler but requires lazy rewriting support from rewrite engines in order to be executed, while the second can be executed by any rewrite engines.

Lazy Equational Representation

Our first representation of Turing machines in equational logic is based on the idea that the infinite tape can be finitely represented by means of self-expanding stream data-structures. In spite of being infinite sequences of cells, like the Turing machine tapes, many interesting streams can be finitely specified using equations. For example, the stream of zeros, $zeros = 0 : 0 : 0 : \cdots$, can be defined as $zeros = 0 : zeros$. Since at any given moment the portions of a Turing machine tape to the left and to the right of the head have a suffix consisting of an infinite sequence of 0 cells, it is natural to represent them as streams of the form $b_1{:}b_2{:}\cdots b_n{:}zeros$. When the head is on cell b_n and the command is to move the head to the right, the self-expanding equational definition of *zeros* can produce one more 0, so that the head can move onto it. To expand *zeros* on a by-need basis and thus to avoid undesired non-termination due to the uncontrolled application of the self-expanding equation of *zeros*, this approach requires an equational/rewrite engine with support for lazy evaluation/rewriting in order to be executed.

Figure 2.2 shows how a Turing machine $\mathcal{M} = (Q, B, q_s, q_h, C, M)$ can be associated a computationally equivalent equational logic theory $\mathcal{E}^{lazy}_{\mathcal{M}}$. Except for the self-expanding equation of the *zeros* stream and our stream representation of the two-infinite-end tape, the equations of $\mathcal{E}^{lazy}_{\mathcal{M}}$ are identical to the transition relation on Turing machine configurations discussed right after Definition 7. The self-expanding equation of *zeros*

guarantees that enough 0's can be provided when the head reaches one or the other end of the sequence of cells visited so far. The result below shows that $\mathcal{E}_{\mathcal{M}}^{lazy}$ is proof-theoretically equivalent to $\mathcal{M}$:

Theorem 7. *The following are equivalent:*

(1) The Turing machine $\mathcal{M}$ terminates on input $b_1 b_2 \ldots b_n$;

(2) $\mathcal{E}_{\mathcal{M}}^{lazy} \models q_s(zeros, b_1{:}\, b_2{:} \cdots {:}\, b_n{:}zeros) = q_h(l, r)$ for some terms l, r of sort Tape.

Proof. ... □

The equations in $\mathcal{E}_{\mathcal{M}}^{lazy}$ can be applied in any direction, so an equational proof of $\mathcal{E}_{\mathcal{M}}^{lazy} \models q_s(zeros, b_1 : b_2 : \cdots : b_n : zeros) = q_h(l, r)$ needs not necessarily correspond step-for-step to the computation of $\mathcal{M}$ on input $b_1 b_2 \ldots b_n$. We will see in Section 2.5.3 that by orienting the specific equations in Figure 2.2 into rewrite rules, we will obtain a rewrite logic theory which will faithfully capture, step-for-step, the computational granularity of $\mathcal{M}$.

Note that in Figure 2.2 we preferred to define a configuration construct $q : Tape \times Tape \to Configuration$ for each $q \in Q$. A natural alternative could have been to define an additional sort *State* for the Turing machine states, a constant $q : \to State$ for each $q \in Q$, and one generic configuration construct $_(_,_) : State \times Tape \times Tape \to Configuration$, as we do in the subsequent representation of Turing machines as rewrite logic theories (see Figure 2.3). The reason for which we did not do that here is twofold: first, in functional languages like Haskell it is very natural to associate a function to each such configuration construct $q : Tape \times Tape \to Configuration$, while it would take some additional effort to implement the second approach; second, the approach in this section is more compact than the one below.

Unrestricted Equational Representation

The equational representation of Turing machines above is almost as simple as it can be and, additionally, can be easily executed on programming languages or rewrite engines with support for lazy evaluation/rewriting, such as Haskell or Maude (see, e.g., Section 2.5.6). However, the fact that it requires lazy evaluation/rewriting and that the equivalence classes of configurations have infinitely many terms, its use is limited to systems that support strategies. Here we show that a simple idea can turn the representation in the previous section into an elementary one which can be executed on any equational/rewrite engines: replace the self-expanding and non-terminating (when regarded as a rewrite rule) equation "$zeros = 0{:}zeros$" with configuration equations of the form "$q(zeros, R) = q(0{:}zeros, R)$" and "$q(L, zeros) = q(L, 0{:}zeros)$"; these equations achieve the same role of expanding *zeros* by need, but avoid non-termination when applied as rewrite rules.

Figure 2.3 shows our unrestricted representation of Turing machines as equational logic theories. There are some minor differences between the representation in Figure 2.3 and the one in Figure 2.2. For example, note that in order to add the two equations above for the expanding of *zeros* in a generic manner for any state, we separate the states from the configuration construct. In other words, instead of having an operation $q : Tape \times Tape \to Configuration$ for each $q \in Q$ like in Figure 2.2, we now have one additional sort *State*, a generic configuration construct $_(_,_) : State \times Tape \times Tape \to Configuration$, and a constant $q : \to State$ for each $q \in Q$. This change still allows us to write configurations as terms $q(l, r)$, so we do not need to change the equations corresponding to the Turing machine transitions. With this modification in the signature, we can now remove the troubling equation $zeros = 0{:}zeros$ from the representation in Figure 2.2 and replace it with the two safe equations in Figure 2.3. Let $\mathcal{E}_{\mathcal{M}}$ be the equational logic theory in Figure 2.3.

Theorem 8. *The following are equivalent:*

sorts:

$Cell,\ Tape,\ State,\ Configuration$

operations:

$0, 1\ :\ \rightarrow Cell$

$zeros\ :\rightarrow Tape$

$_ : _\ :\ Cell \times Tape \rightarrow Tape$

$_(_, _)\ :\ State \times Tape \times Tape \rightarrow Configuration$

$q\ :\rightarrow State$ — one such constant for each $q \in Q$

generic equations:

$S(zeros, R) = S(0{:}zeros, R)$

$S(L, zeros) = S(L, 0{:}zeros)$

specific equations:

$q(L, b{:}R) = q'(L, b'{:}R)$ — one equation for each $q, q' \in Q$, $b, b' \in Cell$ with $M(q, b) = (q', b')$

$q(L, b{:}R) = q'(b{:}L, R)$ — one equation for each $q, q' \in Q$, $b \in Cell$ with $M(q, b) = (q', \rightarrow)$

$q(B{:}L, b{:}R) = q'(L, B : b{:}R)$ — one equations for each $q, q' \in Q$, $b \in Cell$ with $M(q, b) = (q', \leftarrow)$

Figure 2.3: Unrestricted equational logic representation $\mathcal{E}_{\mathcal{M}}$ of Turing machine $\mathcal{M}$

(1) The Turing machine $\mathcal{M}$ terminates on input $b_1 b_2 \ldots b_n$;

(2) $\mathcal{E}_{\mathcal{M}} \models q_s(zeros, b_1{:}\, b_2{:} \cdots : b_n{:}zeros) = q_h(l, r)$ for some terms l, r of sort Tape.

Proof. ... □

One could argue that deduction with the equational theory in Figure 2.3 is not fully faithful to computations with the original Turing machine, because the two generic equations may need to artificially apply from time to time as an artifact of our representation, and their application does not correspond to actual computational steps in the Turing machine. In fact, these generic equations can be completely eliminated, at the expense of more equations. For example, if $M(q, b) = (q', \leftarrow)$ then, in addition to the last equation in Figure 2.3, we can also include the equation:

$$q(zeros, b{:}R) = q'(zeros, 0 : b{:}R)$$

This way, one can expand *zeros* and apply the transition in *one* equational step. Doing that systematically for all the transitions allows us to eliminate the need for the two generic equations entirely.

2.4.4 Initial Algebra

Why are they called "initial" models?

...

The next result shows that the problem of equational satisfaction in an initial algebra is a Π^0_2-complete problem. That tells us that there is no way to automate the process of proving equalities or inequalities in initial algebras in general; in particular, induction cannot be automated in general.

Theorem 9. *Equational satisfaction in an initial algebra is a Π^0_2-complete problem.*

Proof. Let us first show the membership in Π^0_2 part. We have to show that the problem taking as input an equational specification (Σ, E) and a (not necessarily ground) Σ-equation e and saying whether $T_{\Sigma,E} \models e$ is of the form $(\forall i)(\exists j)\ r(i, j, k)$, where i and j range over natural numbers, k is the Gödel number corresponding to the input (Σ, E) and e, and r is some recursive, or decidable, property. Let us fix an input to the initial algebra equational problem, that is, an equational specification (Σ, E) and a Σ-equation e, say $(\forall X)\ t = t'$. Since we can enumerate, or Gödelize, all mappings $X \to T_{\Sigma,E}$, there is a one-to-one correspondence between natural number indexes i and mappings $\theta : X \to T_{\Sigma,E}$. Let i_θ denote the index corresponding to substitution θ and let θ_i denote the i-th substitution; then obviously $i_{\theta_i} = i$ and $\theta_{i_\theta} = \theta$ for any i and θ. The Σ-terms t and t' over variables in X are taken by any map $\theta : X \to T_{\Sigma,E}$ into equivalence classes of ground terms in $T_{\Sigma,E}$. Therefore, $T_{\Sigma,E} \models e$ iff $\theta(t) \equiv_E \theta(t')$ for any mapping $\theta : X \to T_{\Sigma,E}$. However, recall that $\theta(t) \equiv_E \theta(t')$ iff $E \vdash (\forall\emptyset)\theta(t) = \theta(t')$, that is, iff and only if *there exists some proof derivation* of $(\forall\emptyset)\ \theta(t) = \theta(t')$ from E using the equational proof system. Since we can enumerate all the proofs derived from an r.e. set of equations E, there is a one-to-one correspondence between natural number indexes j and proof derivations. Let j_π denote the index corresponding to proof π and let π_j denote the j-th proof; then obviously $j_{\pi_j} = j$ and $\pi_{j_\pi} = \pi$ for any j and π. Therefore,

$$T_{\Sigma,E} \models e \text{ iff } (\forall i)(\exists j)\ r(i, j, k),$$

where i ranges over mappings $\theta : X \to T_{\Sigma,E}$, j over equational proofs, and $r(i, j, k)$ is the test whether proof π_j is correct for the task $E \vdash (\forall\emptyset)\ \theta_i(t) = \theta_i(t')$; since proof checking is decidable, it follows that the equational satisfaction problem in an initial model is a Π^0_2 problem.

Let us now focus on the Π^0_2-hardness part. It suffices to build a particular finite equational specification $\mathcal{E} = (\Sigma, E)$ and a Σ-equation e, such that $T_{\Sigma,E} \models e$ is a Π^0_2-hard problem; note that e must have variables (the satisfaction problem for ground equations is no different in initial algebras than in general, so it is r.e. – see Proposition 8 and the discussion following it). Let us start with $\mathcal{E}$; we build it as an extension of $\mathcal{E}_\mathcal{U}$, the equational specification corresponding to a Turing machine constructed following the general procedure in Section **??** but applied to $\mathcal{U}$, the universal Turing machine in Section **??** that makes the Totality problem Π^0_2-complete. Let $\mathcal{E} = (\Sigma, E)$ be the finite equational specification obtained by adding to $\mathcal{E}_\mathcal{U}$ the following:

- A sort *Nat*, a constant $zero :\to Nat$ and a unary operation $succ : Nat \to Nat$;
- An operation $ones : Nat \times List \to List$ together with equations

$$(\forall L{:}List)\ ones(zero, L) = L,$$
$$(\forall N{:}Nat, L{:}List)\ ones(succ(N), L) = ones(N, 1{:}L);$$

- An operation $u : Nat \times Nat \to State$ together with the equation

$$(\forall J, K{:}Nat)\ u(J, K) = q_s(ones(J, 0 : ones(K, nil))).$$

Then by Proposition **??** and 1 in Proposition 8, the Totality problem is equivalent to the following problem, where $\overline{n}$ is the Peano representation $succ(\cdots(succ(zero))\cdots)$ of the natural number n, that is, the term of sort *Nat* consisting of a sequence of n successor operators applied on *zero*:

Input: An integer $k \geq 0$;
Output: Does $E \models (\forall\emptyset)\ u(\overline{j}, \overline{k}) = \bot$ for all $j \geq 0$?

We would like to transform the infinitary conjunction of satisfaction relations in the output of the problem above into only one satisfaction relation, but in an initial model. We claim that the problem above is entirely equivalent to the following problem:

Input: An integer $k \geq 0$;
Output: Does $T_{\Sigma,E} \models (\forall J\!:\!Nat)\ u(J, \overline{k}) = \bot$?

Indeed,

$T_{\Sigma,E} \models (\forall J\!:\!Nat)\ u(J, \overline{k}) = \bot$	iff
$T_{\Sigma,E} \models (\forall\emptyset)\ \theta(u(J, \overline{k})) = \bot$ for all $\theta : J \to T_{\Sigma,E}$	iff
$T_{\Sigma,E} \models (\forall\emptyset)\ (u(\overline{j}, \overline{k})) = \bot$ for all $j \geq 0$	iff
$E \models (\forall\emptyset)\ u(\overline{j}, \overline{k}) = \bot$ for all $j \geq 0$	(by 1 in Proposition 8).

Therefore, the satisfaction problem in an initial algebra is at least as hard as the Totality problem, Π^0_2. Since we proved that it is in Π^0_2, it follows that it is a Π^0_2-complete problem. □

2.4.5 Completeness

...

Let Σ be a many-sorted signature which contains recursively enumerable (r.e.) sets of sorts and of operations, the latter of finite arity. Let T_Σ denote the *initial* Σ*-algebra* of *ground* (i.e., containing no variables) Σ-terms; if Σ contains no constants then T_Σ is obviously empty. Let *Var* be a set of variables that is infinite and r.e. on each sort that appears in Σ. If $X \subseteq Var$ is a set of variables then we let $\Sigma \cup X$ denote the signature Σ extended with the variables in X regarded as *constants*. Also, let $T_\Sigma(X)$ denote the *free algebra* of Σ-terms over variables in X, that is, $T_{\Sigma\cup X}$. Equations are triples $(\forall X)\ t = t'$, where t and t' are Σ-terms over variables in X. If X is empty then the equation is called *ground*. Let (Σ, E) be any equational theory where E is an r.e. set of Σ-equations. We assume that all equations are unconditional but can contain variables. We let $T_{\Sigma,E}$ denote the *initial* (Σ, E)*-algebra*. Recall that the initial (Σ, E)-algebra contains equivalence classes of ground terms as elements, where two terms are equivalent iff they can be *proved*, or *derived* equal using E; in other words, $T_{\Sigma,E}$ is the quotient of T_Σ by $\equiv_E$, where $t \equiv_E t'$ iff $E \vdash (\forall\emptyset)\ t = t'$, where $\vdash$ is the standard equational derivability relation.

Proposition 8. *[27] If e is any Σ-equation then the following hold:*

1. *(Soundness and completeness) $E \models e$ iff $T_{\Sigma\cup X,E} \models e$ iff $E \vdash e$;*
2. *(Initial algebra semantics) If e is ground then $E \models e$ iff $T_{\Sigma,E} \models e$.*

The first item above tells us that the equational satisfaction problem is r.e.. Indeed, one can use the deduction system of equational logics to enumerate all proofs; the completeness theorem of equational logics ensures that this enumeration process eventually reaches any semantic equational consequence of the equational theory, with or without variables. The second item, which is a corollary of the first when X is

empty, tells us that the initial algebra of an equational theory "knows everything" about *ground* equalities; this is not surprising, because the initial algebra is built by collapsing ground terms that can be shown equal. An immediate consequence of the two items above is that the problem of ground satisfaction in an initial algebra is r.e.

> ***Pitfall.*** One should not get tricked by the results above and conclude that the initial algebra can simply replace the original equational theory, thinking that any property can be tested for satisfaction now in the initial algebra instead of the original theory. That only holds for ground equalities! The initial algebra, of course, satisfies all the equational consequences of the original theory, but it can and typically does also satisfy many other non-ground equational properties. For example, the initial algebra of the Peano equational theory of natural numbers, i.e., the actual natural numbers, satisfies the commutativity of addition; nevertheless, there are many other algebras of that equational theory that do not satisfy commutativity of addition. There is indeed an initial algebra that captures precisely all the equational properties of E quantified over variables in X, but that initial algebra is one over a larger signature, namely $T_{\Sigma \cup X,E}$, not $T_{\Sigma,E}$. Unfortunately, adding constants to an initial algebra significantly reduces the set of properties that hold in that algebra; in particular, the commutativity of addition does *not* hold in the algebra of natural numbers extended with two constants.

On Models

From now on we may use the word *model* to refer to implementations. This terminology comes from mathematical logics and *model theory*, so it has a more mathematical flavor. Anyhow, we should think of both implementations and models as *realizations* of specifications.

A specification can have therefore several models: all those satisfying its properties. However, not all models are always intended. In this lecture we discuss *initial models*, for which *induction* is a valid proof technique, and then we discuss the relationship between these and *recursively defined operations*.

Bad Models

Among the models of a specification, there are some which have quite unexpected properties. For example, a simple specification of lists of bits without any equations, can be defined as follows:

```
mod BIT-LIST is
  sorts Bit BitList .  subsort Bit < BitList .
  ops 0 1 : -> Bit .
  op nil : -> BitList .
  op _,_ : Bit BitList -> BitList .
endm
```

One possible model, $\mathcal{M}$, can do the following:

- Implement elements of sort `BitList` as *real numbers* (yes, it looks strange but there is nothing to forbid this so far),
- Implement `nil` and `0` as *0*, and `1` as *1*,
- Implement `_,_` as *addition*.

The model $\mathcal{M}$ has very strange properties, such as

Junk

There are lots of lists in $\mathcal{M}$ which are not intended to be lists of bits, such as, for example, the number π.

Confusion

There are distinct lists which in fact collapse when interpreted in $\mathcal{M}$, such as, for example, the lists `0,1` and `1,0`. Concatenation becomes commutative in $\mathcal{M}$, which is highly undesirable.

Initial Models

Initial models are those with *no junk* and *no confusion*. How can we define such a model?

For the specification of lists of bits above, we can build an initial model $\mathcal{T} = (\mathcal{T}_{Bit}, \mathcal{T}_{BitList})$ as the pair of *smallest* sets with the following properties:

1. `0` and `1` belong to $\mathcal{T}_{Bit}$;
2. $\mathcal{T}_{Bit}$ is included in $\mathcal{T}_{BitList}$;
3. `nil` is an element of $\mathcal{T}_{BitList}$, and `0,L` and `1,L` are elements of $\mathcal{T}_{BitList}$ whenever `L` is an element of $\mathcal{T}_{BitList}$.

The model $\mathcal{T}$ is exactly the desired model for lists of bits: contains nothing but lists and no two distinct lists are collapsed!

One can similarly define an initial model for any signature. Intuitively, initial models of signatures consist of *exactly* all the well-formed terms over the syntax specified by the signature.

In the case of specifications, which contain not only signatures but also sentences, initial models can be defined as well. Essentially, they are defined by

> *taking the initial models of the corresponding signatures and collapsing all terms which can be shown equal using specification's sentences.*

As an example, let us reconsider the specification of natural numbers `PEANO-NAT`:

```
mod PEANO-NAT is
  sort Nat .
  op zero : -> Nat .
  op succ : Nat -> Nat .
  op plus : Nat Nat -> Nat .
  vars N M : Nat .
  eq plus(zero, M) = M .
  eq plus(succ(N), M) = succ(plus(N, M)) .
endm
```

The initial model of its signature contains all the well-formed terms built over `zero`, `succ` and `plus` (so no variables!). When we also consider the two equations, many distinct such terms will collapse, because they become equal modulo those equations.

E.g., `plus(succ(succ(zero)), succ(succ(succ(zero))))` is equal to `plus(succ(succ(succ(succ(zero)))), succ(zero))`, so they represent only one element in the initial model of `PEANO-NAT`.

The set of all terms which are equivalent modulo the equations of a specification are typically called *equivalence classes*. Therefore, initial models of specifications have equivalence classes as elements.

From now on in this class, when we talk about models or implementations of specifications, we will actually mean *initial models*. Also, when we prove properties of specifications, we prove them as if for their initial models.

Induction

Why are we interested in initial models? Because in these models, *induction*, a very powerful proof technique very related to recursive definitions that will be intensively used in defining programming language features later in the course, is a valid proof technique.

Where is the Mistake?

To understand the important concept of initial model as well as its relationship to induction better, let us play a "where is the mistake" game. We prove by induction a property of a specification, and then we show that there are implementations which satisfy the specification but which do *not* satisfy the "proved" property.

We first prove that commutativity of `_+_` is a consequence of the specification of Peano natural numbers, this time using the mix-fix notation:

```
mod PEANO-NAT is sort Nat .
  op 0 : -> Nat .
  op s : Nat -> Nat .
  op _+_ : Nat Nat -> Nat .
```

```
  vars N M : Nat .
  eq 0 + N = N .
  eq s(M) + N = s(M + N) .
endm
```

To show that `M + N = N + M` for all natural numbers `M` and `N`, we can do a proof by *induction* on either `M` or `N`, say `M`. Since any natural number is either `0` or the successor of a smaller natural number, we need to analyze two cases:

Case 1: `M = 0`. We need to prove that `0 + N = N + 0` for any natural number `N`. By the first equation, we only need to show that for any natural number `N`, it is the case that `N + 0 = N`. The only way to show it is by induction again, this time by `N`. There are two cases again:

Case 1.1: `N = 0`. We have to show that `0 + 0 = 0`, which follows by the first equation.

Case 1.2: Assume `n + 0 = n` for some `n` and prove that `s(n) + 0 = s(n)`. By the second equation, `s(n) + 0 = s(n + 0)`, and by the induction's hypothesis, `s(n + 0) = s(n)`. Done.

Case 2: Assume that `m + N = N + m` for some `m` and for any `N`, and prove that `s(m) + N = N + s(m)` for any `N`. By the second equation and the induction hypothesis, it follows that all what is left to prove is `N + s(m) = s(N + m)`, which we prove again by induction:

Case 2.1: `N = 0`. The equality `0 + s(m) = s(0 + m)` is immediate because of the first equation.

Case 2.2: Assume that `n + s(m) = s(n + m)` for some `n` and show `s(n) + s(m) = s(s(n) + m)`. Indeed, `s(n) + s(m) = s(n + s(m)) = s(s(n + m))` by the second equation and the induction hypothesis, and `s(s(n) + m) = s(s(n + m))` by the second equation.

Therefore, by several applications of induction we proved that addition on Peano natural numbers is commutative. Hence, one would naturally expect that addition should be commutative for *any* implementation of natural numbers satisfying the Peano axioms in `PEANO-NAT`. Well, let us consider the following implementation $\mathcal{S}$:

- Natural numbers are interpreted as strings;
- `0` is interpreted as the empty string;
- `s(N)` is the string `aN`, that is, the character `a` concatenated with the string `N`;
- `_+_` is implemented as string concatenation.

$\mathcal{S}$ obviously satisfies the two axioms of `PEANO-NAT`. However, addition is *not* commutative in $\mathcal{S}$! Where is the problem? What's going on here? There is nothing wrong, just that

> *Proofs by induction are NOT valid for all possible models/implementations, but only for special ones!*

At this stage, one can admittedly say: well, but in the string model you can only build natural numbers by adding an 'a' in front of a string, so you can only get strings of a's, in which case addition is commutative. That means that one actually had in mind a special model all the time, namely one in which all the numbers

need to be *reached* starting with zero and applying successor operations. For this model, induction is indeed a valid proof principle, because this model is the initial one. But note that there is no axiom in the Peano definition of natural numbers saying that in any model natural numbers can only be obtained starting with a zero and applying a finite number of successor operations. That was only implicit in our mind, but not explicit in the axioms.

One thing to keep in mind when we talk about specifications is that they give us a set of *constraints* that the possible acceptable worlds should satisfy. They *do not place us in a unique world*. Also, in the context of programming language design, we do not want to restrict too much the class of implementations by design! One implementation of a programming language can be by using a compiler, another by using an interpreter, yet another by translating it into a different programming language.

When one proves a property about a specification, that property is expected to hold in *all* possible worlds. However, our major points above are:

1. Properties one proves by induction are *not* valid in all possible worlds, so one has to restrict the possible universes to those in which induction is valid! Those are called "reachable" in the literature, but this goes beyond the scope of the class. What's important is that the initial model is reachable; that's the model you should have in mind when you write executable language specifications.

2. Once one accepts the restricted set of possible universes in which induction is valid, then one can also have a methodological clean way to develop specifications: think of what are the constructors that you would use in doing proofs by induction, and then make sure you define your new operations for *each of those*.

From here on, we will consider only specifications whose intended models are initial, so proofs by induction are valid.

Exercise 21. *Show that addition is associative in* `PEANO-NAT` *and that multiplication is commutative and associative in* `PEANO-NAT*`*, where we also replace* `mult` *by its mix-fix variant* `_*_`*. These proofs need to be done by induction. Describe also a model/implementation of* `PEANO-NAT*`*, where multiplication is implemented in such a way that it is neither commutative nor associative. You can extend the one on strings if you wish.*

2.4.6 Defining Data-Types Equationally

---

also known as algebraic data-types

Lists

Bags

Sets

Maps

Make it clear that we define Partial Finite-Domain Maps

---

sort:
$List$
subsort:
$A < List$
operations:
$nil \quad : \quad \rightarrow List$
$_,_ \quad : \quad List \times List \rightarrow List$
equations:
$nil, L = L$
$L, nil = L$
$(L_1, L_2), L_3 = L_1, (L_2, L_3)$

Figure 2.4: Lists of elements on sort A defined as an algebraic datatype. Variables L,L_1,L_2,L_3 have sort $List$.

sort:
Bag
subsort:
$A < Bag$
operations:
$\emptyset \quad : \quad \rightarrow Bag$
$_,_ \quad : \quad Bag \times Bag \rightarrow Bag$
equations:
$\emptyset, B = B$
$B_1, B_2 = B_2, B_1$
$(B_1, B_2), B_3 = B_1, (B_2, B_3)$

Figure 2.5: Bags of elements on sort A defined as an algebraic datatype. Variables B,B_1,B_2,B_3 have sort Bag.

sort:
Set
subsort:
$A < Set$
operations:
$\emptyset \quad : \quad \rightarrow Set$
$_,_ \quad : \quad Set \times Set \rightarrow Set$
equations:
$X, X = X$
$\emptyset, S = S$
$S_1, S_2 = S_2, S_1$
$(S_1, S_2), S_3 = S_1, (S_2, S_3)$

Figure 2.6: Sets of elements on sort A defined as an algebraic datatype. Variables S,S_1,S_2,S_3 have sort Set, and X has sort A.

say that general, possibly infinite-domain partial functions, like any other non-r.e. domains, cannot be defined using any computationally-based technique; one can still define certain infinite structures, in particular partial functions, provided that one has finite presentations of them, such as recursive definitions.

---

define the update to only work on states that are defined in the variable to update.

allow $\sigma[\bot/x]$.

also, add a defined Boolean operation which checks whether a partial function is defined in a certain element.

the above are needed for consistency with the conventional executable semantics.

Partial finite-domain functions can be easily defined equationally as an algebraic data-type of sets of pairs, more precisely as sets of bindings from elements in their sources to elements in their targets, respectively. This works because partial finite-domain functions, when regarded as binary relations, are indeed finite sets of pairs. Following the presentation of partial functions in Section 2.1.2, we also define lookup and update operations on partial function. We show that well-formed structures defined this way indeed correspond to partial finite-domain functions, so they can be regarded as such partial functions for all practical purposes from now on.

Figure 2.7 shows our definition of partial finite-domain functions as an algebraic data-type. We define a sort *PFun* together with a pairing "binding" constructor $_ \mapsto _ : A \times B \to PFun$ for it and together with an associative and commutative union construct $_,_ : PFun \times PFun \to PFun$ with $\emptyset$ as its identity to put together such bindings. The equational definitions of operations $_(_) : PFun \times A \to B$ and $_[_/_] : PFun \times B \times A \to PFun$ are given by equations that operate modulo the associativity and commutativity of $_,_$, but it could also be given many different ways, including ones that do not make use of AC matching.

Note the $a \neq a'$ appearing as a condition is not a negative condition, but rather a Boolean predicate, which can be equationally defined for any constructor-based type such as the type of integers or of variables, for example. Such "built-ins" appearing in these definitions and others in this book are easily definable as initial models of corresponding equational theories. And indeed, when performing formal proofs, one will make use of these equational definitions of the built-ins.

Definition 11. *Let T be the initial algebra of the equational specification in Figure 2.7. A term $t \in T_{PFun}$ is* ***well-formed as a partial functions****, or simply* ***well-formed****, if and only if* $\textsc{Partial-Function}(A,B) \vdash t = (a_1 \mapsto b_1, \ldots, a_n \mapsto b_n)$ *for some $a_i \in A$ and $b_i \in B$ for each $i \in \{1, \ldots, n\}$, such that $a_i \neq a_j$ for any $i \neq j$. Let $T^{wf}_{PFun} \subset T_{PFun}$ denote the set of well-formed terms.*

In other words, a term of sort *PFun* is well-formed if and only if, when regarded as a binary relation, it corresponds to a partial function.

Proposition 9. *The following hold:*

1. $\emptyset \in T^{wf}_{PFun}$;
2. *If $t \in T^{wf}_{PFun}$, $a \in A$, $b \in B$, then $t[b/a] \in T^{wf}_{PFun}$;*

$\boxed{\textsc{Partial-Function}(A,B)}$

sorts:

$PFun$

operations:

$$
\begin{array}{rcl}
_ \mapsto _ & : & A \times B \to PFun \\
\emptyset & : & \to PFun \\
_ , _ & : & PFun \times PFun \to PFun \\
_(_) & : & PFun \times A \to B \\
_[_/_] & : & PFun \times B \times A \to PFun
\end{array}
$$

equations:

$$
\begin{array}{l}
(F_1, F_2), F_3 = F_1, (F_2, F_3) \\
F_1, F_2 = F_2, F_1 \\
\emptyset, F = F \\
\\
(F, a \mapsto b)(a) = b \\
(F, a \mapsto b)[b'/a] = (F, a \mapsto b') \\
(F, a \mapsto b)[b'/a'] = (F[b'/a'], a \mapsto b) \quad \textbf{if} \quad a \neq a' \\
\emptyset[b/a] = a \mapsto b
\end{array}
$$

Figure 2.7: Partial finite-domain functions defined as an algebraic datatype. We assume the definition of the sorts A and B already given. The variables a,a' have the sort A, the variables b,b' have sort B, and F,F_1,F_2,F_3 have sort $PFun$.

3. *If* $(t_1, t_2) \in T^{wf}_{PFun}$ *then* $t_1 \in T^{wf}_{PFun}$ *and* $t_2 \in T^{wf}_{PFun}$.

Thus, if one always starts with the term $\emptyset$ and then only uses the provided update operation to construct terms of sort $PFun$, then one is guaranteed that one only obtains well-formed terms. Moreover, any subterm of a well-formed term is also well-formed.

Exercise 22. *Prove Proposition 9.*

We next show that well-formed terms in T^{wf}_{PFun} and partial finite-domain functions in $[A \rightharpoonup B]^{finite}$ are equivalent. We show it by defining isomorphic mappings between the two:

Definition 12. *Let us consider two functions*

- $\varphi \ : \ T^{wf}_{PFun} \to [A \rightharpoonup B]^{finite}$ *and*
- $\psi \ : \ [A \rightharpoonup B]^{finite} \to T^{wf}_{PFun}$

defined as follows:

- *for any* $t \in T^{wf}_{PFun}$, $a \in A$ *and* $b \in B$, $\varphi(t)$ *is defined in* a *and* $\varphi(t)(a) = b$ *if and only if* $\textsc{Partial-Function}(A,B) \vdash t = (t', a \mapsto b)$ *for some* $t' \in T^{wf}_{PFun}$, *and*
- *for any* $f \ : \ A \rightharpoonup B$ *of finite domain* $\{a_1, a_2, \ldots, a_n\}$, $\psi(f)$ *is defined as the term* $(a_1 \mapsto b_1, a_2 \mapsto b_2, \ldots, a_n \mapsto b_n)$, *where* $b_k = f(a_k)$ *for each* $k \in \{1, 2, \ldots, n\}$.

As expected, the two mappings above are well-defined, are inverse to each other, and commute with the state lookup and update operations:

Proposition 10. *With the notation in Definition 12, the mappings φ and ψ are well-defined modulo the equations in* Partial-Function(A,B)*, that is:*

> *(φ well-defined) If $t_1, t_2 \in T^{wf}_{PFun}$ such that* Partial-Function(A,B) $\vdash t_1 = t_2$*, then $\varphi(t_1) = \varphi(t_2)$;*
>
> *(ψ well-defined) If $f : A \rightharpoonup B$ is a partial function of finite domain $\{a_1, a_2, \ldots, a_n\}$ with $f(a_k) = b_k$ for each $k \in \{1, 2, \ldots, n\}$ and if $\pi : \{1, 2, \ldots, n\} \to \{1, 2, \ldots, n\}$ is a permutation of the numbers $\{1, 2, \ldots, n\}$, then* Partial-Function(A,B) $\vdash (a_1 \mapsto b_1, a_2 \mapsto b_2, \ldots, a_n \mapsto b_n) = (a_{\pi(1)} \mapsto b_{\pi(1)}, a_{\pi(2)} \mapsto b_{\pi(2)}, \ldots, a_{\pi(n)} \mapsto b_{\pi(n)})$.

Moreover, the following hold for any $f \in [A \rightharpoonup B]^{finite}$ and for any $t \in T^{wf}_{PFun}$:

1. *$\varphi(\psi(f)) = f$;*
2. Partial-Function(A,B) $\vdash \psi(\varphi(t)) = t$;
3. *For any $a \in A$ and any $b \in B$, the following hold:*
   - *$f(a) = b$ iff* Partial-Function(A,B) $\vdash \psi(f)(a) = b$*, and*
   - Partial-Function(A,B) $\vdash t(a) = b$ *iff $\varphi(t)(a) = b$;*
4. *For any $a \in A$ and any $b \in B$, the following hold:*
   - Partial-Function(A,B) $\vdash \psi(f[b/a]) = \psi(f)[b/a]$;
   - $\varphi(t[b/a]) = \varphi(t)[b/a]$.

Exercise 23. *Prove Proposition 10.*

---

the following material comes from state definitions and may not be needed anymore

---

We can now naturally define the

Definition 13. *A state sigma is* ***equivalent*** *to a finite partial function $\sigma : Var \rightharpoonup Int$, written sigma $\simeq \sigma$, if sigma is well-formed and for any $x \in Var$ and $i \in Int$, $\sigma(x) = i$ iff* State $\vdash$ *sigma*$[x] = i$.

In other words, *sigma* $\simeq \sigma$ if and only if *sigma*, together with equational reasoning using the definition STATE, behaves like σ.

The proposition below states that, for all practical reasons, our equational definition of the state is completely equivalent to the one defining states as finite partial functions.

Proposition 11. *Let $x, x' \in Var$, $i, i' \in Int$, sigma, sigma' $\in State$ and finite partial functions $\sigma, \sigma' : Var \rightharpoonup Int$. Then the following hold:*

1. *empty $\simeq \emptyset$ where $\emptyset$ is the function undefined everywhere.*
2. *(sigma , $x \mapsto i$) $\simeq \sigma$ implies that sigma $\simeq \sigma \backslash \{x\}$, where $\sigma \backslash \{x\}$ is defined as σ restricted to $Dom(\sigma) \backslash \{x\}$.*
3. *If sigma $\simeq \sigma$ then also sigma$[x \leftarrow i] \simeq \sigma[x \leftarrow i]$.*

Proof. 1. Trivial, since there is no $x \in Var$ and $i \in Int$ such that State $\vdash$ *empty*$[x] = i$.

2. Suppose that $(sigma\ ,\ x \mapsto i) \simeq \sigma$. Let σ' be such that $sigma \simeq \sigma'$. We prove that $Dom(\sigma') = Dom(\sigma) \setminus \{x\}$ and $\sigma'(x) = \sigma(x)$ for any $x' \in Dom(\sigma')$. Consider an arbitrary $x' \in Var$. If $x' = x$ then there is no $i' \in Int$ such that $\textsc{State} \vdash sigma[x] = i'$, since otherwise we would have $\textsc{State} \vdash sigma = (sigma'\ ,\ x \mapsto i')$ for some state *sigma'*, which contradicts the well-formedness of $(sigma\ ,\ x \mapsto i)$; therefore, σ' is not defined on x. If $x' \neq x$, then $\textsc{State} \vdash sigma[x'] = (sigma\ ,\ x \mapsto i)[x']$, therefore σ' is defined on x' iff σ is defined on x', and, if so, then $\sigma'(x') = \sigma(x')$.

3. Suppose $sigma \simeq \sigma$. We distinguish two cases—if σ is defined on x or if σ is not defined on x. Let us first consider the case where σ is defined on x. Let us say that $\sigma(x) = i'$ for some $i' \in Int$. Then it must be the case that $\textsc{State} \vdash sigma[x] = i'$, which can only happen if $\textsc{State} \vdash sigma = (sigma'\ ,\ x \mapsto i')$ for some $sigma' \in State$, so $\textsc{State} \vdash sigma[x \leftarrow i] = (sigma'\ ,\ x \mapsto i)$. We next show that $sigma[x \leftarrow i] \simeq \sigma[x \leftarrow i]$. Let x' be an arbitrary variable in *Var*; there are two cases to analyze, when $x' = x$ and when $x' \neq x$. If $x' = x$ then

$$\textsc{State} \vdash (sigma[x \leftarrow i])[x'] = (sigma'\ ,\ x \mapsto i)[x'] = i$$

and note also that $\sigma[x \leftarrow i][x'] = i$. If $x' \neq x$ then

$$\begin{array}{rcl} \textsc{State} & \vdash & (sigma[x \leftarrow i])[x'] \\ & = & (sigma', x \mapsto i)[x'] \\ & = & sigma'[x'] \\ & = & (sigma', x \mapsto i')[x'] \\ & = & sigma[x'] \end{array}$$

and note also that, since $sigma \simeq \sigma$, we have $\textsc{State} \vdash sigma[x'] = j$ for some $j \in Int$ iff $\sigma[x'] = j$, which, since $x' \neq x$, happens iff $\sigma[x \leftarrow i][x'] = j$. We have therefore shown that $sigma[x \leftarrow i] \simeq \sigma[x \leftarrow i]$ whenever σ is defined in x.

If σ is not defined in x, it means that there is no $j \in Int$ such that $\textsc{State} \vdash sigma[x] = j$, so there is no $j \in Int$ and $sigma' \in State$ such that $\textsc{State} \vdash sigma = (sigma', x \mapsto j)$. If $\textsc{State} \vdash sigma = empty$ then we are done, since $\textsc{State} \vdash (x \mapsto i)[x'] = i'$ iff $x = x'$ and $i = i'$. If it is not the case that $\textsc{State} \vdash sigma = empty$, it must be that $\textsc{State} \vdash sigma = (x_1 \mapsto i_1, \ldots, x_n \mapsto i_n)$ with $x_k \neq x$ for all $k \in \{1, \ldots, n\}$. This leads to $\textsc{State} \vdash sigma[x \leftarrow i] = \cdots = (x_1 \mapsto i_1, \ldots, x_i \mapsto i_k)[x \leftarrow i](x_{k+1} \mapsto i_{k+1}, \ldots, x_n \mapsto i_n) = \cdots = (empty[x \leftarrow i], sigma) = (x \mapsto i, sigma) = (sigma, x \mapsto i)$. Thus $\textsc{State} \vdash sigma[x \leftarrow i][x'] = i'$ iff either $x' \neq x$ and $\textsc{State} \vdash sigma[x'] = i'$ or $x' = x$ and $i' = i$ iff either $x' \neq x$ and $\sigma[x'] = i'$ or $x' = x$ and $i' = i$ iff $\sigma[x \leftarrow i][x'] = i'$.

□

Because of the proposition above, we take the liberty to make no distinction between states represented as finite partial functions and states represented as equationally defined finite sets of bindings as above, and, for simplicity, we use the symbols σ, σ', σ_1, etc., to range over states in either of the two representations.

---

the above material comes from state definitions and may not be needed anymore

---

sort:

Stream

operations:

$$
\begin{array}{rcl}
_ : _ & : & Int \times Stream \to Stream \\
head & : & Stream \to Int \\
tail & : & Stream \to Stream \\
zeros & : & \to Stream \\
zip & : & Stream \times Stream \to Stream \\
add & : & Stream \to Stream \\
fibonacci & : & \to Stream
\end{array}
$$

equations:

$$
\begin{array}{l}
head(X : S) = X \\
tail(X : S) = S \\
zeros = 0 : zeros \\
zip(X : S_1, S_2) = X : zip(S_2, S_1) \\
add(X_1 : X_2 : S) = (X_1 +_{Int} X_2) : add(S) \\
fibonacci = 0 : 1 : add(zip(fibonacci, tail(fibonacci)))
\end{array}
$$

Figure 2.8: Streams of integers defined as an algebraic datatype. The variables S, S_1, S_2 have sort *Stream* and the variables X, X_1, X_2 have sort *Int*.

Streams

Figure 2.8 shows an example of a data-structure whose elements are infinite sequences, called *streams*, together with several particular streams and operations on them. Here we prefer to be more specific than in the previous examples and work with streams of integers. We assume the integers and operations on them already defined; specifically, we assume *Int* to be their sort and operations on them indexed with *Int* to distinguish them from other homonymous operations, e.g., $+_{Int}$, etc. The operation $_ : _$ adds a given integer to the beginning of a given stream, and the dual operations *head* and *tail* extract the head (integer) and the tail (stream) from a stream. The stream *zeros* contains only 0 elements. The stream operation *zip* merges two streams by interleaving their elements, and *add* generates a new stream by adding any two consecutive elements of a given stream. The stream *fibonacci* consists of the Fibonacci sequence (see Exercise 25).

It is interesting to note that the equational specification of streams in Figure 2.8 is one where its initial algebra semantics is likely *not* the model that we want. Indeed, the initial algebra here would consists of infinite classes of finite terms, where any two terms in any class are provably equal, for example $\{zeros,\ 0 : zeros,\ 0 : 0 : zeros, \ldots\}$. While this is a valid and interesting model of streams, it is likely not what one has in mind when one thinks of streams as infinite sequences. Nevertheless, the intended stream model is among the models/algebras of this equational specification, so any equational deduction or reduction that we perform, with or without strategies, is sound (see Exercise 26).

2.4.7 Notes

Equational encodings of general computation into equational deduction are well-known; for example, [7, 1] show such encodings, where the resulting equational specifications, if regarded as term rewrite systems (TRSs), are confluent and terminate whenever the original computation terminates. Our goal in this section is to discuss equational encodings of (Turing machine) computation. These encodings will be used later in the paper to show the Π^0_2-hardness of the equational satisfaction problem in the initial algebra. While we could have used existing encodings of Turing machines as TRSs, however, we found them more complex and intricate for our purpose in this paper than needed. Consequently (and also for the sake of self-containment), we recall the more recent (simple) encoding and corresponding proofs from [65]. Since the subsequent encoding is general purpose rather than specific to our Π^0_2-hardness result, the content of this section may have a more pedagogical than technical nature. For example, the references to TRSs are technically only needed to prove the equational encoding correct, so they could have been removed from the main text and added only in the proofs, but we find them pedagogically interesting and potentially useful for other purposes. The equational encodings that follow can be faithfully used as TRS Turing-complete computational engines, because each rewrite step corresponds to precisely one computation step in the Turing machine; in other words, there are no artificial rewrite steps.

2.4.8 Exercises

Exercise 24. *Eliminate the two equations in Figure 2.3 as discussed right after Theorem 8, and prove a result similar to Theorem 8 for the new representation.*

Exercise 25. *Show that the fibonacci stream defined in Figure 2.8 indeed defines the sequence of Fibonacci numbers. This exercise has two parts: first formally state what to prove, and second prove it.*

Exercise 26. *Consider the equational specification of streams in Figure 2.8. Define the intended model/algebra of streams over integer numbers with constant streams and functions on streams corresponding to the various operations in this specification. Then show that this model indeed satisfies all the equations in Figure 2.8. Describe also its default initial model and compare it with the intended model. Are they isomorphic?*

2.5 Rewrite logic

define $\rightarrow^1$

define or explain RL's concurrent rewriting

discuss how RL, by its AC rewriting and non-sharing approach, forces interleaving in situations where one would not want interleaving: $a* \rightarrow b*$*, etc.*

we use lots of Bool conditions in our subsequent semantics. explain how these relate to the RL approach where one only uses equations or rules in conditions.

cite multi-sorted signatures, instead of redefining everywhere. add them in the first, background section, before we even talk about term rewriting.

move **List{***Sort***}***, etc., in the appropriate section under eq logic, and cite*

Talk about confluence (for example, Theorem 23 needs it). Talk also about confluence criteria, such as orthogonality; needed for example to show the confluence of combinatory logic.

2.5.1 Equations versus Rewrite Rules

In contrast to term rewriting, which is just a method of computation, rewrite logic is a *computational logic* proposed by Meseguer [42] as a unified logic for (true) concurrency, which builds upon equational logic by extending it with rewrite rules. In equational logic, a number of *sorts* (types) and *equations* are defined, specifying which terms are equal. Equal terms are members of the same equivalence class. In other words, equal terms are regarded as being identical; in particular, equations can be soundly applied either from left-to-right or from right-to-left when reasoning about equational or rewrite logic theories. Rewrite logic adds *rules* to equational logic, thought of as irreversible transitions: a rewrite theory is an equational theory extended with rewrite rules, where equations can be applied in any direction, while rules can only be applied from left-to-right. In other words, a *rewrite logic theory* consists of:

- *Syntax*, consisting of sorts (syntactic categories) and operators on them, that can be used to define well-formed uninterpreted terms; as already mentioned, we prefer to use equivalent CFGs to define syntax in this paper;
- *Equations*, defining structural identities on terms: equal terms can be substituted for each other unrestricted anywhere. Intuitively, equal terms are *the same thing represented differently*, the same way π, 3.1459 . . ., or the circumference of a circle of diameter 1 are the same thing; and

- *Rewrite rules*, defining *irreversible transitions* between terms.

Here we define languages and language features as rewrite logic theories: the syntax of the rewrite logic theory captures the syntax of the language possibly extended with auxiliary operators needed for semantics, equations capture structural rearrangements or equivalences of the program configuration of computation structures and thus carry no computational meaning, and rules capture the intended computational steps and can apply in parallel (provided they don't overlap).

Both equations and rules can be parametric in rewrite logic, in the sense that they can contain variables that act as placeholders for subterms that *match* them. To avoid defining the details of rewrite logic, including substitutions and matching, by a *parametric* equation or rule we mean a recursively enumerable set of equations or rules, one per each parameter instance that satisfies the side conditions. In other words, equations and rules in this paper are regarded as *schematas* (when executing them on rewrite engines, one may need to capture side conditions using conditions in rules and/or equations). Unlike in reduction semantics with evaluation contexts [25], there are no context restrictions on applications of equations and/or rules (extensions of rewrite logic have been proposed recently allowing variants of context-sensitivity, but we are not going to use those). This allows for rewrite logic to serve as a very simple and elegant foundation for concurrency, because rules and equations can indeed be applied concurrently. Rewrite logic admits a *complete proof system* and an *initial model semantics* [42] that makes inductive proofs rigorous and valid. The intuition for why rewrite logic is a unified logic for concurrency comes from two observations: (1) the rewriting process is inherently parallel, in the sense that non-overlapping parts of a term can be rewritten concurrently, and thus fits well with current trends in concurrent system architecture; and (2) as shown in [42, 36], many other models of concurrent computation can be represented within rewrite logic, capturing faithfully the intended true or interleaved concurrency of each model.

Rewrite logic is connected to term rewriting in that the latter can be used to *execute* theories in the former. Indeed, most of the equations $l = r$ can be transformed into term rewriting rules $l \to r$, thus providing a means to taking a rewrite logic theory, together with an initial term, and *executing* it using the underlying TRS. Some of the TRS steps therefore correspond to equations, so can be viewed as *structural transformations* rather than *computational steps*. Any of the existing rewrite engines can be used for this purpose. It is important, though, not to confuse orientation of equations for executability purposes with rewrite rules in rewrite logic! As mentioned above, the actual semantics of a rewrite logic theory is that rewrite rules can apply concurrently on equivalence classes of terms obtained *modulo* all the equations! Not all equations can be oriented into rewrite rules for execution purposes; for example, an equation stating commutativity (C) of a binary operator cannot be regarded as a rewrite rule, because it would lead to non-termination; similarly, equations for associativity (A) or even ones defining identities, or units, of binary operators (I) may not always be desired to be regarded as (irreversible) rewrite rules. For that reason, some rewrite engines provide support for rewriting modulo special equations such as A, C, I.

An associative binary operator on a sort S can also be regarded as a list construct for S, an AI one as a list with empty-list construct, while an ACI one as a multiset construct; to obtain a set, one also needs to add an explicit idempotency equation in addition to the ACI equations. Since lists and (multi-)sets admit straightforward equational definitions in rewrite logic, from here on in this paper, we assume lists and (multi-)sets over any sort whenever needed; for a sort *Sort*, **List{***Sort***}** denotes comma-separated lists of terms of sort *Sort*, and **Set{***Sort***}** denotes white space separated (multi-)sets of terms of sort *Sort*. For all these structures, we use "·" as unit (nil, empty, etc.). If one wants a different list or set separator, then one writes it as a subscript, while if one wants a different unit then one writes it as a superscript. For clarity, we take the liberty to occasionally mention as subscripts and/or superscripts even the default constructs and units for lists and sets. For example, $\mathbf{List}^{\cdot}_{\curvearrowright}\{K\}$ stays for $\curvearrowright$-separated lists of terms of sort K. We also assume, whenever

needed, product sorts; these are written conventionally ($S_1 \times S_2$) and are also easy to define in rewrite logic.

We next show some examples of rewrite theories modulo equations (modulo A and AI, and modulo AC and ACI). First, let us consider two simple examples, showing also how intuitively rewrite logic and concurrency fit together. Suppose that one would like to define (comma-separated) sorted lists of integers. Then all one needs to do is to add the following bubble sort rule (to the AI equations for lists):

$$i, j \to j, i \quad \text{when } i > j$$

Here is an example of a nine-step rewrite logic derivation in the one-rule rewrite theory above (underlined subterms are redexes where the rule applies in the corresponding step):

$$\begin{array}{l} \underline{7,6}, \underline{5,4}, 3, \underline{2,1} \to 6, \underline{7,4}, \underline{5,3}, 1, 2 \to \underline{6,4}, \underline{7,3}, 5, 1, 2 \to 4, \underline{6,3}, 7, \underline{5,1}, 2 \to \underline{4,3}, 6, \underline{7,1}, \underline{5,2} \to \\ 3, 4, \underline{6,1}, \underline{7,2}, 5 \to 3, \underline{4,1}, \underline{6,2}, \underline{7,5} \to \underline{3,1}, \underline{4,2}, \underline{6,5}, 7 \to 1, \underline{3,2}, 4, \underline{5,6}, 7 \to 1, 2, 3, \underline{4,5}, \underline{6,7} \end{array}$$

Note that the rewrite rule is allowed to apply concurrently, but that it is not enforced to apply everywhere it can; for example, the rule was purposely not applied on the subterm "5, 1" in the third step above. Also, note that many applications of the associativity equation for $_,_$ were necessary in order to structurally change the term so that the rewrite rule could match and apply (sometimes multiple times). Note also that the applications of rules is non-deterministic and that one rule application choice may disable one or more other potential rule applications; for example, in the first step above one could have chosen the redex "6, 5", which would have disabled the redexes "7, 6" and "5, 4". This "disabling" is, of course, just an informal way of saying that equations (such as associativity) can be applied many different and sometimes exclusive ways in order for the rules to match. It is obvious that the resulting rewrite system terminates (modulo AI), because the number of misplaced numbers decreases at each application of the rule, concurrent or not. It is also obvious that the normal-form lists are sorted, so the one-rule rewrite system above gives a correct sorting algorithm. What may be less obvious is that, when executed on a parallel rewrite engine with plenty of cores, this trivial rewrite system gives a parallel algorithm that can sort in worst-case linear time.

Consider now a simple game. A group of children get a large bag (an ACI operator, giving a multiset) with black and white balls, as well as arbitrarily many additional balls. Each child can remove two balls from the bag, whenever they want and concurrently, but then immediately put back in the bag a black ball if both extracted balls had the same color, or a white ball if they had different colors. Other children may dynamically join the game. We can define this problem as a rewrite theory as follows (b ranges over *Ball*):

$$\begin{array}{l} \mathit{Ball} ::= \circ \mid \bullet \\ \mathit{Bag} ::= \mathbf{Set}\{\mathit{Ball}\} \\ \circ\ \bullet \to \circ \\ b\ b \to \bullet \quad \text{where } b \in \{\circ, \bullet\} \end{array}$$

It is obvious that as far as children keep playing the game it will eventually terminate, because at each extraction the number of balls decreases by at least one. The rules above can apply concurrently as far as there are at least two balls in the bag. What is not immediately obvious is that, despite the high degree of parallelism in this ACI rewrite system, it is actually confluent: the parity of the white balls does not change.

Let us now consider an extension of the simple calculator language above with ","-separated lists and with ";"-separated lists of lists (let us not worry about typing such programs for now) and with a permutation construct taking a number n and generating the list of permutation lists over the elements $\{1, 2, \ldots, n\}$ (here n, m range over natural numbers, p over ","-separated lists of natural numbers, and pl over ";"-separated lists

of ","-separated lists of natural numbers):

$$
\begin{array}{l}
Exp ::= \ldots \mid \mathbf{List}_{,}\{Exp\} \mid \mathbf{List}_{;}\{Exp\} \mid \mathsf{perm}(Exp) \mid \mathsf{insert}(Exp, Exp) \mid \mathsf{mapcons}(Exp, Exp)) \\
\mathsf{perm}(1) \to 1 \\
\mathsf{perm}(n) \to \mathsf{insert}(n, \mathsf{perm}(n-1)) \quad \text{when } n \geq 1 \\
\mathsf{insert}(n, (p; pl)) \to \mathsf{insert}(n, p); \mathsf{insert}(n, pl) \\
\mathsf{insert}(n, (m, p)) \to (n, m, p); \mathsf{mapcons}(m, \mathsf{insert}(n, p)) \\
\mathsf{insert}(n, \cdot) \to n \\
\mathsf{mapcons}(m, (p; pl)) \to \mathsf{mapcons}(m, p); \mathsf{mapcons}(m, pl) \\
\mathsf{mapcons}(m, p) \to (m, p)
\end{array}
$$

Note that the ","-separated lists above have an identity (i.e., it is an AI operator), "·", while the ";"-separated ones have no identity (only A). Also, note that the second occurrence of "1" in the first rule can and should be thought of as a degenerated ";"-separated list of ","-separated lists; indeed, thanks to matching modulo identity (I), the fourth rule also applies when p is $\cdot$ (since (m, p) matches 1 with $m = 1$ and $p = \cdot$). In our experiments, the rewrite theory above, when executed in Maude, outperformed best implementations of permutation generation in ML and Scheme: Maude took 61 seconds to calculate permutations of 10, while ML and Scheme took 83 and 92 seconds, respectively. None of these systems were able to calculate permutations of 11. These simplistic experiments should by no means be considered conclusive; all we are trying to say is that the pragmatic, semantics-reluctant language designer, can safely regard the semantic programming language and related definitions in this paper also as efficient implementations, in addition to their conciseness and mathematical rigor.

To further exemplify the use of ACI operators and corresponding matching, let us now consider an extension of the simple calculator language above with partial maps (from natural numbers to expressions) defined by their graphs (i.e., as sets of pairs (n, e)), as well as update and lookup constructs:

$$
\begin{array}{l}
Exp ::= \ldots \mid \mathbf{Set}\{Nat \times Exp\} \mid Exp[Nat] \mid Exp[Exp/Nat] \\
e\ e = e \\
([n, e]\ g)[n] \to e \\
([n, e]\ g)[e'/n] \to [n, e']\ g \\
g[e'/n] \to [n, e']\ g \quad \text{when } g \text{ contains no pair } [n, _]
\end{array}
$$

In the above, we assumed white-space-separated sets and the notation $[n, e]$ for pairs in the product sort $Nat \times Exp$. The equation assures that maps are indeed sets, not multi-sets (note that this set requirement is not really necessary in this case, because if one constructs maps only with the provided update interface then one can show that maps contain no duplicate pairs. The first rule shows a first example of ACI matching: note that n appears both in the map and as the lookup index, and that g can also be "·". In theory, many applications of associativity and commutativity equations may take place in the background to bring the pair $[n, e]$ on the first position in the set. In practice, ACI rewrite engines like Maude implement efficient data-structures and indexing to perform such matches in essentially logarithmic time (in the size of the set). We here did not bother to produce an error if the map was not defined in n; if g has no pair of the form $[n, _]$, then the term $g[n]$ is stuck and thus can be regarded as a "core dump" error message. On rewrite engines providing support for errors, one can define lookup as a potentially error generating operator; for example, in Maude one would define it to return an element of *kind* $[Exp]$ instead of sort Exp. The remaining two rules define the update operation; the first uses again ACI matching, while the second has a side condition. As mentioned above, in the theoretical developments of rewrite logic semantics and K in this paper we assume only unconditional rules and equations, though we adopt common conventions in logics and regard rules and equations as schematas, so they are allowed to have side conditions.

The simple calculator language above, with or without its extensions, is rather trivial and therefore cannot be used as an argument that rewrite logic can serve as a solid foundation for programming languages. Indeed, the main problem when defining non-trivial languages is that the applications of reductions needs to be somehow controlled, to ensure the correct evaluation flow in a program, and, moreover, applications of rules may need data that is not available locally, such as, e.g., values associated to locations in a shared, top-level store. Rewrite logic semantics (RLS), proposed by Meseguer and Rosu [45, 47], builds upon the strong belief, supported by extensive empirical evidence, that arbitrarily complex programming languages can, in fact, be easily defined as rewrite logic theories. By doing so, one gets essentially for free not only an interpreter and an initial model semantics for the defined language, but also a series of formal analysis tools obtained as instances of existing tools for rewriting logic.

As mentioned above, operationally speaking the major difference between conventional reduction semantics, with [25] or without [61] evaluation contexts, and rewrite logic semantics is that the former typically impose contextual restrictions on applications of reduction steps and the reduction steps happen one at a time, while the latter imposes no such restrictions. To avoid undesired applications of rewrite steps, one has to follow certain techniques and obey certain methodologies when using rewrite logic. In particular, as shown in [74], the more conventional language definitional styles (small-step and big-step SOS, reduction semantics with evaluation contexts, chemical abstract machines, continuation-based semantics, etc.) can be faithfully captured in rewrite logic by appropriate particular uses of its general reduction machinery. Consequently, one can define a language many different ways in rewrite logic, each with its advantages and disadvantages; indeed, the fact that one faithfully captures a small-step SOS definition as a rewrite logic theory does not mean that the resulting rewrite theory will give a true concurrency semantics to the original SOS definition: the adverb "faithfully", justified by theorems proved in [74], ensures that the corresponding rewrite theories have exactly the same strengths and limitations as the original definitions.

In this paper, we discuss the K rewrite logic technique, which was first introduced in the lecture notes of a programming language design course at the University of Illinois at Urbana-Champaign in Fall 2003 [64], as a means to define executable concurrent languages in rewrite logic using the Maude executable specification system [17]. K and its corresponding Maude tool support incrementally improved every year since then, as seen in the series of technical reports [66]. The latest of these reports also contains a more formal and detailed algebraic description of K. K is reminiscent of abstract state machines [31] and continuations [79, 63], and glosses over language-irrelevant rewrite logic details. Analyzing the various faithful translations of language definitional styles into rewrite logic discussed in detail in [74], as well as their advantages and disadvantages when regarded as rewrite logic theories, we confidently believe that K captures and reflects best the strengths of rewrite logic as a semantic framework for programming languages and related features.

---

a Boolean term b that appears as a condition is meant to be an equality b = `true`

---

from I&C

Rewrite logic [42] is a computational logic that can be efficiently implemented and that has good properties as a general and flexible *logical and semantic framework*, in which a wide range of logics and models of computation can be faithfully represented [35]. In particular, for programming language semantics it provides the RLS framework, of which we emphasize the operational semantics aspects in this paper (for the mathematical aspects of RLS see [46, 48]).

Two key points to explain are: (i) how rewrite logic combines equational logic and traditional term rewriting; and (ii) what the intuitive meaning of a rewrite theory is all about. A *rewrite theory* is a triple $\mathcal{R} = (\Sigma, E, R)$ with Σ a signature of function symbols, E a set of (possibly conditional) Σ-equations, and R a set of Σ-rewrite rules which in general may be conditional, with conditions involving both equations and rewrites. That is, a rule in R can have the general form

$$(\forall X)\ t \longrightarrow t' \ \text{ if } \ (\bigwedge_i u_i = u'_i) \wedge (\bigwedge_j w_j \longrightarrow w'_j)$$

Alternatively, such a conditional rule could be displayed with an inference-rule-like notation as

$$\frac{(\bigwedge_i u_i = u'_i) \wedge (\bigwedge_j w_j \longrightarrow w'_j)}{t \longrightarrow t'}$$

Therefore, the logic's atomic sentences are of two kinds: equations, and rewrite rules. Equational theories and traditional term rewriting systems then appear as special cases. An equational theory (Σ, E) can be faithfully represented as the rewrite theory $(\Sigma, E, \emptyset)$; and a term rewriting system (Σ, R) can likewise be faithfully represented as the rewrite theory $(\Sigma, \emptyset, R)$.

Of course, if the equations of an equational theory (Σ, E) are *confluent*, there is another useful representation, namely, as the rewrite theory $(\Sigma, \emptyset, \overrightarrow{E})$, where $\overrightarrow{E}$ are the rewrite rules obtained by orienting the equations E as rules from left to right. This representation is at the basis of much work in term rewriting, but by implicitly suggesting that rewrite rules are just an efficient technique for equational reasoning it can blind us to the fact that rewrite rules can have a much more general *non-equational* semantics. This is the whole *raison d'être* of rewrite logic. In rewrite logic a rewrite theory $\mathcal{R} = (\Sigma, E, R)$ axiomatizes a *concurrent system*, whose states are elements of the algebraic data type axiomatized by (Σ, E), that is, they are E-equivalence classes of ground Σ-terms, and whose *atomic transitions* are specified by the rules R. The inference system of rewrite logic described below then allows us to derive as *proofs* all the possible *concurrent computations* of the system axiomatized by $\mathcal{R}$, that is, concurrent computation and rewrite logic deduction *coincide*.

2.5.2 Proof System and Soundness

The inference rules below assume a typed setting, in which (Σ, E) is a *membership equational theory* [FIXTHIS] having sorts (denoted s, s', s'', etc.), subsort inclusions, and kinds (denoted k, k', k'', etc.), which gather together connected components of sorts. Kinds allow error terms like $3/0$, which has a kind but no sort. Similar inference rules can be given for untyped or simply typed (many-sorted) versions of the logic. Given $\mathcal{R} = (\Sigma, E, R)$, the sentences that $\mathcal{R}$ proves are universally quantified rewrites of the form $(\forall X)\ t \longrightarrow t'$, with $t, t' \in T_\Sigma(X)_k$, for some kind k, which are obtained by finite application of the following *rules of deduction*:

- **Reflexivity**. For each $t \in T_\Sigma(X)$, $\quad \dfrac{}{(\forall X)\ t \longrightarrow t}$

- **Equality**. $\quad \dfrac{(\forall X)\ u \longrightarrow v \quad E \vdash (\forall X) u = u' \quad E \vdash (\forall X) v = v'}{(\forall X)\ u' \longrightarrow v'}$

- **Congruence**. For each $f : s_1 \ldots s_n \longrightarrow s$ in Σ, with $t_i \in T_\Sigma(X)_{s_i}$, $1 \leq i \leq n$, and with $t'_{j_l} \in T_\Sigma(X)_{s_{j_l}}$, $1 \leq l \leq m$,

$$\frac{(\forall X)\ t_{j_1} \longrightarrow t'_{j_1} \quad \ldots \quad (\forall X)\ t_{j_m} \longrightarrow t'_{j_m}}{(\forall X)\ f(t_1, \ldots, t_{j_1}, \ldots, t_{j_m}, \ldots, t_n) \longrightarrow f(t_1, \ldots, t'_{j_1}, \ldots, t'_{j_m}, \ldots, t_n)}$$

Figure 2.9: Visual representation of rewrite logic deduction.

- **Replacement**. For each $\theta : X \longrightarrow T_\Sigma(Y)$ and for each rule in R of the form

$$(\forall X)\ t \longrightarrow t' \ \text{ if } \ (\bigwedge_i u_i = u'_i) \wedge (\bigwedge_j w_j \longrightarrow w'_j),$$

$$\frac{(\bigwedge_x (\forall Y)\ \theta(x) \longrightarrow \theta'(x)) \wedge (\bigwedge_i (\forall Y)\ \theta(u_i) = \theta(u'_i)) \wedge (\bigwedge_j (\forall Y)\ \theta(w_j) \longrightarrow \theta(w'_j))}{(\forall Y)\ \theta(t) \longrightarrow \theta'(t')}$$

  where θ' is the new substitution obtained from the original substitution θ by some possibly complex rewriting of each $\theta(x)$ to some $\theta'(x)$ for each $x \in X$.

- **Transitivity**

$$\frac{(\forall X)\ t_1 \longrightarrow t_2 \qquad (\forall X)\ t_2 \longrightarrow t_3}{(\forall X)\ t_1 \longrightarrow t_3}$$

We can visualize the above inference rules as in Figure 2.9.

The notation $\mathcal{R} \vdash t \longrightarrow t'$ states that the sequent $t \longrightarrow t'$ is *provable* in the theory $\mathcal{R}$ using the above inference rules. Intuitively, we should think of the inference rules as different ways of *constructing* all the (finitary) *concurrent computations* of the concurrent system specified by $\mathcal{R}$. The **Reflexivity** rule says that for any state t there is an *idle transition* in which nothing changes. The **Equality** rule specifies that the states are in fact equivalence classes modulo the equations E. The **Congruence** rule is a very general form of "sideways parallelism," so that each operator f can be seen as a *parallel state constructor*, allowing its arguments to evolve in parallel. The **Replacement** rule supports a different form of parallelism, which could be called "parallelism under one's feet," since besides rewriting an instance of a rule's left-hand-side to the corresponding right-hand-side instance, the state fragments in the substitution of the rule's variables can also be rewritten. Finally, the **Transitivity** rule allows us to build longer concurrent computations by composing them sequentially.

A somewhat more general version of rewrite logic [FIXTHIS]allows rewrite theories of the form $\mathcal{R} = (\Sigma, E \cup A, R, \phi)$, where the additional component ϕ is a function assigning to each function symbol $f \in \Sigma$ with n arguments a subset $\phi(f) \subseteq \{1, \ldots, n\}$ of those argument positions that are *frozen*, that is, positions under which rewriting is forbidden. The above inference rules can then be slightly generalized. Specifically, the **Congruence** rule is restricted to non-frozen positions $\{j_1, \ldots, j_m\}$, and the substitution θ' in the **Replacement** rule should only differ from θ for variables x in non-frozen positions. The generalized form $\mathcal{R} = (\Sigma, E \cup A, R, \phi)$, makes possible a more expressive control of the possibility of rewriting under contexts already supported by the **Congruence** rule; that is, it endows rewrite theories with flexible context-sensitive rewriting capabilities.[1]

Note that, in general, a proof $\mathcal{R} \vdash t \longrightarrow t'$ does not represent an *atomic* step, but can represent a complex concurrent computation. In some of the mathematical proofs that we will give to relate different operational semantics definitions, it will be easier to work with a "one-step" rewrite relation $\rightarrow^1$, defined on ground terms. This relation is just the special case in which: (i) **Transitivity** is excluded; (ii) $m = 1$ in the **Congruence** rule (only one rewrite below); and (iii) **Replacement** is restricted, so that no rewriting of the substitution θ to θ' is allowed; and (iv) there is exactly *one* application of **Replacement**. The relation $\rightarrow^{\leq 1}$ is defined by allowing either one or no applications of **Replacement** in the last condition. Similarly, one can define relations $\rightarrow^n$

[1] We will not consider this general version. The interested reader is referred to [FIXTHIS]

(or $\rightarrow^{\leq n}$) by controlling the number of applications of the **Transitivity** rule. However, it should be noted that rewrite logic *does not* have a builtin one-step rewrite relation, that being the reason for which we need a methodology to encode one-step-based formalisms such as SOS semantics. The one-step relation we define above is only at the deduction level and is introduced solely to help our proofs.

The whole point of RLS is then to define the semantics of a programming language $\mathcal{L}$ as a rewrite theory $\mathcal{R}_{\mathcal{L}}$. RLS uses the fact that rewrite logic deduction is performed *modulo* the equations in $\mathcal{R}_{\mathcal{L}}$ to faithfully capture the desired granularity of a language's computations. This is achieved by making rewriting rules all intended computational steps, while using equations for convenient equivalent structural transformations of the state, or auxiliary "infrastructure" computations, which should not be regarded as computation steps. Note that this does not preclude performing also equational simplification with equations. That is, the set E of equations in a rewrite theory can often be fruitfully decomposed as a disjoint union $E = E_0 \cup A$, where A is a set of *structural axioms*, such as associativity, commutativity and identity of some function symbols, and E_0 is a set of equations that are confluent and terminating *modulo* the axioms A. A rewrite engine supporting rewriting modulo A will then execute both the equations E_0 and the rules R modulo A by rewriting. Under a condition called *coherence* [FIXTHIS] , this form of execution then provides a complete inference system for the given rewrite theory (Σ, E, R). However, both conceptually and operationally, the execution of rules R and equations E_0 must be separated. Conceptually, what we are rewriting with R are E-equivalence classes, so that the E_0-steps become *invisible*. Operationally, the execution of rules R and equations E_0 must be kept separate for soundness reasons. This is particularly apparent in the case of executing *conditional* equations and rules: for a conditional equation it would be *unsound* to use rules in R to evaluate its condition; and for a conditional rule it would likewise be unsound to use rules in R to evaluate the *equational* part of its condition.

There are many systems that either specifically implement term rewriting efficiently, so-called as *rewrite engines*, or support term rewriting as part of a more complex functionality. Any of these systems can be used as an underlying platform for execution and analysis of programming languages defined using the techniques proposed in this paper. Without attempting to be exhaustive, we here only mention (alphabetically) some engines that we are more familiar with, noting that many functional languages and theorem provers provide support for term rewriting as well: ASF+SDF [81], CafeOBJ [21], Elan [10, 11], Maude [18, 17], OBJ [28], and Stratego [85]. Some of these engines can achieve remarkable speeds on today's machines, in the order of tens of millions of rewrite steps per second.

---

lazy evaluation to be added in the next section; it is needed in Section FIXTHIS

---

sorts:
$Cell$, $Tape$, $Configuration$
operations:
$0, 1 \ : \ \rightarrow Cell$
$zeros \ : \rightarrow Tape$
$_ : _ \ : \ Cell \times Tape \rightarrow Tape$
$q \ : \ Tape \times Tape \rightarrow Configuration$ — one such operation for each $q \in Q$
equations:
$zeros = 0 : zeros$
rules:
$q(L, b:R) \rightarrow q'(L, b':R)$ — one rule for each $q, q' \in Q$, $b, b' \in Cell$ with $(q', b') \in M(q, b)$
$q(L, b:R) \rightarrow q'(b:L, R)$ — one rule for each $q, q' \in Q$, $b \in Cell$ with $(q', \rightarrow) \in M(q, b)$
$q(B:L, b:R) \rightarrow q'(L, B:b:R)$ — one rule for each $q, q' \in Q$, $b \in Cell$ with $(q', \leftarrow) \in M(q, b)$

Figure 2.10: Lazy rewrite logic representation $\mathcal{R}_{\mathcal{M}}^{lazy}$ of Turing machine $\mathcal{M}$

2.5.3 Computation as Rewrite Logic Deduction

Building upon the equational representations of deterministic Turing machines in Section 2.4.3, here we show how we can associate rewrite theories to non-deterministic Turing machines so that there is a bijective correspondence between computational steps performed by the original Turing machine and rewrite steps in the corresponding rewrite theory. In non-deterministic Turing machines, the total transition function $M : (Q - \{q_h\}) \times B \rightarrow Q \times C$ generalizes to a total relation, or in other words to a function into the strict powerset of $Q \times C$, $M : (Q - \{q_h\}) \times B \rightarrow \mathcal{P}^+(Q \times C)$, that is, taking each non-halting state q and current cell bit contents b into a non-empty set $M(q, b)$ of non-deterministic (state,action) choices. For example, to turn the successor Turing machine in Figure 2.1 into one which non-deterministically chooses to add one more 1 to the given number *or not* when it reaches its end, all we have to do is to modify its transition function in state q_1 and cell contents 0 to return two possible continuations: $M(q_1, 0) = \{(q_2, 1), (q_2, \leftarrow)\}$. Like in Section 2.4.3, we give both lazy and unrestricted representations.

Lazy Rewrite Logic Representation

Figure 2.10 shows how a Turing machine $\mathcal{M}$ can be associated a computationally equivalent rewrite logic theory $\mathcal{R}_{\mathcal{M}}^{lazy}$. The only difference between this rewrite logic theory and the equational logic theory in Figure 2.2 is that the equations which were specific to the Turing machine have been turned into rewrite rules. The equation expanding the stream of zeros remains an equation. Since in rewrite logic only the rewrite rules count as transitions, and they apply modulo equations, the rewrite theory is in fact more faithful to the actual computational steps embodied in the Turing machine. The result below formalizes this by showing that there is a step-for-step equivalence between computations using $\mathcal{M}$ and rewrites using $\mathcal{R}_{\mathcal{M}}^{lazy}$:

Theorem 10. *The rewrite logic theory $\mathcal{R}_{\mathcal{M}}^{lazy}$ is confluent. Moreover, the Turing machine $\mathcal{M}$ and the rewrite theory $\mathcal{R}_{\mathcal{M}}^{lazy}$ are step-for-step equivalent, that is,*

$$(q, 0^\omega u\underline{b}v0^\omega) \rightarrow_{\mathcal{M}} (q', 0^\omega u'\underline{b'}v'0^\omega) \quad \textit{if and only if} \quad \mathcal{R}_{\mathcal{M}}^{lazy} \models q(\overleftarrow{u}, b : \overrightarrow{v}) \rightarrow^1 q'(\overleftarrow{u'}, b' : \overrightarrow{v'})$$

sorts:
$Cell$, $Tape$, $State$, $Configuration$
operations:
$0, 1 \ : \ \to Cell$
$zeros \ : \to Tape$
$_ : _ \ : \ Cell \times Tape \to Tape$
$_(_, _) \ : \ State \times Tape \times Tape \to Configuration$
$q \ : \to State$ — one such constant for each $q \in Q$
equations:
$S(zeros, R) = S(0{:}zeros, R)$
$S(L, zeros) = S(L, 0{:}zeros)$
rules:
$q(L, b{:}R) \to q'(L, b'{:}R)$ — one rule for each $q, q' \in Q$, $b, b' \in Cell$ with $(q', b') \in M(q, b)$
$q(L, b{:}R) \to q'(b{:}L, R)$ — one rule for each $q, q' \in Q$, $b \in Cell$ with $(q', \to) \in M(q, b)$
$q(B{:}L, b{:}R) \to q'(L, B{:}b{:}R)$ — one rule for each $q, q' \in Q$, $b \in Cell$ with $(q', \leftarrow) \in M(q, b)$

Figure 2.11: Unrestricted rewrite logic representation $\mathcal{R}_{\mathcal{M}}$ of Turing machine $\mathcal{M}$

for any finite sequences of bits $u, v, u', v' \in \{0, 1\}^*$, any bits $b, b' \in \{0, 1\}$, and any states $q, q' \in Q$, where if $u = b_1 b_2 \ldots b_{n-1} b_n$, then $\overleftarrow{u} = b_n : b_{n-1} : \cdots : b_2 : b_1 : zeros$ and $\overrightarrow{u} = b_1 : b_2 : \cdots : b_{n-1} : b_n : zeros$. Finally, the following are equivalent:

(1) The Turing machine $\mathcal{M}$ terminates on input $b_1 b_2 \ldots b_n$;

(2) $\mathcal{R}_{\mathcal{M}}^{lazy} \models q_s(zeros, b_1{:}b_2{:}\cdots{:}b_n{:}zeros) \to q_h(l, r)$ for some terms l, r of sort Tape; note though that $\mathcal{R}_{\mathcal{M}}^{lazy}$ does not terminate on term $q_s(zeros, b_1{:}b_2{:}\cdots{:}b_n{:}zeros)$ as an unrestricted rewrite system, since the equation $zeros = 0 : zeros$ (regarded as a rewrite rule) can apply forever, thus yielding infinite equational classes of configurations with no canonical forms, but $\mathcal{R}_{\mathcal{M}}^{lazy}$ terminates on $q_s(zeros, b_1{:}b_2{:}\cdots{:}b_n{:}zeros)$ if the stream construct operation $_{:}_ : Cell \times Tape \to Tape$ has a lazy rewriting strategy on its second argument;

Proof. ... □

Therefore, unlike the equational logic theory $\mathcal{E}_{\mathcal{M}}^{lazy}$ in Theorem 7, the rewrite logic theory $\mathcal{R}_{\mathcal{M}}^{lazy}$ faithfully captures, step-for-step, the computational granularity of $\mathcal{M}$. Recall that equational deduction does not count as computational, or rewrite steps in rewrite logic, which allows to apply the self-expanding equation of *zeros* silently in the background. Since there are no artificial rewrite steps, we can conclude that $\mathcal{R}_{\mathcal{M}}$ actually *is* precisely $\mathcal{M}$ and not an encoding of it. Theorem 10 thus showed not only that rewriting logic is Turing complete, but also that it faithfully captures the computational granularity of the represented Turing machines.

Unrestricted Rewrite Logic Representations

Figure 2.11 shows our unrestricted representation of Turing machines as rewrite logic theories, following the same idea as the equational representation in Section 2.4.3 (Figure 2.3). Let $\mathcal{R}_{\mathcal{M}}$ be the rewrite logic theory in Figure 2.11. Then the following result holds:

Theorem 11. *The rewrite logic theory $\mathcal{R}_{\mathcal{M}}$ is confluent. Moreover, the Turing machine $\mathcal{M}$ and the rewrite theory $\mathcal{R}_{\mathcal{M}}$ are step-for-step equivalent, that is,*

$$(q, 0^{\omega}u\underline{b}v0^{\omega}) \rightarrow_{\mathcal{M}} (q', 0^{\omega}u'\underline{b'}v'0^{\omega}) \quad \textit{if and only if} \quad \mathcal{R}_{\mathcal{M}} \models q(\overleftarrow{u}, b : \overrightarrow{v}) \rightarrow^{1} q'(\overleftarrow{u'}, b' : \overrightarrow{v'})$$

for any finite sequences of bits $u, v, u', v' \in \{0,1\}^*$, any bits $b, b' \in \{0,1\}$, and any states $q, q' \in Q$, where if $u = b_1 b_2 \ldots b_{n-1} b_n$, then $\overleftarrow{u} = b_n : b_{n-1} : \cdots : b_2 : b_1 : zeros$ and $\overrightarrow{u} = b_1 : b_2 : \cdots : b_{n-1} : b_n : zeros$. Finally, the following are equivalent:

(1) The Turing machine $\mathcal{M}$ terminates on input $b_1 b_2 \ldots b_n$;

(2) $\mathcal{R}_{\mathcal{M}}$ terminates on term $q_s(zeros, b_1 : b_2 : \cdots : b_n : zeros)$ as an unrestricted rewrite system and $\mathcal{R}_{\mathcal{M}} \models q_s(zeros, b_1 : b_2 : \cdots : b_n : zeros) \rightarrow q_h(l, r)$ for some terms l, r of sort Tape;

Proof. ... □

Like for the lazy representation of Turing machines in rewriting logic discussed above, the rewrite theory $\mathcal{R}_{\mathcal{M}}$ *is* the Turing machine $\mathcal{M}$, in that there is a step-for-step equivalence between computational steps in $\mathcal{M}$ and rewrite steps in $\mathcal{R}_{\mathcal{M}}$. Recall, again, that equations do not count as rewrite steps, their role being to structurally rearrange the term so that rewrite rules can apply; indeed, that is precisely the intended role of the two equations in Figure 2.11 (they reveal new blank cells on the tape whenever needed). Similarly to the equational case in Section 2.4.3, the two generic equations can be completely eliminated. However, this time we have to add more Turing-machine-specific rules instead. For example, if $(q', \leftarrow) \in M(q, b)$ then, in addition to the last rule in Figure 2.11, we also include the rule:

$$q(zeros, b : R) \rightarrow q'(zeros, 0 : b : R)$$

This way, one can expand *zeros* and apply the transition in *one* rewrite step, instead of one equational step and one rewrite step. Doing that systematically for all the transitions allows us to eliminate the need for equations entirely; the price to pay is, of course, that the number of rules increases.

2.5.4 Completeness

2.5.5 Rewrite Logic as a Universal Logical Formalism

include this:

In [35], it is suggested that one generic way to incorporate any proof system in rewrite logic is to translate each deduction rule of the form

$$\frac{Seq_1 \quad Seq_2 \quad \ldots \quad Seq_n}{Seq}$$

into a rewrite rule of the form

$$Seq \rightarrow Seq_1, Seq_2, \ldots, Seq_n$$

or into a rewrite rule of the form

$$Seq_1, Seq_2, \ldots, Seq_n \rightarrow Seq$$

2.5.6 Maude: A High Performance Rewrite Logic System

Maude (`http://maude.cs.uiuc.edu`) is a rewrite logic *executable specification language*, which builds upon a fast rewrite engine. Our main use of Maude in this book is as a platform to execute rewrite logic semantic definitions, following the various approaches in Chapter 3. Our goal here is to give a high-level overview of Maude, mentioning only its features needed in this book. We make no attempt to give a systematic presentation of Maude here, meant to replace its manual or other more comprehensive papers or books (some mentioned in Section 2.5.8). The features we need will be introduced on-the-fly, with enough explanations to make this book self-contained, but the reader interested in learning Maude in depth should consult its manual.

We will use Maude to specify programming language features, which, when put together via simple Maude module operations, lead to programming language semantic definitions. Since Maude is executable, *interpreters for programming languages designed this way will be obtained for free*, which will be very useful for understanding, refining and/or changing the languages. Moreover, *formal analysis tools* for the specified languages can also be obtained with little effort, such as exhaustive state-space searchers or model-checkers, simply by using the corresponding generic rewrite logic analysis tools already provided by Maude.

Devising *formal semantic executable models* of desired languages or tools *before these are implemented* is a crucial step towards a deep understanding of the language or tool. In simplistic terms, it is like devising a simulator for an expensive system before building the actual system. However, our simulators in this book will consist of exactly the *semantics*, or the *meaning*, of our desired systems, defined using a very rigorous, mathematical notation. In the obtained formal executable model of a programming language, executing a program will correspond to nothing but *logical inference* within the semantics of the language.

How to Execute Maude

After installing Maude on your platform and setting up the environment path variable, you should be able to type `maude` and immediately see a welcome screen followed by a cursor waiting for user input:

```
Maude>
```

Maude is interpreted, so you can just type your specifications and commands. However, a more practical way is to type everything in one file, say `pgm.maude`, and then include that file with the command

```
Maude> in pgm.maude
```

after starting Maude (the extension is optional), or, alternatively, start Maude with `pgm` as an argument: "`maude pgm`". In both cases, the contents of `pgm.maude` will be loaded and executed as if it was manually typed at the cursor. Use the `quit` command, or simply `q`, to quit Maude. Since Maude's initialization and termination are quite fast, many users end their `pgm.maude` file with a `q` command on a new line, so that Maude terminates as soon as the program is executed. Another useful command in files is `eof`, which tells Maude that the end-of-file is meant there and thus it does not process the code following the `eof` command. Instead, the control is given to the user, who can manually type commands, etc. You can correct/edit your Maude definition in `pgm.maude` and then load it again. However, keep it in mind that Maude maintains only one working session, in particular one module database, until you quit it. This can sometimes lead to unexpected errors for beginners, so if you are not sure about an error just quit and then restart Maude.

Modules

Maude specifications are introduced as *modules*. There are several kinds of modules, but for simplicity we only use general modules in this book, which have the syntax

```
mod <NAME> is
  <BODY>
endm
```

where <NAME> can be any identifier. The <BODY> of a module can include importation of other modules, sort and operation declarations, and a set of sentences. The sorts together with the operations form the *signature* of that module, and can be thought of as the interface of that module to other modules.

To lay the ground for introducing more Maude features, let us define Peano-style natural numbers with addition and multiplication. We define the addition first, in one separate module:

```
mod PEANO-NAT is
  sort Nat .
  op zero : -> Nat .
  op succ : Nat -> Nat .
  op plus : Nat Nat -> Nat .
  vars N M : Nat .
  eq plus(zero, M) = M .
  eq plus(succ(N), M) = succ(plus(N, M)) .
endm
```

Declarations and sentences are always terminated by periods, which should have white spaces before and after. Forgetting a terminal period or a white space before the period are two of the most common errors that Maude beginners make.

The signature of `PEANO-NAT` consists of one sort, `Nat`, and three operations, namely `zero`, `succ`, and `plus`. Sorts are declared with the keywords `sort` or `sorts`, and operations with `op` or `ops`.

The three operations have zero, one and two arguments, respectively, whose sorts are listed between the symbols `:` and `->`. Operations of zero arguments are also called *constants*, those of one argument are called *unary* and those of two *binary*. The result sort appears right after the symbol `->`.

We use `ops` when two or more operations of same arguments are declared together, to save space, and then we use white spaces to separate them:

```
  ops plus mult : Nat Nat -> Nat .
```

There are few special characters in Maude, and users are allowed to define almost any token or combination of tokens as operation names. If you use `op` in the above instead of `ops`, for example, then only one operation, called "`plus mult`", is declared.

The two equations in `PEANO-NAT` are properties, or constraints, that terms built with these operations must satisfy. Another way to look at equations is through the lenses of possible implementations of the specifications they define; in our case, any *correct* implementation of Peano natural numbers should satisfy the two equations. Equations are quantified universally with the variables they contain, and can be applied from *left-to-right* or from *right-to-left* in reasoning, which means that equational proofs may require exponential search, thus making them theoretically intractable. Maude provides limited support for equational reasoning.

`reduce`: Rewriting with Equations

When executing specifications, Maude regards all equations as *rewrite rules*, which means that they are applied only from left to right. Moreover, they are applied iteratively for as long as their left-hand-side terms match any subterm of the term to reduce. This way, any well-formed term can either be derived infinitely often, or be reduced to a *normal form*, which cannot be reduced anymore by applying equations as rewriting rules. Maude's command to reduce a term to its normal form using equations as rewrite rules is `reduce`, or simply `red`. Reduction will be made in the last defined module, which is `PEANO-NAT` in our case:

```
Maude> reduce plus(plus(succ(zero),succ(succ(zero))), succ(succ(succ(zero)))) .
rewrites: 6 in 0ms cpu (0ms real) (~ rewrites/second)
result Nat: succ(succ(succ(succ(succ(succ(zero))))))
```

Make sure commands are terminated with a period. Maude implements state of the art term rewriting algorithms, based on advanced indexing and pattern matching techniques. This way millions of rewrites per second can be performed, making `Maude` usable as a programming language in terms of performance.

Sometimes the results of reductions are repetitive and may be too large to read. To ameliorate this problem, Maude provides an operator attribute called `iter`, which allows to input and print repetitive terms more compactly. For example, if we replace the declaration of operation `succ` with

```
  op succ : Nat -> Nat  [iter] .
```

then Maude uses, e.g., `succ^3(zero)` as a shorthand for `succ(succ(succ(zero)))`. For example,

```
Maude> reduce plus(plus(succ(zero),succ^2(zero)), succ^3(zero)) .
result Nat: succ^6(zero)
```

Importation

Modules can be imported in several different ways. The difference between importation modes is subtle and semantical rather than operational, and it is not relevant in this book. Therefore, we only use the most general of them, `including`. For example, the following module extends `PEANO-NAT` with multiplication:

```
mod PEANO-NAT* is
  including PEANO-NAT .
  op mult : Nat Nat -> Nat .
  vars M N : Nat .
  eq mult(zero, M) = zero .
  eq mult(succ(N), M) = plus(mult(N, M), M) .
endm
```

It is safe to think of `including` as "copy and paste" the contents of the imported module into the importing module, with one exception: variable declarations are *not* imported, so they need to be redeclared.

We can now "execute programs" using features in both modules:

```
red mult(plus(succ(zero),succ(succ(zero))), succ(succ(succ(zero)))) .
```

The following is Maude's output:

```
rewrites: 18 in 0ms cpu (0ms real) (~ rewrites/second)
result Nat: succ^9(zero)
```

Even though this language is very simple and its syntax is ugly, it nevertheless shows a formal and executable definition of a language using equational logic and rewriting. Other languages or formal analyzers discussed in this book will be defined in a relatively similar manner, though, as expected, they will be more involved.

The Mixfix Notation and Parsing

The `plus` and `mult` operations defined above are meant to be written using the *prefix* notation in terms. Maude also supports the *mixfix* notation for operations (see Section 2.1.3), by allowing the user to write underscores in operation names as placeholders for their corresponding arguments.

```
op _+_ : Int Int -> Int .
op _! : Nat -> Nat .
op in_then_else_ : BoolExp Stmt Stmt -> Stmt .
op _?_:_ : BoolExp Exp Exp -> Exp .
```

Users can now also write terms taking advantage of the mixfix notation, for example `3 + 5`, in addition to the usual prefix notation, that is, `_+_(3,5)`.

Recall from Section 2.1.3 that, syntactically speaking, the mixfix notation has the same expressiveness as the context-free grammar notation. Therefore, the mixfix notation comes with the unavoidable *parsing* problem. For example, suppose that we replace the operations `plus` and `mult` in the modules above with their mixfix variants `_+_` and `_*_` (see also Exercise 29). Then the term `X + Y * Z`, with `X`, `Y`, `Z` arbitrary variables (or any terms) of sort `Nat`, admits two ambiguous parsings: `(X + Y) * Z` and `X + (Y * Z)`.

Maude provides a `parse` command, similar to `reduce` except that it only parses the given term:

```
Maude> parse X + Y .
Nat: X + Y
```

Maude generates a warning message whenever it detects more than one parsing of the given term:

```
Maude> parse X + Y * Z .
Warning: <standard input>, line 1: ambiguous term, two parses are:
X + (Y * Z)
-versus-
(X + Y) * Z
Arbitrarily taking the first as correct.
Nat: X + (Y * Z)
```

Similar warning messages are issued when ambiguous terms are detected in the specification (e.g., in equations). In general, we do not want to allow any parsing ambiguity in specifications or in terms to rewrite. One simple way to avoid ambiguities is to use parentheses to specify the desired grouping, for example:

```
Maude> parse X + (Y * Z) .
Nat: X + (Y * Z)
```

To reduce the number of parentheses, Maude allows us to assign precedences to mixfix operations declared in its modules, specifically as operator attributes in square brackets using the `prec` keyword. For example:

```
op _+_ : Nat Nat -> Nat [prec 33] .
op _*_ : Nat Nat -> Nat [prec 31] .
```

The lower the precedence the stronger the binding! As expected, now there is no parsing ambiguity anymore:

```
Maude> parse X + Y * Z .
Nat: X + Y * Z
```

To see how the term was parsed, set the "`print with parentheses`" flag on:

```
Maude> set print with parentheses on .
Maude> parse X + Y * Z .
Nat: (X + (Y * Z))
```

If displaying the parentheses is not sufficient, then disable the mixfix printing completely:

```
Maude> set print mixfix off .
Maude> parse X + Y * Z .
Nat: _+_(X, _*_(Y, Z))
```

Associativity, Commutativity and Identity Attributes

Some of the binary operations used in this book will be associative (A), commutative (C) or have an identity (I), or combinations of these. E.g., _+_ is associative, commutative and has 0 as identity. All these can be added as attributes to operations when declared:

```
op _+_ : Int Int -> Int [assoc comm id: 0 prec 33] .
op _*_ : Int Int -> Int [assoc comm id: 1 prec 31] .
```

Note that each of the A, C, and I attributes are logically equivalent to appropriate equations, such as

```
eq A + (B + C) = (A + B) + C .
eq A + B = B + A .  ---> attention: rewriting does not terminate!
eq A + 0 = A .
```

When applied as rewrite rules, each of the three equations above have limitations. The associativity equation forces all the parentheses to be grouped to the left, which may prevent some other rules from applying. The commutativity equation may lead to non-termination when applied as a rewrite rule. The identity equation would only be able to simplify expressions, but not to add a 0 to an expression, which may be useful in some situations (we will see such situations shortly, in the context of lists). Maude's builtin support for ACI attributes addresses all the problems above. Additionally, the `assoc` attribute of a mixfix operation is also taken into account by Maude's parser, which hereby eliminates the need for some useless parentheses:

```
Maude> parse X + Y + Z .
Nat: X + Y + Z
```

An immediate consequence of the builtin support for the `comm` attribute, which allows rewriting with commutative operations to terminate, is that normal forms will be reported now *modulo commutativity*:

```
Maude> red X + Y + X .
rewrites: 0 in 0ms cpu (0ms real) (~ rewrites/second)
result Nat: X + X + Y
```

As seen, Maude picked to display some equivalent (modulo AC) of the original term (extracted from how the current implementation of Maude stores this term internally). There were 0 rewrites applied in the reduction above, because the internal rearrangements of terms according to the ACI attribute annotations do not count as rule applications.

Matching Modulo Associativity, Commutativity, and Identity

Here we discuss Maude's support for *ACI matching*, which is arguably one of the most distinguished and complex Maude features, and nevertheless the reason and the most important use of the ACI attributes.

We discuss ACI matching by means of a series of examples, starting with lists, which occur in many programming languages. The following module defines lists of integers with a membership operation `_in_`, based on AI (associative and identity) matching:

```
mod INT-LIST is including INT .
  sort IntList .
  subsort Int < IntList .
  op nil : -> IntList .
  op __ : IntList IntList -> IntList [assoc id: nil] .
  op _in_ : Int IntList -> Bool .
  var I : Int .  vars L L' : IntList .
  eq I in L I L' = true .
  eq I in L = false [owise] .
endm
```

We start by including the builtin INT module, which declares a sort Int and provides arbitrary large integers as constants of sort Int, together with the usual operations on these. The builtin module BOOL, which similarly declares a sort Bool and common Boolean operations on it, is automatically included in all modules, so it needs not be included explicitly. To see a an existing module, builtin or not, use the command

```
Maude> show module <NAME> .
```

For example, "show module INT ." will display the INT module. In the INT-LIST module above, note the subsort declaration "Int < IntList", which says that integers are also lists of integers. This, together with the constant nil and the concatenation operation __, can generate any finite list of integers:

```
Maude> parse 1 2 3 4 5 .
IntList: 1 2 3 4 5
Maude> red 1 nil 2 nil 3 nil 4 nil 5 6 7 nil .
rewrites: 0 in 0ms cpu (0ms real) (~ rewrites/second)
result IntList: 1 2 3 4 5 6 7
```

Note how the reduce command above eliminated all the unnecessary nil constants from the list, in zero rewrite steps, for the same reason as above: the internal rearrangements according to the ACI attributes do not count as rewrite steps.

The two equations defining the membership operation make use of AI matching. The first equation says that if we can match the integer I anywhere inside the list, then we are done. Since the list constructor __ was declared associative and with identity nil, Maude is mathematically allowed to bind the variables L and L' of sort IntList to any lists of integers, including the empty one. Maude indeed does this through its efficient AI matching algorithm. Equations with attribute owise are applied only when other equations fail to apply. Therefore, we defined the semantics of the membership operation only by means of AI matching, without having to implement any explicit traversal of the list. Here are some examples testing the semantics above:

```
Maude> red 3 in 2 3 4 .
result Bool: true
Maude> red 3 in 3 4 5 .
result Bool: true
Maude> red 3 in 1 2 4 .
result Bool: false
```

To define sets of integers (see, e.g., Exercise 30), besides likely renaming the sort IntList into IntSet, we would also need to declare the concatenation operation __ commutative; moreover, thanks to Maude's commutative matching, we can also replace the first equation by "eq I in I L = true ."

We next discuss a Maude definition of (partial finite-domain) maps (see Section 2.4.6 and Figure 2.7 for the mathematical definition). We assume that the Source and Target sorts are defined in separate modules SOURCE and TARGET, respectively; one may need to change these in concrete applications. The associativity, commutativity and identity equations in Figure 2.7 are replaced by corresponding Maude operational attributes. Note that the second equation defining the update operation takes advantage of Maude's owise attribute (explained above), so it departs from the more mathematical definition in Figure 2.7:

```
mod MAP is including SOURCE + TARGET .
  sort Map .
  op _|->_ : Source Target -> Map  [prec 0] .
  op empty : -> Map .
  op _,_ : Map Map -> Map  [assoc comm id: empty] .
  op _(_) : Map Source -> Target  [prec 0] .       --- lookup
  op _[_/_] : Map Target Source -> Map [prec 0] .  --- update
```

```
  var M : Map .  var A : Source .  var B B' : Target .
  eq (M, A |-> B)(A) = B .
  eq (M, A |-> B')[B / A ] = (M, A |-> B) .
  eq M[B / A] = (M, A |-> B) [owise] .
endm
```

If module SOURCE defines constants a, b, c, d, . . . , of sort Source, and TARGET defines constants 1, 2, 3, 4, . . . , of sort Target, then the following reduce commands work as shown:

```
Maude> red empty[1 / a][2 / b][3 / c] .
result Map: a |-> 1,b |-> 2,c |-> 3
Maude> red empty[1 / a][2 / b][3 / c][4 / a] .
result Map: a |-> 4,b |-> 2,c |-> 3
Maude> red empty[1 / a][2 / b][3 / c][4 / a](a) .
result Target: 4
Maude> red empty[1 / a][2 / b][3 / c][4 / a](d) .
result Target: (a |-> 4,b |-> 2,c |-> 3)(d)
```

Note that the last reduction above only updated the map, but it got stuck on the lookup of d. That is because we only have one equation defining lookup, which works only when the looked up element is in the domain of the map. Getting stuck terms as above may be acceptable in many applications, but, however, we sometimes want to report specific errors in such situations. Maude has several advanced foundational mechanisms to deal with errors, but they are non-trivial and we do not need them in this book. Instead, we can simply modify the MAP definition above to include a special undefined "value" in the Target domain and then explicitly use this value in equations where we mean that an error occurred:

```
op undefined : -> Target .
eq M(A) = undefined  [owise] .
```

Now the last reduction command above yields undefined. A particularly useful approach to deal with undefinedness in the context of programming language semantics, where a semantics builts upon several mathematical domains in which the syntax is interpreted, is to define the constant undefined to have a generic sort, say Undefined, which is then subsorted to all sorts, including Source, Target, and Map. Then we can add the following to the module MAP to obtain partial finite-domain maps with support for undefinedness:

```
subsort Undefined < Map .
eq M(A) = undefined  [owise] .
eq A |-> undefined = empty .
```

The last equation above is an optimization, allowing us to "garbage collect" the useless bindings in maps once they are explicitly "undefined" in certain elements. For example,

```
Maude> red empty[1 / a][2 / b][3 / c][4 / a][undefined / a] .
result Map: b |-> 2,c |-> 3
```

Pretty Printing

In the MAP example above, the bindings and the comma separating them may be hard to read when the maps are large. We may therefore want to pretty-print the reduced terms. Maude provides the format attribute for this purpose. For example, if we replace the operation declarations of _|->_ and _,_ with

```
op _|->_ : Source Target -> Map  [prec 0 format(d b o d)] .
op _,_ : Map Map -> Map  [assoc comm id: empty format(d sr! oss d)] .
```

then the former will always be displayed in color blue, while the second in bold red and preceded by one space and followed by two spaces. Each group of characters in the argument of `format` refers to a pointcut in the operation name; we have default pointcuts at the beginning and at the end of the operation name, as well pointcuts before and after any special token (underscore, comma, and the various kinds of parentheses and brackets). In each group of characters, `d` means "default" and is used alone to skip that pointcut and move to the next, `b` and `r` the colors blue and red, `o` means to revert to the original color and style, `s` means one space, and `!` means bold font. There are also indentation attributes, which we have not used here but we will use later in the book, such as: `+` and `-` to increment and decrement the global indent counter, respectively, `i` to print the number of spaces determined by indent counter, and `n` to print a newline.

Built-in Modules

Maude provides several builtin modules and has been designed in a way that existing modules can be easily changed and more modules can be added. It is likely that at this moment Maude's builtin modules are not identical to the homonymous ones when this book was written, and it also likely that new modules have been added since then. To avoid depending on particular versions of Maude, and also to avoid unfortunate naming conflicts with existing builtins which prevent us from naming programming language constructs as desired, in this book we actually define a custom version builtins, discussed in Section A.1. Nevertheless, some of Maude's current builtin modules make interesting use of ACI matching and are also present in our custom builtins, albeit using different names, so we briefly discuss them here.

As already discussed, the `INT` module provides a sort `Int` with arbitrarily large integers and common operations on them. All these are essentially hooked to C library functions through a special interface that Maude provides, which we do not discuss here but refer the interested reader to Section A.1 for details.

Similarly, there is a builtin module named `QID`, from "quoted identifiers", which provides a sort `Qid` together with arbitrary large quoted identifiers, as constants of sort `Qid`, such as the following: `'a`, `'b`, `'abc123`, `'a-larger-identifier`, etc. These can be used as identifiers, e.g., as program variable names, in the programming languages that we define and execute using Maude.

Let us next discuss the module `BOOL`, which by default Maude includes in every other module (there are ways to disable the automatic inclusion, discussed in Section A.1). Besides the sort `Bool` with its two Boolean constants `true` and `false`, `BOOL` includes three important *polymorphic* operations and the usual Boolean operations. The polymorphic operations have the following syntax:

```
op if_then_else_fi : Bool Universal Universal -> Universal [...]
op _==_ : Universal Universal -> Bool [...]
op _=/=_ : Universal Universal -> Bool [...]
```

We excluded their specific attributes because they use advanced Maude features which are not necessary anywhere else in this book. Instead, we explain their behavior in words. The builtin sort `Universal` can be thought of as a generic sort, which can be instantiated with any concrete sort. Operation `if_then_else_fi` is strict in its first argument and lazy in its second and third arguments, that is, it only allows rewrites to take place in its first argument but not in the other two arguments. If the first argument rewrites to `true` then the conditional rewrites to its second argument, and if thefirst argument rewrites to `false` then the conditional rewrites to its third argument. We will discuss rewrite strategies in more depth later in this section. The other two operations correspond to operational equality and inequality of terms: the two terms are first rewritten to their normal forms and then those are compared modulo the existing operation ACI attributes. While operational equality implies logical equality, the other implication does not hold and tends to be a source of confusion, sometimes even among Maude experts: two terms t and t' may be provably equal using equational

reasoning, yet $t == t'$ may still rewrite to `false`. In this case, `false` only means that Maude was *not able* to show the two terms equal by rewriting them to their normal forms.

The definitions of the usual Boolean operations make interesting use of AC matching:

```
op _and_ : Bool Bool -> Bool [assoc comm prec 55] .
op _or_ : Bool Bool -> Bool [assoc comm prec 59] .
op _xor_ : Bool Bool -> Bool [assoc comm prec 57] .
op not_ : Bool -> Bool [prec 53] .
op _implies_ : Bool Bool -> Bool [prec 61 gather (e E)] .
vars A B C : Bool .
eq true and A = A .
eq false and A = false .
eq A and A = A .
eq false xor A = A .
eq A xor A = false .
eq A and (B xor C) = A and B xor A and C .
eq not A = A xor true .
eq A or B = A and B xor A xor B .
eq A implies B = not (A xor A and B) .
```

It can be shown that the equations above, when applied as rewrite rules, yield a decision procedure for propositional logic. Specifically, if we only consider `Bool` terms which are variables, any valid proposition rewrites to `true` and any unsatisfiable one to `false`; the remaining (satisfiable but invalid) propositions are rewritten to a canonical form consisting of an exclusive disjunction (`xor`) of conjunctions (`and`). We refer the interested reader to Section 2.3 for terminology and basic propositional logic results.

The attribute `gather(e E)` of `_implies_` tells the Maude parser that we want `_implies_` to be right associative, this way avoiding to add unnecessary parentheses in Boolean expressions. There is a similar attribute, `gather(E e)`, for left associativity. We do not discuss the gather attribute any further in this book.

Constructor versus Defined Operations

Recall the two equations of the module `PEANO-NAT`:

```
eq plus(zero, M) = M .
eq plus(succ(N), M) = succ(plus(N, M)) .
```

These equations constrain the three operations of the module, namely

```
op zero : -> Nat .
op succ : Nat -> Nat .
op plus : Nat Nat -> Nat .
```

But why did we write them in that particular style, which resembles a recursive definition of a *function* `plus` in terms of data-type *constructors* `zero` and `succ`? Intuitively, that is because we want `plus` to be completely *defined* in terms of `zero` and `succ`. Formally, this means that any term over the syntax `zero`, `succ`, and `plus` can be shown, using the given equations, equal to a term containing only `zero` and `succ` operations, that is, `zero` and `succ` alone are sufficient to build any Peano number.

While Maude at its core makes no distinction between operations meant to be data-type constructors and operations meant to be functions, it is still meaningful to distinguish the operations of a module into *constructor* and *defined* operations. Note, however, that it is the way we write the equations in the module, and only that, which makes the operations become constructors or defined. If we forget an equation dealing with a case (e.g., an intended constructor) for an operation intended to be defined, then that operation cannot

be always eliminated from terms, so technically speaking it is also a constructor. Unfortunately, there is no silver-bullet recipe on how to define "defined" operators, but essentially a good methodology is to define the operator's behavior on each intended constructor. That is what we did when we defined `plus` in `PEANO-NAT` and `mult` in `PEANO-NAT*`: we defined them on `zero` and on `succ`. In general, if `c1`, ..., `cn` are the intended constructors of a data-type, in order to define a new operation `d`, make sure that all equations of the form

```
eq d(c1(...)) = ...
...
eq d(cn(...)) = ...
```

are in the specification. If `d` has more arguments, then make sure that the above cases are listed for at least one of its arguments. This gives no guarantee (e.g., one can "define" `plus` as `plus(succ(M),N) = plus(succ(M),N)`), but it is a good enough principle to follow.

Let us demonstrate the above by defining several operations. Consider the following specification of lists:

```
mod INT-LIST is including INT .
  sort IntList .  subsort Int < IntList .
  op __ : Int IntList -> IntList [id: nil] .
  op nil : -> IntList .
endm
```

The two operations are meant to be constructors for lists, namely the empty list and adding an integer to the beginning of a list. Note, however, that the above contains *three* constructors for lists at first sight, because the subsorting of `Int` to `IntList` states that sole integers are also lists. Indeed, without the identity attribute of `__`, we would have to consider three cases when defining operations over lists. However, with the identity declaration Maude will internally identity integers `I` with lists `I nil`, so only two constructors are needed, corresponding to the two declared operations. Let us next define several important and useful operations on lists. Notice that the definition of each operator treats each of the two constructors separately.

The following defines the usual list length operator:

```
mod LENGTH is including INT-LIST .
  op length : IntList -> Nat .
  var I : Int .  var L : IntList .
  eq length(I L) = 1 + length(L) .
  eq length(nil) = 0 .
endm

red length(1 2 3 4 5) .   ***> should be 5
```

The following defines list membership, without speculating matching (in fact, this would not be possible anyway because concatenation is not defined associative as before):

```
mod IN is including INT-LIST .
  op _in_ : Int IntList -> Bool .
  vars I J : Int .  var L : IntList .
  eq I in J L = if I == J then true else I in L fi .
  eq I in nil = false .
endm

red 3 in 2 3 4 .   ***> should be true
red 3 in 3 4 5 .   ***> should be true
red 3 in 1 2 3 .   ***> should be true
red 3 in 1 2 4 .   ***> should be false
```

The next defines list append:

```
mod APPEND is including INT-LIST .
  op append : IntList IntList -> IntList .
  var I : Int .  vars L1 L2 : IntList .
  eq append(I L1, L2) = I append(L1, L2) .
  eq append(nil, L2) = L2 .
endm

red append(1 2 3 4, 5 6 7 8) .   ***> should be 1 2 3 4 5 6 7 8
```

Notice that `append` has two arguments and that we have picked the first one to define our cases on. One can still show that `append` is a defined operation, in that it can be eliminated by equational reasoning from any term of sort `IntList`. The following imports `APPEND` and defines an operation which reverses a list:

```
mod REV is including APPEND .
  op rev : IntList -> IntList .
  var I : Int .  var L : IntList .
  eq rev(I L) = append(rev(L), I) .
  eq rev(nil) = nil .
endm

red rev(1 2 3 4 5) .   ***> should be 5 4 3 2 1
```

The next module defines an operation, `isort`, which sorts a list of integers by insertion sort:

```
mod ISORT is including INT-LIST .
  op isort : IntList -> IntList .
  vars I J : Int .  var L : IntList .
  eq isort(I L) = insert(I, isort(L)) .
  eq isort(nil) = nil .
  op insert : Int IntList -> IntList .
  eq insert(I, J L) = if I > J then J insert(I,L) else I J L fi .
  eq insert(I, nil) = I .
endm

red isort(4 7 8 1 4 6 9 4 2 8 3 2 7 9) .   ***> should be 1 2 2 3 4 4 4 6 7 7 8 8 9 9
```

An auxiliary `insert` operation is also defined, which takes an integer and a sorted list and rewrites to a sorted list inserting the integer argument at its place in the list argument. Notice that this latter operation makes use of the builtn `if_then_else_fi` operation provided by the default `BOOL` module discussed above, as well as of the integer comparison operation “>” provided by the builtin module `INT`.

Let us now consider binary trees, where a tree is either empty or an integer with a left and a right subtree:

```
mod TREE is including INT .
  sort Tree .
  op ___ : Tree Int Tree -> Tree .
  op empty : -> Tree .
endm
```

We next define some operations on trees, following the tree structure given by the two constructors above.

The next operation mirrors a tree, i.e., it replaces left subtrees by the mirrored right siblings and vice-versa:

```
mod MIRROR is including TREE .
  op mirror : Tree -> Tree .
  vars L R : Tree .  var I : Int .
```

```
  eq mirror(L I R) = mirror(R) I mirror(L) .
  eq mirror(empty) = empty .
endm

red mirror((empty 3 (empty 1 empty)) 5 ((empty 6 empty) 2 empty)) .
***> should be (empty 2 (empty 6 empty)) 5 ((empty 1 empty) 3 empty)
```

Searching in binary trees can be defined as follows:

```
mod SEARCH is including TREE .
  op search : Int Tree -> Bool .
  vars I J : Int .  vars L R : Tree .
  eq search(I, L I R) = true .
 ceq search(I, L J R) = search(I, L) or search(I, R) if I =/= J .
  eq search(I, empty) = false .
endm

red search(6, (empty 3 (empty 1 empty)) 5 ((empty 6 empty) 2 empty)) .   ***> should be true
red search(7, (empty 3 (empty 1 empty)) 5 ((empty 6 empty) 2 empty)) .   ***> should be false
```

Note that we used a conditional equation above. Conditional equations are introduced with the keyword `ceq`, and their condition with the keyword `if`. There are several types of conditions in Maude, which we will discuss in the sequel, as needed. Here we used the simplest of them, namely a Bool term. To be faithful to rewrite logic (Section 2.5), we can regard a Boolean condition b as syntactic sugar for the equality b = `true`; in fact, Maude also allows us to write b = `true` instead of b. We can combine the first two equations above into an unconditional one, using an `if_then_else_fi` in its RHS (see Exercise 31).

We next define a module which imports both modules of trees and of lists on integers, and defines an operation which takes a tree and returns the list of all integers in that tree, in an infix traversal:

```
mod FLATTEN is
  including APPEND .
  including TREE .
  op flatten : Tree -> IntList .
  vars L R : Tree .  var I : Int .
  eq flatten(L I R) = append(flatten(L), I flatten(R)) .
  eq flatten(empty) = nil .
endm

red flatten((empty 3 (empty 1 empty)) 5 ((empty 6 empty) 2 empty)) .  ***> should be 3 1 5 6 2
```

Reduction Strategies

We sometimes want to inhibit the application of equations on some subterms, for executability reasons. For example, imagine a conditional expression construct `if_then_else_` in a pure language or calculus, i.e., one whose expression evaluation has no side-effects, say λ-calculus (this is discussed extensively in Section 4.5), whose semantics is governed by equations. While it is semantically safe to apply equations anywhere at any time, including inside the two branches of the conditional, for executability reasons we may prefer not to. Indeed, e.g., any reduction step applied in the negative branch would be a waste of computation if the condition turns out to evaluate to `true`. Worse, the wasteful reduction steps may lead to computational non-termination and resource exhaustion without giving the conditional a chance to evaluate its condition and then discard the non-terminating branch.

Because of reasons like above, many executable equational logic systems provide support for *reduction strategies*. In Maude, reduction strategies are given as operator `strat` attributes taking as argument sequences

of numbers. For example, the conditional operation would be assigned the strategy `strat(1 0)`, meaning that the first argument of the conditional is reduced to its normal form (the `1`), and *then* the conditional itself is allowed to be reduced (the `0`); since `2` and `3` are not mentioned, the second and the third argument of the conditional are never reduced while the conditional statement is still there. By default, operations of `n` arguments have strategy `strat(1 2...n 0)`, which yields a leftmost innermost reduction strategy.

Let us next discuss an interesting application of rewrite strategies in the context and lazy, infinite data-structures. One of the simplest such structure is the *stream*, that is, the infinite sequence. Specifically, we consider streams of integers, as defined in Figure 2.8. The key idea is to assign the stream construct `_:_`, which adds a given integer to the beginning of a given stream, the reduction strategy `strat(1 0)`. In other words, reduction is inhibited in the tail of streams built using the `_:_`. This allows us to have finite representations of infinite streams. To "observe" the actual elements of such stream structures, we define an operation `#` taking a natural number `N` and a stream `S`, and unrolling `S` until its first `N` elements become available. The module below defines this basic stream infrastructure, as well as some common streams and operations on them:

```
mod STREAM is including INT .
  sort Stream .
  op _:_ : Int Stream -> Stream [strat(1 0)] .
  var N : Int .  var S S' : Stream .
  op h : Stream -> Int .              eq h(N : S) = N .
  op t : Stream -> Stream .           eq t(N : S) = S .
  op # : Nat Stream -> Stream .   ***> #(N,S) displays the first N elements of stream S
  eq #(1, S) = h(S) : t(S) .
 ceq #(s(N), S) = h(S) : S' if S' := #(N, t(S)) .
  op zeros : -> Stream .              eq zeros = 0 : zeros .
  op ones : -> Stream .               eq ones = 1 : ones .
  op nat : Nat -> Stream .            eq nat(N) = N : nat(N + 1) .
  op zip : Stream Stream -> Stream .  eq zip(S, S') = h(S) : zip(S',t(S)) .
  op blink : -> Stream .              eq blink = zip(zeros,ones) .
  op add : Stream -> Stream .         eq add(S) = (h(S) + h(t(S))) : add(t(t(S))) .
  op fibonacci : -> Stream .          eq fibonacci = 0 : 1 : add(zip(fibonacci,t(fibonacci))) .
endm
```

The definition of the basic observers `h` (head) and `t` (tail) is self-explanatory. The definition of the general observer `#` involves a conditional equation, and that equation has a *matching* (`:=`) in its condition. In terms of equational logic, the matching symbol `:=` is nothing but the equality `=`; however, because of its special operational nature allowing us to introduce variables that do not appear in the LHS of the equation, Maude prefers to use a different symbol for it. The streams `zeros` and `ones`, of infinitely many 0 and 1 sequences, respectively, can be defined so compactly and then executed exactly because of the reduction strategy of `_:_`, which disallows the uncontrolled, non-terminating unrolling of these streams. However, we can observe as many elements of these streams as we wish, say 7, using controlled unrolling as follows:

```
red #(7, zeros) .                  ***> 0 : 0 : 0 : 0 : 0 : 0 : 0 : t(0 : zeros)
red #(7, ones) .                   ***> 1 : 1 : 1 : 1 : 1 : 1 : 1 : t(1 : ones)
```

Note how the reduction strategy enables the tails of the streams above, starting with their 8-th element, to stay unreduced, thus preventing non-termination.

The module above also defines the stream of natural numbers, an operation `zip` which interleaves two streams element by element, and operation `add` which generates a new stream by adding any two consecutive elements of a given stream, and finally two concrete streams defined using `zip`: one called `blink` which zips the streams `zeros` and `ones`, and one called `fibonacci` which contains the Fibonacci sequence (see also Exercise 25). Here are some sample reductions with these streams:

```
red #(7, nat(1)) .                  ***> 1 : 2 : 3 : 4 : 5 : 6 : 7 : t(7 : nat(1 + 7))
red #(7, blink) .                   ***> 0 : 1 : 0 : 1 : 0 : 1 : 0 : ...
red #(7, add(add(add(ones)))) .     ***> 8 : 8 : 8 : 8 : 8 : 8 : 8 : ...
red #(7, fibonacci) .               ***> 0 : 1 : 1 : 2 : 3 : 5 : 8 : ...
```

To save space, we did not show the remaining tail terms for the last three reductions.

Rewrite Rules

Until now we have only discussed one type of Maude sentences, its equations, and how to perform reductions with them regarded as rewrite rules, oriented from left to right. As seen in Section 2.5.1, rewrite logic has two types of sentences: equations and rewrite rules. Semantically, the rewrite rules establish transitions between equivalence classes of terms obtained using the equations. In other words, the distinction between equations and rewrite rules is that the former can be applied in any direction and do not count as computational steps in the transition systems associated to terms, while the latter can only be irreversibly applied from left to right and count as transitions. Because of the complexity explosion resulting from applying equations bidirectionally, Maude applies them in very restricted but efficient ways, which we have already discussed: equations like associativity, commutativity and identity are given by means of operator attributes and are incorporated within Maude's internal ACI rewrite algorithm, while other equations are only applied from left to right, same like the rewrite rules. While there is no visible distinction in Maude between equations and rewrite rules in terms of executability, several of Maude's tools, including the `search` command discussed below, treat them differently, so it is important to understand the difference between them.

The following module, inspired from the Maude manual, models a simple vending machine selling coffee for one dollar and tea for three quarters. The machine takes only one-dollar coins as input (`$`). For one dollar one can either buy one coffee, or one tea with a quarter rest. Assume that an external agent can silently and automatically change money as needed, that is, four quarters into a dollar (the viceversa is useless here); in other words, changing money does not count as a transaction in this simplistic vending machine model.

```
mod VENDING-MACHINE is
    sorts Coin Item State .
    subsorts Coin Item < State .
    op __ : State State -> State [assoc comm id: null] .
    op null : -> State .
    op $ : -> Coin      [format (r! o)] .
    op q : -> Coin      [format (r! o)] .
    op tea : -> Item    [format (b! o)] .
    op coffee : -> Item [format (b! o)] .

    rl $ => coffee .
    rl $ => tea q .
    eq q q q q = $ .
endm
```

All the coins and all the items that one has at any given moment forms a `State`. The two rules above model the two actions that the machine can perform, and the equation models the money changing agent.

A first distinction between equations and rules can be seen when using the `reduce` command:

```
Maude> red $ q q q .
result State: $ q q q
```

Note that the equation cannot reduce this state any further when applied from left to right, and that no rewrite rules were applied. Indeed, the command `reduce` only applies equations, and only from left-to-right. To apply both equations *and* rewrite rules, we have to use the command `rewrite` or its shortcut `rew`:

```
Maude> rew $ q q q .
result State: q q q coffee
```

Maude chose to apply the first rule and the resulting state cannot be rewritten or reduced anymore, so we are stuck with three useless quarters. If Maude had chosen the second rule, then we could have bought both a coffee and a tea with our money. To see all possible ways to rewrite a given term, we should use the `search` command instead of `rewrite`:

```
Maude> search $ q q q =>! S:State .
Solution 1 (state 1)
S:State --> q q q coffee
Solution 2 (state 3)
S:State --> tea coffee
Solution 3 (state 4)
S:State --> q tea tea
```

The `search` command takes a term and a pattern and attempts to systematically (in breadth-first order) apply the rewrite rules on the original term in order to match the pattern. In our case, the pattern is a state variable `S`, so all states match it. The term and the pattern are separated by a decorated arrow. Different decorations mean different things. The `!` above means that we are interested only in normal forms. Indeed, the above search command has precisely three solutions, as reported by Maude. Another useful decoration is `*`, which shows all intermediate states, not only the normal forms:

```
Maude> search $ q q q =>* S:State .
Solution 1 (state 0)
S:State --> $ q q q
Solution 2 (state 1)
S:State --> q q q coffee
Solution 3 (state 2)
S:State --> $ tea
Solution 4 (state 3)
S:State --> tea coffee
Solution 5 (state 4)
S:State --> q tea tea
```

Notice that we never see four quarters in a state in the solutions above, in spite of the `*` decoration; the equation automatically changed them into a dollar. Remember: *equations do not count as transitions*, so their application is not visible in the transition system explored by `search`. Another way to think of it is *rewrite rules apply modulo equations*; that is, equations structurally rearrange the term so that rules match and apply. Yet another way to think about it is that equations take time zero to apply, i.e., they apply instantaneously no matter how many they are, while rules take time one. In terms of performance, rules are slightly slower in Maude because they require more infrastructure to be maintained in terms, but that should not be the deciding factor when choosing whether a sentence should be an equation or a rule. One typical criterion for deciding what is an equation and what is a rule is that computations performed with the former are meant to be deterministic, while rewrite rules can lead to non-deterministic behaviors (like our two rules above).

One of Maude's major strengths, in addition to its efficient support for rewriting modulo ACI, is its capability to perform reachability analysis in conditions of rules. Consider, for example, the following extension of our vending machine model (`S` and `S'` are variables of sort `State`):

```
 op both? : State -> Bool .
crl both?(S) => true if S => coffee tea S' .
```

The conditional rule rewrites `both?(S)` to `true` when `S` rewrites to at least one coffee and one tea. However, to check the condition, exhaustive search may be needed. Otherwise, wrong normal forms may be reported.

```
Maude> rew both?(q q q q q q q q) .
result Bool: true
```

Maude correctly reported `true` above; however, without search in the condition one could have wrongly reported the term stuck, for example if one would have bought two coffees from the two dollars in the condition instead of one coffee and one tea.

As expected, the `search` command also works with conditional rules:

```
Maude> search both?(q q q q q q q q) =>* B:Bool .
Solution 1 (state 0)
B:Bool --> both?($ $)
Solution 2 (state 1)
B:Bool --> true
Solution 3 (state 2)
B:Bool --> both?($ coffee)
Solution 4 (state 3)
B:Bool --> both?($ q tea)
Solution 5 (state 4)
B:Bool --> both?(coffee coffee)
Solution 6 (state 5)
B:Bool --> both?(q tea coffee)
Solution 7 (state 6)
B:Bool --> both?(q q tea tea)
```

Interestingly, note that many other solutions besides `true` have been reported above, some of them in normal form (use `!` instead of `*` in the search command to see only those in normal form). That is because the rules of the original vending machine applied inside the argument of `both?`, which is something that we did not intend to happen when we wrote the conditional rule above. To prohibit rewrite rules from applying in some arguments of an operation, we have to use the `frozen` operator attribute with corresponding arguments:

```
op both? : State -> Bool [frozen(1)] .
```

Unlike the `strat` attribute which gives permission to reductions inside operator arguments, the `frozen` attribute takes permission to rewrites inside operator arguments. The `strat` attribute only works with equations and is ignored by rewrite rules, while the `frozen` attribute only works with rewrite rules and is ignored by equations. For example, the above search command still reports two solutions:

```
Maude> search both?(q q q q q q q q) =>* B:Bool .
Solution 1 (state 0)
B:Bool --> both?($ $)
Solution 2 (state 1)
B:Bool --> true
```

The first solution is an artifact of `frozen` allowing the equation to apply within the argument of `both?`. If one wants to prohibit both rules and equations, then one should use both `frozen` and `strat` attributes:

```
op both? : State -> Bool [frozen(1) strat(0)] .
```

Rules can have multiple conditions and the conditions can share variables, e.g., (`I`,`I'` of sort `Item`)

```
 op solve : State -> State [frozen(1) strat(0)] .
crl solve(S) => S' if S => I I I S' /\ S => I' I' I' I' S' .
```

The rule above says that `solve(S)` rewrites to `S'` when `S'` is a rest that can be obtained from `S` both after buying three identical items and after buying four identical items. Common sense tells us that the three identical items must be coffee and the four identical items must be tea, and that `S'` is `S` minus the three dollars spent to buy any of these groups of identical items. But Maude does not have this common sense, it needs to search. Specifically, it will do search in the first condition until three identical items are reached, then it does search in the second condition until four identical items are found with the same rest `S'` as in the first search; if this is not possible, then it backtracks and searches for another match of three identical items in the first rule, and so on and so forth until the entire state-space is exhausted (if finite, otherwise possibly forever).

Interestingly, although either three coffees or four teas cost three dollars, we cannot buy each of these with three dollars:

```
Maude> search solve($ $ $) =>! S .
Solution 1 (state 0)
S --> solve($ $ $)
```

The term `solve($ $ $)` is in normal form because there is no way to satisfy the second condition of the conditional rule above. However, if we have one quarter in addition to the three dollars, then we can satisfy the second condition of the rule, too, because we can first buy three teas getting three quarters back, which together with the additional quarter can be changed into one dollar, which gives us one more tea and a quarter back. So from three dollars and a quarter we can buy either three coffees or four teas, with a quarter rest:

```
Maude> rew solve($ $ $ q) .
result Coin: q
Maude> search solve($ $ $ q) =>! S .
Solution 1 (state 1)
S --> q
```

Therefore, both the rewrite and the search commands above have solved the double condition of the conditional rule. Moreover, the search command tells us that there is only one solution to the conditional rule's constraints.

When giving semantics to programming languages using rewrite logic, conditional rules will be used to reduce the semantic task associated to a language construct to similar semantic tasks associated to its arguments. Since some language constructs have a non-deterministic behavior, the search capability of Maude has a crucial role. In spite of the strength and elegance of Maude's conditional rules, note, however, that they are quite expensive to execute. Indeed, due to the combined variable constraints resulting from the various conditions, in the worst case there is no way to avoid an exhaustive search of the entire state-space in rules' conditions. Additionally, each rule used in the search-space of a condition can itself be conditional, which may itself require an exhaustive search to solve its condition, and so on and so forth. All this nested search process is clearly expensive. It is therefore highly recommended to avoid conditional rules whenever possible. As seen in Chapter 3, some semantic styles cannot avoid the use of conditional rules, but others can.

Turing Machines in Maude

Section 2.5.3 showed how to faithfully represent Turing machines in rewrite logic, in a way that any computational step in the original Turing machine corresponds to precisely one rewrite step in its (rewrite theory) representation and viceversa. Here we show how such rewrite theories can be defined and executed in Maude. Since the Turing machine transitions are represented as rewrite rules and not as equations, non-deterministic Turing machines can be represented as well following the same approach and without any additional complexity. Following the model in Section 2.5.3, we first discuss the representation based on lazy reduction strategies and then the unrestricted representation. For the lazy one, the idea is to define the infinite tape of zeros lazily, as we did when we defined the various kinds of streams above:

```
mod TAPE is
  sorts Cell Tape .
  ops 0 1 : -> Cell .
  op _:_ : Cell Tape -> Tape [strat(1 0)] .
  op zeros : -> Tape .
  eq zeros = 0 : zeros .
endm
```

Thanks to the `strat(1 0)` reduction strategy of the `_:_` construct above, the expanding equation of `zeros` only applies when `zeros` is on a position different from the tail of a stream/tape; in particular, it cannot be applied to further expand the `zeros` in its RHS.

Now we can define any Turing machine $\mathcal{M}$ in Maude using the approach in Figure 2.10 by importing `TAPE`, defining operations "`q : Tape Tape -> Tape`" for all $q \in Q$, and adding all the rules corresponding to the Turing machine's transition function (as explained in Figure 2.10). For example, the Maude representation of the Turing machine in Figure 2.1 that computes the successor function is:

```
mod TURING-MACHINE-SUCC is including TAPE .
  sort Configuration .
  ops qs qh q1 q2 : Tape Tape -> Configuration .
  var L R : Tape .  var B : Cell .
  rl qs(L, 0 : R) => q1(0 : L, R) .
  rl q1(L, 0 : R) => q2(L, 1 : R) .
  rl q1(L, 1 : R) => q1(1 : L, R) .
  rl q2(L, 0 : R) => qh(L, 0 : R) .
  rl q2(B : L, 1 : R) => q2(L, B : 1 : R) .
endm
```

Now we can "execute" the Turing machine using Maude's rewriting:

```
Maude> rew qs(zeros, 0 : 1 : zeros) .
result Configuration: qh(0 : zeros, 0 : 1 : 1 : zeros)
Maude> rew qs(zeros, 0 : 1 : 1 : 1 : 1 : zeros) .
result Configuration: qh(0 : zeros, 0 : 1 : 1 : 1 : 1 : 1 : zeros)
```

Recall from Section 2.2.1 that a natural number input n is encoded as $n + 1$ bits of 1 following the start cell where the head of the machine is initially, which holds the bit 0. So the first rewrite command above rewrites the natural number 0 to its successor 1, while the second rewrites 3 to 4. Note that the self-expanding equation of `zeros` is not applied backwards, so resulting streams/tapes of the form `0 : zeros` are not compacted back into `zeros`. One needs specific equations to do so, which we do not show here (see Exercise 34).

Non-deterministic Turing machines can be defined equally easily. For example, the machine non-deterministically choosing to yield the successor or not when it reaches the end of the input, also discussed in Section 2.5.3, can be obtained by adding the rule

```
  rl q1(B : L, 0 : R) => q2(L, B : 0 : R) .
```

Of course, `search` is needed now in order to explore all the non-deterministic behaviors:

```
Maude> search qs(zeros, 0 : 1 : 1 : 1 : 1 : zeros) =>! C:Configuration .
Solution 1 (state 16)
C:Configuration --> qh(0 : zeros, 0 : 1 : 1 : 1 : 1 : 0 : zeros)
Solution 2 (state 18)
C:Configuration --> qh(0 : zeros, 0 : 1 : 1 : 1 : 1 : 1 : zeros)
```

The unrestricted representation of Turing machines in rewrite logic discussed in Section 2.5.3 (Figure 2.11) can also be easily defined and then executed in Maude. In fact, this unrestricted representation has the advantage that it requires no special support for reduction strategies from the underlying rewrite system, so it should be easily adaptable to other rewrite systems than Maude. Since the self-expanding equation of *zeros* is not needed anymore, we can now define the tape as a plain algebraic signature:

```
mod TAPE is
  sorts Cell Tape .
  ops 0 1 : -> Cell .
  op _:_ : Cell Tape -> Tape .
  op zeros : -> Tape .
endm
```

The two new equations in Figure 2.11 can be defined generically as follows, for any Turing machine state:

```
mod TURING-MACHINE is including TAPE .
  sorts State Configuration .
  op _(_,_) : State Tape Tape -> Configuration .
  var S : State .  var L R : Tape .
  eq S(zeros,R) = S(0 : zeros, R) .
  eq S(L,zeros) = S(L, 0 : zeros) .
endm
```

Particular Turing machines can now be defined by including the module `TURING-MACHINE` above and adding specific states and rules. For example, here is the one calculating the successor already discussed above:

```
mod TURING-MACHINE-SUCC is including TURING-MACHINE .
  ops qs qh q1 q2 : -> State .
  var L R : Tape .  var B : Cell .
  rl qs(L, 0 : R) => q1(0 : L, R) .
  rl q1(L, 0 : R) => q2(L, 1 : R) .
  rl q1(L, 1 : R) => q1(1 : L, R) .
  rl q2(L, 0 : R) => qh(L, 0 : R) .
  rl q2(B : L, 1 : R) => q2(L, B : 1 : R) .
endm
```

The Post Correspondence Problem in Maude

We here show how to define in Maude the rewrite theory in Section 2.5.3 which allows to reduce the Post correspondence problem to rewrite logic reachability. We define strings as AI sequences of symbols, but for output reasons we prefer tiles to be triples instead of just pairs of strings, where the additional string acts as the label of the tile. To easily distinguish labels from each other, we prefer to technically work with strings of natural numbers instead of bits, although we will only use the numbers 0 and 1 in strings not meant to serve as labels. The module below is self-explanatory:

```
mod PCP is including NAT .
  sorts Symbol String .  subsort Nat < Symbol < String .
  sorts Tile Tiles .     subsort Tile < Tiles .
  op . : -> String .                                          --- empty string
  op __ : String String -> String [assoc id: .] .             --- string concatenation
  op _[_,_] : String String String -> Tile [prec 3] .         --- first string is the label
  op __ : Tiles Tiles -> Tiles [assoc comm] .                 --- concatenation of tiles
  var L L' R R' S S' : String .
  rl L[R,S] L'[R',S'] => L[R,S] L'[R',S'] (L L')[R R', S S'] .  --- the only rule
endm
```

So a tile has the form *label*$[\alpha,\beta]$, where *label* is the label of the tile and α and β are the tile's two strings. The unique rule matches any two tiles and adds their concatenation to the pool, without removing them. This way, we can start with any set of tiles and eventually reach any combination of them. Obviously this rewrite theory does not terminate. Indeed, if we attempt to execute the command

```
Maude> rewrite 1[0, 1 0 0] 2[0 1, 0 0] 3[1 1 0, 1 1] .
```

then Maude will rewrite forever; in fact, Maude will eventually run out of memory, because each rule application adds one more tile to the pool (and tiles can repeat). The Maude flag `trace` can be used to see what rewrite steps are applied and is typically used during debugging. For example,

```
Maude> set trace on .
Maude> rewrite 1[0, 1 0 0] 2[0 1, 0 0] 3[1 1 0, 1 1] .
```

outputs an infinite trace starting as follows:

```
*********** rule
rl L[R,S] L'[R',S'] => (L'[R',S'] (L L')[R R',S S']) L[R,S] .
L --> 1
R --> 0
S --> 1 0 0
L' --> 2
R' --> 0 1
S' --> 0 0
1[0,1 0 0] 2[0 1,0 0] 3[1 1 0,1 1]
--->
3[1 1 0,1 1] (2[0 1,0 0] (1 2)[0 0 1,(1 0 0) 0 0]) 1[0,1 0 0]
*********** rule
rl L[R,S] L'[R',S'] => (L'[R',S'] (L L')[R R',S S']) L[R,S] .
L --> 1
R --> 0
S --> 1 0 0
L' --> 2
R' --> 0 1
S' --> 0 0
1[0,1 0 0] 2[0 1,0 0] 3[1 1 0,1 1] (1 2)[0 0 1,1 0 0 0 0]
--->
(3[1 1 0,1 1] (1 2)[0 0 1,1 0 0 0 0]) (2[0 1,0 0] (1 2)[0 0 1,(1 0 0) 0 0]) 1[0,1 0 0]
...
```

Each step in the trace shows a rule instance and the resulting term (Maude may re-arrange the AC terms). Here it just happens that Maude chooses the same first two tiles to combine at each step.

Maude also allows to pass an optional number to the rewrite command, indicating how many rewrite steps are desired. For example:

```
Maude> rewrite[2] 1[0, 1 0 0] 2[0 1, 0 0] 3[1 1 0, 1 1] .
result Tiles: 1[0,1 0 0] 2[0 1,0 0] 3[1 1 0,1 1] (1 2)[0 0 1,1 0 0 0 0] (1 2)[0 0 1,1 0 0 0 0]
```

One should be careful when using this option with conditional rewrite rules, because only the top-level applications of rules are counted. The rules used to solve the conditions are *not* counted. It is therefore possible that the command `rewrite[1]` does not terminate, namely when the condition of a matching conditional rule does not terminate.

Consequently, we can only use the `search` command here to solve the PCP. Specifically, we should search for patterns containing a tile of the form *label*$[\gamma,\gamma]$. Any term of sort `Tiles` matching such a pattern

indicates that a successful combination of the original tiles has been found; moreover, *label* will be bound to the desired sequence of labels of the combined tiles and γ to the combined string. For example, the following shows that the specific PCP problem mentioned in Section 2.2.2 is solvable, as well as a solution:

```
Maude> search[1] 1[0, 1 0 0] 2[0 1, 0 0] 3[1 1 0, 1 1] =>*  L[S,S] Ts:Tiles .
L --> 3 2 3 1
S --> 1 1 0 0 1 1 1 0 0
Ts:Tiles --> 1[0,1 0 0] 2[0 1,0 0] 3[1 1 0,1 1] (2 3)[0 1 1 1 0,0 0 1 1] ...
```

It is important to use `search[1]` above, because otherwise Maude will continue to search for all solutions and thus will never terminate. The option `[1]` tells Maude to stop searching after finding one solution.

Since the PCP problem is undecidable, we can conclude that Maude's search is undecidable in general (which is not a surprise, but we now have a formal proof).

2.5.7 Exercises

Exercise 27. *Eliminate the two generic equations in Figure 2.11 as discussed below Theorem 11 and prove that the resulting rewrite theory is confluent and captures the computation in $\mathcal{M}$ faithfully, step-for-step.*

Exercise 28. *Prove that any correct implementation of* `PEANO-NAT` *(Section 2.5.6) should satisfy the property*

```
plus(succ(succ(zero)), succ(succ(succ(zero)))) =
plus(succ(succ(succ(succ(zero)))), succ(zero)).
```

Exercise 29. *Rewrite* `PEANO-NAT` *and* `PEANO-NAT*` *(Section 2.5.6) using Maude's mixfix notation for operations. What happens if we try to reduce an expression containing both* `_+_` *and* `_*_` *without parentheses?*

Exercise 30. *Define a Maude module called* `INT-SET` *specifying sets of integers with membership, union, intersection, difference (elements in the first set but not in the second), and symmetric difference (elements in any of the two sets but not in the other).*

Exercise 31. *Define the* `search` *operation in module* `SEARCH` *(Section 2.5.6) with only two unconditional equations, using the built-in* `if_then_else_fi`.

Exercise 32. *Recall module* `FLATTEN` *(Section 2.5.6) which defines and infix traversal operation on binary trees. Do the same for prefix and for postfix traversals.*

Exercise 33. *Write a Maude module that uses binary trees as defined in module* `TREE` *(Section 2.5.6) to sort lists of integers. You should define an operation* `btsort : IntList -> IntList`, *which sorts the argument list of integers (like the* `isort` *operation in module* `ISORT` *in Section 2.5.6). In order to define* `btsort`, *define another operation,* `bt-insert : IntList Tree -> Tree`, *which inserts each integer in the list at its place in the tree, and also use the* `flatten` *operation already defined in module* `FLATTEN`.

Exercise 34. *When executing Turing machines in Maude as shown in Section 2.5.6, we obtain final configurations which contain (sub)streams/tapes of the form* `0 : zeros`. *While these are semantically equal to* `zeros`, *they decrease the readability of the final Turing machine configurations. Add generic equations to canonicalize the final configurations by iteratively replacing each sub-term* `0 : zeros` *with* `zeros`.
Hint: A special reduction strategy may be needed for the operation `qh` *(or the configuration construct, respectively), to inhibit the application of the self-expanding equation of* `zeros`.

Exercise 35. *Define in Maude the Turing machines corresponding to the addition, the multiplication, and the power operations on natural numbers in Exercise 12. Do it using three different approaches: (1) using the infinite stream* `zeros`*, following the representation in Figure 2.10; (2) without using infinite streams but mimicking them with the by-need expansion using the two equations in Figure 2.11; (3) without any equations, following the style suggested right after Theorem 11, at the expense of adding more rules.*

Exercise 36. *Use the Maude definition of the Post correspondence problem in Section 2.5.6 to calculate the least common multiplier of two natural numbers. The idea here is to create an appropriate set of tiles from the two numbers so that the solution to the search command contains the desired least common multiplier.*

Talk about kinds and MEL? We currently need them in rsec (contextual representation of syntax is a kind), but maybe we can eliminate them from there, too.

Discuss the LTL model checker? If yes, the dining philosophers example with rewrite rules.

2.5.8 Notes

Maude's roots go back to Clear (Edinburgh) in 1970s and to OBJ (Stanford, Oxford) in 1980s. There are many other languages in the same family, such as CafeOBJ (Japan), BOBJ (San Diego, California), ELAN (France).

Maude is currently being developed at Stanford Research Institute and at the University of Illinois at Urbana-Champaign. You can find more about Maude, including a manual and many papers and examples, on its webpage at `http://maude.cs.uiuc.edu`.

Salvador with strategies

2.6 Complete Partial Orders and the Fixed-Point Theorem

This section introduces a fixed-point theorem for complete partial orders. Our main use of this theorem is to give denotational semantics to iterative (in Section 3.4) and to recursive (in Section 4.7) language constructs. Complete partial orders with bottom elements are at the core of both domain theory and denotational semantics, often identified with the notion of "domain" itself.

2.6.1 Posets, Upper Bounds and Least Upper Bounds

Here we recall basic notions of partial and total orders, such as upper bounds and least upper bounds, and discuss several examples and counter-examples.

Definition 14. *A **partial order**, or a **poset** (from partial order set) $(D, \sqsubseteq)$ consists of a set D and a binary relation $\sqsubseteq$ on D, written as an infix operation, which is*

- ***reflexive**, that is, $x \sqsubseteq x$ for any $x \in D$;*
- ***transitive**, that is, for any $x, y, z \in D$, $x \sqsubseteq y$ and $y \sqsubseteq z$ imply $x \sqsubseteq z$; and*
- ***anti-symmetric**, that is, if $x \sqsubseteq y$ and $y \sqsubseteq x$ for some $x, y \in D$ then $x = y$.*

*A partial order $(D, \sqsubseteq)$ is called a **total order** when $x \sqsubseteq y$ or $y \sqsubseteq x$ for any $x, y \in D$.*

Here are some examples of partial or total orders:

- $(\mathcal{P}(S), \subseteq)$ is a partial order, where $\mathcal{P}(S)$ is the powerset of a set S, that is, the set of subsets of S.
- $(Nat, \leq)$, the set of natural numbers ordered by "less than or equal to", is a total order.
- $(Nat, \geq)$, the set of natural numbers ordered by "larger than or equal to", is a total order.
- $(Nat \cup \{\infty\}, \leq)$, the naturals plus infinity, where infinity is larger than any number, is a total order.
- $(Int, \leq)$, the set of integer numbers, is a total order.
- $(Real, \leq)$, the set of real numbers, is a total order.
- $(S, =)$, a flat set S where the only partial ordering is the identity, is a partial order.
- $(S \cup \{\bot\}, \leq_{\bot}^{S})$, a set S extended with a bottom element $\bot$ with the property that for any $a, b \in S \cup \{\bot\}$, $a \leq_{\bot}^{S} b$ if and only if $a = b$ or $a = \bot$, is a partial order. Such extensions of sets with bottom elements are common in denotational semantics (Section 3.4), where the intuition for the $\bot$ is *undefined*. They are commonly written more simply as $S_{\bot}$ and are often called *primitive* or *flat* domains.
- A partial order which is particularly important for denotational semantics (Section 3.4) is $(A \rightharpoonup B, \preceq)$, the set of partial functions from A to B partially ordered by the *informativeness relation* $\preceq$: given partial functions $f, g : A \rightharpoonup B$, f is "less informative than or as informative as" g, written $f \preceq g$, iff for any $a \in A$, it is either the case that $f(a)$ is not defined or both $f(a)$ and $g(a)$ are defined and $f(a) = g(a)$.

- If $(S_1, \leq_1)$ and $(S_2, \leq_2)$ are partial or total orders and $S_1 \cap S_2 = \emptyset$ then $(S_1 \cup S_2, \leq_1 \cup \leq_2)$ is a partial order, where $a(\leq_1 \cup \leq_2)b$ if and only if there is some $i \in \{1, 2\}$ such that $a, b \in S_i$ and $a \leq_i b$. This union partial order is typically not a total order; it is a total order only when $(S_1, \leq_1)$ is total and $S_2 = \emptyset$, or the other way around. More generally, if $\{(S_i, \leq_i)\}_{i \in I}$ is a family of partial orders such that $S_i \cap S_j = \emptyset$ for any $i \neq j \in I$, then $\bigcup_{i \in I}(S_i, \leq_i) \stackrel{\text{def}}{=} (\bigcup_{i \in I} S_i, \bigcup_{i \in I} \leq)$ is a partial order, where $a \ (\bigcup_{i \in I} \leq_i) \ b$ if and only if there is some $i \in I$ such that $a, b \in S_i$ and $a \leq_i b$.

- If $\{(S_i, \leq_i)\}_{i \in I}$ is a family of partial orders then $\prod_{i \in I}(S_i, \leq_i) \stackrel{\text{def}}{=} (\prod_{i \in I} S_i, \prod_{i \in I} \leq_i)$ is also a partial order, where $\{a_i\}_{i \in I} \ (\prod_{i \in I} \leq_i) \ \{b_i\}_{i \in I}$ iff $a_i \leq_i b_i$ for all $i \in I$ (i.e., the product partial order is defined component-wise).

Definition 15. *Given partial order $(D, \sqsubseteq)$ and a set of elements $X \subseteq D$, an element $p \in D$ is called an **upper bound** of X iff $x \sqsubseteq p$ for any $x \in X$. Furthermore, $p \in D$ is called the **least upper bound (LUB)** of X, written $\sqcup X$, iff p is an upper bound and for any other upper bound q of X it is the case that $p \sqsubseteq q$.*

Upper bounds and least upper bounds may not always exist. For example, if $D = X = \{x, y\}$ and $\sqsubseteq$ is the identity relation, then X has no upper bounds. Least upper bounds may not exist even though upper bounds exist. For example, if $D = \{a, b, c, d, e\}$ and $\sqsubseteq$ is defined by $a \sqsubseteq c$, $a \sqsubseteq d$, $b \sqsubseteq c$, $b \sqsubseteq d$, $c \sqsubseteq e$, $d \sqsubseteq e$, then any subset of D admits upper bounds, but the set $\{a, b\}$ does not have a LUB. Due to the anti-symmetry property, least upper bounds are unique when they exist, which justifies the notation $\sqcup X$ in Definition 15.

As an analogy with mathematical analysis, we can think of the LUB of a set of elements as their *limit*. For example, in $(Real, \leq)$ the LUB of the set $\{n/(n+1) \mid n \in Nat\}$ coincides with its limit in the mathematical analysis sense, namely 1. Note that $([0, 1), \leq)$ does not admit LUBs for all its subsets, e.g., $\{n/(n+1) \mid n \in Nat\}$ does not have a LUB. One might think that this happens because $([0, 1), \leq)$ does not even admit upper bounds for all its subsets. If we add the interval $(4, 5]$ to the set above, for example, then the resulting (total) order admits upper bounds for any of its subsets (e.g., 5 is such an upper bound) but the set $\{n/(n+1) \mid n \in Nat\}$ still does not have a LUB. However, if we add the interval $[4, 5]$ to the original interval $[0, 1)$ then the resulting (total) order admits LUBs for all its subsets; for example, the LUB of $\{n/(n+1) \mid n \in Nat\}$ is 4.

2.6.2 Complete Partial Orders

In the decimal representation of real numbers, completeness means that any infinite string of decimal digits is actually the decimal representation for some real number. In mathematical analysis in general, a set of real numbers is complete iff the limit of any converging sequence of real numbers is also in the set. For example, the interval $[0, 1)$ open in 1 is not complete, because the sequence $1 - 1/n$ for $n > 0$ converges to 1 but 1 is not in the set. On the other hand, the interval $[0, 1]$ is complete. Many particular variants of completeness that appear in the literature are in fact instances a general notion or completeness for partial orders.

Definition 16. *Given a poset $(D, \sqsubseteq)$, a **chain** in D is an infinite sequence $d_0 \sqsubseteq d_1 \sqsubseteq d_2 \sqsubseteq \cdots \sqsubseteq d_n \sqsubseteq \ldots$ of elements in D, also written using the set notation as $\{d_n \mid n \in Nat\}$. Such a chain is called **stationary** when there is some $n \in Nat$ such that $d_m = d_{m+1}$ for all $m \geq n$.*

If D is finite then any chain is stationary. More generally, if for a given $x \in D$ there are only finitely many elements $y \in D$ such that $x \sqsubseteq y$, then any chain containing x is stationary.

Definition 17. *A poset $(D, \sqsubseteq)$ is called a **complete partial order (CPO)** iff any of its chains has a LUB. $(D, \sqsubseteq)$ is said to **have bottom** iff it has a minimal element. Such an element is typically written $\perp$, and the poset*

with bottom $\perp$ is written $(D, \sqsubseteq, \perp)$. CPOs with bottom are also called ***bottomed complete partial orders****. If $\{d_n \mid n \in Nat\}$ is a chain in $(D, \sqsubseteq)$, then we also let $\bigsqcup_{n \in Nat} d_n$, or simply $\sqcup d_n$, denote its LUB, $\sqcup\{d_n \mid n \in Nat\}$.*

Let us follow up on the examples underneath Definition 14 and discuss which of them are CPOs with botom and which are not:

- $(\mathcal{P}(S), \subseteq, \emptyset)$ is a bottomed CPO.
- $(Nat, \leq)$ has bottom 0 but is not complete: the chain $0 \leq 1 \leq 2 \leq \cdots \leq n \leq \cdots$ has no upper bound.
- $(Nat, \geq)$ is a CPO but has no bottom.
- $(Nat \cup \{\infty\}, \leq, 0)$ is a CPO with bottom 0: it is a CPO because any chain is either stationary, in which case its LUB is obvious, or is unbounded by any natural number, in which case ∞ is its LUB.
- $(Int, \leq)$ is not a CPO and has no bottom.
- $(S, =)$ is always a CPO, and it has bottom if and only if S has only one element.
- $(S \cup \{\perp\}, \leq^S_\perp, \perp)$ is a bottomed CPO. It is common to use the same notation $S_\perp$, used for the partial order $(S \cup \{\perp\}, \leq^S_\perp)$, to also denote its corresponding bottomed CPO $(S \cup \{\perp\}, \leq^S_\perp, \perp)$. It is interesting and particularly important in denotational semantics to note that the primitive domain $S_\perp$ is equivalent to the set of partial functions $\{*\} \rightharpoonup S$, where $\{*\}$ is the singleton set.
- $(A \rightharpoonup B, \preceq, \perp)$, the set of partial functions $A \rightharpoonup B$ ordered by the informativeness relation $\preceq$, is a CPO with bottom $\perp : A \rightharpoonup B$, the partial function which is undefined in each element of A.
- If $\{(S_i, \leq_i, \perp)\}_{i \in I}$ is a family of bottomed CPOs with same bottom $\perp$ such that $S_i \cap S_j = \{\perp\}$ for any $i \neq j \in I$, then $\bigcup_{i \in I}(S_i, \leq_i, \perp)$ is also a bottomed CPO.
- If $\{(S_i, \leq_i)\}_{i \in I}$ is a family of bottomed CPOs then $\prod_{i \in I}(S_i, \leq_i)$ is also a bottomed CPO.

All the CPOs of interest discussed in this book have a bottom element and, with a few minor exceptions in the next section, all the results that we prove for CPOs require bottom elements. Consequently, unless otherwise explicitly stated, we assume that all CPOs are bottomed CPOs.

2.6.3 Monotone and Continuous Functions

The same way various notions of completeness encountered in the literature are instances of the general notion of complete partial orders in Section 2.6.2, various particular notions of monotone and of continuous functions are instances of homonymous notions defined generally for partial orders and CPOs.

Definition 18. *If $(D, \sqsubseteq)$ and $(D', \sqsubseteq')$ are two posets and $\mathcal{F} : D \to D'$ is a function, then $\mathcal{F}$ is called* ***monotone*** *if and only if $\mathcal{F}(x) \sqsubseteq' \mathcal{F}(y)$ for any $x, y \in D$ with $x \sqsubseteq y$. If $\mathcal{F}$ is monotone, then we simply write $\mathcal{F} : (D, \sqsubseteq) \to (D', \sqsubseteq')$. Let $Mon((D, \sqsubseteq), (D', \sqsubseteq'))$ denote the set of monotone functions from $(D, \sqsubseteq)$ to $(D', \sqsubseteq')$.*

Monotone functions *preserve chains*, that is, $\{\mathcal{F}(d_n) \mid n \in Nat\}$ is a chain in $(D', \sqsubseteq')$ whenever $\{d_n \mid n \in Nat\}$ is a chain in $(D, \sqsubseteq)$. Moreover, if $(D, \sqsubseteq)$ and $(D', \sqsubseteq')$ are CPOs then for any chain $\{d_n \mid n \in Nat\}$ in $(D, \sqsubseteq)$, we have $\sqcup\mathcal{F}(d_n) \sqsubseteq' \mathcal{F}(\sqcup d_n)$. Indeed, since $\mathcal{F}$ is monotone and since $d_n \sqsubseteq \sqcup d_n$ for each $n \in Nat$, it follows

that $\mathcal{F}(d_n) \sqsubseteq' \mathcal{F}(\sqcup d_n)$ for each $n \in Nat$. Therefore, $\mathcal{F}(\sqcup d_n)$ is an upper bound for the chain $\{\mathcal{F}(d_n) \mid n \in Nat\}$. The rest follows because $\sqcup\mathcal{F}(d_n)$ is the LUB of $\{\mathcal{F}(d_n) \mid n \in Nat\}$.

Note that $\mathcal{F}(\sqcup d_n) \sqsubseteq' \sqcup\mathcal{F}(d_n)$ does not hold in general. Let, for example, $(D, \sqsubseteq)$ be $(Nat \cup \{\infty\}, \leq)$, $(D', \sqsubseteq')$ be $(\{0, \infty\}, 0 \leq \infty)$, and $\mathcal{F}$ be the monotone function taking any natural number to 0 and ∞ to ∞. For the chain $\{n \mid n \in Nat\}$, note that $\sqcup n = \infty$, so $\mathcal{F}(\sqcup n) = \infty$. On the other hand, the chain $\{\mathcal{F}(n) \mid n \in Nat\}$ is stationary in 0, so $\sqcup\mathcal{F}(n) = 0$. Therefore, $\mathcal{F}(\sqcup n) = \infty \not\leq 0 = \sqcup\mathcal{F}(n)$.

Recall that one can think of the LUB of a chain as the limit of that chain. The following definition is inspired from the analogous notion of continuous function in mathematical analysis, which is characterized by the property of preserving limits:

Definition 19. *Let $(D, \sqsubseteq)$ and $(D', \sqsubseteq')$ be CPOs. A monotone function $\mathcal{F} : (D, \sqsubseteq) \to (D', \sqsubseteq')$ is* ***continuous*** *iff $\mathcal{F}(\sqcup d_n) \sqsubseteq' \sqcup\mathcal{F}(d_n)$, which is equivalent to $\sqcup\mathcal{F}(d_n) = \mathcal{F}(\sqcup d_n)$, for any chain $\{d_n \mid n \in Nat\}$ in $(D, \sqsubseteq)$. Let $Cont((D, \sqsubseteq), (D', \sqsubseteq'))$ denote the set of continuous functions from $(D, \sqsubseteq)$ to $(D', \sqsubseteq')$. This notation extends seamlessly to bottomed CPO: $Cont((D, \sqsubseteq, \bot), (D', \sqsubseteq', \bot'))$.*

If we let $S_\bot$ denote the bottomed CPO extension of the set S as discussed in Section 2.6.2, then it can be shown that there is a bijective correspondence between the set of partial functions in $A \rightharpoonup B$ and the set of continuous (total) functions in $Cont(A_\bot, B_\bot)$ (see Exercises 37 and 38).

Note that continuous functions in $Cont((D, \sqsubseteq, \bot), (D', \sqsubseteq', \bot'))$ need not take $\bot$ to $\bot'$ in general. In fact, as seen shortly in Section 2.6.4, the interesting continuous functions do not have that property.

Proposition 12. *$Cont((D, \sqsubseteq, \bot), (D', \sqsubseteq', \bot'))$ can be endowed with a bottomed CPO structure, where*

- *Its partial order is defined as $f \preceq g$ iff $f(d) \leq g(d)$ for all $d \in D$;*
- *Its bottom element is the continuous function taking each element $d \in D$ to $\bot'$.*

Proof. ... □

Recall from Section 2.6.2 that for any sets A and B, the set of partial functions $A \rightharpoonup B$ can be organized as a bottomed CPO, $(A \rightharpoonup B, \preceq, \bot)$. Moreover, it can be shown (see Exercise 39) that $(A \rightharpoonup B, \preceq, \bot)$ and $(Cont(A_\bot, B_\bot), \preceq, \bot)$ are in fact isomorphic bottomed CPOs. This way, Proposition 12 allows us to associate a bottomed CPO to any higher-order type. For example, the type `(int -> int) -> (int -> bool)` of functions taking functions from *Int* to *Int* into functions from *Int* to *Bool* can be associated the bottomed CPO $Cont(Cont(Int_\bot, Int_\bot), Cont(Int_\bot, Bool_\bot))$. This uniform view of types as CPOs will allows us in Section 4.7 to use the fixed-point theorem discussed next to elegantly give denotational semantics to higher-order languages with recursive functions of arbitrary types.

Moreover, we can organize all the CPOs above together as one big CPO. Indeed, given any family of bottomed CPOs $\{B_i\}_{i \in I}$ (thought of as CPOs corresponding to some arbitrary set of basic types), let $\mathcal{HO}$ (from higher-order model) be the smallest set closed under the following:

- $B_i \in \mathcal{HO}$ for any $i \in I$;
- If $X, Y \in \mathcal{HO}$ then $Cont(X, Y) \in \mathcal{HO}$.

In Section 2.7 we will see that $\mathcal{HO}$ can be regarded as a cartesian-closed-category. Now let us additionally assume that all CPOs B_i have the same bottom element $\bot$ and that $B_i \cap B_j = \{\bot\}$ for any $i, j \in I$. Also, assume that we use the same $\bot$ element as bottom in all the CPOs $Cont(X, Y) \in \mathcal{HO}$. Since the union of CPOs whose pairwise intersection is their common bottom element $\{\bot\}$ is also a CPO with bottom $\{\bot\}$ (Section 2.6.2), we conclude that the union of all the CPOs in $\mathcal{HO}$ is also a CPO with bottom $\{\bot\}$.

2.6.4 The Fixed-Point Theorem

Any monotone function $\mathcal{F} : (D, \sqsubseteq, \perp) \to (D, \sqsubseteq, \perp)$ defined on a bottomed CPO to itself admits an implicit and important chain, namely $\perp \sqsubseteq \mathcal{F}(\perp) \sqsubseteq \mathcal{F}^2(\perp) \sqsubseteq \cdots \sqsubseteq \mathcal{F}^n(\perp) \sqsubseteq \cdots$, where $\mathcal{F}^n$ denotes n compositions of $\mathcal{F}$ with itself. The next theorem is a key result, which has major implications in many areas of mathematics and computer science, in particular in denotational semantics (Section 3.4):

Theorem 12. ***(The fixed-point theorem)*** *Let $(D, \sqsubseteq, \perp)$ be a bottomed CPO, let $\mathcal{F} : (D, \sqsubseteq, \perp) \to (D, \sqsubseteq, \perp)$ be a continuous function, and let $fix(\mathcal{F})$ be the LUB of the chain $\{\mathcal{F}^n(\perp) \mid n \in Nat\}$. Then $fix(\mathcal{F})$ is the least fix-point of $\mathcal{F}$.*

Proof. We first show that $fix(\mathcal{F})$ is a fix-point of $\mathcal{F}$:

$$\begin{array}{rcl} \mathcal{F}(fix(\mathcal{F})) & = & \mathcal{F}(\bigsqcup_{n \in Nat} \mathcal{F}^n(\perp)) \\ & = & \bigsqcup_{n \in Nat} \mathcal{F}^{n+1}(\perp) \\ & = & \bigsqcup_{n \in Nat} \mathcal{F}^n(\perp) \\ & = & fix(\mathcal{F}). \end{array}$$

Next we show that $fix(\mathcal{F})$ is the least fix-point of $\mathcal{F}$. Let d be another fix-point of $\mathcal{F}$, that is, $\mathcal{F}(d) = d$. We can show by induction that $\mathcal{F}^n(\perp) \sqsubseteq d$ for any $n \in Nat$: first note that $\mathcal{F}^0(\perp) = \perp \sqsubseteq d$; assume $\mathcal{F}^n(\perp) \sqsubseteq d$ for some $n \in Nat$; since $\mathcal{F}$ is monotone, it follows that $\mathcal{F}(\mathcal{F}^n(\perp)) \sqsubseteq \mathcal{F}(d) = d$, that is, $\mathcal{F}^{n+1}(\perp) \sqsubseteq d$. Thus d is an upper bound of the chain $\{\mathcal{F}^n(\perp) \mid n \in Nat\}$, so $fix(\mathcal{F}) \sqsubseteq d$. □

Many recursive definitions in mathematics and computer science are given informally, but they are more subtle than they appear to be. The fixed-point theorem can be used to formally argue that such definitions are indeed correct. Consider, for example, the following common definition of the factorial:

$$f(n) = \begin{cases} 1 & \text{if } \ n = 0 \\ n * f(n-1) & \text{if } \ n > 0 \end{cases}$$

How can we know whether such a mathematical object, i.e., a function f satisfying the above property, actually exists and is unique, as tacitly assumed? To see that such a concern is justified, replace "$n * f(n-1)$ if $n > 0$" by "$n * f(n-1)$ if $n > 1$" in the above; now there are infinitely many functions satisfying the above property. Or replace $n * f(n-1)$ by $n * f(n+1)$; now there is no function with the property above. According to the fixed-point theorem, since the function $\mathcal{F}$ defined on the set of partial functions $Nat \rightharpoonup Nat$ to itself as

$$\mathcal{F}(g)(n) = \begin{cases} 1 & \text{if } \ n = 0 \\ n * g(n-1) & \text{if } \ n > 0 \text{ and } g(n-1) \text{ defined} \\ \text{undefined} & \text{if } \ n > 0 \text{ and } g(n-1) \text{ undefined} \end{cases}$$

is continuous, it has a least fixed point. We thus can take $f = fix(\mathcal{F})$, and get

$$f(n) = \mathcal{F}(f)(n) = \begin{cases} 1 & \text{if } \ n = 0 \\ n * f(n-1) & \text{if } \ n > 0 \text{ and } f(n-1) \text{ defined} \\ \text{undefined} & \text{if } \ n > 0 \text{ and } f(n-1) \text{ undefined} \end{cases}$$

One can easily show (by induction) that f is defined everywhere, so the above is equivalent to the original definition of f. Since f is total, it is the *unique* fixed point of $\mathcal{F}$: any fixed-point f' of $\mathcal{F}$ obeys $f \preceq f'$, so f' is also defined everywhere and equal to f. See also Exercise 40.

Another application of the fixed-point theorem is to show why and how recursively defined languages admit unique solution. Any context-free language over a possibly infinite alphabet, or set of terminals, is the least fixed point of some continuous operator on the power set of the set of words over the given alphabet. Let for instance the alphabet be $A = Var \cup Int \cup \{+, -, *\}$, where *Int* is the set of integers and *Var* is a set of variables, and consider the following context-free grammar giving the syntax for arithmetic expressions:

$$Exp \ ::= \ Var \ | \ Int \ | \ Exp + Exp \ | \ -Exp \ | \ Exp * Exp$$

There are many ways to describe or construct the language of arithmetic expressions corresponding to the grammar above, which we do not discuss here. What we want to highlight is that it can also be described as the least fixed point of the following continuous function, where A^* is the set of finite words with letters in A:

$\mathcal{F} : (\mathcal{P}(A^*), \subseteq, \emptyset) \to (\mathcal{P}(A^*), \subseteq, \emptyset)$, where

$$\mathcal{F}(L) \ = \ Var \ \cup \ Int \ \cup \ L\{+\}L \ \cup \ \{-\}L \ \cup \ L\{*\}L.$$

We used the notation $L_1L_2 = \{w_1w_2 \mid w_1 \in L_1, w_2 \in L_2\}$. Notice that the iterations $\mathcal{F}(\emptyset)$, $\mathcal{F}^2(\emptyset)$, ... correspond to the one-step, two-steps, ... derivations applying the grammar's productions. See Exercise 41.

We shall later see that the fixed-point theorem above can also be used to give denotational semantics both to iterative (in Section 3.4) and to recursive (in Section 4.7.3) constructs. Fixed-points in general and the theorem above in particular have many applications in both computer science and mathematics.

2.6.5 Equational Framework for Fixed-Points

As seen above, fixed-points can be complex mathematical objects, such as functions or infinite sets of words, which may require complex means to define, understand or visualize. In many cases the only option is to *experiment* with the fixed-point, such as to evaluate it on a given input when it is a function, or to check whether it contains a certain element when it is a set. If one wants to formally reason about or to compute fixed points, then one needs to formally support, explicitly or implicitly, the body of mathematics involved in the definition of the function for which the fixed point is desired. For example, the factorial above requires support for the domain of integers, at least with multiplications and subtraction, as well as support for the domain $Int \rightharpoonup Int$ of partial functions from integers to integers (in order to define $\mathcal{F}$). Similarly, the CFG language above requires support for sets with union and possibly other operations on them, and for the domain of finite words over a given alphabet, with at least a concatenation operation on them, as well as a combination of the two, where concatenation is extended to sets of words. Even though in theory we assume the entire arsenal of mathematics, in practice we only have a limited fragment of mathematics available for formal definitions of fixed points. In this section we define such a limited framework, but one which is quite common and particularly useful for the application of fixed-points in denotational semantics (Section 3.4).

Figure 2.12 shows a simple equational theory of a minimal framework for formally defining fixed-points. We call it minimal because it only includes support for defining and applying functions, together with a simple extension for defining cases. One will need to add further extensions to this framework in order to support definitions of more complex fixed-points. For defining cases, we assumed that Booleans are already defined. For convenience, from here on we also assume that integers are already defined, together with all the desired operations on both Booleans and integers. To make it clear from which mathematical domain each operation or relational symbol comes, we add the main domain sort as a subscript to the operator or relational symbol. For example, $+_{Int}$ is the addition on integers, $\leq_{Int}$ is their comparison which evaluates to a Boolean, and $=_{Int}$ is their equality test. The functions, their application and their fixed-points have been defined like in untyped call-by-value λ-calculus (see Section 4.5) extended with the recursion μ construct (see Section 4.7),

$$
\begin{array}{lcl}
\textbf{sorts:} & & \\
\quad \mathit{Var}_{CPO},\ \mathit{CPO} & & \\
\textbf{subsorts:} & & \\
\quad \mathit{Var}_{CPO},\ \mathit{Bool} & < & \mathit{CPO} \\
\textbf{operations:} & & \\
\quad \texttt{fun}_{CPO}\ _\ \texttt{->}\ _ & : & \mathit{Var}_{CPO} \times \mathit{CPO} \to \mathit{CPO} \\
\quad \texttt{app}_{CPO}(_,_) & : & \mathit{CPO} \times \mathit{CPO} \to \mathit{CPO} \\
\quad \texttt{fix}_{CPO}\ _ & : & \mathit{CPO} \to \mathit{CPO} \\
\quad \texttt{if}_{CPO}(_,_,_) & : & \mathit{Bool} \times \mathit{CPO} \times \mathit{CPO} \to \mathit{CPO} \\
\textbf{equations:} & & \\
\quad \texttt{app}_{CPO}(\texttt{fun}_{CPO}\ V\ \texttt{->}\ C, C') & = & C[C'/V] \\
\quad \texttt{fix}_{CPO}\ \texttt{fun}_{CPO}\ V\ \texttt{->}\ C & = & C[(\texttt{fix}_{CPO}\ \texttt{fun}_{CPO}\ V\ \texttt{->}\ C)/V] \\
\quad \texttt{if}_{CPO}(\texttt{true}, C, C') & = & C \\
\quad \texttt{if}_{CPO}(\texttt{false}, C, C') & = & C'
\end{array}
$$

Figure 2.12: Equational framework for CPOs and fixed-points (assuming Booleans and substitution).

using an appropriate notion of substitution (see Section 4.5.3). How these are defined is not important here, so we refer the interested reader to the above-mentioned sections for details. What is important here is that the equational theory in Figure 2.12 gives us a starting point for formally defining fixed-points.

For simplicity, we considered only one major syntactic category in the equational theory in Figure 2.12, namely *CPO*, for complete partial orders. Basic CPOs other than that of Booleans $\mathit{Bool}_\perp$, e.g., the CPO of integers $\mathit{Int}_\perp$, need to be subsorted to *CPO* if one wants to include them. We concentrate on functional domains here, that is, on CPOs whose elements are partial functions, with $\perp$ the partial function which is undefined everywhere. We followed our naming convention and thus we tagged all the newly introduced operations with *CPO*, to distinguish them from possibly homonymous operations from included domains. We implicitly assume that the various domains subsorted to *CPO* are responsible for generating undefinedness (e.g., when a division by zero in the domain of integers is attempted) and appropriately propagating it. The four equations in Figure 2.12 are straightforward, capturing the meaning of the four language constructs in terms of substitution; here we assume substitution given, as explained above.

We can now use the constructs in Figure 2.12 to define fixed-points. For example, the factorial function discussed above can be defined as the term below of sort *CPO*, say `factorial`:

$$\texttt{fix}_{CPO}\ \texttt{fun}_{CPO}\ g\ \texttt{->}\ \texttt{fun}_{CPO}\ n\ \texttt{->}\ \texttt{if}_{CPO}(n =_{Int} 0, 1, \texttt{if}_{CPO}(n >_{Int} 0, n *_{Int} \texttt{app}_{CPO}(g, n -_{Int} 1), \perp))$$

Using the provided equations, one can prove, for example, that $\texttt{factorial}(3) = 6$. The only difference between our definition of factorial above and the fixed-point definition right after Theorem 12 is that we assumed the common mathematical convention that undefinedness is propagated implicitly through the various domain operations, so we did not propagate it explicitly. See Exercise 42 for another example.

The equational theory in Figure 2.12 can be easily defined in rewrite logic systems like Maude, provided that a generic substitution is available. For example, Appendix A.1 shows one way to do it, together with several extensions not discussed here. Once such a Maude definition of the mathematical domain is available, we can use it to calculate and experiment with fixed-points, in particular to execute recursive functions like the factorial above and, more importantly, later (Section 3.4) to execute denotational semantics of programming languages, thus obtaining interpreters for the defined languages directly from their formal semantics.

2.6.6 Exercises

Exercise 37. *Recall from Section 2.6.2 that given a set S, we let $S_\perp$ denote the poset $(S \cup \{\perp\}, \leq^S_\perp)$ with $a \leq^S_\perp b$ if and only if $a = b$ or $a = \perp$, for any $a, b \in S \cup \{\perp\}$. Let A and B be two arbitrary sets.*

1. *Characterize the monotone functions $\mathcal{F} \in Mon(A_\perp, B_\perp)$ with the property that $\mathcal{F}(\perp) \neq \perp$;*
2. *Are there any monotone functions $\mathcal{F} \in Mon(A_\perp, B_\perp)$ such that $\mathcal{F}(a) = \perp$ for some $\perp \neq a \in A$?*
3. *For this item only, suppose that A and B are finite:*
   (a) *How many partial functions are there in $A \rightharpoonup B$?*
   (b) *How many (not necessarily monotone) total functions are there in $A_\perp \to B_\perp$?*
   (c) *How many monotone functions are there in $Mon(A_\perp, B_\perp)$?*
4. *Show that there is a bijective correspondence between partial functions in $A \rightharpoonup B$ and monotone functions in $Mon(A_\perp, B_\perp)$. Give an example of a total function in $A_\perp \to B_\perp$ which does not correspond to a partial function in $A \rightharpoonup B$.*

Exercise 38. *(See also Exercise 37). For any sets A and B, $Mon(A_\perp, B_\perp) = Cont(A_\perp, B_\perp)$ and, moreover, this set of total functions bijectively corresponds to the set of partial functions $A \rightharpoonup B$.*

Exercise 39. *For any sets A and B, $(Cont(A_\perp, B_\perp), \leq, \perp)$ and $(A \rightharpoonup B, \leq, \perp)$ are isomorphic bottomed CPOs.*

Exercise 40. *Prove that the function $\mathcal{F}$ associated to the factorial function discussed right after Theorem 12 satisfies the hypothesis of Theorem 12, in particular that it is continuous. What if we replace "$n * g(n-1)$ if $n > 0$" by "$n * g(n-1)$ if $n > 1$", or $n * g(n-1)$ by $n * g(n+1)$?*

Exercise 41. *Prove that the function $\mathcal{F}$ associated to the context-free grammar discussed after Theorem 12 satisfies the hypothesis of Theorem 12, in particular that it is continuous. Describe a general procedure to associate such a function to any context-free grammar, so that its least fixed point is precisely the language of the grammar.*

Exercise 42. *Define the famous Ackermann function*

$$a(m,n) = \begin{cases} n+1 & \text{if } m = 0 \\ a(m-1,1) & \text{if } m > 0 \text{ and } n = 0 \\ a(m-1, a(m, n-1)) & \text{if } m > 0 \text{ and } n > 0 \end{cases}$$

as a fixed-point and then represent it in the formal domain language in Figure 2.12 in two different ways: first as a (curried) function taking one argument and producing a function taking another argument and then producing an integer results, and second as a function taking a pair argument. For the latter, extend the equational theory in Figure 2.12 with a pair construct and corresponding projection operations.

2.6.7 Notes

Complete partial orders play a central role in theoretical computer science and especially in denotational semantics (Section 3.4) and in domain theory (started with the seminal paper by Dana Scott [71]). Some of the original work in these fields was done using *complete latices* (i.e., partial orders in which all subsets have both a supremum and an infimum), which also admit fixed-point theorems. Nevertheless, CPOs are generally

considered to have better properties than complete latices, particularly in the context of hight-order languages, so these days both denotational semantics and domain theory build upon CPOs instead of complete latices.

Complete partial orders are sometimes called *directed-complete partial orders*, or ω-complete partial orders, and abbreviated DCPOs or ω-CPOs. Also, the notion of continuity as we used in this section is sometimes called *Scott continuity*, named after Dana Scott. There are many fixed-point theorems in mathematics and in computer science; the variant we used in this section is typically attributed to Bronisław Knaster and Alfred Tarski, and is called the Knaster-Tarski fixed-point theorem. At the time this book was being written, there was a conference totally dedicated to fixed points, called the *International Conference on Fixed Point Theory and Its Applications*.

The construction underlying $\mathcal{HO}$ should be made into or at least should relate to a "model" of the equational theory in Figure 2.12. That does not seem to be easy to achieve, though. What we may be able to do instead is to define a morphism from the initial model of the equational theory into BCPO, then show that equational reasoning is sound through this morphism. This will give us the right to use ordinary rewriting to compute fixed points and hereby to execute denotational semantics.

2.7 Category Theory

Category theory appeared for at least two important reasons:

1. to capture general concepts and principles from various particular instances of mathematics; and
2. to eliminate some of the foundational concerns, especially those related to set theory.

While the usefulness of category theory is still debatable in mathematics, there is no doubt that category theory is a very powerful language to express and handle complex computer science concepts. In what follows, we shall use category theory as a means to define some very intuitive models of simply-typed λ-calculus. But first, let us introduce some basic notions of category theory.

Category

A *category* C consists of:

- A *class of objects* written $|C|$, or $Ob(C)$. It is called a "class" to reflect the fact that it does not need to obey the constraints of set theory; one can think of a class as something "potentially larger than a set";
- A *set of morphisms*, or *arrows*, for any two objects $A, B \in |C|$, written $C(A, B)$. The fact that $f \in C(A, B)$ is often expressed using the more familiar notation $f : A \to B$. The object A is called the *domain* of f, or its *source*, and B is called the *codomain*, or the *target* of f;
- A special *identity morphism* $1_A : A \to A$ for any object $A \in |C|$;
- A *composition operator* $_;_ : C(A, B) \times C(B, C) \to C(A, C)$ for any (not necessarily distinct) objects $A, B, C \in |C|$, with the following properties:
  - *(identity)* $1_A; f = f; 1_B = f$ for any $A, B \in |C|$ and any $f : A \to B$, and
  - *(associativity)* $(f; g); h = (f; g); h$ for any $f : A \to B$, $g : B \to C$, and $h : C \to D$.

Categories are everywhere. For example:

- ***Set*** is the category whose objects are sets and whose morphisms are the usual functions;
- ***Set***^{inj} is the category whose objects are sets and whose morphisms are the injective functions;
- ***Set***^{surj} is the category whose objects are sets and whose morphisms are the surjective functions.

Exercise 43. *Prove that the above are indeed categories.*

Note that it may be the case that there are no morphisms between some given objects. For example, there is no morphism in ***Set***$^{inj}(\{1, 2\}, \{1\})$. Let us discuss several other categories:

- ***Mon*** is the category of monoids, i.e., structures $(M, _\cdot_ : M \times M \to M, e \in M)$ with $_\cdot_$ associative and identity e, with structure preserving functions, i.e., functions $f : M \to N$ such that $f(a \cdot_M b) = f(a) \cdot_N f(b)$ and $f(e_M) = e_N$, as morphisms;
- ***Grp*** is the category of groups and morphisms of groups;
- ***Poset*** is the category of partially ordered sets and monotone functions between them;

- ***Real***$^{\leq}$ is the category whose objects are the real numbers and whose morphisms are given by the $\leq$" relation: $p \to q$ iff $p \leq q$.

Exercise 44. *Show that the above are categories.*

Let us continue the discussion on categorical concepts. Given morphisms $f : A \to B$ and $g : B \to A$, one can obtain the morphisms $f; g : A \to A$ and $g; f : B \to A$. If these morphisms are the identities on A and B, respectively, the morphisms f and g are called *isomorphisms* and the objects A and B are said to be *isomorphic*. The notation $A \simeq B$ is often used to denote that A and B are isomorphic objects.

Diagrams A *diagram* in a category C is a directed graph whose nodes are objects and whose arrows are morphisms in C. Formally, a diagram consists of a pair of mappings $d : Nodes \to |C|$ and $d : Arrows \to C$, written compactly $d : (Nodes, Arrows) \to C$, where $(Nodes, Arrows)$ is some (not necessarily finite) labeled digraph (i.e., directed, or oriented, graph), such that for any $\alpha : i \to j$ in $Arrows$, $d(\alpha)$ is a morphism $d(i) \to d(j)$ in C.

To simplify writing, we draw diagrams directly as digraphs and do not specify the mapping explicitly. For example, the following nine figures represent are diagrams:

$\begin{tikzcd} A \arrow[rr, "f"] \arrow[dr, "h"'] & & B \arrow[dl, "g"] \\ & C & \end{tikzcd}$	$\begin{tikzcd} A \arrow[r, "f"] & B \end{tikzcd}$	$\begin{tikzcd} A \arrow[r, "f"] \arrow[d, "u"'] & B \arrow[d, "g"] \\ C \arrow[r, "v"'] & D \end{tikzcd}$
$\begin{tikzcd} & B \\ A \arrow[ur, "f"] \arrow[dr, "g"'] & \\ & C \end{tikzcd}$	$\begin{tikzcd} A \arrow[r, bend left, "f"] \arrow[r, bend right, "g"'] & B \end{tikzcd}$	$\begin{tikzcd} A \\ B \end{tikzcd}$
$\begin{tikzcd} & A \arrow[dl] \arrow[d] \arrow[dr] & \\ A_1 & A_2 \ \ldots \ A_n & \ldots \end{tikzcd}$	$\begin{tikzcd} \ldots \arrow[r] & A_n \arrow[r] & \ldots \arrow[r] & A_1 \end{tikzcd}$	$\begin{tikzcd} & A_1 \arrow[dr, "g_1"] & \\ A \arrow[ur, "f_1"] \arrow[r, "f_2"] \arrow[dr, "f_n"'] & A_2 \arrow[r, "g_2"] & A' \\ & A_n \arrow[ur, "g_n"'] & \end{tikzcd}$

A diagram is said to *commute*, or is a *commutative diagram*, iff any two paths between any two nodes correspond to equal morphisms, where path concatenation is interpreted as morphism composition. For example, the top-left diagram commutes iff $f; g = h$; the top-right diagram commutes iff $f; g = u; v$; the middle diagram commutes iff $f = g$; the bottom-right diagram commutes iff $f_i; g_i = f_j; g_j$ for all numbers $1 \leq i, j \leq n$.

An interesting diagram in the category ***Real***$^{\leq}$ is that of positive numbers, with an arrow $p \to q$ iff $p \leq q$.

Unless explicitly stated differently, from now on we assume all the diagrams that we draw to be commutative.

Cones and Limits

Given a diagram $d : (Nodes, Arrows) \to \mathcal{C}$, a *cone* of d is a pair $(C, \{\gamma_i\}_{i \in Nodes}$, where C is an object in $|\mathcal{C}|$ and $\gamma_i : C \to d(i)$ are morphisms in $\mathcal{C}$ with the property that $\gamma_i; d(\alpha) = \gamma_j$ for any $\alpha : i \to j$ in $Arrows$:

$$\begin{tikzcd} & d(i) \arrow[dd, "d(\alpha)"] \\ C \arrow[ur, "\gamma_i"] \arrow[dr, "\gamma_i"'] & \\ & d(j) \end{tikzcd}$$

In other words, all the diagrams formed by the cone with any edge in the diagram commute. The terminology of "cone" probably comes from graphical resemblance with the 3D figure obtained when one regards the diagram as a disc and C as a point above the plane of the disc, which is connected to every point in the diagram. But, of course, there can be all types of diagrams of all kinds of different shape.

Let us next discuss some examples of cones:

- a cone of a diagram containing just one object A and no morphism is any object C together with some morphism $C \to A$ (hence there is a one-to-one correspondence between morphisms of target A and cones of A);
- a cone of a diagram containing two disconnected objects A, B is any object C together with morphisms $f : C \to A$, $g : C \to B$;
- a cone of an empty diagram is any object C (the existence of a morphism from C to any object in the diagram is vacuously fulfilled);
- a cone of a diagram consisting of just a morphism $f : A \to B$ is an object C together with a morphism $g : C \to A$ (the other morphism of the cone, say $h : C \to B$, is uniquely determined as $g; f$);
- a cone of the diagram of positive real numbers in $\boldsymbol{Real}^{\leq}$ is uniquely determined by any negative number or zero (because these numbers are smaller than or equal to any positive real number); moreover, diagrams in $\boldsymbol{Real}^{\leq}$ admitting cones are precisely those subsets of real numbers which have lower bounds.

A *limit* of a diagram $d : (Nodes, Arrows) \to \mathcal{C}$ is a "maximal" cone d. Formally, a limit of d is a cone $(L, \{\delta_i\}_{i \in Nodes})$ such that for any other cone $(C, \{\gamma_i\}_{i \in Nodes})$ of d, there is a *unique* morphism from C to L, say $h : C \to L$, such that $h; \delta_i = \gamma_i$ for all $i \in Nodes$:

$$\begin{tikzcd} & & d(i) \arrow[ddd, "d(\alpha)"] \\ & & \\ C \arrow[uurr, "\gamma_i"] \arrow[r, "\exists! h"] \arrow[drrr, "\gamma_j"'] & L \arrow[uur, "\delta_i"] \arrow[dr, "\delta_j"] & \\ & & d(j) \end{tikzcd}$$

Exercise 45. *Any two limits of a diagram are isomorphic.*

Because of this, we say that limits are taken up-to-isomorphism.

Let us next discuss some examples of limits:

- a limit of a diagram containing just one object A and no morphism is any object L that is isomorphic to A (the isomorphism is part of the limit);

- a limit of a diagram containing two disconnected objects A, B is called a *product* of A and B, and is usually written $(A \times B, \pi_A, \pi_B)$, or even more simply just $A \times B$ and the two projections π_A and π_B are understood - the product $A \times B$ has therefore the property that for any object C and morphisms $f : C \to A$ and $g : C \to B$, there is a *unique* morphism, usually written $\langle f, g\rangle : C \to A \times B$, such that $\langle f, g\rangle; \pi_A = f$ and $\langle f, g\rangle; \pi_B = g$:

$$\begin{tikzcd}
 & & A \\
C \arrow[urr, "f"] \arrow[r, "\langle f,g\rangle"] \arrow[drr, "g"'] & A \times B \arrow[ur, "\pi_A"'] \arrow[dr, "\pi_B"] & \\
 & & B
\end{tikzcd}$$

- A limit of an empty diagram is called a *final* object of the category $\mathcal{C}$, usually denoted $\star$. Recall that a cone of an empty diagram was any object in $C \in |\mathcal{C}|$. Therefore, final objects $\star$ have the property that for any object $C \in \mathcal{C}$ there is a unique morphism from C to $\star$, usually denoted by $!_C : C \to \star$;

- A limit of a diagram consisting of just a morphism $f : A \to B$ is an object isomorphic to A;

- The limit of the diagram of positive real numbers in $\boldsymbol{Real}^{\leq}$ is the number 0, together with the corresponding "less than" morphisms to any positive number. Moreover, any diagram in $\boldsymbol{Real}^{\leq}$ consisting of a bounded set of numbers admits a limit, which is the infimum of the family; if the diagram is a (countable) decreasing sequence, then this limit is precisely the limit from mathematical analysis (this is perhaps where the name "limit" comes from).

Products

Products will play an important role in our subsequent developments. Therefore, we investigate them in slightly more depth here.

Note first that in particular instances of $\mathcal{C}$, for example sets and functions, products are nothing but the usual cartesian products, consisting of pairs of elements, one in the first component and one in the second. Also, the final objects are typically one-element structures.

Exercise 46. *Explain why in* ***Set****, the product of an empty set of sets is a one-element set.*

Given two morphisms $f_1 : A_1 \to B_1$ and $f_2 : A_2 \to B_2$, note that there is a *unique* morphism, written $f_1 \times f_2 : A_1 \times A_2 \to B_1 \times B_2$, such that the following diagram commutes:

$$\begin{tikzcd}
 & A_1 \arrow[rr, "f_1"] & & B_1 \\
A_1 \times A_2 \arrow[ur, "\pi_{A_1}"] \arrow[rr, "f_1 \times f_2"] \arrow[dr, "\pi_{A_2}"'] & & B_1 \times B_2 \arrow[ur, "\pi_{B_1}"'] \arrow[dr, "\pi_{B_2}"] & \\
 & A_2 \arrow[rr, "f_2"'] & & B_2
\end{tikzcd}$$

Exercise 47. *Show that $A \times B \simeq B \times A$ for any $A, B \in |\mathcal{C}|$.*

Exercise 48. *Why the morphism $f_1 \times f_2$ exists and is unique?*

Exercise 49. *Show that $A \simeq \star \times A$ for any $A \in |\mathcal{C}|$.*

Exercise 50. *Let $f : A \to B$, $g : B \to C$, and $h : B \to D$ be morphisms in a category with products. Show that $f; \langle g, h \rangle = \langle f; g, f; h \rangle$.*

Exercise 51. *Let $f_1 : A_1 \to A_2$, $f_2 : A_2 \to A_3$ and $g_1 : B_1 \to B_2$, $g_2 : B_2 \to B_3$ be morphisms in a category with products. Show that $(f_1; f_2) \times (g_1; g_2) = (f_1 \times g_1); (f_2 \times g_2)$.*

Exponentials

From now on we assume that our categories admit finite products, i.e., limits of finite diagrams of disconnected objects. In particular, the categories are assumed to have final objects.

Given two objects $B, C \in |\mathcal{C}|$, an *exponential of B and C* is an object denoted C^B together with a morphism $app^{B,C} : C^B \times B \to C$ such that for any $f : A \times B \to C$, there is a *unique* $g : A \to C^B$ such that $(g \times 1_B); app^{B,C} = f$:

$$\begin{tikzcd}
A \times B \arrow[rr, "f"] \arrow[dr, "g \times 1_B"'] & & C \\
& C^B \times B \arrow[ur, "app^{B,C}"'] &
\end{tikzcd}
\qquad
\begin{tikzcd}
& A \arrow[rr, "g"] & & C^B \\
A \times B \arrow[ur] \arrow[r, "g \times 1_B"] \arrow[dr] & C^B \times B \arrow[urr] \arrow[drr] & & \\
& B \arrow[rr, "1_B"'] & & B
\end{tikzcd}$$

Proposition 13. *If an exponential C^B of B and C exists in $\mathcal{C}$, then there is a one-to-one correspondence between the sets of morphisms $\mathcal{C}(A \times B, C)$ and $\mathcal{C}(A, C^B)$. The two components of this bijection, inverse to each other, are written:*

$$\begin{tikzcd}
\mathcal{C}(A \times B, C) \arrow[r, bend left, "curry"] & \mathcal{C}(A, C^B) \arrow[l, bend left, "uncurry"]
\end{tikzcd}$$

Proof. Let us first define the functions *curry* and *uncurry*. For any $f : A \times B \to C$, let $curry(f)$ be the unique morphism $g : A \to C^B$ given by the definition of the exponential, with the property that $(g \times 1_B); app^{B,C} = f$. Conversely, for any $g : A \to C^B$, let $uncurry(g)$ be the morphism $(g \times 1_B); app^{B,C} : A \times B \to C$. All we need to prove is that for any $f : A \times B \to C$ and $g : A \to C^B$, it is the case that $uncurry(curry(f)) = f$ and $curry(uncurry(g)) = g$. The first is equivalent to $(curry(f) \times 1_B); app^{B,C} = f$, which is immediate by the definition of *curry*, while the second follows by the unicity of g with the property that $(g \times 1_B); app^{B,C} = f$, where f is $uncurry(g)$. □

Exercise 52. *Prove that $\mathcal{C}(A, B) \simeq \mathcal{C}(\star, B^A)$ whenever the exponential of A and B exists in $\mathcal{C}$.*

A category $\mathcal{C}$ which admits finite products and exponentials for any two objects is called *cartesian closed*. For notational simplicity, a cartesian closed category is called a *CCC*.

Exercise 53. *Show that if A is an object in a CCC, then $A^* \simeq A$ and $*^A \simeq *$.*

Chapter 3

Executable Semantics Approaches

Dorel: make sure that the relationship to proof systems is well explained and referenced. make it clear that many of the semantics are formal proof systems, so they obey the general pattern of rules as explained in Section ??? (write this section as well in the preliminaries); explain also what a proof tree is there and then make the connection that such proof trees capture the idea of computation/derivation in the PL world. It may also be worth explaining, in the preliminaries, how general purpose proof systems can be captures as particular rewrite systems, and also give the theorem by Oliet and Meseguer stating the faithfulness of the representation.

Programming language designers and researchers need to understand the existing programming language semantical approaches well. A good understanding of semantics can help in at least two ways. On the one hand, it helps to understand how various language features can be formally defined and, implicitly, to better understand those language features. On the other hand, it helps to understand the limitations of the existing semantical approaches and, implicitly, to set up the ground for the development of novel and more powerful programming language semantical approaches.

possible and what not, as well as how it can be achieved

and how it can be done.

or to

By an *operational semantics* of a programming language, one typically understands a collection of rules specifying how its expressions, statements, programs, etc., are evaluated or executed. These rules say how a possible implementation of a programming language should "operate" and it is not difficult in practice to derive an implementation of (an interpreter of) a language by just following and translating its operational semantics into the target implementation language.

There is no definite agreement on how an operational semantics of a language should be given, because any description of a programming language which is rigorous enough to quickly lead to a correct implementation of the language can be considered to be a valid operational semantics.

This chapter introduces and discusses several operation semantics approaches in a uniform way, by means of a common computational logic framework, namely rewrite logic. The operational semantics approaches addressed in this chapter are: natural or big-step operational semantics, small-step structural operational semantics (SOS), modular SOS, reduction semantics with evaluation contexts, and the chemical abstract machine (CHAM). The first two are the most known and widely used language definitional approaches; modular SOS and reduction semantics with evaluation contexts have been introduced to enhance the modularity and compactness of SOS, and the CHAM to address the lack of support in the other approaches for true concurrency and distributed state. Each of these language definitional styles is shown to be *faithfully captured* as a rewrite logic theory, in the sense that there is a one-to-one correspondence between computational steps in the original language definition and computational steps in the corresponding rewrite logic theory.

This chapter has three major goals:

1. To serve as a gentle introduction to conventional operational semantics approaches;

2. To present these various operational semantics approaches in a uniform setting, thus being able to more easily compare them to each other and understand the advantages and the disadvantages of each of them; and

3. To convey the fact that rewrite logic does not force or pre-impose any given language definitional style, and that its flexibility and ease of use makes it an appealing framework for understanding and exploring existing as well as new language definitional styles.

Each language definitional style discussed in this chapter can therefore be used as a particular definitional methodology within rewrite logic, thus enabling the language designer to use his/her favorite techniques within rewrite logic with the benefit of a unified logic and generic tool support. It is important to realize that representing a language definitional style in rewrite logic does not make that style more flexible: as it will soon become clear once we start presenting the details, the technique representing it within rewrite logic inherits the same benefits and limitations that the original definitional style had.

Even though we exemplify the techniques and proofs in this chapter with a simple language for concreteness' sake, the process of representing each definitional style in rewrite logic and proving the faithfulness of the representation is completely general and mechanical.

---

merge this:

*We will also characterize the relation between the RLS representations and their corresponding definitional style counterparts, pointing out some strengths and weaknesses for each style. The reader is referred to [34, **?**, 53, 25, 9] for further details on the described operational semantics styles.*

---

following was initially included as part of the IMP *language section; it should be here, perhaps rewritten*

To give an operational semantics to a programming language, one typically needs a special structure, called a *configuration*, which comprises all the semantic ingredients needed to define how the programming language under consideration operates. For example, a configuration typically contains the program or the fragment of program that needs to be executed. In addition to code, configurations typically also contain several other pieces of information, such as the program state, memory (or heap), stacks, other resources held by each thread, etc.

In general, complex languages may require complex configurations and different languages may require different configurations. There is no absolute agreement on how to denote configurations, but it is common practice to consider them as tuples separating the various configuration components with a comma and enclosing the comma-separated list of configuration components with some special brackets, such as angle or square brackets. For example, $\langle c_1, c_2, \ldots, c_n \rangle$ denotes a configuration containing components c_1, c_2, $\ldots$, c_n.

For the simple language IMP, we only need very simple configurations. In this chapter, we follow the comma-and-angle-bracket notational convention, that is, we separate the configuration components by commas and then enclose the entire list with angle brackets. For example, $\langle a, \sigma \rangle$ is a configuration containing an arithmetic expression a and a state σ, and $\langle b, \sigma \rangle$ is a configuration containing a Boolean expression b and a state σ. Some configurations may not need a state while others may not need the code. For example, $\langle i \rangle$ is a configuration holding only the integer number i that can be obtained as a result of evaluating an arithmetic expression, while $\langle \sigma \rangle$ is a configuration holding only one state σ that can be obtained after evaluating a statement. Configurations can therefore be of different types and need not necessarily have the same number of components.

It is common practice when giving "paper" (i.e., not machine executable) definitions of languages to informally and tacitly assume configurations for any types and any number of components.

An operational semantics can be viewed as a means to establish a relationship between configurations. For example, the relation defined by a small-step semantics captures exactly one desirable execution step in the defined language, while the relation defined by a big-step semantics captures several such small steps, typically relating the corresponding program or fragment of program directly to its result after evaluation.

Once one goes beyond paper definitions of languages and wants to have actual rigorous and machine executable definitions, one needs to worry about giving formal definitions of configurations. There are many possible ways to do this. Figure FIXTHIS shows an algebraic signature defining the IMP configurations needed for the subsequent operational semantics. For simplicity, we preferred to explicitly define each type of needed configuration. Consequently, our configurations definition in Figure FIXTHIS may be more verbose than an alternative polymorphic definition, but we believe that it is clearer for this simple language. We assumed that the sorts *AExp*, *BExp*, *Stmt* and *State* come from algebraic definitions of the IMP syntax and state, like those in Sections 3.1.1 and 3.1.2; recall that the latter adapted the algebraic definition of partial functions in Section 2.4.6 (see Figure 2.7) as explained in Section 3.1.2.

In paper definitions of languages, configurations tend to be informally and tacitly assumed available for any types and any number of components. When defining language semantics formally, configurations need to also be defined rigorously.

An SOS defines a relation on *configurations*, which is typically binary and thought of as a *transition relation*. In general, a configuration is a tuple containing a program or a fragment of program, which we may call a *term over the syntax of the language*, and corresponding needed semantic infrastructure, such as a state, various control stacks, etc.; however, in our simple language definition we only need configurations consisting of pairs of a term and a state.

They both consist of defining deduction (or inference, or derivation) systems for binary relations on *configurations*, where a configuration is typically a tuple containing some program fragment together with state infrastructure necessary to evaluate it.

Big-Step SOS, also known as *natural semantics*. Under big-step SOS, the atomic provable entities are relations of configurations, typically written $C \Downarrow C'$, with the meaning that C' is the configuration obtained after the (complete) evaluation of C.

If expression evaluation is side-effect-free, then one can drop the state from the right-side configuration and thus write big-step relations as $\langle a, \sigma \rangle \Downarrow \langle i \rangle$;

A transition takes a configuration to another configuration (the "next step").

An SOS defines a relation on *configurations*, which is typically binary and thought of as a *transition relation*. In general, a configuration is a tuple containing a program or a fragment of program, which we may call a *term over the syntax of the language*, and corresponding needed semantic infrastructure, such as a state, various control stacks, etc.; however, in our simple language definition we only need configurations consisting of pairs of a term and a state.

However, the state needs to be carried as part of the configuration that appears to the left side of each sequent, because it may be needed if expressions contain variables, to look them up.

To avoid treating special cases in the subsequent small-step definitions, we prefer to mention the state in the right-side configuration even if it does not change.

---

get stuff from the I&C paper

---

say that can also learn these semantic definitional styles, in that we do not assume the reader necessarily familiar with those.

In this section we will investigate a particular but very common operational semantics approach, called *structural operational semantics* and abbreviated *SOS*, as well as two variations aiming at improving modularity and compactness of language definitions, namely *modular SOS (MSOS)* and *reduction semantics with evaluation contexts*. Language definitional frameworks or styles are typically called "structural" when their authors want to convey the intuition that programming languages are defined inductively over the structure of their syntax.

There are two common SOS definitional styles of programming languages, big-step SOS and small-step SOS. They both consist of defining deduction (or inference, or derivation) systems for binary relations on *configurations*, where a configuration is typically a tuple containing some program fragment together with state infrastructure necessary to evaluate it. Even though big-step SOS can be technically regarded as a special case of the small-step SOS where the small-step is big enough to take a program or a fragment of it directly to its final result, the two styles tend to be regarded as conceptually rather different.

- *Big-Step SOS*, also known as *natural semantics*. Under big-step SOS, the atomic provable entities are relations of configurations, typically written $C \Downarrow C'$, with the meaning that C' is the configuration obtained after the (complete) evaluation of C. A big-step SOS describes in a divide-and-conquer manner how final evaluation results of language constructs can be obtained by combining the evaluation results of their syntactic counterparts (subexpressions, substatements, etc.). For example, the big-step SOS definition of addition in a language whose expression evaluation can have side effects is

  move rule below in discussions section

$$\frac{\langle a_1, \sigma\rangle \Downarrow \langle i_1, \sigma_1\rangle,\ \langle a_2, \sigma_1\rangle \Downarrow \langle i_2, \sigma_2\rangle}{\langle a_1 + a_2, \sigma\rangle \Downarrow \langle i, \sigma_2\rangle} \text{ where } i \text{ is the sum of } i_1 \text{ and } i_2 \tag{3.1}$$

  Here, the meaning of a relation $\langle a, \sigma\rangle \Downarrow \langle i, \sigma'\rangle$ is that arithmetic expression a is evaluated in state σ to integer i and new state σ'. If expression evaluation is side-effect-free, then one can drop the state from the right-side configuration and thus write big-step relations as $\langle a, \sigma\rangle \Downarrow \langle i\rangle$; big-step SOS definitions transform as expected:

$$\frac{\langle a_1, \sigma\rangle \Downarrow \langle i_1\rangle,\ \langle a_2, \sigma\rangle \Downarrow \langle i_2\rangle}{\langle a_1 + a_2, \sigma\rangle \Downarrow \langle i\rangle} \text{ where } i \text{ is the sum of } i_1 \text{ and } i_2 \tag{3.2}$$

- *Small-Step SOS*, also known as *transitional semantics*, is how SOS was originally introduced. Under small-step SOS, the atomic provable entities are one-computation-step transitions, showing how a fragment of program is advanced *one step* in its evaluation process; a small-step SOS identifies for each language construct typically one of its syntactic counterparts which can be advanced precisely one step, and then shows how that subcomputation step translates into a one-computation step for the language construct. For example, the small-step SOS definition of addition is

$$\frac{\langle a_1, \sigma\rangle \rightarrow \langle a_1', \sigma\rangle}{\langle a_1 + a_2, \sigma\rangle \rightarrow \langle a_1' + a_2, \sigma\rangle} \tag{3.3}$$

$$\frac{\langle a_2, \sigma\rangle \rightarrow \langle a_2', \sigma\rangle}{\langle a_1 + a_2, \sigma\rangle \rightarrow \langle a_1 + a_2', \sigma\rangle} \tag{3.4}$$

$$\frac{\cdot}{\langle i_1 + i_2, \sigma\rangle \to \langle i, \sigma\rangle} \text{ where } i \text{ is the sum of } i_1 \text{ and } i_2 \tag{3.5}$$

We will be able to formally show that a step in a big-step SOS of a language corresponds to many steps in a small-step SOS of the same language.

Many recent works on SOS definitions of languages take the liberty to mix big-step and small-step. For example, one may evaluate the condition of a conditional statement in one big-step, but then transition to the left or to the right branch of the conditional in a small-step; or, more generally, one can use big-step for expressions and small-step for statements.

In our simple language definition, we only need simple *configurations*. We enclose the various components forming a configuration with angle brackets. For example, $\langle a, \sigma\rangle$ is a configuration containing an arithmetic expression a and a state σ, and $\langle b, \sigma\rangle$ is a configuration containing a Boolean expression b and a state σ. Configurations of different types need not necessarily have the same number of components. For example, since all programs evaluate in the initial state, there is no need to mention a state next to a program in a configuration; in this case, a configuration is simply a one element tuple, $\langle p\rangle$, where p is a program. We will introduce configurations tacitly by need in our SOS language definitions; what distinguishes configurations from other structures are the angle brackets.

The core ingredient of an SOS definition is the *sequent*. Like in definitions of deductive systems in general, a sequent can be almost any tuple; however, in our particular SOS definitions, a *sequent* is typically a *transition*. We use arrows for transitions, such as $\Downarrow$ or $\to$. A transition takes a configuration to another configuration. For example, in a big-step SOS, a transition that signifies that arithmetic expression a evaluates in state σ to integer i can be written $\langle a, \sigma\rangle \Downarrow \langle i\rangle$ if the evaluation of expressions is side-effect free, or $\langle a, \sigma\rangle \Downarrow \langle i, \sigma'\rangle$ if the evaluation of expressions may have side effects, where σ' is the state after the evaluation of a to i in σ. Also, in a small-step semantics, a transition $\langle s, \sigma\rangle \to \langle s', \sigma'\rangle$ signifies that statement s in state σ transitions to statement s' (typically a variant of s whose computation is advanced by one step) in state σ'.

An SOS definition is a collection of *parametric rules* of the form:

$$\frac{\mathit{transition}_1,\ \mathit{transition}_2,\ \ldots, \mathit{transition}_k}{\mathit{transition}}$$

The transitions above the line are called the *condition* of the rule, and the transition below the line is called the *conclusion* of the rule. The intuition here is that *transition* is possible whenever $\mathit{transition}_1$, $\mathit{transition}_2$, ..., $\mathit{transition}_k$ are possible. We may also say that *transition* is *derivable*, or can be *inferred*, from $\mathit{transition}_1$, $\mathit{transition}_2$, ..., $\mathit{transition}_k$. This reflects the fact that an SOS definition can (and should) be viewed as a logical system, where one can deduce possible behaviors of programs.

If $k = 0$, then we call the rule *unconditional* and simply write

$$\frac{\cdot}{\mathit{transition}}$$

---

discuss $+_{Int}$ *and other assumed builtins*

---

merge this as well:

Since K makes use only of standard, context-insensitive term rewriting modulo equations, it can be executed on rewrite engines like Maude almost *as is*. Nevertheless, the relationship between K and Maude is

like that between SOS, or any of the above-mentioned styles, and Maude (or ML, or Scheme, or Haskell, etc.): Maude can be used as a means to execute and analyze K language definitions. The interested reader is referred to [74] for a detailed discussion on how the various language definitional styles can be faithfully embedded in rewrite logic and then executed using systems like Maude, or even more conventional programming languages. The basic idea of these faithful embeddings is that a language definition in any of these styles, say $\mathcal{L}$, can be regarded as a rewrite logic theory, say $R_{\mathcal{L}}$, in such a way that there is a *one-to-one computational equivalence* between reductions using $\mathcal{L}$ and rewrites using $R_{\mathcal{L}}$. Note that this is significantly stronger than encoding, or implementing, a framework into another framework: *$R_{\mathcal{L}}$ is $\mathcal{L}$, not an encoding of it*, the only difference between the two being insignificant notational/syntactic conventions. This is totally different from encoding $\mathcal{L}$ on a Turing machine or in a λ-calculus, for example, because such encodings would not preserve the intended computational granularity of $\mathcal{L}$'s reductions (if correct, they would only preserve the relational behavior of $\mathcal{L}$: whatever is reduced with $\mathcal{L}$ can also be reduced, in any number of steps, with its encodings).

---

the essential difference between reduction semantics and rewriting semantics is that the former needs to be given permission, while the latter needs to be taken permission

---

Should we consider also unravelings, or automatic elimination of conditions? Would these make them faster?

See [84] for lots of citations that could go in the intro of Chapter 3. Vardejo also defines several languages in Maude, using several styles. E.g., a simple functional language in both big-step and small-step SOS, a simple imperative language in both big-step and small-step SOS, Kahn's Mini-ML using big-step SOS, Milner's CCS (he uses frozen attributes; are they really needed? I think we can use the $\circ C$ trick to avoid it all together),

But he uses slightly different encodings/embeddings of these styles in rewrite logic, and does not state/prove general results about embeddings; he only demonstrates them by means of concrete examples. His goal is really to implement operational semantics in Maude.

In each figure in this section, say what the sorts of variables or the syntactic categories of metavariables are, like in the figures for big-step

---

Notes

- Gordon 89?
    - Centaur? Typol?

---

say that ★ means a Maude-specific section/problem/exercise

Some ideas from Peter Mosses denotational semantics paper in the handbook. Most of these may go in the introduction of the book.

- syntax is about the structure of programs, whether they are legal or not

- syntax of most languages is defined quite formally; its meaning is typically defined informally, in words; for that reason it is often the case that it is incomplete or inconsistent, unsuitable for reasoning, and useless for generation of implementations; why is the meaning of a PL not also defined formally? well, semantic frameworks are not as obvious as the syntactic ones (CFG), they have advantages and disadvantages, they are not as easy to read and understand, one needs mathematical training, etc. also, can be painful to do it, because they are non-modular (unlike the syntax).

- semantics is about their meaning, the behavior they produce when executed on computers; formal description of PLs;

- a semantics documents the design of the language; it also gives insight about the language one is defining; if executable, then a great tool for design.

- a semantics serves as a standard implementation of the language: ensures that all implementations give the same result when running the same program

- a semantics does NOT tell how the language should be implemented! in particular, it may use additional constructs that may have no equivalent in implementations

- a semantics gives a basis for reasoning about programs

Int	::=	the domain of (unbounded) integer numbers, with usual operations on them
Bool	::=	the domain of Booleans
Id	::=	standard identifiers
AExp	::=	*Int*
	|	*Id*
	|	*AExp* + *AExp*
	|	*AExp* / *AExp*
BExp	::=	*Bool*
	|	*AExp* <= *AExp*
	|	! *BExp*
	|	*BExp* && *BExp*
Block	::=	{}
	|	{ *Stmt* }
Stmt	::=	*Block*
	|	*Id* = *AExp* ;
	|	*Stmt Stmt*
	|	`if` (*BExp*) *Block* `else` *Block*
	|	`while` (*BExp*) *Block*
Pgm	::=	`int` **List**{*Id*}; *Stmt*

Figure 3.1: Syntax of IMP, a small imperative language, using algebraic BNF.

3.1 IMP: A Simple Imperative Language

To illustrate the various semantic styles discussed in this chapter, we have chosen a small imperative language, called IMP, whose syntax is inspired from C and Java. In fact, if we wrap an IMP program in a `main(){...}` function the we get a valid C program. IMP has arithmetic expressions which include the domain of arbitrarily large integer numbers, Boolean expressions, assignment statements, conditional statements, while loop statements, and sequential composition of statements. Statements can be grouped in blocks surrounded with curly brackets, and the branches of the conditional and the loop body are required to be blocks. All variables used in an IMP program need to be declared at the beginning of the program, can only hold integer values (for simplicity, IMP has no Boolean variables), and are initialized with default value 0.

3.1.1 IMP Syntax

We here define the syntax of IMP, first using the Backus-Naur form (BNF) notation for context-free grammars and then using the alternative and completely equivalent mixfix algebraic notation (see Section 2.1.3). The latter is in general more appropriate for semantic developments of a language.

IMP Syntax as a Context-Free Grammar

Figure 3.1 shows the syntax of IMP using the algebraic BNF notation. In this book we implicitly assume parentheses as part of any syntax, without defining them explicitly. Parentheses can be freely used for grouping, to increase clarity and/or to avoid ambiguity in parsing. For example, with the syntax in Figure 3.1, `(x + 3) / y` is a well-formed IMP arithmetic expression.

The only algebraic feature in the IMP syntax in Figure 3.1 is the use of **List{***Id***}** for variable declarations (last production), which in this case is clear: one can declare a comma-separated list of variables. To stay more conventional in notation, we refrained from replacing the productions *Stmt* ::= {} | *Stmt Stmt* with the algebraic production *Stmt* ::= $\mathbf{List}^{\{\}}_{\sqcup}$**{***Stmt***}** which captures the idea of statement sequentialization more naturally. Moreover, our syntax for statement sequential composition allows ambiguous parsing. Indeed, if $s_1, s_2, s_3 \in$ *Stmt* then $s_1\ s_2\ s_3$ can be parsed either as $(s_1\ s_2)\ s_3$ or as $s_1\ (s_2\ s_3)$. However, the semantics of statement sequential composition will be such that the parsing ambiguity is irrelevant (but that may not always be the case). It may be worthwhile pointing out that one should not get tricked by thinking that different parsings mean different evaluation orders. In our case here, both $(s_1\ s_2)\ s_3$ and $s_1\ (s_2\ s_3)$ will proceed by evaluating the three statements in order. The difference between the two is that the former will first evaluate $s_1\ s_2$ and then s_3, while the latter will first evaluate s_1 and then $s_2\ s_3$; in either case, s_1, s_2 and s_3 will end up being evaluated in the same order: first s_1, then s_2, and then s_3.

The IMP language constructs have their usual imperative meaning. For diversity and demonstration purposes, when giving the various semantics of IMP we will assume that `+` is *non-deterministic* (it evaluates the two subexpressions in any order, possibly interleaving their corresponding evaluation steps), `/` is non-deterministic and *partial* (it will stuck the program when a division by zero takes place), `<=` is *left-right sequential* (it first evaluates the left subexpression and then the right subexpression), and that `&&` is left-right sequential and *short-circuited* (it first evaluates the left subexpression and then it conditionally evaluates the right only if the left evaluated to true).

One of the main reasons for which arithmetic language constructs like `+` above are allowed to be non-deterministic in language semantic definitions is because one wants to allow flexibility in how the language is implemented, and not because these operations are indeed intended to have fully non-deterministic, or random, behaviors in all implementations. In other words, their non-determinism is to a large extent an artifact of their intended *underspecification*. Some language manuals actually state explicitly that one should not rely on the order in which the arguments of language constructs are evaluated. In practice, it is considered to be programmers' responsibility to write their programs in such a way that one does not get different behaviors when the arguments are evaluated in different orders.

To better understand the existing semantic approaches and to expose some of their limitations, Section 3.5 discusses extensions of IMP with expression side effects (a variable increment operation), with abrupt termination (a halt statement), with dynamic threads and join synchronization, with local variable declarations, as well as with all of these together; the resulting language is called IMP++. The extension with side effects, in particular, makes the evaluation strategies of `+`, `<=` and `&&` semantically relevant.

Each semantical approach relies on some basic mathematical infrastructure, such as integers, Booleans, etc., because each semantic definition reduces the semantics of the language constructs to those domains. We will assume available any needed mathematical domains, as well as basic operations on them which are clearly tagged (e.g., $+_{Int}$ for the addition of integer numbers, etc.) to distinguish them from homonymous operations which are language constructs. Unless otherwise stated, we assume no implementation-specific restrictions in our mathematical domains; for example, we assume integer numbers to be arbitrarily large rather than representable on 32 bits, etc. We can think of the underlying domains used in language semantics as parameters of the semantics; indeed, changing the meaning of these domains changes the meaning of all language semantics using them. We also assume that each mathematical domain is endowed with a special element, written $\perp$ for all domains to avoid notational clutter, corresponding to *undefined* values of that domain. Some of these mathematical domains are defined in Chapter 2; appropriate references will be given when such domains are used.

sorts:

Int, $Bool$, Id, $AExp$, $BExp$, $Block$, $Stmt$, Pgm

subsorts:

$$
\begin{array}{rcl}
Int,\ Id & < & AExp \\
Bool & < & BExp \\
Block & < & Stmt
\end{array}
$$

operations:

$$
\begin{array}{rcl}
_ \texttt{+} _ & : & AExp \times AExp \to AExp \\
_ \texttt{/} _ & : & AExp \times AExp \to AExp \\
_ \texttt{<=} _ & : & AExp \times AExp \to BExp \\
\texttt{!} _ & : & BExp \to BExp \\
_ \texttt{\&\&} _ & : & BExp \times BExp \to BExp \\
\{\} & : & \to Block \\
\{ _ \} & : & Stmt \to Block \\
_ \texttt{=} _ \texttt{;} & : & Id \times AExp \to Stmt \\
_ \ _ & : & Stmt \times Stmt \to Stmt \\
\texttt{if (} _ \texttt{)} \ _ \texttt{else} \ _ & : & BExp \times Block \times Block \to Stmt \\
\texttt{while (} _ \texttt{)} \ _ & : & BExp \times Block \to Stmt \\
\texttt{int} \ _ \texttt{;} \ _ & : & \mathbf{List}\{Id\} \times Stmt \to Pgm
\end{array}
$$

Figure 3.2: Syntax of IMP as an algebraic signature.

We take the freedom to tacitly use the following naming conventions for meta or mathematical variables[1] ranging over IMP-specific terms throughout the remainder of this chapter: $x, X \in Id$; $a, A \in AExp$; $b, B \in BExp$; $s, S \in Stmt$; $i, I \in Int$; $t, T \in Bool$; $p, P \in Pgm$. Any of these can be primed or indexed.

IMP Syntax as an Algebraic Signature

Following the relationship between the CFG and the mixfix algebraic notations explained in Section 2.1.3, the BNF syntax in Figure 3.1 can be associated the entirely equivalent algebraic signature in Figure 3.2 with one (mixfix) operation per production: the terminals mixed with underscores form the name of the operation and the non-terminals give its arity. This signature is easy to define in any rewrite engine or theorem prover; moreover, it can also be defined as a data-type or corresponding structure in any programming language. We next show how it can be defined in Maude.

★ Definition of IMP Syntax in Maude

Using the Maude notation for algebraic signatures, the algebraic signature in Figure 3.2 can yield the Maude syntax module in Figure 3.3. We have additionally picked some appropriate precedences and formatting attributes for the various language syntactic constructs (see Section 2.5.6 for more details on Maude and the meaning of these attributes).

[1]Recall that we use an *italic* font for such variables, in contrast to the `typewriter` font that we use for code (including program variable identifiers, integers, operation symbols, etc.). For example, if we write $x, \mathtt{x} \in Id$ then we mean an arbitrary identifier that x refers to, and *the concrete* identifier `x`. The latter can appear in programs, while the former cannot. The former is mainly used to define semantics or state properties of the language.

```
mod IMP-SYNTAX is including PL-INT + PL-BOOL + PL-ID .
--- AExp
  sort AExp .  subsorts Int Id < AExp .
  op _+_ : AExp AExp -> AExp [prec 33 gather (E e) format (d b o d)] .
  op _/_ : AExp AExp -> AExp [prec 31 gather (E e) format (d b o d)] .
--- BExp
  sort BExp .  subsort Bool < BExp .
  op _<=_ : AExp AExp -> BExp  [prec 37 format (d b o d)] .
  op !_ : BExp -> BExp [prec 53 format (b o d)] .
  op _&&_ : BExp BExp -> BExp [prec 55 gather (E e) format (d b o d)] .
--- Block and Stmt
  sorts Block Stmt .  subsort Block < Stmt .
  op {} : -> Block [format (b b o)] .
  op {_} : Stmt -> Block [format (d n++i n--i d)] .
  op _=_; : Id AExp -> Stmt [prec 40 format (d b o b o)] .
  op __ : Stmt Stmt -> Stmt [prec 60 gather (e E) format (d ni d)] .
  op if(_)_else_ : BExp Block Block -> Stmt [prec 59 format (b so d d s nib o d)] .
  op while(_)_ : BExp Block -> Stmt [prec 59 format (b so d d s d)] .
--- Pgm
  sort Pgm .
  op int_;_ : List{Id} Stmt -> Pgm [prec 70 format (nb o d ni d)] .
endm
```

Figure 3.3: IMP syntax as an algebraic signature in Maude. This definition assumes appropriate modules `PL-INT`, `PL-BOOL` and `PL-ID` defining corresponding sorts `Int`, `Bool`, and `Id`, respectively.

The module `IMP-SYNTAX` in Figure 3.3 imports three builtin modules, namely: `PL-INT`, which we assume it provides a sort `Int`; `PL-BOOL`, which we assume provides a sort `Bool`; and `PL-ID` which we assume provides a sort `Id`. We do not give the precise definitions of these modules here, particularly because one may have many different ways to define them. In our examples from here on in the rest of this chapter we assume that `PL-INT` contains all the integer numbers as constants of sort `Int`, that `PL-BOOL` contains the constants `true` and `false` of sort `Bool`, and that `PL-ID` contains all the letters in the alphabet as constants of sort `Id`. Also, we assume that the module `PL-INT` comes equipped with as many builtin operations on integers as needed. To avoid operator name conflicts caused by Maude's operator overloading capabilities, we urge the reader to *not* use the Maude builtin `INT` and `BOOL` modules, but instead to overwrite them. Appendix A.1 shows one possible way to do this: we define new modules `PL-INT` and `PL-BOOL` "hooked" to the builtin integer and Boolean values but defining only a subset of operations on them and with clearly tagged names to avoid name overloading, e.g., `_+Int_`, `_/Int_`, etc.

Recall from Sections 2.4.6 and 2.5.6 that lists, sets, bags, and maps are trivial algebraic structures which can be easily defined in Maude; consequently, we take the freedom to use them without definition whenever needed, as we did with using the sort `List{Id}` in Figure 3.3.

To test the syntax, one can now parse various IMP programs, such as:

```
Maude> parse
   int n, s ;
   n = 100 ;
   while (!(n <= 0)) {
     s = s + n ;
     n = n + -1 ;
   }
.
```

```
mod IMP-PROGRAMS is including IMP-SYNTAX .
  ops sumPgm collatzPgm countPrimesPgm : -> Pgm .
  ops collatzStmt multiplicationStmt primalityStmt : -> Stmt .
  eq sumPgm = (
     int n, s ;
     n = 100 ;
     while (!(n <= 0)) {
       s = s + n ;
       n = n + -1 ;
     }    ) .

  eq collatzStmt = (
     while (!(n <= 1)) {
       s = s + 1 ;  q = n / 2 ;  r = q + q + 1 ;
       if (r <= n) { n = n + n + n + 1 ; } else { n = q ; }
     }     ) .

  eq collatzPgm = (
     int m, n, q, r, s ;
     m = 10 ;
     while (!(m <= 2)) {
       n = m ;
       m = m + -1 ;
       collatzStmt
     }     ) .

  eq multiplicationStmt = (        --- fast multiplication (base 2) algorithm
     z = 0 ;
     while (!(x <= 0)) {
       q = x / 2 ;
       r = q + q + 1 ;
       if (r <= x) { z = z + y ; } else {}
       x = q ;
       y = y + y ;
     }     ) .

  eq primalityStmt = (
     i = 2 ;  q = n / i ;  t = 1 ;
     while (i <= q && 1 <= t) {
       x = i ;
       y = q ;
       multiplicationStmt
       if (n <= z) { t = 0 ; } else { i = i + 1 ;  q = n / i ; }
     }     ) .

  eq countPrimesPgm = (
     int i, m, n, q, r, s, t, x, y, z ;
     m = 10 ;  n = 2 ;
     while (n <= m) {
       primalityStmt
       if (1 <= t) { s = s + 1 ; } else {}
       n = n + 1 ;
     }     ) .
endm
```

Figure 3.4: IMP programs defined in a Maude module IMP-PROGRAMS.

Now it is a good time to define a module, say `IMP-PROGRAMS`, containing as many IMP programs as one bears to write. Figure 3.4 shows such a module containing several IMP programs. Note that we took advantage of Maude's rewriting capabilities to save space and reuse some of the defined fragments of programs as "macros". The program `sumPgm` calculates the sum of numbers from 1 to 100; since we do not have subtraction in IMP, we decremented the value of `n` by adding -1.

The program `collatzPgm` in Figure 3.4 tests Collatz' conjecture for all numbers from 1 to 10, counting the total number of steps in `s`. The Collatz conjecture, still unsolved, is named after Lothar Collatz (but also known as the $3n+1$ conjecture), who first proposed it in 1937. Take any natural number n. If n is even, divide it by 2 to get $n/2$, if n is odd multiply it by 3 and add 1 to obtain $3n+1$. Repeat the process indefinitely. The conjecture claims that no matter what number you start with, you will always eventually reach 1. Paul Erdös said about the Collatz conjecture: "Mathematics is not yet ready for such problems." While we do not attempt to solve it, we can test it even in a simple language like IMP. It is a good example program to test IMP semantics because it makes use of almost all IMP's language constructs and also has nested loops. The macro `collatzStmt` detaches the check of a single `n` from the top-level loop iterating `n` through all $2 < \mathtt{n} \leq \mathtt{m}$. Note that, since we do not have multiplication and test for even numbers in IMP, we mimic them using the existing IMP constructs.

Finally, the program `countPrimesPgm` counts all the prime numbers up to `m`. It uses `primalityStmt`, which checks whether `n` is prime or not (writing `t` to 1 or to 0, respectively), and `primalityStmt` makes use of `multiplicationStmt`, which implements a fast base 2 multiplication algorithm. Defining such a module with programs helps us to test the desired language syntax (Maude will report errors if the programs that appear in the right-hand-sides of the equations are not parsable), and will also help us later on to test the various semantics that we will define.

3.1.2 IMP State

Any operational semantics of IMP needs some appropriate notion of *state*, which is expected to map program variables to integer values. Moreover, since IMP disallows uses of undeclared variables, it suffices for the state of a given program to only map the declared variables to integer values and stay undefined in the variables which were not declared.

Fortunately, all these desired IMP state operations correspond to conventional mathematical operations on *partial finite-domain functions* from variables to integers in $[Id \rightharpoonup Int]^{finite}$ (see Section 2.1.2) or, equivalently, to structures of sort $\mathbf{Map}\{Id \mapsto Int\}$ defined using equations (see Section 2.4.6 for details on the notation and the equivalence); we let *State* be an alias for the map sort above. From a semantic point of view, the equations defining such map structures are computationally invisible: semantic transitions that are part of various IMP semantics will be performed *modulo* these equations. In other words, state lookup and update operations will not count as computational steps, so they will not interfere with or undesirably modify the intended computational granularity of the defined language.

We let σ, σ', σ_1, etc., range over states. By defining IMP states as partial finite-domain functions $\sigma : Id \rightharpoonup Int$, we have a very natural notion of undefinedness for a variable that has not been declared and thus has not been initialized in a state: variable x is considered *undefined* in a state σ if and only if $x \notin Dom(\sigma)$. We may use the terminology *state lookup* for the operation $_(_) : State \times Id \rightharpoonup Int$, the terminology *state update* for the operation $_[_/_] : State \times Int \times Id \to State$, and the terminology *state initialization* for the operation $_ \mapsto _ : \mathbf{List}\{Id\} \times Int \to State$.

Recall from Section 2.1.2 that the lookup operation is itself a partial function, because the state to lookup may be undefined in the variable of interest; as usual, we let $\perp$ denote the undefined state and we write as expected $\sigma(x) = \perp$ and $\sigma(x) \neq \perp$ when the state σ is undefined and, respectively, defined in variable x. Recall

```
mod STATE is including PL-INT + PL-ID .
  sort State .

  op _|->_ : List{Id} Int -> State [prec 0] .
  op .State : -> State .
  op _&_ : State State -> State [assoc comm id: .State format(d s s d)] .

  op _(_) : State Id -> Int [prec 0] .          --- lookup
  op _[_/_] : State Int Id -> State [prec 0] .  --- update

  var Sigma : State .  var I I' : Int .  var X X' : Id .  var Xl : List{Id} .

  eq X |-> undefined = .State .                 --- "undefine" a variable in a state

  eq (Sigma & X |-> I)(X) = I .
  eq Sigma(X) = undefined [owise] .

  eq (Sigma &  X |-> I)[I' / X ] = (Sigma & X |-> I') .
  eq Sigma[I / X] = (Sigma & X |-> I) [owise] .

  eq (X,X',Xl) |-> I = X |-> I & X' |-> I & Xl |-> I .
  eq .List{Id} |-> I = .State .
endm
```

Figure 3.5: The IMP state defined in Maude.

also from Section 2.1.2 that the update operation can be used not only to update maps but also to "undefine" particular elements in their domain: $\sigma[\bot/x]$ is the same as σ in all elements different from x and is undefined in x. Finally, recall also from Section 2.1.2 that the initialization operation yields a partial function mapping each element in the first list argument to the element given as second argument. These can be easily defined equationally, following the equational approach to partial finite-domain functions in Section 2.4.6.

★ Definition of IMP State in Maude

Figure 3.5 adapts the generic Maude definition of partial finite-domain functions in Section 2.5.6 for our purpose here: the generic sorts `Source` and `Target` are replaced by `Id` and `Int`, respectively. Recall from Section 2.5.6 that the constant `undefined` has sort `Undefined`, which is a subsort of all sorts corresponding to mathematical domains (e.g., `Int`, `Bool`, etc.). This way, identifiers can be made "undefined" in a state by simply updating them with `undefined` (see the equation dissolving undefined bindings in Figure 3.5).

To avoid overloading the comma "`,`" construct for too many purposes (which particularly may confuse Maude's parser), we took the freedom to rename the associative and commutative construct for states to `&`. The only reason for which we bother to give this obvious module here is because we want the various subsequent semantics of the IMP language, all of them including the module `STATE` in Figure 3.5, to be self-contained and executable in Maude by simply executing all the Maude code in the figures in this chapter.

3.1.3 Notes

The style that we follow in this chapter, namely to pick a simple language and then demonstrate the various language definitional approaches by means of that simple language, is quite common. In fact, we named our language IMP after a similar language introduced by Winskel in his book [87], also called IMP, which

is essentially identical to ours except that it uses a slightly different syntax and does not have variable declarations. For example, Winskel's IMP uses "`:=`" for assignment and "`;`" as statement separator instead of statement terminator, while our IMP's syntax resembles that of common languages like C and Java. Also, since most imperative languages do have variable declarations, we feel it is instructive to include them in our simple language. Winskell gives his IMP a big-step SOS, a small-step SOS, a denotational semantics, and an axiomatic semantics. Later, Nipkow [55] formalized all these semantics of IMP in the Isabelle/HOL proof assistant [56], and used it to formally relate the various semantics, effectively mechanizing most of Winskel's paper proofs; in doing so, Nipkow [55] found several minor errors in Winskel's proofs, thus showing the benefits of mechanization.

Vardejo and Martì-Oliet [83, 84] show how to use Maude to implement executable semantics for several languages following both big-step and small-step SOS approaches. Like us, they also demonstrate how to define different semantics for the same simple language using different styles; they do so both for an imperative language (very similar to our IMP) and for a functional language. Șerbănuță *et al.* [74] use a similar simple imperative language to also demonstrate how to use rewrite logic to define executable semantics. In fact, this chapter is an extension of [74], both in breadth and in depth. For example, we state and prove general faithful rewrite logic representation results for each of the semantic approaches, while [74] did the same only for the particular simple imperative language considered there. Also, we cover new approaches here, such as denotational semantics, which were not covered in [83, 84, 74].

3.2 Big-Step Structural Operational Semantics (Big-Step SOS)

Known also under the names *natural semantics*, *relational semantics*, and *evaluation semantics*, big-step structural operational semantics, or *big-step SOS* for short, is the most "denotational" of the operational semantics: one can view big-step definitions as definitions of functions, or more generally of relations, interpreting each language construct in an appropriate domain. Big-step semantics is so easy and natural to use, that one is strongly encouraged to use it whenever possible. Unfortunately, as discussed in Section 3.10, big-step semantics has a series of limitations making it inconvenient or impossible to use in many situations, such as when defining control-intensive language features, or non-deterministic ones, or concurrency.

A big-step SOS of a programming language or calculus is given as a formal *proof system* (see Section 2.1.5). The *big-step SOS sequents* are relations over configurations, typically written $C \Rightarrow R$ or $C \Downarrow R$, with the meaning that R is the configuration obtained after the (complete) evaluation of C. In this book we prefer the notation $C \Downarrow R$. A *big-step SOS rule* therefore has the form

$$\frac{C_1 \Downarrow R_1 \quad C_2 \Downarrow R_2 \quad \ldots \quad C_n \Downarrow R_n}{C_0 \Downarrow R_0} \text{ [if } \textit{condition}\text{]}$$

where C_0, C_1, C_2, ..., C_n are configurations holding fragments of program together with all the needed semantic components, where R_0, R_1, R_2, ..., R_n are *result configurations*, or *irreducible configurations*, i.e., configurations which cannot be reduced anymore, and where *condition* is an optional *side condition*; as discussed in Section 2.1.5, the role of side conditions is to filter out undesirable instances of the rule.

A big-step semantics compositionally describes how final evaluation results of language constructs can be obtained by combining the evaluation results of their syntactic counterparts (subexpressions, etc.). For example, the big-step semantics of IMP's addition is

$$\frac{\langle a_1, \sigma\rangle \Downarrow \langle i_1\rangle \quad \langle a_2, \sigma\rangle \Downarrow \langle i_2\rangle}{\langle a_1 + a_2, \sigma\rangle \Downarrow \langle i_1 +_{Int} i_2\rangle}$$

Here, the meaning of a relation $\langle a, \sigma\rangle \Downarrow \langle i\rangle$ is that arithmetic expression a is evaluated in state σ to integer i. If expression evaluation has side-effects, then one has to also include a state in the right configurations, so they become of the form $\langle i, \sigma\rangle$ instead of $\langle i\rangle$, as discussed in Section 3.10.

It is common in big-step semantics to not wrap single values in configurations, that is, to write $\langle a, \sigma\rangle \Downarrow i$ instead of $\langle a, \sigma\rangle \Downarrow \langle i\rangle$ and similarly for all the other sequents. Also, while the angle-bracket-and-comma notation $\langle \textit{code}, \textit{state}, \ldots\rangle$ is common for configurations, it is not enforced; some prefer to use a square or curly bracket notation of the form $[\textit{code}, \textit{state}, \ldots]$ or $\{\textit{code}, \textit{state}, \ldots\}$, or the simple tuple notation $(\textit{code}, \textit{state}, \ldots)$, or even to use a different (from comma) symbol to separate the various configuration ingredients, e.g., $\langle \textit{code} \mid \textit{state} \mid \ldots\rangle$, etc. Moreover, we may even encounter in the literature sequent notations of the form $\sigma \vdash a \Rightarrow i$ instead of $\langle a, \sigma\rangle \Downarrow \langle i\rangle$, as well as variants of sequent notations that prefer to move various semantic components from the configurations into special, sometimes rather informal, decorations of the symbols $\Downarrow$, $\vdash$ and/or $\Rightarrow$.

For the sake of a uniform notation, in particular when transitioning from languages whose expressions have no side effects to languages whose expressions do have side effects (as we do in Section 3.10), we prefer to always write big-step sequents as $C \Downarrow R$, and always use the angle brackets to surround both configurations involved. This solution is the most general; for example, any additional semantic data or labels that one may need in a big-step definition can be uniformly included as additional components in the configurations (the left ones, or the right ones, or both).

sorts:
Configuration
operations:

$$
\begin{array}{rcl}
\langle _, _ \rangle & : & \mathit{AExp} \times \mathit{State} \to \mathit{Configuration} \\
\langle _ \rangle & : & \mathit{Int} \to \mathit{Configuration} \\
\langle _, _ \rangle & : & \mathit{BExp} \times \mathit{State} \to \mathit{Configuration} \\
\langle _ \rangle & : & \mathit{Bool} \to \mathit{Configuration} \\
\langle _, _ \rangle & : & \mathit{Stmt} \times \mathit{State} \to \mathit{Configuration} \\
\langle _ \rangle & : & \mathit{State} \to \mathit{Configuration} \\
\langle _ \rangle & : & \mathit{Pgm} \to \mathit{Configuration}
\end{array}
$$

Figure 3.6: IMP big-step configurations as an algebraic signature.

3.2.1 IMP Configurations for Big-Step SOS

For the big-step semantics of the simple language IMP, we only need very simple configurations. We follow the comma-and-angle-bracket notational convention, that is, we separate the configuration components by commas and then enclose the entire list with angle brackets. For example, $\langle a, \sigma \rangle$ is a configuration containing an arithmetic expression a and a state σ, and $\langle b, \sigma \rangle$ is a configuration containing a Boolean expression b and a state σ. Some configurations may not need a state while others may not need the code. For example, $\langle i \rangle$ is a configuration holding only the integer number i that can be obtained as a result of evaluating an arithmetic expression, while $\langle \sigma \rangle$ is a configuration holding only one state σ that can be obtained after evaluating a statement. Configurations can therefore be of different types and need not necessarily have the same number of components. Here are all the configuration types needed for the big-step semantics of IMP:

- $\langle a, \sigma \rangle$ grouping arithmetic expressions a and states σ;
- $\langle i \rangle$ holding integers i;
- $\langle b, \sigma \rangle$ grouping Boolean expressions b and states σ;
- $\langle t \rangle$ holding truth values $t \in \{\mathit{true}, \mathit{false}\}$;
- $\langle s, \sigma \rangle$ grouping statements s and states σ;
- $\langle \sigma \rangle$ holding states σ;
- $\langle p \rangle$ holding programs p.

IMP Big-Step SOS Configurations as an Algebraic Signature

The configurations above were defined rather informally as tuples of syntax and/or states. There are many ways to rigorously formalize them, all building upon some formal definition of state (besides IMP syntax). Since we have already defined states as partial finite-domain functions (Section 3.1.2) and have already shown how partial finite-domain functions can be formalized as algebraic specifications (Section 2.4.6), we also formalize configurations algebraically.

Figure 3.6 shows an algebraic signature defining the IMP configurations needed for the subsequent big-step operational semantics. For simplicity, we preferred to explicitly define each type of needed configuration.

Consequently, our configurations definition in Figure 3.6 may be more verbose than an alternative polymorphic definition, but we believe that it is clearer for this simple language. We assumed that the sorts *AExp*, *BExp*, *Stmt*, *Pgm* and *State* come from algebraic definitions of the IMP syntax and state, like those in Sections 3.1.1 and 3.1.2; recall that the latter adapted the algebraic definition of partial functions in Section 2.4.6 (see Figure 2.7) as explained in Section 3.1.2.

3.2.2 The Big-Step SOS Rules of IMP

Figure 3.7 shows all the rules in our IMP big-step operational semantics proof system. Recall that the role of a proof system is to derive sequents, or facts. The facts that our proof system will derive have the forms $\langle a, \sigma\rangle \Downarrow \langle i\rangle$, $\langle b, \sigma\rangle \Downarrow \langle t\rangle$, $\langle s, \sigma\rangle \Downarrow \langle \sigma'\rangle$, and $\langle p\rangle \Downarrow \langle \sigma\rangle$ where a ranges over *AExp*, b over *BExp*, s over *Stmt*, p over *Pgm*, i over *Int*, t over *Bool*, and σ and σ' over *State*.

Informally[2], the meaning of derived triples of the form $\langle a, \sigma\rangle \Downarrow \langle i\rangle$ is that the arithmetic expression a evaluates/executes/transitions to the integer i in state σ; the meaning of $\langle b, \sigma\rangle \Downarrow \langle t\rangle$ is similar but with Boolean values instead of integers. The reason for which it suffices to derive such simple facts is because the evaluation of expressions in our simple IMP language is side-effect-free. When we add the increment operation ++ x in Section 3.10, we will have to change the big-step semantics to work with 4-tuples of the form $\langle a, \sigma\rangle \Downarrow \langle i, \sigma'\rangle$ and $\langle b, \sigma\rangle \Downarrow \langle t, \sigma'\rangle$ instead. The meaning of $\langle s, \sigma\rangle \Downarrow \langle \sigma'\rangle$ is that the statement s takes state σ to state σ'. Finally, the meaning of pairs $\langle p\rangle \Downarrow \langle \sigma\rangle$ is that the program p yields state σ when executed in the initial state.

In the case of our simple IMP language, the transition relation is going to be *deterministic*, in the sense that $i_1 = i_2$ whenever $\langle a, \sigma\rangle \Downarrow \langle i_1\rangle$ and $\langle a, \sigma\rangle \Downarrow \langle i_2\rangle$ can be deduced (and similarly for Boolean expressions, statements, and programs). However, in the context of non-deterministic languages, triples $\langle a, \sigma\rangle \Downarrow \langle i\rangle$ state that *a may* evaluate to i in state σ, but it may also evaluate to other integers (and similarly for Boolean expressions, statements, and programs).

The proof system in Figure 3.7 contains one or two rules for each language construct, capturing its intended evaluation relation. Recall from Section 2.1.5 that proof rules are in fact *rule schemas*, that is, they correspond to (recursively enumerable) sets of *rule instances*, one for each concrete instance of the rule *parameters* (i.e., a, b, σ, etc.). We next discuss each of the rules in Figure 3.7.

The rules (BigStep-Int) and (BigStep-Lookup) define the obvious semantics of integers and of variable lookup; these rules have no premises because integers and variables are atomic expressions, so one does not need to evaluate any other subexpression in order to evaluate them. The rule (BigStep-Add) has already been discussed at the beginning of Section 3.2, and (BigStep-Div) is similar. Note that the rules (BigStep-Lookup) and (BigStep-Div) have side conditions. We chose not to short-circuit the division operation when a_1 evaluates to 0. Consequently, no matter whether a_1 evaluates to 0 or not, a_2 is still expected to produce a correct value in order for the rule (BigStep-Div) to be applicable (e.g., a_2 cannot perform a division by 0).

Before we continue with the remaining rules, let us clarify, using concrete examples, what it means for rule schemas to admit multiple instances and how these can be used to derive proofs. For example, a possible instance of rule (BigStep-Div) can be the following (assume that $\mathtt{x}, \mathtt{y} \in \mathit{Id}$):

$$\frac{\langle \mathtt{x}, (\mathtt{x} \mapsto 8, \mathtt{y} \mapsto 0)\rangle \Downarrow \langle 8\rangle \quad \langle 2, (\mathtt{x} \mapsto 8, \mathtt{y} \mapsto 0)\rangle \Downarrow \langle 2\rangle}{\langle \mathtt{x} \,/\, 2, (\mathtt{x} \mapsto 8, \mathtt{y} \mapsto 0)\rangle \Downarrow \langle 4\rangle}$$

[2]Formal definitions of these concepts can only be given after one has a formal language definition. We formally define the notions of evaluation and termination in the context of the IMP language in Definition 20.

$$\langle i, \sigma\rangle \Downarrow \langle i\rangle \qquad \text{(BigStep-Int)}$$

$$\langle x, \sigma\rangle \Downarrow \langle \sigma(x)\rangle \quad \text{if } \sigma(x) \neq \bot \qquad \text{(BigStep-Lookup)}$$

$$\frac{\langle a_1, \sigma\rangle \Downarrow \langle i_1\rangle \quad \langle a_2, \sigma\rangle \Downarrow \langle i_2\rangle}{\langle a_1 + a_2, \sigma\rangle \Downarrow \langle i_1 +_{Int} i_2\rangle} \qquad \text{(BigStep-Add)}$$

$$\frac{\langle a_1, \sigma\rangle \Downarrow \langle i_1\rangle \quad \langle a_2, \sigma\rangle \Downarrow \langle i_2\rangle}{\langle a_1 \ / \ a_2, \sigma\rangle \Downarrow \langle i_1 \ /_{Int} \ i_2\rangle} \quad \text{if } i_2 \neq 0 \qquad \text{(BigStep-Div)}$$

$$\langle t, \sigma\rangle \Downarrow \langle t\rangle \qquad \text{(BigStep-Bool)}$$

$$\frac{\langle a_1, \sigma\rangle \Downarrow \langle i_1\rangle \quad \langle a_2, \sigma\rangle \Downarrow \langle i_2\rangle}{\langle a_1 <= a_2, \sigma\rangle \Downarrow \langle i_1 \leq_{Int} i_2\rangle} \qquad \text{(BigStep-Leq)}$$

$$\frac{\langle b, \sigma\rangle \Downarrow \langle \texttt{true}\rangle}{\langle \texttt{!}\ b, \sigma\rangle \Downarrow \langle \texttt{false}\rangle} \qquad \text{(BigStep-Not-True)}$$

$$\frac{\langle b, \sigma\rangle \Downarrow \langle \texttt{false}\rangle}{\langle \texttt{!}\ b, \sigma\rangle \Downarrow \langle \texttt{true}\rangle} \qquad \text{(BigStep-Not-False)}$$

$$\frac{\langle b_1, \sigma\rangle \Downarrow \langle \texttt{false}\rangle}{\langle b_1 \ \texttt{\&\&}\ b_2, \sigma\rangle \Downarrow \langle \texttt{false}\rangle} \qquad \text{(BigStep-And-False)}$$

$$\frac{\langle b_1, \sigma\rangle \Downarrow \langle \texttt{true}\rangle \quad \langle b_2, \sigma\rangle \Downarrow \langle t\rangle}{\langle b_1 \ \texttt{\&\&}\ b_2, \sigma\rangle \Downarrow \langle t\rangle} \qquad \text{(BigStep-And-True)}$$

$$\langle \{\}, \sigma\rangle \Downarrow \langle \sigma\rangle \qquad \text{(BigStep-Empty-Block)}$$

$$\frac{\langle s, \sigma\rangle \Downarrow \langle \sigma'\rangle}{\langle \{\ s\ \}, \sigma\rangle \Downarrow \langle \sigma'\rangle} \qquad \text{(BigStep-Block)}$$

$$\frac{\langle a, \sigma\rangle \Downarrow \langle i\rangle}{\langle x = a\texttt{;}, \sigma\rangle \Downarrow \langle \sigma[i/x]\rangle} \quad \text{if } \sigma(x) \neq \bot \qquad \text{(BigStep-Asgn)}$$

$$\frac{\langle s_1, \sigma\rangle \Downarrow \langle \sigma_1\rangle \quad \langle s_2, \sigma_1\rangle \Downarrow \langle \sigma_2\rangle}{\langle s_1 \ \ s_2, \sigma\rangle \Downarrow \langle \sigma_2\rangle} \qquad \text{(BigStep-Seq)}$$

$$\frac{\langle b, \sigma\rangle \Downarrow \langle \texttt{true}\rangle \quad \langle s_1, \sigma\rangle \Downarrow \langle \sigma_1\rangle}{\langle \texttt{if}\ (b)\ s_1\ \texttt{else}\ s_2, \sigma\rangle \Downarrow \langle \sigma_1\rangle} \qquad \text{(BigStep-If-True)}$$

$$\frac{\langle b, \sigma\rangle \Downarrow \langle \texttt{false}\rangle \quad \langle s_2, \sigma\rangle \Downarrow \langle \sigma_2\rangle}{\langle \texttt{if}\ (b)\ s_1\ \texttt{else}\ s_2, \sigma\rangle \Downarrow \langle \sigma_2\rangle} \qquad \text{(BigStep-If-False)}$$

$$\frac{\langle b, \sigma\rangle \Downarrow \langle \texttt{false}\rangle}{\langle \texttt{while}\ (b)\ s, \sigma\rangle \Downarrow \langle \sigma\rangle} \qquad \text{(BigStep-While-False)}$$

$$\frac{\langle b, \sigma\rangle \Downarrow \langle \texttt{true}\rangle \quad \langle s\ \texttt{while}\ (b)\ s, \sigma\rangle \Downarrow \langle \sigma'\rangle}{\langle \texttt{while}\ (b)\ s, \sigma\rangle \Downarrow \langle \sigma'\rangle} \qquad \text{(BigStep-While-True)}$$

$$\frac{\langle s, xl \mapsto 0\rangle \Downarrow \langle \sigma\rangle}{\langle \texttt{int}\ xl\texttt{;}\ s\rangle \Downarrow \langle \sigma\rangle} \qquad \text{(BigStep-Pgm)}$$

Figure 3.7: BigStep(IMP) — Big-step SOS of IMP ($i, i_1, i_2 \in Int$; $x \in Id$; $xl \in \mathbf{List}\{Id\}$; $a, a_1, a_2 \in AExp$; $t \in Bool$; $b, b_1, b_2 \in BExp$; $s, s_1, s_2 \in Stmt$; $\sigma, \sigma', \sigma_1, \sigma_2 \in State$).

Another instance of rule (BigStep-Div) is the following, which, of course, seems problematic:

$$\frac{\langle \mathtt{x}, (\mathtt{x} \mapsto 8, \mathtt{y} \mapsto 0)\rangle \Downarrow \langle 8\rangle \quad \langle 2, (\mathtt{x} \mapsto 8, \mathtt{y} \mapsto 0)\rangle \Downarrow \langle 4\rangle}{\langle \mathtt{x} \ / \ 2, (\mathtt{x} \mapsto 8, \mathtt{y} \mapsto 0)\rangle \Downarrow \langle 2\rangle}$$

The rule above is indeed a correct instance of (BigStep-Div), but, however, one will never be able to infer $\langle 2, (\mathtt{x} \mapsto 8, \mathtt{y} \mapsto 0)\rangle \Downarrow \langle 4\rangle$, so this rule can never be applied in a correct inference.

Note, however, that the following is *not* an instance of (BigStep-Div), no matter what ? is chosen to be ($\bot$, or $8/_{Int}0$, etc.):

$$\frac{\langle \mathtt{x}, (\mathtt{x} \mapsto 8, \mathtt{y} \mapsto 0)\rangle \Downarrow \langle 8\rangle \quad \langle \mathtt{y}, (\mathtt{x} \mapsto 8, \mathtt{y} \mapsto 0)\rangle \Downarrow \langle 0\rangle}{\langle \mathtt{x} \ / \ \mathtt{y}, (\mathtt{x} \mapsto 8, \mathtt{y} \mapsto 0)\rangle \Downarrow \langle ?\rangle}$$

Indeed, the above does not satisfy the side condition of (BigStep-Div).

The following is a valid proof derivation, where $\mathtt{x}, \mathtt{y} \in \mathit{Id}$ and $\sigma \in \mathit{State}$ with $\sigma(\mathtt{x}) = 8$ and $\sigma(\mathtt{y}) = 0$:

$$\frac{\dfrac{\cdot}{\langle \mathtt{x}, \sigma\rangle \Downarrow \langle 8\rangle} \quad \dfrac{\dfrac{\cdot}{\langle \mathtt{y}, \sigma\rangle \Downarrow \langle 0\rangle} \quad \dfrac{\cdot}{\langle \mathtt{x}, \sigma\rangle \Downarrow \langle 8\rangle}}{\langle \mathtt{y} \ / \ \mathtt{x}, \sigma\rangle \Downarrow \langle 0\rangle} \quad \dfrac{\cdot}{\langle 2, \sigma\rangle \Downarrow \langle 2\rangle}}{\langle \mathtt{y} \ / \ \mathtt{x} + 2, \sigma\rangle \Downarrow \langle 2\rangle}}{\langle \mathtt{x} \ / \ (\mathtt{y} \ / \ \mathtt{x} + 2), \sigma\rangle \Downarrow \langle 4\rangle}$$

The proof above can be regarded as a tree, with dots as leaves and instances of rule schemas as nodes. We call such complete (in the sense that their leaves are all dots and their nodes are correct rule instances) trees *proof trees*. This way, we have a way to mathematically *derive facts*, or *sequents*, about programs directly within their semantics. We may call the root of a proof tree the *fact (or sequent) that was proved or derived*, and the tree *its proof or derivation*.

Recall that our original intention was, for demonstration purposes, to attach various evaluation strategies to the arithmetic operations. We wanted `+` and `/` to be non-deterministic and `<=` to be left-right sequential; a non-deterministic evaluation strategy means that the subexpressions are evaluated in any order, possibly interleaving their evaluation steps, which is different from non-deterministically picking an order and then evaluating the subexpressions sequentially in that order. As an analogy, the former corresponds to evaluating the subexpressions concurrently on a multi-threaded machine, while the latter to non-deterministically queuing the subexpressions and then evaluating them one by one on a sequential machine. The former has obviously potentially many more possible behaviors than the latter. Note that many programming languages opt for non-deterministic evaluation strategies for their expression constructs precisely to allow compilers to evaluate them in any order or even concurrently; some language manuals explicitly warn the reader not to rely on any evaluation strategy of arithmetic constructs when writing programs.

Unfortunately, big-step semantics is not appropriate for defining non-deterministic evaluation strategies, because such strategies are, by their nature, small-step. One way to attempt to do it is to work with *sets* of result configurations instead of just with result configurations and thus associate to each fragment of program in a state the set of all the results that it can non-deterministically yield. However, such an approach would significantly complicate the big-step definition, so we prefer to not do it. Moreover, since IMP has no side effects yet (we will add it side effects in Section 3.5), the non-deterministic evaluation strategies would not lead to non-deterministic results anyway.

We next discuss the big-step rules for Boolean expressions. The rule (BigStep-Bool) is similar to rule (BigStep-Int), but it has only two instances, one for $t = \mathtt{true}$ and one for $t = \mathtt{false}$. The rule (BigStep-Leq) allows to derive Boolean sequents from arithmetic ones; although we want `<=` to evaluate

its arguments from left to right, there is no need to do anything more at this stage, because expressions have no side effects in IMP; in Section 3.5 we will see how to achieve the desired evaluation strategy in the presence of side effects in expressions. The rules (BigStep-Not-True) and (BigStep-Not-False) are clear; they could have been combined into only one rule if we had assumed our builtin *Bool* equipped with a negation operation. Unlike the division, the conjunction has a short-circuited semantics: if the first conjunct evaluates to `false` then the entire conjunction evaluates to `false` (rule (BigStep-And-False)), and if the first conjunct evaluates to `true` then the conjunction evaluates to whatever truth value the second conjunct evaluates (rule (BigStep-And-True)).

The role of statements in a language is to change the program state. Consequently, the rules for statements derive triples of the form $\langle s, \sigma\rangle \Downarrow \langle \sigma'\rangle$ with the meaning that if statement s is executed in state σ and *terminates*, then the resulting state is σ'. We will shortly discuss the aspect of termination in more detail. Rule (BigStep-Empty-Block) states that {} does nothing with the state. (BigStep-Block) states that the block has the same semantics as the enclosed statement, which is correct at this stage because IMP has no local variable declarations; this will change in Section 3.5. (BigStep-Asgn) shows how the state σ gets updated by an assignment statement "x = a;" after a is evaluated in state σ using the rules for arithmetic expressions discussed above. (BigStep-Seq) shows how the state updates are propagated by the sequential composition of statements, and rules (BigStep-If-True) and (BigStep-If-False) show how the conditional first evaluates its condition and then, depending upon the truth value of that, it either evaluates its then-branch or its else-branch, but never both. The rules giving the big-step semantics of the while loop say that if the condition evaluates to `false`then the while loop dissolves and the state stays unchanged, and if the condition evaluates to `true`then the body followed by the very same while loop is evaluated (rule (BigStep-While-True)). Finally, (BigStep-Pgm) gives the semantics of programs as the semantics of their statement in a state instantiating all the declared variables to 0.

On Proof Derivations, Evaluation, and Termination

So far we have used the words "evaluation" and "termination" informally. In fact, without a formal definition of a programming language, there is no other way, but informal, to define these notions. Once one has a formal definition of a language, one can not only formally define important concepts like evaluation and termination, but can also rigorously reason about programs.

Definition 20. *Given appropriate* IMP *configurations C and R, the* IMP *big-step sequent* $C \Downarrow R$ *is* ***derivable****, written* BigStep(IMP) $\vdash C \Downarrow R$*, iff there is some proof tree rooted in* $C \Downarrow R$ *which is derivable using the proof system* BigStep(IMP) *in Figure 3.7. Arithmetic (resp. Boolean) expression* $a \in AExp$ *(resp.* $b \in BExp$*)* ***evaluates*** *to integer* $i \in Int$ *(resp. to truth value* $t \in \{\texttt{true}, \texttt{false}\}$*) in state* $\sigma \in State$ *iff* BigStep(IMP) $\vdash \langle a, \sigma\rangle \Downarrow \langle i\rangle$ *(resp.* BigStep(IMP) $\vdash \langle b, \sigma\rangle \Downarrow \langle t\rangle$*). Statement* s ***terminates*** *in state* σ *iff* BigStep(IMP) $\vdash \langle s, \sigma\rangle \Downarrow \langle \sigma'\rangle$ *for some* $\sigma' \in State$*; if that is the case, then we say that* s ***evaluates*** *in state* σ *to state* σ'*, or that it* ***takes*** *state* σ *to state* σ'*. Finally, program* p ***terminates*** *iff* BigStep(IMP) $\vdash \langle p\rangle \Downarrow \langle \sigma\rangle$ *for some* $\sigma \in State$*.*

There are two reasons for which an IMP statement s may not terminate in a state σ: because it may contain a loop that does not terminate, or because it performs a division by zero and thus the rule (BigStep-Div) cannot apply. In the former case, the process of proof search does not terminate, while in the second case the process of proof search terminates in principle, but with a failure to find a proof. Unfortunately, big-step semantics cannot make any distinction between the two reasons for which a proof derivation cannot be found. Hence, the termination notion in Definition 20 rather means *termination with no error*. To catch

division-by-zero within the semantics, we need to add a special *error* value that a division by zero would evaluate to, and then to propagate it through all the language constructs (see Exercise 56).

A formal definition of a language allows to also formally define what it means for the language to be deterministic and to also prove it. For example, we can prove that if an IMP program p terminates then there is a unique state σ such that $\textsc{BigStep}(\text{IMP}) \vdash \langle p \rangle \Downarrow \langle \sigma \rangle$ (see Exercise 57).

Since each rule schema comprises a *recursively enumerable* collection of concrete instances and since we have a finite set of rule schemata, we can enumerate all the instances of all these rules. Furthermore, since proof trees built with nodes in a recursively enumerable set are themselves recursively enumerable, it follows that the set of proof trees derivable with the proof system in Figure 3.7 is recursively enumerable. In other words, we can find an algorithm that enumerates all the proof trees, in particular one that enumerates all the derivable sequents $C \Downarrow R$. By enumerating all proof trees, given a terminating IMP program p, one can eventually find the unique state σ such that $\langle p \rangle \Downarrow \langle \sigma \rangle$ is derivable. This simple-minded algorithm may take a very long time and a huge amount of resources, but it is insightful to understand that it can be done.

It can be shown that there is no algorithm, based on proof derivation like above or on anything else, which takes as input an IMP program and says whether it terminates or not (see Exercise 58). This follows from the fact that our simple language, due to its while loops and arbitrarily large integers, is Turing-complete. Thus, if one were able to decide termination of programs in our language then one would also be able to decide termination of Turing machines, contradicting one of the basic undecidable problems, the *halting problem* (see Section 2.2.1 for more on Turing machines).

An interesting observation here is that non-termination of a program corresponds to *lack of proof*, and that the latter is not decidable in many interesting logics. Indeed, in *complete* logics, that is, logics that admit a complete proof system, one can enumerate all the truths. However, in general there is not much one can do about non-truths, because the enumeration algorithm will loop forever when run on a non-truth. In decidable logics one can enumerate both truths and non-truths; clearly, decidable logics are not powerful enough for our task of defining programming languages, precisely because of the halting problem argument above.

3.2.3 Big-Step SOS in Rewrite Logic

Due to its straightforward recursive nature, big-step semantics is typically easy to represent in other formalisms and also easy to translate into interpreters for the defined languages in any programming language. (The difficulty with big-step semantics is to actually give big-step semantics to complex constructs, as illustrated and discussed in Section 3.5.) It should therefore come at no surprise to the reader that one can associate a conditional rewrite rule to each big-step rule and hereby obtain a rewrite logic theory that faithfully captures the big-step definition.

In this section we first show that any big-step operational semantics $\textsc{BigStep}$ can be mechanically translated into a rewrite logic theory $\mathcal{R}_{\textsc{BigStep}}$ in such a way that the corresponding derivation relations are step-for-step equivalent, that is, $\textsc{BigStep} \vdash C \Downarrow R$ if and only if $\mathcal{R}_{\textsc{BigStep}} \vdash \mathcal{R}_{C \Downarrow R}$, where $\mathcal{R}_{C \Downarrow R}$ is the corresponding syntactic translation of the big-step sequent $C \Downarrow R$ into a (one-step) rewrite rule. Second, we apply our generic translation technique on the big-step operational semantics $\textsc{BigStep}(\text{IMP})$ and obtain a rewrite logic semantics of IMP that is step-for-step equivalent to the original big-step semantics of IMP. Finally, we show how $\mathcal{R}_{\textsc{BigStep}(\text{IMP})}$ can be seamlessly defined in Maude, thus yielding an interpreter for IMP essentially for free.

Faithful Embedding of Big-Step SOS into Rewrite Logic

To define our translation generically, we need to make some assumptions about the existence of an algebraic axiomatization of configurations. More precisely, as also explained in Section 2.1.3, we assume that for any parametric term t (which can be a configuration, a condition, etc.), the term $\bar{t}$ is an equivalent algebraic variant of t of appropriate sort. For example, a parametric configuration C is a configuration that may possibly make use of parameters, such as $a \in AExp$, $\sigma \in State$, etc.; by equivalent algebraic variant we mean a term $\overline{C}$ of sort *Configuration* over an appropriate signature of configurations like the one that we defined for IMP in Section 3.2.1 (see Figure 3.6); moreover, each parameter in C gets replaced by a *variable* of corresponding sort in $\overline{C}$. Similarly, the algebraic variant of a rule side condition is an appropriate term of sort *Bool*.

To have a formal mechanism for performing reasoning within the employed mathematical domains, which is tacitly assumed in the big step semantics (e.g., $3 +_{Int} 5 = 8$, etc.), in particular for formally evaluating side conditions, we assume that the algebraic signature associated to the various syntactic and semantic categories is extended into a background algebraic specification capable of proving precisely all the equalities over ground terms (i.e., terms containing no variables). As discussed in Section 2.4, this assumption is quite reasonable, because any computational domain is isomorphic to an initial algebra over a finite algebraic specification, and that the latter can prove precisely all the ground equational properties of the domain/initial algebra. Consider, for example, the side condition $\sigma(x) \neq \bot$ of the rules (BigStep-Lookup) and (BigStep-Asgn) in Figure 3.7. Its algebraic variant is the term $\sigma(X) \neq \bot$ of *Bool* sort, where σ and X are variables of sorts *State* and *Id*, respectively. We therefore assume that any ground instance of this *Bool* term (obtained for a ground/concrete instance of the variables σ and X) can be proved using the background algebraic theory equal to either `true` (which means that the map is defined in the given variable) or `false`.

Consider now a general-purpose big-step rule of the form

$$\frac{C_1 \Downarrow R_1 \quad C_2 \Downarrow R_2 \quad \ldots \quad C_n \Downarrow R_n}{C_0 \Downarrow R_0} \ [\text{if } \textit{condition}]$$

where $C_0, C_1, C_2, \ldots, C_n$ are configurations holding fragments of program together with all the needed semantic components, $R_0, R_1, R_2, \ldots, R_n$ are result configurations, and *condition* is some optional side condition. As one may expect, we translate it into the rewrite logic rule

$$(\forall \mathcal{X})\ \overline{C_0} \to \overline{R_0}\ \textbf{if}\ \overline{C_1} \to \overline{R_1} \ \wedge\ \overline{C_2} \to \overline{R_2} \ \wedge\ \ldots\ \wedge\ \overline{C_n} \to \overline{R_n}\ [\wedge\ \overline{\textit{condition}}].$$

where $\mathcal{X}$ is the set of parameters, or meta-variables, that occur in the big-step proof rule (schema), now regarded as variables. Therefore, the big-step SOS rule premises and side conditions are both turned into conditions of the corresponding rewrite rule. The sequent premises become rewrites in the condition, while the side conditions become simple Boolean checks.

We make two reasonable assumptions about big-step semantics: (1) configurations cannot be nested; and (2) result configurations are irreducible.

Theorem 13. ***(Faithful embedding of big-step SOS into rewrite logic)*** *For any big-step operational semantics definition* BigStep*, and any* BigStep *appropriate configuration C and result configuration R, the following equivalence holds*

$$\text{BigStep} \vdash C \Downarrow R \iff \mathcal{R}_{\text{BigStep}} \vdash \overline{C} \to^1 \overline{R},$$

where $\mathcal{R}_{\text{BigStep}}$ is the rewrite logic semantic definition obtained from BigStep *by translating each rule in* BigStep *as above. (Recall from Section 2.5 that $\to^1$ is the one-step rewriting relation obtained by dropping the reflexivity and transitivity rules of rewrite logic. Also, since C and R are parameter-free—parameters only appear in rules—, $\overline{C}$ and $\overline{R}$ are ground terms.)*

Proof. Before we proceed with the proof, let us understand how the assumptions about BigStep and the use of $\rightarrow^1$ affect the rewrite logic proof derivations that can be performed with $\mathcal{R}_{\text{BigStep}}$. First, note that the Reflexivity and Transitivity proof rules of rewrite logic (see Section 2.5) will never apply. Second, since the rules in $\mathcal{R}_{\text{BigStep}}$ correspond to rules in BigStep between configurations and there are no other rules in $\mathcal{R}_{\text{BigStep}}$, and since configurations cannot be nested, we conclude that the Congruence rule of rewrite logic will never apply either. Third, since the rules of BigStep add no equations to $\mathcal{R}_{\text{BigStep}}$, the equations of $\mathcal{R}_{\text{BigStep}}$ correspond all to the background algebraic theory used for domain reasoning; thus, the Equality rule of rewrite logic can be used for domain reasoning and for nothing else. Finally, since configurations cannot be nested and since the result configurations of BigStep are irreducible, we conclude that the Replacement rule of rewrite logic can only instantiate rules in $\mathcal{R}_{\text{BigStep}}$ by substituting their variables with terms (which can be ground or not), but it can perform no "inner" concurrent rewrites. Therefore, the capabilities of rewrite logic are significantly crippled by $\mathcal{R}_{\text{BigStep}}$, as the only deductions that $\mathcal{R}_{\text{BigStep}}$ can perform are domain (equational) reasoning and rule instantiation.

Let Σ be the signature of the assumed background algebraic formalization of configurations.

Let us first assume that BigStep $\vdash C \Downarrow R$ and let us prove that $\mathcal{R}_{\text{BigStep}} \vdash \overline{C} \rightarrow^1 \overline{R}$. We do this proof by structural induction on the BigStep proof/derivation tree of $C \Downarrow R$. The last proof step in the tree deriving $C \Downarrow R$ must correspond to some instance of a rule (schema) of the form

$$\frac{C_1 \Downarrow R_1 \quad \ldots \quad C_m \Downarrow R_m}{C_0 \Downarrow R_0} \text{ if } \mathit{condition}$$

where $m \geq 0$ and where the *condition* may be missing, in which case we just assume it to be *true*. If $\mathcal{X}$ is the set of parameters, or meta-variables, that occur in this proof rule, then the last proof step in the derivation of $C \Downarrow R$ consists of an instance of the parameters in $\mathcal{X}$ that yields corresponding instances $(C, R, C'_1, R'_1, \ldots, C'_m, R'_m, \mathit{true})$ of, respectively, $(C_0, R_0, C_1, R_1, \ldots, C_m, R_m, \mathit{condition})$, such that BigStep $\vdash C'_i \Downarrow R'_i$ for all $1 \leq i \leq m$; moreover, domain reasoning is allowed at any moment, in particular to evaluate the instance of *condition* to *true*. By the (structural) induction hypothesis we have that $\mathcal{R}_{\text{BigStep}} \vdash \overline{C'_i} \rightarrow^1 \overline{R'_i}$ for all $1 \leq i \leq m$. More algebraically, the above say that there is a map $\theta : \mathcal{X} \rightarrow T_\Sigma$ such that:

- (domain reasoning) $\mathcal{R}_{\text{BigStep}} \vdash \overline{C} = \theta(\overline{C}_0)$ and $\mathcal{R}_{\text{BigStep}} \vdash \overline{R} = \theta(\overline{R}_0)$;
- (domain reasoning) $\mathcal{R}_{\text{BigStep}} \vdash \overline{C'_i} = \theta(\overline{C}_i)$ and $\mathcal{R}_{\text{BigStep}} \vdash \overline{R'_i} = \theta(\overline{R}_i)$ for all $1 \leq i \leq m$;
- (domain reasoning) $\mathcal{R}_{\text{BigStep}} \vdash \theta(\overline{\mathit{condition}}) = \texttt{true}$;
- (induction hypothesis) $\mathcal{R}_{\text{BigStep}} \vdash \overline{C'_i} \rightarrow^1 \overline{R'_i}$ for all $1 \leq i \leq m$.

Since $\mathcal{R}_{\text{BigStep}}$ contains the rule

$$(\forall \mathcal{X})\ \overline{C_0} \rightarrow \overline{R_0} \ \textbf{if}\ \overline{C_1} \rightarrow \overline{R_1} \ \wedge\ \overline{C_2} \rightarrow \overline{R_2} \ \wedge\ \ldots \ \wedge\ \overline{C_m} \rightarrow \overline{R_m} \ \wedge\ \overline{\mathit{condition}},$$

the Replacement and Equality rules of rewrite logic then give us $\mathcal{R}_{\text{BigStep}} \vdash \overline{C} \rightarrow^1 \overline{R}$.

Let us now assume that $\mathcal{R}_{\text{BigStep}} \vdash \overline{C} \rightarrow^1 \overline{R}$ and prove that BigStep $\vdash C \Downarrow R$. Since C and R are ground configurations and since all the rewrite rules derived in any given rewrite logic proof contain the same set of variables, it follows that all the rewrite rules derived as part of the proof of $\mathcal{R}_{\text{BigStep}} \vdash \overline{C} \rightarrow^1 \overline{R}$ are also ground. We show by structural induction on the rewrite logic derivation tree that any derivable sequent $\mathcal{R}_{\text{BigStep}} \vdash u \rightarrow^1 v$ with u and v ground is of the form $\mathcal{R}_{\text{BigStep}} \vdash \overline{C_u} \rightarrow^1 \overline{R_v}$, where C_u is a configuration and R_v a result configuration in BigStep such that BigStep $\vdash C_u \Downarrow R_v$. There are two cases to analyze:

- The last step used the Equality proof rule of rewrite logic, that is, $\mathcal{R}_{\text{BigStep}} \vdash u = u'$, $\mathcal{R}_{\text{BigStep}} \vdash u' \to^1 v'$, and $\mathcal{R}_{\text{BigStep}} \vdash v' = v$. Then by the induction hypothesis there is a configuration $C_{u'}$ and a result configuration $R_{v'}$ such that $u' = \overline{C_{u'}}$, $v' = \overline{R_{v'}}$ and $\text{BigStep} \vdash C_{u'} \Downarrow R_{v'}$. Since $\mathcal{R}_{\text{BigStep}} \vdash u = u'$ and $\mathcal{R}_{\text{BigStep}} \vdash v' = v$, and since equational deduction preserves the sorts of the terms proved equal, it follows that u and v are also of configuration sort. Then let C_u and R_v be the configurations corresponding to u and v, respectively, that is, $u = \overline{C_u}$ and $v = \overline{R_v}$. Since domain properties can be tacitly used in big-step derivations, we conclude that $\text{BigStep} \vdash C_u \Downarrow R_v$ and R_v is a result configuration.

- The last step is an instance of the Replacement rule of rewrite logic, with the rewrite rule

$$(\forall \mathcal{X})\ \overline{C_0} \to \overline{R_0}\ \textbf{if}\ \overline{C_1} \to \overline{R_1} \wedge \overline{C_2} \to \overline{R_2} \wedge \ldots \wedge \overline{C_m} \to \overline{R_m} \wedge \overline{condition}$$

  Then there is some ground substitution $\theta : \mathcal{X} \to T_\Sigma$ and ground terms $u_1, v_1, \ldots, u_m, v_m$ such that $u = \theta(\overline{C_0})$ and $v = \theta(\overline{R_0})$, $\theta(\overline{condition}) = \texttt{true}$, and $u_i = \theta(\overline{C_i})$ and $v_i = \theta(\overline{R_i})$ for all $1 \le i \le m$. By the induction hypothesis, there are configurations $C_{u_1}, \ldots, C_{u_m}$ and result configurations $R_{v_1}, \ldots, R_{v_m}$ such that $u_i = \overline{C_{u_i}}$, $v_i = \overline{R_{v_i}}$, and $\text{BigStep} \vdash C_{u_i} \Downarrow R_{v_i}$ for all $1 \le i \le m$. Let C_u and R_v be the configurations corresponding to u and v, respectively, that is, $u = \overline{C_u}$ and $v = \overline{R_v}$. We have thus found instances $(C_u, R_u, C_{u_1}, R_{v_1}, ..., C_{u_m}, R_{v_m}, \mathit{true})$ of $(C_0, R_0, C_1, R_1, ..., C_m, R_m, \mathit{condition})$, such that $\text{BigStep} \vdash C_{u_i} \Downarrow R_{v_i}$ for all $1 \le i \le m$. In other words, we constructed an instance of the BigStep rule

$$\frac{C_1 \Downarrow R_1 \quad ... \quad C_m \Downarrow R_m}{C_0 \Downarrow R_0}\ \text{if } \mathit{condition}$$

  with derivations for all its premises. Hence, $\text{BigStep} \vdash C \Downarrow R$.

□

The non-nestedness assumption on configurations in BigStep guarantees that the resulting rewrite rules in $\mathcal{R}_{\text{BigStep}}$ only apply at the top of the term they rewrite. The irreducibility of the result configurations guarantees that $\mathcal{R}_{\text{BigStep}}$ does not do more rewrite steps than intended, because rewriting is an inherently transitively closed relation, while the big-step relation $\Downarrow$ is not.

Since one typically perceives parameters as variables anyway, the only apparent difference between BigStep and $\mathcal{R}_{\text{BigStep}}$ is the different notational conventions they use ($\to$ instead of $\Downarrow$ and conditional rewrite rules instead of conditional deduction rules). As Theorem 13 shows, there is a one-to-one correspondence also between their corresponding "computations" (or executions, or derivations). Therefore, $\mathcal{R}_{\text{BigStep}}$ *is* the big-step operational semantics BigStep, and *not* an encoding of it.

At our knowledge, there is no rewrite engine[3] that supports the one-step rewrite relation $\to^1$ (that appears in Theorem 13). Indeed, rewrite engines aim at high-performance implementations of the general rewrite relation $\to$, which may even involve parallel rewriting (see Section 2.5 for the precise definitions of $\to^1$ and $\to$); $\to^1$ is meaningful only from a theoretical perspective and there is little to no practical motivation for an efficient implementation of it. Therefore, in order to execute the rewrite theory $\mathcal{R}_{\text{BigStep}}$ resulting from the mechanical translation of big-step semantics BigStep, one needs to take some precautions to ensure that $\to^1$ is actually identical to $\to$.

Given any rewrite theory $\mathcal{R}$, a sufficient condition for $\to^1$ to be the same as $\to$ in $\mathcal{R}$ is for the right-hand-sides of the rules in $\mathcal{R}$ to generate terms which make any context that contains them unmatchable by any rule

[3] Maude's `rewrite[1]` command does not inhibit the transitive closure of the rewrite relation, it only stops the rewrite engine on a given term after *one* application of a rule on that term; however, many (transitive) applications of rules are allowed when solving the condition of that rule.

in $\mathcal{R}$. Fortunately, the two assumptions we made about our original BigStep semantics guarantee this property. First, the non-nestedness assumption guarantees that configurations can only appear at the top of a term, so the only contexts that contain configurations are the configurations themselves. Second, the irreducibility of result configurations assumption ensures that the rules in $\mathcal{R}_{\text{BigStep}}$ will never match any result configurations. Therefore, we can conclude the following important

Corollary 3. *Under the same hypotheses and assumptions as in Theorem 13,*

$$\text{BigStep} \vdash C \Downarrow R \iff \mathcal{R}_{\text{BigStep}} \vdash \overline{C} \to^1 \overline{R} \iff \mathcal{R}_{\text{BigStep}} \vdash \overline{C} \to \overline{R}.$$

Our current BigStep(IMP) semantics verifies our second assumption, since the configurations to the left of $\Downarrow$ and the result configurations to the right of $\Downarrow$ are always distinct. Unfortunately, in general that may not always be the case. For example, when we extend IMP with side effects in Section 3.10, the state also needs to be part of result configurations, so the semantics of integers is going to be given by an unconditional rule of the form $\langle i, \sigma\rangle \Downarrow \langle i, \sigma\rangle$, which after translation becomes the rewrite rule $\langle i, \sigma\rangle \to \langle i, \sigma\rangle$. This rule will make the rewrite relation $\to$ not terminate anymore (although the relation $\to^1$ will still terminate). There are at least two simple ways to ensure the irreducibility of result configurations, and thus make Corollary 3 still hold:

1. It is highly expected that the only big-step rules in BigStep having a result configuration to the left of $\Downarrow$ are unconditional rules of the form $R \Downarrow R$; such rules typically say that a value is already evaluated. If that is the case, then one can simply drop all the corresponding rules $\overline{R} \to \overline{R}$ from $\mathcal{R}_{\text{BigStep}}$ and the resulting rewrite theory, say $\mathcal{R}'_{\text{BigStep}}$ still has the property $\text{BigStep} \vdash C \Downarrow R \iff \mathcal{R}'_{\text{BigStep}} \vdash \overline{C} \to \overline{R}$, which is desirable in order to execute the big-step definition on rewrite engines, although the property $\text{BigStep} \vdash C \Downarrow R \iff \mathcal{R}'_{\text{BigStep}} \vdash \overline{C} \to^1 \overline{R}$ will not hold anymore, because, e.g., even though $R \Downarrow R$ is a rule in BigStep, it is not the case that $\mathcal{R}'_{\text{BigStep}} \vdash \overline{R} \to^1 \overline{R}$.

2. If BigStep contains pairs $R' \Downarrow R$ where R' and R are possibly different result configurations, then one can apply the following general procedure. Change or augment the syntax of the configurations to the left or to the right of $\Downarrow$, so that those changed or augmented configurations will always be different from the other ones. This is the technique employed in our representation of small-step operational semantics in rewriting logic in Section 3.3. More precisely, we prepend all the configurations to the left of the rewrite relation in $\mathcal{R}_{\text{BigStep}}$ with a circle $\circ$, e.g., $\circ C \to R$, with the intuition that the circled configurations are *active*, while the other ones are *inactive*.

Regardless of how the desired property $\text{BigStep} \vdash C \Downarrow R \iff \mathcal{R}_{\text{BigStep}} \vdash \overline{C} \to \overline{R}$ is ensured, note that, unfortunately, $\mathcal{R}_{\text{BigStep}}$ lacks the main strengths of rewrite logic that make it a good formalism for concurrency: in rewrite logic, rewrite rules can apply under any context and in parallel. Indeed, the rules of $\mathcal{R}_{\text{BigStep}}$ can only apply at the top, sequentially.

Big-Step SOS of IMP in Rewrite Logic

Figure 3.8 shows the rewrite logic theory $\mathcal{R}_{\text{BigStep(IMP)}}$ that is obtained by applying the procedure above to the big-step semantics of IMP, BigStep(IMP), in Figure 3.7. We have used the rewrite logic convention that variables start with upper-case letters. For the state variable, we used Σ, that is, a larger σ symbol.

Note how the three side conditions that appear in the proof system in Figure 3.7 turned into normal conditions of rewrite rules. In particular, the two side conditions saying that $\sigma(x)$ is defined became the algebraic term $\Sigma(X) \neq \bot$ of Boolean sort.

The following corollary of Theorem 13 and Corollary 3 establishes the faithfulness of the representation of the big-step operational semantics of IMP in rewrite logic:

$$
\begin{array}{rll}
\langle I, \sigma\rangle \to \langle I\rangle & & \\
\langle X, \sigma\rangle \to \langle \sigma(X)\rangle & \textbf{if} & \sigma(X) \neq \bot \\
\langle A_1 + A_2, \sigma\rangle \to \langle I_1 +_{Int} I_2\rangle & \textbf{if} & \langle A_1, \sigma\rangle \to \langle I_1\rangle \ \wedge\ \langle A_2, \sigma\rangle \to \langle I_2\rangle \\
\langle A_1 \ / \ A_2, \sigma\rangle \to \langle I_1 \ /_{Int} \ I_2\rangle & \textbf{if} & \langle A_1, \sigma\rangle \to \langle I_1\rangle \ \wedge\ \langle A_2, \sigma\rangle \to \langle I_2\rangle \ \wedge\ I_2 \neq 0 \\
\langle A_1 <= A_2, \sigma\rangle \to \langle I_1 \leq_{Int} I_2\rangle & \textbf{if} & \langle A_1, \sigma\rangle \to \langle I_1\rangle \ \wedge\ \langle A_2, \sigma\rangle \to \langle I_2\rangle \\
\langle T, \sigma\rangle \to \langle T\rangle & & \\
\langle \texttt{!}\ B, \sigma\rangle \to \langle \texttt{false}\rangle & \textbf{if} & \langle B, \sigma\rangle \to \langle \texttt{true}\rangle \\
\langle \texttt{!}\ B, \sigma\rangle \to \langle \texttt{true}\rangle & \textbf{if} & \langle B, \sigma\rangle \to \langle \texttt{false}\rangle \\
\langle B_1 \ \texttt{\&\&}\ B_2, \sigma\rangle \to \langle \texttt{false}\rangle & \textbf{if} & \langle B_1, \sigma\rangle \to \langle \texttt{false}\rangle \\
\langle B_1 \ \texttt{\&\&}\ B_2, \sigma\rangle \to \langle T\rangle & \textbf{if} & \langle B_1, \sigma\rangle \to \langle \texttt{true}\rangle \ \wedge\ \langle B_2, \sigma\rangle \to \langle T\rangle \\
\langle \{\}, \sigma\rangle \to \langle \sigma\rangle & & \\
\langle \{\ S\ \}, \sigma\rangle \to \langle \sigma'\rangle & \textbf{if} & \langle S, \sigma\rangle \to \langle \sigma'\rangle \\
\langle X = A\texttt{;}, \sigma\rangle \to \langle \sigma[I/X]\rangle & \textbf{if} & \langle A, \sigma\rangle \to \langle I\rangle \ \wedge\ \sigma(X) \neq \bot \\
\langle S_1 \ S_2, \sigma\rangle \to \langle \sigma_2\rangle & \textbf{if} & \langle S_1, \sigma\rangle \to \langle \sigma_1\rangle \ \wedge\ \langle S_2, \sigma_1\rangle \to \langle \sigma_2\rangle \\
\langle \texttt{if}\,(B)\, S_1\, \texttt{else}\, S_2, \sigma\rangle \to \langle \sigma_1\rangle & \textbf{if} & \langle B, \sigma\rangle \to \langle \texttt{true}\rangle \ \wedge\ \langle S_1, \sigma\rangle \to \langle \sigma_1\rangle \\
\langle \texttt{if}\,(B)\, S_1\, \texttt{else}\, S_2, \sigma\rangle \to \langle \sigma_2\rangle & \textbf{if} & \langle B, \sigma\rangle \to \langle \texttt{false}\rangle \ \wedge\ \langle S_2, \sigma\rangle \to \langle \sigma_2\rangle \\
\langle \texttt{while}\,(B)\, S, \sigma\rangle \to \langle \sigma\rangle & \textbf{if} & \langle B, \sigma\rangle \to \langle \texttt{false}\rangle \\
\langle \texttt{while}\,(B)\, S, \sigma\rangle \to \langle \sigma'\rangle & \textbf{if} & \langle B, \sigma\rangle \to \langle \texttt{true}\rangle \ \wedge\ \langle \{\ S\ \texttt{while}\,(B)\, S\ \}, \sigma\rangle \to \langle \sigma'\rangle \\
\langle \texttt{int}\ Xl\texttt{;}\ S\rangle \to \langle \sigma\rangle & \textbf{if} & \langle S, Xl \mapsto 0\rangle \to \langle \sigma\rangle
\end{array}
$$

Figure 3.8: $\mathcal{R}_{\text{BigStep(IMP)}}$: the big-step SOS of IMP in rewrite logic.

Corollary 4. BigStep(IMP) $\vdash C \Downarrow R \iff \mathcal{R}_{\text{BigStep(IMP)}} \vdash \overline{C} \to \overline{R}$.

Therefore, there is no perceivable computational difference between the IMP-specific proof system BigStep(IMP) and generic rewrite logic deduction using the IMP-specific rewrite rules in $\mathcal{R}_{\text{BigStep(IMP)}}$, so the two are faithfully equivalent.

★ Maude Definition of IMP Big-Step SOS

Figure 3.9 shows a straightforward Maude representation of the rewrite theory $\mathcal{R}_{\text{BigStep(IMP)}}$ in Figure 3.8, including a representation of the algebraic signature in Figure 3.6 for configurations as needed for big-step SOS. The Maude module `IMP-SEMANTICS-BIGSTEP` in Figure 3.9 is executable, so Maude, through its rewriting capabilities, yields an interpreter for IMP; for example, the command

```
rewrite < sumPgm > .
```

where `sumPgm` is the first program defined in the module `IMP-PROGRAMS` in Figure 3.4, produces a result of the form (the exact statistics are irrelevant here, so they were replaced by "...”):

```
rewrites: 5218 in ... cpu (... real) (... rewrites/second)
result Configuration: < n |-> 0 , s |-> 5050 >
```

The obtained IMP interpreter actually has acceptable performance; for example, all the programs in Figure 3.4 together take a fraction of a second to execute on conventional PCs or laptops.

In fact, Maude needs only one rewrite logic step to rewrite any configuration; in particular,

```
rewrite [1] < sumPgm > .
```

```
mod IMP-CONFIGURATIONS-BIGSTEP is including IMP-SYNTAX + STATE .
  sort Configuration .
  op <_,_> : AExp State -> Configuration .
  op   <_> : Int -> Configuration .
  op <_,_> : BExp State -> Configuration .
  op   <_> : Bool -> Configuration .
  op <_,_> : Stmt State -> Configuration .
  op   <_> : State -> Configuration .
  op   <_> : Pgm -> Configuration .
endm

mod IMP-SEMANTICS-BIGSTEP is including IMP-CONFIGURATIONS-BIGSTEP .
  var X : Id .  var Xl : List{Id} .  var Sigma Sigma' Sigma1 Sigma2 : State .
  var I I1 I2 : Int .  var T : Bool .
  var A A1 A2 : AExp .  var B B1 B2 : BExp .  var S S1 S2 : Stmt .

  rl < I,Sigma > => < I > .
 crl < X,Sigma > => < Sigma(X) >
  if Sigma(X) =/=Bool undefined .
 crl < A1 + A2,Sigma > => < I1 +Int I2 >
  if < A1,Sigma > => < I1 > /\ < A2,Sigma > => < I2 > .
 crl < A1 / A2,Sigma > => < I1 /Int I2 >
  if < A1,Sigma > => < I1 > /\ < A2,Sigma > => < I2 > /\ I2 =/=Bool 0 .

  rl < T,Sigma > => < T > .
 crl < A1 <= A2,Sigma > => < I1 <=Int I2 >
  if < A1,Sigma > => < I1 > /\ < A2,Sigma > => < I2 > .
 crl < ! B,Sigma > => < false >
  if < B,Sigma > => <  true > .
 crl < ! B,Sigma > => < true >
  if < B,Sigma > => < false > .
 crl < B1 && B2,Sigma > => < false >
  if < B1,Sigma > => < false > .
 crl < B1 && B2,Sigma > => < T >
  if < B1,Sigma > => < true > /\ < B2,Sigma > => < T > .

  rl < {},Sigma > => < Sigma > .
 crl < {S},Sigma > => < Sigma' >
  if < S,Sigma > => < Sigma' > .
 crl < X = A ;,Sigma > => < Sigma[I / X] >
  if < A,Sigma > => < I > /\ Sigma(X) =/=Bool undefined .
 crl < S1 S2,Sigma > => < Sigma2 >
  if < S1,Sigma > => < Sigma1 > /\ < S2,Sigma1 > => < Sigma2 > .
 crl < if (B) S1 else S2,Sigma > => < Sigma1 >
  if < B,Sigma > => < true > /\ < S1,Sigma > => < Sigma1 > .
 crl < if (B) S1 else S2,Sigma > => < Sigma2 >
  if < B,Sigma > => < false > /\ < S2,Sigma > => < Sigma2 > .
 crl < while (B) S,Sigma > => < Sigma >
  if < B,Sigma > => < false > .
 crl < while (B) S,Sigma > => < Sigma' >
  if < B,Sigma > => < true > /\ < S while (B) S,Sigma > => < Sigma' > .

 crl < int Xl ; S > => < Sigma >
  if < S,(Xl |-> 0) > => < Sigma > .
endm
```

Figure 3.9: The big-step SOS of IMP in Maude, including the definition of configurations.

will give the same output as above. Recall from Section 2.5.6 that Maude performs a potentially exhaustive search to satisfy the rewrites in rule conditions. Thus, a large number of rule instances can be attempted in order to apply one conditional rule, so a `rewrite [1]` command can take a long time; it may not even terminate. Nevertheless, thanks to Theorem 13, Maude's implicit search mechanism in conditions effectively achieves a proof searcher for big-step SOS derivations.

Once one has a rewrite logic definition in Maude, one can use any of the general-purpose tools provided by Maude on that definition; the rewrite engine is only one of them. For example, one can exhaustively search for all possible behaviors of a program using the `search` command:

```
search < sumPgm > =>! Cfg:Configuration .
```

Since our IMP language so far is deterministic, the `search` command will not discover any new behaviors. In fact, the search command will only discover two configurations in total, the original configuration `< sumPgm >` and the result configuration `< n |-> 0 & s |-> 5050 >`. However, as shown in Section 3.5 where we extend IMP with various language features, the `search` command can indeed show all the behaviors of a non-deterministic program (restricted only by the limitations of the particular semantic style employed).

3.2.4 Defining a Type System for IMP Using Big-Step SOS

Big-step SOS is routinely used to define type systems for programming languages, even though in most cases this connection is not made explicit. In this section we demonstrate the use of big-step SOS for defining a type system for IMP, following the same steps as above but more succinctly. Type systems is a broad subject, with many variations and important applications to programming languages. Our intention in this section is twofold: on the one hand we show that big-step SOS is not limited to only defining language semantics, and, on the other hand, we introduce the reader to type systems by means of a very simple example.

The idea underlying big-step SOS definitions of type systems is that a given program or fragment of program in a given type environment reduces, in one big step, to its type. Like states, type environments are also partial mappings, but from variable names into types instead of values. A common notation for a type judgment is $\Gamma \vdash c : \tau$, where Γ is a type environment, c is a program or fragment, and τ is a type. This type judgment reads "in type environment Γ, program or fragment c has type τ". One can find countless variations of the notation for type judgments in the literature, usually adding more items (pieces of information) to the left of $\vdash$, to its right, or as subscripts or superscripts of it. There is, unfortunately, no well-established notation for all type judgments. Nevertheless, type judgments are special big-step sequents relating two special configurations, one including the givens and the other the results. For example, a simple type judgment $\Gamma \vdash c : \tau$ like above can be regarded as a big-step sequent $\langle c, \Gamma \rangle \Downarrow \langle \tau \rangle$. However, this notation is not preferred.

Figure 3.10 depicts our type system for IMP, which is a nothing but a big-step SOS proof system. We, however, follow the more conventional notation for type judgments discussed above, with one slight change: since in IMP variables are intended to hold only integer values, there is no need for type environments; instead, we replace them by lists of variables, each meant to have the type `int`. Therefore, $xl \vdash c : \tau$ with c and τ as above but with xl a list of variables reads as follows: "when the variables in xl are defined, c has the type τ". We drop the list of variables from the typing judgments of programs, because that would be empty anyway. The big-step SOS rules in Figure 3.10 define the typing policy of each language construct of IMP, guaranteeing all together that a program p types, that is, that $\vdash p : \texttt{pgm}$ is derivable if and only if each construct is used according to its intended typing policy and, moreover, that p declares each variable that it uses. For our simple IMP language, a CFG parser using the syntax defined in Figure 3.1 would already guarantee that each construct is used as intended. Note, however, that the second desired property of our type system (each used variable is declared) is context dependent.

$$xl \vdash i : \texttt{int} \qquad \text{(BigStepTypeSystem-Int)}$$

$$xl \vdash x : \texttt{int} \quad \text{if } x \in xl \qquad \text{(BigStepTypeSystem-Lookup)}$$

$$\frac{xl \vdash a_1 : \texttt{int} \quad xl \vdash a_2 : \texttt{int}}{xl \vdash a_1 + a_2 : \texttt{int}} \qquad \text{(BigStepTypeSystem-Add)}$$

$$\frac{xl \vdash a_1 : \texttt{int} \quad xl \vdash a_2 : \texttt{int}}{xl \vdash a_1 \ \texttt{/}\ a_2 : \texttt{int}} \qquad \text{(BigStepTypeSystem-Div)}$$

$$xl \vdash t : \texttt{bool} \quad \text{if } t \in \{\texttt{true}, \texttt{false}\} \qquad \text{(BigStepTypeSystem-Bool)}$$

$$\frac{xl \vdash a_1 : \texttt{int} \quad xl \vdash a_2 : \texttt{int}}{xl \vdash a_1 \texttt{<=} a_2 : \texttt{bool}} \qquad \text{(BigStepTypeSystem-Leq)}$$

$$\frac{xl \vdash b : \texttt{bool}}{xl \vdash \texttt{!}\, b : \texttt{bool}} \qquad \text{(BigStepTypeSystem-Not)}$$

$$\frac{xl \vdash b_1 : \texttt{bool} \quad xl \vdash b_2 : \texttt{bool}}{xl \vdash b_1 \ \texttt{\&\&}\ b_2 : \texttt{bool}} \qquad \text{(BigStepTypeSystem-And)}$$

$$xl \vdash \{\} : \texttt{block} \qquad \text{(BigStepTypeSystem-Empty-Block)}$$

$$\frac{xl \vdash s : \tau}{xl \vdash \{\, s \,\} : \texttt{block}} \quad \text{if } \tau \in \{\texttt{block}, \texttt{stmt}\} \qquad \text{(BigStepTypeSystem-Block)}$$

$$\frac{xl \vdash a : \texttt{int}}{xl \vdash x \texttt{=} a\texttt{;} : \texttt{stmt}} \quad \text{if } x \in xl \qquad \text{(BigStepTypeSystem-Asgn)}$$

$$\frac{xl \vdash s_1 : \tau_1 \quad xl \vdash s_2 : \tau_2}{xl \vdash s_1 \ s_2 : \texttt{stmt}} \quad \text{if } \tau_1, \tau_2 \in \{\texttt{block}, \texttt{stmt}\} \qquad \text{(BigStepTypeSystem-Seq)}$$

$$\frac{xl \vdash b : \texttt{bool} \quad xl \vdash s_1 : \texttt{block} \quad xl \vdash s_2 : \texttt{block}}{xl \vdash \texttt{if}\,(b)\ s_1\ \texttt{else}\ s_2 : \texttt{stmt}} \qquad \text{(BigStepTypeSystem-If)}$$

$$\frac{xl \vdash b : \texttt{bool} \quad xl \vdash s : \texttt{block}}{xl \vdash \texttt{while}\,(b)\ s : \texttt{stmt}} \qquad \text{(BigStepTypeSystem-While)}$$

$$\frac{xl \vdash s : \tau}{\vdash \texttt{int}\ xl\texttt{;}\ s : \texttt{pgm}} \quad \text{if } \tau \in \{\texttt{block}, \texttt{stmt}\} \qquad \text{(BigStepTypeSystem-Pgm)}$$

Figure 3.10: BigStepTypeSystem(IMP) — Type system of IMP using big-step SOS ($xl \in$ **List{***Id***}**; $i \in Int$; $x \in Id$; $a, a_1, a_2 \in AExp$; $b, b_1, b_2 \in BExp$; $s, s_1, s_2 \in Stmt$).

Let us next use the type system in Figure 3.10 to type the program `sumPgm` in Figure 3.4. We split the proof tree in proof subtrees. Note first that using the rules (BigStepTypeSystem-Int) (first level), (BigStepTypeSystem-Asgn) (second level) and (BigStepTypeSystem-Seq) (third level), we can derive the following proof tree, say $tree_1$:

$$tree_1 = \left\{ \begin{array}{c} \dfrac{\dfrac{\cdot}{\texttt{n,s} \vdash \texttt{100} : \texttt{int}}}{\texttt{n,s} \vdash \texttt{(n = 100;)} : \texttt{stmt}} \quad \dfrac{\dfrac{\cdot}{\texttt{n,s} \vdash \texttt{0} : \texttt{int}}}{\texttt{n,s} \vdash \texttt{(s = 0;)} : \texttt{stmt}} \\ \hline \texttt{n,s} \vdash \texttt{(n = 100; s = 0;)} : \texttt{stmt} \end{array} \right.$$

Similarly, using rules (BigStepTypeSystem-Lookup) and (BigStepTypeSystem-Int) (first level), (BigStepTypeSystem-Leq) (second level), and (BigStepTypeSystem-Not) (third level), we can derive the following proof tree, say $tree_2$:

$$tree_2 = \left\{ \begin{array}{c} \dfrac{\dfrac{\cdot}{\texttt{n,s} \vdash \texttt{n} : \texttt{int}} \quad \dfrac{\cdot}{\texttt{n,s} \vdash \texttt{0} : \texttt{int}}}{\texttt{n,s} \vdash \texttt{(n <= 0)} : \texttt{bool}} \\ \hline \texttt{n,s} \vdash \texttt{(!(n <= 0))} : \texttt{bool} \end{array} \right.$$

Similarly, we can derive the following proof tree, say $tree_3$:

$$tree_3 = \left\{ \begin{array}{c} \dfrac{\dfrac{\dfrac{\cdot}{\texttt{n,s} \vdash \texttt{s} : \texttt{int}} \quad \dfrac{\cdot}{\texttt{n,s} \vdash \texttt{n} : \texttt{int}}}{\texttt{n,s} \vdash \texttt{(s + n)} : \texttt{int}}}{\texttt{n,s} \vdash \texttt{(s = s + n;)} : \texttt{stmt}} \quad \dfrac{\dfrac{\dfrac{\cdot}{\texttt{n,s} \vdash \texttt{n} : \texttt{int}} \quad \dfrac{\cdot}{\texttt{n,s} \vdash \texttt{-1} : \texttt{int}}}{\texttt{n,s} \vdash \texttt{(n + -1)} : \texttt{int}}}{\texttt{n,s} \vdash \texttt{(n = n + -1;)} : \texttt{stmt}} \\ \hline \texttt{n,s} \vdash \texttt{(s = s + n; n = n + -1;)} : \texttt{stmt} \\ \hline \texttt{n,s} \vdash \texttt{\{ s = s + n; n = n + -1; \}} : \texttt{block} \end{array} \right.$$

Finally, we can now derive the tree that proves that `sumPgm` is well-typed:

$$\dfrac{\dfrac{tree_1 \quad \dfrac{tree_2 \quad tree_3}{\texttt{n,s} \vdash \texttt{(while (!(n <= 0)) \{s = s + n; n = n + -1;\})} : \texttt{stmt}}}{\texttt{n,s} \vdash \texttt{(n = 100; s = 0; while (!(n <= 0)) \{s = s + n; n = n + -1;\})} : \texttt{stmt}}}{\vdash \texttt{(int n,s; n = 100; s = 0; while (!(n <= 0)) \{s = s + n; n = n + -1;\})} : \texttt{pgm}}$$

A major role of a type system is to filter out a set of programs which are obviously wrong. Unfortunately, it is impossible to filter out precisely those programs which would execute erroneously. For example, note that a division is considered type safe whenever its two arguments type to integers, but no check is being made on whether the denominator is 0 or not. Indeed, statically checking whether an expression has a certain value at a certain point in a program is an undecidable problem. Also, no check is being made on whether a detected type error is reachable or not (if unreachable, the detected type error will never show up at runtime). Statically checking whether a certain point in a program is reachable is also an undecidable problem. One should therefore be aware of the fact that in general a type system may allow programs which run into errors when executed and, moreover, that it may reject programs which would execute correctly.

$$
\begin{array}{rl}
\langle I, Xl\rangle \to \langle \texttt{int}\rangle & \\
\langle X, (Xl, X, Xl')\rangle \to \langle \texttt{int}\rangle & \\
\langle A_1 + A_2, Xl\rangle \to \langle \texttt{int}\rangle & \textbf{if } \langle A_1, Xl\rangle \to \langle \texttt{int}\rangle \wedge \langle A_2, Xl\rangle \to \langle \texttt{int}\rangle \\
\langle \div IMP A_1 A_2, Xl\rangle \to \langle \texttt{int}\rangle & \textbf{if } \langle A_1, Xl\rangle \to \langle \texttt{int}\rangle \wedge \langle A_2, Xl\rangle \to \langle \texttt{int}\rangle \\
\langle A_1 <= A_2, Xl\rangle \to \langle \texttt{bool}\rangle & \textbf{if } \langle A_1, Xl\rangle \to \langle \texttt{int}\rangle \wedge \langle A_2, Xl\rangle \to \langle \texttt{int}\rangle \\
\langle T, Xl\rangle \to \langle \texttt{bool}\rangle & \\
\langle !\, B, Xl\rangle \to \langle \texttt{bool}\rangle & \textbf{if } \langle B, Xl\rangle \to \langle \texttt{bool}\rangle \\
\langle B_1 \;\&\&\; B_2, Xl\rangle \to \langle \texttt{bool}\rangle & \textbf{if } \langle B_1, Xl\rangle \to \langle \texttt{bool}\rangle \wedge \langle B_2, Xl\rangle \to \langle \texttt{bool}\rangle \\
\langle \{\}, Xl\rangle \to \langle \texttt{block}\rangle & \\
\langle \{\, S \,\}, Xl\rangle \to \langle \texttt{block}\rangle & \textbf{if } \langle S, Xl\rangle \to \langle ST\rangle \\
\langle X = A\,\texttt{;}, (Xl, X, Xl')\rangle \to \langle \texttt{stmt}\rangle & \textbf{if } \langle A, (Xl, X, Xl')\rangle \to \langle \texttt{int}\rangle \\
\langle S_1\; S_2, Xl\rangle \to \langle \texttt{stmt}\rangle & \textbf{if } \langle S_1, Xl\rangle \to \langle ST_1\rangle \wedge \langle S_2, Xl\rangle \to \langle ST_2\rangle \\
\langle \texttt{if}\,(B)\, S_1\, \texttt{else}\, S_2, Xl\rangle \to \langle \texttt{stmt}\rangle & \textbf{if } \langle B, Xl\rangle \to \langle \texttt{bool}\rangle \wedge \langle S_1, Xl\rangle \to \langle \texttt{block}\rangle \wedge \langle S_2, Xl\rangle \to \langle \texttt{block}\rangle \\
\langle \texttt{while}\,(B)\, S, Xl\rangle \to \langle \texttt{stmt}\rangle & \textbf{if } \langle B, Xl\rangle \to \langle \texttt{bool}\rangle \wedge \langle S, Xl\rangle \to \langle \texttt{block}\rangle \\
\langle \texttt{int}\; Xl\texttt{;}\; S\rangle \to \langle \texttt{pgm}\rangle & \textbf{if } \langle S, Xl\rangle \to \langle ST\rangle
\end{array}
$$

Figure 3.11: $\mathcal{R}_{\text{BigStepTypeSystem(IMP)}}$: type system of IMP using big-step SOS in rewrite logic. Assume a sort *Type* with constants `int`, `bool`, and `pgm`, and a subsort of it *StmtType* with constants `block` and `stmt`. The sorts of the involved variables is understood (from Figure 3.10), except for T which has the sort *Bool* and for ST, ST_1 and ST_2 which have the sort *StmtType*.

Figure 3.11 shows a translation of the big-step SOS in Figure 3.10 into a rewrite logic theory, following the general technique described in Section 3.2.3 but with two simple optimizations explained shortly. This translation is based on the reinterpretation of type judgments as big-step SOS sequents mentioned above. The following configurations are used in the rewrite theory in Figure 3.11:

- $\langle A, Xl\rangle$ grouping arithmetic expressions A and variable lists Xl;
- $\langle B, Xl\rangle$ grouping Boolean expressions B and variable lists Xl;
- $\langle S, Xl\rangle$ grouping statements S and variable lists Xl;
- $\langle P\rangle$ holding programs P;
- $\langle \tau\rangle$ holding types τ, which can be `int`, `bool`, `block`, `stmt`, or `pgm`.

The two optimizations refer to how to test the various memberships that occur in some of the rules' side conditions. One optimization is to use associative matching to check membership to a list; this applies to the rules (BigStepTypeSystem-Lookup) and (BigStepTypeSystem-Asgn). Another optimization is to use a (sub)sort as an umbrella for a set of terms of interest (constants in our case); for example, we let T be a variable of sort *Bool* (i.e., T can match both `true`and `false`) when defining the rewrite rule corresponding to (BigStepTypeSystem-Bool), and we let ST, ST_1, and ST_2 be variables ranging over a new subsort *StmtType* of *Type* when defining the rewrite rules corresponding to (BigStepTypeSystem-Block), (BigStepTypeSystem-Seq) and (BigStepTypeSystem-Pgm).

By Corollary 3 we have that a program p is well-typed, that is, $\vdash\ p : \texttt{pgm}$ is derivable with the proof system in Figure 3.10, if and only if $\mathcal{R}_{\text{BigStepTypeSystem(IMP)}} \vdash \langle \overline{p}\rangle \to \langle \texttt{pgm}\rangle$.

```
mod IMP-TYPES is
  sorts StmtType Type .  subsort StmtType < Type .
  ops block stmt : -> StmtType .
  ops int bool pgm : -> Type .
endm

mod IMP-TYPE-SYSTEM-CONFIGURATIONS-BIGSTEP is including IMP-SYNTAX + IMP-TYPES .
  sort Configuration .
  op <_,_> : AExp List{Id} -> Configuration .
  op <_,_> : BExp List{Id} -> Configuration .
  op <_,_> : Stmt List{Id} -> Configuration .
  op   <_> : Pgm -> Configuration .
  op   <_> : Type -> Configuration .
endm

mod IMP-TYPE-SYSTEM-BIGSTEP is including IMP-TYPE-SYSTEM-CONFIGURATIONS-BIGSTEP .
  var X : Id .  var Xl Xl' : List{Id} .  var I : Int .  var T : Bool .
  var A A1 A2 : AExp .  var B B1 B2 : BExp .  var S S1 S2 : Stmt .
  var ST ST1 ST2 : StmtType .

  rl < I,Xl > => < int > .
  rl < X,(Xl,X,Xl') > => < int > .
 crl < A1 + A2,Xl > => < int >
  if < A1,Xl > => < int > /\ < A2,Xl > => < int > .
 crl < A1 / A2,Xl > => < int >
  if < A1,Xl > => < int > /\ < A2,Xl > => < int > .

  rl < T,Xl > => < bool > .
 crl < A1 <= A2,Xl > => < bool >
  if < A1,Xl > => < int > /\ < A2,Xl > => < int > .
 crl < ! B,Xl > => < bool >
  if < B,Xl > => < bool > .
 crl < B1 && B2,Xl > => < bool >
  if < B1,Xl > => < bool > /\ < B2,Xl > => < bool > .

  rl < {},Xl > => < block > .
 crl < {S},Xl > => < block >
  if < S,Xl > => < ST > .
 crl < X = A ;,(Xl,X,Xl') > => < stmt >
  if < A,(Xl,X,Xl') > => < int > .
 crl < S1 S2,Xl > => < stmt >
  if < S1,Xl > => < ST1 > /\ < S2,Xl > => < ST2 > .
 crl < if (B) S1 else S2,Xl > => < stmt >
  if < B,Xl > => < bool > /\ < S1,Xl > => < block > /\ < S2,Xl > => < block > .
 crl < while (B) S,Xl > => < stmt >
  if < B,Xl > => < bool > /\ < S,Xl > => < block > .

 crl < int Xl ; S > => < pgm >
  if < S,Xl > => < ST > .
endm
```

Figure 3.12: The type-system of IMP using big-step SOS in Maude, including the definition of types and configurations.

★ Maude Definition of a Type System for IMP using Big-Step SOS

Figure 3.12 shows the Maude representation of the rewrite theory $\mathcal{R}_{\text{BigStepTypeSystem(IMP)}}$ in Figure 3.11, including a representation of the algebraic signature for the needed configurations. The Maude module `IMP-TYPE-SYSTEM-BIGSTEP` in Figure 3.12 is executable, so Maude, through its rewriting capabilities, yields a type checker for IMP; for example, the command

```
rewrite < sumPgm > .
```

where `sumPgm` is the first program defined in the module `IMP-PROGRAMS` in Figure 3.4, produces a result of the form (the exact statistics are irrelevant here, so they were replaced by "...”):

```
rewrites: 20 in ... cpu (... real) (... rewrites/second)
result Configuration: < pgm >
```

A type system is generally expected to be deterministic. Nevertheless, implementations of it (particularly rewrite-based ones) may mistakenly be non-deterministic (non-confluent; see Section 2.1.4). To gain confidence in the determinism of the Maude definition in Figure 3.12, one may exhaustively search for all possible behaviors yielded by the type checker:

```
search < sumPgm > =>! Cfg:Configuration .
```

As expected, this finds only one solution. This Maude definition of IMP's type checker is very simple and one can easily see that it is confluent (it is orthogonal—see Section 2.1.4), so the search is redundant. However, the search command may be useful for testing more complex type systems.

3.2.5 Notes

Big-step structural operational semantics (big-step SOS) was introduced under the name *natural semantics* by Kahn [34] in 1987. Even though he introduced it in the limited context of defining Mini-ML, a simple pure (no side effects) version of the ML language, Kahn's aim was to propose natural semantics as a "unified manner to present different aspects of the semantics of programming languages, such as dynamic semantics, static semantics and translation" (Section 1.1 in [34]). Kahn's original notation for big-step sequents was $\sigma \vdash a \Rightarrow i$, with the meaning that a evaluates to i in state (or environment) σ. Kahn, like many others after him (including ourselves; e.g., Section 3.2.4), took the freedom to using a different notation for type judgments, namely $\Gamma \vdash c : \tau$, where Γ is a type environment, c is a program or fragment of program, and τ is a type. This colon notation for type judgments was already established in 1987; however, Kahn noticed that the way type systems were defined was a special instance of a more general schema, which he called natural semantics (and which is called big-step SOS here and in many other places). Big-step SOS is very natural when defining pure, deterministic and structured languages, so it quickly became very popular. However, Kahn's terminology for "natural" semantics was inspired from its reminiscence to "natural deduction" in mathematical logic, not necessarily from the fact that it is natural to use. He demonstrated how one can use big-step SOS to define all these in his seminal paper [34], using the Mini-ML language.

As Kahn himself acknowledged, the idea of using proof systems to capture the operational semantics of programming languages goes back to Plotkin [60, 61] in 1981. Plotkin was the first to coin the terminology *structural operational semantics* (S0S), but what he meant by that was mostly what we call today *small-step* structural operational semantics (small-step SOS). Note, however, that Plotkin in fact used a combination of small-step and big-step SOS, without calling them as such, using the $\rightarrow$ arrow for small-steps and its transitive closure $\rightarrow^*$ for big-steps. We will discuss small-step SOS in depth in Section 3.3. Kahn and others found big-step SOS more natural and convenient than Plotkin's SOS, essentially because it is more abstract

and denotational in nature (which may help in formal reasoning), and one needs fewer rules to define a language semantics.

One of the most notable uses of natural semantics is the formal semantics of Standard ML by Milner *et al.* [49]. Several types of big-step sequents were used in [49], such as $\rho \vdash p \Rightarrow v/f$ for "in environment ρ, sentence p either evaluates to value v or otherwise an error or failure f takes place", and $\sigma, \rho \vdash p \Rightarrow v, \sigma'$ for "in state σ and environment ρ, sentence p evaluates to v and the resulting state is σ'", and $\rho, v \vdash m \Rightarrow v'/f$ for "in environment ρ, a match m either evaluates to v' or otherwise failure f", among many others. After more than twenty years of natural semantics, it is now common wisdom that big-step semantics is inappropriate as a rigorous formalism for defining languages with complex features such as exceptions or concurrency. To give a reasonably compact and readable definition of Standard ML in [49], Milner *et al.* had to make several informal notational conventions, such as a "state convention" to avoid having to mention the state in every rule, and an "exception convention" to avoid having to more than double the number of rules for the sole purpose of supporting exceptions. As rightfully noticed by Mosses [53], such conventions are not only adhoc and language specific, but may also lead to erroneous definitions. Section 3.5 illustrates in detail the limitations of big-step operational semantics, both with respect to its incapacity of defining certain rather simple language features and with respect to inconvenience in using it (for example, due to its lack of modularity). One of the common uses of natural semantics these days is to define static semantics of programming languages and calculi, such as type systems (see Section 3.2.4).

Hennessy [32] (1990) and Winskel [87] (1993) are perhaps the first textbooks proposing big-step SOS in teaching programming language semantics. They define big-step SOS for several simple languages, including ones similar to the IMP language presented in this chapter. Hennessy [32] defines languages incrementally, starting with a small core and then adding new features one by one, highlighting a major problem with big-step SOS: its lack of modularity. Indeed, the big-step SOS of a language is entirely redefined several times in [32] as new features are added to the language, because adding new features requires changes in the structure of judgments. For example, some big-step SOS judgments in [32] evolve from $a \Rightarrow i$, to $\sigma \vdash a \Rightarrow i$, to $D, \sigma \vdash i$ during the language design experiment, where a is an expression, i an integer, σ a state (or environment), and D a set of function definitions.

While the notations of Hennessy [32] and of Milner *et al.* [49] are somehow reminiscent of original Kahn's notation, Winskel [87] uses a completely different notation. Specifically, he prefers to use big-step sequents of the form $\langle a, \sigma \rangle \to i$ instead of $\sigma \vdash a \Rightarrow i$. There seems to be, unfortunately, no uniform and/or broadly accepted notation for SOS sequents in the literature, be they big-step or small-step. As already explained earlier in this section, for the sake of uniformity at least throughout this book, we will make an effort to consider sequents of the form $C \Downarrow R$ in our big-step SOS definitions, where C and R are configurations. Similarly, we will make an effort to use the notation $C \to C'$ for small-step sequents (see Section 3.3). We will make it explicit when we deviate from our uniform notation, explaining how the temporary notation relates to the uniform one, as we did in Section 3.2.4.

Big-step SOS is the semantic approach which is probably the easiest to implement in any language or to represent in any computational logic. There are countless approaches to implementing or encoding big-step SOS in various languages or logics, which we cannot enumerate here. We only mention rewriting-based ones which are directly related to the approach followed in this book. Vardejo and Martí-Oliet [84] proposed big-step SOS implementations in Maude for several languages, including Hennessy's languages [32] and Kahn's Mini-ML [34]. Vardejo and Martí-Oliet were mainly interested in demonstrating the strengths of Maude 2 to give executable semantics to concrete languages, rather than in proposing general representations of big-step SOS into rewrite logic that work for any language. Besides Vardejo and Martí-Oliet, several other authors used rewrite logic and Maude to define and implement language semantics for languages or calculi

following a small-step approach. These are discussed in Section 3.3.4; we here only emphasize that most of those can likely be adapted into big-step SOS definitional or implementation styles, because big-step SOS can be regarded as a special case of small-step SOS, one in which the small step is "big".

There is a common misconception that big-step SOS is "efficient" when executed. This misconception is fueled by case studies where efficient interpreters for various (deterministic or deterministic fragments of) languages, were more or less mechanically derived from the big-step SOS of those languages. Recall that a big-step SOS is a formal proof system, and that proof systems are typically meant to tell *what* is possible in the defined language, calculus or system, and not *how* to implement it. A big-step SOS *can* indeed be used as a basis to develop efficient interpreters, but one should be aware of the fact that when that happens it happens either because the big-step SOS has a particularly convenient form, where at most one proof rule can apply at any given moment[4], or because one cuts corners in the implementation by deliberately[5] ignoring the proof search process needed to detect which proof rule applies, and instead arbitrarily picking one matching rule and stopping the execution if its premises cannot be proved. Section 3.5 illustrates a big-step SOS whose rules are not syntax-driven (e.g., expressions have side effects and arithmetic operators are non-deterministic); as discussed there, its faithful Maude implementation is indeed very slow, requiring exponential complexity in some cases to derive the rule premises.

3.2.6 Exercises

Prove the following exercises, all referring to the IMP big-step SOS in Figure 3.7.

Exercise 54. *Change the rule* (BigStep-Div) *so that division short-circuits when* a_1 *evaluates to* 0. *(Hint: may need to replace it with two rules, like for the semantics of conjunction).*

Exercise 55. *Change the big-step semantics of the* IMP *conjunction so that it is not short-circuited.*

Exercise 56. *Add an "error" state and modify the big-step semantics in Figure 3.7 to allow derivations of sequents of the form* $\langle s, \sigma\rangle \Downarrow \langle error\rangle$ *or* $\langle p\rangle \Downarrow \langle error\rangle$ *when s evaluated in state* σ *or when p evaluated in the initial state performs a division by zero.*

Exercise 57. *Prove that the transition relation defined by the* BigStep(IMP) *proof system in Figure 3.7 is* ***deterministic****, that is:*

- *If* BigStep(IMP) $\vdash \langle a, \sigma\rangle \Downarrow \langle i\rangle$ *and* BigStep(IMP) $\vdash \langle a, \sigma\rangle \Downarrow \langle i'\rangle$ *then* $i = i'$*;*
- *If* BigStep(IMP) $\vdash \langle b, \sigma\rangle \Downarrow \langle t\rangle$ *and* BigStep(IMP) $\vdash \langle b, \sigma\rangle \Downarrow \langle t'\rangle$ *then* $t = t'$*;*
- *If s terminates in* σ *then there is a unique* σ' *such that* BigStep(IMP) $\vdash \langle s, \sigma\rangle \Downarrow \langle \sigma'\rangle$*;*
- *If p terminates then there is a unique* σ *such that* BigStep(IMP) $\vdash \langle p\rangle \Downarrow \langle \sigma\rangle$.

Prove the same results above for the proof system detecting division-by-zero as in Exercise 56.

Exercise 58. *Show that there is no algorithm, based on the big-step proof system in Figure 3.7 or on anything else, which takes as input an* IMP *program and says whether it terminates* ***or not****.*

[4]We call such big-step SOS definitions *syntax-driven*.

[5]Either because one proved that this is sound, or because one's intention is to trade soundness for performance.

Is this enough? Should I add more citations here? Ask Narciso, Peter and Jose for their and others' works embedding big-step into rewrite logic.

change =/=Bool in =/=Int, etc.

make sure only the IMP macros are used above, e.g., plusIMP, etc.

3.3 Small-Step Structural Operational Semantics (Small-Step SOS)

Known also under the names *transition semantics*, *reduction semantics*, *one-step operational semantics*, and *computational semantics*, small-step structural operational semantics, or small-step SOS for short, formally captures the intuitive notion of one atomic computational step. Unlike in big-step SOS where one defines all computation steps in one transition, in a small-step SOS a transition encodes only one computation step. To distinguish small-step from big-step transitions, we use a plain arrow $\rightarrow$ instead of $\Downarrow$. To execute a small-step SOS, or to relate it to a big-step SOS, we need to transitively close the small-step transition relation. Indeed, the conceptual relationship between big-step SOS and small-step SOS is that for any configuration C and any result configuration R, $C \Downarrow R$ if and only if $C \rightarrow^{\star} R$. Small-step SOS is typically preferred over big-step SOS when defining languages with a high-degree of non-determinism, such as, for example, concurrent languages, because in a small-step semantics one has direct control over what can execute and when.

Like big-step SOS, a small-step SOS of a programming language or calculus is also given as a formal proof system (see Section 2.1.5). The *small-step SOS sequents* are also binary relations over configurations like in big-step SOS, but in small-step SOS they are written $C \rightarrow C'$ and have the meaning that C' is a configuration obtained from C after *one step* of computation. A *small-step SOS rule* therefore has the form

$$\frac{C_1 \rightarrow C_1' \quad C_2 \rightarrow C_2' \quad \ldots \quad C_n \rightarrow C_n'}{C_0 \rightarrow C_0'} \quad [\text{if } \textit{condition}]$$

where $C_0, C_0', C_1, C_1', C_2, C_2', \ldots, C_n, C_n'$ are configurations holding fragments of program together with all the needed semantic components, like in big-step SOS, and *condition* is an optional side condition. Unlike in big-step SOS, the result configurations do not need to be explicitly defined. In small-step SOS they are implicit: they are precisely those configurations which cannot be reduced anymore using the one-step relation.

Given a configuration holding a fragment of program, a small-step of computation typically takes place in some subpart of the fragment. However, when each of the subparts is already reduced, then the small-step can apply on the part itself. A small-step SOS is therefore finer-grain than big-step SOS, and thus more verbose, because one has to cover all the cases where a computation step can take place. For example, the small-step SOS of addition in IMP is

$$\frac{\langle a_1, \sigma\rangle \rightarrow \langle a_1', \sigma\rangle}{\langle a_1 + a_2, \sigma\rangle \rightarrow \langle a_1' + a_2, \sigma\rangle}$$

$$\frac{\langle a_2, \sigma\rangle \rightarrow \langle a_2', \sigma\rangle}{\langle a_1 + a_2, \sigma\rangle \rightarrow \langle a_1 + a_2', \sigma\rangle}$$

$$\langle i_1 + i_2, \sigma\rangle \rightarrow \langle i_1 +_{Int} i_2, \sigma\rangle$$

Here, the meaning of a relation $\langle a, \sigma\rangle \rightarrow \langle a', \sigma\rangle$ is that arithmetic expression a in state σ is reduced, in one small-step, to arithmetic expression a' and the state stays unchanged. Like for big-step SOS, one can encounter various other notations for small-step SOS configurations in the literature, e.g., $[a, \sigma]$, or (a, σ), or $\{a, \sigma\}$, or $\langle a \mid \sigma\rangle$, etc. Like for big-step SOS, we prefer to uniformly use the angle-bracket-and-comma notation $\langle a, \sigma\rangle$. Also, like for big-step SOS, one can encounter various decorations on the transition arrow $\rightarrow$, a notable situation being when the transition is *labeled*. Again like for big-step SOS, we assume that such transition decorations are incorporated in the (source and/or target) configurations. How this can be effectively achieved is discussed in detail in Section 3.6 in the context of modular SOS (which allows rather complex transition labels).

The rules above rely on the fact that expression evaluation in IMP has no side effects. If there were side effects, like in the IMP extension in Section 3.5, then the σ's in the right-hand-side configurations above

would need to change to a different symbol, say σ', to account for the possibility that the small-step in the condition of the rules, and implicitly in their conclusion, may change the state as well. While in big-step SOS it is more common to derive sequents of the form $\langle a, \sigma \rangle \Downarrow \langle i \rangle$ than of the form $\langle a, \sigma \rangle \Downarrow \langle i, \sigma \rangle$, in small-step SOS the opposite tends to be norm, that is, it is more common to derive sequents of the form $\langle a, \sigma \rangle \rightarrow \langle a', \sigma \rangle$ than of the form $\langle a, \sigma \rangle \rightarrow \langle a' \rangle$. Nevertheless, the latter sequent type also works when defining languages like IMP whose expressions are side-effect-free (see Exercise 68). Some language designers may prefer this latter style, to keep sequents minimal. However, even if one prefers these simpler sequents, we still keep the angle brackets in the right-hand-sides of the transition relations (for the same reason like in big-step SOS—to maintain a uniform notation); in other words, we write $\langle a, \sigma \rangle \rightarrow \langle a' \rangle$ and not $\langle a, \sigma \rangle \rightarrow a'$.

In addition to rules, a small-step SOS may also include *structural identities*. For example, we can state that sequential composition is associative using the following structural identity:

$$(s_1 \ s_2) \ s_3 \equiv s_1 \ (s_2 \ s_3)$$

The small-step SOS rules apply *modulo* structural identities. In other words, the structural identities can be used anywhere in any configuration and at any moment during the derivation process, without counting as computational steps. In practice, they are typically used to rearrange the syntactic term so that some small-step SOS rule can apply. In particular, the structural rule above allows the designer of the small-step SOS to rely on the fact that the first statement in a sequential composition is *not* a sequential composition, which may simplify the actual SOS rules; this is indeed the case in Section 3.5.4, where we extend IMP with dynamic threads (we do not need structural identities in the small-step SOS definition of the simple IMP language in this section). Structural identities are not easy to execute and/or implement in their full generality, because they can quickly yield an exponential explosion in the number of terms that need to be matched by rules. Their role in SOS is the same as the role of equations in rewriting logic definitions; in fact, we effectively turn them into equations when we embed small-step SOS into rewrite logic (see Section 3.3.3).

3.3.1 IMP Configurations for Small-Step SOS

The configurations needed for the small-step SOS of IMP are a subset of those needed for its big-step SOS in Section 3.2.1. Indeed, we still need all the two-component configurations containing a fragment of program and a state, but, for the particular small-step SOS style that we follow in this section, we do not need those result configurations of big-step SOS containing only a value or only a state. If one prefers to instead follow the minimalist style as in Exercise 68, then one would also need the other configuration types. Here are all the configuration types needed for the small-step SOS of IMP given in the remainder of this section:

- $\langle a, \sigma \rangle$ grouping arithmetic expressions a and states σ;
- $\langle b, \sigma \rangle$ grouping Boolean expressions b and states σ;
- $\langle s, \sigma \rangle$ grouping statements s and states σ;
- $\langle p \rangle$ holding programs p.

We still need the one-component configuration holding only a program, because we still want to reduce a program in the default initial state (empty) without having to mention the empty state.

sorts:
Configuration
operations:

$$\begin{array}{rcl} \langle _, _ \rangle & : & \mathit{AExp} \times \mathit{State} \to \mathit{Configuration} \\ \langle _, _ \rangle & : & \mathit{BExp} \times \mathit{State} \to \mathit{Configuration} \\ \langle _, _ \rangle & : & \mathit{Stmt} \times \mathit{State} \to \mathit{Configuration} \\ \langle _ \rangle & : & \mathit{Pgm} \to \mathit{Configuration} \end{array}$$

Figure 3.13: IMP small-step SOS configurations as an algebraic signature.

IMP Small-Step SOS Configurations as an Algebraic Signature

Figure 3.13 shows an algebraic signature defining the IMP configurations above, which is needed for the subsequent small-step operational semantics. We defined this algebraic signature in the same style and following the same assumptions as those for big-step SOS in Section 3.2.1.

3.3.2 The Small-Step SOS Rules of IMP

Figures 3.14 and 3.15 show all the rules in our IMP small-step SOS proof system, the former showing the rules corresponding to expressions, both arithmetic and Boolean, and the latter showing those rules corresponding to statements. The sequents that this proof system derives have the forms $\langle a, \sigma \rangle \to \langle a', \sigma \rangle$, $\langle b, \sigma \rangle \to \langle b', \sigma \rangle$, $\langle s, \sigma \rangle \to \langle s', \sigma' \rangle$, and $\langle p \rangle \to \langle s, \sigma \rangle$, where a ranges over *AExp*, b over *BExp*, s over *Stmt*, p over *Pgm*, and σ and σ' over *State*.

The meaning of $\langle a, \sigma \rangle \to \langle a', \sigma \rangle$ is that given state σ, the arithmetic expression a reduces in one (small) step to the arithmetic expression a' and the state σ stays unchanged. The meaning of $\langle b, \sigma \rangle \to \langle b', \sigma \rangle$ is similar but with Boolean expressions instead of arithmetic expressions. The meaning of $\langle s, \sigma \rangle \to \langle s', \sigma' \rangle$ is that statement s in state σ reduces in one step to statement s' in a potentially modified state σ'. The meaning of $\langle p \rangle \to \langle s, \sigma \rangle$ is that program p reduces in one step to statement s in state σ (as expected, whenever such sequents can be derived, the statement s is the body of p and the state σ initializes to 0 the variables declared by p). The reason for which the state stays unchanged in the sequents corresponding to arithmetic and Boolean expressions is because, as discussed, IMP's expressions currently have no side effects; we will have to change these rules later on in Section 3.5 when we add a variable increment arithmetic expression construct to IMP. A small-step reduction of a statement may or may not change the state, so we use a different symbol in the right-hand-side of statement transitions, σ', to cover both cases.

We next discuss each of the small-step SOS rules of IMP in Figures 3.14 and 3.15. Before we start, note that we have no rules for reducing constant (integer or Boolean) expressions to their corresponding values as we had in big-step SOS (i.e., no rules corresponding to (BigStep-Int) and (BigStep-Bool) in Figure 3.7). Indeed, we do not want to have small-step rules of the form $\langle v, \sigma \rangle \to \langle v, \sigma \rangle$ because no one-step reductions are further desired on values v: adding such rules would lead to undesired divergent SOS reductions later on when we consider the transitive closure of the one-step relation $\to$. Recall that the big-step SOS relation $\Downarrow$ captured all the reduction steps at once, including zero steps, and thus it did not need to be transitively closed like the small-step relation $\to$, so evaluating values to themselves was not problematic in big-step SOS.

The rule (SmallStep-Lookup) happens to be almost the same as in big-step SOS; that's because variable lookup is an atomic-step operation both in big-step and in small-step SOS. The rules (SmallStep-Add-Arg1), (SmallStep-Add-Arg2), and (SmallStep-Add) give the small-step semantics of addition, and (SmallStep-Div-Arg1), (SmallStep-Div-Arg2), and (SmallStep-Div) that of division, each covering all the three cases where

$$\langle x, \sigma \rangle \to \langle \sigma(x), \sigma \rangle \quad \text{if } \sigma(x) \neq \bot \qquad \text{(SmallStep-Lookup)}$$

$$\frac{\langle a_1, \sigma \rangle \to \langle a_1', \sigma \rangle}{\langle a_1 + a_2, \sigma \rangle \to \langle a_1' + a_2, \sigma \rangle} \qquad \text{(SmallStep-Add-Arg1)}$$

$$\frac{\langle a_2, \sigma \rangle \to \langle a_2', \sigma \rangle}{\langle a_1 + a_2, \sigma \rangle \to \langle a_1 + a_2', \sigma \rangle} \qquad \text{(SmallStep-Add-Arg2)}$$

$$\langle i_1 + i_2, \sigma \rangle \to \langle i_1 +_{Int} i_2, \sigma \rangle \qquad \text{(SmallStep-Add)}$$

$$\frac{\langle a_1, \sigma \rangle \to \langle a_1', \sigma \rangle}{\langle a_1 \ / \ a_2, \sigma \rangle \to \langle a_1' \ / \ a_2, \sigma \rangle} \qquad \text{(SmallStep-Div-Arg1)}$$

$$\frac{\langle a_2, \sigma \rangle \to \langle a_2', \sigma \rangle}{\langle a_1 \ / \ a_2, \sigma \rangle \to \langle a_1 \ / \ a_2', \sigma \rangle} \qquad \text{(SmallStep-Div-Arg2)}$$

$$\langle i_1 \ / \ i_2, \sigma \rangle \to \langle i_1 \ /_{Int} \ i_2, \sigma \rangle \quad \text{if } i_2 \neq 0 \qquad \text{(SmallStep-Div)}$$

$$\frac{\langle a_1, \sigma \rangle \to \langle a_1', \sigma \rangle}{\langle a_1 <= a_2, \sigma \rangle \to \langle a_1' <= a_2, \sigma \rangle} \qquad \text{(SmallStep-Leq-Arg1)}$$

$$\frac{\langle a_2, \sigma \rangle \to \langle a_2', \sigma \rangle}{\langle i_1 <= a_2, \sigma \rangle \to \langle i_1 <= a_2', \sigma \rangle} \qquad \text{(SmallStep-Leq-Arg2)}$$

$$\langle i_1 <= i_2, \sigma \rangle \to \langle i_1 \leq_{Int} i_2, \sigma \rangle \qquad \text{(SmallStep-Leq)}$$

$$\frac{\langle b, \sigma \rangle \to \langle b', \sigma \rangle}{\langle \, ! \ b, \sigma \rangle \to \langle \, ! \ b', \sigma \rangle} \qquad \text{(SmallStep-Not-Arg)}$$

$$\langle \, ! \ \texttt{true}, \sigma \rangle \to \langle \texttt{false}, \sigma \rangle \qquad \text{(SmallStep-Not-True)}$$

$$\langle \, ! \ \texttt{false}, \sigma \rangle \to \langle \texttt{true}, \sigma \rangle \qquad \text{(SmallStep-Not-False)}$$

$$\frac{\langle b_1, \sigma \rangle \to \langle b_1', \sigma \rangle}{\langle b_1 \ \&\& \ b_2, \sigma \rangle \to \langle b_1' \ \&\& \ b_2, \sigma \rangle} \qquad \text{(SmallStep-And-Arg1)}$$

$$\langle \texttt{false} \ \&\& \ b_2, \sigma \rangle \to \langle \texttt{false}, \sigma \rangle \qquad \text{(SmallStep-And-False)}$$

$$\langle \texttt{true} \ \&\& \ b_2, \sigma \rangle \to \langle b_2, \sigma \rangle \qquad \text{(SmallStep-And-True)}$$

Figure 3.14: SmallStep(IMP)— Small-step SOS of IMP expressions ($i_1, i_2 \in Int$; $x \in Id$; $a_1, a_1', a_2, a_2' \in AExp$; $b, b', b_1, b_1', b_2 \in BExp$; $\sigma \in State$).

a small-step reduction can take place. The first two cases in each group may apply non-deterministically. Recall from Section 3.2 that big-step SOS was inappropriate for defining the desired non-deterministic evaluation strategies for `+` and `/`. Fortunately, that was not a big problem for IMP, because its intended non-deterministic constructs are side-effect free. Therefore, the intended non-deterministic evaluation strategies of these particular language constructs did not affect the overall determinism of the IMP language, thus making its deterministic (see Exercise 57) big-step SOS definition in Section 3.2 acceptable. As expected, the non-deterministic evaluation strategies of `+` and `/`, which this time can be appropriately captured within the small-step SOS, will not affect the overall determinism of the IMP language (that is, the reflexive/transitive closure $\rightarrow^\star$ of $\rightarrow$; see Theorem 14). These will start making a difference when we add side effects to expressions in Section 3.5.

The rules (SmallStep-Leq-Arg1), (SmallStep-Leq-Arg2), and (SmallStep-Leq) give the deterministic, sequential small-step SOS of <=. The first rule applies whenever a_1 is not an integer, then the second rule applies when a_1 is an integer but a_2 is not an integer, and finally, when both a_1 and a_2 are integers, the third rule applies. The rules (SmallStep-Not-Arg), (SmallStep-Not-True), and (SmallStep-Not-False) are self-explanatory, while the rules (SmallStep-And-Arg1), (SmallStep-And-False) and (SmallStep-And-True) give the short-circuited semantics of `and`: indeed, b_2 will not be reduced unless b_1 is first reduced to `true`.

Before we continue with the remaining small-step SOS rules for statements, let us see an example of a small-step SOS reduction using the rules discussed so far; as in the case of the big-step SOS rules in Section 3.2, recall that the small-step SOS rules are also rule schemas, that is, they are parametric in the (meta-)variables a, a_1, b, s, σ, etc. The following is a correct derivation, where `x` and `y` are program variables and σ is any state with $\sigma(\texttt{x}) = 1$:

$$\dfrac{\dfrac{\dfrac{\dfrac{\cdot}{\langle \texttt{x}, \sigma\rangle \rightarrow \langle 1, \sigma\rangle}}{\langle \texttt{y} \ / \ \texttt{x}, \sigma\rangle \rightarrow \langle \texttt{y} \ / \ 1, \sigma\rangle}}{\langle \texttt{x} + (\texttt{y} \ / \ \texttt{x}), \sigma\rangle \rightarrow \langle \texttt{x} + (\texttt{y} \ / \ 1), \sigma\rangle}}{\langle (\texttt{x} + (\texttt{y} \ / \ \texttt{x})) <= \texttt{x}, \sigma\rangle \rightarrow \langle (\texttt{x} + (\texttt{y} \ / \ 1)) <= \texttt{x}, \sigma\rangle}$$

The above can be regarded as a proof of the fact that replacing the second occurrence of `x` by 1 is a correct one-step computation of IMP, as defined using the small-step SOS rules discussed so far.

Let us now discuss the small-step SOS rules of statements in Figure 3.15. Unlike the reduction of expressions, a reduction step of a statement may also change the state. Note that the empty block, {}, acts as a value that statements evaluate to. Like for other values (integers and Booleans), there is no rule for the empty block, because such a rule would lead to non-termination. Rule (SmallStep-Block) simply dissolves the block construct and keeps the inner statement. We can afford to do this in IMP only because its blocks currently have no local variable declarations (this will change in IMP++ in Section 3.5). An alternative could be to keep the block and instead advance the inner statement (see Exercise 61). Rule (SmallStep-Asgn-Arg2) reduces the second argument—which is an arithmetic expression—of an assignment statement whenever possible, regardless of whether the assigned variable was declared or not. Exercise 62 proposes an alternative semantics where the arithmetic expression is only reduced when the assigned variable has been declared. When the second argument is already fully reduced (i.e., it is an integer value), the rule (SmallStep-Asgn) reduces the assignment statement to {}, at the same time updating the state accordingly. Therefore, two steps are needed in order to assign an already evaluated expression to a variable: one step to write the variable in the state and modify the assignment to {}, and another step to dissolve the resulting {}. Exercise 63 proposes an alternative semantics where these operations can be done in one step.

The rules (SmallStep-Seq-Arg1) and (SmallStep-Seq-Empty-Block) give the small-step SOS of sequential composition: if the first statement is reducible then reduce it, otherwise, if it is {}, move on in a

$$\langle \{\, s \,\}, \sigma \rangle \to \langle s, \sigma \rangle \qquad \text{(SmallStep-Block)}$$

$$\frac{\langle a, \sigma \rangle \to \langle a', \sigma \rangle}{\langle x = a\,\texttt{;}, \sigma \rangle \to \langle x = a'\,\texttt{;}, \sigma \rangle} \qquad \text{(SmallStep-Asgn-Arg2)}$$

$$\langle x = i\,\texttt{;}, \sigma \rangle \to \langle \{\}, \sigma[i/x] \rangle \quad \text{if } \sigma(x) \neq \bot \qquad \text{(SmallStep-Asgn)}$$

$$\frac{\langle s_1, \sigma \rangle \to \langle s_1', \sigma' \rangle}{\langle s_1\ \ s_2, \sigma \rangle \to \langle s_1'\ \ s_2, \sigma' \rangle} \qquad \text{(SmallStep-Seq-Arg1)}$$

$$\langle \{\}\ \ s_2, \sigma \rangle \to \langle s_2, \sigma \rangle \qquad \text{(SmallStep-Seq-Empty-Block)}$$

$$\frac{\langle b, \sigma \rangle \to \langle b', \sigma \rangle}{\langle \texttt{if}\,(b)\ s_1\ \texttt{else}\ s_2, \sigma \rangle \to \langle \texttt{if}\,(b')\ s_1\ \texttt{else}\ s_2, \sigma \rangle} \qquad \text{(SmallStep-If-Arg1)}$$

$$\langle \texttt{if}\,(\texttt{true})\ s_1\ \texttt{else}\ s_2, \sigma \rangle \to \langle s_1, \sigma \rangle \qquad \text{(SmallStep-If-True)}$$

$$\langle \texttt{if}\,(\texttt{false})\ s_1\ \texttt{else}\ s_2, \sigma \rangle \to \langle s_2, \sigma \rangle \qquad \text{(SmallStep-If-False)}$$

$$\langle \texttt{while}\,(b)\ s, \sigma \rangle \to \langle \texttt{if}\,(b)\,\{\, s\ \texttt{while}\,(b)\ s \,\}\,\texttt{else}\,\{\}, \sigma \rangle \qquad \text{(SmallStep-While)}$$

$$\langle \texttt{int}\ xl\texttt{;}\ s \rangle \to \langle s, xl \mapsto 0 \rangle \qquad \text{(SmallStep-Pgm)}$$

Figure 3.15: SmallStep(IMP)— Small-step SOS of IMP statements ($i \in \mathit{Int}$; $x \in \mathit{Id}$; $xl \in \mathbf{List}\{\mathit{Id}\}$; $a, a' \in \mathit{AExp}$; $b, b' \in \mathit{BExp}$; $s, s_1, s_1', s_2 \in \mathit{Stmt}$; $\sigma, \sigma' \in \mathit{State}$).

$$C \to^* C \qquad \text{(SmallStep-Closure-Stop)}$$

$$\frac{C \to C'', \quad C'' \to^* C'}{C \to^* C'} \qquad \text{(SmallStep-Closure-More)}$$

Figure 3.16: SmallStep(IMP)—Reflexive/transitive closure of the small-step SOS relation, which is the same for any small-step SOS of any language or calculus ($C, C', C'' \in$ *Configuration*).

small-step to the second statement. Another possibility (different from that in Exercise 63) to avoid wasting the computational step generated by reductions to {}like in the paragraph above, is to eliminate the rule (SmallStep-Seq-Empty-Block) and instead to add a rule that allows the reduction of the second statement provided that the first one is terminated. This approach is proposed by Hennessy [32], where he introduces a new sequent for *terminated configurations*, say $C\surd$, and then includes a rule like the following (and no rule like (SmallStep-Seq-Empty-Block)):

$$\frac{\langle s_1, \sigma\rangle\surd \quad \langle s_2, \sigma\rangle \to \langle s_2', \sigma'\rangle}{\langle s_1 \;\; s_2, \sigma\rangle \to \langle s_2', \sigma'\rangle}$$

The three rules for the conditional, namely (SmallStep-If-Arg1), (SmallStep-If-True), and (SmallStep-If-False), are straightforward; note that the two branches are never reduced when the condition can still be reduced. Exercise 65 proposes an alternative semantics for the conditional which wastes no computational step on switching to one of the two branches once the condition is evaluated.

The (SmallStep-While) rule unrolls the loop once; this unrolling semantics seems as natural as it can be, but one should notice that it actually also generates an artificial computational step. Exercise 66 proposes an alternative loop semantics which wastes no computational step.

Finally, (SmallStep-Pgm) gives the semantics of programs by reducing them to their body statement in the expected state formed by initializing all the declared variables to 0. Note, however, that this rule also wastes a computational step; indeed, one may not want the initialization of the state with default values for variables to count as a step. Exercise 67 addresses this problem.

It is worthwhile noting that one has some flexibility in how to give a small-step SOS semantics to a language. The same holds true for almost any language definitional style, not only for SOS.

On Proof Derivations, Evaluation, and Termination

To formally capture the notion of "sequence of transitions", in Figure 3.16 we define the relation of *reflexive/transitive closure* of the small-step SOS transition.

Definition 21. *Given appropriate* IMP *small-step SOS configurations C and C', the* IMP *small-step SOS sequent $C \to C'$ is* ***derivable****, written* SmallStep(IMP) $\vdash C \to C'$*, iff there is some proof tree rooted in $C \to C'$ which is derivable using the proof system* SmallStep(IMP) *in Figures 3.14 and 3.15. In this case, we also say that C* ***reduces in one step*** *to C'. Similarly, the many-step sequent $C \to^\star C'$ is* ***derivable****, written* SmallStep(IMP) $\vdash C \to^\star C'$*, iff there is some proof tree rooted in $C \to^\star C'$ which is derivable using the proof system in Figures 3.14, 3.15, and 3.16. In this case, we also say that C* ***reduces*** *(in zero, one, or more steps) to C'. Configuration R is* ***irreducible*** *iff there is no configuration C such that* SmallStep(IMP) $\vdash R \to C$*, and is a* ***result*** *iff it has one of the forms $\langle i, \sigma\rangle$, $\langle t, \sigma\rangle$, or $\langle\{\}, \sigma\rangle$, where $i \in$ Int, $t \in$ Bool, and $\sigma \in$ State. Finally, configuration C* ***terminates*** *under* SmallStep(IMP) *iff there is no infinite sequence of configurations $C_0, C_1, \ldots$ such that $C_0 = C$ and C_i reduces in one step to C_{i+1} for any natural number i.*

Result configurations are irreducible, but there are irreducible configurations which are not necessarily result configurations. For example, the configuration $\langle i \ / \ o, \sigma \rangle$ is irreducible but it is not a result. Like for big-step SOS, to catch division-by-zero within the semantics we need to add special error values/states and propagate them through all the language constructs (see Exercise 70).

The syntax of IMP (Section 3.1.1, Figure 3.1) was deliberately ambiguous with regards to sequential composition, and that was motivated by the fact that the semantics of the language will be given in such a way that the syntactic ambiguity will become irrelevant. We can now rigorously prove that is indeed the case, that is, we can prove properties of the like " $\textsc{SmallStep}(\text{IMP}) \vdash \langle (s_1 \ s_2) \ s_3, \sigma \rangle \to \langle (s_1' \ s_2) \ s_3, \sigma' \rangle$ if and only if $\textsc{SmallStep}(\text{IMP}) \vdash \langle s_1 \ (s_2 \ s_3), \sigma \rangle \to \langle s_1' \ (s_2 \ s_3), \sigma' \rangle$", etc. Exercise 71 discusses several such properties which, together with the fact that the semantics of no language construct is structurally defined in terms of sequential composition, also says that adding the associativity of sequential composition as a structural identity to the small-step SOS of IMP does not change the set of global behaviors of any IMP program (although we have not added it). However, that will not be the case anymore when we extend IMP with dynamic threads in Section 3.5.4, because the semantics of thread spawning will be given making use of sequential composition in such a way that adding this structural identity will be necessary in order to capture the desired set of behaviors.

Note that there are non-terminating sequences which repeat configurations, as well as non-terminating sequences in which all the configurations are distinct; an example of the former is a sequence generated by reducing the statement `while (true)` {}, while an example of the latter is a sequence generated by reducing `while (!(n <= 0)) { n = n + 1; }`. Nevertheless, in the case of IMP, a configuration terminates if and only if it reduces to some irreducible configuration (see Exercise 72). This is not necessarily the case for non-deterministic languages, such as the IMP++ extension in Section 3.5, because reductions of configurations in such language semantics may non-deterministically choose steps that lead to termination, as well as steps that may not lead to termination. In the case of IMP though, the local non-determinism given by rules like (SmallStep-Add-Arg1) and (SmallStep-Add-Arg2) does not affect the overall determinism of the IMP language (Exercise 73).

Relating Big-Step and Small-Step SOS

As expected, the reflexive/transitive closure $\to^\star$ of the small-step SOS relation captures the same complete evaluation meaning as the $\Downarrow$ relation in big-step SOS (Section 3.2). Since for demonstrations reasons we deliberately worked with different result configurations in big-step and in small-step SOS, our theorem below looks slightly involved; if we had the same configurations in both semantics, then the theorem below would simply state "for any configuration C and any result configuration R, $\textsc{SmallStep}(\text{IMP}) \vdash C \to^\star R$ if and only if $\textsc{BigStep}(\text{IMP}) \vdash C \Downarrow R$":

Theorem 14. *The following equivalences hold for any $a \in AExp$, $i \in Int$, $b \in BExp$, $t \in Bool$, $s \in Stmt$, $p \in Pgm$, and $\sigma, \sigma' \in State$:*

- $\textsc{SmallStep}(\text{IMP}) \vdash \langle a, \sigma \rangle \to^\star \langle i, \sigma' \rangle$ *for a state* σ' *iff* $\textsc{BigStep}(\text{IMP}) \vdash \langle a, \sigma \rangle \Downarrow \langle i \rangle$*;*
- $\textsc{SmallStep}(\text{IMP}) \vdash \langle b, \sigma \rangle \to^\star \langle t, \sigma' \rangle$ *for a state* σ' *iff* $\textsc{BigStep}(\text{IMP}) \vdash \langle b, \sigma \rangle \Downarrow \langle t \rangle$*;*
- $\textsc{SmallStep}(\text{IMP}) \vdash \langle s, \sigma \rangle \to^\star \langle \{\}, \sigma' \rangle$ *for a state* σ' *iff* $\textsc{BigStep}(\text{IMP}) \vdash \langle s, \sigma \rangle \Downarrow \langle \sigma' \rangle$*;*
- $\textsc{SmallStep}(\text{IMP}) \vdash \langle p \rangle \to^\star \langle \{\}, \sigma \rangle$ *for a state* σ *iff* $\textsc{BigStep}(\text{IMP}) \vdash \langle p \rangle \Downarrow \langle \sigma \rangle$.

Note that the small-step SOS relation for IMP is a *recursive*, or *decidable* problem: indeed, given configurations C and C', one can use the small-step proof system in Figures 3.14 and 3.15 to exhaustively

check whether indeed $C \to C'$ is derivable or not. Moreover, since the rules for the reflexive/transitive closure relation $\to^\star$ in Figure 3.16 can be used to systematically generate any sequence of reductions, we conclude that the relation $\to^\star$ is *recursively enumerable*. Theorem 14 together with the discussion at the end of Section 3.2.2 and Exercise 58, tell us that $\to^\star$ is properly recursively enumerable, that is, it cannot be recursive. This tells us, in particular, that non-termination of a program p is equivalent to saying that, no matter what the state σ is, SmallStep(IMP) $\vdash \langle p \rangle \to^\star \langle \{\}, \sigma \rangle$ cannot be derived. However, unlike for big-step SOS where nothing else can be said about non-terminating programs, in the case of small-step SOS definitions one can use the small-step relation, $\to$, to observe program executions for any number of steps.

3.3.3 Small-Step SOS in Rewrite Logic

Like for big-step SOS, we can also associate a conditional rewrite rule to each small-step SOS rule and hereby obtain a rewrite logic theory that faithfully (i.e., step-for-step) captures the small-step SOS definition. Additionally, we can associate a rewrite logic equation to each SOS structural identity, because in both cases rules are applied modulo structural identities or equations. An important technical aspect needs to be resolved, though. The rewriting relation of rewrite logic is by its own nature reflexively and transitively closed. On the other hand, the small-step SOS relation is not reflexively and transitively closed by default (its reflexive/transitive closure is typically defined a posteriori, as we did in Figure 3.16). Therefore, we need to devise mechanisms to inhibit rewrite logic's reflexive, transitive and uncontrolled application of rules.

We first show that any small-step SOS, say SmallStep, can be mechanically translated into a rewrite logic theory, say $\mathcal{R}_{\text{SmallStep}}$, in such a way that the corresponding derivation relations are step-for-step equivalent, that is, SmallStep $\vdash C \to C'$ if and only if $\mathcal{R}_{\text{SmallStep}} \vdash \mathcal{R}_{C \to C'}$, where $\mathcal{R}_{C \to C'}$ is the corresponding syntactic translation of the small-step SOS sequent $C \to C'$ into a rewrite logic sequent. Second, we apply our generic translation technique on the small-step SOS formal system SmallStep(IMP) defined in Section 3.3.2 and obtain a rewrite logic semantics of IMP that is step-for-step equivalent to the original small-step SOS of IMP. Finally, we show how $\mathcal{R}_{\text{SmallStep(IMP)}}$ can be seamlessly defined in Maude, thus yielding another interpreter for IMP (in addition to the one similarly obtained from the big-step SOS of IMP in Section 3.2.3).

Faithful Embedding of Small-Step SOS into Rewrite Logic

Like for big-step SOS (Section 3.2.3), to define our translation from small-step SOS to rewrite logic generically, we assume that each parametric configuration C admits an equivalent algebraic variant $\overline{C}$ as a term of sort *Configuration* over an appropriate signature of configurations like the one that we defined for IMP in Section 3.2.1 (see Figure 3.6); moreover, each *parameter* in C (e.g., arithmetic expression $a \in AExp$) gets replaced by a *variable* of corresponding sort in $\overline{C}$ (e.g., variable A of sort $AExp$). Consider now a general-purpose small-step SOS rule of the form

$$\frac{C_1 \to C_1' \quad C_2 \to C_2' \quad \ldots \quad C_n \to C_n'}{C_0 \to C_0'} \ [\text{if } \textit{condition}]$$

where C_0, C_0', C_1, C_1', C_2, C_2', ..., C_n, C_n' are configurations holding fragments of program together with all the needed semantic components, and *condition* is an optional side condition. Before we introduce our transformation, let us first discuss why the same straightforward transformation that we used in the case of big-step SOS,

$$(\forall \mathcal{X})\ \overline{C_0} \to \overline{C_0'} \ \textbf{if}\ \overline{C_1} \to \overline{C_1'} \wedge \overline{C_2} \to \overline{C_2'} \wedge \ldots \wedge \overline{C_n} \to \overline{C_n'}\ [\wedge\ \overline{\textit{condition}}],$$

where $\mathcal{X}$ is the set of parameters, or meta-variables, that occur in the small-step SOS rule, does not work in the case of small-step SOS. For example, with that transformation, the rewrite rules corresponding to the small-step SOS rules of IMP for assignment (SmallStep-Asgn-Arg2) and (SmallStep-Asgn) in Figure 3.15 would be

$$\langle X = A\,\mathbf{;}, \sigma\rangle \to \langle X = A'\,\mathbf{;}, \sigma\rangle \quad \textbf{if} \quad \langle A, \sigma\rangle \to \langle A', \sigma\rangle$$
$$\langle X = I\,\mathbf{;}, \sigma\rangle \to \langle \{\}, \sigma[I/X]\rangle \quad \textbf{if} \quad \sigma(X) \neq \bot$$

The problem with these rules is that the rewrite of a configuration of the form $\langle \mathtt{x} = i\,\mathbf{;}, \sigma\rangle$ for some $\mathtt{x} \in \mathit{Id}$, $i \in \mathit{Int}$ and $\sigma \in \mathit{State}$ may not terminate, applying forever the first rule: in rewrite logic, $\langle i, \sigma\rangle \to \langle i, \sigma\rangle$ because $\to$ is closed under reflexivity. Even if we may somehow solve this reflexivity aspect by defining and then including an additional condition $A \neq A'$, such rules still fail to capture the intended small-step transition, because $\to$ is also closed transitively in rewrite logic, so there could be many small-steps taking place in the condition of the first rule before the rule is applied.

To capture *exactly one step* of reduction, thus avoiding the inherent automatic reflexive and transitive closure of the rewrite relation which is desirable in rewrite logic but not in reduction semantics, we can mark the left-hand-side (or, alternatively, the right-hand-side) configuration in each rewrite sequent to always be distinct from the other one; then each rewrite sequent comprises precisely one step, from a marked to an unmarked configuration (or vice versa). For example, let us place a $\circ$ in front of all the left-hand-side configurations and keep the right-hand-side configurations unchanged. Then the generic small-step SOS rule above translates into the rewrite logic rule

$$(\forall \mathcal{X})\ \circ\overline{C_0} \to \overline{C'_0}\ \ \textbf{if}\ \ \circ\overline{C_1} \to \overline{C'_1}\ \wedge\ \circ\overline{C_2} \to \overline{C'_2}\ \wedge\ \ldots\ \wedge\ \circ\overline{C_n} \to \overline{C'_n}\ [\wedge\ \overline{\mathit{condition}}],$$

where $\mathcal{X}$ is the same as above. One can metaphorically think of a marked configuration $\circ\overline{C}$ as a "hot" configuration that needs to be "cooled down" in one step, while of an unmarked configuration $\overline{C}$ as a cool one. Theorem 15 below states as expected that a small-step SOS sequent $C \to C'$ is derivable if and only if the term $\circ\overline{C}$ rewrites in the corresponding rewrite theory to $\overline{C'}$ (which is a normal form). Thus, to enable the resulting rewrite system on a given configuration, one needs to first mark the configuration to be reduced (by placing a $\circ$ in front of it) and then to let it rewrite to its normal form. Since the one-step reduction always terminates, the corresponding rewrite task also terminates.

If the original small-step SOS had structural identities, then we translate them into equations in a straightforward manner: each identity $t \equiv t'$ is translated into an equation $(\forall \mathcal{X})\ \overline{t} = \overline{t'}$, where $\mathcal{X}$ is the set of meta-variables appearing in the structural identity. The only difference between the original structural identity and the resulting equation is that the meta-variables of the former become variables in the latter. The role of the two is the same in their corresponding frameworks and whatever we can do with one in one framework we can equivalently do with the other in the other framework; consequently, to simplify the notation and the presentation, we will make abstraction of structural identities and equations in our theoretical developments in the remainder of this chapter.

To obtain the reflexive and transitive many-step closure of the small-step SOS relation in the resulting rewrite setting and thus to be able to obtain an interpreter for the defined language when executing the rewrite system, we need to devise some mechanism to iteratively apply the one-step reduction step captured by rewriting as explained above. There could be many ways to do that, but one simple and uniform way is to add a new configuration marker, say $\star\overline{C}$, with the meaning that $\overline{C}$ must be iteratively reduced, small-step after small-step, either forever or until an irreducible configuration is reached. Figure 3.17 shows how one can define both configuration markers algebraically (assuming some existing *Configuration* sort, e.g., the one in Figure 3.13). To distinguish the marked configurations from the usual configurations and to also possibly allow several one-step markers at the same time, e.g., $\circ\circ\circ\overline{C}$, which could be useful for debugging/tracing

sorts:
ExtendedConfiguration
subsorts:
Configuration $<$ *ExtendedConfiguration*
operations:
$\circ_$: *Configuration* $\rightarrow$ *ExtendedConfiguration* // reduce one step
$\star_$: *Configuration* $\rightarrow$ *ExtendedConfiguration* // reduce all steps
rule:
$\star Cfg \rightarrow \star Cfg'$ **if** $\circ\, Cfg \rightarrow Cfg'$ // where Cfg, Cfg' are variables of sort *Configuration*

Figure 3.17: Representing small-step and many-step SOS reductions in rewrite logic.

reasons, we preferred to define the sort of marked configurations as a supersort of *Configuration*. Note that the rule in Figure 3.17 indeed gives $\star$ its desired reflexive and transitive closure property (the reflexivity follows from the fact that the rewrite relation in rewrite logic is reflexive, so $\star C \rightarrow \star C$ for any configuration term C).

Theorem 15. ***(Faithful embedding of small-step SOS into rewrite logic)*** *For any small-step SOS* SmallStep*, and any* SmallStep *appropriate configurations C and C', the following equivalences hold:*

$$\text{SmallStep} \vdash C \rightarrow C' \iff \mathcal{R}_{\text{SmallStep}} \vdash \circ\, \overline{C} \rightarrow^1 \overline{C'} \iff \mathcal{R}_{\text{SmallStep}} \vdash \circ\, \overline{C} \rightarrow \overline{C'}$$

$$\text{SmallStep} \vdash C \rightarrow^\star C' \iff \mathcal{R}_{\text{SmallStep}} \vdash \star\, \overline{C} \rightarrow \star\, \overline{C'}$$

where $\mathcal{R}_{\text{SmallStep}}$ is the rewrite logic semantic definition obtained from SmallStep *by translating each rule in* SmallStep *as above. (Recall from Section 2.5 that $\rightarrow^1$ is the one-step rewriting relation obtained by dropping the reflexivity and transitivity rules of rewrite logic. Also, as C and C' are parameter-free—parameters only appear in rules—, $\overline{C}$ and $\overline{C'}$ are ground terms.)*

Except for transforming parameters into variables, the only apparent difference between SmallStep and $\mathcal{R}_{\text{SmallStep}}$ is that the latter marks (using $\circ$) all the left-hand-side configurations and, naturally, uses conditional rewrite rules instead of conditional deduction rules. As Theorem 15 shows, there is a step-for-step correspondence between their corresponding computations (or executions, or derivations). Therefore, similarly to the big-step SOS representation in rewrite logic, the rewrite theory $\mathcal{R}_{\text{SmallStep}}$ *is* the small-step SOS SmallStep, and *not* an encoding of it.

Recall from Section 3.2.3 that in the case of big-step SOS there were some subtle differences between the one-step $\rightarrow^1$ (obtained by dropping the reflexivity and transitivity rules of rewrite logic) and the usual $\rightarrow$ relations in the rewrite theory corresponding to the big-step SOS. The approach followed in this section based on marking configurations, thus keeping the left-hand and the right-hand-sides always distinct, eliminates all the differences between the two rewrite relations in the case of the one-step reduction (the two relations are identical on the terms of interest). The second equivalence in Theorem 15 tells us that we can turn the rewrite logic representation of the small-step SOS language definition into an interpreter by simply marking the configuration to be completely reduced with a $\star$ and then letting the rewrite engine do its job.

It is worthwhile noting that like in the case of the big-step SOS representation in rewrite logic, unfortunately, $\mathcal{R}_{\text{SmallStep}}$ lacks the main strengths of rewrite logic: in rewrite logic, rewrite rules can apply under any context and in parallel. Indeed, the rules of $\mathcal{R}_{\text{SmallStep}}$ can only apply at the top, sequentially. This should not surprise because, as stated, $\mathcal{R}_{\text{SmallStep}}$ *is* SmallStep, with all its strengths and limitations. By all means,

both the $\mathcal{R}_{\text{SmallStep}}$ above and the $\mathcal{R}_{\text{BigStep}}$ in Section 3.2.3 are rather poor-style rewrite logic specifications. However, that is normal, because neither big-step SOS nor small-step SOS were meant to have the capabilities of rewrite logic w.r.t. context-insensitivity and parallelism; since their representations in rewrite logic are faithful, one should not expect that they inherit the additional capabilities of rewriting logic (if they did, then the representations would not be step-for-step faithful, so something would be wrong).

Small-Step SOS of IMP in Rewrite Logic

Figure 3.18 gives the rewrite logic theory $\mathcal{R}_{\text{SmallStep(IMP)}}$ that is obtained by applying the procedure above to the small-step SOS of IMP, namely the formal system SmallStep(IMP) presented in Figures 3.14 and 3.15. As usual, we used the rewrite logic convention that variables start with upper-case letters, and like in the rewrite theory corresponding to the big-step SOS of IMP in Figure 3.8, we used σ (a larger σ symbol) for variables of sort *State*. Besides the parameter vs. variable subtle (but not unexpected) aspect, the only perceivable difference between SmallStep(IMP) and $\mathcal{R}_{\text{SmallStep(IMP)}}$ is the different notational conventions they use. The following corollary of Theorem 15 establishes the faithfulness of the representation of the small-step SOS of IMP in rewriting logic:

Corollary 5. $\text{SmallStep(IMP)} \vdash C \to C' \iff \mathcal{R}_{\text{SmallStep(IMP)}} \vdash \circ\overline{C} \to \overline{C'}$.

Therefore, there is no perceivable computational difference between the IMP-specific proof system SmallStep(IMP) and generic rewrite logic deduction using the IMP-specific rewrite rules in $\mathcal{R}_{\text{SmallStep(IMP)}}$; the two are faithfully equivalent.

★ Maude Definition of IMP Small-Step SOS

Figure 3.19 shows a straightforward Maude representation of the rewrite theory $\mathcal{R}_{\text{SmallStep(IMP)}}$ in Figure 3.18, including representations of the algebraic signatures of small-step SOS configurations in Figure 3.13 and of their extensions in Figure 3.17, which are needed to capture small-step SOS in rewrite logic. The Maude module `IMP-SEMANTICS-SMALLSTEP` in Figure 3.19 is executable, so Maude, through its rewriting capabilities, yields a small-step SOS interpreter for IMP the same way it yielded a big-step SOS interpreter in Section 3.2.3; for example, the command

```
rewrite * < sumPgm > .
```

where `sumPgm` is the first program defined in the module `IMP-PROGRAMS` in Figure 3.4, produces a result of the form (the exact statistics are also irrelevant, so they were replaced by "...”):

```
rewrites: 7632 in ... cpu (... real) (... rewrites/second)
result ExtendedConfiguration: * < {},n |-> 0 & s |-> 5050 >
```

Like for the big-step SOS definition in Maude, one can also use any of the general-purpose tools provided by Maude on the small-step SOS definition above. For example, one can exhaustively search for all possible behaviors of a program using the `search` command:

```
search * < sumPgm > =>! Cfg:ExtendedConfiguration .
```

As expected, only one behavior will be discovered because our IMP language so far is deterministic. However, a relatively large number of states will be explored, 1709, due to the non-deterministic evaluation strategy of the various language constructs:

```
Solution 1 (state 1708)
states: 1709  rewrites: 9232 in ... cpu (... real) (... rewrites/second)
Cfg:ExtendedConfiguration --> * < {},n |-> 0 & s |-> 5050 >
```

$$\circ\langle X, \sigma\rangle \to \langle \sigma(X), \sigma\rangle \ \textbf{if}\ \sigma(X) \neq \bot$$

$$\circ\langle A_1 + A_2, \sigma\rangle \to \langle A_1' + A_2, \sigma\rangle \ \textbf{if}\ \circ\langle A_1, \sigma\rangle \to \langle A_1', \sigma\rangle$$
$$\circ\langle A_1 + A_2, \sigma\rangle \to \langle A_1 + A_2', \sigma\rangle \ \textbf{if}\ \circ\langle A_2, \sigma\rangle \to \langle A_2', \sigma\rangle$$
$$\circ\langle I_1 + I_2, \sigma\rangle \to \langle I_1 +_{Int} I_2, \sigma\rangle$$

$$\circ\langle A_1 \ / \ A_2, \sigma\rangle \to \langle A_1' \ / \ A_2, \sigma\rangle \ \textbf{if}\ \circ\langle A_1, \sigma\rangle \to \langle A_1', \sigma\rangle$$
$$\circ\langle A_1 \ / \ A_2, \sigma\rangle \to \langle A_1 \ / \ A_2', \sigma\rangle \ \textbf{if}\ \circ\langle A_2, \sigma\rangle \to \langle A_2', \sigma\rangle$$
$$\circ\langle I_1 \ / \ I_2, \sigma\rangle \to \langle I_1 \ /_{Int} I_2, \sigma\rangle \ \textbf{if}\ I_2 \neq 0$$

$$\circ\langle A_1 <= A_2, \sigma\rangle \to \langle A_1' <= A_2, \sigma\rangle \ \textbf{if}\ \circ\langle A_1, \sigma\rangle \to \langle A_1', \sigma\rangle$$
$$\circ\langle I_1 <= A_2, \sigma\rangle \to \langle I_1 <= A_2', \sigma\rangle \ \textbf{if}\ \circ\langle A_2, \sigma\rangle \to \langle A_2', \sigma\rangle$$
$$\circ\langle I_1 <= I_2, \sigma\rangle \to \langle I_1 \leq_{Int} I_2, \sigma\rangle$$

$$\circ\langle !\ B, \sigma\rangle \to \langle !\ B', \sigma\rangle \ \textbf{if}\ \circ\langle B, \sigma\rangle \to \langle B', \sigma\rangle$$
$$\circ\langle !\ \texttt{true}, \sigma\rangle \to \langle \texttt{false}, \sigma\rangle$$
$$\circ\langle !\ \texttt{false}, \sigma\rangle \to \langle \texttt{true}, \sigma\rangle$$

$$\circ\langle B_1 \ \&\&\ B_2, \sigma\rangle \to \langle B_1' \ \&\&\ B2, \sigma\rangle \ \textbf{if}\ \circ\langle B_1, \sigma\rangle \to \langle B_1', \sigma\rangle$$
$$\circ\langle \texttt{false} \ \&\&\ B_2, \sigma\rangle \to \langle \texttt{false}, \sigma\rangle$$
$$\circ\langle \texttt{true} \ \&\&\ B_2, \sigma\rangle \to \langle B_2, \sigma\rangle$$

$$\circ\langle \{\ S\ \}, \sigma\rangle \to \langle S, \sigma\rangle$$

$$\circ\langle X = A\,\texttt{;}, \sigma\rangle \to \langle X = A'\,\texttt{;}, \sigma\rangle \ \textbf{if}\ \circ\langle A, \sigma\rangle \to \langle A', \sigma\rangle$$
$$\circ\langle X = I\,\texttt{;}, \sigma\rangle \to \langle \{\}, \sigma[I/X]\rangle \ \textbf{if}\ \sigma(X) \neq \bot$$

$$\circ\langle S_1\ S_2, \sigma\rangle \to \langle S_1'\ S_2, \sigma'\rangle \ \textbf{if}\ \circ\langle S_1, \sigma\rangle \to \langle S_1', \sigma'\rangle$$
$$\circ\langle \{\}\ S_2, \sigma\rangle \to \langle S_2, \sigma\rangle$$

$$\circ\langle \texttt{if}\ (B)\ S_1\ \texttt{else}\ S_2, \sigma\rangle \to \langle \texttt{if}\ (B')\ S_1\ \texttt{else}\ S_2, \sigma\rangle \ \textbf{if}\ \circ\langle B, \sigma\rangle \to \langle B', \sigma\rangle$$
$$\circ\langle \texttt{if}\ (\texttt{true})\ S_1\ \texttt{else}\ S_2, \sigma\rangle \to \langle S_1, \sigma\rangle$$
$$\circ\langle \texttt{if}\ (\texttt{false})\ S_1\ \texttt{else}\ S_2, \sigma\rangle \to \langle S_2, \sigma\rangle$$

$$\circ\langle \texttt{while}\ (B)\ S, \sigma\rangle \to \langle \texttt{if}\ (B)\ \{S\ \ \texttt{while}\ (B)\ S\ \}\ \texttt{else}\ \{\}, \sigma\rangle$$

$$\circ\langle \texttt{int}\ Xl\texttt{;}\ S\rangle \to \langle S, (Xl \mapsto 0)\rangle$$

Figure 3.18: $\mathcal{R}_{\text{SmallStep(IMP)}}$: the small-step SOS of IMP in rewrite logic.

```
mod IMP-CONFIGURATIONS-SMALLSTEP is including IMP-SYNTAX + STATE .
  sorts Configuration ExtendedConfiguration .
  subsort Configuration < ExtendedConfiguration .
  op <_,_> : AExp State -> Configuration .
  op <_,_> : BExp State -> Configuration .
  op <_,_> : Stmt State -> Configuration .
  op <_> : Pgm -> Configuration .
  op o_ : Configuration -> ExtendedConfiguration [prec 80] .   --- one step
  op *_ : Configuration -> ExtendedConfiguration [prec 80] .   --- all steps
  var Cfg Cfg' : Configuration .
 crl * Cfg => * Cfg' if o Cfg => Cfg' .
endm

mod IMP-SEMANTICS-SMALLSTEP is including IMP-CONFIGURATIONS-SMALLSTEP .
  var X : Id .  var Sigma Sigma' : State .  var I I1 I2 : Int .  var Xl : List{Id} .
  var A A' A1 A1' A2 A2' : AExp .  var B B' B1 B1' B2 : BExp .  var S S' S1 S1' S2 : Stmt .

 crl o < X,Sigma > => < Sigma(X),Sigma > if Sigma(X) =/=Bool undefined .

 crl o < A1 + A2,Sigma > => < A1' + A2,Sigma > if o < A1,Sigma > => < A1',Sigma > .
 crl o < A1 + A2,Sigma > => < A1 + A2',Sigma > if o < A2,Sigma > => < A2',Sigma > .
  rl o < I1 + I2,Sigma > => < I1 +Int I2,Sigma > .

 crl o < A1 / A2,Sigma > => < A1' / A2,Sigma > if o < A1,Sigma > => < A1',Sigma > .
 crl o < A1 / A2,Sigma > => < A1 / A2',Sigma > if o < A2,Sigma > => < A2',Sigma > .
 crl o < I1 / I2,Sigma > => < I1 /Int I2,Sigma > if I2 =/=Bool 0 .

 crl o < A1 <= A2,Sigma > => < A1' <= A2,Sigma > if o < A1,Sigma > => < A1',Sigma > .
 crl o < I1 <= A2,Sigma > => < I1 <= A2',Sigma > if o < A2,Sigma > => < A2',Sigma > .
  rl o < I1 <= I2,Sigma > => < I1 <=Int I2,Sigma > .

 crl o < ! B,Sigma > => < ! B',Sigma > if o < B,Sigma > => < B',Sigma > .
  rl o < ! true,Sigma > => < false,Sigma > .
  rl o < ! false,Sigma > => < true,Sigma > .

 crl o < B1 && B2,Sigma > => < B1' && B2,Sigma > if o < B1,Sigma > => < B1',Sigma > .
  rl o < false && B2,Sigma > => < false,Sigma > .
  rl o < true && B2,Sigma > => < B2,Sigma > .

  rl o < {S},Sigma > => < S,Sigma > .

 crl o < X = A ;,Sigma > => < X = A' ;,Sigma > if o < A,Sigma > => < A',Sigma > .
 crl o < X = I ;,Sigma > => < {},Sigma[I / X] > if Sigma(X) =/=Bool undefined .

 crl o < S1 S2,Sigma > => < S1' S2,Sigma' > if o < S1,Sigma > => < S1',Sigma' > .
  rl o < {} S2,Sigma > => < S2,Sigma > .

 crl o < if (B) S1 else S2,Sigma > => < if (B') S1 else S2,Sigma >
  if o < B,Sigma > => < B',Sigma  > .
  rl o < if (true) S1 else S2,Sigma > => < S1,Sigma > .
  rl o < if (false) S1 else S2,Sigma > => < S2,Sigma > .

  rl o < while (B) S,Sigma > => < if (B) {S while (B) S} else {},Sigma > .

  rl o < int Xl ; S > => < S,(Xl |-> 0) > .
endm
```

Figure 3.19: The small-step SOS of IMP in Maude, including the definition of configurations.

3.3.4 Notes

Small-step structural operational semantics was introduced as just *structural operational semantics* (SOS; no "small-step" qualifier at that time) by Plotkin in a 1981 technical report (University of Aarhus Technical Report DAIMI FN-19, 1981) that included his lecture notes of a programming language course [60]. For more than 20 years this technical report was cited as the main SOS reference by hundreds of scientists who were looking for mathematical rigor in their programming language research. It was only in 2004 that Plotkin's SOS was finally published in a journal [61].

Small-step SOS is pedagogically discussed in several textbooks, two early notable ones being Hennessy [32] (1990) and Winskel [87] (1993). Hennessy [32] uses the same notation as Plotkin, but Winskel [87] prefers a different one to make it clear that it is a one step semantics: $\langle s, \sigma\rangle \rightarrow_1 \langle s', \sigma'\rangle$. Like for big-step SOS, there is no well-established notation for small-step SOS sequents. There is a plethora of research projects and papers that explicitly or implicitly take SOS as *the* formal language semantics framework. Also, SOS served as a source of inspiration, or of problems to be fixed, to other semantic framework designers, including the author. There is simply too much work on SOS, using it, or modifying it, to attempt to cover it here. We limit ourselves to directly related research focused on capturing SOS as a methodological fragment of rewrite logic.

The marked configuration style that we adopted in this section to faithfully represent small-step SOS in rewrite logic was borrowed from Șerbănuță *et al.* [74]; there, the configuration marker "$\circ$" was called a "configuration modifier". An alternative way to keep the left-hand and the right-hand-side configurations distinct was proposed by Meseguer and Braga in [44, 14] in the context of representing MSOS into rewrite logic (see Section 3.6); the idea there was to use two different types of configuration wrappers, one for the left-hand-side of the transitions and one for the right-hand-side, yielding rewrite logic rules of the form:

$$(\forall \mathcal{X})\ \{\overline{C_0}\} \rightarrow [\overline{C'_0}]\ \ \textbf{if}\ \ \{\overline{C_1}\} \rightarrow [\overline{C'_1}]\ \wedge\ \{\overline{C_2}\} \rightarrow [\overline{C'_2}]\ \wedge \cdots \wedge \{\overline{C_n}\} \rightarrow [\overline{C'_n}]\ [\wedge\ \overline{condition}].$$

The solution proposed by Meseguer and Braga in [44, 14] builds upon experience with a previous representation of MSOS in rewrite logic in [15] as well as with an implementation of it in Maude [13, 16], where the necessity of being able to inhibit the default reflexivity and transitivity of the rewrite relation took shape. We preferred to follow the configuration modifier approach proposed by Șerbănuță *et al.* [74] because it appears to be slightly less intrusive (we only tag the already existing left-hand terms of rules) and more general (the left-hands of rules can have any structure, not only configurations, including no structure at all, as it happens in most of the rules of reduction semantics with evaluation contexts—see Section 3.7, e.g., Figure 3.40).

Vardejo and Martì-Oliet [84] give a Maude implementation of a small-step SOS definition for a simple imperative language similar to our IMP (Hennessy's *WhileL* language [32]), in which they do not attempt to prevent the inherent transitivity of rewriting. While they indeed obtain an executable semantics that is reminiscent of the original small-step SOS of the language, they actually define directly the transitive closure of the small-step SOS relation; they explicitly disable the reflexive closure by checking $C \neq C'$ next to rewrites $C \rightarrow C'$ in rule conditions. A small-step SOS of a simple functional language (Hennessy's *Fpl* language [32]) is also given in [84], following a slightly different style, which avoids the problem above. They successfully inhibit rewriting's inherent transitivity in their definition by using a rather creative rewriting representation style for sequents. More precisely, they work with sequents which appear to the user as having the form $\sigma \vdash a \rightarrow a'$, where σ is a state and a, a' are arithmetic expressions, etc., but they actually are rewrite relations between terms $\sigma \vdash a$ and a' (an appropriate signature to allow that to parse is defined). Indeed, there is no problem with the automatic reflexive/transitive closure of rewriting here because the LHS and the RHS of each rewrite rule have different structures. The simple functional language in [84] was pure (no side effects), so there was no need to include a resulting state in the RHS of their rules; if the language had

side effects, then this Vardejo and Martì-Oliet's representation of small-step SOS sequents in [84] would effectively be the same as the one by Meseguer and Braga in [44, 14] (but using a different notation).

3.3.5 Exercises

Prove the following exercises, all referring to the IMP small-step SOS in Figures 3.14 and 3.15.

Exercise 59. *Change the small-step rules for* / *so that it short-circuits when* a_1 *evaluates to* 0.

Exercise 60. *Change the small-step SOS of the* IMP *conjunction so that it is not short-circuited.*

Exercise 61. *Change the small-step SOS of blocks so that the block is kept but its inner statement is advanced one step.*

Exercise 62. *One can rightfully argue that the arithmetic expression in an assignment should not be reduced any step when the assigned variable is not declared. Change the small-step SOS of* IMP *to only reduce the arithmetic expression when the assigned variable is declared.*

Exercise 63. *A sophisticated language designer could argue that the reduction of the assignment statement to* $emptyBlockIMP$ *is an artifact of using small-step SOS, therefore an artificial and undesired step which affects the intended computational granularity of the language. Change the small-step SOS of* IMP *to eliminate this additional small-step.*
Hint: Follow the style in Exercise 68; note, however, that that style will require more rules and more types of configurations, so from that point of view is more complex.

Exercise 64. *Give a proof system for deriving "terminated configuration" sequents* $C\surd$.

Exercise 65. *One could argue that our small-step SOS rules for the conditional waste a computational step when switching to one of the two branches once the condition is evaluated.*

1. *Give an alternative small-step SOS for the conditional which does not require a computational step to switch to one of the two branches.*
2. *Can one do better than that? That is, can one save an additional step by reducing the corresponding branch one step at the same time with reducing the condition to true or false in one step? Hint: one may need terminated configurations, like in Exercise 64.*

Exercise 66. *Give an alternative small-step SOS definition of while loops which wastes no computational step. Hint: do a case analysis on b, like in the rules for the conditional.*

Exercise 67. *Give an alternative small-step SOS definition of variable declarations which wastes no computational steps. Hint: one may need terminated configurations, like in Exercise 64.*

Exercise 68. *Modify the small-step SOS definition of* IMP *such that the configurations in the right-hand-sides of the transition sequents are minimal (they should contain both a fragment of program and a state only when absolutely needed). What are the drawbacks of this minimalistic approach, compared to the small-step SOS semantics that we chose to follow?*

Exercise 69. *Show that the small-step SOS resulting from Exercise 68 is equivalent to the one in Figure 3.14 on arithmetic and Boolean expressions, that is,* $\langle a, \sigma\rangle \to \langle a', \sigma\rangle$ *is derivable with the proof system in Figure 3.14 if and only if* $\langle a, \sigma\rangle \to \langle a'\rangle$ *is derivable with the proof system in Exercise 68, and similarly for Boolean expressions. However, show that the equivalence does not hold true for statements.*

Exercise 70. *To handle division-by-zero, add "error" values and statements, and modify the small-step SOS in Figures 3.14 and 3.15 to allow derivations of sequents whose right-hand-side configurations contain "error" as their syntactic component. See also Exercise 56 (same problem but for big-step SOS).*

Exercise 71. *For any* IMP *statements* s_1, s_1', s_2, s_3 *and any states* σ, σ', *the following hold:*

1. $\textsc{SmallStep}(\text{IMP}) \vdash \langle(\{\}\ s_2)\ s_3, \sigma\rangle \to \langle s_2\ s_3, \sigma\rangle$ *and*
   $\textsc{SmallStep}(\text{IMP}) \vdash \langle\{\}\ (s_2\ s_3), \sigma\rangle \to \langle s_2\ s_3, \sigma\rangle$*; and*

2. $\textsc{SmallStep}(\text{IMP}) \vdash \langle(s_1\ s_2)\ s_3, \sigma\rangle \to \langle(s_1'\ s_2)\ s_3, \sigma'\rangle$ *if and only if*
   $\textsc{SmallStep}(\text{IMP}) \vdash \langle s_1\ (s_2\ s_3), \sigma\rangle \to \langle s_1'\ (s_2\ s_3), \sigma'\rangle$.

Consequently, the following also hold (prove them by structural induction on s_1*):*
$\textsc{SmallStep}(\text{IMP}) \vdash \langle(s_1\ s_2)\ s_3, \sigma\rangle \to^\star \langle s_2\ s_3, \sigma'\rangle$ *if and only if*
$\textsc{SmallStep}(\text{IMP}) \vdash \langle s_1\ (s_2\ s_3), \sigma\rangle \to^\star \langle s_2\ s_3, \sigma'\rangle$ *if and only if*
$\textsc{SmallStep}(\text{IMP}) \vdash \langle s_1, \sigma\rangle \to^\star \langle\{\}, \sigma'\rangle$.

Exercise 72. *With the* $\textsc{SmallStep}(\text{IMP})$ *proof system in Figures 3.14, 3.15, and 3.16, configuration C terminates iff* $\textsc{SmallStep}(\text{IMP}) \vdash C \to^\star R$ *for some irreducible configuration R.*

Exercise 73. *The small-step SOS of* IMP *is globally deterministic: if* $\textsc{SmallStep}(\text{IMP}) \vdash C \to^\star R$ *and* $\textsc{SmallStep}(\text{IMP}) \vdash C \to^\star R'$ *for irreducible configurations R and R′, then R = R′. Show the same result for the proof system detecting division-by-zero as in Exercise 70.*

Exercise 74. *Show that if* $\textsc{SmallStep}(\text{IMP}) \vdash C \to^\star \langle i, \sigma\rangle$ *for some configuration C, integer i, and state* σ*, then C must be of the form* $\langle a, \sigma\rangle$ *for some arithmetic expression a. Show a similar result for Boolean expressions. For statements, show that if* $\textsc{SmallStep}(\text{IMP}) \vdash C \to^\star \langle\{\}, \sigma\rangle$ *then C must be either of the form* $\langle s, \sigma'\rangle$ *for some statement s and some state* σ'*, or of the form* $\langle p\rangle$ *for some program p.*

Exercise 75. *Prove Theorem 14.*

Exercise 76. *State and show a result similar to Theorem 14 but for the small-step and big-step SOS proof systems in Exercises 70 and 56, respectively.*

Is this enough? Should I add more citations here? Ask Narciso, Peter and Jose for their and others' works embedding small-step into rewrite logic.

add the variables in rewrite rules

3.4 Denotational Semantics

Denotational semantics, also known as *fixed-point semantics*, associates to each syntactically well-defined fragment of program a well-defined, rigorous *mathematical object*. This mathematical object denotes the complete behavior of the fragment of program, no matter in what context it will be used. In other words, the denotation of a fragment of program represents its contribution to the meaning of any program containing it. In particular, equivalence of programs or fragments is immediately translated into equivalence of mathematical objects. The later can be then shown using the entire arsenal of mathematics, which is supposedly better understood and more well-established than that of the relatively much newer field of programming languages. There are no theoretical requirements on the nature of the mathematical domains in which the fragments of program are interpreted, although a particular approach became quite well-established, to an extent that it is by many identified with denotational semantics itself: choose the domains to be appropriate bottomed complete partial orders (abbreviated CPOs; see Section 2.6), and give the denotation of recursive language constructs (including loops, recursive functions, recursive data-structures, recursive types, etc.) as least fixed-points, which exist thanks to Theorem 12.

Each language requires customized denotational semantics, the same way each language required customized big-step or small-step structural operational semantics in the previous sections in this chapter. For the sake of concreteness, below we discuss general denotational semantics notions and notations by means of our running example language, IMP, without attempting to completely define it. Note that the complete denotational semantics of IMP is listed in Figure 3.20 in Section 3.4.1. Consider, for example, arithmetic expressions in IMP (which are side-effect free). Each arithmetic expression can be thought of as the mathematical object which is a partial function taking a state to an integer value, namely the value that the expression evaluates to in the given state. It is a partial function and not a total one because the evaluation of some arithmetic expressions may not be defined in some states, for example due to illegal operations such as division by zero. Thus, we can define the *denotation* of arithmetic expressions as a *total* function

$$[\![_]\!] : AExp \to (State \rightharpoonup Int)$$

taking arithmetic expressions to *partial* functions from states to integer numbers. As in all the semantics discussed in this chapter, states are themselves partial maps from names to values. In what follows we will follow a common notational simplification and will write $[\![a]\!]\sigma$ instead of $[\![a]\!](\sigma)$ whenever $a \in AExp$ and $\sigma \in State$, and similarly for other syntactic or semantic categories. To avoid ambiguity in the presence of multiple denotation functions, many works on denotational semantics tag the denotation functions with their corresponding syntactic categories, e.g., $AExp[\![_]\!]$ or $[\![_]\!]_{AExp}$. Our IMP language is simple enough that we can afford to not add such tags.

Denotation functions are defined inductively, over the structure of the language constructs. For example, if $i \in Int$ then $[\![i]\!]$ is the constant function i, that is, $[\![i]\!]\sigma = i$ for any $\sigma \in State$. Similarly, if $x \in Id$ then $[\![x]\!]\sigma = \sigma(x)$. As it is the case in mathematics, if an undefined value is used to calculate another value, then the resulting value is also undefined. In particular, $[\![x]\!]\sigma$ is undefined when $x \notin Dom(\sigma)$. The denotation of compound constructs is defined in terms of the denotations of the parts. In other words, we say that denotational semantics is *compositional*. For example, $[\![a_1 + a_2]\!]\sigma = [\![a_1]\!]\sigma +_{Int} [\![a_2]\!]\sigma$ for any $a_1, a_2 \in AExp$ and any $\sigma \in State$. For the same reason as above, if any of $[\![a_1]\!]\sigma$ or $[\![a_2]\!]\sigma$ is undefined then $[\![a_1 + a_2]\!]\sigma$ is also implicitly undefined. One can also chose to explicitly keep certain functions undefined in certain states, such as the denotation of division in those states in which the denominator is zero:

$$[\![a_1 \ / \ a_2]\!]\sigma = \begin{cases} [\![a_1]\!]\sigma \ /_{Int} \ [\![a_2]\!]\sigma & \text{if } [\![a_2]\!]\sigma \neq 0 \\ \bot & \text{if } [\![a_2]\!]\sigma = 0 \end{cases}$$

Note that even though the case where $[\![a_2]\!]\sigma$ is undefined ($\perp$) was not needed to be explicitly listed above (because it falls under the first case, $[\![a_2]\!]\sigma \neq 0$), it is still the case that $[\![a_1 \ / \ a_2]\!]\sigma$ is undefined whenever any of $[\![a_1]\!]\sigma$ or $[\![a_2]\!]\sigma$ is undefined.

An immediate use of denotational semantics is to prove properties about programs. For example, we can show that the addition operation on *AExp* whose denotational semantics was given above is associative. Indeed, we can prove for any $a_1, a_2, a_3 \in AExp$ the following equality of partial functions

$$[\![(a_1 + a_2) + a_3]\!] = [\![a_1 + (a_2 + a_3)]\!]$$

using conventional mathematical reasoning and the fact that the sum $+_{Int}$ in the *Int* domain is associative (see Exercise 77). Note that denotational semantics allows us not only to prove properties about programs or fragments of programs relying on properties of their mathematical domains of interpretation, but also, perhaps even more importantly, it allows us to elegantly formulate such properties. Indeed, what does it mean for a language construct to be associative or, in general, for any desired property over programs or fragments of programs to hold? While one could use any of the operational semantics discussed in this chapter to answer this, denotational semantics gives us one of the most direct means to state and prove program properties.

Each syntactic category is interpreted into its corresponding mathematical domain. For example, the denotations of Boolean expressions and of statements are total functions of the form:

$$\begin{aligned} [\![_]\!] &: \ BExp \to (State \rightharpoonup Bool) \\ [\![_]\!] &: \ Stmt \to (State \rightharpoonup State) \end{aligned}$$

The former is similar to the one for arithmetic expressions above, so we do not discuss it here. The latter is more interesting. Statements can indeed be regarded as partial functions taking states into resulting states. In addition to partiality due to illegal operations in expressions that statements may involve, such as division by zero, partiality in the denotation of statements may also occur for another important reason: *loops may not terminate*. For example, the statement `while (x <= y) {}` will not terminate in those states in which the value that `x` denotes is less than or equal to that of `y`. Mathematically, we say that the function from states to states that this loop statement denotes is undefined in those states in which the loop statement does not terminate. This will be elaborated shortly, after we discuss other statement constructs.

Since {} does not change the state, its denotation is the identity function, i.e., $[\![\{\}]\!] = 1_{State}$. The assignment statement updates the given state when defined in the assigned variable, that is, $[\![\texttt{x = a;}]\!]\sigma = \sigma[[\![a]\!]\sigma/x]$ when $\sigma(x) \neq \perp$ and $[\![a]\!]\sigma \neq \perp$, and $[\![\texttt{x = a;}]\!]\sigma = \perp$ otherwise. Sequential composition accumulates the state changes of the denotations of the composed statements, so it is precisely the mathematical composition of the corresponding partial functions: $[\![s_1 \ s_2]\!] = [\![s_2]\!] \circ [\![s_1]\!]$.

As an example, let us calculate the denotation of the statement "`x = 1; y = 2; x = 3;`" when $\texttt{x} \neq \texttt{y}$, i.e., the function $[\![\texttt{x = 1; y = 2; x = 3;}]\!]$. Applying the denotation of sequential composition twice, we obtain $[\![\texttt{x = 3;}]\!] \circ [\![\texttt{y = 2;}]\!] \circ [\![\texttt{x = 1;}]\!]$. Applying this composed function on a state σ, one gets $([\![\texttt{x = 3;}]\!] \circ [\![\texttt{y = 2;}]\!] \circ [\![\texttt{x = 1;}]\!])\sigma$ equals $\sigma[1/\texttt{x}][2/\texttt{y}][3/\texttt{x}]$ when $\sigma(\texttt{x})$ and $\sigma(\texttt{y})$ are both defined, and equals $\perp$ when any of $\sigma(\texttt{x})$ or $\sigma(\texttt{y})$ is undefined; let σ' denote $\sigma[1/\texttt{x}][2/\texttt{y}][3/\texttt{x}]$. By the definition of function update, one can easily see that σ' can be defined as

$$\sigma'(z) = \begin{cases} 3 & \text{if } z = \texttt{x} \\ 2 & \text{if } z = \texttt{y} \\ \sigma(z) & \text{otherwise,} \end{cases}$$

which is nothing but $\sigma[2/\texttt{y}][3/\texttt{x}]$. We can therefore conclude that the statements "`x = 1; y = 2; x = 3;`" and "`y = 2; x = 3;`" are equivalent, because they have the same denotation.

The denotation of a conditional statement `if (`b`)` s_1 `else` s_2 in a state σ is either the denotation of s_1 in σ or that of s_2 in σ, depending upon the denotation of b in σ:

$$[\![\texttt{if}\ (b)\ s_1\ \texttt{else}\ s_2]\!]\sigma = \begin{cases} [\![s_1]\!]\sigma & \text{if} \quad [\![b]\!]\sigma = \texttt{true} \\ [\![s_2]\!]\sigma & \text{if} \quad [\![b]\!]\sigma = \texttt{false} \\ \bot & \text{if} \quad [\![b]\!]\sigma = \bot \end{cases}$$

The third case above was necessary, because the first two cases do not cover the entire space of possibilities and, in such situations, one may (wrongly in our context here) understand that the function is underspecified in the remaining cases rather than undefined. Using the denotation of the conditional statement above and conventional mathematical reasoning, we can show, for example, that $[\![\texttt{if (y <= z) \{ x = 1; \} else \{ x = 2; \} x = 3;}]\!]$ is the function taking states σ defined in `x`, `y` and `z` to $\sigma[3/\texttt{x}]$.

The language constructs which admit non-trivial and interesting denotational semantics tend to be those which have a recursive nature. One of the simplest such constructs, and the only one we discuss here (see Section 4.7 for other recursive constructs), is IMP's `while` looping construct. Thus, the question we address next is how to define the denotation functions of the form

$$[\![\texttt{while}\ (b)\ s]\!] : \mathit{State} \rightharpoonup \mathit{State}$$

where $b \in \mathit{BExp}$ and $s \in \mathit{Stmt}$. What we want is $[\![\texttt{while}\ (b)\ s]\!]\sigma = \sigma'$ iff the while loop correctly terminates in state σ' when executed in state σ. Such a σ' may not always exist for two reasons:

1. Because b or s is undefined (e.g., due to illegal operations) in σ or in other states encountered during the loop execution; or

2. Because s (which may contain nested loops) or the while loop itself does not terminate.

If w is the partial function $[\![\texttt{while}\ (b)\ s]\!]$, then its most natural definition would appear to be:

$$w(\sigma) = \begin{cases} w([\![s]\!]\sigma) & \text{if} \quad [\![b]\!]\sigma = \texttt{true} \\ \sigma & \text{if} \quad [\![b]\!]\sigma = \texttt{false} \\ \bot & \text{if} \quad [\![b]\!]\sigma = \bot \end{cases}$$

Mathematically speaking, this is a problematic definition for several reasons:

1. The partial function w is defined in terms of itself;

2. It is not clear that such a w exists; and

3. In case it exists, it is not clear that such a w is unique.

To see how easily one can yield inappropriate recursive definitions of functions, we refer the reader to the discussion immediately following Theorem 12, which shows examples of recursive definitions which admit no solutions or which are ambiguous.

We next develop the mathematical machinery needed to rigorously define and reason about partial functions like the w above. More precisely, we frame the mathematics needed here as an instance of the general setting and results discussed in Section 2.6. We strongly encourage the reader to familiarize herself with the definitions and results in Section 2.6 before continuing.

A convenient interpretation of partial functions that may ease the understanding of the subsequent material is as *information* or *knowledge bearers*. More precisely, a partial function $\alpha : \mathit{State} \rightharpoonup \mathit{State}$ can be thought

of as carrying knowledge about some states in *State*, namely exactly those on which α is defined. For such a state $\sigma \in State$, the knowledge that α carries is $\alpha(\sigma)$. If α is not defined in a state $\sigma \in State$ then we can think of it as "*α does not have any information about σ*".

Recall from Section 2.6 that the set of partial functions between any two sets can be organized as a bottomed complete partial order (CPO). In our case, if $\alpha, \beta : State \rightharpoonup State$ then we say that *α is less informative than or as informative as β*, written $\alpha \preceq \beta$, if and only if for any $\sigma \in State$, it is either the case that $\alpha(\sigma)$ is not defined, or both $\alpha(\sigma)$ and $\beta(\sigma)$ are defined and $\alpha(\sigma) = \beta(\sigma)$. If $\alpha \preceq \beta$ then we may also say that *β refines α* or that *β extends α*. Then $(State \rightharpoonup State, \preceq \bot)$ is a CPO, where $\bot : State \rightharpoonup State$ is the partial function which is undefined everywhere.

One can think of each possible iteration of a while loop as an opportunity to refine the knowledge about its denotation. Before the Boolean expression b of the loop $\texttt{while}\, b\, \texttt{do}\, s$ is evaluated the first time, the knowledge that one has about its denotation function w is the empty partial function $\bot : State \rightharpoonup State$, say w_0. Therefore, w_0 corresponds to no information.

Now suppose that we evaluate the Boolean expression b in some state σ and that it is false. Then the denotation of the while loop should return σ, which suggests that we can refine our knowledge about w from w_0 to the partial function $w_1 : State \rightharpoonup State$, which is an identity on all those states $\sigma \in State$ for which $[\![b]\!]\sigma = \texttt{false}$ and which remains undefined in any other state.

So far we have not considered any state in which the loop needs to evaluate its body. Suppose now that for some state σ, it is the case that $[\![b]\!]\sigma = \texttt{true}$, $[\![s]\!]\sigma = \sigma'$, and $[\![b]\!]\sigma' = \texttt{false}$, that is, that the while loop terminates in one iteration. Then we can extend w_1 to a partial function $w_2 : State \rightharpoonup State$, which, in addition to being an identity on those states on which w_1 is defined, that is $w_1 \preceq w_2$, takes each σ as above to $w_2(\sigma) = \sigma'$.

By iterating this process, one can define a partial function $w_k : State \rightharpoonup State$ for any natural number k, which is defined on all those states on which the while loop terminates in *at most* k evaluations of its Boolean condition (i.e., $k - 1$ executions of its body). An immediate property of the partial functions w_0, w_1, w_2, $\ldots$, w_k is that they increasingly refine each other, that is, $w_0 \preceq w_1 \preceq w_2 \preceq \cdots \preceq w_k$. Informally, the partial functions w_k approximate w as k increases; more precisely, for any $\sigma \in State$, if $w(\sigma) = \sigma'$, that is, if the while loop terminates and σ' is the resulting state, then there is some k such that $w_k(\sigma) = \sigma'$. Moreover, $w_n(\sigma) = \sigma'$ for any $n \geq k$.

But the main question still remains unanswered: how to define the denotation $w : State \rightharpoonup State$ of the looping statement $\texttt{while}\ (b)\ s$? According to the intuitions above, w should be some sort of *limit* of the (infinite) sequence of partial functions $w_0 \preceq w_1 \preceq w_2 \preceq \cdots \preceq w_k \preceq \cdots$. We next formalize all the intuitions above. Let us define the total function

$$\mathcal{F} : (State \rightharpoonup State) \to (State \rightharpoonup State)$$

taking partial functions $\alpha : State \rightharpoonup State$ to partial functions $\mathcal{F}(\alpha) : State \rightharpoonup State$ as follows:

$$\mathcal{F}(\alpha)(\sigma) = \begin{cases} \alpha([\![s]\!]\sigma) & \text{if} \quad [\![b]\!]\sigma = \texttt{true} \\ \sigma & \text{if} \quad [\![b]\!]\sigma = \texttt{false} \\ \bot & \text{if} \quad [\![b]\!]\sigma = \bot \end{cases}$$

The partial functions w_k defined informally above can be now rigorously defined as $\mathcal{F}^k(\bot)$, where $\mathcal{F}^k$ stays for k compositions of $\mathcal{F}$, and $\mathcal{F}^0$ is by convention the identity function, i.e., $1_{(State \rightharpoonup State)}$ (which is total). Indeed, one can show by induction on k the following property, where $[\![s]\!]^i$ stays for i compositions of

$[\![s]\!] : State \rightharpoonup State$ and $[\![s]\!]^0$ is by convention the identity (total) function on *State*:

$$\mathcal{F}^k(\bot)(\sigma) = \begin{cases} [\![s]\!]^i\sigma & \text{if there is } 0 \leq i < k \text{ s.t. } [\![b]\!][\![s]\!]^i\sigma = \texttt{false} \\ & \text{and } [\![b]\!][\![s]\!]^j\sigma = \texttt{true} \text{ for all } 0 \leq j < i \\ \bot & \text{otherwise} \end{cases}$$

We can also show that the following is a chain of partial functions (Exercise 81 requires the reader to prove, for the IMP language, all these facts mentioned above and below)

$$\bot \preceq \mathcal{F}(\bot) \preceq \mathcal{F}^2(\bot) \preceq \cdots \preceq \mathcal{F}^n(\bot) \preceq \cdots$$

in the CPO $(State \rightharpoonup State, \preceq, \bot)$. As intuitively discussed above, this chain incrementally approximates the desired denotation of `while (`b`)` s. The final step is to realize that $\mathcal{F}$ is a continuous function and thus satisfies the hypotheses of the fixed-point Theorem 12, so we can conclude that the least upper bound (LUB) of the chain above, which by Theorem 12 is the least fixed-point $fix(\mathcal{F})$ of $\mathcal{F}$, is the desired denotation of the while loop, that is,

$$[\![\texttt{while (}b\texttt{)}\ s]\!] = fix(\mathcal{F})$$

Remarks. First, note that we indeed want the *least* fixed-point of $\mathcal{F}$, and not some arbitrary fixed-point of $\mathcal{F}$, to be the denotation of the while statement. Indeed, any other fixed-points would define states in which the while loop is intended to be undefined. To be more concrete, consider the simple IMP while loop "`while (! (k <= 10)) k = k + 1;`" whose denotation is defined only on those states σ with $\sigma(k) \leq 10$ and, on those states, it is the identity. That is,

$$[\![\texttt{while (! (k <= 10)) k = k + 1;}]\!](\sigma) = \begin{cases} \sigma & \text{if } \sigma(k) \leq 10 \\ \bot & \text{otherwise} \end{cases}$$

Consider now another fixed-point $\gamma : State \rightharpoonup State$ of its corresponding $\mathcal{F}$. While γ must still be the identity on those states σ with $\sigma(k) \leq 10$ (indeed, $\gamma(\sigma) = \mathcal{F}(\gamma)(\sigma) = \sigma$ for any such $\sigma \in State$), it is not enforced to be undefined on any other states. In fact, it can be shown (see Exercise 82) that the fixed-points of $\mathcal{F}$ are precisely those γ as above with the additional property that $\gamma(\sigma) = \gamma(\sigma')$ for any $\sigma, \sigma' \in State$ with $\sigma(k) > 10$, $\sigma'(k) > 10$, and $\sigma(x) = \sigma'(x)$ for any $x \neq k$. Such a γ can be, for example, the following:

$$\gamma(\sigma) = \begin{cases} \sigma & \text{if } \sigma(k) \leq 10 \\ \iota & \text{otherwise} \end{cases}$$

where $\iota \in State$ is some arbitrary but fixed state. Such fixed-points are too informative for our purpose here, since we want the denotation of while to be undefined in all states in which the loop does not terminate. Any other fixed-point of $\mathcal{F}$ which is strictly more informative than $fix(\mathcal{F})$ is simply too informative.

Second, note that the chain $\bot \preceq \mathcal{F}(\bot) \preceq \mathcal{F}^2(\bot) \preceq \cdots \preceq \mathcal{F}^n(\bot) \preceq \cdots$ can be stationary in some cases, but in general it is not. For example, when the loop is well-defined and terminates in any state in some fixed maximum number of iterations which does not depend on the state, its denotation is the (total) function in which the chain stabilizes (which in that case is its LUB and, by Theorem 12, the fixed-point of $\mathcal{F}$). For example, the chain corresponding to the loop "`while (1 <= k && k <= 10) k = k + 1;`" stabilizes in 12 steps, each step adding more states to the domain of the corresponding partial function until nothing can be added anymore: at step 1 all states σ with $\sigma(\texttt{k}) > 10$ or $\sigma(\texttt{k}) < 1$, at step 2 those with $\sigma(\texttt{k}) = 10$, at step 3 those with $\sigma(\texttt{k}) = 9$, ..., at step 11 those with $\sigma(\texttt{k}) = 1$; then no other state is added at step 12, that is, $\mathcal{F}^{12}(\bot) = \mathcal{F}^{11}(\bot)$. However, the chain associated to a loop is not stationary in general. For example, "`while (k <= 0) k = k + 1;`" terminates in any state, but there is no bound on the number of iterations. Consequently, there is no n such that $\mathcal{F}^n(\bot) = \mathcal{F}^{n+1}(\bot)$. Indeed, the later has strictly more information than the former: $\mathcal{F}^{n+1}$ is defined on all those states σ with $\sigma(\texttt{k}) = -n$, while $\mathcal{F}^n$ is not.

3.4.1 The Denotational Semantics of IMP

Figure 3.20 shows the complete denotational semantics of IMP. There is not much to comment on the denotational semantics of the various IMP language constructs, because they have already been discussed above. Note though that the denotation of conjunction captures the desired short-circuited semantics, in that the second conjunct is evaluated only when the first evaluates to `true`. Also, note that the denotation of programs is still a total function for uniformity (in spite of the fact that some programs may not be well-defined or may not terminate), but one into the CPO $State_\perp$ (see Section 2.6); thus, the denotation of a program which is not well-defined is $\perp$. Finally, note that, like in the big-step SOS of IMP in Section 3.2.2, we ignore the non-deterministic evaluation strategies of the `+` and `/` arithmetic expression constructs. In fact, since the denotations of the various language constructs are *functions*, non-deterministic constructs cannot be handled in denotational semantics the same way they were handled in operational semantics. There are ways to deal with non-determinism and concurrency in denotational semantics as discussed at the end of this section, but those are more complex and lead to inefficient interpreters when executed, so we do not consider them in this book. We here limit ourselves to denotational semantics of deterministic languages.

3.4.2 Denotational Semantics in Equational/Rewrite Logic

In order to formalize and execute denotational semantics one needs to formalize and execute the fragment of mathematics that is used by the denotation functions. The size of the needed fragment of mathematics is arbitrary and is typically determined by the particular programming language in question. Since a denotational semantics associates to each program or fragment of program a mathematical object expressed using the formalized language of the corresponding mathematical domain, the faithfulness of any representation/encoding/implementation of denotational semantics into any formalism directly depends upon the faithfulness of the formalizations of the mathematical domains.

The faithfulness of formalizations of mathematical domains is, however, not trivial to characterize in general. The formalization of each mathematical domain requires its own proofs of correctness, and in order for such proofs to make sense we need an alternative, trusted definition of the domain. Consider, for example, the basic domain of natural numbers. One may choose to formalize it using, e.g., Peano-style equational axioms or λ-calculus (see Section 4.5); nevertheless, each of these formalizations needs to be shown correct w.r.t. the mathematical domain of natural numbers, and none of these formalizations is powerful enough to mechanically derive all properties over natural numbers. It is therefore customary when formalizing denotational semantics to simply assume that the formalizations of the mathematical domains are correct; or put differently, to separate the problem of verifying the mathematical domains themselves from the problem of giving a language a denotational semantics using those domains. We here do the same thing: we assume that our equational/rewrite logic formalizations of mathematical domains in Section 2.6.5, which allow us in particular to define and execute higher-order functions and fixed-points for them, are correct. This automatically implies that our equational/rewrite logic representation of denotational semantics is faithful to the original denotational semantics; in other words, unlike for other semantic approaches, no faithfulness theorems for our representation of denotational semantics in equational/rewrite logic are needed.

Denotational Semantics of IMP in Equational/Rewrite Logic

Figure 3.21 shows a direct representation of the denotational semantics of IMP in Figure 3.20 using the mathematical domains formalized in equational/rewrite logic in Section 2.6.5.

To reduce the number of denotation functions defined, we chose to collapse all the syntactic categories under only one sort, *Syntax*. Similarly, to reuse existing operations of the CPO domain in Section 2.6.5 (e.g.,

Arithmetic expression constructs

$[\![_]\!] : AExp \to (State \rightharpoonup Int)$

$[\![i]\!]\sigma = i$

$[\![x]\!]\sigma = \sigma(x)$

$[\![a_1 + a_2]\!]\sigma = [\![a_1]\!]\sigma +_{Int} [\![a_2]\!]\sigma$

$$[\![a_1 \ / \ a_2]\!]\sigma = \begin{cases} [\![a_1]\!]\sigma \ /_{Int} \ [\![a_2]\!]\sigma & \text{if} \quad [\![a_2]\!]\sigma \neq 0 \\ \bot & \text{if} \quad [\![a_2]\!]\sigma = 0 \end{cases}$$

Boolean expression constructs

$[\![_]\!] : BExp \to (State \rightharpoonup Bool)$

$[\![t]\!]\sigma = t$

$[\![a_1 <= a_2]\!]\sigma = [\![a_1]\!]\sigma \leq_{Int} [\![a_2]\!]\sigma$

$[\![\,!\ b]\!]\sigma = \neg_{Bool}([\![b]\!]\sigma)$

$$[\![b_1 \ \&\& \ b_2]\!]\sigma = \begin{cases} [\![b_2]\!]\sigma & \text{if} \quad [\![b_1]\!]\sigma = \texttt{true} \\ \texttt{false} & \text{if} \quad [\![b_1]\!]\sigma = \texttt{false} \\ \bot & \text{if} \quad [\![b_1]\!]\sigma = \bot \end{cases}$$

Statement constructs

$[\![_]\!] : Stmt \to (State \rightharpoonup State)$

$[\![\{\}]\!]\sigma = \sigma$

$[\![\{\ s\ \}]\!]\sigma = [\![s]\!]\sigma$

$$[\![x = a\,;]\!]\sigma = \begin{cases} \sigma[[\![a]\!]\sigma/x] & \text{if} \quad \sigma(x) \neq \bot \\ \bot & \text{if} \quad \sigma(x) = \bot \end{cases}$$

$[\![s_1 \ \ s_2]\!]\sigma = [\![s_2]\!][\![s_1]\!]\sigma$

$$[\![\texttt{if}\ (b)\ s_1\ \texttt{else}\ s_2]\!]\sigma = \begin{cases} [\![s_1]\!]\sigma & \text{if} \quad [\![b]\!]\sigma = \texttt{true} \\ [\![s_2]\!]\sigma & \text{if} \quad [\![b]\!]\sigma = \texttt{false} \\ \bot & \text{if} \quad [\![b]\!]\sigma = \bot \end{cases}$$

$[\![\texttt{while}\ (b)\ s]\!] = fix(\mathcal{F})$, where $\mathcal{F} : (State \rightharpoonup State) \to (State \rightharpoonup State)$ defined as

$$\mathcal{F}(\alpha)(\sigma) = \begin{cases} \alpha([\![s]\!]\sigma) & \text{if} \quad [\![b]\!]\sigma = \texttt{true} \\ \sigma & \text{if} \quad [\![b]\!]\sigma = \texttt{false} \\ \bot & \text{if} \quad [\![b]\!]\sigma = \bot \end{cases}$$

Programs

$[\![_]\!] : Pgm \to State_\bot$

$[\![\texttt{int}\ xl;\ s]\!] = [\![s]\!](xl \mapsto 0)$

Figure 3.20: Denot(IMP): The denotational semantics of IMP.

sort:

Syntax // generic sort for syntax

subsorts:

AExp, *BExp*, *Stmt*, *Pgm* $<$ *Syntax* // syntactic categories fall under *Syntax*

Int, *Bool*, *State* $<$ *CPO* // basic domains are regarded as CPOs

operation:

$[\![_]\!] : Syntax \to CPO$ // denotation of syntax

equations:

// Arithmetic expression constructs:

$[\![I]\!] = \texttt{fun}_{CPO}\ \sigma\ \texttt{->}\ I$

$[\![X]\!] = \texttt{fun}_{CPO}\ \sigma\ \texttt{->}\ \sigma(X)$

$[\![A_1 + A_2]\!] = \texttt{fun}_{CPO}\ \sigma\ \texttt{->}\ \texttt{app}_{CPO}([\![A_1]\!], \sigma) +_{Int} \texttt{app}_{CPO}([\![A_2]\!], \sigma)$

$[\![A_1\ /\ A_2]\!] = \texttt{fun}_{CPO}\ \sigma\ \texttt{->}\ \texttt{if}_{CPO}(\texttt{app}_{CPO}([\![A_2]\!], \sigma) \neq_{Int} 0, \texttt{app}_{CPO}([\![A_1]\!], \sigma) /_{Int} \texttt{app}_{CPO}([\![A_2]\!], \sigma), \bot)$

// Boolean expression constructs:

$[\![T]\!] = \texttt{fun}_{CPO}\ \sigma\ \texttt{->}\ T$

$[\![A_1 \texttt{<=} A_2]\!] = \texttt{fun}_{CPO}\ \sigma\ \texttt{->}\ \texttt{app}_{CPO}([\![A_1]\!], \sigma) \leq_{Int} \texttt{app}_{CPO}([\![A_2]\!], \sigma)$

$[\![\texttt{!}\ B]\!] = \texttt{fun}_{CPO}\ \sigma\ \texttt{->}\ \neg_{Bool}\texttt{app}_{CPO}([\![B]\!], \sigma)$

$[\![B_1\ \texttt{\&\&}\ B_2]\!] = \texttt{fun}_{CPO}\ \sigma\ \texttt{->}\ \texttt{if}_{CPO}(\texttt{app}_{CPO}([\![B_1]\!], \sigma), \texttt{app}_{CPO}([\![B_2]\!], \sigma), \texttt{false})$

// Statement constructs:

$[\![\{\}]\!] = \texttt{fun}_{CPO}\ \sigma\ \texttt{->}\ \sigma$

$[\![\{\ S\ \}]\!] = \texttt{fun}_{CPO}\ \sigma\ \texttt{->}\ \texttt{app}_{CPO}([\![S]\!], \sigma)$

$[\![X = A\texttt{;}]\!] = \texttt{fun}_{CPO}\ \sigma\ \texttt{->}\ \texttt{app}_{CPO}(\texttt{fun}_{CPO}\ arg\ \texttt{->}\ \texttt{if}_{CPO}(\sigma(X) \neq_{Int} \bot, \sigma[arg/X], \bot), \texttt{app}_{CPO}([\![A]\!], \sigma))$

$[\![S_1\ S_2]\!] = \texttt{fun}_{CPO}\ \sigma\ \texttt{->}\ \texttt{app}_{CPO}([\![S_2]\!], \texttt{app}_{CPO}([\![S_1]\!], \sigma))$

$[\![\texttt{if}\ \texttt{(}B\texttt{)}\ S_1\ \texttt{else}\ S_2]\!] = \texttt{fun}_{CPO}\ \sigma\ \texttt{->}\ \texttt{if}_{CPO}(\texttt{app}_{CPO}([\![B]\!], \sigma), \texttt{app}_{CPO}([\![S_1]\!], \sigma), \texttt{app}_{CPO}([\![S_2]\!], \sigma))$

$[\![\texttt{while}\ \texttt{(}B\texttt{)}\ S]\!] = \texttt{fix}_{CPO}\ \texttt{fun}_{CPO}\ \alpha\ \texttt{->}\ \texttt{fun}_{CPO}\ \sigma\ \texttt{->}\ \texttt{if}_{CPO}(\texttt{app}_{CPO}([\![B]\!], \sigma), \texttt{app}_{CPO}(\alpha, \texttt{app}_{CPO}([\![S]\!], \sigma)), \sigma)$

// Programs:

$[\![\texttt{int}\ Xl\texttt{;}\ S]\!] = \texttt{app}_{CPO}([\![S]\!], (Xl \mapsto 0))$

Figure 3.21: $\mathcal{R}_{\text{DENOT(IMP)}}$: Denotational semantics of IMP in equational/rewrite logic.

the substitution) on the new mathematical domains without any additional definitional effort, we collapse all the mathematical domains under CPO; in other words, we now have only one large mathematical domain, CPO, which includes the domains of integer numbers, Booleans and states as subdomains.

There is not much to say about the equations in Figure 3.21; they restate the mathematical definitions in Figure 3.20 using our particular CPO language. The equational formalization of the CPO domain in Section 2.6.5 propagates undefinedness through the CPO operations according to their evaluation strategies. This allows us to ignore some cases, such as the last case in the denotation of the assignment or the conditional in Figure 3.20; indeed, if $\mathtt{app}_{CPO}(\llbracket A \rrbracket, \sigma)$ is undefined in some state σ then $\llbracket X = A\,; \rrbracket\sigma$ will also be undefined, because $\mathtt{app}_{CPO}(\ldots, \bot)$ equals $\bot$ according to our CPO formalization in Section 2.6.5.

★ Denotational Semantics of IMP in Maude

Figure 3.22 shows the Maude module corresponding to the rewrite theory in Figure 3.21.

3.4.3 Notes

Denotational semantics is the oldest semantic approach to programming languages. The classic paper that introduced denotational semantics as we know it today, then called "mathematical semantics", was published in 1971 by Scott and Strachey [70]. However, Strachey's interest in the subject started much earlier; he published two papers in 1966 and 1967, [77] and [78], respectively, which mark the beginning of denotational semantics. Before Strachey and Scott published their seminal paper [70], Scott published in 1970 a paper which founded what we call today *domain theory* [71]. Initially, Scott formalized domains as complete latices (which also admit a fixed-point theorem); in time, bottomed complete partial orders (CPOs, see Section 2.6) turned out to have better properties and they eventually replaced the complete latices.

We have only used very simple domains in our semantics of IMP in this section, such as domains of integers, Booleans, and partial functions. Moreover, for simplicity, we are not going to use complex domains for the IMP++ extension in Section 3.5 either. However, complex languages or better semantics may require more complex domains. In fact, choosing the right domains is one of the most important aspects of a denotational semantics. Poor domains may lead to behavioral limitations or to non-modular denotational semantic definitions. There are two additional important contributions to domain theory that are instrumental in making denotational semantics more usable:

- *Continuation domains.* The use of *continuations* in denotational semantics was proposed in 1974, in a paper by Strachey and Wadsworth [79]. Wadsworth was the one who coined the term "continuation", as representing "the meaning of the rest of the program". Continuations allow to have direct access to the execution flow, in particular to modify it, as needed for the semantics of abrupt termination, exceptions, or call/cc (Scheme was the first language to support call/cc). This way, continuations bring modularity and elegance to denotational definitions. However, they come at a price: using continuations affects the entire language definition (so one needs to change almost everything) and the resulting semantics are harder to read and reason about. There are countless uses of continuations in the literature, not only in denotational semantics; we refer the interested reader to a survey paper by Reynolds, which details continuations and their discovery from various perspectives [63].

- *Powerdomains.* The usual domains of partial functions that we used in our denotational semantics of IMP are not sufficient to define non-deterministic and/or concurrent languages. Consider, for example, the denotation of statements, which are partial functions from states to states. If the language is non-deterministic or concurrent, then a statement may take a state into any of many possible different

```
mod IMP-SEMANTICS-DENOTATIONAL is including IMP-SYNTAX + STATE + CPO .
  sort Syntax .
  subsorts AExp BExp Stmt Pgm < Syntax .
  subsorts Int Bool State < CPO .

  op [[_]] : Syntax  -> CPO .    --- Syntax interpreted in CPOs

  var X : Id . var Xl : List{Id} .  var I : Int .  var A1 A2 A : AExp .
  var T : Bool .  var B1 B2 B : BExp .  var S1 S2 S : Stmt .
  ops sigma alpha arg : -> CPOVar .

  eq [[I]] = funCPO sigma -> I .

  eq [[X]] = funCPO sigma -> _`(_`)(sigma,X) .

  eq [[A1 + A2]] = funCPO sigma -> appCPO([[A1]],sigma) +Int appCPO([[A2]],sigma) .

  eq [[A1 / A2]] = funCPO sigma -> ifCPO(appCPO([[A2]],sigma) =/=Bool 0,
                                         appCPO([[A1]],sigma) /Int appCPO([[A2]],sigma),
                                         undefined) .

  eq [[T]] = funCPO sigma -> T .

  eq [[A1 <= A2]] = funCPO sigma -> appCPO([[A1]],sigma) <=Int appCPO([[A2]],sigma) .

  eq [[! B]] = funCPO sigma -> notBool appCPO([[B]],sigma) .

  eq [[B1 && B2]] = funCPO sigma -> ifCPO(appCPO([[B1]],sigma),appCPO([[B2]],sigma),false) .

  eq [[{}]] = funCPO sigma -> sigma .

  eq [[{S}]] = funCPO sigma -> appCPO([[S]],sigma) .

  eq [[X = A ;]]
   = funCPO sigma
     -> appCPO(funCPO arg
               -> ifCPO(_`(_`)(sigma,X) =/=Bool undefined, sigma[arg / X], undefined),
               appCPO([[A]],sigma)) .

  eq [[S1 S2]] = funCPO sigma -> appCPO([[S2]],appCPO([[S1]],sigma)) .

  eq [[if (B) S1 else S2]]
   = funCPO sigma -> ifCPO(appCPO([[B]],sigma),appCPO([[S1]],sigma),appCPO([[S2]],sigma)) .

  eq [[while (B) S]]
   = fixCPO(funCPO alpha
            -> funCPO sigma
               -> ifCPO(appCPO([[B]],sigma),appCPO(alpha,appCPO([[S]],sigma)),sigma)) .

  eq [[(int Xl ; S)]] = appCPO([[S]],(Xl |-> 0)) .
endm
```

Figure 3.22: The denotational semantics of IMP in Maude

states, or, said differently, it may take a state into a *set* of states. To give denotational semantics to such languages, Plotkin proposed and formalized the notion of *powerdomain* [59]; the elements of a powerdomain are sets of elements of an underlying domain. Powerdomains make it thus possible to give denotational semantics to non-deterministic languages; used in combination with "resumptions", powerdomains can also be used to give interleaving semantics to concurrent languages.

As seen above, most of the denotational semantics ideas and principles have been proposed and developed in the 1970s. While it is recommended to read the original papers for historical reasons, some of them may actually use notational conventions and constructions which are not in current use today, making them less accessible. The reader interested in a more modern presentation of denotational semantics and domain theory is referred to Schmidt's denotational semantics book [69], and to Mosses' denotational semantics chapter [50] and Gunter and Scott's domain theory chapter [30] in the Handbook of Theoretical Computer Science (1990).

Denotational semantics are commonly defined and executed using functional languages. A particularly appealing aspect of functional languages is that the domains of partial functions, which are crucial for almost any denotational semantics, and fixed-point operators for them can be very easily defined using the already existing functional infrastructure of these languages; in particular, one needs to define no λ-like calculus as we did in Section 2.6.5. There is a plethora of works on implementing and executing denotational semantics on functional languages. We here only mention Papaspyrou's denotational semantics of C [57], implemented in Haskell; it uses about 40 domains in total and spreads over about 5000 lines of Haskell code. An additional advantage of defining denotational semantics in functional languages is that one can relatively easily port them into theorem provers and then prove properties or meta-properties about them. We refer the interested reader to Nipkow [55] for a simple example of how this is done in the context of the Isabelle/HOL prover.

An insightful exercise is to regard domain theory and denotational semantics through the lenses of *initial algebra semantics* [26], which was proposed by Goguen *et al.* in 1977. The initial algebra semantics approach is simple and faithful to equational logic (Section 2.4); it can be summarized with the following steps:

1. Define a language syntax as an algebraic signature, say Σ, which admits an initial (term) algebra T_Σ;
2. Define any needed semantic domain as the carrier of corresponding sort in a multi-sorted set, say D;
3. Give D a Σ-algebra structure, by defining operations corresponding to all symbols in Σ;
4. Conclude that the meaning of the language in D is the unique morphism $D_{_} : T_\Sigma \to D$, which gives meaning D_t in D to any fragment of program t.

Let us apply the initial algebra semantics steps above to IMP:

1. Σ is the signature in Figure 3.2;
2. D_{AExp} is the CPO $(State \rightharpoonup Int, \leq, \perp)$ and similarly for the other sorts;
3. $D_{_+_} : D_{AExp} \times D_{AExp} \to D_{AExp}$ is the (total) function defined as $D_{_+_}(f_1, f_2)(\sigma) = f_1(\sigma) +_{Int} f_2(\sigma)$ for all $f_1, f_2 \in D_{AExp}$ and $\sigma \in State$, and similarly for the other syntactic constructs.
4. The meaning of IMP is given by the unique morphism $D_{_} : T_\Sigma \to D_{AExp}$.

Therefore, one can regard a denotational semantics of a language as an initial algebra semantics applied to a *particular* algebra D; in our IMP case, for example, what we defined as $[\![a]\!]$ for $a \in AExp$ in denotational semantics is nothing but D_a. Moreover, we can now *prove* our previous denotational semantics definitions; for example, we can prove $D_{a_1+a_2}(\sigma) = D_{a_1}(\sigma) +_{Int} D_{a_2}(\sigma)$. The above was a very brief account of initial

algebra semantics, but sufficient to appreciate the foundational merits of initial algebra semantics in the context of denotational semantics (initial algebra semantics has many other applications). From an initial algebra semantics perspective, a denotational semantics is all about defining an algebra D in which the syntax is interpreted. How each fragment gets a meaning follows automatically, from more basic principles. In that regard, initial algebra semantics achieves a cleaner separation of concerns: syntax is defined as a signature, and semantics is defined as an algebra. There are no equations mixing syntax and semantics, like $[\![a_1 + a_2]\!]\sigma = [\![a_1]\!]\sigma +_{Int} [\![a_2]\!]\sigma$.

We are not aware of any other approaches to define denotational semantics using rewriting and then executing it on rewrite engines as we did in Section 3.4.2. While this is not difficult in principle, as seen in this section, it requires one to give executable rewriting definitions of semantic domains and of fixed points. This may be a tedious and repetitive process on simplistic rewrite engines; for example, the use of membership equational logic, which allows computations to take place also on terms whose intermediate sorts cannot be determined, was crucial for our formalization in Section 2.6.5. Perhaps the closest approach to ours is the one by Goguen and Malcolm in [29]; they define a simple imperative language using the OBJ system (a precursor of Maude) and a style which is a mixture of initial algebra semantics and operational semantics. For example, no fixed-points are used in [29], the loops being simply unrolled like in small-step SOS (see Section 3.3).

3.4.4 Exercises

Prove the following exercises, all referring to the IMP denotational semantics in Figure 3.20.

Exercise 77. *Show the associativity of the addition expression construct, that is, that*

$$[\![(a_1 + a_2) + a_3]\!] = [\![a_1 + (a_2 + a_3)]\!]$$

for any $a_1, a_2, a_3 \in AExp$.

Exercise 78. *Show the associativity of the statement sequential composition, that is, that*

$$[\![s_1\ (s_2\ s_3)]\!] = [\![(s_1\ s_2)\ s_3]\!]$$

for any $s_1, s_2, s_3 \in Stmt$. Compare the elegance of formulating and proving this result using denotational semantics with the similar task using small-step SOS (see Exercise 71).

Exercise 79. *State and prove the (correct) distributivity property of division over addition.*

Exercise 80. *Prove that the sequential composition statement "`if (b)` s_1 `else` s_2 s" (i.e., the conditional statement composed sequentially with s) and the conditional statement "`if (b)` $\{\, s_1\ s \,\}$ `else` $\{\, s_2\ s \,\}$" are equivalent, where s_1 and s_2 are blocks and where s is any statement.*

Exercise 81. *Prove that the functions $\mathcal{F} : (State \to State) \to (State \to State)$ associated to* IMP `while` *loops satisfy the hypotheses of the fixed-point Theorem 12, so that the denotation of* IMP *loops is indeed well-defined. Also, prove that the partial functions $w_k : State \to State$ defined as*

$$w_k(\sigma) = \begin{cases} [\![s]\!]^i\sigma & \text{if there is } 0 \le i < k \text{ s.t. } [\![b]\!][\![s]\!]^i\sigma = \texttt{false} \text{ and } [\![b]\!][\![s]\!]^j\sigma = \texttt{true} \text{ for all } 0 \le j < i \\ \bot & \text{otherwise} \end{cases}$$

are well-defined, that is, that if an i as above exists then it is unique. Then prove that $w_k = \mathcal{F}^k(\bot)$.

Exercise 82. *Describe all the fixed-points of the function $\mathcal{F}$ associated to the* IMP `while` *loop.*

Exercise 83. *The semantics in Figure 3.20 evaluates a program performing a division-by-zero to* $\perp$*. Modify the denotational semantics of* IMP *so that it returns a state like for correct programs, namely the state in which the division-by-zero took place. In other words, we want a variant of* IMP *where programs fail silently when they perform illegal operations.*

should I add definitions for entailments and prove the faithful representation theorem?

add denotational semantics as well in Section 3.5

discuss/add monads as well, as an interesting development besides continuations and powerdomains

use CPO instead of BCPO as an abbreviation for bottomed complete partial orders

make initial algebra semantics into a section

results relating denotational semantics to other semantics

3.5 IMP++: IMP Extended with Several Features

Our goal in this section is to challenge the modularity, reuse capability and flexibility to changes in language design of the basic semantic approaches discussed so far in this chapter, namely big-step SOS, small-step SOS and denotational semantics. We do this by means of a simple programming language design experiment. Specifically, we extend the IMP language in Section 3.1 with several common language features and attempt to give the resulting language a formal semantics following each of the approaches. In each case, we aim at reusing the existing semantics of IMP as much as possible.

Below we describe the features that we add to IMP, together with a short motivation for each:

1. A variable increment operation, ++ *Id*, whose role is to infuse side effects in expressions.

2. An input expression construct, `read()`, and an output statement construct, "`print(`*AExp*`);`", whose role is to modify the configurations (one needs to add input/output buffers).

3. Abrupt termination, both by means of an explicit "`halt;`" statement and by means of implicit division-by-zero, whose role is to enforce a sudden change of the evaluation context.

4. Spawning a new thread with an arithmetic expression construct `spawn` *Block*, which evaluates to a unique thread identifier associated to the newly created thread, and synchronizing with any thread using a "`join(`*AExp*`);`" statement construct. The newly created thread executes the given block concurrently with the rest of the program, sharing all the variables. The thread encountering a join statement gets blocked until the thread it wants to join terminates. The role of thread spawning and joining is to test the support of the various semantic approaches for concurrency.

5. Blocks allowing local variable declarations, { *Stmt* }, where *Stmt* can include declarations of the form "`int` **List{***Id***};**" (regarded as ordinary statements). The scope of local declarations is the remainder of the current block. The introduction of blocks with locals requires changing some of the existing syntax and semantics. For example, there is no need for global variable declarations anymore, because they can be replaced by locals. The role of this extension is threefold: (1) to demonstrate how execution environment recovery can be achieved in each semantic approach; (2) to generate some non-trivial feature interactions (e.g., spawned threads share the spawning environment); (3) to highlight a language design scenario where the introduction of a new feature may affect the design of the previous ones.

Two criteria guided us in selecting these particular language extensions: first, these are quite ordinary features encountered in many languages; and second, each of them exposes limitations of one or more of the conventional semantic approaches in this chapter (both before and after this section). We refer to the resulting language extension of IMP as IMP++. Both IMP and IMP++ are admittedly toy languages. However, if a certain programming language semantic approach has difficulties in supporting any of the features of IMP or any of the above IMP extensions in IMP++, then one should most likely expect the same problems, but of course amplified, to occur in practical attempts to define real-life programming languages. By "difficulty" we here mostly mean lack of modularity, that is, that in order to define a new feature or to change an existing one we need to make unrelated changes in the already existing semantics of other features. Such formalism-artifact changes are not only tedious and thus inconvenient and demotivate the language designer to extend or experiment with her language, but are also error prone.

IMP++ extends and modifies IMP both syntactically and semantically. Syntactically, it removes from

IMP the global declarations and adds the following constructs:

$$
\begin{array}{rcl}
\mathit{AExp} & ::= & \texttt{++}\,\mathit{Id} \\
 & | & \texttt{read()} \\
 & | & \texttt{spawn}\,\mathit{Block} \\
\mathit{Stmt} & ::= & \texttt{print(}\mathit{AExp}\texttt{);} \\
 & | & \texttt{halt;} \\
 & | & \texttt{join(}\mathit{AExp}\texttt{);} \\
 & | & \texttt{int}\,\mathbf{List}\{\mathit{Id}\}\texttt{;} \\
\mathit{Pgm} & ::= & \mathit{Stmt}
\end{array}
$$

Semantically, in addition to defining the new language constructs above, we prefer that division-by-zero implicitly halts the program, same like the explicit use of "`halt;`", but in the middle of an expression evaluation. When such an error takes place, one could also generate an error message. However, for simplicity, we do not consider error messages here; we only consider silent termination.

As part or our language design experiment to test the modularity, reuse capability and flexibility to language design changes of the various semantic approaches, we will do the following:

1. We will take each of the IMP++ language features one by one, discussing in detail what it takes to add each of them to IMP making complete abstraction of the other features. In other words, each time we define a feature we pretend that we do not know what other features will be added to the language.Hence, we attempt to achieve local optima for each feature independently. Then, in Section 3.5.6, we put all the features together in the IMP++ language.

2. We will first define `spawn` as a statement construct, pretending that join thread synchronization were not intended to be an explicit language construct, and will use this variant of `spawn` when putting together all the features of the IMP++ language in Section 3.5.6. Then, in Section 3.5.7, we add join thread synchronization to the resulting IMP++ language by turning `spawn` into an expression construct that evaluates to the identifier of the newly created thread, and by adding the "`join(`*AExp*`);`" statement construct that takes the identifier of the thread to join as argument.

We believe that our experiment in this section is not unrealistic. It is quite possible that language designers extend their language one feature at a time, for example to more easily experiment with it or to test it, and that while doing so they would like to only concentrate on the feature at hand and not to think of other possible extensions of the language. It is also reasonable to not worry about thread synchronization from the very beginning when designing a multi-threaded language, particularly because synchronization can also be achieved using shared variables and busy waiting (e.g., using Dekker's algorithm).

3.5.1 Adding Variable Increment

Like in several main-stream programming languages, ++ x increments the value of x in the state and evaluates to the incremented value. This way, the increment operation makes the evaluation of expressions to now have side effects. Consider, for example, the following two programs:

```
int m, n, s ;
n = 100 ;
while (++ m <= n) { s = s + m ; }
```

```
int x ;
x = 1 ;
x = ++ x / (++ x / x) ;
```

The first program shows that the side effect of variable increment can take place anywhere, even in the condition of a while loop; this is actually quite a common programming pattern in languages with variable increment. The second program shows that the addition of side-effects makes the originally intended evaluation strategies of the various expression constructs important. Indeed, recall that for demonstration purposes we originally wanted `+` and `/` to be non-deterministic (i.e., to evaluate their arguments stepwise non-deterministically, possibly interleaving their evaluations), while `<=` to be left-to-right sequential. These different evaluation strategies can now lead to different program behaviors. For example, the second program above has five different behaviors! Indeed, the expression assigned to x can evaluate to 0, 1, 2, 3, and can also perform a division-by-zero. Unfortunately, not all semantic approaches are able to capture all these behaviors.

Big-Step SOS

Big-step SOS is one of the semantics which is the most affected by the inclusion of side effects in expressions, because the previous triples $\langle a, \sigma\rangle \Downarrow \langle i\rangle$ and $\langle b, \sigma\rangle \Downarrow \langle t\rangle$ need to change to four-tuples of the form $\langle a, \sigma\rangle \Downarrow \langle i, \sigma'\rangle$ and $\langle b, \sigma\rangle \Downarrow \langle t, \sigma'\rangle$. These changes are necessary to account for collecting the possible side effects generated by the evaluation of expressions (note that the evaluation of Boolean expressions, because of `<=`, can also have side effects). The big-step SOS of almost all the language constructs needs to change as well. For example, the original big-step SOS of division, namely

$$\frac{\langle a_1, \sigma\rangle \Downarrow \langle i_1\rangle \quad \langle a_2, \sigma\rangle \Downarrow \langle i_2\rangle}{\langle a_1 \ / \ a_2, \sigma\rangle \Downarrow \langle i_1 \ /_{Int} \ i_2\rangle} \text{ if } i_2 \neq 0$$

changes as follows:

$$\frac{\langle a_1, \sigma\rangle \Downarrow \langle i_1, \sigma_1\rangle \quad \langle a_2, \sigma_1\rangle \Downarrow \langle i_2, \sigma_2\rangle}{\langle a_1 \ / \ a_2, \sigma\rangle \Downarrow \langle i_1 \ /_{Int} \ i_2, \sigma_2\rangle} \text{ if } i_2 \neq 0$$

$$\frac{\langle a_1, \sigma_2\rangle \Downarrow \langle i_1, \sigma_1\rangle \quad \langle a_2, \sigma\rangle \Downarrow \langle i_2, \sigma_2\rangle}{\langle a_1 \ / \ a_2, \sigma\rangle \Downarrow \langle i_1 \ /_{Int} \ i_2, \sigma_1\rangle} \text{ if } i_2 \neq 0$$

The rules above make an attempt to capture the intended nondeterministic evaluation strategy of the division operator. We will shortly explain why they fail to fully capture the desired behaviors. One should similarly consider the side effects of expressions in the semantics of statements that need to evaluate expressions. For example, the semantics of the while loop needs to change to propagate the side effects of its condition both when the loop is taken and when the loop is not taken.

Let us next include the big-step semantics of the increment operation; once all the changes to the existing semantics of IMP are applied, the big-step semantics of increment is straightforward:

$$\langle ++\, x, \sigma\rangle \Downarrow \langle \sigma(x) +_{Int} 1, \sigma[(\sigma(x) +_{Int} 1)/x]\rangle \qquad \text{(BigStep-Inc)}$$

Indeed, the problem with big-step is not necessarily to define the semantics of variable increment, but what it takes to be able to do it. One needs to redefine configurations as explained above and, consequently, to change the semantics of all the already existing features of IMP to use the new configurations. This, and other features defined later on, show how non-modular big-step semantics is.

In addition to being non-modular, big-step SOS cannot properly deal with non-determinism. While it can capture some limited degree of non-determinism as shown above with `/`, namely it can *non-deterministically choose* which of the subexpressions to evaluate first, it cannot define the full non-deterministic strategy (unless we make radical changes to the definition, such as working with sets of values instead of values, which

significantly complicate everything and still fail to capture the non-deterministic behaviors—as it would only capture the non-deterministic evaluation results). To see how the non-deterministic choice evaluation strategy in big-step semantics fails to capture all the desired behaviors, consider the expression $++x \mathbin{/} (++x \mathbin{/} x)$ with x initially 1, as in the second program at the beginning of Section 3.5.1. This expression can only evaluate to 1, 2 or 3 under non-deterministic choice strategy, like we get in big-step SOS. Nevertheless, as explained at the beginning of Section 3.5.1, it could also evaluate to 0 and even perform a division-by-zero under a fully non-deterministic evaluation strategy, as we will see when we use small-step semantic approaches.

Big-step semantics not only misses behaviors due to its lack of support for non-deterministic evaluation strategies, like shown above, but also hides misbehaviors that it, in principle, detects. For example, assuming $x > 0$, the expression $1 \mathbin{/} (x \mathbin{/} ++x)$ can either evaluate to 1 or perform an erroneous division-by-zero. If one searches for all the possible evaluations of a program containing such an expression using the big-step semantics in this section, one will only see the behavior where this expression evaluates to 1; one will never see the erroneous behavior where the division by zero takes place. This will be fixed in Section 3.5.3, where we modify the big-step SOS to support abrupt termination. However, without modifying the semantics, the language designer using big-step semantics may wrongly think that the program is correct. Contrast that with small-step semantics, which, even when one does not add support for abrupt termination, one still detects the wrong behavior by getting stuck on the configuration obtained right before the division by zero.

Additionally, as already explained in Section 3.2.3, the new configurations may be problematic when one wants to execute big-step definitions using rewriting. Indeed, one needs to remove resulting rewrite rules that lead to non-termination, such as rules of the form $R \to R$ corresponding to big-step sequents $R \Downarrow R$ where R are result configurations (e.g., $\langle i, \sigma\rangle$ with $i \in \mathit{Int}$ or $\langle t, \sigma\rangle$ with $t \in \{\texttt{true}, \texttt{false}\}$). We do not use this argument against big-step SOS (its poor modularity is sufficient to disqualify big-step in the competition for an ideal language definitional framework), but rather as a warning to the reader who wants to execute it using rewriting engines (like Maude).

Type System using Big-Step SOS

The typing policy of variable increment is the same as that of variable lookup: provided it has been declared, the incremented variable types to an integer. All we need is to add the following typing rule for increment to the already existing typing rules in Figure 3.10:

$$(xl, x, xl') \vdash ++x : \texttt{int} \qquad \text{(BigStepTypeSystem-Inc)}$$

Small-Step SOS

Including side effects in expressions is not as bad in small-step semantics as in big-step semantics, because, as discussed in Section 3.3, in small-step SOS one typically uses sequents whose left and right configurations have the same structure even in cases where only some of the configuration components change (e.g., one typically uses sequents of the form $\langle a, \sigma\rangle \to \langle a', \sigma\rangle$ instead of $\langle a, \sigma\rangle \to \langle a'\rangle$); thus, expressions, like any other syntactic categories including statements, can seamlessly modify the state if they need to. However, since we deliberately did not anticipate the inclusion of side effects in expression evaluation, we still have to go back through the existing definition and modify *all* the rules involving expressions to propagate the side effects. For example, the small-step rule

$$\frac{\langle a_1, \sigma\rangle \to \langle a_1', \sigma\rangle}{\langle a_1 \mathbin{/} a_2, \sigma\rangle \to \langle a_1' \mathbin{/} a_2, \sigma\rangle}$$

for the first argument of `/` does not apply anymore when the next step in a_1 is an increment operation (since the state σ changes), so it needs to change to

$$\frac{\langle a_1, \sigma\rangle \to \langle a_1', \sigma'\rangle}{\langle a_1 \ / \ a_2, \sigma\rangle \to \langle a_1' \ / \ a_2, \sigma'\rangle}$$

Of course, all these changes due to side-effect propagation would have not been necessary if we anticipated that side effects may be added to the language, but the entire point of this exercise is to study the strengths of the various semantic approaches without knowing what comes next.

Once all the changes are applied, one can define the small-step SOS of the increment operation almost identically to its big-step SOS (increment is one atomic step, so small equals big):

$$\langle ++\, x, \sigma\rangle \to \langle \sigma(x) +_{Int} 1, \sigma[(\sigma(x) +_{Int} 1)/x]\rangle \qquad \text{(SmallStep-Inc)}$$

Denotational Semantics

The introduction of expression side effects affects denotational semantics even worse than it affects the big-step SOS. Not only that one has to change the denotation of almost every language construct, but the changes are also heavier and more error prone than for big-step SOS. The first change that needs to be made is the type of the denotation functions for expressions:

$$[\![_]\!] : AExp \to (State \rightharpoonup Int)$$
$$[\![_]\!] : BExp \to (State \rightharpoonup Bool)$$

need to change into

$$[\![_]\!] : AExp \to (State \rightharpoonup Int \times State)$$
$$[\![_]\!] : BExp \to (State \rightharpoonup Bool \times State)$$

respectively, to take into account the fact that expressions also return a new possibly changed state besides a result when evaluated. Then one has to change the definitions of the denotation functions for each expression construct to propagate the side effects and to properly extract/combine values and states from/in pairs. For example, the previous denotation function of division, where $[\![a_1 \ / \ a_2]\!]\sigma$ was defined as

$$\begin{cases} [\![a_1]\!]\sigma +_{Int} [\![a_2]\!]\sigma & \text{if} \quad [\![a_2]\!]\sigma \neq 0 \\ \bot & \text{if} \quad [\![a_2]\!]\sigma = 0 \end{cases}$$

needs to change to be defined as

$$\begin{cases} (1^{\text{st}}([\![a_1]\!]\sigma) +_{Int} 1^{\text{st}}([\![a_2]\!](2^{\text{nd}}([\![a_1]\!]\sigma))), 2^{\text{nd}}([\![a_2]\!](2^{\text{nd}}([\![a_1]\!]\sigma))) & \text{if} \quad 1^{\text{st}}([\![a_2]\!](2^{\text{nd}}([\![a_1]\!]\sigma))) \neq 0 \\ \bot & \text{if} \quad 1^{\text{st}}([\![a_2]\!](2^{\text{nd}}([\![a_1]\!]\sigma))) = 0 \end{cases}$$

The above is a bit heavy, repetitive and thus error prone. In implementations of denotational semantics, and sometimes even on paper definitions, one typically uses let binders, or λ-abstractions (see Section 4.5), to bind each subexpression appearing more than once in a denotation function to some variable and then using that variable in each place.

In addition to the denotations of expressions, the denotation functions of all statements except for those of the empty block {} and sequential composition also need to change, because they involve expressions and need to take their side effects into account. For example, the denotation of the while loop statement `while` (b) s is the fixed-point of the total function

$$\mathcal{F} : (State \rightharpoonup State) \to (State \rightharpoonup State)$$

defined as

$$\mathcal{F}(\alpha)(\sigma) = \begin{cases} \alpha([\![s]\!](2^{\text{nd}}([\![b]\!]\sigma))) & \text{if } 1^{\text{st}}([\![b]\!]\sigma) = \texttt{true} \\ 2^{\text{nd}}([\![b]\!]\sigma) & \text{if } 1^{\text{st}}([\![b]\!]\sigma) = \texttt{false} \\ \bot & \text{if } [\![b]\!]\sigma = \bot \end{cases}$$

All the changes above were necessary to support the side effects generated by the increment construct. The denotational semantics of programs does not need to change. All programs that make no use of increment should still have exactly the same semantics as before (to be precise, the program denotation functions are different as syntactic terms, but they evaluate to the same values when invoked). We are now ready to give the denotational semantics of increment:

$$[\![{++}\,x]\!]\sigma = \begin{cases} (\sigma(x) +_{Int} 1, \sigma[(\sigma(x) +_{Int} 1)/x]) & \text{if } \sigma(x) \neq \bot \\ \bot & \text{if } \sigma(x) = \bot \end{cases} \qquad \text{(DENOTATIONAL-INC)}$$

Like for big-step SOS, giving the denotational semantics of increment is not difficult; the difficulty stays in what it takes to be able to do so.

Needless to say that denotational semantics, as we used it here, is very non-modular. The brute force approach above is the most straightforward approach when one's goal is to exclusively add increment to IMP—recall that our experiment in this section assumes that each language extension is the last one. When one expects many extensions to a language that in an operational setting would yield changes to the program configuration, in denotational semantics one is better served using a *continuation* based or a *monadic* style. These styles were briefly mentioned in Section 3.4.3 and will be further discussed in Section 3.9. They are more involved, and thus less accessible to non-expert language designers. Moreover, switching to such styles is a radical change which requires a complete redefinition of the language. It is therefore highly recommended that one starts directly with a continuation or monadic style if one expects many and non-trivial language extensions. We here, however, prefer the straightforward denotational approach because it is easier to understand and because our overall focus of this book is more operational than denotational.

Besides lacking modularity, the denotational semantics above also lacks non-determinism. Indeed, note that, for example, our denotation of division first evaluates the first expression and then the second expression. Since expressions have side effects, different orders of evaluation can lead to different behaviors. Since the denotations of expressions are partial functions, they cannot have two different behaviors in the same state; therefore, unlike in big-step SOS, we cannot simply add another equation for the other order of evaluation because that would yield an inconsistent mathematical/equational theory, which would violate the basic property of a function to produce no more than one output for any given input. The consecrated method to define non-determinism in denotational semantics is to use *powerdomains*, as briefly discussed in Section 3.4.3 and further discussed in Section 3.9. Like using continuations or monads, the use of powerdomains also requires a complete redesign of the entire semantics and makes it less accessible to non-experts. Moreover, one cannot obtain a feasible executable model of the language anymore, because the use of powerdomains requires to collect all possible behaviors of any fragment of program at any point in execution; this will significantly slow down the execution of the semantics, making it, for example, infeasible or even unusable as an interpreter anymore.

3.5.2 Adding Input/Output

The semantics of the input expression construct `read()` is that it consumes the next integer from the "input buffer" and evaluates to that integer. The semantics of the output statement construct `print(`a`);` is that a is first evaluated to some integer, which is then collected in an "output buffer". By a "buffer" we here mean a list structure over integers. The semantics of the input/output constructs will be given such that the input

buffer can only be removed integers from its beginning and the output buffer can only be appended integers to its end. If there is no integer left in the input buffer then `read()` blocks. The output buffer is unbounded, so "`print(`a`);`" never blocks when outputting the value of a. Consider the following two programs:

```
int m, n, s ;
n = read() ;
while (m <= n)
  {  print(m) ;  s = s + m ;  m = m + 1 ;  }
print(s) ;
```

```
int s ;
s = 0 ;
while (!(read() <= 0))
  {  s = s + read() ;  }
print(s) ;
```

The first reads one integer i from the beginning of the input buffer and then it appends $i + 2$ integers to the end of the output buffer (the numbers 0, 1, 2, ..., i followed by their sum). The second reads a potentially unbounded number of integers from the input buffer, terminating if and only if it reads a non-positive integer on an odd position in the input buffer; when that happens, it outputs the sum of the elements on the even positions in the input buffer up to that point.

The addition of `read()` to IMP means that expression evaluation becomes non-deterministic, regardless of whether we have variable increment or not in our language. Indeed, since `/` is non-deterministic, an expression of the form `read() / read()` can evaluate the two reads in any order; for example, if the first two integers in the input buffer are 7 and 3, then this expression can evaluate to either 2 or 0.

Let us formalize buffers. Assume colon-separated integer lists with ϵ as identity, $\mathbf{List}^{\epsilon}_{:}\{Int\}$, and let ω, ω', ω_1, etc., range over such lists. The same way we decided for notational convenience to let *State* be an alias for the map sort $\mathbf{Map}\{Id \mapsto Int\}$ (Section 3.1.2), from here on we also let *Buffer* alias the list sort $\mathbf{List}^{\epsilon}_{:}\{Int\}$.

In a formal language semantics, providing the entire input as part of the initial configuration and collecting the entire output in the result configuration is acceptable, although in implementations of the language one will most likely want the input/output to be interactive. There is some flexibility as to where the input and output buffers should be located in the configuration. One possibility is as new top-level components in the configuration. Another possibility is as special variables in the already existing state. The latter would require some non-trivial changes in the mathematical model of the state, so we prefer to follow the former approach in the sequel. An additional argument in favor of our choice is that sooner or later one needs to add new components to the configuration anyway, so we take this opportunity to discuss how robust/modular the various semantic styles are with regards to changes in the structure of the configuration.

Big-Step SOS

To accommodate the input/output buffers, all configurations and all sequents we had in the original big-step SOS of IMP in Sections 3.2.1 and 3.2.2 need to change. For example, since expressions can now consume input, the original expression sequents of form $\langle a, \sigma\rangle \Downarrow \langle i\rangle$ and $\langle b, \sigma\rangle \Downarrow \langle t\rangle$ need to change into sequents $\langle a, \sigma, \omega\rangle \Downarrow \langle i, \omega'\rangle$ and $\langle b, \sigma, \omega\rangle \Downarrow \langle t, \omega'\rangle$ (recall that we add one feature at a time, so expression evaluation currently does not have side effects on the state), respectively, where $\omega, \omega' \in \mathit{Buffer}$. Also, the big-step SOS rules for expressions need to change to take into account both the new configurations and the fact that expression evaluation can now affect the input buffer. For example, the original big-step SOS of division,

$$\frac{\langle a_1, \sigma\rangle \Downarrow \langle i_1\rangle \quad \langle a_2, \sigma\rangle \Downarrow \langle i_2\rangle}{\langle a_1 \ / \ a_2, \sigma\rangle \Downarrow \langle i_1 +_{Int} i_2\rangle} \ \text{if } i_2 \neq 0$$

changes as follows:

$$\frac{\langle a_1, \sigma, \omega\rangle \Downarrow \langle i_1, \omega_1\rangle \quad \langle a_2, \sigma, \omega_1\rangle \Downarrow \langle i_2, \omega_2\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega\rangle \Downarrow \langle i_1 \ /_{Int} \ i_2, \omega_2\rangle} \text{ if } i_2 \neq 0$$

$$\frac{\langle a_1, \sigma, \omega_2\rangle \Downarrow \langle i_1, \omega_1\rangle \quad \langle a_2, \sigma, \omega\rangle \Downarrow \langle i_2, \omega_2\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega\rangle \Downarrow \langle i_1 \ /_{Int} \ i_2, \omega_1\rangle} \text{ if } i_2 \neq 0$$

Like for the variable increment, the rules above make an attempt to capture the intended nondeterministic evaluation strategy of the division operator. Unfortunately, they also only capture a non-deterministic choice strategy, failing to capture the intended full non-determinism of division.

Since statements can both consume input and produce output, their big-step SOS sequents need to change from $\langle s, \sigma\rangle \Downarrow \langle \sigma'\rangle$ to $\langle s, \sigma, \omega_{in}\rangle \Downarrow \langle \sigma', \omega'_{in}, \omega_{out}\rangle$, where $\omega_{in}, \omega'_{in} \in \mathit{Buffer}$ are the input buffers before and, respectively, after the evaluation of statement s, and where $\omega_{out} \in \mathit{Buffer}$ is the output produced during the evaluation of s. Unfortunately, all big-step SOS rules for statements also have to change, to accommodate the additional input and/or output components in configurations. For example, the semantics of sequential composition needs to change from

$$\frac{\langle s_1, \sigma\rangle \Downarrow \langle \sigma_1\rangle \quad \langle s_2, \sigma_1\rangle \Downarrow \langle \sigma_2\rangle}{\langle s_1 \ \ s_2, \sigma\rangle \Downarrow \langle \sigma_2\rangle}$$

to

$$\frac{\langle s_1, \sigma, \omega_{in}\rangle \Downarrow \langle \sigma_1, \omega^1_{in}, \omega^1_{out}\rangle \quad \langle s_2, \sigma_1, \omega^1_{in}\rangle \Downarrow \langle \sigma_2, \omega^2_{in}, \omega^2_{out}\rangle}{\langle s_1 \ \ s_2, \sigma, \omega_{in}\rangle \Downarrow \langle \sigma_2, \omega^2_{in}, \omega^1_{out} : \omega^2_{out}\rangle}$$

Note that the outputs of s_1 and of s_2 have been appended in order to yield the output of $s_1 \ \ s_2$.

Finally, we also have to change the initial configuration holding programs to also take an input, as well as the big-step SOS rule for programs from

$$\frac{\langle s, xl \mapsto 0\rangle \Downarrow \langle \sigma\rangle}{\langle \texttt{int}\, xl\texttt{;}\ s\rangle \Downarrow \langle \sigma\rangle}$$

into

$$\frac{\langle s, xl \mapsto 0, \omega_{in}\rangle \Downarrow \langle \sigma, \omega'_{in}, \omega_{out}\rangle}{\langle \texttt{int}\, xl\texttt{;}\ s, \omega_{in}\rangle \Downarrow \langle \sigma, \omega'_{in}, \omega_{out}\rangle}$$

We keep the input buffer in the final configuration for two reasons: to avoid having to define a new configuration holding only the state and the output, and to allow the language designer to more easily debug her semantics, in particular to see whether there is any input left unused at the end of the program. One can argue that now, since we have output in our language, the result configuration may actually contain only the output (that is, it can also drop the state). The reader is encouraged to experiment with different final configurations.

Hence, almost everything changed in the original big-step SOS of IMP in order to prepare for the addition of input/output. All theses necessary changes highlight, again, the lack of modularity of big-step SOS. Once all the changes above are applied, one can easily define the semantics of the input/output constructs:

$$\langle \texttt{read()}, \sigma, i : \omega_{in}\rangle \Downarrow \langle i, \omega_{in}\rangle \qquad \text{(BigStep-Read)}$$

$$\frac{\langle a, \sigma, \omega_{in}\rangle \Downarrow \langle i, \omega'_{in}\rangle}{\langle \texttt{print(}a\texttt{);}, \sigma, \omega_{in}\rangle \Downarrow \langle \sigma, \omega'_{in}, i\rangle} \qquad \text{(BigStep-Print)}$$

Type System using Big-Step SOS

The typing policy of the input/output constructs is straightforward, though recall that we decided to only allow to read and to print integers. To type programs using input/output, we therefore add the following typing rules to the already existing typing rules in Figure 3.10:

$$xl \vdash \texttt{read()} : \texttt{int} \qquad \text{(BigStepTypeSystem-Read)}$$

$$\frac{xl \vdash a : \texttt{int}}{xl \vdash \texttt{print(}a\texttt{);} : \texttt{stmt}} \qquad \text{(BigStepTypeSystem-Print)}$$

Small-Step SOS

Like in big-step SOS, in small-step SOS we also need to change all IMP's configurations in Section 3.3.1 in order to add input/output. In the spirit of making only minimal changes, we modify the configurations holding expressions to also hold an input buffer, and the configurations holding statements to also hold both an input and an output buffer. Implicitly, all IMP's small-step SOS rules in Section 3.3.2 also need to change. The changes are straightforward, essentially having to just propagate the output through each statement construct, but they are still changes and thus expose, again, the lack of modularity of small-step SOS. Here is, for example, how the small-step SOS rules for division and for sequential composition need to change:

$$\frac{\langle a_1, \sigma, \omega_{in}\rangle \to \langle a_1', \sigma, \omega_{in}'\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega_{in}\rangle \to \langle a_1' \ / \ a_2, \sigma, \omega_{in}'\rangle}$$

$$\frac{\langle a_2, \sigma, \omega_{in}\rangle \to \langle a_2', \sigma, \omega_{in}'\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega_{in}\rangle \to \langle a_1 \ / \ a_2', \sigma, \omega_{in}'\rangle}$$

$$\frac{\langle s_1, \sigma, \omega_{in}, \omega_{out}\rangle \to \langle s_1', \sigma', \omega_{in}', \omega_{out}'\rangle}{\langle s_1 \ \ s_2, \sigma, \omega_{in}, \omega_{out}\rangle \to \langle s_1' \ \ s_2, \sigma', \omega_{in}', \omega_{out}'\rangle}$$

The expression configurations do not need to consider an output buffer because, as already discussed, in this language design experiment we assume at each stage only the current feature, without attempting to anticipate other features that will be possibly added in the future, and we attempts to do minimal changes. For example, if functions were to be added to the language later, in which case expressions will also possibly affect the output through function calls, then all the expression configurations and their corresponding small-step SOS rules will need to change again.

Finally, we also have to change the initial configuration holding programs to also take an input, as well as the small-step SOS rule for programs to initialize the output buffer to ϵ as follows:

$$\langle \texttt{int}\, xl\texttt{;} \ \ s, \omega_{in}\rangle \to \langle s, (xl \mapsto 0), \omega_{in}, \epsilon\rangle$$

Once all the changes are applied, we can give the small-step SOS of input/output as follows:

$$\langle \texttt{read()}, \sigma, i : \omega_{in}\rangle \to \langle i, \sigma, \omega_{in}\rangle \qquad \text{(SmallStep-Read)}$$

$$\frac{\langle a, \sigma, \omega_{in}\rangle \to \langle a', \sigma, \omega_{in}'\rangle}{\langle \texttt{print(}a\texttt{);}, \sigma, \omega_{in}, \omega_{out}\rangle \to \langle \texttt{print(}a'\texttt{);}, \sigma, \omega_{in}', \omega_{out}\rangle} \qquad \text{(SmallStep-Print-Arg)}$$

$$\langle \texttt{print(}i\texttt{);}, \sigma, \omega_{in}, \omega_{out}\rangle \to \langle \{\}, \sigma, \omega_{in}, \omega_{out} : i\rangle \qquad \text{(SmallStep-Print)}$$

Denotational Semantics

To accommodate the input and the output buffers, the denotation functions associated to IMP's syntactic categories need to change their types from

$$\begin{aligned}
&[\![_]\!] : AExp \to (State \rightharpoonup Int)\\
&[\![_]\!] : BExp \to (State \rightharpoonup Bool)\\
&[\![_]\!] : Stmt \to (State \rightharpoonup State)\\
&[\![_]\!] : Pgm \to State_{\perp}
\end{aligned}$$

to

$$\begin{aligned}
&[\![_]\!] : AExp \to (State \times Buffer \rightharpoonup Int \times Buffer)\\
&[\![_]\!] : BExp \to (State \times Buffer \rightharpoonup Bool \times Buffer)\\
&[\![_]\!] : Stmt \to (State \times Buffer \rightharpoonup State \times Buffer \times Buffer)\\
&[\![_]\!] : Pgm \to (Buffer \rightharpoonup State \times Buffer \times Buffer)
\end{aligned}$$

We next discuss the definitions of the new denotation functions.

The denotations of expressions now take a state and an input buffer and produce a value and a possibly modified input buffer. For example, the denotation of division becomes:

$$[\![a_1 \ / \ a_2]\!]\pi = \begin{cases} (1^{\text{st}}(arg_1) \ /_{Int} \ 1^{\text{st}}(arg_2),\ 2^{\text{nd}}(arg_2)) & \text{if} \quad 1^{\text{st}}(arg_2) \neq 0 \\ \perp & \text{if} \quad 1^{\text{st}}(arg_2) = 0 \end{cases}$$

where $arg_1 = [\![a_1]\!]\pi$ and $arg_2 = [\![a_2]\!](1^{\text{st}}(\pi), 2^{\text{nd}}(arg_1))$.

The denotations of statements can now produce an output buffer in addition to a modified input buffer (and a state). For example, the denotation of sequential composition becomes:

$$[\![s_1 \ \ s_2]\!]\pi = (1^{\text{st}}(arg_2),\ 2^{\text{nd}}(arg_2),\ 3^{\text{rd}}(arg_1) : 3^{\text{rd}}(arg_2))$$

where $arg_1 = [\![s_1]\!]\pi$ and $arg_2 = [\![s_2]\!](1^{\text{st}}(arg_1),\ 2^{\text{nd}}(arg_1))$. As another example of statement denotational semantics, the denotational semantics of while loops remains a fixed-point, but in order to be consistent with the new type of the denotation function for statements, it needs to be the fixed point of a (total) function

$$\mathcal{F} : (State \times Buffer \rightharpoonup State \times Buffer \times Buffer) \to (State \times Buffer \rightharpoonup State \times Buffer \times Buffer)$$

It is not difficult to see that the following definition of $\mathcal{F}$ has the right type and that $\mathcal{F}(\alpha)$ indeed captures the information that is added to α by unrolling the loop `while` (b) s once:

$$\mathcal{F}(\alpha)(\pi) = \begin{cases} (1^{\text{st}}(arg_3),\ 2^{nd}(arg_3),\ 3^{\text{rd}}(arg_2) : 3^{\text{rd}}(arg_3)) & \text{if} \quad 1^{\text{st}}(arg_1) = \texttt{true} \\ (1^{\text{st}}(\pi),\ 2^{\text{nd}}(arg_1),\ \epsilon) & \text{if} \quad 1^{\text{st}}(arg_1) = \texttt{false} \\ \perp & \text{if} \quad arg_1 = \perp \end{cases}$$

where $arg_1 = [\![b]\!]\pi$, $arg_2 = [\![s]\!](1^{\text{st}}(\pi),\ 2^{\text{nd}}(arg_1))$, and $arg_3 = \alpha(1^{\text{st}}(arg_2),\ 2^{\text{nd}}(arg_2))$.

Programs now take an input as well; like for big-step SOS, we prefer to also make the remaining input available at the end of the program execution, in addition to the state and the output:

$$[\![\texttt{int}\, xl;\ s]\!]\omega = [\![s]\!]((xl \mapsto 0), \omega)$$

Once all the changes on the denotations of the various syntactic categories are applied as discussed above, adding the semantics of the new input/output constructs is immediate:

$$[\![\texttt{read()}]\!]\pi = (\mathit{head}(2^{\text{nd}}(\pi)),\ \mathit{tail}(2^{\text{nd}}(\pi))) \qquad \text{(DENOTATIONAL-READ)}$$

$$[\![\texttt{print}(a)\texttt{;}]\!]\pi = (1^{\text{st}}(\pi),\ 2^{\text{nd}}([\![a]\!]\pi),\ 1^{\text{st}}([\![a]\!]\pi)) \qquad \text{(DENOTATIONAL-PRINT)}$$

Like in the case of IMP's extension with the variable increment expression construct, the denotational semantics of the extension with the input/output constructs would have been more modular if we had adopted a continuation or monadic style from the very beginning.

3.5.3 Adding Abrupt Termination

IMP++ adds both implicit and explicit abrupt program termination. The implicit one is given by division by zero, while the explicit abrupt termination is given by a new statement added to the language, "`halt;`". For uniformity and demonstration purposes, we choose a silent termination semantics. That is, in both cases of abrupt termination, the resulting configuration has the same structure as if the program terminated normally; for example, in the case of big-step SOS, we would like the result configuration for statements to be $\langle\sigma\rangle$, where σ is the state when the program was terminated abruptly. Specifically, we want the programs

```
int m, n, s ;
n = 100 ;
while (true) {
  if (m <= n) {
    s = s + m ;
    m = m + 1 ;
  }
  else { halt ; }
}
```

```
int m, n, s ;
n = 100 ;
while (true) {
  if (m <= n) {
    s = s + m ;
    m = m + 1 ;
  }
  else { s = s / (n / m) ; }
}
```

to yield the result configuration `< m |-> 101 & n |-> 100 & s |-> 5050 >` instead of a special configuration of the form `< halting, m |-> 101 & n |-> 100 & s |-> 5050 >` or similar. Unfortunately, that is not possible in all cases without intrusively modifying the syntax of the IMP language (to catch the exceptional behavior and explicitly discard the additional information), since some operational styles need to make a sharp distinction between a halting configuration and a normal configuration (for propagation reasons). Proponents of those semantic styles may argue that our semantic choice above seems inappropriate, since giving more information in the result configuration, such as "this is a halting configuration", is better for all purposes than giving less information. There are, however, also reasons to always want a normal result configuration upon termination. For example, one may want to include IMP in a larger context, such as in a distributed system, where all the context wants to know about the embedded language is that it takes a statement and produces a state and/or an output; IMP's internal exceptional situations are of no concern to the outer context. There is no absolute better or worse language design, both in what regards syntax and in what regards semantics. Our task here is to make the language designer aware of the subtleties and the limitations of the various semantic approaches, and not to propose a better programming language.

Big-Step SOS

The lack of modularity of big-step semantics will be, again, emphasized here. Let us first add the semantic definition for the implicit abrupt termination generated by division by zero. Recall that the big-step SOS rule for division was the following:

$$\frac{\langle a_1,\sigma\rangle \Downarrow \langle i_1\rangle \qquad \langle a_2,\sigma\rangle \Downarrow \langle i_2\rangle}{\langle a_1 \ / \ a_2,\sigma\rangle \Downarrow \langle i_1 +_{Int} i_2\rangle} \ \text{if } i_2 \neq 0$$

We keep that unchanged, but we also add the following new rule:

$$\frac{\langle a_1, \sigma\rangle \Downarrow \langle i_1\rangle \quad \langle a_2, \sigma\rangle \Downarrow \langle 0\rangle}{\langle a_1 \ / \ a_2, \sigma\rangle \Downarrow \langle \texttt{error}\rangle} \qquad \text{(BigStep-Div-By-Zero)}$$

In the above rule, `error` can be regarded as a special value; alternatively, one can regard $\langle \texttt{error}\rangle$ as a special result configuration.

But what if the evaluation of a_1 or of a_2 in the above rule generates itself an error? If that is the case, then one needs to propagate that error through the division construct:

$$\frac{\langle a_1, \sigma\rangle \Downarrow \langle \texttt{error}\rangle}{\langle a_1 \ / \ a_2, \sigma\rangle \Downarrow \langle \texttt{error}\rangle} \qquad \text{(BigStep-Div-Error-Left)}$$

$$\frac{\langle a_2, \sigma\rangle \Downarrow \langle \texttt{error}\rangle}{\langle a_1 \ / \ a_2, \sigma\rangle \Downarrow \langle \texttt{error}\rangle} \qquad \text{(BigStep-Div-Error-Right)}$$

Note that in case one of a_1 or a_2 generates an error, then the other one is not even evaluated anymore, to faithfully capture the intended meaning of abrupt termination.

Unfortunately, one has to do the same for all the expression language constructs. This way, for each expression construct, one has to add at least as many error-propagation big-step SOS rules as arguments that expression construct takes. Moreover, when the evaluation error reaches a statement, one needs to transform it into a "halting signal". This can be achieved by introducing a new type of result configuration, namely $\langle \texttt{halting}, \sigma\rangle$, and then adding appropriate halting propagation rules for all the statements. For example, the assignment statement needs to be added the new rule

$$\frac{\langle a, \sigma\rangle \Downarrow \langle \texttt{error}\rangle}{\langle x = a\texttt{;}, \sigma\rangle \Downarrow \langle \texttt{halting}, \sigma\rangle} \qquad \text{(BigStep-Asgn-Halt)}$$

The halting signal needs to be propagated through statement constructs, collecting the appropriate state. For example, the following two rules need to be included for sequential composition, in addition to the existing rule (which stays unchanged):

$$\frac{\langle s_1, \sigma\rangle \Downarrow \langle \texttt{halting}, \sigma_1\rangle}{\langle s_1 \ \ s_2, \sigma\rangle \Downarrow \langle \texttt{halting}, \sigma_1\rangle} \qquad \text{(BigStep-Seq-Halt-Left)}$$

$$\frac{\langle s_1, \sigma\rangle \Downarrow \langle \sigma_1\rangle, \ \langle s_2, \sigma_1\rangle \Downarrow \langle \texttt{halting}, \sigma_2\rangle}{\langle s_1 \ \ s_2, \sigma\rangle \Downarrow \langle \texttt{halting}, \sigma_2\rangle} \qquad \text{(BigStep-Seq-Halt-Right)}$$

In addition to all the propagation rules, we also have to define the semantics of the explicit halt statement:

$$\langle \texttt{halt;}, \sigma\rangle \Downarrow \langle \texttt{halting}, \sigma\rangle \qquad \text{(BigStep-Halt)}$$

Therefore, when using big-step SOS, one has to more than *double* the number of rules in order to support abrupt termination. Indeed, any argument of any language construct can yield the termination signal, so a rule is necessary to propagate that signal through the current language construct. It is hard to imagine worse in a language design framework. An unfortunate language designer choosing big-step semantics as language definition framework will incrementally become reluctant to add or experiment with any new feature in her language. For example, imagine that one wants to add exceptions and break/continue of loops to IMP++.

Finally, unless one extends the language syntax, there appears to be no way to get rid of the junk result configurations $\langle \texttt{halting}, \sigma \rangle$ that have been artificially added in order to propagate the error or the halting signals. For example, one cannot simply add the rule

$$\frac{\langle s, \sigma \rangle \Downarrow \langle \texttt{halting}, \sigma' \rangle}{\langle s, \sigma \rangle \Downarrow \langle \sigma' \rangle}$$

because it may interfere with other rules and thus wrongly hide the halting signal; for example, it can be applied on the second hypothesis of the rule (BigStep-Seq-Halt-Right) above hiding the halting signal and thus wrongly making the normal rule (BigStep-Seq) applicable. While having junk result configurations of the form $\langle \texttt{halting}, \sigma \rangle$ may seem acceptable in our scenario here, perhaps even desirable for debugging reasons, in general one may find it inconvenient to have many types of result configurations; indeed, one would need similar junk configurations for exceptions, for break/continue of loops, for functions return, etc.

Consequently, the halting signal needs to be caught at the top-level of the derivation. Fortunately, IMP provides a top-level syntactic category, *Pgm*, so we can add the following rule which dissolves the potential junk configuration at the top:

$$\frac{\langle s, xl \mapsto 0 \rangle \Downarrow \langle \texttt{halting}, \sigma \rangle}{\langle \texttt{int}\, xl;\ s \rangle \Downarrow \langle \sigma \rangle} \qquad \text{(BigStep-Halt)}$$

Now we have silent abrupt termination, but note that the variable declaration construct now acts as an exception catching and dissolving the abrupt termination signal generated by "`halt;`" or by division-by-zero. If one does not like to use a language construct for something which has not been originally intended, then one can add an auxiliary `top` statement or program construct, then reduce the semantics of programs to that of `top`, and then give `top` an exception-handling-like big-step SOS, as we will do for the MSOS definition of abrupt termination in Section 3.6; see Exercise 93. This latter solution is also more general, because it does not rely on a fortunate earlier decision to have a top-level language construct.

In addition to the lack of modularity due to having to more than double the number of rules in order to add abrupt termination, the inclusion of all these rules can also have a significant impact on performance when one wants to execute the big-step SOS. Indeed, there are now four rules for division, each having the same left-hand-side, $\langle a_1 \ / \ a_2, \sigma \rangle$, and some of these rules even sharing some of the hypotheses. That means that any general-purpose proof or rewrite system attempting to execute such a definition will unavoidably face the problem of searching a large space of possibilities in order to find one or all possible reductions.

Type System using Big-Step SOS

The typing policy of abrupt termination is clear: "`halt;`" types to a statement and division-by-zero is ignored. Indeed, one cannot expect that a type checker, or any technique, procedure or algorithm, can detect division by zero in general: division-by-zero, like almost any other runtime property of any Turing-complete programing language, is an undecidable problem. Consequently, it is common to limit typing division to checking that the two expressions have the expected type, integer in our case, which our existing type checker for IMP already does (see Figure 3.10). We therefore only add the following typing rule for "`halt;`":

$$xl \vdash \texttt{halt;} : \texttt{stmt} \qquad \text{(BigStepTypeSystem-Halt)}$$

Small-Step SOS

Small-step SOS turns out to be almost as non-modular as big-step SOS when defining control-intensive constructs like abrupt termination. Like for big-step SOS, we need to invent special configurations to signal

steps corresponding to implicit division by zero or to explicit halt statements. In small-step SOS, a single type of such special configurations suffices, namely one of the form $\langle \texttt{halting}, \sigma \rangle$, where σ is the state in which the program was abruptly terminated. However, for uniformity with big-step SOS, we also define two types of special configurations, one for expressions and one for statements; since we carry the state in the right-hand-side configuration of all sequents in our small-step SOS definitions, the only difference between the two configuration types is their tag, namely $\langle \texttt{error}, \sigma \rangle$ for the former versus $\langle \texttt{halting}, \sigma \rangle$ for the latter.

We can then define the small-step SOS of division by zero as follows (recall that the original SmallStep-Div rule in Figure 3.14 is "$\langle i_1 \ \texttt{/} \ i_2, \sigma \rangle \rightarrow \langle i_1 \ /_{Int} \ i_2, \sigma \rangle \quad \text{if } i_2 \neq 0$"):

$$\langle i_1 \ \texttt{/} \ 0, \sigma \rangle \rightarrow \langle \texttt{error}, \sigma \rangle \qquad \text{(SmallStep-Div-By-Zero)}$$

Like for the big-step SOS extension above, we have to make sure that the halting signal is correctly propagated. Here are, for example, the propagation rules through the division construct:

$$\frac{\langle a_1, \sigma \rangle \rightarrow \langle \texttt{error}, \sigma \rangle}{\langle a_1 \ \texttt{/} \ a_2, \sigma \rangle \rightarrow \langle \texttt{error}, \sigma \rangle} \qquad \text{(SmallStep-Div-Error-Left)}$$

$$\frac{\langle a_2, \sigma \rangle \rightarrow \langle \texttt{error}, \sigma \rangle}{\langle a_1 \ \texttt{/} \ a_2, \sigma \rangle \rightarrow \langle \texttt{error}, \sigma \rangle} \qquad \text{(SmallStep-Div-Error-Right)}$$

The two rules above are given in such a way that the semantics is faithful to the intended *computational granularity* of the defined language feature. Indeed, we want division by zero to take one computational step to be reported as an error, as opposed to as many steps as the depth of the context in which the error has been detected; for example, a configuration containing expression (3 / 0) / 3 should reduce to a halting configurations in one step, not in two. If we added a special error value and replaced the two rules above by

$$\langle \texttt{error} \ \texttt{/} \ a_2, \sigma \rangle \rightarrow \langle \texttt{error}, \sigma \rangle$$

$$\langle a_1 \ \texttt{/} \ \texttt{error}, \sigma \rangle \rightarrow \langle \texttt{error}, \sigma \rangle$$

then errors would be propagated to the top level of the program in as many small-steps as the depth of the context in which the error was generated; we do not want that.

Like in the big-step SOS above, the implicit expression errors need to propagate through the statements and halt the program. One way to do it is to generate an explicit `halt;` statement and then to propagate that "`halt;`" statement through all the statement constructs as if it was a special statement value, until it reaches the top. However, as discussed in the paragraph above, that would generate as many steps as the depth of the evaluation contexts in which the "`halt;`" statement is located, instead of just one step as desired. Alternatively, we can use the same approach to propagate the halting configuration through the statement constructs as we used to propagate it through the expression constructs. Specifically, we add transition rules from expressions to statements, like the one below (a similar one needs to be added for the conditional):

$$\frac{\langle a, \sigma \rangle \rightarrow \langle \texttt{error}, \sigma \rangle}{\langle x = a\texttt{;}, \sigma \rangle \rightarrow \langle \texttt{halting}, \sigma \rangle} \qquad \text{(SmallStep-Asgn-Halt)}$$

Once the halting signal due to a division by zero reaches the statement level, it needs to be further propagated through the sequential composition, that is, we need to add the following rule:

$$\frac{\langle s_1, \sigma \rangle \rightarrow \langle \texttt{halting}, \sigma \rangle}{\langle s_1 \ \ s_2, \sigma \rangle \rightarrow \langle \texttt{halting}, \sigma \rangle} \qquad \text{(SmallStep-Seq-Halt)}$$

Note that we assumed that a halting step does not change the state (we used the same σ in both the left and the right configurations). One can prove by structural induction that in our simple language scenario that is indeed the case, so there is no need to propagate state changes when the program halts.

Finally, we can also generate a special halting configuration when a "`halt;`" statement is reached:

$$\langle \texttt{halt;}, \sigma\rangle \to \langle \texttt{halting}, \sigma\rangle \qquad \text{(SmallStep-Halt)}$$

Now any abruptly terminated program reduces to a special configuration of the form $\langle \texttt{halting}, \sigma\rangle$. Recall that our plan, however, was to terminate the computation with a normal configuration of the form $\langle \{\}, \sigma\rangle$, regardless of whether the program terminates normally or abruptly. Like in the big-step SOS above, the naive solution to transform a step producing a halting configuration into a normal step using the rule

$$\frac{\langle s, \sigma\rangle \to \langle \texttt{halting}, \sigma\rangle}{\langle s, \sigma\rangle \to \langle \{\}, \sigma\rangle}$$

does not work. Indeed, consider a situation where the rule (SmallStep-Seq-Halt) above could apply. There is nothing to prevent the naive rule above to interfere and transform the halting premise of (SmallStep-Seq-Halt) into a normal step producing a $\{\}$, which can be further fed to the conventional rule for sequential composition, (SmallStep-Seq-Arg1) in Figure 3.15, hereby continuing the execution of the program as if no abrupt termination took place.

If one is willing to waste a computational step in order to explicitly dissolve the halting configuration replacing it by a normal one, then one can add the following simple small-step SOS rule:

$$\langle \texttt{halting}, \sigma\rangle \to \langle \{\}, \sigma\rangle \qquad \text{(SmallStep-Halting)}$$

Although it may look like this rule has the same final effect as the wrong conditional conditional rule above, in the sense that it reduces the halting statement in two steps instead of one to the empty-block configuration, note that this difference in the number of steps is actually crucial here! The above works because small-step SOS performs only one step at a time, both globally and in the rule premises.

Although not perfect, because it wastes one step, we find that the rule above gives us a good trade-off between elegance and computational intrusiveness (after all, wasting a step in a deterministic manner may be acceptable in many situations). Supposing that one wants to waste no computational steps as an artifact of the particular small-step SOS approach chosen, there is no immediate way to terminate the program with a normal result configuration of the form $\langle \{\}, \sigma\rangle$ both when the program terminates abruptly and when it terminates normally. One possibility, also suggested for big-step SOS above and followed in the subsequent MSOS definition for abrupt termination in Section 3.6, is to add an auxiliary `top` language construct. With that, we can change the small-step SOS rule for variable declarations to reduce the top-level program to its body statement wrapped under this `top` construct, and then add corresponding rules to propagate normal steps under the `top` while catching the halting steps and transforming them into normal steps at the top-level. Here are four small-step SOS rules which almost achieve this (the first rule replaces the previous one for variable declarations); see also Exercise 96:

$$\langle \texttt{int}\, xl\texttt{;}\ s\rangle \to \langle \texttt{top}\ s, (xl \mapsto 0)\rangle$$

$$\frac{\langle s, \sigma\rangle \to \langle s', \sigma'\rangle}{\langle \texttt{top}\ s, \sigma\rangle \to \langle \texttt{top}\ s', \sigma'\rangle}$$

$$\langle \texttt{top}\ \{\}, \sigma\rangle \to \langle \{\}, \sigma\rangle$$

$$\frac{\langle s, \sigma\rangle \to \langle \texttt{halting}, \sigma\rangle}{\langle \texttt{top}\ s, \sigma\rangle \to \langle \{\}, \sigma\rangle}$$

We said the rules above "almost" achieve non-wasteful reduction because the third rule still wastes one step to eliminate the unnecessary `top` construct once its statement has been reduced. This rule is easy to avoid though. All we have to do is to add an additional case for each of the first two rules, namely a case where the argument statement of the `top` construct is {}, and in that case to replace `top` {} with {}; see also Exercise 97.

Denotational Semantics

Since both expressions and statements can abruptly terminate, like in the previous semantics we have to provide a means for the denotation functions for expressions and statements to flag when abrupt termination is intended. This way, the denotation of programs can catch the abrupt termination flag and yield the expected state (recall that we want to see normal termination of programs regardless of whether that happens abruptly or not). Specifically, we change the previous denotation functions

$$\begin{aligned}
&[\![_]\!] : AExp \to (State \rightharpoonup Int)\\
&[\![_]\!] : BExp \to (State \rightharpoonup Bool)\\
&[\![_]\!] : Stmt \to (State \rightharpoonup State)
\end{aligned}$$

into denotation functions of the form

$$\begin{aligned}
&[\![_]\!] : AExp \to (State \rightharpoonup Int \cup \{error\})\\
&[\![_]\!] : BExp \to (State \rightharpoonup Bool \cup \{error\})\\
&[\![_]\!] : Stmt \to (State \rightharpoonup State \times \{halting, ok\})
\end{aligned}$$

as described below. Before we proceed, note that the new denotation functions will still associate partial functions to syntactic categories. While the new semantics will be indeed able now to catch and terminate when a division by zero takes place, it will still not be able to catch non-termination of programs; the denotation of those programs will still stay undefined. Strictly technically speaking, the denotations of expressions will now always be defined, because the only source of expression undefinedness in IMP was division by zero. However, for the same reason it is good practice in small-step SOS to have the same type of configurations both to the left and to the right of the transition arrow ($\to$), it is good practice in denotational semantics to work with domains of partial functions instead of ones of total functions. This way, we wouldn't have to change these later on if we add new expression constructs to the language that yield undefinedness (such as, e.g., recursive functions).

The denotation functions of expressions need to change in order to initiate a "catchable" error when division by zero takes place, and then to propagate it through all the other expression constructs. We only discuss the denotation of division, the other being simpler. The previous denotation function of division $[\![a_1 \ / \ a_2]\!]$ was defined as

$$[\![a_1 \ / \ a_2]\!]\sigma = \begin{cases} [\![a_1]\!]\sigma \ /_{Int} \ [\![a_2]\!]\sigma & \text{if} \quad [\![a_2]\!]\sigma \neq 0 \\ \bot & \text{if} \quad [\![a_2]\!]\sigma = 0 \end{cases}$$

which is not good enough anymore. To catch and propagate the division-by-zero error, we can modify the denotation of division as follows:

$$[\![a_1 \ / \ a_2]\!]\sigma = \begin{cases} [\![a_1]\!]\sigma \ /_{Int} \ [\![a_2]\!]\sigma & \text{if} \quad [\![a_1]\!]\sigma \neq error \text{ and } [\![a_2]\!]\sigma \neq error \text{ and } [\![a_2]\!]\sigma \neq 0 \\ \bot & \text{if} \quad [\![a_1]\!]\sigma = \bot \\ error & \text{if} \quad [\![a_1]\!]\sigma = error \ \text{ or } \ [\![a_2]\!]\sigma = error \ \text{ or } \ [\![a_2]\!]\sigma = 0 \end{cases}$$

The second case above is necessary, because we want $[\![a_1 \ / \ a_2]\!]\sigma$ to be undefined, and not *error*, when $[\![a_1]\!]\sigma = \bot$ and $[\![a_2]\!]\sigma = 0$.

Like in the previous semantics, the implicit expression errors need to propagate through the statements and halt the program. The denotation function of statements now returns both a state and a flag. The flag tells whether the state resulted from a normal evaluation or whether it is a halting state that needs to be propagated. Here is the new denotation of assignment:

$$[\![x = a\,;]\!]\sigma = \begin{cases} (\sigma, \mathit{halting}) & \text{if } [\![a]\!]\sigma = \mathit{error} \\ \bot & \text{if } \sigma(x) = \bot \text{ or } [\![a]\!]\sigma = \bot \\ (\sigma[[\![a]\!]\sigma/x], \mathit{ok}) & \text{if otherwise} \end{cases}$$

Above, we chose to halt when $[\![a]\!]\sigma = \mathit{error}$ and $\sigma(x) = \bot$ (the cases are handled in order). The alternative would be to choose undefined instead of halt (see Exercise 100). Our choice to assume that the expression being assigned is always evaluated no matter whether the assigned variable is declared or not, is consistent with the small-step SOS of assignment in IMP: first evaluate the expression being assigned step by step, then write the resulting value to the assigned variable if declared; if undeclared then get stuck (Section 3.3.2). The statement sequential composition construct needs to also properly propagate the halting situation:

$$[\![s_1\ \ s_2]\!]\sigma = \begin{cases} [\![s_1]\!]\sigma & \text{if } 2^{\text{nd}}([\![s_1]\!]\sigma) = \mathit{halting} \\ [\![s_2]\!](1^{\text{st}}([\![s_1]\!]\sigma)) & \text{if } 2^{\text{nd}}([\![s_1]\!]\sigma) = \mathit{ok} \\ \bot & \text{if } [\![s_1]\!]\sigma = \bot \end{cases}$$

We also discuss the denotational semantics of loops `while` (b) s. It now needs to be the fixed point of a (total) function of the form

$$\mathcal{F} : (\mathit{State} \to \mathit{State} \times \{\mathit{halting}, \mathit{ok}\}) \to (\mathit{State} \to \mathit{State} \times \{\mathit{halting}, \mathit{ok}\})$$

The following defines such an $\mathcal{F}$ which has the right type and captures the information that is added by unrolling the loop once (the cases are handled in order, from top to bottom):

$$\mathcal{F}(\alpha)(\sigma) = \begin{cases} \bot & \text{if } [\![b]\!]\sigma = \bot \\ (\sigma, \mathit{halting}) & \text{if } [\![b]\!]\sigma = \mathit{error} \\ (\sigma, \mathit{ok}) & \text{if } [\![b]\!]\sigma = \texttt{false} \\ [\![s]\!]\sigma & \text{if } [\![b]\!]\sigma = \texttt{true} \text{ and } 2^{\text{nd}}([\![s]\!]\sigma) = \mathit{halting} \\ \alpha(1^{\text{st}}([\![s]\!]\sigma)) & \text{if } [\![b]\!]\sigma = \texttt{true} \text{ and } 2^{\text{nd}}([\![s]\!]\sigma) = \mathit{ok} \end{cases}$$

After we modify the denotation functions of almost all expression and statement constructs as explained above (except for the denotations of variable lookup, and of builtin integers and Booleans), we have to also modify the denotation of programs to silently discard the halting signal in case its body statement terminated abruptly (the type of the denotation of programs does not change):

$$[\![\texttt{int}\ xl\texttt{;}\ \ s]\!] = 1^{\text{st}}([\![s]\!](xl \mapsto 0))$$

Finally, we can now also give the denotational semantics of "`halt;`":

$$[\![\texttt{halt;}]\!]\sigma = (\sigma, \mathit{halting})$$

Like for the other semantic extensions in this section, adding the semantics of abrupt termination was easy; the tedious part was to modify the existing semantics to make it aware of abrupt termination.

3.5.4 Adding Dynamic Threads

IMP++ adds threads to IMP, which can be dynamically created and terminated. Like with the previous IMP extensions we keep the syntax and the semantics of threads minimal, and pretend no other features will be added to the language. Recall that as part of our language design experiment, we first only consider thread spawning as a statement, without worrying about thread synchronization. Then, once all the features are put together in the IMP++ language in Section 3.5.6, we will add thread joining synchronization in Section 3.5.7 and, as part of that, will modify thread spawning to be an expression instead of a statement construct, evaluating to the unique identifier of the newly created thread. Therefore, for now we only add a `spawn` statement construct:

$$\textit{Stmt} \quad ::= \quad \texttt{spawn}\ \textit{Block}$$

The semantics of `spawn` S is that the block statement S executes concurrently with the rest of the program, sharing and possibly concurrently modifying the same variables, like threads do. The thread corresponding to S terminates when S terminates and, when that happens, we remove the thread. Like in the previous language extensions, we want programs to terminate normally, no matter whether they make use of threads or not. For example, the programs below create 101 and, respectively, 2 new threads during their execution:

```
int m, n, s ;
n = 100 ;
while (m <= n) {
  spawn {s = s + m ;}
  m = m + 1 ;
}
```

```
int x ;
{
   spawn {x = x + 1 ;}
   spawn {x = x + 10 ;}
}
x = x + 100 ;
```

Yet, we want the result configurations at the end of the execution to look like before in IMP, that is, like `< skip, m |-> 101 & n |-> 100 & s |-> 5050 >` and `< skip, x |-> 111 >`, respectively, in the case of small-step SOS. We grouped the two `spawn` statements in the second program on purpose, to highlight the need for structural equivalences (after the block is eliminated; this will be discussed shortly, in the context of small-step SOS). Recall that the syntax of IMP's sequential composition in Section 3.1 (see Figure 3.1) was deliberately left ambiguous, based on the hypothesis that the semantics of IMP will be given in such a way (left-to-right statement evaluation) that the syntactic ambiguity is irrelevant. Unfortunately, the addition of threads makes the above hard or impossible to achieve modularly in some of the semantic approaches.

Concurrency often implies non-determinism, which is not always desirable. For example, the first program above can also evaluate to a result configuration in which `s` is 0. This happens when the first spawned thread calculates the sum `s + m`, which is 0, but postpones writing it to `s` until all the subsequent 100 new threads complete their execution. Similarly, after the execution of the second program above, `x` can have any of the values 1, 10, 11, 100, 101, 110, 111 (see Exercise 101).

A language designer or semanticist may find it very useful to execute and analyze programs like above in their semantics. Indeed, the existence of certain behaviors or their lack of, may suggest changes in the syntax and the semantics of the language at an early and therefore cheap design stage. For example, one may decide that one's language must be race-free on any variable except some special semaphore variables used specifically for synchronization purposes (this particular decision may be too harsh on implementations, though, which may end up having to rely on complex static analysis front-ends or even ignoring it). We make no such difficult decisions in our simple language here, limiting our goal to the bottom of the spectrum of semantic possibilities: we only aim at giving a semantics that faithfully captures all the program behaviors due to spawning threads.

We make the simplifying assumptions that we have a sequentially consistent memory and that variable read and write operations are atomic and thus unaffected by potential races. For example, if `x` is 0 in a

two-threaded program where one thread is about to write 5 to `x` and the other is about to read `x`, then the only global next steps can be either that the first thread writes 5 to `x` or that the second thread reads 0 as the value of `x`; in other words, we assume that it is impossible for the second thread to read 5 or any other value except 0 as the next small-step step in the program.

Big-Step SOS

Big-step SOS and denotational semantics are the semantical approaches which are the most affected by concurrency extensions of the base language. That is because their holistic view of computation makes it hard or impossible to capture the fine-grained execution steps and behavior interleavings that are inherent to concurrent program executions. Consider, for example, a statement of the form (`spawn` S_1) S_2 in some program state σ. The only thing we can do in big-step SOS is to evaluate S_1 and S_2 in some appropriate states and then to combine their resulting states into a result state. We do not have much room for imagination here: we either evaluate S_1 in state σ and then S_2 in the resulting state, or evaluate S_2 in state σ and then S_1 in the resulting state. The big-step SOS rule for sequential composition already implies the former case provided that `spawn` S can evaluate to whatever state S evaluates to, which is true and needs to be considered anyway. Thus, we can formalize the above into the following two big-step SOS rules, which can be regarded as a rather desperate attempt to use big-step SOS for defining concurrency:

$$\frac{\langle s, \sigma\rangle \Downarrow \langle \sigma'\rangle}{\langle \texttt{spawn}\ s, \sigma\rangle \Downarrow \langle \sigma'\rangle} \qquad \text{(BigStep-Spawn-Arg)}$$

$$\frac{\langle s_2, \sigma\rangle \Downarrow \langle \sigma_2\rangle \quad \langle s_1, \sigma_2\rangle \Downarrow \langle \sigma_1\rangle}{\langle (\texttt{spawn}\ s_1)\ s_2, \sigma\rangle \Downarrow \langle \sigma_1\rangle} \qquad \text{(BigStep-Spawn-Wait)}$$

As expected, the two big-step SOS rules above capture only a limited number of the possible concurrent behaviors even for small and simple programs like the ones discussed above. One may try to change the entire big-step SOS definition of IMP to collect in result configurations all possible ways in which the corresponding fragments of program can evaluate. However, in spite of its non-modularity, there seems to be no easy way to combine, for example, the behaviors of S_1 and of S_2 into the behaviors of (`spawn` S_1) S_2.

Type System using Big-Step SOS

From a typing perspective, `spawn` is nothing but a language construct expecting a statement as argument and producing a statement as result. To type programs using `spawn` statements we therefore add the following typing rule to the already existing typing rules in Figure 3.10:

$$\frac{xl \vdash s : \texttt{stmt}}{xl \vdash \texttt{spawn}\ s : \texttt{stmt}} \qquad \text{(BigStepTypeSystem-Spawn)}$$

Small-Step SOS

Small-step semantics are more appropriate for concurrency, because they allow a finer-grain view of computation. For example, they allow to say that the next computational step of a statement of the form (`spawn` S_1) S_2 comes either from S_1 or from S_2 (which is different from saying that either S_1 or S_2 is evaluated next all the way through, like in big-step SOS). Since in this context `spawn` S_1 is already permitted to advance by the small-step SOS rule for sequential composition, the following three small-step SOS rules achieve the desired behavioral non-determinism caused by concurrent threads ... or at least it may look like:

$$\frac{\langle s, \sigma\rangle \to \langle s', \sigma'\rangle}{\langle \texttt{spawn}\ s, \sigma\rangle \to \langle \texttt{spawn}\ s', \sigma'\rangle} \qquad \text{(SmallStep-Spawn-Arg)}$$

$$\langle \texttt{spawn}\ \{\}, \sigma\rangle \to \langle \{\}, \sigma\rangle \qquad \text{(SmallStep-Spawn-Skip)}$$

$$\frac{\langle s_2, \sigma\rangle \to \langle s_2', \sigma'\rangle}{\langle (\texttt{spawn}\ s_1)\ s_2, \sigma\rangle \to \langle (\texttt{spawn}\ s_1)\ s_2', \sigma'\rangle} \qquad \text{(SmallStep-Spawn-Wait)}$$

The rule (SmallStep-Spawn-Skip) cleans up terminated threads.

Unfortunately, the three rules above are not sufficient to capture all the intended behaviors. Consider, for example, the second program at the beginning of Section 3.5.4. That program was intended to have seven different behaviors with respect to the final value of `x`. Our current small-step SOS misses two of those behaviors, namely those in which `x` results in 1 and 100, respectively.

In order for the program to terminate with `x = 1`, it needs to start the first new thread, calculate the sum `x + 1` (which is 1), then delay writing it back to `x` until after the second and the main threads do their writes of `x`. However, in order for the main thread to be allowed to execute its assignment statement, the two grouped `spawn` statements need to either terminate and become {} so that the rule (SmallStep-Seq-Skip) (see Figure 3.15) applies, or to reduce to only one `spawn` statement so that the rule (SmallStep-Spawn-Wait) above applies. Indeed, these are the only two rules which allow access to the second statement in a sequential composition. The first case is not possible, because, as explained, the first newly created thread cannot be terminated. In order for the second case to happen, since the first `spawn` statement cannot terminate, the only possibility is for the second `spawn` statement to be executed all the way through (which is indeed possible, thanks to the rules (SmallStep-Seq-Arg1) in Figure3.15 and (SmallStep-Spawn-Wait) above) and then eliminated. To achieve this elimination, we may think of adding a new rule, which appears to be so natural that one may even wonder "how did we miss it in our list above?":

$$\langle \texttt{spawn}\ s\ \{\}, \sigma\rangle \to \langle \texttt{spawn}\ s, \sigma\rangle$$

This rule turns out to be insufficient and, once we fix the semantics properly, it will actually become unnecessary, which is why we did not add it above. Nevertheless, if we add this rule, the resulting small-step SOS can also produce the behavior in which `x` = 1 at the end of the execution of the second program at the beginning of Section 3.5.4. However, it is still insufficient to produce the behavior in which `x` = 100.

In order to produce a behavior in which `x` = 100 when executing the second program, the main thread should first execute its `x+100` step (which evaluates to 100), then let the two child threads do their writes to `x`, and then write the 100 to `x`. We have, unfortunately, no rule that allows computations within s_2 in a sequential composition $s_1\ s_2$ where s_1 is different from {} or a `spawn` statement, as it is our case here. What we want is some generalization of the rule (SmallStep-Spawn-Wait) above which allows computations in s_2 whenever it is preceded by a sequence of spawns, possibly grouped in possibly nested blocks. On paper definitions, one can do that rather informally by means of some informal side condition saying so. If one needs to be formal, which is a must when one needs to execute the resulting language definitions as we do here, one can define a special sequent saying that a statement only spawns new threads and does nothing else (in the same spirit as the the $C\surd$ sequents in Exercise 64), and then use it to generalize the rule (SmallStep-Spawn-Wait) above. However, that would be a rather particular and potentially non-modular solution (what if later on we add agents or other mechanisms for concurrency or grouping?).

Our general solution is to instead enforce the sequential composition of IMP to be structurally associative, using the following structural identity:

$$(s_1\ s_2)\ s_3 \equiv s_1\ (s_2\ s_3) \qquad \text{(SmallStep-Seq-Assoc)}$$

That means that the small-step SOS reduction rules now apply *modulo* the associativity of sequential composition, that is, that it suffices to find a structurally equivalent representative of a syntactic term which allows a small-step SOS rule to apply. In our case, the program obtained from the original program after eliminating the block surrounding the two `spawn` statements, is structurally equivalent to one whose first statement is the first `spawn` and whose second statement is the sequential composition of the second `spawn` and the assignment of the main thread, and that structurally equivalent program allows all seven desired behaviors, so the original program also allows them. It is important to understand that we cannot avoid enforcing associativity (or, alternatively, the more expensive solution discussed above) by simply parsing the original program so that we start with a right-associative arrangement of the sequentially composed statements. The problem is that right-associativity may be destroyed as the program executes, for example when applying the true/false rules for the `if` statement, so it needs to be dynamically enforced.

Structural identities are not easy to execute and implement, because they can quickly yield an exponential explosion in the number of terms that need to be matched by rules. Since in our particular case we only need the fully right-associative representative of each sequential composition, we can even replace the structural identity above by a small-step SOS rule. The problem with doing that, though, is that the intended computational granularity of the language is significantly modified; for example, the application of a true/false rule for the conditional statement may trigger as many such rearrangement steps as statements in the chosen branch; such rearrangement steps could dominate the total number of steps seen in some computations.

Unfortunately, there is an important syntactic detail that we purposely left out in order to highlight a common problem when using syntactic approaches to operational semantics: it may be possible that terms change their syntactic category during reduction, which may require extending the syntax of some constructs or otherwise require significant changes in the existing semantics of other language constructs. Consider, for example, the rule (SmallStep-Spawn-Arg) above. Syntactically, the rule does not parse, because `spawn` takes a block, not a statement, as argument. We should reduce its applicability only to blocks (i.e., require s, s' range over blocks only) in order to obey the declared syntax, but note that if we do so then the rule can only be applied in very particular situations. Indeed, if s is a block $\{\ s_1\ \}$, then the rule (SmallStep-Block) in Figure 3.15 can be applied to reduce s to s_1, but, however, the restricted rule (SmallStep-Spawn-Arg) can only be applied with this premise if s_1 is a also block, so s has the form $\{\{\ldots\}\}$, which is unlikely.

One could argue that the culprit for the above is in fact the semantic rule of blocks, (SmallStep-Block) in Figure 3.15, because it is allowed to change the syntactic category of its code from *Block* to *Stmt*. Although there is no SOS restriction saying that the type of the code cannot be changed during the reduction process (in fact this happens quite frequently, e.g., expressions reduce to integers, etc.), let us briefly discuss how the semantics of blocks needs to change in order to reduce blocks to blocks, and what additional problems that generates. First, we would need to propagate the reduction relation through the block construct:

$$\frac{\langle s, \sigma\rangle \rightarrow \langle s', \sigma\rangle}{\langle\{\ s\ \}, \sigma\rangle \rightarrow \langle\{\ s'\ \}, \sigma\rangle}$$

Then, once the statement within the block reduces completely, the entire block reduces to the empty block:

$$\langle\{\ \{\}\ \}, \sigma\rangle \rightarrow \langle\{\}, \sigma\rangle$$

While the above may seem a reasonable alternative to the semantics of blocks, it is more involved than our original semantics, slower when we execute it, non-modular (because in order to fix the semantics of a language construct we had to significantly modify the semantics of another language construct which worked

fine before), and worse, still does not solve our problem. Indeed, there is no way now to reduce S_2 in a fragment of the form { `spawn` S_1 } S_2, because we have to first completely eliminate the block before we move on to reducing S_2. The rule (SmallStep-Spawn-Wait) above does not work anymore. More involved changes to the semantics are needed in order to fix the above.

Instead of changing the semantics of existing constructs, let us now investigate more modular approaches to fix the semantics of `spawn`. One possibility is to structurally enforce `spawn` to only take blocks and to only reduce the statements inside the blocks, which can be achieved by changing the rule (SmallStep-Spawn-Arg) above as follows (and keeping the rules (SmallStep-Spawn-Skip) and (SmallStep-Spawn-Wait) unchanged):

$$\frac{\langle s, \sigma\rangle \to \langle s', \sigma'\rangle}{\langle \texttt{spawn}\ \{\ s\ \}, \sigma\rangle \to \langle \texttt{spawn}\ \{\ s'\ \}, \sigma'\rangle}$$

We also have to add one rule to eliminate the block once its enclosed statement completely reduces:

$$\langle \texttt{spawn}\ \{\ \{\}\ \}, \sigma\rangle \to \langle \{\}, \sigma\rangle$$

The last rule above is somewhat artificial, particularly in combination with the rule (SmallStep-Spawn-Skip), which is still necessary. Additionally, both rules above make the connection between `spawn` and blocks too tight, indicating that the semantics of `spawn` may need to be revisited if we extend the language with other constructs for blocks (e.g., blocks with local variable declarations), or if we extend `spawn` to take other statements besides blocks are argument (e.g., function calls).

Another solution, which we prefer here, is to keep the three small-step SOS rules (SmallStep-Spawn-Arg), (SmallStep-Spawn-Skip) and (SmallStep-Spawn-Wait) unchanged, but to extend the syntax of `spawn` to take any statements as argument, not only blocks:

$$\textit{Stmt} \quad ::= \ \mid\ \texttt{spawn}\ \textit{Stmt}$$

This can be done either by replacing the previous `spawn` *Block* construct with the above or by simply adding the construct above to the already existing grammar of IMP and thus overloading the previous `spawn` *Block* construct. In either case, we should make sure that the original program only uses the syntax `spawn` *Block*.

Denotational Semantics

As already mentioned when we discussed the big-step SOP of `spawn` above, big-step SOS and denotational semantics are the semantic approaches which are the most affected by the addition of concurrency to IMP. While big-step SOS was somewhat able to capture some of the non-determinism due to concurrency, unfortunately, denotational semantics cannot do even that easily. The notes on denotational semantics in Section 3.4.3 mention the use of powerdomains and resumptions when giving concurrent semantics to languages. These are complex denotational semantics topics, which are not easy to use even by experts. Moreover, they yield semantics which are either non-executable at all or very slow. Since the main emphasis of this book is on operational semantics, we do not discuss these advanced topics in this book. Instead, we simply dissolve the `spawn` statements, so we can still execute IMP++ programs using denotational semantics:

$$[\![\texttt{spawn}\ s]\!] = [\![s]\!]$$

Of course, spawning threads is completely useless with our denotational semantics here.

3.5.5 Adding Local Variables

Blocks with local variable declarations are common to many imperative, object oriented and functional languages. In IMP++ we follow the common imperative approach where variables can be declared anywhere inside a block, their *scope* being the rest of the block (whatever follows the declaration); in other words, a declared variable is not visible before its declaration or outside the block declaring it. A declaration of a variable that appears in the scope of another declaration of the same variable is said to *shadow* the original one. For example, the values of `y` and `z` are 1 and 2, respectively, right before the end of the following two IMP++ blocks (none of the variables are visible outside the given blocks):

```
{    int x, y, z ;
     x = 1 ;
     y = x ;
     int x ;
     x = 2 ;
     z = x ;    }
```

```
{    int x, y, z ;
     x = 1 ;
     { int x ;
       x = 2 ;
       z = x ; }
     y = x ;    }
```

As already explained in the preamble of Section 3.5, the introduction of local variables suggests some syntactic and semantic simplifications in the already existing definition of IMP. For example, since local variable declarations generalize the original global variable declarations of IMP, there is no need for the original global declarations. Thus, programs can be just statements. Therefore, we remove the top-level variable declaration and add the following new syntax:

$$\begin{array}{lcl} \mathit{Stmt} & ::= & \texttt{int}\ \mathbf{List}\{\mathit{Id}\}\texttt{;} \\ \mathit{Pgm} & ::= & \mathit{Stmt} \end{array}$$

In each of the semantics, we assume that all the previous rules referring to global variable declarations are removed. Moreover, for semantic clarity, we assume that variable declarations can only appear in blocks (a hypothetical parser can reject those programs which do not conform).

An immediate consequence of the language extension and conventions above is that programs now evaluate to empty states. Indeed, since the initial state in which a program is evaluated is empty and since variable declarations are local to the blocks in which they occur, the state obtained after evaluating a program is empty. This makes it somewhat difficult to test this IMP extension. To overcome this problem, one can either add an output statement to the language like in Section 3.5.2 (we will do this in Section 3.5.6), or manually initialize the state with some "global" variables and then use them undeclared in the program.

It would be quite tedious to give semantics directly to the syntactic constructs above. Instead, we are going to propose another construct which is quite common and easy to give semantics to in each of the semantic approaches, and then statically translate the constructs above into the new construct. The new construct has the following syntax:

$$\mathit{Stmt} \quad ::= \quad \texttt{let}\ \mathit{Id} = \mathit{AExp}\ \texttt{in}\ \mathit{Stmt}$$

Its semantics is as expected: the arithmetic expression is first evaluated to an integer, then the declared variable is bound to that integer possibly shadowing an already existing binding of the same variable, then the statement is evaluated in the new state, and finally the environment before the execution of the `let` is recovered. The latter step is executed by replacing the value of the variable after the execution of the statement with whatever it was before the `let`, possibly undefined. All the other side effects generated by the statement are kept.

Here we propose a simple set of macros which automatically desugar any program using the block and local variable constructs into a program containing only `let` and the existing IMP constructs:

$$
\begin{array}{rcl}
(s_1\ s_2)\ s_3 & = & s_1\ (s_2\ s_3) \\
\texttt{int}\, xl, x\texttt{;}\ s & = & \texttt{int}\, xl\texttt{;}\ \texttt{let}\ x = 0\ \texttt{in}\ s \\
\texttt{int}\, \cdot\texttt{;}\ s & = & s \\
s\ \texttt{int}\, xl\texttt{;} & = & s \\
\{\,\texttt{int}\, xl\texttt{;}\,\} & = & \{\}
\end{array}
$$

The macros above are expected to be iteratively applied in order, from top to bottom, until no macro can be applied anymore. When that happens, there will be no variable declaration left; all of these would have been systematically replaced by the `let` construct. The first macro enforces right-associativity of sequential composition. This way, any non-terminal variable declaration (i.e., one which is not the last statement in a block) will be followed, via a sequential composition, by the remainder of the block. The second and the third macros iteratively replace each non-terminal variable declaration by a corresponding `let` statement, while the fourth and the fifth eliminate the remaining (and useless) terminal variable declarations. From here on we assume that these syntactic desugaring macros are applied statically, before any of the semantic rules is applied; this way, the subsequent semantics will only be concerned with giving semantics to `let`.

Big-Step SOS

The big-step SOS rule of `let` follows quite closely its informal description above:

$$
\frac{\langle a, \sigma\rangle \Downarrow \langle i\rangle \qquad \langle s, \sigma[i/x]\rangle \Downarrow \langle \sigma'\rangle}{\langle \texttt{let}\ x = a\ \texttt{in}\ s, \sigma\rangle \Downarrow \langle \sigma'[\sigma(x)/x]\rangle} \qquad \text{(BigStep-Let)}
$$

In words, the arithmetic expression a is first evaluated to some integer i. Then the statement s is evaluated in state $\sigma[i/x]$, resulting in a state σ'. Then we return $\sigma'[\sigma(x)/x]$ as the result of the `let` statement, that is, the state σ' in which we update the value of x to whatever x was bound to originally in σ. We *cannot* return σ' as the result of the `let`, because σ' binds x to some value which is likely different from what x was bound to in σ (note that s is allowed to assign to x, although that is not the main problem here). If x is undefined in σ, that is, if $\sigma(x) = \bot$, then x is also undefined in $\sigma'[\sigma(x)/x]$: indeed, recall from Section 2.4.6 that $\sigma'[\bot/x]$ "undefines" x in σ'.

Since programs are now just statements, their big-step SOS simply reduces to that of statements:

$$
\frac{\langle s, \cdot\rangle \Downarrow \langle \sigma\rangle}{\langle s\rangle \Downarrow \langle \sigma\rangle} \qquad \text{(BigStep-Pgm)}
$$

Hence, programs are regarded as statements that execute in the empty state. However, since variable accesses in IMP require the variable to be declared and since all variable declarations are translated into `let` statements, which recover the state in the variable they bind after their execution, we can conclude that σ will always be empty whenever a sequent of the form $\langle s\rangle \Downarrow \langle \sigma\rangle$ is derivable using the big-step SOS above. The rule above is, therefore, not very useful. All it tells us is that if $\langle s\rangle \Downarrow \langle \sigma\rangle$ is derivable then s is a well-formed program which terminates. The idea of reducing the semantics of statement-programs to that of statements in an initial state is general though, and it becomes practical when we add other features to the language (see Section 3.5.6).

Type System using Big-Step SOS

Following the intuitions above, to type programs using `let` statements we add the following typing rules to the already existing typing rules in Figure 3.10:

$$\frac{xl \vdash a : \texttt{int} \quad xl, x \vdash s : \texttt{stmt}}{xl \vdash \texttt{let}\ x = a\ \texttt{in}\ s : \texttt{stmt}} \qquad \text{(BigStepTypeSystem-Let)}$$

$$\frac{\cdot \vdash s : \texttt{stmt}}{\vdash s : \texttt{pgm}} \qquad \text{(BigStepTypeSystem-Pgm)}$$

Small-Step SOS

The small-step SOS of `let` $x = a$ `in` s can be described in words as follows: first evaluate a stepwise, until it becomes some integer; then evaluate s stepwise in a state originally binding x to the integer to which a evaluates, but making sure that the value bound to x is properly updated during each step in the evaluation of s and it is properly recovered after each step to whatever it was in the environment outside the `let` (so other potentially interleaved rules taking place outside the `let` see a consistent state); finally, dissolve the `let` when its enclosed statement becomes {}. All these can be achieved with the following three rules, without having to change anything in the already existing small-step SOS of IMP:

$$\frac{\langle a, \sigma\rangle \to \langle a', \sigma\rangle}{\langle \texttt{let}\ x = a\ \texttt{in}\ s, \sigma\rangle \to \langle \texttt{let}\ x = a'\ \texttt{in}\ s, \sigma\rangle} \qquad \text{(SmallStep-Let-Exp)}$$

$$\frac{\langle s, \sigma[i/x]\rangle \to \langle s', \sigma'\rangle}{\langle \texttt{let}\ x = i\ \texttt{in}\ s, \sigma\rangle \to \langle \texttt{let}\ x = \sigma'(x)\ \texttt{in}\ s', \sigma'[\sigma(x)/x]\rangle} \qquad \text{(SmallStep-Let-Stmt)}$$

$$\langle \texttt{let}\ x = i\ \texttt{in}\ \{\}, \sigma\rangle \to \langle \{\}, \sigma\rangle \qquad \text{(SmallStep-Let-Done)}$$

Note that if x was undeclared before the `let` then so it stays after each application of the rule (SmallStep-Let-Stmt), because $\sigma'[\bot/x]$ "undefines" σ' in x (see Section 2.4.6).

Like in big-step SOS, the semantics of programs (which are now statements) reduces to that of statements. One simple way to achieve that in small-step SOS is to add a rule $\langle s\rangle \to \langle s, \cdot\rangle$, in the same spirit as the small-step SOS of IMP in Section 3.3.2 (Figure 3.15). However, like in Exercise 67, one could argue that this approach is wasteful, since one does not want to spend a step only to initialize the empty state (this can be regarded as poor style). For demonstration purposes and for the sake of a semantic variation, we here prefer a non-wasteful small-step SOS rule of programs:

$$\frac{\langle s, \cdot\rangle \to \langle s', \sigma\rangle}{\langle s\rangle \to \langle s', \sigma\rangle} \qquad \text{(SmallStep-Pgm)}$$

One could still argue that the rule above is not perfect, because the configuration $\langle\{\}\rangle$ is frozen; thus, while any other (terminating) program eventually reduces to a configuration of the form $\langle\{\}, \cdot\rangle$, {} itself does not. To address this non-uniformity problem, one can add a rule $\langle\{\}\rangle \to \langle\{\}, \cdot\rangle$; this wastes a step, indeed, but this case when the entire program is just {} is expected to be very uncommon. A conceptually cleaner alternative is to replace the rule (SmallStep-Pgm) above with a structural identity $\langle s\rangle \equiv \langle s, \cdot\rangle$ identifying each program configuration with a statement configuration holding an empty state. This can be easily achieved in Maude using an equation, but it can be harder to achieve in other rewrite systems providing support and semantics only for rewrite rules but not for equations.

Denotational Semantics

The denotational semantics of the `let` construct is very compact and elegant:

$$[\![\texttt{let}\ x = a\ \texttt{in}\ s]\!]\sigma = ([\![s]\!](\sigma[[\![a]\!]\sigma/x]))[\sigma(x)/x] \qquad \text{(DENOTATIONAL-LET)}$$

Like in the other semantics above, this works because $\sigma'[\bot/x]$ "undefines" x in σ' (see Section 2.4.6).

3.5.6 Putting Them All Together: First Attempt

In this section and the next we analyze the modularity of the various semantic approaches discussed so far by defining the IMP++ language, which puts together all the language features discussed so far plus more. Recall from the preamble of Section 3.5 that, as part of our language design experiment, we first consider `spawn` to be a statement construct with no explicit thread synchronization constructs. This is also what we did in Section 3.5.4. Our objective in this section is therefore to "make everything work" under this assumption, and then, in Section 3.5.7, we change `spawn` into an expression construct that evaluates to a unique thread identifier, and add `join` thread synchronization. Therefore, the IMP++ language discussed in this section removes from IMP the global variable declarations and adds the following constructs:

$$\begin{array}{rcl} \mathit{AExp} & ::= & \texttt{++}\,\mathit{Id} \\ & | & \texttt{read()} \\ \mathit{Stmt} & ::= & \texttt{print(}\mathit{AExp}\texttt{);} \\ & | & \texttt{halt;} \\ & | & \texttt{spawn}\ \mathit{Block} \\ & | & \texttt{int}\ \mathbf{List}\{\mathit{Id}\}\texttt{;} \\ \mathit{Pgm} & ::= & \mathit{Stmt} \end{array}$$

We consider the semantics of these constructs adopted in the previous sections.

To make the design of IMP++ more permissive in what regards its possible implementations, we shall opt for maximum non-determinism whenever such design choices can be made. For example, in the case of division expressions a_1 / a_2, we want to capture all possible behaviors (recall that division is non-deterministic) due to the possibly interleaved evaluations of a_1 and a_2, including all possible abruptly terminated behaviors generated when a_2 evaluates to 0. In particular, we want to also capture those behaviors where a_1 is not completely evaluated. The rationale for this language design decision is that we want to allow maximum flexibility to implementations of IMP++; for example, some implementations may choose to evaluate a_1 and a_2 in two concurrent threads and to stop with abrupt termination as soon as the thread evaluating a_2 yields 0.

Somewhat surprisingly, when adding several new features together to a language, it is not always sufficient to simply apply all the global, non-modular changes that are required for each feature in isolation. We sometimes have to additionally consider the semantic implications of the various combinations of features. For example, the addition of side-effects in combination with division-by-zero abrupt termination requires the addition of new rules to catch specific new behaviors due to this particular combination. Indeed, the evaluation of a_1 / a_2, for example, may abruptly terminate with the current state precisely when a_2 is evaluated to zero, but it can also terminate with the state obtained after evaluating a_1 or parts of a_1, as discussed above.

Also, a language design decision needs to be made in what regards the state of abruptly terminated programs. One option is to simply enclose the local state when the abrupt termination flag was issued. This option is particularly useful for debugging. However, as argued in Section 3.5.3, we want abruptly terminated programs to behave the same way as the normally terminated programs. Since normally terminated programs now empty the state after their execution, we will give the semantics of abruptly terminated programs to also

empty the state. Whenever easily possible, we will give the semantics of abruptly terminated statements to return after their evaluation a state binding precisely the same variables as before their evaluation.

A design decision also needs to be made in what regards the interaction between abrupt termination and threads. We choose that abrupt termination applies to the entire program, no matter whether it is issued by the main program or by a spawned thread. An alternative would be that abrupt termination only applies to the spawned thread if issued by a spawned thread, or to the entire program if issued by the main program. Yet another alternative is to consider the main program as an ordinary thread, and an abrupt termination issued by the main program to only stop that thread, allowing the other spawned threads to continue their executions.

Finally, there is an interesting aspect regarding the interaction between blocks and threads. In conventional programming languages, spawned threads continue to execute concurrently with the rest of the program regardless of whether the language construct which generated them completed its execution or not. For example, if function $f()$ spawns a thread and then immediately returns 1, then the expression $f() + f()$ evaluates to 2 and the two spawned threads continue to execute concurrently with the rest of the program. We do not have functions in IMP++, but we still want the spawned threads to continue to execute concurrently with the rest of the program even after the completion of the block within which the spawn statements were executed. For example, we would like the IMP++ program (its `let`-desugared variant is shown to the right)

```
{
   int x ;
   {
      int y ;
      spawn { x := x + 1 ; }
   }
   spawn { x := x + 10 ; }
   print(x) ;
}
```

```
{
   let x = 0 in (
      {
         let y = 0 in
            spawn { x := x + 1 ; }
      }
      spawn { x := x + 10 ; }
      print(x) ; )
}
```

to manifest four behaviors, where x is 0, 1, 10, and 11, and not only two (where x is 1, 11) as it would be the case if the first spawn statement were not allowed to transcend its surrounding block.

Below we discuss, for each semantic approach, the changes that we have to apply to the semantics of IMP in order to extend it to IMP++, highlighting changes that cannot be mechanically derived from the changes required by each of IMP++'s features when considered in isolation.

Big-Step SOS

To accommodate the side effects generated by variable increment on the state and by `read()` on the input buffer, and the possible abrupt termination generated by division-by-zero, the arithmetic expression sequents need to change from $\langle a, \sigma\rangle \Downarrow \langle i\rangle$ to $\langle a, \sigma, \omega\rangle \Downarrow \langle i, \sigma', \omega'\rangle$ for normal termination and to $\langle a, \sigma, \omega\rangle \Downarrow \langle \texttt{error}, \sigma', \omega'\rangle$ for abrupt termination, and similarly for Boolean expressions, where σ, ω and σ',ω' are the states and input buffers before and after the evaluation of a, respectively. Also, the original elegant big-step SOS rules for expressions need to change to take into account the new configurations, the various side effects, and the abrupt termination due to division-by-zero. For example, the IMP big-step SOS rule for division in Section 3.2.2, namely

$$\frac{\langle a_1, \sigma\rangle \Downarrow \langle i_1\rangle \quad \langle a_2, \sigma\rangle \Downarrow \langle i_2\rangle}{\langle a_1 \ / \ a_2, \sigma\rangle \Downarrow \langle i_1 +_{Int} i_2\rangle} \text{ if } i_2 \neq 0$$

changes into the following six rules:

$$\frac{\langle a_1, \sigma, \omega\rangle \Downarrow \langle i_1, \sigma_1, \omega_1\rangle \quad \langle a_2, \sigma_1, \omega_1\rangle \Downarrow \langle i_2, \sigma_2, \omega_2\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega\rangle \Downarrow \langle i_1 \ /_{Int} \ i_2, \sigma_2, \omega_2\rangle} \text{ if } i_2 \neq 0$$

$$\frac{\langle a_1, \sigma, \omega\rangle \Downarrow \langle i_1, \sigma_1, \omega_1\rangle \quad \langle a_2, \sigma_1, \omega_1\rangle \Downarrow \langle \texttt{error}, \sigma_2, \omega_2\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega\rangle \Downarrow \langle \texttt{error}, \sigma_2, \omega_2\rangle}$$

$$\frac{\langle a_1, \sigma, \omega\rangle \Downarrow \langle \texttt{error}, \sigma_1, \omega_1\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega\rangle \Downarrow \langle \texttt{error}, \sigma_1, \omega_1\rangle}$$

$$\frac{\langle a_1, \sigma_2, \omega_2\rangle \Downarrow \langle i_1, \sigma_1, \omega_1\rangle \quad \langle a_2, \sigma, \omega\rangle \Downarrow \langle i_2, \sigma_2, \omega_2\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega\rangle \Downarrow \langle i_1 \ /_{Int} \ i_2, \sigma_1, \omega_1\rangle} \text{ if } i_2 \neq 0$$

$$\frac{\langle a_1, \sigma_2, \omega_2\rangle \Downarrow \langle \texttt{error}, \sigma_1, \omega_1\rangle \quad \langle a_2, \sigma, \omega\rangle \Downarrow \langle i_2, \sigma_2, \omega_2\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega\rangle \Downarrow \langle \texttt{error}, \sigma_1, \omega_1\rangle}$$

$$\frac{\langle a_2, \sigma, \omega\rangle \Downarrow \langle \texttt{error}, \sigma_2, \omega_2\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega\rangle \Downarrow \langle \texttt{error}, \sigma_2, \omega_2\rangle}$$

Like for the individual features, rules like the above attempt to capture the intended nondeterministic evaluation strategy of the arithmetic operators, but they end up capturing only the non-deterministic choice semantics. In the case of division, we also have to add the rule for abrupt termination in the case of a division-by-zero, like the rule (BigStep-Div-By-Zero) in Section 3.5.3. However, since we want to capture all the non-deterministic behaviors that big-step SOS can detect, we actually need three such rules:

$$\frac{\langle a_1, \sigma, \omega\rangle \Downarrow \langle i_1, \sigma_1, \omega_1\rangle \quad \langle a_2, \sigma_1, \omega_1\rangle \Downarrow \langle 0, \sigma_2, \omega_2\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega\rangle \Downarrow \langle \texttt{error}, \sigma_2, \omega_2\rangle}$$

$$\frac{\langle a_1, \sigma_2, \omega_2\rangle \Downarrow \langle i_1, \sigma_1, \omega_1\rangle \quad \langle a_2, \sigma, \omega\rangle \Downarrow \langle 0, \sigma_2, \omega_2\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega\rangle \Downarrow \langle \texttt{error}, \sigma_1, \omega_1\rangle}$$

$$\frac{\langle a_2, \sigma, \omega\rangle \Downarrow \langle 0, \sigma_2, \omega_2\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega\rangle \Downarrow \langle \texttt{error}, \sigma_2, \omega_2\rangle}$$

The big-step SOS sequents for statements $\langle s, \sigma\rangle \Downarrow \langle \sigma'\rangle$ also need to change, to hold both the input/output buffers and the termination flag. We use sequents of the form $\langle s, \sigma, \omega_{in}\rangle \Downarrow \langle \sigma', \omega'_{in}, \omega_{out}\rangle$ for the case when s terminates normally and sequents of the form $\langle s, \sigma, \omega_{in}\rangle \Downarrow \langle \texttt{halting}, \sigma', \omega'_{in}, \omega_{out}\rangle$ for the case when s terminates abruptly; here $\omega_{in}, \omega'_{in} \in \mathit{Buffer}$ are the input buffers before and, respectively, after the evaluation of statement s, and $\omega_{out} \in \mathit{Buffer}$ is the output produced during the evaluation of s. All the big-step SOS rules for statements need to change to accommodate the new sequents, making sure that side effects and abrupt termination are properly propagated, like we did in Sections 3.5.1 and 3.5.3 (but for the new configurations). We also have to include big-step SOS rules for input/output like those in Section 3.5.2, for local variable declarations like those in Section 3.5.5, and for dynamic threads like those in Section 3.5.4, but also modified

to work with the new configurations and to propagate abrupt termination. Recall from the preamble of this section that we want abruptly terminated programs to terminate similarly to the normal programs, that is, with an empty state in the configuration. This can be easily achieved in the rule for programs by simply emptying the state when the program statement terminates abruptly. However, in the case of big-step SOS it is relatively easy to ensure a stronger property, namely that each statement leaves the state in a consistent shape after its evaluation, no matter whether that is abruptly terminated or not. All we have to do is to also clean up the state when the halting signal is propagated through the `let` construct. For clarity, we show both big-step SOS rules for `let`:

$$\frac{\langle a, \sigma, \omega_{in}\rangle \Downarrow \langle i, \sigma', \omega'_{in}\rangle \quad \langle s, \sigma'[i/x], \omega'_{in}\rangle \Downarrow \langle \sigma'', \omega''_{in}, \omega_{out}\rangle}{\langle \texttt{let}\ x = a\ \texttt{in}\ s, \sigma, \omega_{in}\rangle \Downarrow \langle \sigma''[\sigma'(x)/x], \omega''_{in}, \omega_{out}\rangle}$$

$$\frac{\langle a, \sigma, \omega_{in}\rangle \Downarrow \langle i, \sigma', \omega'_{in}\rangle \quad \langle s, \sigma'[i/x], \omega'_{in}\rangle \Downarrow \langle \texttt{halting}, \sigma'', \omega''_{in}, \omega_{out}\rangle}{\langle \texttt{let}\ x = a\ \texttt{in}\ s, \sigma, \omega_{in}\rangle \Downarrow \langle \texttt{halting}, \sigma''[\sigma'(x)/x], \omega''_{in}, \omega_{out}\rangle}$$

Recall from Section 2.4.6 that $\sigma''[\bot/x]$ "undefines" x in σ''.

Finally, the big-step SOS sequents and rules for programs also have to change, to take into account the fact that programs are now just statements like in Section 3.5.5, that they take an input and that they yield both the unconsumed input and an output like in Section 3.5.2, and that programs manifest normal termination behavior no matter whether their corresponding statement terminates normally or not:

$$\frac{\langle s, xl \mapsto 0, \omega_{in}\rangle \Downarrow \langle \sigma, \omega'_{in}, \omega_{out}\rangle}{\langle s, \omega_{in}\rangle \Downarrow \langle \sigma, \omega'_{in}, \omega_{out}\rangle}$$

$$\frac{\langle s, xl \mapsto 0, \omega_{in}\rangle \Downarrow \langle \texttt{halting}, \sigma, \omega'_{in}, \omega_{out}\rangle}{\langle s, \omega_{in}\rangle \Downarrow \langle \sigma, \omega'_{in}, \omega_{out}\rangle}$$

Unfortunately, as seen above, all the configurations, sequents and rules of IMP extended with any one of the features of IMP++ had to change again when we added all the features together. This highlights, again, the poor modularity of big-step SOS. But, even accepting the poor modularity of big-step SOS, do we at least get all the behaviors expressible in a big-step SOS style by simply putting together and adjusting accordingly all the rules of the individual features? Unfortunately, not. Recall from the preamble of Section 3.5.6 that we want spawned threads to execute concurrently with the rest of the program also after their surrounding blocks complete. In other words, we would like to also capture behaviors of, e.g., ($\texttt{let}\ x = a\ \texttt{in}\ s_1$) s_2, where a is first evaluated, then s_2, and then s_1. We can easily add a big-step SOS rule to capture this particular situation, but is that enough? Of course not, because there are many other similar patters in which we would like to allow the evaluation of s_2 before the preceding statement completes its evaluation. For example, one can replace `spawn` s_1 above by another `let` holding a spawn statement, or by `spawn` s'_1 `spawn` s'_2, or by combinations of such patterns. A tenacious reader could probably find some complicated way to allow all these behaviors. However, it is fair to say that big-step SOS has simply not been conceived to deal with concurrent languages, and can only partially deal with non-determinism.

Type System using Big-Step SOS

The IMP++ type system is quite simple and modular. We simply put together all the typing rules of the individual language features discussed in Sections 3.5.1, 3.5.2, 3.5.3, 3.5.4, and 3.5.5.

Small-Step SOS

It is conceptually easy, though not entirely mechanical, to combine the ideas and changes to the original IMP small-step SOS discussed in Sections 3.5.1, 3.5.2, and 3.5.3, to obtain the small-step SOS rules of IMP++.

The arithmetic expression sequents need to change from $\langle a, \sigma\rangle \rightarrow \langle a', \sigma'\rangle$ to $\langle a, \sigma, \omega\rangle \rightarrow \langle a', \sigma', \omega'\rangle$ for normal steps and to $\langle a, \sigma, \omega\rangle \rightarrow \langle \texttt{error}, \sigma', \omega'\rangle$ for halting steps, and similarly for Boolean expressions, where σ, ω and σ',ω' are the states and input buffers before and after the small-step applied to a, respectively. Also, the original small-step SOS rules for expressions need to change to take into account the new configurations, the various side effects, and the abrupt termination due to division-by-zero. For example, here are the new rules for division:

$$\frac{\langle a_1, \sigma, \omega_{in}\rangle \rightarrow \langle a_1', \sigma', \omega_{in}'\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega_{in}\rangle \rightarrow \langle a_1' \ / \ a_2, \sigma', \omega_{in}'\rangle}$$

$$\frac{\langle a_1, \sigma, \omega_{in}\rangle \rightarrow \langle \texttt{error}, \sigma', \omega_{in}'\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega_{in}\rangle \rightarrow \langle \texttt{error}, \sigma', \omega_{in}'\rangle}$$

$$\frac{\langle a_2, \sigma, \omega_{in}\rangle \rightarrow \langle a_2', \sigma', \omega_{in}'\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega_{in}\rangle \rightarrow \langle a_1 \ / \ a_2', \sigma', \omega_{in}'\rangle}$$

$$\frac{\langle a_2, \sigma, \omega_{in}\rangle \rightarrow \langle \texttt{error}, \sigma', \omega_{in}'\rangle}{\langle a_1 \ / \ a_2, \sigma, \omega_{in}\rangle \rightarrow \langle \texttt{error}, \sigma', \omega_{in}'\rangle}$$

$$\langle i_1 \ / \ i_2, \sigma, \omega_{in}\rangle \rightarrow \langle i_1 \ /_{Int} \ i_2, \sigma', \omega_{in}'\rangle \quad \text{if } i_2 \neq 0$$

$$\langle a_1 \ / \ 0, \sigma, \omega_{in}\rangle \rightarrow \langle \texttt{error}, \sigma, \omega_{in}\rangle$$

Note that the last rule above does not require the full evaluation of a_1 in order to flag abrupt termination. This aspect was irrelevant when we added abrupt termination in isolation to IMP in Section 3.5.3, because expressions did not have side effects there. However, since expressions can now modify both the state (via variable increment) and the input buffer (via `read()`), the rule above captures more behaviors than a rule replacing a_1 by an integer i_1, which would be obtained by mechanically translating the corresponding rule from Section 3.5.3.

The small-step SOS rules of statements and programs result from those of the individual features discussed in Sections 3.5.1, 3.5.2, and 3.5.3, by modifying the configurations to include all the new semantic components. Sequents $\langle s, \sigma\rangle \rightarrow \langle s', \sigma'\rangle$ change into sequents $\langle s, \sigma, \omega_{in}, \omega_{out}\rangle \rightarrow \langle s', \sigma', \omega_{in}', \omega_{out}'\rangle$ and $\langle s, \sigma, \omega_{in}, \omega_{out}\rangle \rightarrow \langle \texttt{halting}, \sigma', \omega_{in}', \omega_{out}'\rangle$ for normal and for halting steps, respectively, where σ, ω_{in}, ω_{out} and σ', ω_{in}', ω_{out}' are the states, input buffers and output buffers before and after the small-step applied to s, respectively. The only rule that deviates from the expected pattern is the rule that propagates the halting signal through the `let` construct. Like in the case of big-step SOS discussed above, we can do it in such a way that the state is consistently cleaned up after each `let`, regardless of whether its body statement terminated

abruptly or not. Here are all three small-step SOS rules for `let`:

$$\frac{\langle s, \sigma[i/x], \omega_{in}, \omega_{out}\rangle \to \langle s', \sigma', \omega'_{in}, \omega'_{out}\rangle}{\langle \texttt{let}\ x = i\ \texttt{in}\ s, \sigma, \omega_{in}, \omega_{out}\rangle \to \langle \texttt{let}\ x = \sigma'(x)\ \texttt{in}\ s', \sigma'[\sigma(x)/x], \omega'_{in}, \omega'_{out}\rangle}$$

$$\langle \texttt{let}\ x = i\ \texttt{in}\ \{\}, \sigma, \omega_{in}, \omega_{out}\rangle \to \langle \{\}, \sigma, \omega_{in}, \omega_{out}\rangle$$

$$\frac{\langle s, \sigma[i/x], \omega_{in}, \omega_{out}\rangle \to \langle \texttt{halting}, \sigma', \omega'_{in}, \omega'_{out}\rangle}{\langle \texttt{let}\ x = i\ \texttt{in}\ s, \sigma, \omega_{in}, \omega_{out}\rangle \to \langle \texttt{halting}, \sigma'[\sigma(x)/x], \omega'_{in}, \omega'_{out}\rangle}$$

Recall again from Section 2.4.6 that $\sigma'[\bot/x]$ "undefines" x in σ'.

Even though strictly speaking all the small-step SOS rules above are different from their corresponding rules in the IMP extension introducing only the feature they define, they are somewhat mechanically derivable. In fact, MSOS (see Section 3.6) formalizes and mechanizes this process. The main question is then whether these new small-step SOS rules indeed capture the intended semantics of IMP++. Unfortunately, like in the case of big-step SOS, they fail to capture the intended semantics of `spawn`. Indeed, since the `let` construct does not dissolve itself until its body statement becomes {}, statements of the form (`let` $x = i$ `in spawn` s_1) s_2 will never allow reductions in s_2, thus limiting the concurrency of spawn statements to their defining blocks.

There are several ways to address the problem above, each with its advantages and limitations. One possibility is to attempt to syntactically detect all situations in which a statement allows a subsequent statement to execute, that is, all situations in which the former can only perform spawn steps. Like in Section 3.3.2 where we introduced special configurations of the form $C\surd$ (following Hennessy [32]) called "terminated configurations", we can introduce special "parallel configurations" $C\|$ stating that C can only spawn statements or discard terminated spawned statements, that is, C's statement can be executed in parallel with subsequent statements. Assuming such $C\|$ special configurations, we can remove the existing small-step SOS rule corresponding to rule (SmallStep-Spawn-Wait) in Section 3.5.4 and add instead the following rule:

$$\frac{\langle s_1, \sigma, \omega_{in}, \omega_{out}\rangle\| \quad \langle s_2, \sigma, \omega_{in}, \omega_{out}\rangle \to \langle s'_2, \sigma', \omega'_{in}, \omega'_{out}\rangle}{\langle s_1\ \ s_2, \sigma, \omega_{in}, \omega_{out}\rangle \to \langle s_1\ \ s'_2, \sigma', \omega'_{in}, \omega'_{out}\rangle}$$

Alternatively, one can define $\|$ as a predicate only taking a statement instead of a configuration, and then move the $\|$ sequent out from the rule premise into a side condition. Nevertheless, the $\|$ configurations or predicates need to be defined such that they include only statements of the form `spawn` s and `spawn` s `spawn` s' and `let` $x = i$ `in spawn` s and combinations of such statements. The idea is that such statements can safely be executed in parallel with what follows them. For example, a statement of the form `let` $x = a$ `in spawn` s where a is not a value does not fall into this pattern. Exercise 116 further explores this approach.

One problem with the approach above is that it is quite non-modular. Indeed, the definition of $\|$ is strictly dependent upon the current language syntax. Adding or removing syntax to the language will require us to also revisit the definition of $\|$. Another and bigger problem with this approach is that it does not seem to work for other concurrent language constructs. For example, in many languages including our extension of IMP++ in Section 3.5.7, the creation of a thread is an expression (and not a statement) construct, returning to the calling context the new thread identifier as a value. The calling context can continue its execution using the returned value in parallel with the spawned thread. In our context, for example, if `spawn` s returned an integer value, we would have to allow expressions of the form `spawn` { x = x + 1 ; } <= `spawn` { x = x + 10 ; } + x and be able to execute the two threads concurrently with the lookup for x and the evaluation of the <= Boolean expression. It would be quite hard to adapt the approach above to work with such common concurrent constructs which both return a value to the calling context and execute their code in parallel with the context.

Another way to address the loss of concurrent behaviors due to the syntactic constraints imposed by `let` on the `spawn` statements in its body, is to eliminate the `let` as soon as it is semantically unnecessary. Indeed, once the expression a is evaluated in a construct `let` $x = a$ `in` s, the `let` is semantically unnecessary. The only reason we kept it was because it offered us an elegant syntactic means to keep track of the execution contexts both for its body statement and for the outside environment. Unfortunately, it is now precisely this "elegant syntactic means" that inhibits the intended concurrency of the `spawn` statement. One way to eliminate the semantically unnecessary `let` statements is to try to add a small-step SOS rule of the form:

$$\langle \texttt{let}\ x = i\ \texttt{in}\ s, \sigma, \omega_{in}, \omega_{out}\rangle \rightarrow \langle s\ \ x = \sigma(x)\texttt{;}, \sigma[i/x], \omega_{in}, \omega_{out}\rangle$$

The idea here is to reduce the `let` statement to its body statement in a properly updated state, making sure that the state is recovered after the execution of the body statement. The infusion of the assignment statement "$x = \sigma(x)$`;`" is a bit unorthodox, because $\sigma(x)$ can also be undefined; it works in our case here because we allow state updates of the form $\sigma[\perp/x]$, which have the effect to undefine x in σ (see Section 2.4.6), and the assignment rule generates such a state update. This trick is not crucial for our point here; if one does not like it, then one can split the rule above in two cases, one where $\sigma(x)$ is defined and one where is it undefined, and then add a special syntactic statement construct to undefine x in the second case. The real problem with this `let`-elimination approach is that it is simply *wrong* when we also have `spawn` statements in the language. For example, if s is a spawn statement then the `let` reduces to the `spawn` statement followed by the assignment to x; the small-step SOS rule for `spawn` allowing the spawned statement to be executed in parallel with its subsequent statement then kicks in and allows the assignment to x to be possibly evaluated before the spawned statement, thus resulting in a wrong behavior: the spawned statement should only see the x bound by its `let` construct, not the outside x (possibly undefined). A correct way to eliminate the `let` construct is to rename the bound variable into a fresh variable visible only to `let`'s body:

$$\langle \texttt{let}\ x = i\ \texttt{in}\ s, \sigma, \omega_{in}, \omega_{out}\rangle \rightarrow \langle s[x'/x], \sigma[i/x'], \omega_{in}, \omega_{out}\rangle \quad \text{if } x' \text{ is a fresh variable}$$

Besides having to provide and maintain (as the language changes) a substitution[6] operation and then having to pay linear or worse complexity each time a `let` is eliminated, and besides creating potentially unbounded garbage bindings in the state (e.g., the let statement can be inside a loop), the solution above appears to only solve our particular problem with our particular combination of `let` and `spawn`. It still does not seem to offer us a general solution for dealing with arbitrary constructs for concurrency, in particular with `spawn` constructs that evaluate to a value which is immediately returned to the calling context, as described a few paragraphs above. For example, while it allows for renaming the variable x into a fresh variable when the expression `spawn` $\{\ x = x + 1\texttt{;}\ \}$ `<=` `spawn` $\{\ x = x + 10\texttt{;}\ \} + x$ appears inside a `let` $x = i$ `in` ... construct, we still have no clear way to evaluate both spawn expressions and the expression containing them concurrently.

The correct solution to deal with concurrency in small-step SOS is to place all the concurrent threads or processes in a syntactic "soup" where any particular thread or process, as well as any communicating subgroups of them, can be picked and advanced one step. Since in our IMP++ language we want all threads to execute concurrently without any syntactic restrictions, we have to place all of them in some top-level "soup", making sure that each of them is unambiguously provided its correct execution environment. For example, if a thread was spawned from inside a let statement, then it should correctly see precisely the execution environment available at the place where it was spawned, possibly interacting with other threads seeing the same environment; it should not wrongly interfere with other threads happening to have had variables with the same names in their creation environments. This can be done either by using a substitution

[6]See Section 4.5.3.

like above to guarantee that each bound variable is distinct, or by splitting the current state mapping variable identifiers to values into an *environment* mapping identifiers to memory locations and a *store* (or memory, or heap) mapping locations to values. In the latter case, each spawned thread would be packed together with its creation environment and all threads would share the store. The environment-store approach is the one that we will follow shortly in Section 3.5.7, so we do not insist on it here.

The morale of the discussion above is that putting features together in a language defined using small-step SOS is a highly non-trivial matter even when the language is trivial, like our IMP++. On the one hand, one has to non-modularly modify the configurations to hold all the required semantic data of all the features and then to modify all the rules to make sure that all these data are propagated and updated appropriately by each language construct. Addressing this problem is the main motivation of the MSOS approach discussed in Section 3.6. One the other hand, the desired feature interactions can require quite subtle changes to the semantics, which are sometimes hard to achieve purely syntactically. One cannot avoid the feeling that syntax is sometimes just too rigid, particularly when concurrency is concerned. This is actually one of the major motivations for the chemical abstract machine computational and semantic model discussed in Section 3.8.

Denotational Semantics

In order to accommodate all the semantic data needed by all the features, the denotation functions will now have the following types:

$$\begin{array}{l}
[\![_]\!] : AExp \to (State \times Buffer \rightharpoonup Int \cup \{error\} \times State \times Buffer) \\
[\![_]\!] : BExp \to (State \times Buffer \rightharpoonup Bool \cup \{error\} \times State \times Buffer) \\
[\![_]\!] : Stmt \to (State \times Buffer \rightharpoonup State \times Buffer \times Buffer \times \{halting, ok\})
\end{array}$$

Moreover, since programs are now statements and since we want their denotation to only take an input buffer and to return a state, the remainder of the input buffer and the output buffer (recall that we deliberately discard the halting status), we replace the original denotation function of programs with the following (we use a different name, to distinguish it from the last denotation function above):

$$[\![_]\!]_{\texttt{pgm}} : Pgm \to (Buffer \rightharpoonup State \times Buffer \times Buffer)$$

For example, the denotation of division becomes

$$[\![a_1 \ / \ a_2]\!]\pi = \begin{cases} (1^{\text{st}}(arg_1) +_{Int} 1^{\text{st}}(arg_2),\ 2^{\text{nd}}(arg_2), 3^{\text{rd}}(arg_2)) & \text{if} \quad 1^{\text{st}}(arg_1) \neq error \\ & \text{and} \quad 1^{\text{st}}(arg_2) \neq error \\ & \text{and} \quad 1^{\text{st}}(arg_2) \neq 0 \\ arg_1 & \text{if} \quad 1^{\text{st}}(arg_1) = error \\ (error, 2^{\text{nd}}(arg_2), 3^{\text{rd}}(arg_2)) & \text{if} \quad 1^{\text{st}}(arg_2) = error \\ & \text{or} \quad 1^{\text{st}}(arg_2) = 0 \end{cases}$$

where $arg_1 = [\![a_1]\!]\pi$ and $arg_2 = [\![a_2]\!](2^{\text{nd}}(arg_1), 3^{\text{rd}}(arg_1))$, the denotation of sequential composition becomes

$$[\![s_1 \ \ s_2]\!]\pi = \begin{cases} (1^{\text{st}}(arg_2),\ 2^{\text{nd}}(arg_2),\ 3^{\text{rd}}(arg_1) : 3^{\text{rd}}(arg_2),\ 4^{\text{th}}(arg_2)) & \text{if} \quad 4^{\text{th}}(arg_1) = ok \\ arg_1 & \text{if} \quad 4^{\text{th}}(arg_1) = halting \end{cases}$$

where $arg_1 = [\![s_1]\!]\pi$ and $arg_2 = [\![s_2]\!](1^{\text{st}}(arg_1),\ 2^{\text{nd}}(arg_1))$, the denotational semantics of while loops `while` (b) s become the fixed-points of total functions

$$\begin{array}{rcl} \mathcal{F} & : & (State \times Buffer \rightharpoonup State \times Buffer \times Buffer \times \{halting, ok\}) \\ & \to & (State \times Buffer \rightharpoonup State \times Buffer \times Buffer \times \{halting, ok\}) \end{array}$$

defined as

$$\mathcal{F}(\alpha)(\pi) = \begin{cases} (1^{\text{st}}(arg_3),\ 2^{nd}(arg_3),\ 3^{\text{rd}}(arg_2) : 3^{\text{rd}}(arg_3),\ 4^{\text{th}}(arg_3)) & \text{if} \quad 1^{\text{st}}(arg_1) = \texttt{true} \\ & \text{and} \quad 4^{\text{th}}(arg_2) = ok \\ arg_2 & \text{if} \quad 1^{\text{st}}(arg_1) = \texttt{true} \\ & \text{and} \quad 4^{\text{th}}(arg_2) = halting \\ (2^{\text{nd}}(arg_1),\ 3^{\text{rd}}(arg_1),\ \epsilon,\ ok) & \text{if} \quad 1^{\text{st}}(arg_1) = \texttt{false} \\ (2^{\text{nd}}(arg_1),\ 3^{\text{rd}}(arg_1),\ \epsilon,\ halting) & \text{if} \quad 1^{\text{st}}(arg_1) = error \end{cases}$$

where $arg_1 = [\![b]\!]\pi$, $arg_2 = [\![s]\!](2^{\text{nd}}(arg_1),\ 3^{\text{rd}}(arg_1))$, and $arg_3 = \alpha(1^{\text{st}}(arg_2),\ 2^{\text{nd}}(arg_2))$, and the denotation of programs is defined as the function

$$[\![s]\!]_{\texttt{pgm}}\omega = (1^{\text{st}}(arg),\ 2^{\text{nd}}(arg),\ 3^{\text{rd}}(arg))$$

where $arg = [\![s]\!](\cdot, \omega)$, that is, the program statement s with input buffer ω is evaluated in the empty state "$\cdot$" and the input buffer ω, and the resulting halting flag is discarded.

Once all the changes above are applied to correctly handle the semantic requirements of the various features of IMP++, the denotational semantics of those features is relatively easy, basically a simple adaptation of their individual denotational semantics in Sections 3.5.1, 3.5.2, 3.5.3, 3.5.4, and 3.5.5. We let their precise definitions as an exercise to the reader (see Exercise 119). Note, however, that the resulting denotational semantics of IMP++ is still non-deterministic and non-concurrent. Because of this accepted limitation, we do not worry about the loss of concurrent behaviors due to the interaction between `spawn` and `let`.

3.5.7 Putting Them All Together: Second Attempt

...

3.5.8 Notes

The first to pinpoint the limitations of plain SOS and denotational semantics when defining non-trivial languages were the inventors of alternative semantic frameworks, such as Berry and Boudol [8, 9] who proposed the chemical abstract machine model (see Section 3.8), Felleisen and his collaborators [25, 88] who proposed evaluation contexts (see Section 3.7), and Mosses and his collaborators [51, 52, 53] who proposed the modular SOS approach (see Section 3.6). Among these, Mosses is perhaps the one who most vehemently criticized the lack of modularity of plain SOS, bringing as evidence natural features like the ones we proposed for IMP++, which require the structure of configurations and implicitly the existing rules to change no matter whether there is any semantic interaction or not between the new and the old features.

The lack of modularity of SOS was visible even in Plotkin's original notes [60, 61], where he had to modify the definition of simple arithmetic expressions several times as his initial language evolved. Hennessy also makes it even more visible in his book [32]. Each time he adds a new feature, he also has to change the configurations and the entire existing semantics, similarly to each of our IMP extensions in this section. However, the lack of modularity of language definitional frameworks was not perceived as a major problem until late 1990es, partly because there were few attempts to give complete and rigorous semantics to real programming languages. Hennessy actually used each language extension as a pedagogical opportunity to teach the reader what new semantic components the feature needs and how and where those are located in each sequent. Note also that Hennessy's languages were rather simple and pure. His imperative language, called *WhileL*, was actually simpler even than our IMP (*WhileL* had no global variable declarations). Hennessy's

approach was somewhat different from ours, namely he defined a series of different paradigmatic languages, each highlighting certain semantic aspects in a pure form, without including features that lead to complex semantic interactions (like the side effects, blocks with local variables, and threads as in our IMP++).

Wadler [86] proposes a language design experiment similar in spirit to our extensions of IMP. His core language is purely functional, but some of its added features overlap with those of IMP and IMP++: state and side effects, output, non-determinism. Wadler's objective in [86] was to emphasize the elegance and usefulness of monads in implementing interpreters in pure functional languages like Haskell, but his motivations for doing so are similar to ours: the existing approaches for programming language design are not modular enough. Monads can also be used to add modularity to denotational semantics, to avoid having to modify the mathematical domains into products of domains as we did in this section. The monadic approach to denotational semantics will be further discussed in Section 3.9. However, as also discussed in Section 3.4.3, the denotational approach to non-determinism and concurrency is to collect all possible behaviors, hereby programs evaluating to sets or lists of values. The same holds true in Wadler's monadic approach to implement interpreters in [86]. The problem with this is that the resulting interpreters or executable semantics are quite inefficient. Contrast that with the small-step SOS approach in Section 3.3 which allows us, for example using our implementation of it in Maude, to both execute programs non-deterministically, making a possibly non-deterministic choice at each step, and search for all possible behaviors of programs.

3.5.9 Exercises

Variable Increment

The exercises below refer to the IMP extension with variable increment discussed in Section 3.5.1.

Exercise 84. *Add variable increment to* IMP*, using big-step SOS:*

1. *Write the complete big-step SOS as a proof system;*
2. *Translate the proof system at 1 above into a rewrite logic theory, like in Figure 3.8;*
3. ★ *Implement in Maude the rewrite logic theory at 2 above, like in Figure 3.9. To test it, add to the* IMP *programs in Figure 3.4 the following two:*

```
op sum++Pgm : -> Pgm .
eq sum++Pgm = (
   int m, n, s ;
   n = 100 ;
   while (++ m <= n) {
         s = s + m ;
   }     ) .
```

```
op nondet++Pgm : -> Pgm .
eq nondet++Pgm = (
   int x ;
   x = 1 ;
   x = ++ x / (++ x / x) ;
) .
```

The first program should have only one behavior, so, for example, both Maude commands below

```
rewrite < sum++Pgm > .
search  < sum++Pgm > =>! Cfg:Configuration .
```

should show the same result configuration, `< m |-> 101 & n |-> 100 & s |-> 5050 >`*. The second program should have (only) three different behaviors under big-step semantics; the first command below will show one of the three behaviors, but the second will show all three of them:*

```
rewrite < nondet++Pgm > .
search  < nondet++Pgm > =>! Cfg:Configuration .
```

The three behaviors captured by the big-step SOS discussed in Section 3.5.1 result in configurations `< x |-> 1 >`, `< x |-> 2 >`, *and* `< x |-> 3 >`. *As explained in Section 3.5.1, this big-step SOS should not be not able to expose the behaviors in which* `x` *is* `0` *and in which a division by zero takes place.*

Exercise 85. *Type* IMP *extended with variable increment:*

1. *Translate the big-step rule above into a rewrite logic rule that can be added to those in Figure 3.11 corresponding to the type system of* IMP*;*

2. ★ *Implement the above in Maude, extending the implementation in Figure 3.12. Test it on the two additional programs in Example 84.*

Exercise 86. *Same as Exercise 84, but for small-step SOS instead of big-step SOS.*
★ *Make sure that the small-step definition in Maude exhibits all five behaviors of program* `nondet++Pgm` *defined in Exercise 84 (the three behaviors exposed by the big-step definition in Maude in Exercise 84, plus one where* `x` *is* `0` *and one where the program gets stuck right before a division by zero).*

Exercise 87. *Same as Exercise 84, but for denotational semantics instead of big-step SOS.*
★ *The definition in Maude should lack any non-determinism, so only one behavior should be observed for any program, including* `nondet++Pgm` *in Exercise 84.*

Input/Output

The exercises below refer to the IMP extension with input/output discussed in Section 3.5.2.

Exercise 88. *Add input/output to* IMP*, using big-step SOS:*

1. *Write the complete big-step SOS as a proof system;*

2. *Translate the above into a rewrite logic theory, like in Figure 3.8;*

3. ★ *Implement the resulting rewrite logic theory in Maude, like in Figure 3.9. To test it, add to the* IMP *programs in Figure 3.4 the following three:*

```
op sumIOPgm : -> Pgm .
eq sumIOPgm = (
   int m, n, s ;
   n = read() ;
   while (m <= n) {
     print(m) ;
     s = s + m ;
     m = m + 1 ;
   }
   print(s) ;    ) .
```

```
op whileIOPgm : -> Pgm .
eq whileIOPgm = (
   int s ;
   s = 0 ;
   while (!(read() <= 0)) {
     s = s + read() ;
   }
   print(s) ;    ) .
```

```
op nondetIOStmt : -> Stmt .
eq nondetIOStmt = (
     print(read()
           / (read()
              / read())) ;     ) .
```

The first two programs are deterministic, so both the rewrite and the search commands should only show one solution. The initial configuration in which the first program is executed should contain at least one integer in the input buffer, otherwise it does not evaluate; for example, the initial configuration `< sumIOPgm, 100 >` *yields a result configuration whose input buffer is empty and whose output buffer contains the numbers 0,1,2,...,100,5050. The initial configuration in which the second program is executed should eventually contain some 0 on an odd position in the input buffer; for example,* `< whileIOPgm,10:1:17:2:21:3:0:5:8:-2:-5:10 >` *yields a result configuration whose input buffer still contains the remaining input* `5:8:-2:-5:10` *and whose output buffer contains only the*

integer `6`*. The third program, which is actually a statement, is non-deterministic. Unfortunately, big-step SOS misses behaviors because, as explained, it only achieves a non-deterministic choice semantics. For example, the commands*

```
rewrite < nondetIOStmt, .State, 10 : 20 : 30 > .
search  < nondetIOStmt, .State, 10 : 20 : 30 > =>! Cfg:Configuration .
```

yield configurations `< .State,epsilon,10 >` *and, respectively,* `< .State,epsilon,10 >` *and* `< .State,epsilon,15 >`*. The configuration whose output is 6 (i.e.,* $20/_{Int}(30/_{Int}10)$*) and the three undefined configurations due to division by zero are not detected.*

Exercise 89. *Type* IMP *extended with input/output:*

1. *Translate the discussed big-step SOS typing rules for input/output into corresponding rewrite logic rules that can be added to the already existing rewrite theory in Figure 3.11 corresponding to the type system of* IMP*;*

2. ★ *Implement the above in Maude, extending the implementation in Figure 3.12. Test it on the additional programs in Example 88.*

Exercise 90. *Same as Exercise 88, but for small-step SOS instead of big-step SOS. Make sure that the resulting small-step SOS definition detects all six behaviors of* `nondetIOStmt` *when executed with the input buffer* `10:20:30`

Exercise 91. *Same as Exercise 88, but for denotational semantics instead of big-step SOS. Since our denotational semantics is not nondeterministic, only one behavior of* `nondetIOStmt` *is detected. Interestingly, since our denotational semantics of division was chosen to evaluate the two expressions in order, it turns out that the detected behavior is undefined (due to a division by zero). Note that although it also misses non-deterministic behaviors, big-step SOS can still detect (and even search for) valid behaviors of non-deterministic programs (see Exercise 88, where it generated a valid behavior by rewriting and found one additional behavior by searching).*

Abrupt Termination

The exercises below refer to the IMP extension with abrupt termination discussed in Section 3.5.3.

Exercise 92. *Add abrupt termination to* IMP*, using big-step SOS:*

1. *Write the complete big-step SOS as a proof system;*

2. *Translate the above into a rewrite logic theory, like in Figure 3.8;*

3. ★ *Implement the resulting rewrite logic theory in Maude, like in Figure 3.9. To test it, execute the two programs at the beginning of Section 3.5.3. The resulting big-step SOS definition may be very slow when executed in Maude, even for small values of* `n` *(such as 2,3,4 instead of 100), which is normal (the search space is now much larger).*

Exercise 93. *Same as Exercise 92, but using a specific* `top` *construct as explained in Section 3.5.3 to catch the halting signal instead of the existing* `var` *which has a different purpose in the language.*

Exercise 94. *Type* IMP *extended with abrupt termination:*

1. *Translate the big-step SOS typing rule of* `halt` *into a corresponding rewrite logic rule that can be added to the already existing rewrite logic theory in Figure 3.11;*

2. ★ *Implement the above in Maude, extending the implementation in Figure 3.12. Test it on the additional programs in Example 92.*

Exercise 95. *Same as Exercise 92, but for the small-step SOS instead of big-step SOS.*

Exercise 96. *Same as Exercise 92, but use the* `top` *construct approach instead of the rule* (SmallStep-Halting) *as discussed right after the rule* (SmallStep-Halting) *is introduced.*

Exercise 97. *Same as Exercise 93, but eliminate the third rule of the* `top` *construct to avoid wasting any computational step.*

Exercise 98. *One could argue that the introduction of the halting configurations* $\langle \texttt{halting}, \sigma \rangle$ *was unnecessary, because we could have instead used the already existing configurations of the form* $\langle \texttt{halt}, \sigma \rangle$*. Give an alternative small-step SOS definition of abrupt termination which does not add special halting configurations. Can we avoid the introduction of the* `top` *construct discussed above? Comment on the disadvantages of this approach.*

Exercise 99. *Same as Exercise 92, but for denotational semantics instead of big-step SOS.*

Exercise 100. *Same as Exercise 99, but modifying the denotation of assignment so that it is always undefined when the assigned variable has not been declared.*

Dynamic Threads

The exercises below refer to the IMP extension with dynamic threads discussed in Section 3.5.4.

Exercise 101. *Consider the two programs at the beginning of Section 3.5.4. Propose hypothetical executions of the second program corresponding to any of the seven possible values of* `x`*. What is the maximum value that* `s` *can have when the first program, as well as all its dynamically created threads, terminate? Is there some execution of the first program corresponding to each smaller value of* `s` *(but larger than or equal to 0)?*

Exercise 102. *Add dynamic threads to* IMP*, using big-step SOS:*

1. *Write the complete big-step SOS as a proof system;*

2. *Translate the above into a rewrite logic theory, like in Figure 3.8;*

3. ★ *Implement the resulting rewrite logic theory in Maude, like in Figure 3.9. To test it, execute the two programs at the beginning of Section 3.5.4. The resulting Maude big-step SOS definition may be slow on the first program for large initial values for n: even though it does not capture all possible behaviors, it still comprises many of them. For example, searching for all the result configurations when* $n = 10$ *gives 12 possible values for s, namely 55,56,...,66. On the other hand, the second program only shows one out of the seven behaviors, namely the one where x results in 111.*

Exercise 103. *Same as Exercise 102, but for the type system instead of big-step SOS.*

Exercise 104. *Same as Exercise 102, but for small-step SOS instead of big-step SOS. When representing the resulting small-step SOS into rewrite logic, the structural identity can be expressed as a rewrite logic equation, this way capturing faithfully the intended computational granularity of the small-step SOS.* ★ *When implementing the resulting rewrite logic theory into Maude, this equation can either be added as a normal equation (using* `eq`*) or as an* `assoc` *attribute to the sequential composition construct. The former will only be applied from left-to-right when executed using Maude rewriting and search commands, but that is sufficient in our particular case here.*

Exercise 105. *Same as Exercise 102, but for denotational semantics instead of big-step SOS.*

Local Variables

The exercises below refer to the IMP extension with blocks and local variables discussed in Section 3.5.5.

Exercise 106. ★ *Implement in Maude the seven macros in the preamble of Section 3.5.5, which desugar blocks with local variable declarations into* `let` *constructs.*
Hint: In Maude, equations are applied in order. One should typically not rely on that, but in this case it may give us a simpler and more compact implementation.

Exercise 107. *Add blocks with local variables to* IMP*, using big-step SOS:*

1. *Write the complete big-step SOS as a proof system;*
2. *Translate the above into a rewrite logic theory, like in Figure 3.8;*
3. ★ *Implement the resulting rewrite logic theory in Maude, like in Figure 3.9. To test it, modify the* IMP *programs in Figure 3.4 to use local variables and also add the two programs at the beginning of Section 3.5.5. To check whether the programs evaluate as expected, you can let some relevant variables purposely undeclared and bind them manually (to 0) in the initial state. When the programs terminate, you will see the new values of those variables. If you execute closed programs (i.e., programs declaring all the variables they use) then the resulting states will be empty, because our semantics of* `let` *recovers the state, so it will be difficult or impossible to know whether they executed correctly.*

Exercise 108. *Same as Exercise 107, but for the type system instead of big-step SOS.*

Exercise 109. *Same as Exercise 107, but for small-step SOS instead of big-step SOS.*

Exercise 110. *Same as Exercise 107, but for denotational semantics instead of big-step SOS.*

Putting Them All Together

The exercises below refer to the IMP extension with blocks and local variables discussed in Section 3.5.6.

Exercise 111. *Define* IMP++ *using big-step SOS, assuming that abrupt termination applies to the entire program, no matter whether the abrupt termination signal has been issued from inside a spawned thread or from the main program, and assuming that nothing special is done to enhance the parallelism of* `spawn` *within* `let` *blocks:*

1. *Write the complete big-step SOS as a proof system;*
2. *Translate the above into a rewrite logic theory, like in Figure 3.8;*

3. ★ *Implement the resulting rewrite logic theory in Maude, like in Figure 3.9. To test it, propose five tricky* IMP++ *programs. You programs will be added to our test-suite before grading this exercise. You get extra-points if your programs reveal limitations in the number of behaviors captured by other students' definitions.*

Exercise 112. *Same as Exercise 111, but assume that abrupt termination applies to the issuing thread only, letting the rest of the threads continue. The main program is considered to be a thread itself.*

Exercise 113. *Same as Exercise 111, but for the type system instead of big-step SOS.*

Exercise 114. *Same as Exercise 111, but for small-step SOS instead of big-step SOS.*

Exercise 115. *Same as Exercise 112, but for small-step SOS instead of big-step SOS.*

Exercise⋆ 116. *Same as Exercise 114, but define and use "parallel configurations" of the form C*|| *in order to enhance the parallelism of* `spawn` *statements from inside* `let` *blocks.*

Exercise⋆ 117. *Same as Exercise 116, but eliminate the* `let` *construct when semantically unnecessary using a substitution operation (which needs to also be defined).*

Exercise⋆ 118. *Same as Exercise 117, but use an environment-store approach to the state instead of substitution.*

Exercise 119. *Same as Exercise 111, but for denotational semantics instead of big-step SOS.*

Anything else to add here?

Mention Moggi/Wadler among those who criticized plain SOS and plain, non-monadic denotational semantics?

Give a bit of history on who split for the first time a state into an environment and a store, as an alternative to using substitutions.

VERY IMPORTANT!

Make IMP *have the following syntax:*

Int	::=	*the domain of (unbounded) integer numbers, with usual operations on them*
Bool	::=	*the domain of Booleans*
Id	::=	*standard identifiers*
AExp	::=	*Int*
	|	*Id*
	|	*AExp* + *AExp*
	|	*AExp* / *AExp*
BExp	::=	*Bool*
	|	*AExp* <= *AExp*
	|	`not` *BExp*
	|	*BExp* `and` *BExp*
Stmt	::=	{}
	|	{*Stmt*}
	|	*Id* = *AExp* `;`
	|	*Stmt Stmt*
	|	`if(`*BExp*`)` *Stmt* `else` *Stmt*
	|	`while(`*BExp*`)` *Stmt*
Pgm	::=	`int` **List{***Id***}**`;` *Stmt*

Add the following to IMP++*, removing the above production for Pgm:*

AExp	::=	++*Id*
	|	`read()`
	|	`spawn` *Stmt*
Stmt	::=	*AExp* `;`
	|	`print(`*AExp*`);`
	|	`halt;;`
	|	`join(`*AExp*`)`
	|	`int` **List{***Id***}**`;`
Pgm	::=	*Stmt*

To make this language design experiment more interesting, first consider spawn to be a statement construct, as it is right now all over this chapter. This way we also keep all this material.

Then argue that we would like to also be able to synchronize somehow with the spawned thread, at least to join it. In this case, we need an id or something similar to be returned by the spawn command. Then propose to make spawn an expression construct as above, which returns an integer. Then comment what it takes for each style to support this change. Finally, make all Maude code to consider this as the final IMP++ *language.*

3.6 Modular Structural Operational Semantics (MSOS)

Modular structural operational semantics (MSOS) was introduced, as its name implies, to address the non-modularity aspects of (big-step and/or small-step) SOS. As already seen in Section 3.5, there are several reasons why big-step and small-step SOS are non-modular, as well as several facets of non-modularity in general. In short, a definitional framework is *non-modular* when, in order to add a new feature to an existing language or calculus, one needs to revisit and change some or all of the already defined unrelated features. For example, recall the IMP extension with input/output in Section 3.5.2. We had to add new semantic components in the IMP configurations, both in the big-step and in the small-step SOS definitions, to hold the input/output buffers. That meant, in particular, that *all* the existing big-step and/or small-step SOS rules of IMP had to change. That was, at best, very inconvenient.

Before we get into the technicalities of MSOS, one natural question to address is why we need modularity of language definitions. One may argue that defining a programming language is a major endeavor, done once and for all, so having to go through the semantic rules many times is, after all, not such a bad idea, because it gives one the chance to find and fix potential errors in them. Here are several reasons why modularity is desirable in language definitions, in no particular order:

- Having to modify many or all rules whenever a new rule is added that modifies the structure of the configuration is actually more error prone than it may seem, because rules become heavier to read and debug; for example, one can write σ instead of σ' in a right-hand-side of a rule and a different or wrong language is defined.

- A modular semantic framework allows us to more easily reuse semantics of existing and probably already well-tested features in other languages or language extensions, thus increasing our productivity as language designers and our confidence in the correctness of the resulting language definition.

- When designing a new language, as opposed to an existing language, one needs to experiment with features and combinations of features; having to do unrelated changes whenever a new feature is added to or removed from the language burdens the language designer with boring tasks taking considerable time that could have been otherwise spent on actual interesting language design issues.

- There is an increasing number of domain-specific languages, resulting from the need to abstract away from low-level programming language details to important, domain-specific application aspects. Hence, there is a need for language design and experimentation for various domains. Moreover, domain-specific languages tend to be dynamic, being added or removed features frequently as the domain knowledge evolves. It would be nice to have the possibility to "drag-and-drop" features in one's language, such as functions, exceptions, etc.; however, modularity is crucial for that.

To our knowledge, MSOS was the first framework that explicitly recognizes the importance of modular language design and provides explicit support to achieve it in the context of SOS. Reduction semantics with evaluation contexts (see Section 3.7) was actually proposed before MSOS and also offers modularity in language semantic definitions, but its modularity comes as a consequence of a different way to propagate reductions through language constructs and not as an explicit goal that it strives to achieve.

There are both big-step and small-step variants of MSOS, but we discuss only small-step MSOS here. We actually generically call MSOS the small-step, implicitly-modular variant of MSOS (see Section 3.6.4). To bring modularity to SOS, MSOS proposes the following:

- Separate the syntax (i.e., the fragment of program under consideration) from the non-syntactic components in configurations, and treat them differently, as explained below;

- Make the transitions only relate syntax to syntax (as opposed to configurations), and hide the non-syntactic components in a transition label, as explained below;
- Encode in the transition label all the changes in the non-syntactic components of the configuration that need to be applied together with the syntactic reduction given by the transition;
- Use specialized notation in transition labels together with a discipline to refer to the various semantic components and to say that some of them stay unchanged; also, labels can be explicitly or implicitly shared by the conditions and the conclusion of a rule, elegantly capturing the idea that "changes are propagated" through desired language constructs.

A transition in MSOS is of the form

$$P \xrightarrow{\Delta} P'$$

where P and P' are programs or fragments of programs and Δ is a *label describing the semantic configuration components both before and after the transition*. Specifically, Δ is a record containing fields denoting the semantic components of the configuration. The preferred notation in MSOS for stating that in label Δ the semantic component associated to the field name field *before* the transition takes place is α is $\Delta = \{\mathsf{field} = \alpha, \ldots\}$. Similarly, the preferred notation for stating that the semantic component associated to field field *after* the transition takes place is β is $\Delta = \{\mathsf{field'} = \beta, \ldots\}$ (the field name is primed). For example, the second MSOS rule for variable assignment (when the assigned arithmetic expression is already evaluated) is (this is rule (MSOS-Asgn) in Figure 3.24):

$$x = i\,; \xrightarrow{\{\mathsf{state}=\sigma,\ \mathsf{state'}=\sigma[i/x],\ \ldots\}} \{\} \quad \text{if } \sigma(x) \neq \bot$$

It is easy to desugar the rule above into a more familiar SOS rule of the form:

$$\langle x = i\,;, \sigma\rangle \to \langle \{\}, \sigma[i/x]\rangle \quad \text{if } \sigma(x) \neq \bot$$

The above is precisely the rule (SmallStep-Asgn) in the small-step SOS of IMP (see Figure 3.15). The MSOS rule is actually more modular than the SOS one, because of the "...", which says that everything else in the configuration stays unchanged. For example, if we want to extend the language with input/output language constructs as we did in Section 3.5.2, then new semantic components, namely the input and output buffers, need to be added to the configuration. Moreover, as seen in Section 3.5.2, the SOS rule above needs to be changed into a rule of the form

$$\langle x = i\,;, \sigma, \omega_{in}, \omega_{out}\rangle \to \langle \{\}, \sigma[i/x], \omega_{in}, \omega_{out}\rangle \quad \text{if } \sigma(x) \neq \bot$$

where ω_{in} and ω_{out} are the input and output buffers, respectively, which stay unchanged during the variable assignment operation, while the MSOS rule does not need to be touched.

To impose a better discipline on the use of labels, at the same time making the notation even more compact, MSOS splits the fields into three categories: *read-only*, *read-write*, and *write-only*. The field state above was read-write, meaning that the transition label can both read its value before the transition takes place and write its value after the transition takes place. Unlike the state, which needs to be both read and written, there are semantic configuration components that only need to be read, as well as ones that only need to be written. In these cases, it is recommended to use read-only or write-only fields.

Read-only fields are only inspected by the rule, but not modified, so they only appear unprimed in labels. For example, the following can be one of the MSOS rules for the `let` binding language construct in a pure

functional language where expressions yield no side-effects:

$$\frac{e_2 \xrightarrow{\{\mathsf{env}=\rho[v_1/x],\ldots\}} e_2'}{\texttt{let}\ x = v_1\ \texttt{in}\ e_2 \xrightarrow{\{\mathsf{env}=\rho,\ldots\}} \texttt{let}\ x = v_1\ \texttt{in}\ e_2'}$$

Indeed, transitions do not modify the environment in a pure functional language. They only use it in a read-only fashion to lookup the variable values. A new environment is created in the premise of the rule above to reduce the body of the `let`, but neither of the transitions in the premise nor in the conclusion of the rule change their environment. Note that this was not the case for IMP extended with `let` in Section 3.5.5, because there we wanted blocks and local variable declarations to desugar to `let` statements. Since we allowed variables to be assigned new values in blocks, like in conventional imperative and object-oriented languages, we needed an impure variant of `let`. As seen in Section 3.6.2, our MSOS definition of IMP++'s `let` uses a read-write attribute (the state attribute). We do not discuss read-only fields any further here.

Write-only fields are used to record data that is not analyzable during program execution, such as the output or the trace. Their names are always primed and they have a free monoid semantics—everything written on them is actually added to the end (see [53] for technical details). Consider, for example, the extension of IMP with an output (or print) statement in Section 3.5.2, whose MSOS second rule (after the argument is evaluated, namely rule (MSOS-Print) in Section 3.6.2) is:

$$\texttt{print}(i);\ \xrightarrow{\{\mathsf{output'}=i,\ \ldots\}} \{\}$$

Compare the rule above with the one below, which uses a read-write attribute instead:

$$\texttt{print}(i);\ \xrightarrow{\{\mathsf{output}=\omega_{out},\ \mathsf{output'}=\omega_{out}:i,\ \ldots\}} \{\}$$

Indeed, mentioning the ω_{out} like in the second rule above is unnecessary, error-prone (e.g., one may forget to add it to the primed field or may write $i : \omega_{out}$ instead of $\omega_{out} : i$), and non-modular (e.g., one may want to change the monoid construct, say to write $\omega_{out} \cdot i$ instead of $\omega_{out} : i$, etc.).

MSOS achieves modularity in two ways:

1. By making intensive use of the record comprehension notation "...", which, as discussed, indicates that more fields could follow but that they are not of interest. In particular, if the MSOS rule has no premises, like in the rules for the assignment and print statements discussed above, than the "..." says that the remaining contents of the label stays unchanged after the application of the transition; and

2. By reusing the same label or portion of label both in the premise and in the conclusion of an MSOS proof system rule. In particular, if "..." is used in the labels of both the premise and the conclusion of an MSOS rule, then all the occurrences of "..." stand for the same portion of label, that is, the same fields bound to the same semantic components.

For example, the following MSOS rules for first-statement reduction in sequential composition are equivalent

and say that all the changes generated by reducing s_1 to s_1' are propagated when reducing $s_1\ s_2$ to $s_1'\ s_2$:

$$\frac{s_1 \xrightarrow{\Delta} s_1'}{s_1\ s_2 \xrightarrow{\Delta} s_1'\ s_2}$$

$$\frac{s_1 \xrightarrow{\{...\}} s_1'}{s_1\ s_2 \xrightarrow{\{...\}} s_1'\ s_2}$$

$$\frac{s_1 \xrightarrow{\{\mathsf{state}=\sigma,\ ...\}} s_1'}{s_1\ s_2 \xrightarrow{\{\mathsf{state}=\sigma,\ ...\}} s_1'\ s_2}$$

Indeed, advancing the first statement in a sequential composition of statements one step has the same effect on the configuration as if the statement was advanced the same one step in isolation, without the other statement involved; said differently, the side effects are all properly propagated.

MSOS (the implicitly-modular variant of it, see Section 3.6.4) has been refined to actually allow for dropping such redundant labels like above from rules. In other words, if a label is missing from a transition then the *implicit label* is assumed: if the rule is unconditional then the implicit label is the identity label (in which the primed fields have the same values as the corresponding unprimed ones, etc.), but if the rule is conditional then the premise and the conclusion transitions share the same label, that is, they perform the same changes on the semantic components of the configuration. With this new notational convention, the most elegant and compact way to write the rule above in MSOS is:

$$\frac{s_1 \rightarrow s_1'}{s_1\ s_2 \rightarrow s_1'\ s_2}$$

This is precisely the rule (MSOS-Seq-Arg1) in Figure 3.24, part of the MSOS semantics of IMP.

One of the important merits of MSOS is that it captures formally many of the tricks that language designers informally use to avoid writing awkward and heavy SOS definitions.

Additional notational shortcuts are welcome in MSOS if properly explained and made locally rigorous, without having to rely on other rules. For example, the author of MSOS finds the rule

$$x \xrightarrow{\{\mathsf{state},\ ...\}} \mathsf{state}(x) \quad \text{if } \mathsf{state}(x) \neq \bot$$

to be an acceptable variant of the lookup rule:

$$x \xrightarrow{\{\mathsf{state}=\sigma,\ ...\}} \sigma(x) \quad \text{if } \sigma(x) \neq \bot$$

despite the fact that, strictly speaking, $\mathsf{state}(x)$ does not make sense by itself (recall that state is a field name, not the state) and that field names are expected to be paired with their semantic components in labels. Nevertheless, there is only one way to make sense of this rule, namely to replace any use of state by its semantic contents, which therefore does not need to be mentioned.

A major goal when using MSOS to define languages or calculi is to write on the labels as little information as possible and to use the implicit conventions for the missing information. That is because everything written on labels may work against modularity if the language is later on extended or simplified. As an extreme case,

if one uses only read/write fields in labels and mentions all the fields together with all their semantic contents on every single label, then MSOS becomes conventional SOS and therefore suffers from the same limitations as SOS with regards to modularity.

Recall the rules in Figure 3.16 for deriving the transitive closure $\rightarrow^\star$ of the small-step SOS relation $\rightarrow$. In order for two consecutive transitions to compose, the source configuration of the second had to be identical to the target configuration of the first. A similar property must also hold in MSOS, otherwise one may derive inconsistent computations. This process is explained in MSOS by making use of category theory (see [53] for technical details on MSOS; see Section 2.7 for details on category theory), associating MSOS labels with morphisms in a special category and then using the morphism composition mechanism of category theory.

However, category theory is not needed in order to understand how MSOS works in practice. A simple way to explain its label composition is by translating, or desugaring MSOS definitions into SOS, as we implicitly suggested when we discussed the MSOS rule for variable assignment above. Indeed, once one knows all the fields in the labels, which happens once a language definition is complete, one can automatically associate a standard small-step SOS definition to the MSOS one by replacing each MSOS rule with an SOS rule over configurations including, besides the syntactic contents, the complete semantic contents extracted from the notational conventions in the label. The resulting SOS configurations will not have fields anymore, but will nevertheless contain all the semantic information encoded by them. For example, in the context of a language containing only a state and an output buffer as semantic components in its configuration (note that IMP++ contained an input buffer as well), the four rules discussed above for variable assignment, output, sequential composition, and lookup desugar, respectively, into the following conventional SOS rules:

$$\langle x = i\,\texttt{;}, \sigma, \omega\rangle \rightarrow \langle \{\}, \sigma[i/x], \omega\rangle \quad \text{if } \sigma(x) \neq \bot$$

$$\langle \texttt{print(}i\texttt{);}, \sigma, \omega\rangle \rightarrow \langle \{\}, \sigma, \omega : i\rangle$$

$$\frac{\langle s_1, \sigma, \omega\rangle \rightarrow \langle s_1', \sigma', \omega'\rangle}{\langle s_1\ s_2, \sigma, \omega\rangle \rightarrow \langle s_1'\ s_2, \sigma', \omega'\rangle}$$

$$\langle x, \sigma, \omega\rangle \rightarrow \langle \sigma(x), \sigma, \omega\rangle \quad \text{if } \sigma(x) \neq \bot$$

Recall that for unconditional MSOS rules the meaning of the missing label fields is "stay unchanged", while in the case of conditional rules the meaning of the missing fields is "same changes in conclusion as in the premise". In order for all the changes explicitly or implicitly specified by MSOS rules to apply, one also needs to provide an initial state for all the attributes, or in terms of SOS, an initial configuration. The initial configuration is often left unspecified in MSOS or SOS paper language definitions, but it needs to be explicitly given when one is concerned with executing the semantics. In our SOS definition of IMP in Section 3.3.2 (see Figure 3.15), we created the appropriate initial configuration in which the top-level statement was executed using the proof system itself, more precisely the rule (SmallStep-Pgm) created a configuration holding a statement and a state from a configuration holding only the program. That is not possible in MSOS, because MSOS assumes that the structure of the label record does not change dynamically as the rules are applied. Instead, it assumes all the attributes given and fixed. Therefore, one has to explicitly state the initial values corresponding to each attribute in the initial state. However, in practice those initial values are understood and, consequently, we do not bother defining them. For example, if an attribute holds a list or a set, then its initial value is the empty list or set; if it holds a partial function, then its initial value is the partial function undefined everywhere; etc.

This way of regarding MSOS as a convenient front-end to SOS also supports the introduction of further

notational conventions in MSOS if desired, like the one discussed above using $\mathsf{state}(x)$ instead of $\sigma(x)$ in the right-hand-side of the transition, provided that one explains how such conventions are desugared when going from MSOS to SOS. Finally, the translation of MSOS into SOS also allows MSOS to borrow from SOS the reflexive/transitive closure $\rightarrow^\star$ of the one-step relation.

3.6.1 The MSOS of IMP

Figures 3.23 and 3.24 show the MSOS definition of IMP. There is not much to comment on the MSOS rules in these figures, except, perhaps, to note how compact and elegant they are compared to the corresponding SOS definition in Figures 3.14 and 3.15. Except for the three rules (MSOS-Lookup), (MSOS-Asgn), and (MSOS-Var), which make use of labels, they are as compact as they can be in any SOS-like setting for any language including the defined constructs. Also, the above-mentioned three rules only mention those components from the labels that they really need, so they allow for possible extensions of the language, like the IMP++ extension in Section 3.5.

The rule (MSOS-Var) is somehow different from the other rules that need the information in the label, in that it uses an attribute which has the type read-write but it only writes it without reading it. This is indeed possible in MSOS. The type of an attribute cannot be necessarily inferred from the way it is used in some of the rules, and not all rules must use the same attribute in the same way. One should explicitly clarify the type of each attribute before one gives the actual MSOS rules, and one is not allowed to change the attribute types dynamically, during derivations. Indeed, if the type of the output attribute in the MSOS rules for output above (and also in Section 3.6.2) were read-write, then the rules would wrongly imply that the output buffer will only store the last value, the previous ones being lost (this could be a desirable semantics in some cases).

Since the MSOS proof system in Figures 3.23 and 3.24 translates, following the informal procedure described above, in the SOS proof system in Figures 3.14 and 3.15, basically all the small-step SOS intuitions and discussions for IMP in Section 3.3.2 carry over here almost unchanged. In particular:

Definition 22. *We say that $C \rightarrow C'$ is derivable with the MSOS proof system in Figures 3.23 and 3.24, written* $\text{MSOS(IMP)} \vdash C \rightarrow C'$*, iff* $\text{SmallStep(IMP)} \vdash C \rightarrow C'$ *(using the proof system in Figures 3.14 and 3.15). Similarly,* $\text{MSOS(IMP)} \vdash C \rightarrow^\star C'$ *iff* $\text{SmallStep(IMP)} \vdash C \rightarrow^\star C'$.

Note, however, that MSOS is more syntactic in nature than SOS, in that each of its reduction rules requires syntactic terms in both sides of the transition relation. In particular, that means that, unlike in SOS (see Exercise 68), in MSOS one does not have the option to dissolve statements from configurations anymore. Instead, one needs to reduce them to {} or some similar syntactic constant; if the original language did not have such a constant then one needs to invent one and add it to the original language or calculus syntax.

3.6.2 The MSOS of IMP++

We next discuss the MSOS of IMP++, playing the same language design scenario as in Section 3.5: we first add each feature separately to IMP, as if that feature was the final extension of the language, and then we add all the features together and investigate the modularity of the resulting definition as well as possibly unexpected feature interactions.

Variable Increment

In MSOS, one can define the increment modularly:

$$++\,x \xrightarrow{\{\mathsf{state}=\sigma,\ \mathsf{state'}=\sigma[(\sigma(x)+_{Int}1)/x],\ \ldots\}} \sigma(x) +_{Int} 1 \qquad \text{(MSOS-Inc)}$$

$$x \xrightarrow{\{\text{state}=\sigma,\ \ldots\}} \sigma(x) \quad \text{if } \sigma(x) \neq \bot \qquad \text{(MSOS-Lookup)}$$

$$\frac{a_1 \to a_1'}{a_1 + a_2 \to a_1' + a_2} \qquad \text{(MSOS-Add-Arg1)}$$

$$\frac{a_2 \to a_2'}{a_1 + a_2 \to a_1 + a_2'} \qquad \text{(MSOS-Add-Arg2)}$$

$$i_1 + i_2 \to i_1 +_{Int} i_2 \qquad \text{(MSOS-Add)}$$

$$\frac{a_1 \to a_1'}{a_1 \ / \ a_2 \to a_1' \ / \ a_2} \qquad \text{(MSOS-Div-Arg1)}$$

$$\frac{a_2 \to a_2'}{a_1 \ / \ a_2 \to a_1 \ / \ a_2'} \qquad \text{(MSOS-Div-Arg2)}$$

$$i_1 \ / \ i_2 \to i_1 \ /_{Int} \ i_2 \quad \text{if } i_2 \neq 0 \qquad \text{(MSOS-Div)}$$

$$\frac{a_1 \to a_1'}{a_1 <= a_2 \to a_1' <= a_2} \qquad \text{(MSOS-Leq-Arg1)}$$

$$\frac{a_2 \to a_2'}{i_1 <= a_2 \to i_1 <= a_2'} \qquad \text{(MSOS-Leq-Arg2)}$$

$$i_1 <= i_2 \to i_1 \leq_{Int} i_2 \qquad \text{(MSOS-Leq)}$$

$$\frac{b \to b'}{!\ b \to\ !\ b'} \qquad \text{(MSOS-Not-Arg)}$$

$$!\ \texttt{true} \to \texttt{false} \qquad \text{(MSOS-Not-True)}$$

$$!\ \texttt{false} \to \texttt{true} \qquad \text{(MSOS-Not-False)}$$

$$\frac{b_1 \to b_1'}{b_1\ \&\&\ b_2 \to b_1'\ \&\&\ b_2} \qquad \text{(MSOS-And-Arg1)}$$

$$\texttt{false}\ \&\&\ b_2 \to \texttt{false} \qquad \text{(MSOS-And-False)}$$

$$\texttt{true}\ \&\&\ b_2 \to b_2 \qquad \text{(MSOS-And-True)}$$

Figure 3.23: MSOS(IMP) — MSOS of IMP Expressions ($i_1, i_2 \in Int$; $x \in Id$; $a_1, a_1', a_2, a_2' \in AExp$; $b, b', b_1, b_1', b_2 \in BExp$; $\sigma \in State$).

$$\frac{a \to a'}{x = a\texttt{;} \to x = a'\texttt{;}} \qquad \text{(MSOS-Asgn-Arg2)}$$

$$x = i\texttt{;} \xrightarrow{\{\text{state}=\sigma,\ \text{state'}=\sigma[i/x],\ ...\}} \{\} \quad \text{if } \sigma(x) \neq \bot \qquad \text{(MSOS-Asgn)}$$

$$\frac{s_1 \to s_1'}{s_1\ s_2 \to s_1'\ s_2} \qquad \text{(MSOS-Seq-Arg1)}$$

$$\{\}\ s_2 \to s_2 \qquad \text{(MSOS-Seq-Skip)}$$

$$\frac{b \to b'}{\texttt{if}\,(b)\ s_1\ \texttt{else}\ s_2 \to \texttt{if}\,(b')\ s_1\ \texttt{else}\ s_2} \qquad \text{(MSOS-If-Arg1)}$$

$$\texttt{if}\,(\texttt{true})\ s_1\ \texttt{else}\ s_2 \to s_1 \qquad \text{(MSOS-If-True)}$$

$$\texttt{if}\,(\texttt{false})\ s_1\ \texttt{else}\ s_2 \to s_2 \qquad \text{(MSOS-If-False)}$$

$$\texttt{while}\,(b)\ s \to \texttt{if}\,(b)\,\{\ s\ \texttt{while}\,(b)\ s\ \}\,\texttt{else}\,\{\} \qquad \text{(MSOS-While)}$$

$$\texttt{int}\,xl\texttt{;}\ s \xrightarrow{\{\text{state'}=xl\mapsto 0,\ ...\}} s \qquad \text{(MSOS-Var)}$$

Figure 3.24: MSOS(IMP) — MSOS of IMP Statements ($i \in Int$; $x \in Id$; $xl \in \mathbf{List}\{Id\}$; $a, a' \in AExp$; $b, b' \in BExp$; $s, s_1, s_1', s_2 \in Stmt$; $\sigma \in State$).

No other rule needs to be changed, because MSOS already assumes that, unless otherwise specified, each rule propagates all the configuration changes in its premise(s).

Input/Output

MSOS can modularly support the input/output extension of IMP. We need to add new label attributes holding the input and the output buffers, say input and output, respectively, and then to add the corresponding rules for the input/output constructs. Note that the input attribute is read-write, while the output attribute is write-only. Here are the MSOS rules for input/output:

$$\texttt{read()} \xrightarrow{\{\text{input}=i:\omega,\ \text{input'}=\omega,\ ...\}} i \qquad \text{(MSOS-Read)}$$

$$\frac{a \to a'}{\texttt{print}(a)\texttt{;} \to \texttt{print}(a')\texttt{;}} \qquad \text{(MSOS-Print-Arg)}$$

$$\texttt{print}(i)\texttt{;} \xrightarrow{\{\text{output'}=i,\ ...\}} \{\} \qquad \text{(MSOS-Print)}$$

Note that, since output is a write-only attribute, we only need to mention the new value that is added to the output in the label of the second rule above. If output was declared as a read-write attribute, then the label of the second rule above would have been $\{\text{output} = \omega, \text{output'} = \omega : i, \ldots\}$. A major implicit objective of MSOS is to minimize the amount of information that the user needs to write in each rule. Indeed, anything written by a user can lead to non-modularity and thus work against the user when changes are performed to

the language. For example, if for some reason one declared output as a read-write attribute and then later on one decided to change the list construct for the output integer list from colon "_ : _" to something else, say "_ · _", then one would need to change the label in the second rule above from $\{\mathsf{output} = \omega, \mathsf{output'} = \omega : i, \ldots\}$ to $\{\mathsf{output} = \omega, \mathsf{output'} = (\omega \cdot i), \ldots\}$. Therefore, in the spirit of enhanced modularity and clarity, the language designer using MSOS is strongly encouraged to use write-only (or read-only) attributes instead of read-write attributes whenever possible.

Notice the lack of expected duality between the rules (MSOS-Read) and (MSOS-Print) for input and for output above. Indeed, for all the reasons mentioned above, one would like to write the rule (MSOS-Read) more compactly and modularly as follows:

$$\texttt{read()} \xrightarrow{\{\mathsf{input}=i,\ \ldots\}} i$$

Unfortunately, this is not possible with the current set of label attributes provided by MSOS. However, there is no reason why MSOS could not be extended to include more attributes. For example, an attribute called "consumable" which would behave as the dual of write-only, i.e., it would only have an unprimed variant in the label holding a monoid (or maybe a group?) structure like the read-only attributes but it would consume from it whatever is matched by the rule label, would certainly be very useful in our case here. If such an attribute type were available, then our input attribute would be of that type and our MSOS rule for `read()` would be like the one above.

A technical question regarding the execution of the resulting MSOS definition is how to provide input to programs. Or, put differently, how to initialize configurations. One possibility is to assume that the user is fully responsible for providing the initial attribute values. This is, however, rather inconvenient, because the user would then always have to provide an empty state and an empty output buffer in addition to the desired input buffer in each configuration. A more convenient approach is to invent a special syntax allowing the user to provide precisely a program and an input to it, and then to automatically initialize all the attributes with their expected values. Let us pair a program p and an input ω for it using a configuration-like notation of the form $\langle p, \omega \rangle$. Then we can replace the rule (MSOS-Var) in Figure 3.24 with the following rule:

$$\langle \texttt{int}\, xl\texttt{;}\ s,\ \omega \rangle \xrightarrow{\{\mathsf{state'}=xl\mapsto 0,\ \mathsf{input'}=\omega,\ \ldots\}} s$$

Abrupt Termination

MSOS allows for a more modular semantics of abrupt termination than the more conventional semantic approaches discussed in Section 3.5.3. However, in order to achieve modularity, we need to extend the syntax of IMP with a `top` construct, similarly to the small-step SOS variant discussed in Section 3.5.3. The key to modularity here is to use the labeling mechanism of MSOS to carry the information that a configuration is in a halting status. Let us assume an additional write-only field in the MSOS labels, called halting, which is *true* whenever the program needs to halt, otherwise it is *false*[7]. Then we can add the following two MSOS rules that set the halting field to *true*:

$$i_1 \ \texttt{/}\ 0 \xrightarrow{\{\mathsf{halting'}=\mathit{true},\ \ldots\}} i_1 \ \texttt{/}\ 0 \qquad \text{(MSOS-Div-By-Zero)}$$

$$\texttt{halt;} \xrightarrow{\{\mathsf{halting'}=\mathit{true},\ \ldots\}} \texttt{halt;} \qquad \text{(MSOS-Halt)}$$

[7]Strictly speaking, MSOS requires that the write-only attributes take values from a free monoid; if one wants to be faithful to that MSOS requirement, then one can replace *true* with some letter word and *false* with the empty word.

As desired, it is indeed the case now that a sequent of the form $s \xrightarrow{\{\mathsf{halting}'=\mathit{true},\ ...\}} s'$ is derivable if and only if $s = s'$ and the next executable step in s is either a "`halt;`" statement or a division-by-zero expression. If one does not like keeping the syntax unchanged when an abrupt termination takes place, then one can add a new syntactic construct, say `stuck` like in [53], and replace the right-hand-side configurations above with `stuck`; that does not conceptually change anything in what follows. The setting seems therefore perfect for adding a rule of the form

$$\frac{s \xrightarrow{\{\mathsf{halting}'=\mathit{true},\ ...\}} s}{s \xrightarrow{\{\mathsf{halting}'=\mathit{false},\ ...\}} \{\}}$$

and declare ourselves done, because now an abruptly terminated statement terminates just like any other statement, with a {} statement as result and with a label containing a non-halting status. Unfortunately, that does not work, because such a rule would interfere with other rules taking statement reductions as preconditions, for example with the first precondition of the (MSOS-Seq) rule, and thus hide the actual halting status of the precondition. To properly capture the halting status, we define a top level statement construct like we discussed in the context of big-step and small-step SOS above, say `top` *Stmt*, modify the rule (MSOS-Var) from

$$\texttt{int}\,xl\texttt{;}\ s \xrightarrow{\{\mathsf{state}'=xl\mapsto 0,\ ...\}} s$$

to

$$\texttt{int}\,xl\texttt{;}\ s \xrightarrow{\{\mathsf{state}'=xl\mapsto 0,\ \mathsf{halting}'=\mathit{false},\ ...\}} \texttt{top}\ s$$

to mark the top level statement, and then finally include the following three rules:

$$\frac{s \xrightarrow{\{\mathsf{halting}'=\mathit{false},\ ...\}} s'}{\texttt{top}\ s \xrightarrow{\{\mathsf{halting}'=\mathit{false},\ ...\}} \texttt{top}\ s'} \qquad \text{(MSOS-Top-Normal)}$$

$$\texttt{top}\ \{\} \to \{\} \qquad \text{(MSOS-Top-Skip)}$$

$$\frac{s \xrightarrow{\{\mathsf{halting}'=\mathit{true},\ ...\}} s}{\texttt{top}\ s \xrightarrow{\{\mathsf{halting}'=\mathit{false},\ ...\}} \{\}} \qquad \text{(MSOS-Top-Halting)}$$

The use of a `top` construct like above seems unavoidable if we want to achieve modularity. Indeed, we managed to avoid it in the small-step SOS definition of abrupt termination in Section 3.5.3 (paying one additional small-step to dissolve the halting configuration), because the halting configurations were explicitly, and thus non-modularly propagated through each of the language constructs, so the entire program reduced to a halting configuration whenever a division by zero or a "`halt;`" statement was encountered. Unfortunately, that same approach does not work with MSOS (unless we want to break its modularity, like in SOS), because the syntax is not mutilated when an abrupt termination occurs. The halting signal is captured by the label of the transition. However, the label does not tell us when we are at the top level in order to dissolve the halting status. Adding a new label to hold the depth of the derivation, or at least whether we are the top or not, would require one to (non-modularly) change it in each rule. The use of an additional `top` construct like we did above appears to be the best trade-off between modularity and elegance here.

Note that, although MSOS can be mechanically translated into SOS by associating to each MSOS attribute an SOS configuration component, the solution above to support abrupt termination modularly in MSOS is *not*

modular when applied in SOS via the translation. Indeed, adding a new attribute in the label means adding a new configuration component, which already breaks the modularity of SOS. In other words, the MSOS technique above cannot be manually used in SOS to obtain a modular definition of abrupt termination in SOS.

Dynamic Threads

The small-step SOS rules for spawning threads in Section 3.5.4 straightforwardly turn into MSOS rules:

$$\frac{s \to s'}{\texttt{spawn}\ s \to \texttt{spawn}\ s'} \qquad \text{(MSOS-Spawn-Arg)}$$

$$\texttt{spawn}\ \{\} \to \{\} \qquad \text{(MSOS-Spawn-Skip)}$$

$$\frac{s_2 \to s_2'}{(\texttt{spawn}\ s_1)\ s_2 \to (\texttt{spawn}\ s_1)\ s_2'} \qquad \text{(MSOS-Spawn-Wait)}$$

$$(s_1\ s_2)\ s_3 \equiv s_1\ (s_2\ s_3) \qquad \text{(MSOS-Seq-Assoc)}$$

$$\mathit{Stmt}\ ::=\ |\ \texttt{spawn}\ \mathit{Stmt}$$

Even though the MSOS rules above are conceptually identical to the original small-step SOS rules, they are more modular because, unlike the former, they carry over unchanged when the configuration needs to change. Note that the structural identity stating the associativity of sequential composition, called (MSOS-Seq-Assoc) above, is still necessary, and so is the syntactic extension of `spawn` to take statements instead of blocks.

Local Variables

Section 3.5.5 showed how blocks with local variables can be desugared into a uniform `let` construct, and also gave the small-step SOS rules defining the semantics of `let`. Those rules can be immediately adapted into the following MSOS rules:

$$\frac{a \to a'}{\texttt{let}\ x = a\ \texttt{in}\ s \to \texttt{let}\ x = a'\ \texttt{in}\ s} \qquad \text{(MSOS-Let-Exp)}$$

$$\frac{s \xrightarrow{\{\text{state}=\sigma[i/x],\ \text{state}'=\sigma',\ \ldots\}} s'}{\texttt{let}\ x = i\ \texttt{in}\ s \xrightarrow{\{\text{state}=\sigma,\ \text{state}'=\sigma'[\sigma(x)/x],\ \ldots\}} \texttt{let}\ x = \sigma'(x)\ \texttt{in}\ s'} \qquad \text{(MSOS-Let-Stmt)}$$

$$\texttt{let}\ x = i\ \texttt{in}\ \{\} \to \{\} \qquad \text{(MSOS-Let-Done)}$$

Like for the other features, the MSOS rules are more modular than their small-step SOS variants.

Since programs are now just ordinary (closed) expressions and they are now executed in the empty state, the rule (MSOS-Var), namely

$$\texttt{int}\ xl;\ s \xrightarrow{\{\text{state}'=xl\mapsto 0,\ \ldots\}} s$$

needs to change into a rule of the from

$$s \xrightarrow{\{\text{state}'=\cdot,\ \ldots\}} ?$$

Unfortunately, regardless of what we place instead of "?", such a rule will not work. That is because there is nothing to prevent it to apply to any statement at any step during the reduction. To enforce it to happen only at the top of the program and only at the beginning of the reduction, we can wrap the original program (which is a statement) into a one-element configuration-like term $\langle s \rangle$. Then the rule (MSOS-Var) can be replaced with the following rule:

$$\langle s \rangle \xrightarrow{\{\mathsf{state}' = \cdot,\ \ldots\}} s$$

Putting Them All Together

The modularity of MSOS makes it quite easy to put all the features discussed above together and thus define the MSOS of IMP++. Effectively, we have to do the following:

1. We add the three label attributes used for the semantics of the individual features above, namely the read-write input attribute and the two write-only attributes output and halting.

2. We add all the MSOS rules of all the features above *unchanged* (nice!), except for the rule (MSOS-Var) for programs (which changed several times, anyway).

3. To initialize the label attributes, we add a pairing construct $\langle s, \omega \rangle$ like we did when we added the input/output extension of IMP, where s is a statement (programs are ordinary statements now) and ω is a buffer, and replace the rule (MSOS-Var) in Figure 3.24 with the following:

$$\langle s, \omega \rangle \xrightarrow{\{\mathsf{state}' = xl \mapsto 0,\ \mathsf{input}' = \omega,\ \mathsf{halting}' = \mathit{false},\ \ldots\}} \texttt{top}\ s$$

It is important to note that the MSOS rules of the individual IMP extensions can be very elegantly combined into one language (IMP++). The rule (MSOS-Var) had to globally change in order to properly initialize the attribute values, but nothing had to be done in the MSOS rules of any of the features in order to put it together with the other MSOS rules of the other features.

Unfortunately, even though each individual feature has its intended semantics, the resulting IMP++ language does not. We still have the same semantic problems with regards to concurrency that we had in the context of small-step SOS in Section 3.5.6. For example, the concurrency of `spawn` statements is limited to the blocks in which they appear. For example, a statement of the form (`let` $x = i$ `in spawn` s_1) s_2 does not allow s_2 to be evaluated concurrently with s_1. The statement s_1 has to evaluate completely and then the `let` statement dissolved, before any step in s_2 can be performed. To fix this problem, we would have to adopt a different solution, like the one proposed in Section 3.5.7 in the context of small-step SOS. Thanks to its modularity, MSOS would make such a solution easier to implement than small-step SOS. Particularly, one can use the label mechanism to pass a spawned thread and its execution environment all the way to the top modularly, without having to propagate it explicitly through language constructs.

3.6.3 MSOS in Rewrite Logic

Like big-step and small-step SOS, we can also associate a conditional rewrite rule to each MSOS rule and hereby obtain a rewrite logic theory that faithfully (i.e., step-for-step) captures the MSOS definition. There could be different ways to do this. One way to do it is to first desugar the MSOS definition into a step-for-step equivalent small-step SOS definition as discussed above, and then use the faithful embedding of small-step SOS into rewrite logic discussed in Section 3.3.3. The problem with this approach is that the resulting small-step SOS definition, and implicitly the resulting rewrite logic definition, lack the modularity of the

original MSOS definition. In other words, if one wanted to extend the MSOS definition with rules that would require global changes to its corresponding SOS definition (e.g., ones adding new semantic components into the label/configuration), then one would also need to manually incorporate all those global changes in the resulting rewrite logic definition.

We first show that any MSOS proof system, say MSOS, can be mechanically translated into a rewrite logic theory, say $\mathcal{R}_{\text{MSOS}}$, in such a way that two important aspects of the original MSOS definition are preserved: 1) the corresponding derivation relations are step-for-step equivalent, that is, MSOS $\vdash C \to C'$ if and only if $\mathcal{R}_{\text{MSOS}} \vdash \mathcal{R}_{C \to C'}$, where $\mathcal{R}_{C \to C'}$ is the corresponding syntactic translation of the MSOS sequent $C \to C'$ into a rewrite logic sequent; and 2) $\mathcal{R}_{\text{MSOS}}$ is as modular as MSOS. Second, we apply our generic translation technique to the MSOS formal system MSOS(IMP) defined in Section 3.6.1 and obtain a rewrite logic semantics of IMP that is step-for-step equivalent to and as modular as MSOS(IMP). The modularity of MSOS(IMP) and of $\mathcal{R}_{\text{MSOS(IMP)}}$ will pay off when we extend IMP in Section 3.5. Finally, we show how $\mathcal{R}_{\text{MSOS(IMP)}}$ can be seamlessly defined in Maude, yielding another interpreter for IMP (in addition to those corresponding to the big-step and small-step SOS definitions of IMP in Sections 3.2.3 and 3.3.3).

Computationally and Modularly Faithful Embedding of MSOS into Rewrite Logic

Our embedding of MSOS into rewrite logic is very similar to that of small-step SOS, with one important exception: the non-syntactic components of the configuration are all grouped into a *record*, which is a multiset of *attributes*, each attribute being a pair associating appropriate semantic information to a *field*. This allows us to use multiset *matching* (or matching modulo associativity, commutativity, and identity) in the corresponding rewrite rules to extract the needed semantic information from the record, thus achieving not only a computationally equivalent embedding of MSOS into rewrite logic, but also one with the same degree of modularity as MSOS.

Formally, let us assume an arbitrary MSOS formal proof system. Let *Attribute* be a fresh sort and let *Record* be the sort **Bag{***Attribute***}** (that means that we assume all the infrastructure needed to define records as comma-separated bags, or multisets, of attributes). For each field *Field* holding semantic contents *Contents* that appears unprimed or primed in any of the labels on any of the transitions in any of the rules of the MSOS proof system, let us assume an operation "*Field* = _ : *Contents* → *Attribute*" (the name of this postfix unary operation is "*Field* = _"). Finally, for each syntactic category *Syntax* used in any transition that appears anywhere in the MSOS proof system, let us define a configuration construct "$\langle _, _ \rangle$: *Syntax* × *Record* → *Configuration*". We are now ready to define our transformation of an MSOS rule into a rewrite logic rule:

1. Translate it into an SOS rule, as discussed right above Section 3.6.1; we could also go directly from MSOS rules to rewrite logic rules, but we would have to repeat most of the steps from MSOS to SOS that were already discussed;

2. Group all the semantic components in the resulting SOS configurations into a corresponding record, where each semantic component translates into a corresponding attribute using a corresponding label;

3. Replace those attributes whose semantic components are not used anywhere in the rule by a generic variable of sort *Record*;

4. Finally, use the technique in Section 3.3.3 to transform the resulting SOS-like rules into rewrite rules, tagging the left-hand-side configurations with the $\circ$ symbol.

Applying the steps above, the four MSOS rules discussed right above Section 3.6.1 (namely the ones for variable assignment, printing to the output, sequential composition of statements, and variable lookup) translate into the following rewrite logic rules:

$$\begin{aligned}
\circ\langle X = I\texttt{;}, (\mathsf{state} = \sigma,\ \rho)\rangle &\to \langle\{\}, (\mathsf{state} = \sigma[I/X],\ \rho)\rangle \\
\circ\langle \texttt{print}(I)\texttt{;}, (\mathsf{output} = \omega,\ \rho)\rangle &\to \langle\{\}, (\mathsf{output} = \omega : I,\ \rho)\rangle \\
\circ\langle S_1\ S_2, \rho\rangle &\to \langle S'_1\ S_2, \rho'\rangle \quad \textbf{if} \quad \circ\langle S_1, \rho\rangle \to \langle S'_1, \rho'\rangle \\
\circ\langle X, (\mathsf{state} = \sigma,\ \rho)\rangle &\to \langle \sigma(X), (\mathsf{state} = \sigma,\ \rho)\rangle
\end{aligned}$$

We use the same mechanism as for small-step SOS to obtain the reflexive and transitive many-step closure of the MSOS one-step transition relation. This mechanism was discussed in detail in Section 3.3.3; it essentially consists of adding a configuration marker $\star$ together with a rule "$\star Cfg \to \star Cfg'$ **if** $\circ Cfg \to Cfg'$" iteratively applying the one-step relation.

Theorem 16. ***(Faithful embedding of MSOS into rewrite logic)*** *For any MSOS definition* MSOS*, and any appropriate configurations C and C', the following equivalences hold:*

$$\text{MSOS} \vdash C \to C' \iff \mathcal{R}_{\text{MSOS}} \vdash \circ\,\overline{C} \to^1 \overline{C'} \iff \mathcal{R}_{\text{MSOS}} \vdash \circ\,\overline{C} \to \overline{C'}$$

$$\text{MSOS} \vdash C \to^\star C' \iff \mathcal{R}_{\text{MSOS}} \vdash \star\,\overline{C} \to \star\,\overline{C'}$$

where $\mathcal{R}_{\text{MSOS}}$ is the rewrite logic semantic definition obtained from MSOS *by translating each rule in* MSOS *as above. (Recall from Section 2.5 that $\to^1$ is the one-step rewriting relation obtained by dropping the reflexivity and transitivity rules of rewrite logic. Also, as C and C' are parameter-free—parameters only appear in rules—, $\overline{C}$ and $\overline{C'}$ are ground terms.)*

Like for the previous embeddings of big-step and small-step SOS into rewrite logic, let us elaborate on the apparent differences between MSOS and $\mathcal{R}_{\text{MSOS}}$ from a user perspective. The most visible difference is the SOS-like style of writing the rules, namely using configurations instead of labels, which also led to the inheritance of the $\circ$ mechanism from the embedding of SOS into rewrite logic. Therefore, the equivalent rewrite logic definition is slightly more verbose than the original MSOS definition. On the other hand, it has the advantage that it is more direct than the MSOS definition, in that it eliminates all the notational conventions. Indeed, if we strip MSOS out of its notational conventions and go straight to its essence, we find that that essence is precisely its use of multiset matching to modularly access the semantic components of the configuration. MSOS chose to do this on the labels, using specialized conventions for read-only/write-only/read-write components, while our rewrite logic embedding of MSOS does it in the configurations, uniformly for all semantic components. Where precisely this matching takes place is, in our view, less relevant. What is relevant and brings MSOS its modularity is that multiset matching *does* happen. Therefore, similarly to the big-step and small-step SOS representations in rewrite logic, we conclude that the rewrite theory $\mathcal{R}_{\text{MSOS}}$ *is* MSOS, and *not* an encoding of it.

Like for the previous embeddings of big-step and small-step SOS into rewriting logic, unfortunately, $\mathcal{R}_{\text{MSOS}}$ (and implicitly MSOS) still lacks the main strengths of rewrite logic, namely context-insensitive and parallel application of rewrite rules. Indeed, the rules of $\mathcal{R}_{\text{MSOS}}$ can only apply at the top, sequentially, so these rewrite theories corresponding to the faithful embedding of MSOS follow a rather poor rewrite logic style. Like for the previous embeddings, this is not surprising though and does not question the quality of our embeddings. All it says is that MSOS was not meant to have the capabilities of rewrite logic with regards to context-insensitivity and parallelism; indeed, all MSOS attempts to achieve is to address the lack of modularity of SOS and we believe that it succeeded in doing so. Unfortunately, SOS has several other major problems, which are discussed in Section 3.5.

sorts:
Attribute, *Record*, *Configuration*, *ExtendedConfiguration*
subsorts and aliases:
Record = **Bag**{*Attribute*}
Configuration < *ExtendedConfiguration*
operations:

$$
\begin{array}{rcll}
\mathsf{state} = _ & : & \mathit{State} \to \mathit{Attribute} & \text{// more fields can be added by need} \\
\langle _, _ \rangle & : & \mathit{AExp} \times \mathit{Record} \to \mathit{Configuration} & \\
\langle _, _ \rangle & : & \mathit{BExp} \times \mathit{Record} \to \mathit{Configuration} & \\
\langle _, _ \rangle & : & \mathit{Stmt} \times \mathit{Record} \to \mathit{Configuration} & \\
\langle _ \rangle & : & \mathit{Pgm} \to \mathit{Configuration} & \\
\circ _ & : & \mathit{Configuration} \to \mathit{ExtendedConfiguration} & \text{// reduce one step} \\
\star _ & : & \mathit{Configuration} \to \mathit{ExtendedConfiguration} & \text{// reduce all steps}
\end{array}
$$

rule:
$\star \mathit{Cfg} \to \star \mathit{Cfg}'$ **if** $\circ\, \mathit{Cfg} \to \mathit{Cfg}'$ // where $\mathit{Cfg}, \mathit{Cfg}'$ are variables of sort *Configuration*

Figure 3.25: Configurations and infrastructure for the rewrite logic embedding of MSOS(IMP).

MSOS of IMP in Rewrite Logic

We here discuss the complete MSOS definition of IMP in rewrite logic, obtained by applying the faithful embedding technique discussed above to the MSOS definition of IMP in Section 3.6.1. Figure 3.25 gives an algebraic definition of configurations as well as needed additional record infrastructure; all the sorts, operations, and rules in Figure 3.25 were already discussed either above or in Section 3.3.3. Figure 3.26 gives the rules of the rewrite logic theory $\mathcal{R}_{\text{MSOS(IMP)}}$ that is obtained by applying the procedure above to the MSOS of IMP in Figures 3.23 and 3.24. Like before, we used the rewrite logic convention that variables start with upper-case letters; if they are greek letters, then we use a similar but larger symbol (e.g., Σ instead of σ for variables of sort *State*, or $\Rho$ instead of ρ for variables of sort *Record*). The following corollary of Theorem 16 establishes the faithfulness of the representation of the MSOS of IMP in rewrite logic:

Corollary 6. MSOS(IMP) $\vdash C \to C' \iff \mathcal{R}_{\text{MSOS(IMP)}} \vdash \circ\overline{C} \to \overline{C'}$.

Therefore, there is no perceivable computational difference between the IMP-specific proof system MSOS(IMP) and generic rewrite logic deduction using the IMP-specific rewrite rules in $\mathcal{R}_{\text{MSOS(IMP)}}$; the two are faithfully equivalent. Moreover, by the discussion following Theorem 16, $\mathcal{R}_{\text{MSOS(IMP)}}$ is also as modular as MSOS(IMP). This will be further emphasized in Section 3.5, where we will extend IMP with several features, some of which requiring more attributes.

★ Maude Definition of IMP MSOS

Figure 3.27 shows a straightforward Maude representation of the rewrite theory $\mathcal{R}_{\text{MSOS(IMP)}}$ in Figures 3.26 and 3.25. The Maude module `IMP-SEMANTICS-MSOS` in Figure 3.27 is executable, so Maude, through its rewriting capabilities, yields an MSOS interpreter for IMP the same way it yielded big-step and small-step SOS interpreters in Sections 3.2.3 and 3.3.3, respectively; for example, the Maude rewrite command

```
rewrite * < sumPgm > .
```

where `sumPgm` is the first program defined in the module `IMP-PROGRAMS` in Figure 3.4, produces a result of the form (the exact statistics are also irrelevant, so they were replaced by "...”):

$$
\begin{array}{rll}
\circ\langle X, (\mathsf{state} = \sigma, \rho)\rangle \to \langle \sigma(X), (\mathsf{state} = \sigma, \rho)\rangle & \textbf{if} & \sigma(X) \neq \bot \\
\\
\circ\langle A_1 + A_2, \rho\rangle \to \langle A_1' + A_2, \rho'\rangle & \textbf{if} & \circ\langle A_1, \rho\rangle \to \langle A_1', \rho'\rangle \\
\circ\langle A_1 + A_2, \rho\rangle \to \langle A_1 + A_2', \rho'\rangle & \textbf{if} & \circ\langle A_2, \rho\rangle \to \langle A_2', \rho'\rangle \\
\circ\langle I_1 + I_2, \rho\rangle \to \langle I_1 +_{Int} I_2, \rho\rangle & & \\
\\
\circ\langle A_1 \ / \ A_2, \rho\rangle \to \langle A_1' \ / \ A_2, \rho'\rangle & \textbf{if} & \circ\langle A_1, \rho\rangle \to \langle A_1', \rho'\rangle \\
\circ\langle A_1 \ / \ A_2, \rho\rangle \to \langle A_1 \ / \ A_2', \rho'\rangle & \textbf{if} & \circ\langle A_2, \rho\rangle \to \langle A_2', \rho'\rangle \\
\circ\langle I_1 \ / \ I_2, \rho\rangle \to \langle I_1 \ /_{Int} \ I_2, \rho\rangle & \textbf{if} & I_2 \neq 0 \\
\\
\circ\langle A_1 <= A_2, \rho\rangle \to \langle A_1' <= A_2, \rho'\rangle & \textbf{if} & \circ\langle A_1, \rho\rangle \to \langle A_1', \rho'\rangle \\
\circ\langle I_1 <= A_2, \rho\rangle \to \langle I_1 <= A_2', \rho'\rangle & \textbf{if} & \circ\langle A_2, \rho\rangle \to \langle A_2', \rho'\rangle \\
\circ\langle I_1 <= I_2, \rho\rangle \to \langle I_1 \leq_{Int} I_2, \rho\rangle & & \\
\\
\circ\langle \texttt{!}\ B, \rho\rangle \to \langle \texttt{!}\ B', \rho'\rangle & \textbf{if} & \circ\langle B, \rho\rangle \to \langle B', \rho'\rangle \\
\circ\langle \texttt{!}\ \texttt{true}, \rho\rangle \to \langle \texttt{false}, \rho\rangle & & \\
\circ\langle \texttt{!}\ \texttt{false}, \rho\rangle \to \langle \texttt{true}, \rho\rangle & & \\
\\
\circ\langle B_1 \ \&\& \ B2, \rho\rangle \to \langle B_1' \ \&\& \ B2, \rho'\rangle & \textbf{if} & \circ\langle B_1, \rho\rangle \to \langle B_1', \rho'\rangle \\
\circ\langle \texttt{false} \ \&\& \ B2, \rho\rangle \to \langle \texttt{false}, \rho\rangle & & \\
\circ\langle \texttt{true} \ \&\& \ B2, \rho\rangle \to \langle B_2, \rho\rangle & & \\
\\
\circ\langle \{\ S\ \}, \rho\rangle \to \langle S, \rho\rangle & & \\
\\
\circ\langle X = A\texttt{;}, \rho\rangle \to \langle X = A'\texttt{;}, \rho'\rangle & \textbf{if} & \circ\langle A, \rho\rangle \to \langle A', \rho'\rangle \\
\circ\langle X = I\texttt{;}, (\mathsf{state} = \sigma, \rho)\rangle \to \langle \{\}, (\mathsf{state} = \sigma[I/X], \rho)\rangle & \textbf{if} & \sigma(X) \neq \bot \\
\\
\circ\langle S_1 \ S_2, \rho\rangle \to \langle S_1' \ S_2, \rho'\rangle & \textbf{if} & \circ\langle S_1, \rho\rangle \to \langle S_1', \rho'\rangle \\
\circ\langle \{\} \ S_2, \rho\rangle \to \langle S_2, \rho\rangle & & \\
\\
\circ\langle \texttt{if}\ (B)\ S_1\ \texttt{else}\ S_2, \rho\rangle \to \langle \texttt{if}\ (B')\ S_1\ \texttt{else}\ S_2, \rho'\rangle & \textbf{if} & \circ\langle B, \rho\rangle \to \langle B', \rho'\rangle \\
\circ\langle \texttt{if}\ (\texttt{true})\ S_1\ \texttt{else}\ S_2, \rho\rangle \to \langle S_1, \rho\rangle & & \\
\circ\langle \texttt{if}\ (\texttt{false})\ S_1\ \texttt{else}\ S_2, \rho\rangle \to \langle S_2, \rho\rangle & & \\
\\
\circ\langle \texttt{while}\ (B)\ S, \rho\rangle \to \langle \texttt{if}\ (B)\ \{\ S\ \ \texttt{while}\ (B)\ S\ \}\ \texttt{else}\ \{\}, \rho\rangle & & \\
\\
\circ\langle \texttt{int}\ Xl\texttt{;}\ S\rangle \to \langle S, (\mathsf{state} = Xl \mapsto 0)\rangle & &
\end{array}
$$

Figure 3.26: $\mathcal{R}_{\text{MSOS(IMP)}}$: the complete MSOS of IMP in rewrite logic.

```
mod IMP-CONFIGURATIONS-MSOS is including IMP-SYNTAX + STATE .
  sorts Attribute Record Configuration ExtendedConfiguration .
  subsort Attribute < Record .
  subsort Configuration < ExtendedConfiguration .
  op empty : -> Record .
  op _,_ : Record Record -> Record [assoc comm id: empty] .
  op state`=_ : State -> Attribute .
  op <_,_> : AExp Record -> Configuration .
  op <_,_> : BExp Record -> Configuration .
  op <_,_> : Stmt Record -> Configuration .
  op <_> : Pgm -> Configuration .
  op o_ : Configuration -> ExtendedConfiguration [prec 80] .   --- one step
  op *_ : Configuration -> ExtendedConfiguration [prec 80] .   --- all steps
  var Cfg Cfg' : Configuration .
 crl * Cfg => * Cfg' if o Cfg => Cfg' .
endm

mod IMP-SEMANTICS-MSOS is including IMP-CONFIGURATIONS-MSOS .
  var X : Id .   var R R' : Record . var Sigma Sigma' : State .
  var I I1 I2 : Int .  var Xl : List{Id} .  var S S1 S1' S2 : Stmt .
  var A A' A1 A1' A2 A2' : AExp .  var B B' B1 B1' B2 B2' : BExp .

 crl o < X,(state = Sigma, R) > => < Sigma(X),(state = Sigma, R) >
  if Sigma(X) =/=Bool undefined .
 crl o < A1 + A2,R > => < A1' + A2,R' > if o < A1,R > => < A1',R' > .
 crl o < A1 + A2,R > => < A1 + A2',R' > if o < A2,R > => < A2',R' > .
  rl o < I1 + I2,R > => < I1 +Int I2,R > .
 crl o < A1 / A2,R > => < A1' / A2,R' > if o < A1,R > => < A1',R' > .
 crl o < A1 / A2,R > => < A1 / A2',R' > if o < A2,R > => < A2',R' > .
 crl o < I1 / I2,R > => < I1 /Int I2,R > if I2 =/=Bool 0 .
 crl o < A1 <= A2,R > => < A1' <= A2,R' > if o < A1,R > => < A1',R' > .
 crl o < I1 <= A2,R > => < I1 <= A2',R' > if o < A2,R > => < A2',R' > .
  rl o < I1 <= I2,R > => < I1 <=Int I2,R > .
 crl o < ! B,R > => < ! B',R' > if o < B,R > => < B',R' > .
  rl o < ! true,R > => < false,R > .
  rl o < ! false,R > => < true,R > .
 crl o < B1 && B2,R > => < B1' && B2,R' > if o < B1,R > => < B1',R' > .
  rl o < false && B2,R > => < false,R > .
  rl o < true && B2,R > => < B2,R > .
  rl o < {S}, R > => < S,R > .
 crl o < X = A ;,R > => < X = A' ;,R' > if o < A,R > => < A',R' > .
 crl o < X = I ;,(state = Sigma, R) > => < {},(state = Sigma[I / X], R) >
  if Sigma(X) =/=Bool undefined .
 crl o < S1 S2,R > => < S1' S2,R' > if o < S1,R > => < S1',R' > .
  rl o < {} S2,R > => < S2,R > .
 crl o < if (B) S1 else S2,R > => < if (B') S1 else S2,R' > if o < B,R > => < B',R'  > .
  rl o < if (true) S1 else S2,R > => < S1,R > .
  rl o < if (false) S1 else S2,R > => < S2,R > .
  rl o < while (B) S,R > => < if (B) {S while (B) S} else {},R > .
  rl o < int Xl ; S > => < S,(state = Xl |-> 0) > .
endm
```

Figure 3.27: The MSOS of IMP in Maude, including the definition of configurations.

```
rewrites: 7632 in ... cpu (... real) (... rewrites/second)
result ExtendedConfiguration: * < {},state = (n |-> 0 & s |-> 5050) >
```

Note that the rewrite command above took the same number of rewrite steps as the similar command executed on the small-step SOS of IMP in Maude discussed in Section 3.3.3, namely 7632. This is not unexpected, because matching is not counted as rewrite steps, no matter how complex it is.

Like for the big-step and small-step SOS definitions in Maude, one can also use any of the general-purpose tools provided by Maude on the MSOS definition above. For example, one can exhaustively search for all possible behaviors of a program using the `search` command:

```
search * < sumPgm > =>! Cfg:ExtendedConfiguration .
```

As expected, only one behavior will be discovered because our IMP language so far is deterministic. The same number of states as in the case of small-step SOS will be generated by this search command, 1709.

3.6.4 Notes

Modular Structural Operational Semantics (MSOS) was introduced in 1999 by Mosses [51] and since then mainly developed by himself and his collaborators (e.g., [52, 53, 54]). In this section we used the implicitly-modular variant of MSOS introduced in [54], which, as acknowledged by the authors of [54], was partly inspired from discussions with us[8] To be more precise, we used a slightly simplified version of implicitly-modular MSOS here. In MSOS in its full generality, one can also declare some transitions *unobservable*; to keep the presentation simpler, we here omitted all the observability aspects of MSOS.

The idea of our representation of MSOS into rewrite logic adopted in this section is taken over from Serbanuta *et al.* [74]. At our knowledge, Meseguer and Braga [44] give the first representation of MSOS into rewrite logic. The representation in [44] also led to the development of the Maude MSOS tool [16], which was the core of Braga's doctoral thesis. What is different in the representation of Meseguer and Braga in [44] from ours is that the former uses two different types of configuration wrappers, one for the left-hand-side of the transitions and one for the right-hand-side; this was already discussed in Section 3.3.4.

3.6.5 Exercises

Exercise 120. *Redo all the exercises in Section 3.3.5 but for the MSOS of* IMP *discussed in Section 3.6.1 instead of its small-step SOS in Section 3.3.2. Skip Exercises 68, 69 and 76, since the SOS proof system there drops the syntactic components in the RHS configurations in transitions, making it unsuitable for MSOS. For the MSOS variant of Exercise 70, just follow the same non-modular approach as in the case of SOS and not the modular MSOS approach discussed in Section 3.6.2 (Exercise 123 addresses the modular MSOS variant of abrupt termination).*

Exercise 121. *Same as Exercise 86, but for MSOS instead of small-step SOS: add variable increment to* IMP*, like in Section 3.6.2.*

Exercise 122. *Same as Exercise 90, but for MSOS instead of small-step SOS: add input/output to* IMP*, like in Section 3.6.2.*

Exercise 123. *Same as Exercise 95, but for MSOS instead of small-step SOS: add abrupt termination to* IMP*, like in Section 3.6.2.*

[8] In fact, drafts of this book preceding [54] had already dropped the implicit labels in MSOS rules, for notational simplicity.

Exercise 124. *Same as Exercise 104, but for MSOS instead of small-step SOS: add dynamic threads to* IMP*, like in Section 3.6.2.*

Exercise 125. *Same as Exercise 109, but for MSOS instead of small-step SOS: add local variables using* `let` *to* IMP*, like in Section 3.6.2.*

Exercise 126. *This exercise asks to define* IMP++ *in MSOS, in various ways. Specifically, redo Exercises 114, 115, 116, 117, and 118, but for the MSOS of* IMP++ *discussed in Section 3.6.2 instead of its small-step SOS in Section 3.5.6.*

look into ThinkerType

should I add more stuff from [74], in particular the more precise embedding transformation from there? i mean, is the transformation discussed here detailed enough?

should I say more about [44]?

3.7 Reduction Semantics with Evaluation Contexts

The small-step SOS/MSOS approaches discussed in Sections 3.3 and 3.6 define a language semantics as a proof system whose rules are mostly conditional. The conditions of such rules allow to implicitly capture the program execution context as a *proof context*. This shift of focus from the informal notion of execution context to the formal notion of proof context has a series of advantages and it was, in fact, the actual point of formalizing language semantics using SOS. However, as the complexity of programming languages increased, in particular with the adoption of control-intensive statements like call/cc (e.g., Scheme) that can arbitrarily change the execution context, the need for an explicit representation of the execution context as a first-class citizen in the language semantics also increased. Reduction semantics with evaluation contexts (RSEC) is a variant of small-step SOS where the evaluation context may appear explicit in the term being reduced.

In an RSEC language definition one starts by defining the syntax of *evaluation contexts*, or simply just *contexts*, which is typically done by means of a context-free grammar (CFG). A context is a program or a fragment of program with a *hole*, where the hole, which is written □, is a placeholder for where the next computational step can take place. If c is an evaluation context and e is some well-formed appropriate fragment (expression, statement, etc.), then $c[e]$ is the program or fragment obtained by replacing the hole of c by e. Reduction semantics with evaluation contexts relies on a tacitly assumed (but rather advanced) parsing mechanism that takes a program or a fragment p and decomposes it into a context c and a subprogram or fragment e, called a *redex*, such that $p = c[e]$. This decomposition process is called *splitting* (of p into c and e). The inverse process, composing a redex e and a context c into a program or fragment p, is called *plugging* (of e into c). These splitting/plugging operations are depicted in Figure 3.28.

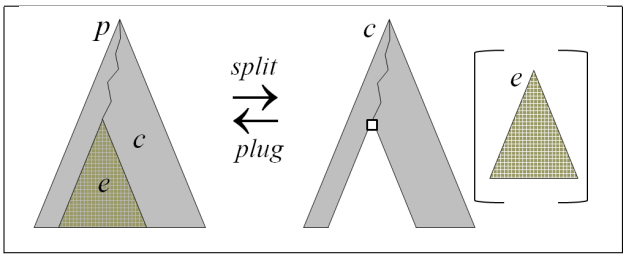


Figure 3.28: Decomposition of syntactic term p into context c and redex e, written $p = c[e]$: we say p *splits* into c and e, or e *plugs* into c yielding p. These operations are assumed whenever needed.

Consider a language with arithmetic/Boolean expressions and statements like our IMP language in Section 3.1. A possible CFG definition of evaluation contexts for such a language may include the following productions (the complete definition of IMP evaluation contexts is given in Figure 3.30):

```
Context ::= □
          | Context <= AExp
          | Int <= Context
          | Id = Context;
          | Context Stmt
          | if (Context) Stmt else Stmt
          | ...
```

Note how the intended evaluation strategies of the various language constructs are reflected in the definition of evaluation contexts: `<=` is sequentially strict (□ allowed to go into the first subexpression until evaluated to an *Int*, then into the second subexpression), the assignment is strict only in its second argument, while the sequential composition and the conditional are strict only in their first arguments. If one thinks of language constructs as operations taking a certain number of arguments of certain types, then note that the operations appearing in the grammar defining evaluation contexts are *different* from their corresponding operations

appearing in the language syntax; for example, "*Id* = *Context*;" is different from "*Id* = *AExp*;" because the former takes a context as second argument while the latter takes an arithmetic expression.

Here are some examples of correct evaluation contexts for the grammar above:

$\square$

3 <= $\square$

$\square$ <= 3

$\square$ x = 5;, where x is any variable.

`if (`$\square$`)` s_1 `else` s_2, where s_1 and s_2 are any well-formed statements.

Here are some examples of incorrect evaluation contexts:

$\square$ <= $\square$ — a context can have only one hole.

x <= 3 — a context must contain a hole.

x <= $\square$ — the first argument of <= must be an integer number in order to allow the hole in the second argument.

x = 5; $\square$ — the hole can only appear in the first statement in a sequential composition.

$\square$ = 5; — the hole cannot appear as first argument of the assignment construct.

`if (`x <= 7`)` $\square$ `else` x = 5; — the hole is only allowed in the condition of a conditional.

Here are some examples of decompositions of syntactic terms into a context and a redex (recall that we can freely use parentheses for disambiguation; here we enclose evaluation contexts in parentheses for clarity):

7 = ($\square$)[7]

3 <= x = (3 <= $\square$)[x] = ($\square$ <= x)[3] = ($\square$)[3 <= x]

3 <= (2 + x) + 7 = (3 <= $\square$ + 7)[2 + x] = ($\square$ <= (2 + x) + 7)[3] = ...

For simplicity, we consider only one type of context in this section, but in general one can have various types, depending upon the types of their holes and of their result.

Reduction semantics with evaluation contexts tends to be a purely syntactic definitional framework (following the slogan "everything is syntax"). If semantic components are necessary in a particular definition, then they are typically "swallowed by the syntax". For example, if one needs a state as part of the configuration for a particular language definition (like we need for our hypothetical IMP language discussed here), then one adds a context production of the form

$$\mathit{Context} \quad ::= \quad \langle \mathit{Context}, \mathit{State} \rangle$$

where the *State*, an inherently semantic entity, becomes part of the evaluation context. Note that once one adds additional syntax to evaluation contexts that does not correspond to constructs in the syntax of the original language, such as our pairing of a context and a state above, one needs to also extend the original syntax with corresponding constructs, so that the parsing-like mechanism decomposing a syntactic term into a context and a redex can be applied. In our case, the production above suggests that a pairing configuration construct of the form $\langle \mathit{Stmt}, \mathit{State} \rangle$, like for SOS, also needs to be defined. Unlike in SOS, we do not need configurations pairing other syntactic categories with a state, such as $\langle \mathit{AExp}, \mathit{State} \rangle$ and $\langle \mathit{BExp}, \mathit{State} \rangle$; the

reason is that, unlike in SOS, transitions with left-hand-side configurations $\langle Stmt, State\rangle$ are not derived anymore from transitions with left-hand-side configurations of the form $\langle AExp, State\rangle$ or $\langle BExp, State\rangle$.

Evaluation contexts are defined in such a way that whenever e is reducible, $c[e]$ is also reducible. For example, consider the term $c[i_1 \texttt{ <= } i_2]$ stating that expression $i_1 \texttt{ <= } i_2$ is in a proper evaluation context. Since $i_1 \texttt{ <= } i_2$ reduces to $i_1 \leq_{Int} i_2$, we can conclude that $c[i_1 \texttt{ <= } i_2]$ reduces to $c[i_1 \leq_{Int} i_2]$. Therefore, in reduction semantics with evaluation contexts we can define the semantics of `<=` using the following rule:

$$c[i_1 \texttt{ <= } i_2] \to c[i_1 \leq_{Int} i_2]$$

This rule is actually a rule schema, containing one rule instance for each concrete integers i_1, i_2 and for each appropriate evaluation context c. For example, here are instances of this rule when the context is $\square$, $\texttt{if (}\square\texttt{) \{\} else } x \texttt{ = 5;}$, and $\langle \square, (x \mapsto 1, y \mapsto 2)\rangle$, respectively:

$$\begin{array}{c} i_1 \texttt{ <= } i_2 \to i_1 \leq_{Int} i_2 \\ \texttt{if (}i_1 \texttt{ <= } i_2\texttt{) \{\} else } x \texttt{ = 5;} \to \texttt{if (}i_1 \texttt{ <= } i_2\texttt{) \{\} else } x \texttt{ = 5;} \\ \langle i_1 \texttt{ <= } i_2 \texttt{ ,} \ (x \mapsto 1, y \mapsto 2)\rangle \to \langle i_1 \leq_{Int} i_2 \texttt{ ,} \ (x \mapsto 1, y \mapsto 2)\rangle \end{array}$$

What is important to note here is that propagation rules, such as (MSOS-Leq-Arg1) and (MSOS-Leq-Arg2) in Figure 3.23, are not necessary anymore when using evaluation contexts, because the evaluation contexts already achieve the role of the propagation rules.

To reflect the fact that reductions take place only in appropriate contexts, RSEC typically introduces a rule schema of the form:

$$\frac{e \to e'}{c[e] \to c[e']} \qquad \text{(RSEC-Characteristic-Rule)}$$

where e, e' are well-formed fragments and c is any appropriate evaluation context (i.e., such that $c[e]$ and $c[e']$ are well-formed programs or fragments of program). This rule is called the *characteristic rule* of RSEC. When this rule is applied, we say that *e reduces to e' in context c*. If c is the empty context $\square$ then $c[e]$ is e and thus the characteristic rule is useless; for that reason, the characteristic rule may be encountered with a side condition "if $c \neq \square$". Choosing good strategies to search for splits of terms into contextual representations can be a key factor in obtaining efficient implementations of RSEC execution engines.

The introduction of the characteristic rule allows us to define reduction semantics of languages or calculi quite compactly. For example, here are all the rules needed to completely define the semantics of the comparison, sequential composition and conditional language constructs for which we defined evaluation contexts above:

$$\begin{array}{r} i_1 \texttt{ <= } i_2 \to i_1 \leq_{Int} i_2 \\ \texttt{\{\}}\ s_2 \to s_2 \\ \texttt{if (true)}\ s_1 \texttt{ else } s_2 \to s_1 \\ \texttt{if (false)}\ s_1 \texttt{ else } s_2 \to s_2 \end{array}$$

The characteristic rule tends to be the only conditional rule in an RSEC, in the sense that the remaining rules take no reduction premises (though they may still have side conditions). Moreover, as already pointed out, the characteristic rule is actually unnecessary, because one can very well replace each rule $l \to r$ by a rule $c[l] \to c[r]$. The essence of reduction semantics with evaluation contexts is not its characteristic reduction rule, but its specific approach to defining evaluation contexts as a grammar and then using them as an explicit part of languages or calculi definitions. The characteristic reduction rule can therefore be regarded as "syntactic sugar", or convenience to the designer allowing her to write more compact definitions.

To give the semantics of certain language constructs, one may need to access specific information that is stored inside an evaluation context. For example, consider a term $\langle x <= 3,\ (x \mapsto 1, y \mapsto 2)\rangle$, which can be split as $c[x]$, where c is the context $\langle \square <= 3,\ (x \mapsto 1, y \mapsto 2)\rangle$. In order to reduce $c[x]$ to $c[1]$ as desired, we need to look inside c and find out that the value of x in the state held by c is 1. Therefore, following the purely syntactic style adopted so far in this section, the reduction semantics with evaluation contexts rule for variable lookup in our case here is the following:

$$\langle c, \sigma\rangle[x] \to \langle c, \sigma\rangle[\sigma(x)] \quad \text{if } \sigma(x) \neq \bot$$

Indeed, the same way we add as much structure as needed in ordinary terms, we can add as much structure as needed in evaluation contexts. Similarly, below is the rule for variable assignment:

$$\langle c, \sigma\rangle[x = i\,;] \to \langle c, \sigma[i/x]\rangle[\{\}] \quad \text{if } \sigma(x) \neq \bot$$

Note that in this case both the context and the redex were changed by the rule. In fact, as discussed in Section 3.10, one of the major benefits of reduction semantics with evaluation contexts consists in precisely the fact that one can arbitrarily modify the evaluation context in rules; this is crucial for giving semantics to control-intensive language constructs such as call/cc.

Splitting of a term into an evaluation context and a redex does not necessarily need to take place at the top of the left-hand-side of a rule. For example, the following is an alternative way to give reduction semantics with evaluation contexts to variable lookup and assignment:

$$\langle c[x], \sigma\rangle \to \langle c[\sigma(x)], \sigma\rangle \quad \text{if } \sigma(x) \neq \bot$$
$$\langle c[x = i\,;], \sigma\rangle \to \langle c[\{\}], \sigma[i/x]\rangle \quad \text{if } \sigma(x) \neq \bot$$

Note that, even if we follow this alternative style, we still need to include the production *Context* ::= $\langle$*Context*, *State*$\rangle$ to the evaluation context CFG if we want to write rules as $c[i_1 <= i_2] \to c[i_1 \leq_{Int} i_2]$ or to further take advantage of the characteristic rule and write elegant and compact rules such as $i_1 <= i_2 \to i_1 \leq_{Int} i_2$. If we want to completely drop evaluation context productions that mix syntactic and semantic components, such as *Context* ::= $\langle$*Context*, *State*$\rangle$, then we may adopt one of the styles discussed in Exercises 127 and 128, respectively, though one should be aware of the fact that those styles also have their disadvantages.

Figure 3.29 shows a reduction sequence using the evaluation contexts and the rules discussed so far. We used the following (rather standard) notation for instantiated contexts whenever we applied the characteristic rule: the redex is placed in a box replacing the hole of the context. For example, the fact that expression $3 <= x$ is split into contextual representation $(3 <= \square)[x]$ is written compactly and intuitively as $3 <= \boxed{x}$. Note that the evaluation context changes almost at each step during the reduction sequence in Figure 3.29.

Like in small-step SOS and MSOS, we can also transitively and reflexively close the one-step transition relation $\to$. As usual, we let $\to^\star$ denote the resulting multi-step transition relation.

3.7.1 The Reduction Semantics with Evaluation Contexts of IMP

Figure 3.30 shows the definition of evaluation contexts for IMP and Figure 3.31 shows all the reduction semantics rules of IMP using the evaluation contexts defined in Figure 3.30. The evaluation context productions capture the intended evaluation strategies of the various language constructs. For example, `+` and `/` are non-deterministically strict, so any one of their arguments can be reduced one step whenever the sum or the division expression can be reduced one step, respectively, so the hole $\square$ can go in any of their two subexpressions. As previously discussed, in the case of `<=` one can reduce its second argument only after its first argument is fully reduced (to an integer). The evaluation strategy of `!` is straightforward. For `&&`, note

$$
\begin{array}{ll}
 & \langle \boxed{x = 1;}\ y = 2;\ \texttt{if}\,(x <= y)\,\{\,x = 0;\,\}\,\texttt{else}\,\{\,y = 0;\,\},\ (x \mapsto 0, y \mapsto 0)\rangle \\
\to & \langle \boxed{\{\}\ y = 2;}\ \texttt{if}\,(x <= y)\,\{\,x = 0;\,\}\,\texttt{else}\,\{\,y = 0;\,\},\ (x \mapsto 1, y \mapsto 0)\rangle \\
\to & \langle \boxed{y = 2;}\ \texttt{if}\,(x <= y)\,\{\,x = 0;\,\}\,\texttt{else}\,\{\,y = 0;\,\},\ (x \mapsto 1, y \mapsto 0)\rangle \\
\to & \langle \{\}\ \texttt{if}\,(x <= y)\,\{\,x = 0;\,\}\,\texttt{else}\,\{\,y = 0;\,\},\ (x \mapsto 1, y \mapsto 2)\rangle \\
\to & \langle \texttt{if}\,(\boxed{x} <= y)\,\{\,x = 0;\,\}\,\texttt{else}\,\{\,y = 0;\,\},\ (x \mapsto 1, y \mapsto 2)\rangle \\
\to & \langle \texttt{if}\,(1 <= \boxed{y})\,\{\,x = 0;\,\}\,\texttt{else}\,\{\,y = 0;\,\},\ (x \mapsto 1, y \mapsto 2)\rangle \\
\to & \langle \texttt{if}\,(\boxed{1 <= 2})\,\{\,x = 0;\,\}\,\texttt{else}\,\{\,y = 0;\,\},\ (x \mapsto 1, y \mapsto 2)\rangle \\
\to & \langle \texttt{if}\,(\texttt{true})\,\{\,x = 0;\,\}\,\texttt{else}\,\{\,y = 0;\,\},\ (x \mapsto 1, y \mapsto 2)\rangle \\
\to & \langle x = 0;,\ (x \mapsto 1, y \mapsto 2)\rangle \\
\to & \langle \{\},\ (x \mapsto 0, y \mapsto 2)\rangle
\end{array}
$$

Figure 3.29: Sample reduction sequence.

IMP evaluation contexts syntax	IMP language syntax
$\mathit{Context} ::= \square$	$\mathit{AExp} ::= \mathit{Int} \mid \mathit{Id}$
$\mid\ \mathit{Context} + \mathit{AExp} \mid \mathit{AExp} + \mathit{Context}$	$\mid\ \mathit{AExp} + \mathit{AExp}$
$\mid\ \mathit{Context}\ /\ \mathit{AExp} \mid \mathit{AExp}\ /\ \mathit{Context}$	$\mid\ \mathit{AExp}\ /\ \mathit{AExp}$
	$\mathit{BExp} ::= \mathit{Bool}$
$\mid\ \mathit{Context} <= \mathit{AExp} \mid \mathit{Int} <= \mathit{Context}$	$\mid\ \mathit{AExp} <= \mathit{AExp}$
$\mid\ !\ \mathit{Context}$	$\mid\ !\ \mathit{BExp}$
$\mid\ \mathit{Context}\ \&\&\ \mathit{BExp}$	$\mid\ \mathit{BExp}\ \&\&\ \mathit{BExp}$
	$\mathit{Block} ::= \{\} \mid \{\,\mathit{Stmt}\,\}$
	$\mathit{Stmt} ::= \mathit{Block}$
$\mid\ \mathit{Id} = \mathit{Context};$	$\mid\ \mathit{Id} = \mathit{AExp};$
$\mid\ \mathit{Context}\ \mathit{Stmt}$	$\mid\ \mathit{Stmt}\ \mathit{Stmt}$
$\mid\ \texttt{if}\,(\mathit{Context})\ \mathit{Stmt}\ \texttt{else}\ \mathit{Stmt}$	$\mid\ \texttt{if}\,(\mathit{BExp})\ \mathit{Block}\ \texttt{else}\ \mathit{Block}$
	$\mid\ \texttt{while}\,(\mathit{BExp})\ \mathit{Block}$
	$\mathit{Pgm} ::= \texttt{int}\ \mathbf{List}\{\mathit{Id}\};\ \mathit{Stmt}$

Figure 3.30: Evaluation contexts for IMP (left column); the syntax of IMP (from Figure 3.1) is recalled in the right column only for reader's convenience, to more easily compare the two grammars.

$$\mathit{Context} ::= \ldots \mid \langle \mathit{Context}, \mathit{State} \rangle$$

$$\frac{e \to e'}{c[e] \to c[e']}$$

$$\begin{array}{rcl}
\langle c, \sigma \rangle [x] & \to & \langle c, \sigma \rangle [\sigma(x)] \quad \text{if } \sigma(x) \neq \bot \\
i_1 + i_2 & \to & i_1 +_{Int} i_2 \\
i_1 \ \texttt{/} \ i_2 & \to & i_1 \ /_{Int} \ i_2 \quad \text{if } i_2 \neq 0 \\
i_1 \ \texttt{<=} \ i_2 & \to & i_1 \leq_{Int} i_2 \\
\texttt{!}\ \texttt{true} & \to & \texttt{false} \\
\texttt{!}\ \texttt{false} & \to & \texttt{true} \\
\texttt{true} \ \texttt{\&\&} \ b_2 & \to & b_2 \\
\texttt{false} \ \texttt{\&\&} \ b_2 & \to & \texttt{false} \\
\{\, s \,\} & \to & s \\
\langle c, \sigma \rangle [x = i\texttt{;}] & \to & \langle c, \sigma[i/x] \rangle [\{\}] \quad \text{if } \sigma(x) \neq \bot \\
\{\} \ s_2 & \to & s_2 \\
\texttt{if}\,\texttt{(true)}\ s_1\ \texttt{else}\ s_2 & \to & s_1 \\
\texttt{if}\,\texttt{(false)}\ s_1\ \texttt{else}\ s_2 & \to & s_2 \\
\texttt{while}\,\texttt{(}b\texttt{)}\ s & \to & \texttt{if}\,\texttt{(}b\texttt{)}\,\{\, s\ \texttt{while}\,\texttt{(}b\texttt{)}\ s \,\}\,\texttt{else}\,\{\} \\
\langle \texttt{int}\ xl\texttt{;}\ s \rangle & \to & \langle s, (xl \mapsto 0) \rangle
\end{array}$$

Figure 3.31: RSEC(IMP): The reduction semantics with evaluation contexts of IMP ($e, e' \in \mathit{AExp} \cup \mathit{BExp} \cup \mathit{Stmt}$; $c \in \mathit{Context}$ appropriate (that is, the respective terms involving c are well-formed); $i, i_1, i_2 \in \mathit{Int}$; $x \in \mathit{Id}$; $b, b_2 \in \mathit{BExp}$; $s, s_1, s_2 \in \mathit{Stmt}$; $xl \in \mathbf{List}\{\mathit{Id}\}$; $\sigma \in \mathit{State}$).

that only its first argument is reduced. Indeed, recall that `&&` has a short-circuited semantics, so its second argument is reduced only after the first one is completely reduced (to a Boolean) and only if needed; this is defined using rules in Figure 3.31. The evaluation contexts for assignment, sequential composition, and the conditional have already been discussed.

Many of the rules in Figure 3.31 have already been discussed or are trivial. Note that there is no production *Context* ::= `while` (*Context*) *Stmt* as a hasty reader may (mistakenly) expect. That is because such a production would allow the evaluation of the Boolean expression in the while loop's condition to a Boolean value in the current context; supposing that value is `true`, then, unless one modifies the syntax in some rather awkward way, we cannot recover the original Boolean expression to evaluate it again after the evaluation of the while loop's body statement. The solution to handle loops remains the same as in SOS, namely to explicitly unroll them into conditional statements, as shown in Figure 3.31. Note that the evaluation contexts allow the loop unrolling to only happen when the `while` statement is a redex. In particular, after an unrolling reduction takes place, subsequent unrolling steps are disallowed inside the `then` branch; to unroll it again, the loop statement must become again a redex, which can only happen after the conditional statement is itself reduced. The initial program configuration (containing only the program) is reduced also like in SOS (last rule in Figure 3.31; note that one cannot instead define a production *Context* ::= `int` **List{***Id***}**`;` *Context*, because, for example, there is no way to reduce "x = 5;" in $\langle \texttt{int}\ x\texttt{;}\ \boxed{x = 5\texttt{;}} \rangle$.

3.7.2 The Reduction Semantics with Evaluation Contexts of IMP++

We next discuss the reduction semantics of IMP++ using evaluation contexts. Like for the other semantics, we first add each feature separately to IMP and then we add all of them together and investigate the modularity and appropriateness of the resulting definition.

Variable Increment

The use of evaluation contexts makes the definition of variable increment quite elegant and modular:

$$\langle c, \sigma\rangle[{++}\,x] \quad \rightarrow \quad \langle c, \sigma[(\sigma(x) +_{Int} 1)/x]\rangle[\sigma(x) +_{Int} 1]$$

No other rule needs to change, because: (1) unlike in SOS, all the language-specific rules are unconditional, each rule matching and modifying only its relevant part of the configuration; and (2) the language-independent characteristic rule allows reductions to match and modify only their relevant part of the configuration, propagating everything else in the configuration automatically.

Input/Output

We need to first change the configuration employed by our reduction semantics with evaluation contexts of IMP from $\langle s, \sigma\rangle$ to $\langle s, \sigma, \omega_{in}, \omega_{out}\rangle$, to also include the input/output buffers. This change, unfortunately, generates several other changes in the existing semantics, some of them non-modular in nature. First, we need to change the syntax of (statement) configurations to include input and output buffers, and the syntax of contexts from *Context* ::= ... | $\langle$*Context*, *State*$\rangle$ to *Context* ::= ... | $\langle$*Context*, *State*, *Buffer*, *Buffer*$\rangle$. No matter what semantic approach one employs, some changes in the configuration (or its equivalent) are unavoidable when one adds new language features that require new semantic data, like input/output constructs that require their own buffers (recall that, e.g., in MSOS, we had to add new attributes in transition labels instead). Hence, this change is acceptable. What is inconvenient (and non-modular), however, is that the rules for variable lookup and for assignment need the complete configuration, so they have to change from

$$\begin{array}{rcl} \langle c, \sigma\rangle[x] & \rightarrow & \langle c, \sigma\rangle[\sigma(x)] \\ \langle c, \sigma\rangle[x = i\,;] & \rightarrow & \langle c, \sigma[i/x]\rangle[\{\}] \end{array}$$

to

$$\begin{array}{rcl} \langle c, \sigma, \omega_{in}, \omega_{out}\rangle[x] & \rightarrow & \langle c, \sigma, \omega_{in}, \omega_{out}\rangle[\sigma(x)] \\ \langle c, \sigma, \omega_{in}, \omega_{out}\rangle[x = i\,;] & \rightarrow & \langle c, \sigma[i/x], \omega_{in}, \omega_{out}\rangle[\{\}] \end{array}$$

Also, the initial configuration which previously held only the program, now has to change to hold both the program and an input buffer, and the rule for programs (the last in Figure 3.31) needs to change as follows:

$$\langle \texttt{int}\, xl\texttt{;}\ s, \omega_{in}\rangle \rightarrow \langle s, (xl \mapsto 0), \omega_{in}, \epsilon\rangle$$

Once the changes above are applied, we are ready for adding the evaluation context for the print statement as well as the reduction semantics rules of both input/output constructs:

$$\begin{array}{rcl} \textit{Context} & ::= & \ldots \mid \texttt{print(}\textit{Context}\texttt{);} \\ \langle c, \sigma, i : \omega_{in}, \omega_{out}\rangle[\texttt{read()}] & \rightarrow & \langle c, \sigma, \omega_{in}, \omega_{out}\rangle[i] \\ \langle c, \sigma, \omega_{in}, \omega_{out}\rangle[\texttt{print(}i\texttt{);}] & \rightarrow & \langle c, \sigma, \omega_{in}, \omega_{out} : i\rangle[\{\}] \end{array}$$

Other possibilities to add input/output buffers to the configuration and to give the reduction semantics with evaluation contexts of the above language features in a way that appears to be more modular (but which yields other problems) are discussed in Section 3.10.

Abrupt Termination

For our language, reduction semantics allows very elegant, natural and modular definitions of abrupt termination, without having to extent the syntax of the original language and without adding any new reduction steps as an artifact of the approach chosen:

$$\begin{array}{rcl} \langle c, \sigma \rangle [i \ / \ o] & \rightarrow & \langle \{\}, \sigma \rangle \\ \langle c, \sigma \rangle [\texttt{halt;}] & \rightarrow & \langle \{\}, \sigma \rangle \end{array}$$

Therefore, the particular evaluation context in which the abrupt termination is being generated, c, is simply discarded. This is not possible in any of the big-step, small-step or MSOS styles above, because in there the evaluation context c is captured by the proof context, which, like in any logical system, cannot be simply discarded. The elegance of the two rules above suggests that having the possibility to explicitly match and change the evaluation context is a very powerful and convenient feature of a language semantic framework.

Dynamic Threads

The rules (SmallStep-Spawn-Arg) (resp. (MSOS-Spawn-Arg)) and (SmallStep-Spawn-Wait) (resp. (MSOS-Spawn-Wait)) in Section 3.5.4 (resp. Section 3.6.2) are essentially computation propagation rules. In reduction semantics with evaluation contexts the role of such rules is taken over by the splitting/plugging mechanism, which in turn relies on parsing and therefore needs productions for evaluation contexts. We can therefore replace those rules by appropriate productions for evaluation contexts:

$$\begin{array}{rcl} \mathit{Context} & ::= & \ldots \\ & | & \texttt{spawn}\ \mathit{Context} \\ & | & \texttt{spawn}\ \mathit{Stmt}\ \mathit{Context} \end{array}$$

The second evaluation context production above involves two language constructs, namely `spawn` and sequential composition. The desired non-determinism due to concurrency is captured by deliberate ambiguity in parsing evaluation contexts and, implicitly, in the splitting/plugging mechanism.

The remaining rule (SmallStep-Spawn-Skip) (resp. (MSOS-Spawn-Skip)) in Section 3.5.4 (resp. Section 3.6.2) is turned into an equivalent rule here, and the structural identity stating the associativity of sequential composition and the syntactic extension of `spawn` to take statements (instead of blocks) are also still necessary:

$$\begin{array}{rcl} \texttt{spawn}\ \{\} & \rightarrow & \{\} \\ (s_1\ s_2)\ s_3 & \equiv & s_1\ (s_2\ s_3) \\ \mathit{Stmt} & ::= & |\ \texttt{spawn}\ \mathit{Stmt} \end{array}$$

Local Variables

We make use of the procedure presented in Section 3.5.5 for desugaring blocks with local variables into `let` constructs, to reduce the problem to only give semantics to `let`. Recall from Section 3.5.5 that the semantics of `let` $x = a$ `in` s in a state σ is to first evaluate arithmetic expression a in σ to some integer i and then evaluate statement s in state $\sigma[i/x]$; after the evaluation of s, the value of x is recovered to $\sigma(x)$ (i.e., whatever it was before the execution of the block) but all the other state updates produced by the evaluation of s are kept. These suggest the following:

$$\mathit{Context} \quad ::= \quad \ldots \ |\ \texttt{let}\ \mathit{Id} = \mathit{Context}\ \texttt{in}\ \mathit{Stmt}$$

$$\langle c, \sigma \rangle [\texttt{let}\ x = i\ \texttt{in}\ s] \quad \rightarrow \quad \langle c, \sigma[i/x] \rangle [s\ \ x = \sigma(x)\texttt{;}]$$

Notice that a solution similar to that in small-step SOS and MSOS (see rules (SmallStep-Let-Stmt) and (MSOS-Let-Stmt) in Sections 3.5.5 and 3.6.2, respectively) does not work here, because rules in reduction semantics with evaluation contexts are unconditional. In fact, a solution similar to the one we adopted above was already discussed in Section 3.5.6 in the context of small-step SOS, where we also explained that it works as shown because of a syntactic trick, namely because we allow assignment statements of the form "$x = \bot$;" (in our case here when $\sigma(x) = \bot$), which have the effect to undefine x in σ (see Section 2.4.6). Section 3.5.6 also gives suggestions on how to avoid allowing $\bot$ to be assigned to x, if one does not like it. One suggestion was to rename the bound variable into a fresh one, this way relieving us from having to recover its value after the `let`:

$$\langle c, \sigma\rangle[\texttt{let}\ x = i\ \texttt{in}\ s] \to \langle c, \sigma[i/x']\rangle[s[x'/x]] \quad \text{if } x' \text{ is a fresh variable}$$

This approach comes with several problems, though: it requires that we define and maintain a substitution operation (for $s[x'/x]$), we have to pay a complexity linear with the size of the `let` body each time a `let` statement is eliminated, and the state may grow indefinitely (since the `let` can be inside a loop).

Note also that, with the current reduction semantics with evaluation contexts of IMP, we cannot add the following evaluation context production

$$\textit{Context} \quad ::= \quad \ldots \quad | \quad \texttt{let}\ \textit{Id} = \textit{Int}\ \texttt{in}\ \textit{Context}$$

stating that once the binding expression becomes an integer then we can evaluate the `let` body. We cannot add it simply because we have to bind the variable to the integer before we evaluate the `let` body statement. An evaluation context production like above would result in evaluating the `let` body statement in the same state as before the `let`, which is obviously wrong. However, the existence of a `let` binder allows us to possibly rethink the overall reduction semantics of IMP, to make it more syntactic. Indeed, the `let` binders can be used in a nested manner and hereby allow us to syntactically mimic a state. For example, a statement of the form `let` $x = 5$ `in let` $y = 7$ `in` s can be thought of as the statement s being executed in a "state" where x is bound to 5 and where y is bound to 7. We can then drop the configurations of IMP completely and instead add the evaluation context above allowing reductions inside `let` body statements. The IMP rules that refer to configurations, namely those for lookup and assignment, need to change to work with the new "state", and a new rule to eliminate unnecessary `let` statements needs to be added:

$$\begin{array}{rcl} \texttt{let}\ x = i\ \texttt{in}\ c[x] & \to & \texttt{let}\ x = i\ \texttt{in}\ c[i] \\ \texttt{let}\ x = i\ \texttt{in}\ c[x = j;] & \to & \texttt{let}\ x = j\ \texttt{in}\ c[\{\}] \\ \texttt{let}\ x = i\ \texttt{in}\ \{\} & \to & \{\} \end{array}$$

The above works correctly only if one ensures that the evaluation contexts c do not contain other `let` $x =$ _ `in` _ evaluation context constructs, with the same x variable name as in the rules (the underscores can be any integer and context, respectively). One can do this by statically renaming the bound variables to have different names at parse-time, or by employing a substitution operation to do it dynamically. Note that the static renaming approach requires extra-care as the language is extended, particularly if new `let` statements can be generated dynamically by other language constructs. Another approach to ensure the correct application of the rules above, which is theoretically more complex and practically more expensive and harder to implement, is to add a side condition to the first two rules of the form "where c does not contain any evaluation context production instance of the form `let` $x =$ _ `in` _".

To conclude, the discussion above suggests that there are various ways to give a reduction semantics of `let` using evaluation contexts, none of them absolutely better than the others: some are simpler, others

are more syntactic but require special external support, others require non-modular changes to the existing language. The approaches above are by no means exhaustive. For example, we have not even discussed environment-store based approaches.

Putting Them All Together

It is relatively easy to combine all the reduction semantics with evaluation contexts of all the features above, although not as modularly as it was for MSOS. Specifically, we have to do the following:

1. Apply all the changes that we applied when we added input/output to IMP above, namely: add input/output buffers to both configurations and configuration evaluation contexts; change the semantic rules involving configurations to work with the extended configurations, more precisely the rules for variable lookup, for variable assignment, and for programs.

2. Add all the evaluation contexts and the rules for the individual features above, making sure we *change* those of them using configurations or configuration evaluation contexts (i.e., almost all of them) to work with the new configurations including input/output buffers.

Unfortunately, the above is not giving us the desired language. Worse, it actually gives us a wrong language, namely one with a disastrous feature interaction. This problem has already been noted in Section 3.5.6, where we discussed the effect of a similar semantics to that of `let` above, but using small-step SOS instead of evaluation contexts. The problem here is that, if the body of a `let` statement contains a `spawn` statement, then the latter will be allowed, according to its semantics, to be executed in parallel with the statements following it. In our case, the assignment statement $x = \sigma(x)$`;` in the `let` semantics, originally intended to recover the value of x, can be now potentially executed before the spawned statement, resulting in a wrong behavior; in particular, the assignment can even "undefine" x in case $\sigma(x) = \bot$, in which case the `spawn` statement can even get stuck.

As already indicated in Section 3.5.6, the correct way to eliminate the `let` construct is to rename the bound variable into a fresh variable visible only to `let`'s body statement, this way eliminating the need to recover the value of the bound variable to what it was before the `let`:

$$\langle c, \sigma, \omega_{in}, \omega_{out}\rangle[\texttt{let}\ x = i\ \texttt{in}\ s] \to \langle c, \sigma[i/x'], \omega_{in}, \omega_{out}\rangle[s[x'/x]] \quad \text{if } x' \text{ is a fresh variable}$$

We have used configurations already extended with input/output buffers, as needed for IMP++. This solution completely brakes any (intended or unintended) relationship between the `let` construct and any other language constructs that may be used inside its body, although, as discussed in Section 3.5.6, the use of the substitution comes with a few (relatively acceptable) drawbacks.

3.7.3 Reduction Semantics with Evaluation Contexts in Rewrite Logic

In this section we show how to automatically and faithfully embed reduction semantics with evaluation contexts into rewrite logic. After discussing how to embed evaluation contexts into rewrite logic, we first give a straightforward embedding of reduction semantics, which is easy to prove correct but which does not take advantage of performance-improving techniques currently supported by rewrite engines, so consequently it is relatively inefficient when executed or formally analyzed. We then discuss simple optimizations which increase the performance of the resulting rewrite definitions an order of magnitude or more. We only consider evaluation contexts which can be defined by means of context-free grammars (CFGs). However, the CFG that we allow for defining evaluation contexts can be non-deterministic, in the sense that a term is allowed to split many different ways into a context and a redex (like the CFG in Figure 3.30).

sort:
Syntax // includes all syntactic terms, in contextual representation *context*[*redex*] or not
subsorts:
$N_1, N_2, \ldots < Syntax$ // $N_1, N_2, \ldots,$ are sorts whose terms can be regarded as *context*[*redex*]
operations:
$_[_] : Context \times Syntax \to Syntax$ // constructor for terms in contextual representation
$split : Syntax \to Syntax$ // puts syntactic terms into contextual representation
$plug : Syntax \to Syntax$ // the dual of split
rules and equations:
$split(Syn) \to \Box[Syn]$ // generic rule; it initiates the splitting process for the rules below
$plug(\Box[Syn]) = Syn$ // generic equation; it terminates the plugging process
// for each context production $Context ::= \pi(N_1, \ldots, N_n, Context)$ add the following:
$split(\pi(T_1, \ldots, T_n, T)) \to \pi(T_1, \ldots, T_n, C)[Syn]$ **if** $split(T) \to C[Syn]$
$plug(\pi(T_1, \ldots, T_n, C)[Syn]) = \pi(T_1, \ldots, T_n, plug(C[Syn]))$

Figure 3.32: Embedding evaluation contexts into rewrite logic theory $\mathcal{R}^{\Box}_{\text{RSEC}}$. The implicit split/plug mechanism is replaced by explicit rewrite logic sentences achieving the same task (the involved variables have the sorts $Syn : Syntax$, $C : Context$, $T_1 : N_1, \ldots, T_n : N_n$, and $T : N$).

Faithful Embedding of Evaluation Contexts into Rewrite Logic

Our approach to embedding reduction semantics with evaluation contexts in rewrite logic builds on an embedding of evaluation contexts and their implicit splitting/plugging mechanism in rewrite logic. More precisely, each evaluation context production is associated with an equation (for plugging) and a conditional rewrite rule (for splitting). The conditional rewrite rules allow to non-deterministically split a term into a context and a redex. Moreover, when executing the resulting rewrite logic theory, the conditional rules allow for finding *all* splits of a term into a context and a redex, provided that the underlying rewrite engine has search capabilities (like Maude does).

Figure 3.32 shows a general and automatic procedure to generate a rewrite logic theory from any CFG defining evaluation contexts for some given language syntax. Recall that, for simplicity, in this section we assume only one *Context* syntactic category. What links the CFG of evaluation contexts to the CFG of the language to be given a semantics, which is also what makes our embedding into rewrite logic discussed here work, is the assumption that for any context production

$$Context \quad ::= \quad \pi(N_1, \ldots, N_n, Context)$$

there are some syntactic categories N, N' (different or not) in the language CFG (possibly extended with configurations and semantic components as discussed above) such that $\pi(t_1, \ldots, t_n, t) \in N'$ for any $t_1 \in N_1$, $\ldots$, $t_n \in N_n$, $t \in N$. We here used a notation which needs to be explained. The actual production above is $Context ::= \pi$, where π is a string of terminals and non-terminals, but we write $\pi(N_1, \ldots, N_n, Context)$ instead of π to emphasize that $N_1, \ldots, N_n, Context$ are all the non-terminals appearing in π; we listed the *Context* last for simplicity. Also, by abuse of notation, we let $\pi(t_1, \ldots, t_n, t)$ denote the term obtained by substituting $(t_1, \ldots, t_n, t)$ for $(N_1, \ldots, N_n, Context)$ in π, respectively. So our assumption is that $\pi(t_1, \ldots, t_n, t)$ is well-formed under the syntax of the language whenever $t_1, \ldots, t_n, t$ are well-defined in the appropriate syntactic categories, that is, $t_1 \in N_1, \ldots, t_n \in N_n, t \in T$. This is indeed a very natural property of well-defined evaluation contexts for a given language, so natural that one may even ask how it can be otherwise (it is

easy to violate this property though, e.g., *Context* ::= <= *Context* = *AExp* ;). Without this property, our embedding of evaluation contexts in rewrite logic in Figure 3.32 would not be well-formed, because the left-hand-side terms of some of the conditional rule(s) for split would not be well-formed terms.

For simplicity, in Figure 3.32 we prefer to subsort all the syntactic categories whose terms are intended to be allowed contextual representations *context*[*redex*] under one top sort[9], *Syntax*. The implicit notation *context*[*term*] for contextual representations, as well as the implicitly assumed *split* and *plug* operations, are defined explicitly in the corresponding rewrite theory. The split operation is only defined on terms over the original language syntax, while the plug operation is defined only over terms in contextual representation. One generic rule and one generic equation are added: $split(Syn) \rightarrow \square[Syn]$ initiates the process of splitting a term into a contextual representation and $plug(\square[Syn]) = Syn$ terminates the process of plugging a term into a context. It is important that the first be a rewrite rule (because it can lead to non-determinism; this is explained below), while the second can safely be an equation.

Each evaluation context production translates into one equation and one conditional rewrite rule. The equation tells how terms are plugged into contexts formed with that production, while the conditional rule tells how that production can be used to split a term into a context and a redex. The equations defining plugging are straightforward: for each production in the original CFG of evaluation contexts, iteratively plug the subterm in the smaller context; when the hole is reached, replace it by the subterm via the generic equation. The conditional rules for splitting also look straightforward, but how and why they work is more subtle. For any context production, if the term to split matches the pattern of the production, then first split the subterm corresponding to the position of the subcontext and then use that contextual representation of the subterm to construct the contextual representation of the original term; at any moment, one has the option to stop splitting thanks to the generic rule $split(Syn) \rightarrow \square[Syn]$. For example, for the five *Context* productions in the evaluation context CFG in the preamble of this section, namely

$$
\begin{array}{lcl}
\mathit{Context} & ::= & \mathit{Context} \texttt{ <= } \mathit{AExp} \\
 & | & \mathit{Int} \texttt{ <= } \mathit{Context} \\
 & | & \mathit{Id} \texttt{ = } \mathit{Context} \texttt{;} \\
 & | & \mathit{Context}\ \mathit{Stmt} \\
 & | & \texttt{if (}\mathit{Context}\texttt{)}\ \mathit{Stmt}\ \texttt{else}\ \mathit{Stmt}
\end{array}
$$

the general procedure in the rewrite logic embedding of evaluation contexts in Figure 3.32 yields the following five rules and five equations (variable I_1 has sort *Int*; X has sort *Id*; A_1 and A_2 have sort *AExp*; B has sort

[9] An alternative, which does not involve subsorting, is to rename all syntactic categories into one, *Syntax*. Our construction also works without subsorting and without collapsing of syntactic categories, but it is more technical, requires more operations, rules, and equations, and it is likely not worth the effort without a real motivation to use it in term rewrite settings without support for subsorting. We have made experiments with both approaches and found no penalty on performance when collapsing syntactic categories.

BExp; S_1 and S_2 have sort *Stmt*; *C* has sort *Context*; *Syn* has sort *Syntax*):

$$split(A_1 <= A_2) \to (C <= A_2)[Syn] \ \textbf{if} \ split(A_1) \to C[Syn]$$
$$plug((C <= A_2)[Syn]) = plug(C[Syn]) <= A_2$$

$$split(I_1 <= A_2) \to (I_1 <= C)[Syn] \ \textbf{if} \ split(A_2) \to C[Syn]$$
$$plug((I_1 <= C)[Syn]) = I_1 <= plug(C[Syn])$$

$$split(X = A\,\texttt{;}) \to (X = C\,\texttt{;})[Syn] \ \textbf{if} \ split(A) \to C[Syn]$$
$$plug((X = C\,\texttt{;})[Syn]) = (X = plug(C[Syn])\,\texttt{;})$$

$$split(S_1\ S_2) \to (C\ S_2)[Syn] \ \textbf{if} \ split(S_1) \to C[Syn]$$
$$plug((C\ S_2)[Syn]) = plug(C[Syn])\ S_2$$

$$split(\texttt{if}\,(B)\ S_1\ \texttt{else}\ S_2) \to (\texttt{if}\,(C)\ S_1\ \texttt{else}\ S_2)[Syn] \ \textbf{if} \ split(B) \to C[Syn]$$
$$plug((\texttt{if}\,(C)\ S_1\ \texttt{else}\ S_2)[Syn]) = \texttt{if}\,(plug(C[Syn]))\ S_1\ \texttt{else}\ S_2$$

The reason for which we used rules instead of equations for splitting in our embedding in Figure 3.32 is that splitting, unlike plugging, can be non-deterministic. Recall that the use of an arrow/transition in the condition of a rule has an existential nature (Sections 2.5). In particular, rewrite logic engines should execute conditional rules by performing an exhaustive search, or reachability analysis of all the zero-, one- or more-step rewrites of the condition left-hand-side term (*split*(*T*) in our case) into the condition right-hand-side term (*C*[*Syn*] in our case). Such an exhaustive search explores all possible splits, or parsings, of a term into a contextual representation.

Theorem 17. ***(Embedding splitting/plugging into rewrite logic)*** *Given an evaluation context CFG as discussed above, say as part of some reduction semantics with evaluation contexts definition* RSEC*, let* $\mathcal{R}^{\square}_{\text{RSEC}}$ *be the rewrite logic theory associated to it as in Figure 3.32. Then the following are equivalent for any* $t, r \in$ *Syntax and* $c \in$ *Context:*

- *t can be split as* $c[r]$ *using the evaluation context CFG of* RSEC*;*
- $\mathcal{R}^{\square}_{\text{RSEC}} \vdash split(t) \to c[r]$*;*
- $\mathcal{R}^{\square}_{\text{RSEC}} \vdash plug(c[r]) = t$.

The theorem above says that the process of splitting a term *t* into a context and a redex in reduction semantics with evaluation contexts, which can be non-deterministic, reduces to reachability in the corresponding rewrite logic theory of a contextual representation pattern $c[r]$ of the original term marked for splitting, *split*(*t*). Rewrite engines such as Maude provide a search command that does precisely that. We will shortly see how Maude's search command can find all splits of a term.

Faithful Embedding of RSEC in Rewriting Logic

In this section we discuss three faithful rewrite logic embeddings of reduction semantics with evaluation contexts. The first two assume that the embedded reduction semantics has no characteristic rule, in that all reductions take place at the top of the original term to reduce (e.g., a configuration in the case of our IMP language); this is not a limitation because, as already discussed, the characteristic rule can be regarded as syntactic sugar anyway, its role being to allow one to write reduction semantics definitions more compactly and elegantly. The first embedding is the simplest and easiest to prove correct, but it is the heaviest in notation

rules:
// for each reduction semantics rule $l(c_1[l_1], \ldots, c_n[l_n]) \to r(c'_1[r_1], \ldots, c'_{n'}[r_{n'}])$
// add the following conditional semantic rewrite rule:

$$\circ\, \overline{l}(T_1, \ldots, T_n) \to \overline{r}(plug(\overline{c'_1}[\overline{r_1}]), \ldots, plug(\overline{c'_n}[\overline{r_{n'}}])) \ \textbf{if}\ split(T_1) \to \overline{c_1}[\overline{l_1}] \wedge \ \ldots \ \wedge split(T_n) \to \overline{c_n}[\overline{l_n}]$$

Figure 3.33: First embedding of RSEC into rewrite logic (RSEC $\rightsquigarrow \mathcal{R}^{\boxed{1}}_{\text{RSEC}}$).

and the resulting rewrite theories tend to be inefficient when executed because most of the left-hand-side terms of rules end up being identical, thus making the task of matching and selecting a rule to apply rather complex for rewrite engines. The second embedding results in rewrite rules whose left-hand-sides are mostly distinct, thus taking advantage of current strengths of rewrite engines to index terms so that rules to be applied can be searched for quickly. Our third embedding is as close to the original reduction semantics in form and shape as one can hope it to be in a rewriting setting; in particular, it also defines a characteristic rule, which can be used to write a more compact semantics. The third embedding yields rewrite theories which are as efficient as those produced by the second embedding. The reason we did not define directly the third embedding is because we believe that the transition from the first to the second and then to the third is instructive.

Since reduction semantics with evaluation contexts is an inherently small-step semantical approach, we use the same mechanism to control the rewriting as for small-step SOS (Section 3.3) and MSOS (Section 3.6). This mechanism was discussed in detail in Section 3.3.3. It essentially consists of: (1) tagging each left-hand-side term appearing in a rule transition with a $\circ$, to capture the desired notion of a one-step reduction of that term; and (2) tagging with a $\star$ the terms to be multi-step (zero, one or more steps) reduced, where $\star$ can be easily defined with a conditional rule as the transitive and reflexive closure of $\circ$ (see Section 3.3.3).

Figure 3.33 shows our first embedding of reduction semantics with evaluation contexts into rewrite logic, which assumes that the characteristic rule, if any, has already been desugared. Each reduction semantics rule translates into one conditional rewrite rule. We allow the reduction rules to have in their left-hand-side and right-hand-side terms an arbitrary number of subterms that are in contextual representation. For example, if the left-hand-side l of a reduction rule has n such subterms, say $c_1[l_1]$, ..., $c_n[l_n]$, then we write it $l(c_1[l_1], \ldots, c_n[l_n])$ (this is similar with our previous notation $\pi(N_1, \ldots, N_n, N)$ in the section above on embedding of evaluation contexts into rewrite logic, except that we now single out all the subterms in contextual representation instead of all the non-terminals). In particular, a rule $l \to r$ in which l and r contain no subterms in contextual representation (like the last rule in Figure 3.31) is translated exactly like in small-step SOS, that is, into $\overline{l} \to \overline{r}$. Also, note that we allow evaluation contexts to have any pattern (since we overline them, like any other terms); we do not restrict them to only be context variables. Consider, for example, the six reduction rules discussed in the preamble of Section 3.7, which after the desugaring of the characteristic rule are as follows:

$$\begin{aligned}
c[i_1 \texttt{<=} i_2] &\to c[i_1 \leq_{Int} i_2] \\
c[\{\}\ s_2] &\to c[s_2] \\
c[\texttt{if (true)}\ s_1\ \texttt{else}\ s_2] &\to c[s_1] \\
c[\texttt{if (false)}\ s_1\ \texttt{else}\ s_2] &\to c[s_2] \\
\langle c, \sigma\rangle[x] &\to \langle c, \sigma\rangle[\sigma(x)] \quad \text{if } \sigma(x) \neq \bot \\
\langle c, \sigma\rangle[x = i\texttt{;}] &\to \langle c, \sigma[i/x]\rangle[\{\}] \quad \text{if } \sigma(x) \neq \bot
\end{aligned}$$

Since all these rules have left-hand-side terms already in contextual representation, their corresponding l in

Figure 3.33 is just a non-terminal (*Configuration*), which means that $\bar{l}$ is just a variable (of sort *Configuration*). Therefore, the rewrite logic rules associated to these RSEC rules are:

$$\circ\, Cfg \to plug(C[I_1 \leq_{Int} I_2])\ \textbf{if}\ split(Cfg) \to C[I_1 <= I_2]$$
$$\circ\, Cfg \to plug(C[S_2])\ \textbf{if}\ split(Cfg) \to C[\{\}\ S_2]$$
$$\circ\, Cfg \to plug(C[S_1])\ \textbf{if}\ split(Cfg) \to C[\texttt{if (true)}\ S_1\ \texttt{else}\ S_2]$$
$$\circ\, Cfg \to plug(C[S_2])\ \textbf{if}\ split(Cfg) \to C[\texttt{if (false)}\ S_1\ \texttt{else}\ S_2]$$
$$\circ\, Cfg \to plug(\langle C, \sigma\rangle[\sigma(X)])\ \textbf{if}\ split(Cfg) \to \langle C, \sigma\rangle[X]\ \wedge\ \sigma(X) \neq \bot$$
$$\circ\, Cfg \to plug(\langle C, \sigma[I/X]\rangle[\{\}])\ \textbf{if}\ split(Cfg) \to \langle C, \sigma\rangle[X = I\,\texttt{;}]\ \wedge\ \sigma(X) \neq \bot$$

Recall from the preamble of Section 3.7 that the RSEC rules for variable lookup and assignment can also be given as follows, so that their left-hand-side terms are not in contextual representation:

$$\langle c[x], \sigma\rangle \to \langle c[\sigma(x)], \sigma\rangle \quad \text{if } \sigma(x) \neq \bot$$
$$\langle c[x = i\,\texttt{;}], \sigma\rangle \to \langle c[\{\}], \sigma[i/x]\rangle \quad \text{if } \sigma(x) \neq \bot$$

In these cases, the string l in Figure 3.33 has the form $\langle Stmt, \sigma\rangle$, which means that $\bar{l}$ is the term $\langle S, \sigma\rangle$, where S is a variable of sort *Stmt*. Then their corresponding rewrite logic rules are:

$$\circ\, \langle S, \sigma\rangle \to \langle plug(C[\sigma(X)]), \sigma\rangle\ \textbf{if}\ split(S) \to C[X]\ \wedge\ \sigma(X) \neq \bot$$
$$\circ\, \langle S, \sigma\rangle \to \langle plug(C[\{\}]), \sigma[I/X]\rangle\ \textbf{if}\ split(S) \to C[X = I\,\texttt{;}]\ \wedge\ \sigma(X) \neq \bot$$

Once the characteristic rule is desugared as explained in the preamble of Section 3.7, an RSEC rule operates as follows: (1) attempt to match the left-hand-side pattern of the rule at the top of the term to reduce, making sure that each of the subterms corresponding to subpatterns in contextual representation form can indeed be split as indicated; and (2) if the matching step above succeeds, then reduce the original term to the right-hand-side pattern instantiated accordingly, plugging all the subterms appearing in contextual representations in the right-hand-side. Note that the conditional rewrite rule associated to an RSEC rule as indicated in Figure 3.33 achieves precisely the desired steps above: the $\circ$ in the left-hand-side term guarantees that the rewrite step takes place at the top of the original term, the condition exhaustively searches for the desired splits of the subterms in question into contextual representations, and the right-hand-side plugs back all the contextual representations into terms over the original syntax. The only difference between the original RSEC rule and its corresponding rewriting logic conditional rule is that the rewrite logic rule makes explicit the splits and plugs that are implicit in the RSEC rule.

Theorem 18. ***(First faithful embedding of reduction semantics into rewrite logic)*** *Let* RSEC *be any reduction semantics with evaluation contexts definition and let* $\mathcal{R}^{\boxed{1}}_{\text{RSEC}}$ *be the rewrite logic theory associated to* RSEC *using the embedding procedures in Figures 3.32 and 3.33. Then*

1. ***(step-for-step correspondence)*** RSEC $\vdash t \to t'$ *using a reduction semantics with evaluation contexts rule iff* $\mathcal{R}^{\boxed{1}}_{\text{RSEC}} \vdash \circ\,\bar{t} \to^1 \overline{t'}$ *using the corresponding conditional rewrite rule obtained like in Figure 3.33; moreover, the reduction rule and the corresponding rewrite rule apply similarly (same contexts, same substitution; all modulo the correspondence in Theorem 17);*

2. ***(computational correspondence)*** RSEC $\vdash t \to^\star t'$ *iff* $\mathcal{R}^{\boxed{1}}_{\text{RSEC}} \vdash \star\,\bar{t} \to \star\,\overline{t'}$.

The first item in Theorem 18 says that the resulting rewriting logic theory captures faithfully the small-step reduction relation of the original reduction semantics with evaluation contexts definition. The faithfulness

of this embedding (i.e., there is precisely one top-level application of a rewrite rule that corresponds to an application of a reduction semantics rule), comes from the fact that the consistent use of the $\circ$ tag inhibits any other application of any other rule on the tagged term. Therefore, like in small-step SOS and MSOS, a small-step in a reduction semantics definition also reduces to reachability analysis in the corresponding rewrite theory; one can also use the search capability of a system like Maude to find all the next terms that a given term evaluates to (Maude provides the capability to search for the first n terms that match a given pattern using up to m rule applications, where n and m are user-provided parameters).

The step-for-step correspondence above is stronger (and better) than the strong bisimilarity of the two definitions; for example, if a reduction semantics rule in RSEC can be applied in two different ways on a term to reduce, then its corresponding rewrite rule in $\mathcal{R}^{\boxed{1}}_{\text{RSEC}}$ can also be applied in two different ways on the tagged term. The second item in Theorem 18 says that the resulting rewrite theory can be used to perform any computation possible in the original RSEC, and vice versa (the step-for-step correspondence is guaranteed in combination with the first item). Therefore, there is absolutely no difference between computations using RSEC and computations using $\mathcal{R}^{\boxed{1}}_{\text{RSEC}}$, except for irrelevant syntactic conventions/notations. This strong correspondence between reductions in RSEC and rewrites in $\mathcal{R}^{\boxed{1}}_{\text{RSEC}}$ tells that $\mathcal{R}^{\boxed{1}}_{\text{RSEC}}$ *is precisely* RSEC*, not an encoding of it*. In other words, RSEC can be faithfully regarded as a methodological fragment of rewrite logic, same like big-step SOS, small-step SOS, and MSOS.

The discussion above implies that, from a theoretical perspective, the rewrite logic embedding of reduction semantics in Figure 3.33 is as good as one can hope. However, its simplicity comes at a price in performance, which unfortunately tends to be at its worst precisely in the most common cases. Consider, for example, the six rewrite rules used before Theorem 18 to exemplify the embedding in Figure 3.33 (consider the variant for lookup and assignment rules where the contextual representation in the left-hand-side appears at the top—first variant). They all have the form:

$$\circ\, Cfg \rightarrow \ldots \;\; \textbf{if} \;\; split(Cfg) \rightarrow \ldots$$

In fact, as seen in Figure 3.38, all the rewrite rules in the rewrite logic theory corresponding to the RSEC of IMP have the same form. The reason the left-hand-side terms of these rewrite rules are the same and lack any structure is because the contextual representations in the left-hand-side terms of the RSEC rules appear at the top, with no structure above them, which is the most common type of RSEC rule encountered.

To apply a conditional rewrite rule, a rewrite engine first matches the left-hand-side and then performs the (exhaustive) search in the condition. In other words, the structure of the left-hand-side acts as a cheap guard for the expensive search. Unfortunately, since the left-hand-side of the conditional rewrite rules above has no structure, it will always match. That means that the searches in the conditions of all the rewrite rules will be, in the worst case, executed one after another until a split is eventually found (if any). If one thinks in terms of implementing RSEC in general, then this is what a naive implementation would do. If one thinks in terms of executing term rewrite systems, then this fails to take advantage of some important performance-increasing advances in term rewriting, such as *indexing* [72, 73, 2]. In short, indexing techniques use the structure of the left-hand-sides to augment the term structure with information about which rule can potentially be applied at which places. This information is dynamically updated, as the term is rewritten. If the rules' left-hand-sides do not significantly overlap, it is generally assumed that it takes constant time to find a matching rewrite rule. This is similar in spirit to hashing, where the access time into a hash table is generally assumed to take constant time when there are no or few key collisions. Thinking intuitively in terms of hashing, from an indexing perspective a rewrite system with rules having the same left-hand-sides is as bad as a hash table in which all accesses are collisions.

Ideally, in an efficient implementation of RSEC one would like to adapt/modify indexing techniques,

rules:

// for each term l that appears as left-hand-side of a reduction rule
// $l(c_1[l_1], \ldots, c_n[l_n]) \to \ldots$ with $n > 0$, add the following
// conditional rewrite rule (there could be one l for many reduction rules):

$$\circ\, \bar{l}(T_1, \ldots, T_n) \to T \ \textbf{if} \ \circ\, \bar{l}(split(T_1), \ldots, split(T_n)) \to T$$

// for each reduction semantics rule $l(c_1[l_1], \ldots, c_n[l_n]) \to r(c'_1[r_1], \ldots, c'_{n'}[r_{n'}])$
// add the following (unconditional) semantic rewrite rule:

$$\circ\, \bar{l}(\overline{c_1}[\overline{l_1}], \ldots, \overline{c_n}[\overline{l_n}]) \to \bar{r}(plug(\overline{c'_1}[\overline{r_1}]), \ldots, plug(\overline{c'_n}[\overline{r_{n'}}]))$$

Figure 3.34: Second embedding of RSEC into rewrite logic (RSEC $\rightsquigarrow \mathcal{R}^{\boxed{2}}_{\text{RSEC}}$).

which currently work for context-insensitive term rewriting, or to invent new techniques serving the same purpose. This seems highly non-trivial and tedious, though. An alternative is to device embedding transformations of RSEC into rewrite logic that take better or full advantage of existing, context-insensitive indexing. Without context-sensitive indexing or other bookkeeping mechanisms hardwired in the reduction engine, due to the inherent non-determinism in parsing/splitting syntax into contextual representations, in the worst case one needs to search the entire term to find a legal position where a reduction can take place. While there does not seem that we can do much to avoid such an exhaustive search in the worst case, note that our first embedding in Figure 3.33 initiates such a search in the condition of every rewrite rule: since in practice many/most of the rewrite rules generated by the procedure in Figure 3.33 end up having the same left-hand-side, the expensive search for appropriate splittings is potentially invoked many times. What we'd like to achieve is: (1) activate the expensive search for splitting only once; and (2) for each found split, quickly test which rule applies and apply it. Such a quick test as desired in (2) can be achieved for free on existing rewrite systems that use indexing, such as Maude, if one slightly modifies the embedding translation of RSEC into rewrite logic as shown in Figure 3.34.

The main idea is to keep the structure of the left-hand-side of the RSEC rules in the left-hand-side of the corresponding rewrite rules. This structure is crucial for indexing. To allow it, one needs to do the necessary splitting as a separate step. The first type of rewrite rules in Figure 3.34, one per term appearing as a left-hand-side in any of the conditional rules generated following the first embedding in Figure 3.33, enables the splitting process on the corresponding contextual representations in the left-hand-side of the original RSEC rule. We only define such rules for left-hand-side terms having at least one subterm in contextual representation, because if the left-hand-side l has no such terms then the rule would be $\circ\, \bar{l} \to T$ **if** $\circ\, \bar{l} \to T$, which is useless and does not terminate.

The second type of rules in Figure 3.34, one per RSEC rule, have almost the same left-hand-sides as the original RSEC rules; the only difference is the algebraic notation (as reflected by the overlining). Their right-hand-sides plug the context representations, so that they always yield terms which are well-formed over the original syntax (possibly extended with auxiliary syntax for semantics components—configurations, states, etc.). Consider, for example, the six RSEC rules discussed in the preamble of Section 3.7, whose translation into rewrite rules following our first embedding in Figure 3.33 was discussed right above Theorem 18. Let us first consider the variant for lookup and assignment rules where the contextual representation in the left-hand side appears at the top. Since in all these rules the contextual representation appears at the top of their

left-hand-side, which in terms of the first embedding in Figure 3.33 means that their corresponding rewrite rules (in the first embedding) had the form $\circ\, Cfg \to \ldots$ **if** $split(Cfg) \to \ldots$, we only need to add one rule of the first type in Figure 3.34 for them, namely (l is the identity pattern, i.e., $\bar{l}$ is a variable):

$$\circ\, Cfg \to Cfg' \ \textbf{if} \ \circ\, split(Cfg) \to Cfg'$$

With this, the six rewrite rules of the second type in Figure 3.34 corresponding to the six RSEC rules under discussion are the following:

$$\begin{aligned}
\circ\, C[I_1 \texttt{<=} I_2] &\to plug(C[I_1 \leq_{Int} I_2]) \\
\circ\, C[\{\}\ S_2] &\to plug(C[S_2]) \\
\circ\, C[\texttt{if}\,(\texttt{true})\, S_1\, \texttt{else}\, S_2] &\to plug(C[S_1]) \\
\circ\, C[\texttt{if}\,(\texttt{false})\, S_1\, \texttt{else}\, S_2] &\to plug(C[S_2]) \\
\circ\, \langle C, \sigma\rangle[X] &\to plug(\langle C, \sigma\rangle[\sigma(X)]) \ \textbf{if} \ \sigma(X) \neq \bot \\
\circ\, \langle C, \sigma\rangle[X = I\,\texttt{;}] &\to plug(\langle C, \sigma[I/X]\rangle[\{\}]) \ \textbf{if} \ \sigma(X) \neq \bot
\end{aligned}$$

If one prefers the second variant for the reduction rules of lookup and assignment, namely

$$\begin{aligned}
\langle c[x], \sigma\rangle &\to \langle c[\sigma(x)], \sigma\rangle \quad \text{if } \sigma(x) \neq \bot \\
\langle c[x = i\,\texttt{;}], \sigma\rangle &\to \langle c[\{\}], \sigma[i/x]\rangle \quad \text{if } \sigma(x) \neq \bot
\end{aligned}$$

then, since the left-hand-side of these rules is a pattern of the form $\langle Stmt, \sigma\rangle$ which in algebraic form (overlined) becomes a term of the form $\langle S, \sigma\rangle$, we need to add one more rewrite rule of the first type in Figure 3.34, namely

$$\circ\, \langle S, \sigma\rangle \to Cfg' \ \textbf{if} \ \circ\, \langle split(S), \sigma\rangle \to Cfg',$$

and to replace the rewrite rules for lookup and assignment above with the following two rules:

$$\begin{aligned}
\circ\, \langle C[X], \sigma\rangle &\to \langle plug(C[\sigma(X)]), \sigma\rangle \ \textbf{if} \ \sigma(X) \neq \bot \\
\circ\, \langle C[X = I\,\texttt{;}], \sigma\rangle &\to \langle plug(C[\{\}]), \sigma[I/X]\rangle \ \textbf{if} \ \sigma(X) \neq \bot
\end{aligned}$$

Theorem 19. ***(Second faithful embedding of reduction semantics in rewrite logic)*** *Let* RSEC *be any reduction semantics with evaluation contexts definition and let* $\mathcal{R}^{\boxed{2}}_{\text{RSEC}}$ *be the rewrite logic theory associated to* RSEC *using the embedding procedures in Figures 3.32 and 3.34. Then*

1. ***(step-for-step correspondence)*** RSEC $\vdash t \to t'$ *using a reduction semantics with evaluation contexts rule iff* $\mathcal{R}^{\boxed{2}}_{\text{RSEC}} \vdash \circ\, \bar{t} \to^1 \overline{t'}$ *using the corresponding rewrite rules obtained like in Figure 3.34 (first a conditional rule of the first type whose left-hand-side matches t, then a rule of the second type which solves, in one rewrite step, the condition of the first rule); moreover, the reduction rule and the corresponding rewrite rules apply similarly (same contexts, same substitution; all modulo the correspondence in Theorem 17);*

2. ***(computational correspondence)*** RSEC $\vdash t \to^\star t'$ *iff* $\mathcal{R}^{\boxed{2}}_{\text{RSEC}} \vdash \star\, \bar{t} \to \star\, \overline{t'}$.

Theorem 19 tells us that we can use our second rewriting logic embedding transformation in Figure 3.34 to seamlessly execute RSEC definitions on context-insensitive rewrite engines, such as Maude. This was also the case for our first embedding (Figure 3.33 and its corresponding Theorem 18). However, as explained above, in our second embedding the left-hand-side terms of the rewrite rules corresponding to the actual reduction semantics rules (the second type of rule in Figure 3.34) preserve the structure of the left-hand-side

rules:

// for each term l that appears as the left-hand-side of a reduction rule
// $l(c_1[l_1], \ldots, c_n[l_n]) \to \ldots$, add the following conditional
// rewrite rule (there could be one l for many reduction rules):

$$\circ\, \bar{l}(T_1, \ldots, T_n) \to T \ \textbf{if}\ plug(\circ\, \bar{l}(split(T_1), \ldots, split(T_n))) \to T$$

// for each non-identity term r appearing as right-hand-side in a reduction rule
// $\ldots \to r(c_1[r_1], \ldots, c_n[r_n])$, add the following equation
// (there could be one r for many reduction rules):

$$plug(\bar{r}(Syn_1, \ldots, Syn_n)) = \bar{r}(plug(Syn_1), \ldots, plug(Syn_n))$$

// for each reduction semantics rule $l(c_1[l_1], \ldots, c_n[l_n]) \to r(c'_1[r_1], \ldots, c'_{n'}[r_{n'}])$
// add the following semantic rewrite rule:

$$\circ\, \bar{l}(\overline{c_1}[\overline{l_1}], \ldots, \overline{c_n}[\overline{l_n}]) \to \bar{r}(\overline{c'_1}[\overline{r_1}], \ldots, \overline{c'_n}[\overline{r_{n'}}])$$

Figure 3.35: Third embedding of RSEC in rewrite logic (RSEC $\rightsquigarrow \mathcal{R}^{\boxed{3}}_{\text{RSEC}}$).

terms of the original corresponding reduction rules. This important fact has two benefits. On the one hand, the underlying rewrite engines can use that structure to enhance the efficiency of rewriting by means of indexing, as already discussed above. On the other hand, the resulting rewrite rules resemble the original reduction rules, so the language designer who wants to use our embedding feels more comfortable. Indeed, since the algebraic representation of terms (the overline) should not change the way they are perceived by a user, the only difference between the left-hand-side of the original reduction rule and the left-hand-side of the resulting rewrite rule is the $\circ$ symbol: $l(c_1[l_1], \ldots, c_n[l_n])$ versus $\circ\, \bar{l}(\overline{c_1}[\overline{l_1}], \ldots, \overline{c_n}[\overline{l_n}])$, e.g., $\langle c[x = i\,;], \sigma\rangle$ versus $\circ\, \langle C[X = I\,;], \mathcal{O}\rangle$, where c, σ, x, i are reduction rule parameters while C, $\mathcal{O}$, X, I are corresponding variables of appropriate sorts.

Even though the representational distance between the left-hand-side terms in the original reduction rules and the left-hand-side terms in the resulting rewrite rules is minimal (one cannot eliminate the $\circ$, as extensively discussed in Section 3.3.3), unfortunately, the same does not hold true for the right-hand-side terms. Indeed, a right-hand-side $r(c'_1[r_1], \ldots, c'_n[r_{n'}])$ of a reduction rule becomes the right-hand-side $\bar{r}(plug(\overline{c'_1}[\overline{r_1}]), \ldots, plug(\overline{c'_n}[\overline{r_{n'}}]))$ of its corresponding rewrite rule, e.g., $\langle c[\{\}], \sigma\rangle$ becomes $\langle plug(C[\{\}]), \mathcal{O}\rangle$.

Figure 3.35 shows our third and final embedding of RSEC in rewrite logic, which has the advantage that it completely isolates the uses of *split*/*plug* from the semantic rewrite rules. Indeed, the rewrite rule associated to a reduction rule has the same left-hand-side as in the second embedding, but now the right-hand-side is actually the algebraic variant of the right-hand-side of the original reduction rule. This is possible because of two simple adjustments of the second embedding:

1. To avoid having to explicitly use the *plug* operation in the semantic rewrite rules, we replace the first type of conditional rewrite rules in the second embedding, namely

$$\circ\, \bar{l}(T_1, \ldots, T_n) \to T \ \textbf{if}\ \circ\, \bar{l}(split(T_1), \ldots, split(T_n)) \to T,$$

with slightly modified conditional rewrite rules of the form

$$\circ\,\bar{l}(T_1,\ldots,T_n) \to T \ \textbf{if}\ plug(\circ\,\bar{l}(split(T_1),\ldots,split(T_n))) \to T.$$

Therefore, the left-hand-side term of the condition is wrapped with the *plug* operation. Since rewriting is context-insensitive, the *plug* wrapper does not affect the rewrites that happen underneath in the $\circ\,\bar{l}(\ldots)$ term. Like in the second embedding, the only way for $\circ$ to disappear from the condition left-hand-side is for a semantic rule to apply. When that happens, the left-hand-side of the condition is rewritten to a term of the form $plug(t)$, where t matches the right-hand-side of some reduction semantics rule, which may potentially contain some subterms in contextual representation.

2. To automatically plug all the subterms in contextual representation that appear in t after the left-hand-side term of the condition in the rule above rewrites to $plug(t)$, we add equations of the form

$$plug(\bar{r}(Syn_1,\ldots,Syn_n)) = \bar{r}(plug(Syn_1),\ldots,plug(Syn_n)),$$

one for each non-identity pattern r appearing as a right-hand side of an RSEC rule; if r is an identity pattern then the equation becomes $plug(Syn) = plug(Syn)$, so we omit it.

Let us exemplify our third rewrite logic embedding transformation of reduction semantics with evaluation contexts using the same six reduction rules used so far in this section, but, to make it more interesting, considering the second variant of reduction rules for variable lookup and assignment. We have two left-hand-side patterns in these reduction rules, namely *Configuration* and $\langle Stmt, \sigma\rangle$, so we have the following two rules of the first type in Figure 3.35:

$$\circ\, Cfg \to Cfg' \ \textbf{if}\ plug(\circ\, split(Cfg)) \to Cfg'$$
$$\circ\,\langle S, \sigma\rangle \to Cfg' \ \textbf{if}\ plug(\circ\,\langle split(S), \sigma\rangle) \to Cfg'$$

We also have two right-hand-side patterns in these reduction rules, the same two as above, but the first one is an identity pattern so we only add one equation of the second type in Figure 3.35:

$$plug(\langle C[Syn], \sigma\rangle) = \langle plug(C[Syn]), \sigma\rangle$$

We can now give the six rewrite rules corresponding to the six reduction rules in discussion:

$$\begin{aligned}
\circ\, C[I_1 \texttt{ <= } I_2] &\to C[I_1 \leq_{Int} I_2]\\
\circ\, C[\{\}\ S_2] &\to C[S_2]\\
\circ\, C[\texttt{if (true)}\ S_1\ \texttt{else}\ S_2] &\to C[S_1]\\
\circ\, C[\texttt{if (false)}\ S_1\ \texttt{else}\ S_2] &\to C[S_2]\\
\circ\,\langle C[X], \sigma\rangle &\to \langle C[\sigma(X)], \sigma\rangle \ \textbf{if}\ \sigma(X) \neq \bot\\
\circ\,\langle C[X = I\,\texttt{;}], \sigma\rangle &\to \langle C[\{\}], \sigma[I/X]\rangle \ \textbf{if}\ \sigma(X) \neq \bot
\end{aligned}$$

The six rewrite rules above are as close to the original reduction semantics rules as one can hope them to be in a rewriting setting. Note that, for simplicity, we preferred to desugar the characteristic rule of reduction semantics with evaluation contexts in all our examples in this subsection. At this moment we have all the infrastructure needed to also include a rewrite equivalent of it:

$$\circ\, C[Syn] \to C[Syn'] \ \textbf{if}\ C \neq \Box \wedge \circ\, Syn \to Syn'$$

Note that we first check whether the context is proper in the condition of the characteristic rewrite rule above, and then we initiate a (small-step) reduction of the redex (by tagging it with the symbol $\circ$). The condition

is well-defined in rewrite logic because, as explained in Figure 3.32, we subsorted all the syntactic sorts together with the configuration under the top sort *Syntax*, so all these sorts belong to the same kind (see Section 2.5), which means that the operation $\circ$ can apply to any of them, including to *Syntax*, despite the fact that it was declared to take a *Configuration* to an *ExtendedConfiguration* (like in Section 3.3.3). With this characteristic rewrite rule, we can now restate the six rewrite rules corresponding to the six reduction rules above as follows:

$$\begin{aligned}
\circ\, I_1 <= I_2 &\to I_1 \leq_{Int} I_2 \\
\circ\, \{\}\; S_2 &\to S_2 \\
\circ\, \texttt{if}\,\texttt{(true)}\; S_1\; \texttt{else}\; S_2 &\to S_1 \\
\circ\, \texttt{if}\,\texttt{(false)}\; S_1\; \texttt{else}\; S_2 &\to S_2 \\
\circ\, \langle C[X], \sigma\rangle &\to \langle C[\sigma(X)], \sigma\rangle \;\; \textbf{if}\;\; \sigma(X) \neq \bot \\
\circ\, \langle C[X = I\,\texttt{;}], \sigma\rangle &\to \langle C[\{\}], \sigma[I/X]\rangle \;\; \textbf{if}\;\; \sigma(X) \neq \bot
\end{aligned}$$

Note, again, that $\circ$ is applied on arguments of various sorts in the same kind with *Configuration*.

The need for $\circ$ in the left-hand-side terms of rules like above is now even more imperative than before. In addition to all the reasons discussed so far, there are additional reasons now for which the dropping of $\circ$ would depart us from the intended faithful capturing of reduction semantics in rewrite logic. Indeed, if we drop $\circ$ then there is nothing to stop the applications of rewrite rules at any places in the term to rewrite, potentially including places which are not allowed to be evaluated yet, such as, for example, in the branches of a conditional. Moreover, such applications of rules could happen concurrently, which is strictly disallowed by reduction semantics with or without evaluation contexts. The role of $\circ$ is precisely to inhibit the otherwise unrestricted potential to apply rewrite rules everywhere and concurrently: rules are now applied sequentially and only at the top of the original term, exactly like in reduction semantics.

Theorem 20. ***(Third faithful embedding of reduction semantics into rewrite logic)*** *Let* RSEC *be any reduction semantics with evaluation contexts definition (with or without a characteristic reduction rule) and let* $\mathcal{R}^{\boxed{3}}_{\text{RSEC}}$ *be the rewrite logic theory associated to* RSEC *using the embedding procedures in Figures 3.32 and 3.35 (plus the characteristic rewrite rule above in case* RSEC *comes with a characteristic reduction rule). Then*

1. ***(step-for-step correspondence)*** RSEC $\vdash t \to t'$ *using a reduction semantics with evaluation contexts rule iff* $\mathcal{R}^{\boxed{3}}_{\text{RSEC}} \vdash \circ\, \bar{t} \to^1 \overline{t'}$*;*

2. ***(computational correspondence)*** RSEC $\vdash t \to^\star t'$ *iff* $\mathcal{R}^{\boxed{3}}_{\text{RSEC}} \vdash \star\, \bar{t} \to \star\, \overline{t'}$.

We can therefore safely conclude that RSEC has been captured as a methodological fragment of rewrite logic. The faithful embeddings of reduction semantics into rewrite logic above can be used in at least two different ways. On the one hand, they can be used as compilation steps transforming a context-sensitive reduction system into an equivalent context-insensitive rewrite system, which can be further executed/compiled/analyzed using conventional rewrite techniques and existing rewrite engines. On the other hand, the embeddings above are so simple, that one can simply use them manually and thus "think reduction semantics" in rewrite logic.

Reduction Semantics with Evaluation Contexts of IMP in Rewrite Logic

We here discuss the complete reduction semantics with evaluation contexts definition of IMP in rewrite logic, obtained by applying the faithful embedding techniques discussed above to the reduction semantics definition of IMP in Figure 3.31 in Section 3.7.1. We start by defining the needed configurations, then we give all the

sorts:
Configuration, *ExtendedConfiguration*
subsort:
Configuration < *ExtendedConfiguration*
operations:

$$\begin{array}{rcll} \langle _, _ \rangle & : & \mathit{Stmt} \times \mathit{State} \to \mathit{Configuration} & \\ \langle _ \rangle & : & \mathit{Pgm} \to \mathit{Configuration} & \\ \circ _ & : & \mathit{Configuration} \to \mathit{ExtendedConfiguration} & \text{// reduce one step} \\ \star _ & : & \mathit{Configuration} \to \mathit{ExtendedConfiguration} & \text{// reduce all steps} \end{array}$$

rule:
$\star \mathit{Cfg} \to \star \mathit{Cfg}'$ **if** $\circ\, \mathit{Cfg} \to \mathit{Cfg}'$ // where $\mathit{Cfg}, \mathit{Cfg}'$ are variables of sort *Configuration*

Figure 3.36: Configurations and infrastructure for the rewrite logic embedding of RSEC(IMP).

rewrite rules and equations embedding the evaluation contexts and their splitting/plugging mechanism in rewrite logic, and then we finally give three rewrite theories corresponding to the three embeddings discussed above, each including the (same) configurations definition and embedding of evaluation contexts.

Figure 3.36 gives an algebraic definition of IMP configurations as needed for reduction semantics with evaluation contexts, together with the additional infrastructure needed to represent the one-step and multi-step transition relations. Everything defined in Figure 3.36 has already been discussed in the context of small-step SOS (see Figures 3.13 and 3.17 in Section 3.3.3). Note, however, that we only defined a subset of the configurations needed for small-step SOS, more precisely only the top-level configurations (ones holding a program and ones holding a statement and a state). The intermediate configurations holding expressions and a state in small-step SOS are not needed here because reduction semantics with evaluation contexts does not need to explicitly decompose bigger reduction tasks into smaller ones until a redex is eventually found, like small-step SOS does; instead, the redex is found atomically by splitting the top level configuration into a context and the redex.

Figure 3.37 shows the rewrite logic theory $\mathcal{R}^{\square}_{\text{RSEC(IMP)}}$ associated to the evaluation contexts of IMP in RSEC(IMP) (Figure 3.30) following the procedure described in Section 3.7.3 and summarized in Figure 3.32. Recall that all language syntactic categories and configurations are sunk into a top sort *Syntax*, and that one rule for splitting and one equation for plugging are generated for each context production. In general, the embedding of evaluation contexts tends to be the largest and the most boring portion of the rewrite logic embedding of a reduction semantics language definition. However, fortunately, this can be generated fully automatically. An implementation of the rewrite logic embedding techniques discussed in this section may even completely hide this portion from the user. We show it in Figure 3.37 only for the sake of completeness.

Figure 3.38 shows the rewrite logic theory $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$ corresponding to the rules in the reduction semantics with evaluation contexts of IMP in Section 3.7.1, following our first embedding transformation depicted in Figure 3.33. Like before, we used the rewrite logic convention that variables start with upper-case letters; if they are Greek letters, then we use a similar but larger symbol (e.g., Σ instead of σ for variables of sort *State*). These rules are added, of course, to those corresponding to evaluation contexts in Figure 3.37 (which are common to all three embeddings). Note that there is precisely one conditional rewrite rule in Figure 3.38 corresponding to each reduction semantics rule of IMP in Figure 3.31. Also, note that if a rule does not make use of evaluation contexts, then its corresponding rewrite rule is identical to the rewrite rule corresponding to the small-step SOS embedding discussed in Section 3.3.3. For example, the last reduction rule in Figure 3.31 results in the last rewrite rule in Figure 3.38, which is identical to the last rewrite rule

sorts:

$Syntax,\ Context$

subsorts:

$AExp, BExp, Stmt, Configuration < Syntax$

operations:

$\square : \to Context$ $\qquad$ $_[_] : Context \times Syntax \to Syntax$

$split : Syntax \to Syntax$ $\qquad$ $plug : Syntax \to Syntax$

$\langle _, _ \rangle : Context \times State \to Context$

$_ + _ : Context \times AExp \to Context$ $\qquad$ $_ + _ : AExp \times Context \to Context$

$_ / _ : Context \times AExp \to Context$ $\qquad$ $_ / _ : AExp \times Context \to Context$

$_ <= _ : Context \times AExp \to Context$ $\qquad$ $_ <= _ : Int \times Context \to Context$

$! _ : Context \to Context$

$_ \&\& _ : Context \times BExp \to Context$

$_ = _; : Id \times Context \to Context$

$_ _ : Context \times Stmt \to Context$

$\texttt{if}\,(_)\ _\,\texttt{else}\ _ : Context \times Stmt \times Stmt \to Context$

rules and equations:

$split(Syn) \to \square[Syn]$ $\qquad$ $plug(\square[Syn]) = Syn$

$split(\langle S, \sigma \rangle) \to \langle C, \sigma \rangle[Syn]$ **if** $split(S) \to C[Syn]$

$plug(\langle C, \sigma \rangle[Syn]) = \langle plug(C[Syn]), \sigma \rangle$

$split(A_1 + A_2) \to (C + A_2)[Syn]$ **if** $split(A_1) \to C[Syn]$

$plug((C + A_2)[Syn]) = plug(C[Syn]) + A_2$

$split(A_1 + A_2) \to (A_1 + C)[Syn]$ **if** $split(A_2) \to C[Syn]$

$plug((A_1 + C)[Syn]) = A_1 + plug(C[Syn])$

$split(A_1 \ / \ A_2) \to (C \ / \ A_2)[Syn]$ **if** $split(A_1) \to C[Syn]$

$plug((C \ / \ A_2)[Syn]) = plug(C[Syn]) \ / \ A_2$

$split(A_1 \ / \ A_2) \to (A_1 \ / \ C)[Syn]$ **if** $split(A_2) \to C[Syn]$

$plug((A_1 \ / \ C)[Syn]) = A_1 \ / \ plug(C[Syn])$

$split(A_1 <= A_2) \to (C <= A_2)[Syn]$ **if** $split(A_1) \to C[Syn]$

$plug((C <= A_2)[Syn]) = plug(C[Syn]) <= A_2$

$split(I_1 <= A_2) \to (I_1 <= C)[Syn]$ **if** $split(A_2) \to C[Syn]$

$plug((I_1 <= C)[Syn]) = I_1 <= plug(C[Syn])$

$split(!\ B) \to (!\ C)[Syn]$ **if** $split(B) \to C[Syn]$

$plug((!\ C)[Syn]) = !\ plug(C[Syn])$

$split(B_1 \ \&\& \ B2) \to (C \ \&\& \ B_2)[Syn]$ **if** $split(B_1) \to C[Syn]$

$plug((C \ \&\& \ B_2)[Syn]) = plug(C[Syn]) \ \&\& \ B_2$

$split(X = A\,;) \to (X = C\,;)[Syn]$ **if** $split(A) \to C[Syn]$

$plug((X = C\,;)[Syn]) = X = plug(C[Syn])\,;$

$split(S_1\ S_2) \to (C\ S_2)[Syn]$ **if** $split(S_1) \to C[Syn]$

$plug((C\ S_2)[Syn]) = plug(C[Syn])\ S_2$

$split(\texttt{if}\,(B)\ S_1\ \texttt{else}\ S_2) \to (\texttt{if}\,(C)\ S_1\ \texttt{else}\ S_2)[Syn]$ **if** $split(B) \to C[Syn]$

$plug((\texttt{if}\,(C)\ S_1\ \texttt{else}\ S_2)[Syn]) = \texttt{if}\,(plug(C[Syn]))\ S_1\ \texttt{else}\ S_2$

Figure 3.37: $\mathcal{R}^{\square}_{\text{RSEC(IMP)}}$: Rewrite logic embedding of IMP evaluation contexts. The implicit split/plug reduction semantics mechanism is replaced by explicit rewrite logic sentences.

$\circ\, Cfg \to plug(\langle C, \sigma\rangle[\sigma(X)])$ **if** $split(Cfg) \to \langle C, \sigma\rangle[X] \ \wedge\ \sigma(X) \neq \bot$
$\circ\, Cfg \to plug(C[I_1 +_{Int} I_2])$ **if** $split(Cfg) \to C[I_1 + I_2]$
$\circ\, Cfg \to plug(C[I_1 /_{Int} I_2])$ **if** $split(Cfg) \to C[I_1 \ / \ I_2] \ \wedge\ I_2 \neq 0$
$\circ\, Cfg \to plug(C[I_1 \leq_{Int} I_2])$ **if** $split(Cfg) \to C[I_1 <= I_2]$
$\circ\, Cfg \to plug(C[\texttt{false}])$ **if** $split(Cfg) \to C[\texttt{!}\ \texttt{true}]$
$\circ\, Cfg \to plug(C[\texttt{true}])$ **if** $split(Cfg) \to C[\texttt{!}\ \texttt{false}]$
$\circ\, Cfg \to plug(C[B_2])$ **if** $split(Cfg) \to C[\texttt{true}\ \&\&\ B2]$
$\circ\, Cfg \to plug(C[\texttt{false}])$ **if** $split(Cfg) \to C[\texttt{false}\ \&\&\ B2]$
$\circ\, Cfg \to plug(\langle C, \sigma[I/X]\rangle[\{\}])$ **if** $split(Cfg) \to \langle C, \sigma\rangle[X = I\,\texttt{;}] \ \wedge\ \sigma(X) \neq \bot$
$\circ\, Cfg \to plug(C[S_2])$ **if** $split(Cfg) \to C[\{\}\ S_2]$
$\circ\, Cfg \to plug(C[S_1])$ **if** $split(Cfg) \to C[\texttt{if}\,(\texttt{true})\ S_1\ \texttt{else}\ S_2]$
$\circ\, Cfg \to plug(C[S_2])$ **if** $split(Cfg) \to C[\texttt{if}\,(\texttt{false})\ S_1\ \texttt{else}\ S_2]$
$\circ\, Cfg \to plug(C[\texttt{if}\,(B)\,\{\,S\ \texttt{while}\,(B)\ S\,\}\,\texttt{else}\,\{\}])$ **if** $split(Cfg) \to C[\texttt{while}\,(B)\ S]$
$\circ\, \langle \texttt{int}\ Xl\texttt{;}\ S\rangle \to \langle S, (Xl \mapsto 0)\rangle$

Figure 3.38: $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$ — rewrite logic theory corresponding to the first embedding of the reduction semantics with evaluation contexts of IMP.

corresponding to the small-step SOS of IMP in Figure 3.18. The rules that make use of evaluation contexts perform explicit splitting (in the left-hand-side of the condition) and plugging (in the right-hand-side of the conclusion) operations. As already discussed but worth reemphasizing, the main drawbacks of this type of rewrite logic embedding are: (1) the expensive, non-deterministic search involving splitting of the original term is performed for any rule, and (2) it does not take advantage of one of the major optimizations of rewrite engines, indexing, which allows for quick detection of matching rules based on the structure of their left-hand-side terms.

Figure 3.39 shows the rewrite logic theory $\mathcal{R}^{\boxed{2}}_{\text{RSEC(IMP)}}$ that follows our second embedding transformation depicted in Figure 3.34. These rules are also added to those corresponding to evaluation contexts in Figure 3.37. Note that now there is precisely one *unconditional* rewrite rule corresponding to each reduction semantics rule of IMP in Figure 3.31 and that, unlike in the first embedding in Figure 3.38, the left-hand-side of each rule preserves the exact structure of the left-hand-side of the original reduction rule (after desugaring of the characteristic rule), so this embedding takes advantage of indexing optimizations in rewrite engines. Like in the first embedding, if a reduction rule does not make use of evaluation contexts, then its corresponding rewrite rule is identical to the rewrite rule corresponding to the small-step SOS embedding discussed in Section 3.3.3 (e.g., the last rule). Unlike in the first embedding, we also need to add a generic conditional rule, the first one in Figure 3.39, which initiates the splitting. We need only one rule of this type because all the left-hand-side terms of reduction rules of IMP in Figure 3.31 that contain a subterm in contextual representation contain that term at the top. As already discussed, if one preferred to write, e.g., the lookup RSEC rule as $\langle c[x], \sigma\rangle \to \langle c[\sigma(x)], \sigma\rangle$ if $\sigma(x) \neq \bot$, then one would need an additional generic rule, namely $\circ\, \langle S, \sigma\rangle \to Cfg'$ **if** $\circ\, \langle split(S), \sigma\rangle \to Cfg'$. While these generic rules take care of splitting and can be generated relatively automatically, the remaining rewrite rules that correspond to the reduction rules still make explicit use of the internal (to the embedding) *plug* operation, which can arguably be perceived by language designers as an inconvenience.

Figure 3.40 shows the rewrite logic theory $\mathcal{R}^{\boxed{3}}_{\text{RSEC(IMP)}}$ obtained by applying our third embedding (shown in Figure 3.35). These rules are also added to those corresponding to evaluation contexts in Figure 3.37 and, like in the second embedding, there is precisely one unconditional rewrite rule corresponding to each RSEC

$$
\begin{aligned}
\circ\, Cfg &\to Cfg' \ \textbf{if}\ \ \circ\, split(Cfg) \to Cfg' \\
\\
\circ\, \langle C, \sigma\rangle[X] &\to plug(\langle C, \sigma\rangle[\sigma(X)])\ \textbf{if}\ \sigma(X) \neq \bot \\
\circ\, C[I_1 + I_2] &\to plug(C[I_1 +_{Int} I_2]) \\
\circ\, C[I_1 \ / \ I_2] &\to plug(C[I_1 \ /_{Int} \ I_2])\ \textbf{if}\ I_2 \neq 0 \\
\circ\, C[I_1 <= I_2] &\to plug(C[I_1 \leq_{Int} I_2]) \\
\circ\, C[\texttt{!}\ \texttt{true}] &\to plug(C[\texttt{false}]) \\
\circ\, C[\texttt{!}\ \texttt{false}] &\to plug(C[\texttt{true}]) \\
\circ\, C[\texttt{true}\ \&\&\ B2] &\to plug(C[B_2]) \\
\circ\, C[\texttt{false}\ \&\&\ B2] &\to plug(C[\texttt{false}]) \\
\circ\, \langle C, \sigma\rangle[X = I\,\texttt{;}] &\to plug(\langle C, \sigma[I/X]\rangle[\{\}])\ \textbf{if}\ \sigma(X) \neq \bot \\
\circ\, C[\{\}\ S_2] &\to plug(C[S_2]) \\
\circ\, C[\texttt{if}\ (\texttt{true})\ S_1\ \texttt{else}\ S_2] &\to plug(C[S_1]) \\
\circ\, C[\texttt{if}\ (\texttt{false})\ S_1\ \texttt{else}\ S_2] &\to plug(C[S_2]) \\
\circ\, C[\texttt{while}\ (B)\ S] &\to plug(C[\texttt{if}\ (B)\ \{\, S\ \ \texttt{while}\ (B)\ S\ \}\ \texttt{else}\ \{\}]) \\
\circ\, \langle \texttt{int}\ Xl\texttt{;}\ S\rangle &\to \langle S, (Xl \mapsto 0)\rangle
\end{aligned}
$$

Figure 3.39: $\mathcal{R}^{\boxed{2}}_{\text{RSEC(IMP)}}$ — rewrite logic theory corresponding to the second embedding of the reduction semantics with evaluation contexts of IMP.

$$
\begin{aligned}
\circ\, Cfg &\to Cfg' \ \textbf{if}\ \ \circ\, plug(split(Cfg)) \to Cfg' \\
\circ\, C[Syn] &\to C[Syn']\ \textbf{if}\ C \neq \Box \wedge \circ\, Syn \to Syn' \\
\\
\circ\, \langle C, \sigma\rangle[X] &\to \langle C, \sigma\rangle[\sigma(X)]\ \textbf{if}\ \sigma(X) \neq \bot \\
\circ\, I_1 + I_2 &\to I_1 +_{Int} I_2 \\
\circ\, I_1 \ / \ I_2 &\to I_1 \ /_{Int} \ I_2]\ \textbf{if}\ I_2 \neq 0 \\
\circ\, I_1 <= I_2 &\to I_1 \leq_{Int} I_2 \\
\circ\, \texttt{!}\ \texttt{true} &\to \texttt{false} \\
\circ\, \texttt{!}\ \texttt{false} &\to \texttt{true} \\
\circ\, \texttt{true}\ \&\&\ B2 &\to B_2 \\
\circ\, \texttt{false}\ \&\&\ B2 &\to \texttt{false} \\
\circ\, \langle C, \sigma\rangle[X = I\,\texttt{;}] &\to \langle C, \sigma[I/X]\rangle[\{\}]\ \textbf{if}\ \sigma(X) \neq \bot \\
\circ\, \{\}\ S_2 &\to S_2 \\
\circ\, \texttt{if}\ (\texttt{true})\ S_1\ \texttt{else}\ S_2 &\to S_1 \\
\circ\, \texttt{if}\ (\texttt{false})\ S_1\ \texttt{else}\ S_2 &\to S_2 \\
\circ\, \texttt{while}\ (B)\ S &\to \texttt{if}\ (B)\ \{\, S\ \ \texttt{while}\ (B)\ S\ \}\ \texttt{else}\ \{\} \\
\circ\, \langle \texttt{int}\ Xl\texttt{;}\ S\rangle &\to \langle S, (Xl \mapsto 0)\rangle
\end{aligned}
$$

Figure 3.40: $\mathcal{R}^{\boxed{3}}_{\text{RSEC(IMP)}}$ — rewrite logic theory corresponding to the third embedding of the reduction semantics with evaluation contexts of IMP.

rule of IMP. We also need to add a generic conditional rule, the first one, which completely encapsulates the rewrite logic representation of the splitting/plugging mechanism, so that the language designer can next focus exclusively on the semantic rules rather than on their representation in rewrite logic. The second rewrite rule in

discussed, it is optional; if one includes it, as we did, we think that its definition in Figure 3.40 is as simple and natural as it can be. In what regards the remaining rewrite rules, the only perceivable difference between them and their corresponding reduction rules is that they are preceded by $\circ$.

All the above suggest that, in spite of its apparently advanced context-sensitivity and splitting/plugging mechanism, reduction semantics with evaluation contexts can be safely regarded as a methodological fragment of rewrite logic. Or, put differently, while context-sensitive reduction seems crucial for programming language semantics, it is in fact unnecessary. A conditional rewrite framework can methodologically achieve the same results, and as discussed in this chapter, so can do for the other conventional language semantics approaches.

The following corollary of Theorems 18, 19, and 20 establishes the faithfulness of the representations of the reduction semantics with evaluation contexts of IMP in rewrite logic:

Corollary 7. *For any* IMP *configurations C and C', the following equivalences hold:*

$$\begin{aligned}
\text{RSEC(IMP)} \vdash C \to C' &\iff \mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}} \vdash \circ\,\overline{C} \to \overline{C'} \\
&\iff \mathcal{R}^{\boxed{2}}_{\text{RSEC(IMP)}} \vdash \circ\,\overline{C} \to \overline{C'} \\
&\iff \mathcal{R}^{\boxed{3}}_{\text{RSEC(IMP)}} \vdash \circ\,\overline{C} \to \overline{C'}
\end{aligned}$$

and

$$\begin{aligned}
\text{RSEC(IMP)} \vdash C \to^{\star} C' &\iff \mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}} \vdash \star\,\overline{C} \to \star\,\overline{C'} \\
&\iff \mathcal{R}^{\boxed{2}}_{\text{RSEC(IMP)}} \vdash \star\,\overline{C} \to \star\,\overline{C'} \\
&\iff \mathcal{R}^{\boxed{3}}_{\text{RSEC(IMP)}} \vdash \star\,\overline{C} \to \star\,\overline{C'}
\end{aligned}$$

Therefore, there is no perceivable computational difference between the reduction semantics with evaluation contexts RSEC(IMP) and its corresponding rewrite logic theories.

★ Reduction Semantics with Evaluation Contexts of IMP in Maude

Figure 3.41 shows a Maude representation of the rewrite theory $\mathcal{R}^{\square}_{\text{RSEC(IMP)}}$ in Figure 3.37 that embeds IMP's evaluation contexts by making explicit the split/plug mechanism which is implicit in RSEC. Figure 3.41 also includes the Maude definition of configurations (see Figure 3.36).

We took the freedom to implement a simple optimization which works well in Maude, but which may not work as well in other engines or systems (which is why we did not incorporate it as part of the general procedure to represent reduction semantics with evaluation contexts in rewrite logic): we defined the contextual representation operation `_[_]` to have as result the *kind* (see Section 2.5) `[Syntax]` instead of the sort `Syntax`. This allows us to include the equation `plug(Syn) = Syn`, where `Syn` is a variable of *sort* `Syntax`, which gives us the possibility to also use terms which do not make use of contexts in the right-hand-sides of rewrite rules. To test the rules for splitting, one can write Maude commands such as the one below, asking Maude to search for all splits of a given term:

```
search split(3 <= (2 + X) / 7) =>! Syn:[Syntax] .
```

The `!` tag on the arrow `=>` in the command above tells Maude to only report the normal forms, in this case the completed splits. As expected, Maude finds all seven splits and outputs the following:

```
mod IMP-CONFIGURATIONS-EVALUATION-CONTEXTS is including IMP-SYNTAX + STATE .
  sorts Configuration ExtendedConfiguration .
  subsort Configuration < ExtendedConfiguration .
  op <_,_> : Stmt State -> Configuration .
  op <_> : Pgm -> Configuration .
  ops (o_) (*_) : Configuration -> ExtendedConfiguration [prec 80] .   --- one step
  var Cfg Cfg' : Configuration .
 crl * Cfg => * Cfg' if o Cfg => Cfg' .
endm

mod IMP-SPLIT-PLUG-EVALUATION-CONTEXTS is including IMP-CONFIGURATIONS-EVALUATION-CONTEXTS .
  sorts Syntax Context .  subsorts AExp BExp Stmt Configuration < Syntax .
  op [] : -> Context .  op _[_] : Context Syntax -> [Syntax] [prec 1] .
  ops split plug : Syntax -> Syntax . --- to split Syntax into context[redex]

  var X : Id .  var A A1 A2 : AExp .  var B B1 B2 : BExp .  var S S1 S2 : Stmt .
  var Sigma : State .  var I1 : Int .  var Syn : Syntax .  var C : Context .

  rl split(Syn) => [][Syn] .  eq plug([][Syn]) = Syn .  eq plug(Syn) = Syn .

  op <_,_> : Context State -> Context .  eq plug(< C, Sigma > [Syn]) = < plug(C[Syn]), Sigma > .
 crl split(< S, Sigma >) => < C, Sigma > [Syn] if split(S) => C[Syn] .

  op _+_ : Context AExp -> Context .  eq plug((C + A2)[Syn]) = plug(C[Syn]) + A2 .
 crl split(A1 + A2) => (C + A2)[Syn] if split(A1) => C[Syn] .
  op _+_ : AExp Context -> Context .  eq plug((A1 + C)[Syn]) = A1 + plug(C[Syn]) .
 crl split(A1 + A2) => (A1 + C)[Syn] if split(A2) => C[Syn] .

  op _/_ : Context AExp -> Context .  eq plug((C / A2)[Syn]) = plug(C[Syn]) / A2 .
 crl split(A1 / A2) => (C / A2)[Syn] if split(A1) => C[Syn] .
  op _/_ : AExp Context -> Context .  eq plug((A1 / C)[Syn]) = A1 / plug(C[Syn]) .
 crl split(A1 / A2) => (A1 / C)[Syn] if split(A2) => C[Syn] .

  op _<=_ : Context AExp -> Context .  eq plug((C <= A2)[Syn]) = plug(C[Syn]) <= A2 .
 crl split(A1 <= A2) => (C <= A2)[Syn] if split(A1) => C[Syn] .
  op _<=_ : Int Context -> Context .  eq plug((I1 <= C)[Syn]) = I1 <= plug(C[Syn]) .
 crl split(I1 <= A2) => (I1 <= C)[Syn] if split(A2) => C[Syn] .

  op !_ : Context -> Context .  eq plug((! C)[Syn]) = ! plug(C[Syn]) .
 crl split(! B) => (! C)[Syn] if split(B) => C[Syn] .

  op _&&_ : Context BExp -> Context .  eq plug((C && B2)[Syn]) = plug(C[Syn]) && B2 .
 crl split(B1 && B2) => (C && B2)[Syn] if split(B1) => C[Syn] .

  op _=_; : Id Context -> Context .  eq plug((X = C ;)[Syn]) = X = plug(C[Syn]) ; .
 crl split(X = A ;) => (X = C ;)[Syn] if split(A) => C[Syn] .

  op __ : Context Stmt -> Context .  eq plug((C S2)[Syn]) = plug(C[Syn]) S2 .
 crl split(S1 S2) => (C S2)[Syn] if split(S1) => C[Syn] .

  op if(_)_else_ : Context Stmt Stmt -> Context .
 crl split(if (B) S1 else S2) => (if (C) S1 else S2)[Syn] if split(B) => C[Syn] .
  eq plug((if (C) S1 else S2)[Syn]) = if (plug(C[Syn])) S1 else S2 .
endm
```

Figure 3.41: The configuration and evaluation contexts of IMP in Maude, as needed for the three variants of reduction semantics with evaluation contexts of IMP in Maude.

```
Solution 1 (state 1)
states: 8  rewrites: 19 in ... cpu (... real) (0 rewrites/second)
Syn --> [][3 <= (2 + X) / 7]

Solution 2 (state 2)
states: 8  rewrites: 19 in ... cpu (. real) (0 rewrites/second)
Syn --> ([] <= (2 + X) / 7)[3]

Solution 3 (state 3)
states: 8  rewrites: 19 in ... cpu (... real) (0 rewrites/second)
Syn --> (3 <= [])[(2 + X) / 7]

Solution 4 (state 4)
states: 8  rewrites: 19 in ... cpu (... real) (0 rewrites/second)
Syn --> (3 <= ([] / 7))[2 + X]

Solution 5 (state 5)
states: 8  rewrites: 19 in ... cpu (... real) (0 rewrites/second)
Syn --> (3 <= (([] + X) / 7))[2]

Solution 6 (state 6)
states: 8  rewrites: 19 in ... cpu (... real) (0 rewrites/second)
Syn --> (3 <= ((2 + []) / 7))[X]

Solution 7 (state 7)
states: 8  rewrites: 19 in ... cpu (... real) (0 rewrites/second)
Syn --> (3 <= 2 + X / [])[7]
```

If, however, we replace any of the rules for splitting with equations, then, as expected, one looses some of the splitting behaviors. For example, if we replace the generic rule for splitting `rl split(Syn) => [][Syn]` by an apparently equivalent equation `eq split(Syn) = [][Syn]`, then Maude will be able to detect no other splitting of a term t except for $\square[t]$ (because Maude executes the equations before the rules; see Section 2.5.6).

Figure 3.42 shows two Maude modules implementing the first two rewrite logic theories $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$ (Figure 3.38) and $\mathcal{R}^{\boxed{2}}_{\text{RSEC(IMP)}}$ (Figure 3.39), and Figure 3.43 shows the Maude module implementing the third rewrite theory $\mathcal{R}^{\boxed{3}}_{\text{RSEC(IMP)}}$ (Figure 3.40), respectively. Each of these three Maude modules imports the module `IMP-CONFIGURATION-EVALUATION-CONTEXTS` defined in Figure 3.41 and is executable. Maude, through its rewriting capabilities, therefore yields an IMP reduction semantics with evaluation contexts interpreter for each of the three modules in Figures 3.42 and 3.43. For any of them, the Maude rewrite command

```
rewrite * < sumPgm > .
```

where `sumPgm` is the first program defined in the module `IMP-PROGRAMS` in Figure 3.4, produces a result of the form (the exact statistics are also irrelevant, so they were replaced by "...”):

```
rewrites: 42056 in ... cpu (... real) (... rewrites/second)
result ExtendedConfiguration: * < skip, n |-> 0 & s |-> 5050 >
```

The reason for this is ...

One can use any of the general-purpose tools provided by Maude on the reduction semantics with evaluation contexts definitions above. For example, one can exhaustively search for all possible behaviors of a program using the `search` command:

```
mod IMP-SEMANTICS-EVALUATION-CONTEXTS is including IMP-SPLIT-PLUG-EVALUATION-CONTEXTS .
  var X : Id .  var I I1 I2 : Int .  var B B2 : BExp .  var S S1 S2 : Stmt .
  var Xl : List{Id} .  var Sigma : State .  var Cfg : Configuration .  var C : Context .

 crl o Cfg => plug(< C,Sigma >[Sigma(X)]) if split(Cfg) => < C,Sigma >[X]
  /\ Sigma(X) =/=Bool undefined .
 crl o Cfg => plug(C[I1 +Int I2]) if split(Cfg) => C[I1 + I2] .
 crl o Cfg => plug(C[I1 /Int I2]) if split(Cfg) => C[I1 / I2]
  /\ I2 =/=Bool 0 .
 crl o Cfg => plug(C[I1 <=Int I2]) if split(Cfg) => C[I1 <= I2] .
 crl o Cfg => plug(C[false]) if split(Cfg) => C[! true] .
 crl o Cfg => plug(C[true]) if split(Cfg) => C[! false] .
 crl o Cfg => plug(C[B2]) if split(Cfg) => C[true && B2] .
 crl o Cfg => plug(C[false]) if split(Cfg) => C[false && B2] .
 crl o Cfg => plug(C[S]) if split(Cfg) => C[{S}] .
 crl o Cfg => plug(< C,Sigma[I / X] >[{}]) if split(Cfg) => < C,Sigma >[X = I ;]
  /\ Sigma(X) =/=Bool undefined .
 crl o Cfg => plug(C[S2]) if split(Cfg) => C[{} S2] .
 crl o Cfg => plug(C[S1]) if split(Cfg) => C[if (true) S1 else S2] .
 crl o Cfg => plug(C[S2]) if split(Cfg) => C[if (false) S1 else S2] .
 crl o Cfg => plug(C[if (B) {S while (B) S} else {}]) if split(Cfg) => C[while (B) S] .
  rl o < int Xl ; S > => < S,(Xl |-> 0) > .
endm
```

```
mod IMP-SEMANTICS-EVALUATION-CONTEXTS is including IMP-SPLIT-PLUG-EVALUATION-CONTEXTS .
  var X : Id .  var I I1 I2 : Int .  var B B2 : BExp .  var S S1 S2 : Stmt .
  var Xl : List{Id} .  var Sigma : State .  var Cfg Cfg' : Configuration .  var C : Context .

 crl o Cfg => Cfg' if o split(Cfg) => Cfg' .   --- generic rule enabling splitting

 crl o < C,Sigma >[X] => plug(< C,Sigma >[Sigma(X)])
  if Sigma(X) =/=Bool undefined .
  rl o C[I1 + I2] => plug(C[I1 +Int I2]) .
 crl o C[I1 / I2] => plug(C[I1 /Int I2])
  if I2 =/=Bool 0 .
  rl o C[I1 <= I2] => plug(C[I1 <=Int I2]) .
  rl o C[! true] => plug(C[false]) .
  rl o C[! false] => plug(C[ true]) .
  rl o C[true && B2] => plug(C[B2]) .
  rl o C[false && B2] => plug(C[false]) .
  rl o C[{S}] => plug(C[S]) .
 crl o < C,Sigma >[X = I ;] => plug(< C,Sigma[I / X] >[{}])
  if Sigma(X) =/=Bool undefined .
  rl o C[{} S2] => plug(C[S2]) .
  rl o C[if (true) S1 else S2] => plug(C[S1]) .
  rl o C[if (false) S1 else S2] => plug(C[S2]) .
  rl o C[while (B) S] => plug(C[if (B) {S while (B) S} else {}]) .
  rl o < int Xl ; S > => < S,(Xl |-> 0) > .
endm
```

Figure 3.42: The first two reduction semantics with evaluation contexts of IMP in Maude.

```
mod IMP-SEMANTICS-EVALUATION-CONTEXTS is including IMP-SPLIT-PLUG-EVALUATION-CONTEXTS .
  var X : Id .  var I I1 I2 : Int .  var B B2 : BExp .  var S S1 S2 : Stmt .  var Xl : List{Id} .
  var Sigma : State .  var Cfg Cfg' : Configuration .  var Syn Syn' : Syntax  .  var C : Context .

  crl o Cfg => Cfg' if plug(o split(Cfg)) => Cfg' .           --- generic rule enabling splitting
  crl o C[Syn] => C[Syn'] if C =/=Bool [] /\ o Syn => Syn' .  --- characteristic rule

 crl o < C,Sigma >[X] => < C,Sigma >[Sigma(X)]
  if Sigma(X) =/=Bool undefined .
  rl o I1 + I2 => I1 +Int I2 .
 crl o I1 / I2 => I1 /Int I2
  if I2 =/=Bool 0 .
  rl o I1 <= I2 => I1 <=Int I2 .
  rl o ! true => false .
  rl o ! false => true  .
  rl o true && B2 => B2 .
  rl o false && B2 => false .
  rl o {S} => S .
 crl o < C,Sigma >[X = I ;] => < C,Sigma[I / X] >[{}]
  if Sigma(X) =/=Bool undefined .
  rl o {} S2 => S2 .
  rl o if (true) S1 else S2 => S1 .
  rl o if (false) S1 else S2 => S2 .
  rl o while (B) S => if (B) {S while (B) S} else {} .
  rl o < int Xl ; S > => < S,(Xl |-> 0) > .
endm
```

Figure 3.43: The third reduction semantics with evaluation contexts of IMP in Maude.

```
search * < sumPgm > =>! Cfg:ExtendedConfiguration .
```

As expected, only one behavior will be discovered because our IMP language so far is deterministic. Not unexpectedly, the same number of states as in the case of small-step SOS and MSOS will be discovered by this search command, namely 1709. Indeed, the splitting/cooling mechanism of RSEC is just another way to find where the next reduction step should take place; it does not generate any different reductions of the original configuration.

3.7.4 Notes

Reduction semantics with evaluation contexts was introduced by Felleisen and his collaborators (see, e.g., [25, 88]) as a variant small-step structural operational semantics. By making the evaluation context explicit and modifiable, reduction semantics with evaluation contexts is considered by many to be a significant improvement over small-step SOS. Like small-step SOS, reduction semantics with evaluation contexts has been broadly used to give semantics to programming languages and to various calculi. We here only briefly mention some strictly related work.

How expensive is the splitting of a term into an evaluation context and a redex? Unfortunately, it cannot be more efficient than testing the membership of a word to a context-free grammar and the latter is expected to be cubic in the size of the original term (folklore). Indeed, consider G an arbitrary CFG whose start symbol is S and let G_C be the "evaluation context" CFG grammar adding a fresh "context" nonterminal C, a fresh terminal #, and productions $C \rightarrow \square \mid CS\#$. Then it is easy to see that a word α is in the language of G if and only if $\#\alpha\#$ can be split as a contextual representation (can only be $(\square\alpha\#)[\#]$). Thus, we should expect, in the worst case, a *cubic* complexity to split a term into an evaluation context and a redex. An additional exponent needs to be added, thus making splitting expected to be a *quadratic* operation in the worst case, when nested contexts are allowed in rules (i.e., when the redex is itself a contextual representation). Unfortunately, this terrible complexity needs to be paid at each step of reduction, not to mention that the size of the program to reduce can also grow as it is reduced. One possibility to decrease this complexity is to attempt to incrementally compute at each step the evaluation context that is needed at the next step (like in refocusing; see below); however, in the worst case the right-hand-sides of rules may contain no contexts, in which case a fresh split is necessary at each step.

Besides our own efforts, we are aware of three other attempts to develop executable engines for reduction semantics with evaluation contexts, which we discuss here in chronological order:

1. A specification language for syntactic theories with evaluation contexts is proposed by Xiao *et al.* [90, 89], together with a system which generates Ocaml interpreters from specifications. Although the compiler in [90, 89] is carefully engineered, as rightfully noticed by Danvy and Nielsen in [20] it cannot avoid the quadratic overhead due to the context-decomposition step. This is consistent with our own observations expressed at several places in this section, namely that the advanced parsing underlying reduction semantics with evaluation contexts is the most expensive part when one is concerned with execution. Fortunately, the splitting of syntax into context and redex can be and typically is taken for granted in theoretical developments, making abstraction of the complexity of its implementation.

2. A technique called *refocusing* is proposed by Danvy and Nielsen in [20, 19]. The idea underlying refocusing is to keep the program decomposed at all times (in a first-order continuation-like form) and to perform minimal changes to the resulting structure to find the next redex. Unfortunately, refocusing appears to work well only with restricted RSEC definitions, namely ones whose evaluation contexts grammar has the property of unique decomposition of a term into a context and a redex (so constructs like the non-deterministic addition of IMP are disallowed), and whose reduction rules are deterministic.

3. PLT-Redex, which is implemented in Scheme by Findler and his collaborators [38, 24], is perhaps the most advanced tool developed specifically to execute reduction semantics with evaluation contexts. PLT-Redex builds upon a direct implementation of context-sensitive reduction, so it cannot avoid the worst-case quadratic complexity of context decomposition, same as the interpreters generated by the system in [90, 89] discussed above. Several large language semantics engineering case studies using PLT-Redex are discussed in [24].

Our embeddings of reduction semantics with evaluation contexts into rewrite logic are inspired from a related embedding by Șerbănuță *et al.* in [74]. The embedding in [74] was similar to our third embedding here, but it included splitting rules also for terms in reducible form, e.g., $split(I_1 <= I_2) \to \square[I_1 <= I_2]$. Instead, we preferred to include a generic rule $split(Syn) \to \square[Syn]$ here, which allows us to more mechanically derive the rewrite rules for splitting from the CFG of evaluation contexts. Calculating the exact complexity of our approach seems to be hard, mainly because of optimizations employed by rewrite engines, e.g., indexing. Since at each step we still search for all the relevant splits of the term into an evaluation context and a redex, in the worst case we still pay the quadratic complexity. However, as suggested by the performance numbers in [74] comparing Maude running the resulting rewrite theory against PLT-Redex, which favor the former by a large margin, our embeddings may serve as alternative means to getting more efficient implementations of reduction semantics engines. There are strong reasons to believe that our third embedding can easily be automated in a way that the user never sees the split/plug operations.

3.7.5 Exercises

Exercise 127. *Suppose that one does not like mixing semantic components with syntactic evaluation contexts as we did above (by including the production* $Context ::= \langle Context, State \rangle$*). Instead, suppose that one prefers to work with configuration tuples like in SOS, holding the various components needed for the language semantics, the program or fragment of program being just one of them. In other words, suppose that one wants to make use of the contextual representation notation only on the syntactic component of configurations. In this case, the characteristic rule becomes*

$$\frac{\langle e, \gamma \rangle \to \langle e', \gamma' \rangle}{\langle c[e], \gamma \rangle \to \langle c[e'], \gamma' \rangle}$$

where γ *and* γ' *consist of configuration semantic components that are necessary to evaluate* e *and* e'*, respectively, such as states, outputs, stacks, etc. Modify accordingly the six reduction semantics with evaluation contexts rules discussed at the beginning of Section 3.7.*

The advantage of this approach is that it allows the evaluation contexts to be defined exclusively over the syntax of the language. However, configurations holding code and state still need to be defined. Moreover, many rules which looked compact before, such as $i_1 <= i_2 \to i_1 \leq_{Int} i_2$*, will now look heavier, e.g.,* $\langle i_1 <= i_2, \sigma \rangle \to \langle i_1 \leq_{Int} i_2, \sigma \rangle$*.*

Exercise 128. *Like in Exercise 127, suppose that one does not like to mix syntactic and semantic components in evaluation contexts, but that, instead, one is willing to accept to slightly enrich the syntax of the programming language with a special statement construct "Stmt ::=* `int` *Id = AExp;" which both declares and initializes a variable*[10]*. Then*

1. *Write structural identities that desugar the current top-level program variable declarations* `int` $x_1, ..., x_n$ `;` s *into statements of the form* `int` $x_1 = 0$ `;` ... `int` $x_n = 0$ `;` s*.*

[10] Similar language constructs exist in many programming language (C, Java, etc.).

2. *Add a new context production that allows evaluation after the new variable declarations.*

3. *Modify the variable lookup and assignment rules discussed above so that one uses the new declarations instead of a state. Hint: the context should have the form "*`int` *x* `=` *i*`;` *c*".*

The advantage of this approach is that one does not need an explicit state anymore, so the resulting definition is purely syntactic. In fact, the state is there anyway, but encoded syntactically as a sequence of variable initializations preceding any other statement. This trick works in this case, but it cannot be used as a general principle to eliminate configurations in complex languages.

Exercise⋆ 129. *Exercise 127 suggests that one can combine MSOS (Section 3.6) and evaluation contexts, in that one can use MSOS's labels to obtain modularity at the configuration level and one can use the evaluation contexts idea to detect and modify the contexts/redexes in the syntactic component of a configuration. Rewrite the six rules discussed at the beginning of Section 3.7 as they would appear in a hypothetical framework merging MSOS and evaluation contexts.*

Exercise 130. *Modify the reduction semantics with evaluation contexts of* IMP *in Figures 3.30 and 3.31 so that* `/` *short-circuits when its numerator evaluates to* 0.
Hint: Make `/` *strict in only the first argument, then use a rule to reduce* 0 `/` a_2 *to 0 and a rule to reduce* i_1 `/` a_2 *to* i_1 `/'` a_2 *when* $i_1 \neq 0$*, where* `/'` *is strict in its second argument, and finally a rule to reduce* i_1 `/'` i_2 *to* $i_1 /_{Int} i_2$ *when* $i_2 \neq 0$.

Exercise 131. *Modify the reduction semantics with evaluation contexts of* IMP *in Figures 3.30 and 3.31 so that conjunction is not short-circuited anymore but, instead, is non-deterministically strict in both its arguments.*

Exercise 132. *Give an alternative reduction semantics of* IMP *with evaluation contexts following the approach in Exercise 127 (that is, use evaluation contexts only for the* IMP *language syntax, and handle the semantic components using configurations, like in SOS).*

Exercise 133. *Give an alternative reduction semantics of* IMP *with evaluation contexts following the approach in Exercise 128.*

Exercise⋆ 134. *Give a semantics of* IMP *using the hypothetical framework combining reduction semantics with evaluation contexts and MSOS proposed in Exercise 129.*

Exercise 135. *Modify the rewrite theory* $\mathcal{R}^{\square}_{\text{RSEC(IMP)}}$ *in Figure 3.37 so that later on one can define the reduction semantics of* `/` *to short-circuit when the numerator evaluates to* 0 *(as required in Exercises 137, 143, and 149).*

Exercise 136. *Modify the rewrite theory* $\mathcal{R}^{\square}_{\text{RSEC(IMP)}}$ *in Figure 3.37 so that one can later on define the reduction semantics of conjunction to be non-deterministically strict in both its arguments (as required in Exercises 138, 144, and 150).*

Exercise 137. *Modify the rewrite theory* $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$ *in Figure 3.38 to account for the reduction semantics of* `/` *that short-circuits when the numerator evaluates to* 0 *(see also Exercise 135).*

Exercise 138. *Modify the rewrite theory* $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$ *in Figure 3.38 to account for the reduction semantics of conjunction that defines it as non-deterministically strict in both its arguments (see also Exercise 136).*

Exercise 139. *As discussed in several places so far in Section 3.7, the reduction semantics rules for variable lookup and assignment can also be given in a way in which their left-hand-side terms are not in contextual representation (i.e.,* $\langle c[x], \sigma\rangle$ *instead of* $\langle c, \sigma\rangle[x]$*, etc.). Modify the corresponding rewrite rules of* $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$ *in Figure 3.38 to account for this alternative reduction semantics.*

Exercise 140. *Modify the rewrite logic theory* $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$ *in Figure 3.38 to account for the alternative reduction semantics with evaluation contexts of* IMP *in Exercise 132.*

Exercise 141. *Modify the rewrite logic theory* $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$ *in Figure 3.38 to account for the alternative reduction semantics with evaluation contexts of* IMP *in Exercise 133.*

Exercise⋆ 142. *Combining the underlying ideas of the embedding of MSOS in rewrite logic discussed in Section 3.6.3 and the embedding of reduction semantics with evaluation contexts in Figure 3.33, give a rewrite logic semantics of* IMP *corresponding to the semantics of* IMP *in Exercise 134.*

Exercise 143. *Same as Exercise 137, but for* $\mathcal{R}^{\boxed{2}}_{\text{RSEC(IMP)}}$ *in Figure 3.39 (instead of* $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$*).*

Exercise 144. *Same as Exercise 138, but for* $\mathcal{R}^{\boxed{2}}_{\text{RSEC(IMP)}}$ *in Figure 3.39 (instead of* $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$*).*

Exercise 145. *Same as Exercise 139, but for* $\mathcal{R}^{\boxed{2}}_{\text{RSEC(IMP)}}$ *in Figure 3.39 (instead of* $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$*).*

Exercise 146. *Same as Exercise 140, but for* $\mathcal{R}^{\boxed{2}}_{\text{RSEC(IMP)}}$ *in Figure 3.39 (instead of* $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$*).*

Exercise 147. *Same as Exercise 141, but for* $\mathcal{R}^{\boxed{2}}_{\text{RSEC(IMP)}}$ *in Figure 3.39 (instead of* $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$*).*

Exercise⋆ 148. *Same as Exercise 142, but for Figure 3.39 (instead of Figure 3.38).*

Exercise 149. *Same as Exercise 137, but for* $\mathcal{R}^{\boxed{3}}_{\text{RSEC(IMP)}}$ *in Figure 3.40 (instead of* $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$*).*

Exercise 150. *Same as Exercise 138, but for* $\mathcal{R}^{\boxed{3}}_{\text{RSEC(IMP)}}$ *in Figure 3.40 (instead of* $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$*).*

Exercise 151. *Same as Exercise 139, but for* $\mathcal{R}^{\boxed{3}}_{\text{RSEC(IMP)}}$ *in Figure 3.40 (instead of* $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$*).*

Exercise 152. *Same as Exercise 140, but for* $\mathcal{R}^{\boxed{3}}_{\text{RSEC(IMP)}}$ *in Figure 3.40 (instead of* $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$*).*

Exercise 153. *Same as Exercise 141, but for* $\mathcal{R}^{\boxed{3}}_{\text{RSEC(IMP)}}$ *in Figure 3.40 (instead of* $\mathcal{R}^{\boxed{1}}_{\text{RSEC(IMP)}}$*).*

Exercise⋆ 154. *Same as Exercise 142, but for Figure 3.40 (instead of Figure 3.38).*

Exercise 155. *Modify the Maude code in Figures 3.41 and 3.42, 3.43 so that / short-circuits when its numerator evaluates to* 0 *(see also Exercises 130, 135, 137, 143, and 149).*

Exercise 156. *Modify the Maude code in Figures 3.41 and 3.42, 3.43 so that conjunction is not short-circuited anymore but, instead, is non-deterministically strict in both its arguments (see also Exercises 131, 136, 138, 144, and 150).*

Exercise 157. *Modify the Maude code in Figures 3.41 and 3.42, 3.43 to account for the alternative reduction semantics in Exercises 139, 145, and 151.*

Exercise 158. *Modify the Maude code in Figures 3.41 and 3.42, 3.43 to account for the alternative reduction semantics in Exercises 140, 146, and 152.*

Exercise 159. *Modify the Maude code in Figures 3.41 and 3.42, 3.43 to account for the alternative reduction semantics in Exercises 141, 147, and 153.*

Exercise⋆ 160. *Modify the Maude code in Figures 3.41 and 3.42, 3.43 to account for the semantics in Exercises 142, 148, and 154.*

Exercise 161. *Same as Exercise 86, but for reduction semantics with evaluation contexts instead of small-step SOS: add variable increment to* IMP*, like in Section 3.7.2.*

Exercise 162. *Same as Exercise 90, but for reduction semantics with evaluation contexts instead of small-step SOS: add input/output to* IMP*, like in Section 3.7.2.*

Exercise⋆ 163. *Consider the hypothetical framework combining MSOS with reduction semantics with evaluation contexts proposed in Exercise 129, and in particular the* IMP *semantics in such a framework in Exercise 134, its rewrite logic embeddings in Exercises 142, 148, and 154, and their Maude implementation in Exercise 160. Define the semantics of the input/output constructs above modularly first in the framework in discussion, then using the rewrite logic embeddings, and finally in Maude.*

Exercise 164. *Same as Exercise 95, but for reduction semantics with evaluation contexts instead of small-step SOS: add abrupt termination to* IMP*, like in Section 3.7.2.*

Exercise 165. *Same as Exercise 104, but for reduction semantics with evaluation contexts instead of small-step SOS: add dynamic threads to* IMP*, like in Section 3.7.2.*

Exercise 166. *Same as Exercise 109, but for reduction semantics with evaluation contexts instead of small-step SOS: add local variables using* `let` *to* IMP*, like in Section 3.7.2.*

Exercise⋆ 167. *This exercise asks to define* IMP++ *in reduction semantics, in various ways. Specifically, redo Exercises 114, 115, 116, 117, and 118, but for the reduction semantics with evaluation contexts of* IMP++ *discussed in Section 3.7.2 instead of its small-step SOS in Section 3.5.6.*

Try to get rid of kinds in the Maude implementation, so that I don't need to introduce them in the EL, RL and Maude sections.

---

should I include the paragraph below?

Like for the first embedding, one can now use a context-insensitive rewrite engine, in particular its search capabilities, to explore computations in the original context reduction definition. For example, if one is interested in listing all terms that are reachable from a given term t using the original context reduction definition, then all one needs to do is to invoke a search procedure on the term $\{split(t)\}$ *and limit its responses to those matching the pattern* $\{\Box[Syntax]\}$. *Moreover, if one is only interested in the values that a program t can reduce to, then, assuming that one defined a syntactic category Value, all one needs to do is to filter the results to those matching the pattern* $\{\Box[Value]\}$. *Even more interestingly, if one is interested in all those reachable configurations in which* $x = 0$ *and* $y = z$, *then, assuming that State is defined as a comma-separated set of pairs variable* $\mapsto$ *value, all one needs to do is to limit the results of the search to those terms matching the pattern* $\{\Box[\langle Syntax, (x \mapsto 0, y \mapsto v, z \mapsto v, State)\rangle]\}$. *However, unlike for the first embedding, with this second embedding one cannot blindly let the rewrite engine execute context reduction definitions. That is because the rewrite engine may choose to stop the rewriting of the term* $\{split(t)\}$ *after each application of a rule of the form* $rule_i$, *by choosing to apply the rule* $split(N) \to \Box[N]$ *(see Figure* **??***).*

---

should we have a more indepth discussion on the complexity of the various techniques?

Narciso: it would be good to add numbers comparing plt-redex with Maude on context reduction definitions.

G: it would be good to add numbers everywhere, in all semantic styles, comparing the various realizations in various languages.

---

make sure the exercise scheme is consistent everywhere; for example, there is no exercise on generating error at div by 0 in RSEC, and there are no exercises in the RL and Maude subsections of the previous semantics

---

make the point that rsec separates concerns when compared to sos: it separates defining evaluation strategies from the actual reduction steps

---

3.8 The Chemical Abstract Machine (CHAM)

The *chemical abstract machine*, or the *CHAM*, is both a model of concurrency and a specific operational semantics style. The states of a CHAM are metaphorically regarded as chemical solutions formed with floating molecules. Molecules can interact with each other by means of reactions. A reaction can involve several molecules and can change them, delete them, and/or create new molecules. One of the most appealing aspects of the chemical abstract machine is that its reactions can take place concurrently, unrestricted by context. To facilitate local computation and to represent complex data-structures, molecules can be nested by encapsulating groups of molecules as sub-solutions. The chemical abstract machine was proposed as an alternative to SOS and its variants, including reduction semantics with evaluation contexts, in an attempt to circumvent their limitations, particularly their lack of support for true concurrency.

CHAM Syntax

The *basic molecules* of a CHAM are ordinary algebraic terms over a user-defined syntax. Several molecules wrapped within a membrane form a *solution*, which is also a molecule. The CHAM uses the symbols $\{\!|$ and $|\!\}$ to denote membranes. For example, $\{\!|m_1\ m_2\ \ldots\ m_k|\!\}$ is a solution formed with the molecules $m_1, m_2, \ldots, m_k$. The order of molecules in a solution is irrelevant, so a solution can be regarded as a multi-set (or bag) of molecules wrapped within a membrane. Since solutions are themselves molecules, we can have arbitrarily nested molecules. This nesting mechanism is generic for all CHAMs and has the following (algebraic) syntax:

$$\begin{array}{rcl} \mathit{Molecule} & ::= & \mathit{Solution} \mid \mathit{Molecule} \triangleleft \mathit{Solution} \\ \mathit{Solution} & ::= & \{\!|\mathbf{Bag}\{\mathit{Molecule}\}|\!\} \end{array}$$

The operator $_ \triangleleft _$ is called the *airlock* operator and will be discussed shortly (under general CHAM laws), after we discuss the CHAM rules. When defining a CHAM, one is only allowed to extend the syntax of molecules, which implicitly also extends the syntax that the solution terms can use. However, one is not allowed to explicitly extend the syntax of solutions. In other words, solutions can only be built using the generic syntax above, on top of user-defined syntactic extensions of molecules. Even though we do not formalize it here (and we are not aware of other formulations elsewhere either), it is understood that one can have multiple types of molecules in a CHAM.

Specific CHAM Rules

In addition to extending the syntax of molecules, a CHAM typically also defines a set of *rules*, each rule being a rule schemata but called a rule for simplicity. A CHAM rule has the form

$$m_1\ m_2\ \ldots\ m_k \rightarrow m'_1\ m'_2\ \ldots m'_l$$

where $m_1, m_2, \ldots, m_k$ and $m'_1, m'_2, \ldots, m'_l$ are not necessarily distinct molecules (since CHAM rules are schemata, these molecule terms may contain meta-variables). Molecules appearing in a rule are restricted to contain only subsolution terms which are either solution meta-variables or otherwise have the form $\{\!|m|\!\}$, where m is some molecule term. For example, a CHAM rule cannot contain subsolution terms of the form $\{\!|m\ s|\!\}$, $\{\!|m_1\ m_2|\!\}$, or $\{\!|m_1\ m_2\ s|\!\}$, with m, m_1, m_2 molecule terms and s solution term, but it can contain ones of the form $\{\!|m|\!\}$, $\{\!|m_1 \triangleleft \{\!|m_2|\!\}|\!\}$, $\{\!|m \triangleleft s|\!\}$, etc. This restriction is justified by chemical intuitions, namely that matching inside a solution is a rather complex operation which needs special handling (the airlock operator $_ \triangleleft _$ is used for this purpose). Note that CHAM rules are unconditional, that is, they have no premises.

General CHAM Laws

Any chemical abstract machine obeys the four laws below. Let CHAM be[11] a chemical abstract machine. Below we assume that mol, mol', mol_1, etc., are arbitrary concrete molecules of CHAM (i.e., no meta-variables) and that sol, sol', etc., are concrete solutions of it. If sol is the solution $\{\!|mol_1\ mol_2\ \ldots\ mol_k|\!\}$ and sol' is the solution $\{\!|mol'_1\ mol'_2\ \ldots\ mol'_l|\!\}$, then $sol \uplus sol'$ is the solution $\{\!|mol_1\ mol_2\ \ldots\ mol_k\ mol'_1\ mol'_2\ \ldots\ mol'_l|\!\}$.

1. ***The Reaction Law.*** Given a CHAM rule

$$m_1\ m_2\ \ldots\ m_k \rightarrow m'_1\ m'_2\ \ldots m'_l \quad \in \quad \text{CHAM}$$

   if $mol_1, mol_2, \ldots, mol_k$ and $mol'_1, mol'_2, \ldots, mol'_l$ are (concrete) instances of $m_1\ m_2\ \ldots\ m_k$ and of $m'_1\ m'_2\ \ldots m'_l$ by a common substitution, respectively, then

$$\text{CHAM} \vdash \{\!|mol_1\ mol_2\ \ldots\ mol_k|\!\} \rightarrow \{\!|mol'_1\ mol'_2\ \ldots mol'_l|\!\}$$

2. ***The Chemical Law.*** Reactions can be performed freely within any solution:

$$\frac{\text{CHAM} \vdash sol \rightarrow sol'}{\text{CHAM} \vdash sol \uplus sol'' \rightarrow sol' \uplus sol''}$$

3. ***The Membrane Law.*** A subsolution can evolve freely in any solution context $\{\!|cxt[\square]|\!\}$:

$$\frac{\text{CHAM} \vdash sol \rightarrow sol'}{\text{CHAM} \vdash \{\!|cxt[sol]|\!\} \rightarrow \{\!|cxt[sol']|\!\}}$$

4. ***The Airlock Law.***

$$\text{CHAM} \vdash \{\!|mol|\!\} \uplus sol \leftrightarrow \{\!|mol \triangleleft sol|\!\}$$

Note the unusual fact that $m_1\ m_2\ \ldots\ m_k \rightarrow m'_1\ m'_2\ \ldots m'_l$ being a rule in CHAM does not imply that $\text{CHAM} \vdash m_1\ m_2\ \ldots\ m_k \rightarrow m'_1\ m'_2\ \ldots m'_l$. Indeed, the CHAM rules are regarded as descriptors of changes that can take place in solutions and only in solutions, while CHAM sequents are incarnations of those otherwise purely abstract rules. What may be confusing is that the same applies also when k and l (the numbers of molecules in the left-hand and right-hand-sides of the CHAM rule) happen to be 1 and m_1 and m'_1 happen to be solutions that contain no meta-variables. The two $\rightarrow$ arrows, namely the one in CHAM rules and the one in CHAM sequents, ought to be different symbols; however, we adhere to the conventional CHAM notation which uses the same symbol for both. Moreover, when the CHAM is clear from context, we also follow the conventional notation and drop it from sequents, that is, we write $sol \rightarrow sol'$ instead of $\text{CHAM} \vdash sol \rightarrow sol'$. While we admit that these conventions may sometimes be confusing, in that $sol \rightarrow sol'$ can be a rule or a sequent or even both, we hope that the context makes it clear which one is meant.

The Reaction Law says that CHAM rules can only apply in solutions (wrapped by a membrane), and not arbitrarily wherever they match. The Chemical Law says that once a reaction take place in a certain solution, it can take place in any other larger solution. In other words, the fact that a solution has more molecules than required by the rule does not prohibit the rule from applying. The Reaction and the Chemical laws together say that CHAM rules can apply inside any solutions having some molecules that match the left-had side

[11] To avoid inventing new names, it is common to use CHAM both as an abbreviation for "the chemical abstract machine" and as a name of an arbitrary but fixed chemical abstract machine.

of the CHAM rule. An interesting case is when the left-hand-side term of the CHAM rule has only one molecule, i.e., when $k = 1$, because the CHAM rule is still allowed to only apply within a solution; it cannot apply in other places where the left-hand-side happens to match.

The Membrane Law says that reactions can take place in any solution context. Indeed, $\{\!|cxt[sol]|\!\}$ says that the solution *sol* (which is wrapped in a membrane) appears somewhere, anywhere, inside a solution context $\{\!|cxt[\square]|\!\}$. Here *cxt* can be any bag-of-molecule context, and we write $cxt[\square]$ to highlight the fact that it is a context with a hole $\square$. By wrapping $cxt[\square]$ in a membrane we enforce a solution context. This rule also suggests that, at any given moment, the global term to rewrite using the CHAM rules should be a solution. Indeed, the CHAM rewriting process gets stuck as soon as the term becomes a proper molecule (not a solution), because the Membrane Law cannot apply.

The Airlock Law is reversible (i.e., it comprises two rewrite rules, one from left-to-right and one from right-to-left) and it allows to extract a molecule from a solution, putting the rest of the solution within a membrane. Using this law one can, for example, rewrite a solution $\{\!|mol_1\ mol_2\ \ldots, mol_k|\!\}$ into $\{\!|mol_1 \triangleleft \{\!|mol_2\ \ldots\ mol_k|\!\}|\!\}$. The advantage of doing so is that one can now match the molecule mol_1 within other rules. Indeed, recall that sub-solutions that appear in rules cannot specify any particular molecule term among the rest of the solution, unless the solution contains precisely that molecule. Since $mol_1 \triangleleft \{\!|mol_2\ \ldots\ mol_k|\!\}$ is a molecule, $\{\!|mol_1 \triangleleft \{\!|mol_2\ \ldots\ mol_k|\!\}|\!\}$ can match molecule terms of the form $\{\!|m \triangleleft s|\!\}$ appearing in CHAM rules, this way one effectively matching (and possibly modifying) the molecule mol_1 via the specific CHAM rules. The Airlock Law is the only means provided by the CHAM to extract or put molecules in a solution.

The four laws above do not completely define the CHAM rewriting; they are only properties that the CHAM rewriting should satisfy. In particular, they do not capture the concurrency potential of the CHAM.

Definition 23. *The four laws above give us a proof system for CHAM sequents. As usual,* $\text{CHAM} \vdash sol \rightarrow sol'$ *in isolation means mean that it is derivable. Also, let* $\rightarrow^*$ *denote the reflexive and transitive closure of* $\rightarrow$*, that is,* $\text{CHAM} \vdash sol \rightarrow^* sol'$ *iff* $sol = sol'$ *or there is some* sol'' *such that* $\text{CHAM} \vdash sol \rightarrow sol''$ *and* $\text{CHAM} \vdash sol'' \rightarrow^* sol'$*. Finally,* $\text{CHAM} \vdash sol \leftrightarrow sol'$ *is a shorthand for the seqents* $\text{CHAM} \vdash sol \rightarrow sol'$ *and* $\text{CHAM} \vdash sol' \rightarrow sol$*, and we say that it is derivable iff the two sequents are derivable.*

None of the two sequents in Definition 23 captures the underlying concurrent computation of the CHAM. Indeed, $\text{CHAM} \vdash sol \rightarrow sol'$ says that one and only one reaction takes place somewhere in *sol*, while $\text{CHAM} \vdash sol \rightarrow^* sol'$ says that arbitrarily many steps take place, including ones which can be done concurrently but also ones which can only take place sequentially. Therefore, we can think of the four laws above, and implicitly of the sequents $\text{CHAM} \vdash sol \rightarrow sol'$ and $\text{CHAM} \vdash sol \rightarrow^* sol'$, as expressing the descriptive capability of the CHAM: what is possible and what is not possible to compute using the CHAM, and not how it operates. Nevertheless, it is recommended to think of CHAM reactions as taking place concurrently whenever they do not involve the same molecules, even though this "concurrent reaction" notion is not formalized here. We are actually not aware of any works that formalize the CHAM concurrency.

A common source of misunderstanding the CHAM is to wrongly think of CHAM rules as ordinary rewriting rules modulo the associativity, commutativity and identity of the molecule grouping (inside a solution) operation. The major distinction between CHAM rules and such rewrite rules is that the former only apply within solutions (i.e., wrapped by membranes) no matter whether the rule contains one or more molecules in its left-hand or right-hand terms, while the latter apply anywhere they match. For example, supposing that we extend the syntax of molecules with the syntax of IMP in Section 3.1.1 and add a CHAM rule $m + 0 \rightarrow m$, then we can rewrite the solution $\{\!|(3 + 0)\ 7|\!\}$ to solution $\{\!|3\ 7|\!\}$, but we cannot rewrite the molecule $5 / (3 + 0)$ to molecule $5 / 3$ regardless of what context it is in, because $3 / 0$ is not in a solution. We cannot even rewrite the isolated (i.e., not in a solution context) term $3 + 0$ to 3 in CHAM, for the same reason.

Classification of CHAM Rules

The rules of a CHAM are typically partitioned into three intuitive categories, namely *heating*, *cooling* and *reaction* rules, although there are no formal requirements imposing a rule to be into one category or another. Moreover, the same laws discussed above apply the same way to all categories of rules, and the same restrictions preventing multiset matching apply to all of them.

- *Heating* rules, distinguished by using the relation symbol $\rightharpoonup$ instead of $\rightarrow$, are used to structurally rearrange the solution so that reactions can take place.
- *Cooling* rules, distinguished by using the relation symbol $\rightharpoondown$ instead of $\rightarrow$, are used after reactions take place to structurally rearrange the solution back into a convenient form, including to remove useless molecules or parts of them.
- *Reaction* rules, which capture the intended computational steps and use the conventional rewrite symbol $\rightarrow$, are used to evolve the solution in an irreversible way.

The heating and cooling rules can typically be paired, with each heating rule $l \rightharpoonup r$ having a symmetric cooling rule $l \leftharpoondown r$, so that we can view them as a single bidirectional *heating/cooling* rule. The CHAM notation for writing such heating/cooling rules is the following:

$$l \rightleftharpoons r$$

In particular, it makes sense to regard the airlock axiom as an example of such a heating/cooling bidirectional rule, that is,

$$\{\!\!\{m_1\ m_2\ \ldots\ m_k\}\!\!\} \rightleftharpoons \{\!\!\{m_1 \triangleleft \{\!\!\{m_2\ \ldots\ m_k\}\!\!\}\}\!\!\}$$

where $m_1, m_2, \ldots, m_k$ are molecule meta-variables. The intuition here is that we can heat the solution to extract m_1 in an airlock, or we can cool it down so that the airlock m_1 is diffused within the solution. However, we need to assume one such rule for each $k > 0$.

As one may expect, the reaction rules are the heart of the CHAM and properly correspond to state transitions. The heating and cooling rules express *structural rearrangements*, so that the reaction rules can match and apply. In other words, we can view the reaction rules as being applied *modulo* the heating and cooling rules. We are going to suggestively use the notation CHAM $\vdash sol \rightharpoonup sol'$, respectively CHAM $\vdash sol \rightharpoondown sol'$ whenever the rewrite step taking sol to sol' is a heating rule, respectively a cooling rule. Similarly, we may use the notations CHAM $\vdash sol \rightharpoonup^* sol'$ and CHAM $\vdash sol \rightharpoondown^* sol'$ for the corresponding reflexive/transitive closures. Also, to emphasize the fact that there is only one reaction rule applied, we take the freedom to (admittedly ambiguously) write CHAM $\vdash sol \rightarrow sol'$ instead of CHAM $\vdash sol(\rightarrow \cup \rightharpoonup \cup \rightharpoondown)^* sol'$ whenever all the involved rules but one are heating or cooling rules.

3.8.1 The CHAM of IMP

We next show how to give IMP a CHAM semantics. CHAM is particularly well-suited to giving semantics to concurrent distributed calculi and languages, yielding considerably simpler definitions than those afforded by SOS. Since IMP is sequential, it cannot take full advantage of the CHAM's true concurrency capabilities; the multi-threaded IMP++ language discussed in Section 3.5 will make better use of CHAM's capabilities. Nevertheless, some of CHAM's capabilities turn out to be useful even in this sequential language application, others turn out to be deceiving. Our CHAM semantics for IMP below follows in principle the reduction semantics with evaluation contexts definition discussed in Section 3.7.1. One can formally show that a step

performed using reduction under evaluation contexts is equivalent to a suite of heating steps, followed by one reaction step, followed by a suite of cooling steps.

The CHAM defined below is just one possible way to give IMP a CHAM semantics. CHAM, like rewriting, is a general framework which does not impose upon its users any particular definitional style. In our case, we chose to conceptually distinguish two types of molecules; we say "conceptually" because, for simplicity, we prefer to define only one *Molecule* syntactic category in our CHAM:

- *Syntactic molecules*, which include all the syntax of IMP in Section 3.1.1, plus all the syntax of its evaluation contexts in Section 3.7.1, plus a mechanism to flatten evaluation contexts; for simplicity, we prefer to not include a distinct type of molecule for each distinct syntactic category of IMP.
- *State molecules*, which are pairs $x \mapsto i$, where $x \in Id$ and $i \in Int$.

For clarity, we prefer to keep the syntactic and the state molecules in separate solutions. More precisely, we work with top-level configurations which are solutions of the form

$$\{\!\{\mathit{Syntax}\}\!\}\ \ \{\!\{\mathit{State}\}\!\}$$

Syntax and *State* are solutions containing syntactic and state molecules, respectively. For example,

$$\{\!\{x = 3\ /\ (x + 2)\,\mathbf{;}\}\!\}\ \{\!\{x \mapsto 1\ \ y \mapsto 0\}\!\}$$

is a CHAM configuration containing the statement "$x = 3\ /\ (x + 2)\,\mathbf{;}$" and state "$x \mapsto 1, y \mapsto 0$".

The state molecules and implicitly the state solution are straightforward. State molecules are not nested and state solutions are simply multisets of molecules of the form $x \mapsto i$. The CHAM does not allow us to impose constraints on solutions, such as that the molecules inside the state solution indeed define a partial function and not some arbitrary relation (i.e., there is at most one molecule $x \mapsto i$ for each $x \in Id$). Instead, the state solution will be used in such a way that the original state solution will embed a proper partial function and each rule will preserve this property. For example, the CHAM rule for variable assignment, say when assigning integer i to variable x, will rewrite the state molecule from $\{\!\{x \mapsto j \triangleleft \sigma\}\!\}$ to $\{\!\{x \mapsto i \triangleleft \sigma\}\!\}$.

The top level syntactic solution holds the current program or fragment of program that is still left to be processed. It is not immediately clear how the syntactic solution should be represented in order to be able to give IMP a CHAM semantics. The challenge here is that the IMP language constructs have evaluation strategies and the subterms that need to be next processed can be arbitrarily deep into the program or fragment of program, such as the framed x in "$x = 3\ /\ (\boxed{x} + 2)\,\mathbf{;}$". If the CHAM allowed conditional rules, then we could have followed an approach similar to that of SOS described in Section 3.3, reducing the semantics of each language construct to that of its subexpressions or substatements. Similarly, if the CHAM allowed matching using evaluation contexts, then we could have followed a reduction semantics with evaluation contexts approach like the one in Section 3.7.1 which uses only unconditional rules. Unfortunately, the CHAM allows neither conditional rules nor evaluation contexts in matching, so a different approach is needed.

Failed attempts to represent syntax. A natural approach to represent the syntax of a programming language in CHAM may be to try to use CHAM's heating/cooling and molecule/solution nesting mechanisms to decompose syntax unambiguously in such a way that the redex (i.e., the subterm which can be potentially reduced next; see Section 3.7) appears as a molecule in the top syntactic solution. That is, if $p = c[t]$ is a program or fragment of program which can be decomposed in evaluation context c and redex t, then one may attempt to represent it as a solution of the form $\{\!\{t\ \gamma_c\}\!\}$, where γ_c is some CHAM representation of the evaluation context c. If this worked, then we could use an airlock operation to isolate that redex from the

rest of the syntactic solution, i.e. $\{\!|p|\!\} \rightleftharpoons \{\!|t\ \gamma_c|\!\} \rightleftharpoons \{\!|t \triangleleft \{\!|\gamma_c|\!\}|\!\}$, and thus have it at the same level with the state solution in the configuration solution; this would allow to have rules that match both a syntactic molecule and a state molecule (after an airlock operation is applied on the state solution as well) in the same rule, as needed for the semantics of lookup and assignment. In our example above, we would obtain

$$\{\!|\{\!|x = 3\ /\ (x + 2)\,;|\!\}\ \{\!|x \mapsto 1\ \ y \mapsto 0|\!\}|\!\} \rightleftharpoons \{\!|\{\!|x \triangleleft \{\!|\gamma_{x=3/(\square+2)\,;}|\!\}|\!\}\ \{\!|x \mapsto 1 \triangleleft \{\!|y \mapsto 0|\!\}|\!\}|\!\}$$

and the latter could be rewritten with a natural CHAM reaction rule for variable lookup such as

$$\{\!|x \triangleleft c|\!\}\ \{\!|x \mapsto i \triangleleft \sigma|\!\} \rightarrow \{\!|i \triangleleft c|\!\}\ \{\!|x \mapsto i \triangleleft \sigma|\!\}$$

Unfortunately, there seems to be no way to achieve such a desirable CHAM representation of syntax. We next attempt and fail to do it in two different ways, and then give an argument why such a representation is actually impossible.

Consider, again, the statement "$x = 3\ /\ (x + 2)\,;$". A naive approach to represent this statement term as a syntactic solution (by means of appropriate heating/cooling rules) is to flatten it into its redex, namely x, and into all its atomic evaluation subcontexts, that is, to represent it as the following solution:

$$\{\!|x\ \ (\square + 2)\ \ (3\ /\ \square)\ \ (x = \square\,;)|\!\}$$

Such a representation can be relatively easily achieved by adding heating/cooling pair rules that correspond to the evaluation strategies (or contexts) of the various language constructs. For example, we can add the following rules corresponding to the evaluation strategies of the assignment and the addition constructs (and two similar ones for the division construct):

$$\begin{array}{rcl} x = a\,; & \rightleftharpoons & a \triangleleft \{\!|x = \square\,;|\!\} \\ a_1 + a_2 & \rightleftharpoons & a_1 \triangleleft \{\!|\square + a_2|\!\} \\ a_1 + a_2 & \rightleftharpoons & a_2 \triangleleft \{\!|a_1 + \square|\!\} \end{array}$$

With such rules, one can now heat or cool syntax as desired, for example:

$$\begin{array}{rcll} \{\!|x = 3\ /\ (x + 2)\,;|\!\} & \rightleftharpoons & \{\!|(3\ /\ (x + 2)) \triangleleft \{\!|x = \square\,;|\!\}|\!\} & \text{(Reaction)} \\ & \rightleftharpoons & \{\!|(3\ /\ (x + 2))\ \ (x = \square\,;)|\!\} & \text{(Airlock)} \\ & \rightleftharpoons & \{\!|(x + 2)\ \ (3\ /\ \square)\ \ (x = \square\,;)|\!\} & \text{(Reaction, Chemical, Airlock)} \\ & \rightleftharpoons & \{\!|x\ \ (\square + 2)\ \ (3\ /\ \square)\ \ (x = \square\,;)|\!\} & \text{(Reaction, Chemical, Airlock)} \end{array}$$

Unfortunately this naive approach is ambiguous, because it cannot distinguish the above from the representation of, say, $x = (3\ /\ x) + 2\,;$. The problem here is that the precise structure of the evaluation context is "lost in translation", so the approach above does not work.

Let us attempt a second approach, namely to guarantee that there is precisely one hole $\square$ molecule in each syntactic subsolution by using the molecule/solution nesting mechanism available in CHAM. More precisely, let us try to unambiguously represent the statements "$x = 3\ /\ (x + 2)\,;$" and "$x = (3\ /\ x) + 2\,;$" as the following two distinct syntactic solutions:

$$\begin{array}{l} \{\!|x\ \ \{\!|(\square + 2)\ \ \{\!|(3\ /\ \square)\ \ \{\!|(x = \square\,;)|\!\}|\!\}|\!\}|\!\} \\ \{\!|x\ \ \{\!|(3\ /\ \square)\ \ \{\!|(\square + 2)\ \ \{\!|(x = \square\,;)|\!\}|\!\}|\!\}|\!\} \end{array}$$

To achieve this, we modify the heating/cooling rules above as follows:

$$\begin{array}{rcl} (x = a\,;) \triangleleft c & \rightleftharpoons & a \triangleleft \{\!|\{\!|(x = \square\,;) \triangleleft c|\!\}|\!\} \\ (a_1 + a_2) \triangleleft c & \rightleftharpoons & a_1 \triangleleft \{\!|\{\!|(\square + a_2) \triangleleft c|\!\}|\!\} \\ (a_1 + a_2) \triangleleft c & \rightleftharpoons & a_2 \triangleleft \{\!|\{\!|(a_1 + \square) \triangleleft c|\!\}|\!\} \end{array}$$

With these modified rules, one may now think that one can heat and cool syntax unambiguously:

$$
\begin{array}{rcll}
\{\!\{x = 3 \,/\, (x+2)\,;\}\!\} & \rightleftharpoons & \{\!\{(x = 3 \,/\, (x+2)\,;) \triangleleft \{\!\{\cdot\}\!\}\}\!\} & \text{(Airlock)} \\
& \rightleftharpoons & \{\!\{(3 \,/\, (x+2)) \triangleleft \{\!\{(x = \square\,;) \triangleleft \{\!\{\cdot\}\!\}\}\!\}\}\!\} & \text{(Reaction)} \\
& \rightleftharpoons & \{\!\{(3 \,/\, (x+2)) \triangleleft \{\!\{x = \square\,;\}\!\}\}\!\} & \text{(Airlock, Membrane)} \\
& \rightleftharpoons & \{\!\{(x+2) \triangleleft \{\!\{(3 \,/\, \square) \triangleleft \{\!\{x = \square\,;\}\!\}\}\!\}\}\!\} & \text{(Reaction)} \\
& \rightleftharpoons & \{\!\{(x+2) \triangleleft \{\!\{(3 \,/\, \square)\ \{\!\{x = \square\,;\}\!\}\}\!\}\}\!\} & \text{(Airlock, Membrane)} \\
& \rightleftharpoons & \{\!\{x \triangleleft \{\!\{(\square + 2) \triangleleft \{\!\{(3 \,/\, \square)\ \{\!\{x = \square\,;\}\!\}\}\!\}\}\!\}\}\!\} & \text{(Reaction)} \\
& \rightleftharpoons & \{\!\{x \triangleleft \{\!\{(\square + 2)\ \{\!\{(3 \,/\, \square)\ \{\!\{x = \square\,;\}\!\}\}\!\}\}\!\}\}\!\} & \text{(Airlock, Membrane)} \\
& \rightleftharpoons & \{\!\{x\ \{\!\{(\square + 2)\ \{\!\{(3 \,/\, \square)\ \{\!\{x = \square\,;\}\!\}\}\!\}\}\!\}\}\!\} & \text{(Airlock)}
\end{array}
$$

Unfortunately, the above is not the only way one can heat the solution in question. For example, the following is also a possible derivation, showing that these heating/cooling rules are still problematic:

$$
\begin{array}{rcll}
\{\!\{x = 3 \,/\, (x+2)\,;\}\!\} & \rightleftharpoons & \{\!\{(x = 3 \,/\, (x+2)\,;) \triangleleft \{\!\{\cdot\}\!\}\}\!\} & \text{(Airlock)} \\
& \rightleftharpoons & \{\!\{(3 \,/\, (x+2)) \triangleleft \{\!\{(x = \square\,;) \triangleleft \{\!\{\cdot\}\!\}\}\!\}\}\!\} & \text{(Reaction)} \\
& \rightleftharpoons & \{\!\{(3 \,/\, (x+2)) \triangleleft \{\!\{x = \square\,;\}\!\}\}\!\} & \text{(Airlock, Membrane)} \\
& \rightleftharpoons & \{\!\{(3 \,/\, (x+2))\ \{\!\{x = \square\,;\}\!\}\}\!\} & \text{(Airlock)} \\
& \rightleftharpoons & \{\!\{(x+2)\ \{\!\{3 \,/\, \square\}\!\}\ \{\!\{x = \square\,;\}\!\}\}\!\} & \text{(All four laws)} \\
& \rightleftharpoons & \{\!\{x\ \{\!\{\square + 2\}\!\}\ \{\!\{3 \,/\, \square\}\!\}\ \{\!\{x = \square\,;\}\!\}\}\!\} & \text{(All four laws)}
\end{array}
$$

Indeed, one can similarly show that

$$\{\!\{x = (3 \,/\, x) + 2\,;\}\!\} \quad \rightleftharpoons \quad \{\!\{x\ \{\!\{\square + 2\}\!\}\ \{\!\{3 \,/\, \square\}\!\}\ \{\!\{x = \square\,;\}\!\}\}\!\}$$

Therefore, this second syntax representation attempt is also ambiguous.

We claim that it is impossible to devise heating/cooling rules in CHAM and representations $\gamma_{_}$ of evaluation contexts with the property that

$$\{\!\{c[t]\}\!\} \rightleftharpoons \{\!\{t\ \gamma_c\}\!\} \quad \text{or, equivalently,} \quad \{\!\{c[t]\}\!\} \rightleftharpoons \{\!\{t \triangleleft \{\!\{\gamma_c\}\!\}\}\!\}$$

for any term t and any appropriate evaluation context c. Indeed, if that was possible, then the following derivation could be possible:

$$
\begin{array}{rcll}
\{\!\{x = 3 \,/\, (x+2)\,;\}\!\} & \rightleftharpoons & \{\!\{(3 \,/\, (x+2))\ \gamma_{x=\square\,;}\}\!\} & \text{(hypothesis)} \\
& \rightleftharpoons & \{\!\{(x+2)\ \gamma_{3/\square}\ \gamma_{x=\square\,;}\}\!\} & \text{(hypothesis, Chemical)} \\
& \rightleftharpoons & \{\!\{x\ \gamma_{\square+2}\ \gamma_{3/\square}\ \gamma_{x=\square\,;}\}\!\} & \text{(hypothesis, Chemical)} \\
& \rightleftharpoons & \{\!\{(3 \,/\, x)\ \gamma_{\square+2}\ \gamma_{x=\square\,;}\}\!\} & \text{(hypothesis, Chemical)} \\
& \rightleftharpoons & \{\!\{((3 \,/\, x) + 2)\ \gamma_{x=\square\,;}\}\!\} & \text{(hypothesis, Chemical)} \\
& \rightleftharpoons & \{\!\{x = (3 \,/\, x) + 2\,;\}\!\} & \text{(hypothesis)}
\end{array}
$$

This general impossibility result explains why both our representation attempts above failed, as well as why many other similar attempts are also expected to fail.

The morale of the exercise above is that one should be very careful when using CHAM's airlock, because in combination with the other CHAM laws it can yield unexpected behaviors. In particular, the Chemical Law makes it impossible to state that a term matches the entire contents of a solution molecule, so one should not rely on the fact that all the remaining contents of a solution is in the membrane following the airlock. In our heating/cooling rules above, for example

$$
\begin{array}{rcl}
(x = a\,;) \triangleleft c & \rightleftharpoons & a \triangleleft \{\!\{(x = \square\,;) \triangleleft c\}\!\}\}\!\} \\
(a_1 + a_2) \triangleleft c & \rightleftharpoons & a_1 \triangleleft \{\!\{(\square + a_2) \triangleleft c\}\!\}\}\!\} \\
(a_1 + a_2) \triangleleft c & \rightleftharpoons & a_2 \triangleleft \{\!\{(a_1 + \square) \triangleleft c\}\!\}\}\!\}
\end{array}
$$

$$
\begin{array}{rcl}
a_1 + a_2 \curvearrowright c & \rightleftharpoons & a_1 \curvearrowright \square + a_2 \curvearrowright c \\
a_1 + a_2 \curvearrowright c & \rightleftharpoons & a_2 \curvearrowright a_1 + \square \curvearrowright c \\
a_1 \,/\, a_2 \curvearrowright c & \rightleftharpoons & a_1 \curvearrowright \square \,/\, a_2 \curvearrowright c \\
a_1 \,/\, a_2 \curvearrowright c & \rightleftharpoons & a_2 \curvearrowright a_1 \,/\, \square \curvearrowright c \\
a_1 <= a_2 \curvearrowright c & \rightleftharpoons & a_1 \curvearrowright \square <= a_2 \curvearrowright c \\
i_1 <= a_2 \curvearrowright c & \rightleftharpoons & a_2 \curvearrowright i_1 <= \square \curvearrowright c \\
!\, b \curvearrowright c & \rightleftharpoons & b \curvearrowright !\, \square \curvearrowright c \\
b_1 \,\&\&\, b_2 \curvearrowright c & \rightleftharpoons & b_1 \curvearrowright \square \,\&\&\, b_2 \curvearrowright c \\
x = a\,; \curvearrowright c & \rightleftharpoons & a \curvearrowright x = \square\,; \curvearrowright c \\
s_1\ s_2 \curvearrowright c & \rightleftharpoons & s_1 \curvearrowright \square\ s_2 \curvearrowright c \\
s & \rightleftharpoons & s \curvearrowright \square \\
\texttt{if}\,(b)\ s_1\ \texttt{else}\ s_2 \curvearrowright c & \rightleftharpoons & b \curvearrowright \texttt{if}\,(\square)\ s_1\ \texttt{else}\ s_2 \curvearrowright c
\end{array}
$$

Figure 3.44: CHAM heating-cooling rules for IMP.

our intuition that c matches *all* the evaluation context solution representation was wrong precisely for that reason. Indeed, it can just as well match a solution representation of a subcontext, which is why we got the unexpected derivation.

Correct representation of syntax. We next discuss an approach to representing syntax which is *not* based on CHAM's existing solution/membrane mechanism. We borrow from K (see Section 3.12) the idea of flattening syntax in an explicit list of computational tasks. Like in K, we use the symbol $\curvearrowright$, read "then" or "followed by", to separate such computational tasks; to avoid writing parentheses, we here assume that $\curvearrowright$ is right-associative and binds less tightly than any other construct. For example, the term "$x = 3 \,/\, (x + 2)\,;$" gets represented as the list term

$$x \curvearrowright \square + 2 \curvearrowright 3 \,/\, \square \curvearrowright x = \square\,; \curvearrowright \square$$

which reads "process x, followed by adding 2 to it, followed by dividing 3 by the result, followed by assigning the obtained result to x, which is the final task". Figure 3.44 shows all the heating/cooling rules that we associate to the various evaluation strategies of the IMP language constructs. These rules allow us to structurally rearrange any well-formed syntactic term so that the next computational task is at the top (left side) of the computation list.

The only rule in Figure 3.44 which does not correspond to the evaluation strategy of some evaluation construct is $s \rightleftharpoons s \curvearrowright \square$. Its role is to initiate the decomposition process whenever an unheated statement is detected in the syntax solution. According to CHAM's laws, these rules can only apply in solutions, so we can derive

$$\{\!|x = 1\,;\ x = 3 \,/\, (x + 2)\,;|\!\} \rightleftharpoons^* \{\!|x = 1\,;\ \curvearrowright \square\ x = 3 \,/\, (x + 2)\,;\ \curvearrowright \square|\!\}$$

but there is no way to derive, for example,

$$\{\!|x = 1\,;\ x = 3 \,/\, (x + 2)\,;|\!\} \rightleftharpoons^* \{\!|x = 1\,;\ (x \curvearrowright \square{+}2 \curvearrowright 3 \,/\, \square \curvearrowright x = \square\,; \curvearrowright \square)|\!\}$$

The syntactic solution will contain only one syntactic molecule at any given moment, with no subsolutions, which is the reason why the heating/cooling rules in Figure 3.44 that correspond to language construct evaluation strategies need to mention the remaining of the list of computational tasks in the syntactic molecule, c, instead of just the interesting part (e.g., $a_1 + a_2 \rightleftharpoons a_1 \curvearrowright \square + a_2$, etc.), as we do in K (see

$$
\begin{array}{rcl}
\{\!\!\{x \curvearrowright c\}\!\!\}\ \{\!\!\{x \mapsto i \rhd \sigma\}\!\!\} & \to & \{\!\!\{i \curvearrowright c\}\!\!\}\ \{\!\!\{x \mapsto i \rhd \sigma\}\!\!\} \\
i_1 + i_2 \curvearrowright c & \to & i_1 +_{Int} i_2 \curvearrowright c \\
i_1 \ / \ i_2 \curvearrowright c & \to & i_1 /_{Int} i_2 \curvearrowright c \quad \text{when } i_2 \neq 0 \\
i_1 <= i_2 \curvearrowright c & \to & i_1 \leq_{Int} i_2 \curvearrowright c \\
!\ \texttt{true} \curvearrowright c & \to & \texttt{false} \curvearrowright c \\
!\ \texttt{false} \curvearrowright c & \to & \texttt{true} \curvearrowright c \\
\texttt{true}\ \&\&\ b_2 \curvearrowright c & \to & b_2 \curvearrowright c \\
\texttt{false}\ \&\&\ b_2 \curvearrowright c & \to & \texttt{false} \curvearrowright c \\
\{\ s\ \} \curvearrowright c & \to & s \curvearrowright c \\
\{\!\!\{x = i;\ \curvearrowright c\}\!\!\}\ \{\!\!\{x \mapsto j \rhd \sigma\}\!\!\} & \to & \{\!\!\{\{\}\ \curvearrowright c\}\!\!\}\ \{\!\!\{x \mapsto i \rhd \sigma\}\!\!\} \\
\{\}\ s_2 \curvearrowright c & \to & s_2 \curvearrowright c \\
\texttt{if}\ (\texttt{true})\ s_1\ \texttt{else}\ s_2 \curvearrowright c & \to & s_1 \curvearrowright c \\
\texttt{if}\ (\texttt{false})\ s_1\ \texttt{else}\ s_2 \curvearrowright c & \to & s_2 \curvearrowright c \\
\texttt{while}\ (b)\ s \curvearrowright c & \to & \texttt{if}\ (b)\ \{\ s\ \texttt{while}\ (b)\ s\ \}\ \texttt{else}\ \{\} \curvearrowright c \\
\texttt{int}\ xl;\ s & \to & \{\!\!\{s\}\!\!\}\ \{\!\!\{xl \mapsto 0\}\!\!\} \\
 & & \\
(x, xl) \mapsto i & \rightharpoonup & x \mapsto i \rhd \{\!\!\{xl \mapsto i\}\!\!\}
\end{array}
$$

Figure 3.45: CHAM(IMP): The CHAM of IMP, obtained by adding to the heating/cooling rules in Figure 3.44 the semantic rules for IMP plus the heating rule for state initialization above.

Section 3.12). The heating/cooling rules in Figure 3.44 decompose the syntactic term into any of its possible splits into a redex (the top of the resulting list of computational tasks) and an evaluation context (represented flattened as the rest of the list). In fact, these heating/cooling rules have been almost mechanically derived from the syntax of evaluation contexts for the IMP language constructs in Section 3.7.1 (see Figure 3.30).

Figure 3.45 shows the remaining CHAM rules of IMP, giving the actual semantics of each language construct. Like the heating/cooling rules in Figure 3.44, these rules are also almost mechanically derived from the rules of the reduction semantics with evaluation contexts of IMP in Section 3.7.1 (see Figure 3.31), with the following notable differences:

- Each rule needs to mention the remaining list of computational tasks, c, for the same reason the heating/cooling rules in Figure 3.44 need to mention it (which is explained above).

- There is no equivalent of the characteristic rule of reduction semantics with evaluation contexts. The Membrane Law looks somehow similar, but we cannot take advantage of that because we were not able to use the inherent airlock mechanism of the CHAM to represent syntax (see the failed attempts to represent syntax above).

- The state is organized as a solution using CHAM's airlock mechanism, instead of just imported as an external data-structure as we did in our previous semantics. We did so because the state data-structure that we used in our previous semantics was a finite-domain partial function (see Section 3.1.2), which was represented as a set of pairs (Section 2.1.2), and it is quite natural to replace any set structures by the inherent CHAM solution mechanism.

Figure 3.46 shows a possible execution of IMP's CHAM defined above, mentioning at each step which of CHAM's laws have been applied. Note that the final configuration contains two subsolutions, one for the

$$
\begin{array}{rcl}
\{\!\{\texttt{int}\ x, y;\ x = 1;\ x = 3 \,/\, (x+2);\}\!\} & \rightarrow & \text{(Reaction)} \\
\{\!\{\{\!\{x = 1;\ x = 3 \,/\, (x+2);\}\!\}\ \{\!\{x, y \mapsto 0\}\!\}\}\!\} & \rightharpoonup & \text{(Heating, Membrane)} \\
\{\!\{\{\!\{x = 1;\ x = 3 \,/\, (x+2);\ \curvearrowright \square\}\!\}\ \{\!\{x, y \mapsto 0\}\!\}\}\!\} & \rightharpoonup^* & \text{(Heating, Membrane, Airlock)} \\
\{\!\{\{\!\{x = 1;\ x = 3 \,/\, (x+2);\ \curvearrowright \square\}\!\}\ \{\!\{x \mapsto 0\ \ y \mapsto 0\}\!\}\}\!\} & \rightharpoonup & \text{(Heating, Membrane)} \\
\{\!\{\{\!\{x = 1;\ \curvearrowright \square\ x = 3 \,/\, (x+2);\ \curvearrowright \square\}\!\}\ \{\!\{x \mapsto 0\ \ y \mapsto 0\}\!\}\}\!\} & \rightharpoonup & \text{(Airlock, Membrane)} \\
\{\!\{\{\!\{x = 1;\ \curvearrowright \square\ x = 3 \,/\, (x+2);\ \curvearrowright \square\}\!\}\ \{\!\{x \mapsto 0 \rhd \{\!\{y \mapsto 0\}\!\}\}\!\}\}\!\} & \rightarrow & \text{(Reaction)} \\
\{\!\{\{\!\{\{\}\ \curvearrowright \square\ x = 3 \,/\, (x+2);\ \curvearrowright \square\}\!\}\ \{\!\{x \mapsto 1 \rhd \{\!\{y \mapsto 0\}\!\}\}\!\}\}\!\} & \rightharpoonup & \text{(Cooling, Membrane)} \\
\{\!\{\{\!\{\{\}\ x = 3 \,/\, (x+2);\ \curvearrowright \square\}\!\}\ \{\!\{x \mapsto 1 \rhd \{\!\{y \mapsto 0\}\!\}\}\!\}\}\!\} & \rightharpoonup & \text{(Reaction, Membrane)} \\
\{\!\{\{\!\{x = 3 \,/\, (x+2);\ \curvearrowright \square\}\!\}\ \{\!\{x \mapsto 1 \rhd \{\!\{y \mapsto 0\}\!\}\}\!\}\}\!\} & \rightharpoonup^* & \text{(Heating, Membrane)} \\
\{\!\{\{\!\{x \curvearrowright \square + 2 \curvearrowright 3 \,/\, \square \curvearrowright x = \square;\ \curvearrowright \square\}\!\}\ \{\!\{x \mapsto 1 \rhd \{\!\{y \mapsto 0\}\!\}\}\!\}\}\!\} & \rightarrow & \text{(Reaction)} \\
\{\!\{\{\!\{1 \curvearrowright \square + 2 \curvearrowright 3 \,/\, \square \curvearrowright x = \square;\ \curvearrowright \square\}\!\}\ \{\!\{x \mapsto 1 \rhd \{\!\{y \mapsto 0\}\!\}\}\!\}\}\!\} & \rightharpoonup & \text{(Cooling)} \\
\{\!\{\{\!\{1 + 2 \curvearrowright 3 \,/\, \square \curvearrowright x = \square;\ \curvearrowright \square\}\!\}\ \{\!\{x \mapsto 1 \rhd \{\!\{y \mapsto 0\}\!\}\}\!\}\}\!\} & \rightarrow & \text{(Reaction, Membrane)} \\
\{\!\{\{\!\{3 \curvearrowright 3 \,/\, \square \curvearrowright x = \square;\ \curvearrowright \square\}\!\}\ \{\!\{x \mapsto 1 \rhd \{\!\{y \mapsto 0\}\!\}\}\!\}\}\!\} & \rightarrow^* & \text{(Reaction, Cooling, Membrane)} \\
\{\!\{\{\!\{x = 1;\ \curvearrowright \square\}\!\}\ \{\!\{x \mapsto 1 \rhd \{\!\{y \mapsto 0\}\!\}\}\!\}\}\!\} & \rightarrow & \text{(Reaction)} \\
\{\!\{\{\!\{\{\}\ \curvearrowright \square\}\!\}\ \{\!\{x \mapsto 1 \rhd \{\!\{y \mapsto 0\}\!\}\}\!\}\}\!\} & \rightarrow & \text{(Cooling, Membrane)} \\
\{\!\{\{\!\{\{\}\}\!\}\ \{\!\{x \mapsto 1 \rhd \{\!\{y \mapsto 0\}\!\}\}\!\}\}\!\} & &
\end{array}
$$

Figure 3.46: An execution of IMP's CHAM.

syntax and one for the state. Since the syntactic subsolution in the final configuration solution is expected to always contain only {}, one can safely eliminated it.

3.8.2 The CHAM of IMP++

We next discuss the CHAM of IMP++, discussing like in the other semantics each feature separately first and then putting all of them together. When putting them together, we also investigate the modularity and appropriateness of the resulting definition.

Variable Increment

The chemical abstract machine can also define the increment modularly:

$$\{\!\{{+}{+}\ x \curvearrowright c\}\!\}\ \{\!\{x \mapsto i \rhd \sigma\}\!\} \rightarrow \{\!\{i +_{Int} 1 \curvearrowright c\}\!\}\ \{\!\{x \mapsto i +_{Int} 1 \rhd \sigma\}\!\} \qquad \text{(CHAM-Inc)}$$

Input/Output

All we have to do is to add new molecules in the top-level solution that hold the input and the output buffers, then define the evaluation strategy of `print` by means of a heating/cooling pair like we did for other strict constructs, and finally add the reaction rules corresponding to the input/output constructs. Since solutions are not typed, to distinguish the solution holding the input buffer from the one holding the output buffer, we introduce two artificial molecules, called input and output, respectively, and place them upfront in their corresponding solutions. Since the input buffer needs to also be provided within the initial solution, we modify the reaction rule for programs to take program and input molecules and initialize the output and state molecules accordingly:

$$\texttt{print}(a)\texttt{;} \curvearrowright c \rightleftharpoons a \curvearrowright \texttt{print}(\square)\texttt{;} \curvearrowright c$$

$$\{\!\!\{\texttt{read}() \curvearrowright c\}\!\!\}\ \{\!\!\{\mathsf{input}\ i : w\}\!\!\} \rightarrow \{\!\!\{i \curvearrowright c\}\!\!\}\ \{\!\!\{\mathsf{input}\ w\}\!\!\} \qquad \text{(CHAM-Read)}$$

$$\{\!\!\{\texttt{print}(i)\texttt{;} \curvearrowright c\}\!\!\}\ \{\!\!\{\mathsf{output}\ w\}\!\!\} \rightarrow \{\!\!\{\{\} \curvearrowright c\}\!\!\}\ \{\!\!\{\mathsf{output}\ w : i\}\!\!\} \qquad \text{(CHAM-Print)}$$

$$\{\!\!\{\texttt{int}\ xl\texttt{;}\ s\}\!\!\}\ \{\!\!\{w\}\!\!\} \rightarrow \{\!\!\{s\}\!\!\}\ \{\!\!\{xl \mapsto 0\}\!\!\}\ \{\!\!\{\mathsf{input}\ w\}\!\!\}\ \{\!\!\{\mathsf{output}\ \epsilon\}\!\!\} \qquad \text{(CHAM-Pgm)}$$

Abrupt Termination

The CHAM semantics of abrupt termination is even more elegant and modular than that using evaluation contexts above, because the other components of the configuration need not be mentioned:

$$i\ /\ 0 \curvearrowright c \ \rightarrow\ \{\} \qquad \text{(CHAM-Div-By-Zero)}$$

$$\texttt{halt;} \curvearrowright c \ \rightarrow\ \{\} \qquad \text{(CHAM-Halt)}$$

Dynamic Threads

As stated in Section 3.8, the CHAM has been specifically proposed as a model of concurrent computation, based on the chemical metaphor that molecules in solutions can get together and react, with possibly many reactions taking place concurrently. Since there was no concurrency so far in our language, the actual strength of the CHAM has not been seen yet. Recall that the configuration of the existing CHAM semantics of IMP consists of one top-level solution, which contains two subsolutions: a syntactic subsolution holding the remainder of the program organized as a molecule sequentializing computation tasks using the special construct $\curvearrowright$; and a state subsolution containing binding molecules, each binding a different program variable to a value. As seen in Figures 3.44 and 3.45, most of the CHAM rules involve only the syntactic molecule. The state subsolution is only mentioned when the language construct involves program variables.

The above suggests that all a `spawn` statement needs to do is to create an additional syntactic subsolution holding the spawned statement, letting the newly created subsolution molecule to float together with the original syntactic molecule in the same top-level solution. Minimalistically, this can be achieved with the following CHAM rule (which does not consider thread termination yet):

$$\{\!\!\{\texttt{spawn}\ s \curvearrowright c\}\!\!\} \rightarrow \{\!\!\{\{\} \curvearrowright c\}\!\!\}\ \{\!\!\{s\}\!\!\}$$

Since the order of molecules in a solution is irrelevant, the newly created syntactic molecule has the same rights as the original molecule in reactions involving the state molecule. We can (correctly) think of each syntactic subsolution as an independently running execution thread. The same CHAM rules we had before (see Figures 3.44 and 3.45) can now also apply to the newly created threads. Moreover, reactions taking place only in the syntactic molecules, which are a majority by a large number, can apply truly concurrently. For example, a thread may execute a loop unrolling step while another thread may concurrently perform an addition. The only restriction regarding concurrency is that rule instances must involve disjoint molecules in order to proceed concurrently. That means that it is also possible for a thread to read or write the state while another thread, truly concurrently, performs a local computation. This degree of concurrency was not possible within the other semantic approaches discussed so far in this chapter.

The rule above only creates threads. It does not collect threads when they complete their computation. One could do that with the simple solution-dissolving rule

$$\{\!\!\{\{\}\}\!\!\} \rightarrow \cdot$$

but the problem is that such a rule cannot distinguish between the original thread and the others, so it would also dissolve the original thread when it completes. This could be considered correct behavior, but, however,

we prefer to distinguish the original thread created statically from the others, which are created dynamically. Specifically, we collect only the terminated threads which were created dynamically. To achieve that, we can flag the newly created threads for collection as below. Here is our complete CHAM semantics of `spawn`:

$$\textit{Molecule} \ ::= \ \ldots \mid \texttt{die}$$

$$\{\!\!\{\texttt{spawn}\ s \curvearrowright c\}\!\!\} \rightarrow \{\!\!\{\{\} \curvearrowright c\}\!\!\}\ \{\!\!\{s \curvearrowright \texttt{die}\}\!\!\} \qquad \text{(CHAM-Spawn)}$$

$$\{\!\!\{\{\} \curvearrowright \texttt{die}\}\!\!\} \rightarrow \cdot \qquad \text{(CHAM-Die)}$$

We conclude this section with a discussion on the concurrency of the CHAM above. As already argued, it allows for truly concurrent computations to take place, provided that their corresponding CHAM rule instances do not overlap. While this already goes far beyond the other semantical approaches in terms of concurrency, it still enforces interleaving where it should not.

Consider, for example, a global configuration in which two threads are about to lookup two different variables in the state cell. Even though there are good reasons to allow the two threads to proceed concurrently, the CHAM above will not, because the two rule instances (of the same CHAM lookup rule) overlap on the state molecule. This problem can be ameliorated, to some extent, by changing the structure of the top-level configuration to allow all the variable binding molecules currently in the state subsolution to instead float in the top-level solution at the same level with the threads: this way, each thread can independently grab the binding it is interested in without blocking the state anymore. Unfortunately, this still does not completely solve the true concurrency problem, because one could argue that different threads should also be allowed to concurrently read the same variable. Thus, no matter where the binding of that variable is located, the two rule instances cannot proceed concurrently. Moreover, flattening all the syntactic and the semantic ingredients in a top level solution, as the above "fix" suggests, does not scale. Real-life languages can have many configuration items of various kinds, such as, environments, heaps, function/exception/loop stacks, locks held, and so on. Collapsing the contents of all these items in one flat solution would not only go against the CHAM philosophy, but it would also make it hard to understand and control. The K framework (see Section 3.12) solves this problem by allowing its rules to state which parts of the matched subterm are shared with and which can be concurrently modified by other rules.

Local Variables

The simplest approach to adding blocks with local variables to IMP is to follow an idea similar to the one for reduction semantics with evaluation contexts discussed in Section 3.7.2, assuming the procedure presented in Section 3.5.5 for desugaring blocks with local variables into `let` constructs:

$$\texttt{let}\ x = a\ \texttt{in}\ s \curvearrowright c \rightleftharpoons a \curvearrowright \texttt{let}\ x = \square\ \texttt{in}\ s \curvearrowright c$$
$$\{\!\!\{\texttt{let}\ x = i\ \texttt{in}\ s \curvearrowright c\}\!\!\}\ \{\!\!\{\sigma\}\!\!\} \rightarrow \{\!\!\{s\ x = \sigma(x);\ \curvearrowright c\}\!\!\}\ \{\!\!\{\sigma[i/x]\}\!\!\} \qquad \text{(CHAM-Let)}$$

As it was the case in Section 3.7.2, the above is going to be problematic when we add `spawn` to the language, too. However, recall that in this language experiment we pretend each language extension is final, in order to understand the modularity and flexibility to change of each semantic approach.

The approach above is very syntactic in nature, following the intuitions of evaluation contexts. In some sense, the above worked because we happened to have an assignment statement in our language, which we used for recovering the value of the bound variable. Note, however, that several computational steps were wasted because of the syntactic translations. What we would have really liked to say is "`let` *Id* = *Int* `in` *Context* is a special evaluation context where the current state is updated with the binding

whenever `let` is passed through top-down, and where the state is recovered whenever `let` is passed through bottom-up". This was not possible to say with evaluation contexts. When using the CHAM, we are free to disobey the syntax. For example, the alternative definition below captures the essence of the problem and wastes no steps (although it is still problematic when combined with `spawn`):

$$\texttt{let}\ x = a\ \texttt{in}\ s \curvearrowright c \rightleftharpoons a \curvearrowright \texttt{let}\ x = \square\ \texttt{in}\ s \curvearrowright c$$
$$\{\!\{\texttt{let}\ x = i\ \texttt{in}\ s \curvearrowright c\}\!\}\ \{\!\{\sigma\}\!\} \rightharpoonup \{\!\{s \curvearrowright \texttt{let}\ x = \sigma(x)\ \texttt{in}\ \square \curvearrowright c\}\!\}\ \{\!\{\sigma[i/x]\}\!\}$$
$$\{\!\{\{\} \curvearrowright \texttt{let}\ x = v\ \texttt{in}\ \square \curvearrowright c\}\!\}\ \{\!\{\sigma\}\!\} \rightharpoonup \{\!\{\{\} \curvearrowright c\}\!\}\ \{\!\{\sigma[v/x]\}\!\}$$

In words, the `let` is first heated/cooled in the binding expression. Once that becomes an integer, the `let` is then only heated in its body statement, at the same time updating the state molecule with the binding and storing the return value in the residual `let` construct. Once the `let` body statement becomes {}, the solution is cooled down by discarding the residual `let` construct and recovering the state appropriately (we used a "value" v instead of an integer i in the latter rule to indicate the fact that v can also be $\perp$).

Of course, the substitution-based approach discussed in detail in Sections 3.5.6 and 3.7.2 can also be adopted here if one is willing to pay the price for using it:

$$\texttt{let}\ x = a\ \texttt{in}\ s \curvearrowright c \rightleftharpoons a \curvearrowright \texttt{let}\ x = \square\ \texttt{in}\ s \curvearrowright c$$
$$\{\!\{\texttt{let}\ x = i\ \texttt{in}\ s \curvearrowright c\}\!\}\ \{\!\{\sigma\}\!\} \rightarrow \{\!\{s[x'/x] \curvearrowright c\}\!\}\ \{\!\{\sigma[i/x']\}\!\} \quad \text{if } x' \text{ is a fresh variable}$$

Putting Them All Together

Putting together all the language features defined in CHAM above is a bit simpler and more modular than in MSOS (see Section 3.6.2): all we have to do is to take the union of all the syntax and semantics of all the features, removing the original rule for the initialization of the solution (that rule was already removed as part of the addition of input/output to IMP); in MSOS, we also had to add the halting attribute to the labels, which we do not have to do in the case of the CHAM.

Unfortunately, like in the case of the reduction semantics with evaluation contexts of IMP++ in Section 3.7.2, the resulting language is flawed. Indeed, a thread spawned from inside a `let` would be created its own molecule in the top-level solution, which would execute concurrently with all the other execution threads, including its parent. Thus, there is the possibility that the parent will advance to the assignment recovering the value of the `let`-bound variable before the spawned thread terminates, in which case the bound variable would be changed in the state by the parent thread, "unexpectedly" for the spawned thread. One way to address this problem is to rename the bound variable into a fresh variable within the `let` body statement, like we did above, using a substitution operation. Another is to split the state into an environment mapping variables to locations and a store mapping locations to values, and to have each thread consist of a solution holding both its code and its environment. Both these solutions were also suggested in Section 3.5.6, when we discussed how to make small-step SOS correctly capture all the behaviors of the resulting IMP++.

3.8.3 CHAM in Rewrite Logic

As explained above, CHAM rewriting cannot be immediately captured as ordinary rewriting modulo solution multiset axioms such as associativity, commutativity and identity. The distinction between the two arises essentially when a CHAM rule involves only one top-level molecule which is not a solution, because the CHAM laws restrict the applications of such a rule only in solutions while ordinary rewriting allows such rules to apply everywhere. To solve this problem, we wrap each rule in a solution context, that is, we translate CHAM rules of the form

$$m_1\ m_2\ \ldots\ m_k \rightarrow m'_1\ m'_2\ \ldots\ m'_l$$

into corresponding rewrite logic rules of the form

$$\{\!|\overline{m_1}\ \overline{m_2}\ \ldots\ \overline{m_k}\ Ms|\!\} \rightarrow \{\!|\overline{m'_1}\ \overline{m'_2}\ \ldots\ \overline{m'_l}\ Ms|\!\}$$

where the only difference between the original CHAM terms $m_1, m_2, \ldots, m_k, m'_1, m'_2, \ldots, m'_l$ and their algebraic variants $\overline{m_1}, \overline{m_2}, \ldots, \overline{m_k}, \overline{m'_1}, \overline{m'_2}, \ldots, \overline{m'_l}$ is that the meta-variables appearing in the former (recall that CHAM rules are schemata) are turned into variables of corresponding sorts in the latter, and where Ms is a variable of sort **Bag{***Molecule***}** that does not appear anywhere else in $\overline{m_1}, \overline{m_2}, \ldots, \overline{m_k}, \overline{m'_1}, \overline{m'_2}, \ldots, \overline{m'_l}$.

With this representation of CHAM rules into rewrite logic rules, it is obvious that rewrite logic's rewriting captures both the Reaction Law and the Chemical Law of the CHAM. What is less obvious is that it also captures the Membrane Law. Indeed, note that the Membrane Law allows rewrites to take place only when the global term is a solution, while rewriting logic allows rewrites to take place anywhere. However, the rewrite logic rule representation above generates only rules that rewrite solutions into solutions. Thus, if the original term to rewrite is a solution, then so it will stay during the entire rewriting process, and so the Membrane Law is also naturally captured by rewrite logic derivations. However, if the original term to rewrite is a proper molecule (which is not a solution), then so it will stay during the entire rewrite logic's rewriting while the CHAM will not reduce it at all. Still it is important to understand that in this case the corresponding rewrite logic theory can perform rewrite steps (in subsolutions of the original term) which are not possible under the CHAM, in particular that it may lead to non-termination in situations where the CHAM is essentially stuck. To reconcile this inherent difference between the CHAM and rewrite logic, we make the reasonable assumption that the original terms to rewrite can only be solutions. Note that the CHAM sequents in Definition 23 already assume that one only derives solution terms.

The only CHAM law which has not been addressed above is the Airlock Law. Rewrite logic has no builtin construct resembling CHAM's airlock, but its multiset matching is powerful enough to allow us to capture the airlock's behavior through rewrite rules. One possibility is to regard the airlock operation like any other molecular construct. Indeed, from a rewrite logic perspective, the Airlock Law says that any molecule inside a solution can be matched and put into an airlock next to the remaining solution wrapped into a membrane, and this process is reversible. This behavior can be achieved through the following two (opposite) rewrite logic rules, where M is a molecule variable and Ms is a bag-of-molecules variable:

$$\begin{array}{rcl} \{\!|M\ Ms|\!\} & \rightarrow & \{\!|M \rhd \{\!|Ms|\!\}|\!\} \\ \{\!|M \rhd \{\!|Ms|\!\}|\!\} & \rightarrow & \{\!|M\ Ms|\!\} \end{array}$$

Another possibility to capture airlock's behavior in rewrite logic is to attempt to eliminate it completely and replace it with matching modulo multiset axioms. While this appears to be possible in many concrete situations, we are however not aware of any general solution to do so systematically for any CHAM. The question is whether the elimination of the airlock is indeed safe, in the sense that the resulting rewrite logic theory does not lose any of the original CHAM's behaviors. One may think that thanks to the restricted form of CHAM's rules, the answer is immediately positive. Indeed, since the CHAM disallows any other constructs for solutions except its builtin membrane operation (a CHAM can only add new syntactic constructs for molecules, but not for solutions) and since solution subterms can either contain only one molecule or otherwise be meta-variables (to avoid multiset matching), we can conclude that in any CHAM rule, a subterm containing an airlock operation at its top can only be of the form $m \rhd \{\!|m'|\!\}$ or of the form $m \rhd s$ with s a meta-variable. Both these cases can be uniformly captured as subterms of the form $\overline{m} \rhd \{\!|ms|\!\}$ with ms a term of sort **Bag{***Molecule***}** in rewrite logic, the former by taking $k = 1$ and $ms = m'$ and the latter by replacing the metavariable s with a term of the form $\{\!|Ms|\!\}$ everywhere in the rule, where Ms is a fresh variable of sort **Bag{***Molecule***}**. Unfortunately, it is not clear how we can eliminate the airlock from subterms of the form

sorts:
Molecule, *Solution*, **Bag{***Molecule***}**
subsorts:
Solution < *Molecule*
// One may also need to subsort to *Molecule* specific syntactic categories (*Int*, *Bool*, etc.)
operations:
$\{\!|_|\!\}$: **Bag{***Molecule***}** $\rightarrow$ *Solution* // membrane operator
$_ \rhd _$: *Molecule* $\times$ *Solution* $\rightarrow$ *Molecule* // airlock operator
// One may also need to define specific syntactic constructs for *Molecule* ($_ + _$, $_ \mapsto _$, etc.)
rules:
// Add the following two generic (i.e., same for all CHAMs) airlock rewrite logic rules:
$\{\!|M\ Ms|\!\} \leftrightarrow \{\!|M \rhd \{\!|Ms|\!\}|\!\}$ // M, Ms variables of sorts *Molecule*, **Bag{***Molecule***}**, resp.
// For each specific CHAM rule $m_1\ m_2\ \ldots m_k \rightarrow m'_1\ m'_2\ \ldots m'_l$ add a rewrite logic rule
$\{\!|\overline{m_1}\ \overline{m_2}\ \ldots\ \overline{m_k}\ Ms|\!\} \rightarrow \{\!|\overline{m'_1}\ \overline{m'_2}\ \ldots\ \overline{m'_l}\ Ms|\!\}$ // Ms variable of sort **Bag{***Molecule***}**
// where $\overline{m}$ replaces each meta-variable in m by a variable of corresponding sort.

Figure 3.47: Embedding of a chemical abstract machine into rewrite logic (CHAM $\rightsquigarrow \mathcal{R}_{\text{CHAM}}$).

$\overline{m} \rhd \{\!|ms|\!\}$. If such terms appear in a solution or at the top of the rule, then one can replace them by their corresponding bag-of-molecule terms $\overline{m}\ ms$. However, if they appear in a proper molecule context (i.e., not in a solution context), then they cannot be replaced by $\overline{m}\ ms$ (first, we would get a parsing error by placing a bag-of-molecule term in a molecule place; second, reactions cannot take place in *ms* anymore, because it is not surrounded by a membrane). We cannot replace $\overline{m} \rhd \{\!|ms|\!\}$ by $\{\!|\overline{m}\ ms|\!\}$ either, because that would cause a double membrane when the thus modified airlock reaches a solution context. Therefore, we keep the airlock.

Putting all the above together, we can associate a rewrite logic theory to any CHAM as shown in Figure 3.47; for simplicity, we assumed that the CHAM has only one *Molecule* syntactic category and, implicitly, only one corresponding *Solution* syntactic category. In Figure 3.47 and elsewhere in this section, we use the notation *left* $\leftrightarrow$ *right* as a shorthand for two opposite rewrite rules, namely for both *left* $\rightarrow$ *right* and *right* $\rightarrow$ *left*. The discussion above implies the following result:

Theorem 21. ***(Embedding of the chemical abstract machine into rewrite logic)*** *If* CHAM *is a chemical abstract machine, sol and sol′ are two solutions, and* $\mathcal{R}_{\text{CHAM}}$ *is the rewrite logic theory associated to* CHAM *as in Figure 3.47, then the following hold:*

1. CHAM $\vdash$ *sol* $\rightarrow$ *sol′ if and only if* $\mathcal{R}_{\text{CHAM}} \vdash \overline{sol} \rightarrow^1 \overline{sol'}$*;*

2. CHAM $\vdash$ *sol* $\rightarrow^*$ *sol′ if and only if* $\mathcal{R}_{\text{CHAM}} \vdash \overline{sol} \rightarrow \overline{sol'}$*.*

Therefore, one-step solution rewriting in $\mathcal{R}_{\text{CHAM}}$ corresponds precisely to one-step solution rewriting in the original CHAM and thus, one can use $\mathcal{R}_{\text{CHAM}}$ as a replacement for CHAM for any reduction purpose. Unfortunately, this translation of CHAM into rewrite logic does not allow us to borrow the latter's concurrency to obtain the desired concurrent rewriting computational mechanism of the former. Indeed, the desired CHAM concurrency says that "different rule instances can apply concurrently in the same solution as far as they act on different molecules". Unfortunately, the corresponding rewrite logic rule instances cannot apply concurrently according to rewrite logic's semantics because both instances would match the entire solution, including the membrane (and rule instances which overlap cannot proceed concurrently in rewrite logic—see Section 2.5).

The CHAM of IMP in Rewrite Logic

Figure 3.48 shows the rewrite theory $\mathcal{R}_{\text{CHAM(IMP)}}$ obtained by applying the generic transformation procedure in Figure 3.47 to the CHAM of IMP discussed in this section and summarized in Figures 3.44 and 3.45. In addition to the generic CHAM syntax, as indicated in the comment under subsorts in Figure 3.47 we also subsort the builtin sorts of IMP, namely *Int*, *Bool* and *Id*, to *Molecule*. The "followed by" $_ \curvearrowright _$ construct for molecules is necessary for defining the evaluation strategies of the various IMP language constructs as explained above; we believe that some similar operator is necessary when defining any programming language whose constructs have evaluation strategies because, as explained above, it appears that the CHAM airlock operator is not suitable for this task. For notational simplicity, we stick to our previous convention that $_ \curvearrowright _$ is right associative and binds less tight than any other molecular construct. Finally, we add molecular constructs corresponding to all the syntax that we need in order to define the IMP semantics, which includes syntax for language constructs, for evaluation contexts, and for the state and state initialization.

The rewrite rules in Figure 3.48 are straightforward, following the transformation described in Figure 3.47. We grouped them in four categories: (1) the airlock rule is reversible and precisely captures the CHAM Airlock Law; (2) the heating/cooling rules, also reversible, capture the evaluation strategies of IMP's language constructs (see Figure 3.44); (3) the semantic rules are irreversible and capture the computational steps of the IMP semantics (see Figure 3.45); (4) the state initialization rule corresponds to the heating rule in Figure 3.45.

Our CHAM-based rewrite logic semantics of IMP in Figure 3.48 follows blindly the CHAM of IMP in Figures 3.44 and 3.45. All it does is to mechanically apply the transformation in Figure 3.47, without making any attempts to optimize the resulting rewrite logic theory. For example, it is easy to see that the syntactic molecules will always contain only one molecule. Also, it is easy to see that the state molecule can be initialized in such a way that at any moment during the initialization rewriting sequence any subsolution containing a molecule of the form $xl \mapsto i$, with xl a proper list, will contain no other molecule. Finally, one can also notice that the top level solution will always contain only two molecules, namely what we called a syntactic solution and a state solution. All these observations suggest that we can optimize the rewrite logic theory in Figure 3.48 by deleting the variable *Ms* from every rule except the airlock ones. While one can do that for our simple IMP language here, one has to be careful with such optimizations in general. For example, we later on add threads to IMP (see Section 3.5), which implies that the top level solution will contain a dynamic number of syntactic subsolutions, one per thread; if we remove the *Ms* variable from the lookup and assignment rules in Figure 3.48, then those rules will not work whenever there are more than two threads running concurrently. On the other hand, the *Ms* variable in the rule for variable declarations can still be eliminated. The point is that one needs to exercise care when one attempts to hand-optimize the rewrite logic theories resulting from mechanical semantic translations.

★ The CHAM of IMP in Maude

Like for the previous semantics, it is relatively straightforward to mechanically translate the corresponding rewrite theories into Maude modules. However, unlike in the previous semantics, the resulting Maude modules are not immediately executable. The main problem is that, in spite of its elegant chemical metaphor, the CHAM was not conceived to be blindly executable. For example, most of the heating and cooling rules tend to be reversible, leading to non-termination of the underlying rewrite relation. Non-termination is not a problem per se in rewrite logic and in Maude, because one can still use other formal analysis capabilities of these such as search and model checking, but from a purely pragmatic perspective it is rather inconvenient not to be able to execute an operational semantics of a language, particularly of a simple one like our IMP. Moreover, since the state space of a CHAM can very quickly grow to unmanageable sizes even when the

sorts:

$Molecule$, $Solution$, $\mathbf{Bag}\{Molecule\}$ // generic CHAM sorts

subsorts:

$Solution < Molecule$ // generic CHAM subsort

Int, $Bool$, $Id < Molecule$ // additional IMP-specific syntactic categories

operations:

$\{\!|_|\!\} : \mathbf{Bag}\{Molecule\} \to Solution$ // generic membrane

$_ \triangleright _ : Molecule \times Solution \to Molecule$ // generic airlock

$_ \curvearrowright _ : Molecule \times Molecule \to Molecule$// "followed by" operator, for evaluation strategies

// Plus all the IMP language constructs and evaluation contexts (all listed in Figure 3.30),

// collapsing all syntactic categories different from Int, $Bool$ and Id into $Molecule$

rules:

// Airlock:

$\{\!|M\ Ms|\!\} \leftrightarrow \{\!|M \triangleright \{\!|Ms|\!\}|\!\}$

// Heating/cooling rules corresponding to the evaluation strategies of IMP's constructs:

$\{\!|(A_1 + A_2 \curvearrowright C)\ Ms|\!\} \leftrightarrow \{\!|(A_1 \curvearrowright \square + A_2 \curvearrowright C)\ Ms|\!\}$

$\{\!|(A_1 + A_2 \curvearrowright C)\ Ms|\!\} \leftrightarrow \{\!|(A_2 \curvearrowright A_1 + \square \curvearrowright C)\ Ms|\!\}$

$\{\!|(A_1 \ / \ A_2 \curvearrowright C)\ Ms|\!\} \leftrightarrow \{\!|(A_1 \curvearrowright \square \ / \ A_2 \curvearrowright C)\ Ms|\!\}$

$\{\!|(A_1 \ / \ A_2 \curvearrowright C)\ Ms|\!\} \leftrightarrow \{\!|(A_2 \curvearrowright A_1 \ / \ \square \curvearrowright C)\ Ms|\!\}$

$\{\!|(A_1 <= A_2 \curvearrowright C)\ Ms|\!\} \leftrightarrow \{\!|(A_1 \curvearrowright \square <= A_2 \curvearrowright C)\ Ms|\!\}$

$\{\!|(I_1 <= A_2 \curvearrowright C)\ Ms|\!\} \leftrightarrow \{\!|(A_2 \curvearrowright I_1 <= \square \curvearrowright C)\ Ms|\!\}$

$\{\!|(!\ B \curvearrowright C)\ Ms|\!\} \leftrightarrow \{\!|(B \curvearrowright !\ \square \curvearrowright C)\ Ms|\!\}$

$\{\!|(B_1 \ \&\& \ B2 \curvearrowright C)\ Ms|\!\} \leftrightarrow \{\!|(B_1 \curvearrowright \square \ \&\& \ B_2 \curvearrowright C)\ Ms|\!\}$

$\{\!|(X = A\texttt{;} \curvearrowright C)\ Ms|\!\} \leftrightarrow \{\!|(A = \square\texttt{;} \curvearrowright C)\ Ms|\!\}$

$\{\!|(S_1\ S_2 \curvearrowright C)\ Ms|\!\} \leftrightarrow \{\!|(S_1 \curvearrowright \square\ S_2 \curvearrowright C)\ Ms|\!\}$

$\{\!|S\ Ms|\!\} \leftrightarrow \{\!|(S \curvearrowright \square)\ Ms|\!\}$

$\{\!|(\texttt{if}\ (B)\ S_1\ \texttt{else}\ S_2 \curvearrowright C)\ Ms|\!\} \leftrightarrow \{\!|(B \curvearrowright \texttt{if}\ (\square)\ S_1\ \texttt{else}\ S_2 \curvearrowright C)\ Ms|\!\}$

// Semantic rewrite rules corresponding to reaction computational steps

$\{\!|\{\!|X \curvearrowright C|\!\}\ \{\!|X \mapsto I \triangleright \sigma|\!\}\ Ms|\!\} \to \{\!|\{\!|I \curvearrowright C|\!\}\ \{\!|X \mapsto I \triangleright \sigma|\!\}\ Ms|\!\}$

$\{\!|(I_1 + I_2 \curvearrowright C)\ Ms|\!\} \to \{\!|(I_1 +_{Int} I_2 \curvearrowright C)\ Ms|\!\}$

$\{\!|(I_1 \ / \ I_2 \curvearrowright C)\ Ms|\!\} \to \{\!|(I_1 \ /_{Int} \ I_2 \curvearrowright C)\ Ms|\!\}$ **if** $I_2 \neq 0$

$\{\!|(I_1 <= I_2 \curvearrowright C)\ Ms|\!\} \to \{\!|(I_1 \leq_{Int} I_2 \curvearrowright C)\ Ms|\!\}$

$\{\!|(!\ \texttt{true} \curvearrowright C)\ Ms|\!\} \to \{\!|(\texttt{false} \curvearrowright C)\ Ms|\!\}$

$\{\!|(!\ \texttt{false} \curvearrowright C)\ Ms|\!\} \to \{\!|(\texttt{true} \curvearrowright C)\ Ms|\!\}$

$\{\!|(\texttt{true} \ \&\& \ B2 \curvearrowright C)\ Ms|\!\} \to \{\!|(B_2 \curvearrowright C)\ Ms|\!\}$

$\{\!|(\texttt{false} \ \&\& \ B2 \curvearrowright C)\ Ms|\!\} \to \{\!|(\texttt{false} \curvearrowright C)\ Ms|\!\}$

$\{\!|(\{\ S\ \} \curvearrowright C)\ Ms|\!\} \to \{\!|(S \curvearrowright C)\ Ms|\!\}$

$\{\!|\{\!|X = I\texttt{;} \curvearrowright C|\!\}\ \{\!|X \mapsto J \triangleright \sigma|\!\}\ Ms|\!\} \to \{\!|\{\!|\{\}\ \curvearrowright C|\!\}\ \{\!|X \mapsto I \triangleright \sigma|\!\}\ Ms|\!\}$

$\{\!|(\{\}\ S_2 \curvearrowright C)\ Ms|\!\} \to \{\!|(S_2 \curvearrowright C)\ Ms|\!\}$

$\{\!|(\texttt{if}\ (\texttt{true})\ S_1\ \texttt{else}\ S_2 \curvearrowright C)\ Ms|\!\} \to \{\!|(S_1 \curvearrowright C)\ Ms|\!\}$

$\{\!|(\texttt{if}\ (\texttt{false})\ S_1\ \texttt{else}\ S_2 \curvearrowright C)\ Ms|\!\} \to \{\!|(S_2 \curvearrowright C)\ Ms|\!\}$

$\{\!|(\texttt{while}\ (B)\ S \curvearrowright C)\ Ms|\!\} \to \{\!|(\texttt{if}\ (B)\ \{\ S\ \texttt{while}\ (B)\ S\ \}\ \texttt{else}\ \{\} \curvearrowright C)\ Ms|\!\}$

$\{\!|(\texttt{int}\ xl\texttt{;}\ s)\ Ms|\!\} \to \{\!|\{\!|s|\!\}\ \{\!|xl \mapsto 0|\!\}\ Ms|\!\}$

// State initialization:

$\{\!|(X, Xl \mapsto I)\ Ms|\!\} \to \{\!|(X \mapsto I \triangleright \{\!|Xl \mapsto I|\!\})\ Ms|\!\}$

Figure 3.48: $\mathcal{R}_{\text{CHAM(IMP)}}$: The CHAM of IMP in rewrite logic.

```
mod CHAM is
  sorts Molecule Solution Bag{Molecule} .
  subsort Solution < Molecule < Bag{Molecule} .
  op empty : -> Bag{Molecule} .
  op _#_ : Bag{Molecule} Bag{Molecule} -> Bag{Molecule} [assoc comm id: empty] .
  op {|_|} : Bag{Molecule} -> Solution .
  op _<|_ : Molecule Solution -> Molecule [prec 110] .
  var M : Molecule .  var Ms : Bag{Molecule} .
  rl {| M # Ms |} => {| M <| {| Ms |} |} .
  rl {| M <| {| Ms |} |} => {| M # Ms |} .
endm

mod IMP-CHAM-SYNTAX is including PL-INT + CHAM .
  subsort Int < Molecule .
--- Define all the IMP constructs as molecule constructs
  op _+_ : Molecule Molecule -> Molecule [prec 33 gather (E e) format (d b o d)] .
  op _/_ : Molecule Molecule -> Molecule [prec 31 gather (E e) format (d b o d)] .
--- ... and so on
--- Add the hole as basic molecular construct, to allow for building contexts as molecules
  op [] : -> Molecule .
endm

mod IMP-HEATING-COOLING-CHAM-FAILED-1 is including IMP-CHAM-SYNTAX .
  var A1 A2 : Molecule .  var Ms : Bag{Molecule} .
--- + strict in its first argument
  rl {| (A1 + A2) # Ms |} => {| (A1 <| {| [] + A2 |}) # Ms |} .
  rl {| (A1 <| {| [] + A2 |}) # Ms |} => {| (A1 + A2) # Ms |} .
--- / strict in its second argument
  rl {| (A1 / A2) # Ms |} => {| (A2 <| {| A1 / [] |}) # Ms |} .
  rl {| (A2 <| {| A1 / [] |}) # Ms |} => {| (A1 / A2) # Ms |} .
--- ... and so on
endm

mod IMP-HEATING-COOLING-CHAM-FAILED-2 is including IMP-CHAM-SYNTAX .
  var A1 A2 : Molecule .  var Ms : Bag{Molecule} .  var C : Solution .
--- + strict in its first argument
  rl {| (A1 + A2 <| C) # Ms |} => {| (A1 <| {| {| [] + A2 <| C |} |}) # Ms |} .
  rl {| (A1 <| {| {| [] + A2 <| C |} |}) # Ms |} => {| (A1 + A2 <| C) # Ms |} .
--- / strict in its second argument
  rl {| (A1 / A2 <| C) # Ms |} => {| (A2 <| {| {| A1 / [] <| C |} |}) # Ms |} .
  rl {| (A2 <| {| {| A1 / [] <| C |} |}) # Ms |} => {| (A1 / A2 <| C) # Ms |} .
--- ... and so on
endm
```

Figure 3.49: Failed attempts to represent the CHAM of IMP in Maude using the airlock mechanism to define evaluation strategies. This figure also highlights the inconvenience of redefining IMP's syntax (the module `IMP-CHAM-SYNTAX` needs to redefine the IMP syntax as molecule constructs).

state space of the represented program is quite small, a direct representation of a CHAM in Maude can easily end up having only a theoretical relevance.

Before addressing the non-termination issue, it is instructive to discuss how a tool like Maude can help us pinpoint and highlight potential problems in our definitions. For example, we have previously seen how we failed, in two different ways, to use CHAM's airlock operator to define the evaluation strategies of the various IMP language constructs. We have noticed those problems with using the airlock for evaluation strategies by actually experimenting with CHAM definitions in Maude, more precisely by using Maude's search command to explore different behaviors of a program. We next discuss how one can use Maude to find out that both our attempts to use airlock for evaluation strategies fail. Figure 3.49 shows all the needed modules. `CHAM` defines the generic syntax of the chemical abstract machine together with its airlock rules in Maude, assuming only one type of molecule. `IMP-CHAM-SYNTAX` defines the syntax of IMP as well as the syntax of IMP's evaluation contexts as a syntax for molecules; since there is only one syntactic category for syntax now, namely `Molecule`, adding $\square$ as a `Molecule` constant allows for `Molecule` to also include all the IMP evaluation contexts, as well as many other garbage terms (e.g., $\square + \square$, etc.). The module `IMP-HEATING-COOLING-CHAM-FAILED-1` represents in Maude, using the general translation of CHAM rules into rewrite logic rules shown in Figure 3.47, heating/cooling rules of the form

$$a_1 + a_2 \quad \rightleftharpoons \quad a_1 \triangleleft \{\!\{\square + a_2\}\!\}$$

Only two such groups of rules are shown, which is enough to show that we have a problem. Indeed, all four Maude search commands below succeed:

```
search[1] {| 3 / (1 + 2) |} =>* {| 1 ([] + 2) (3 / []) |} .
search[1] {| 1 ([] + 2) (3 / []) |} =>* {| 3 / (1 + 2) |} .
search[1] {| (3 / 1) + 2 |} =>* {| 1 ([] + 2) (3 / []) |} .
search[1] {| 1 ([] + 2) (3 / []) |} =>* {| (3 / 1) + 2 |} .
```

That means that the Maude solution terms `{| 3 / (1 + 2) |}` and `{| (3 / 1) + 2 |}` can rewrite into each other, which is clearly wrong. Similarly, `IMP-HEATING-COOLING-CHAM-FAILED-2` represents in Maude heating/cooling rules of the form

$$(a_1 + a_2) \triangleleft c \quad \rightleftharpoons \quad a_1 \triangleleft \{\!\{(\square + a_2) \triangleleft c\}\!\}$$

One can now check that this second approach gives us what we wanted, that is, a chemical representation of syntax where each subsolution directly contains no more than one $\square$:

```
search[1] {| 3 / (1 + 2) |} =>* {| 1 {| ([] + 2) {| 3 / [] |} |} |} .
search[1] {| (3 / 1) + 2 |} =>* {| 1 {| (3 / []) {| [] + 2 |} |} |} .
```

Indeed, both Maude search commands above succeed. Unfortunately, the following commands

```
search[1] {| 3 / (1 + 2) |} =>* {| 1 {| [] + 2 |} {| 3 / [] |} |} .
search[1] {| 1 {| [] + 2 |} {| 3 / [] |} |} =>* {| 3 / (1 + 2) |} .
search[1] {| (3 / 1) + 2 |} =>* {| 1 {| [] + 2 |} {| 3 / [] |} |} .
search[1] {| 1 {| [] + 2 |} {| 3 / [] |} |} =>* {| (3 / 1) + 2 |} .
```

also succeed, showing that our second attempt to use the airlock for evaluation strategies fails, too.

One could also try the following, which should also succeed (proof is the searches above):

```
search[1] {| 3 / (1 + 2) |} =>* {| (3 / 1) + 2 |} .
search[1] {| (3 / 1) + 2 |} =>* {| 3 / (1 + 2) |} .
```

```
mod CHAM is
  sorts Molecule Solution Bag{Molecule} .
  subsort Solution < Molecule < Bag{Molecule} .
  op empty : -> Bag{Molecule} .
  op _#_ : Bag{Molecule} Bag{Molecule} -> Bag{Molecule} [assoc comm id: empty] .
  op {|_|} : Bag{Molecule} -> Solution .
  op _<|_ : Molecule Solution -> Molecule [prec 110] .
--- The airlock is unnecessary in this particular example.
--- We keep it, though, just in case will be needed as we extend the language.
--- We comment the two airlock rules below out to avoid non-termination.
--- Otherwise we would have to use slower search commands instead of rewrite.
---  var M : Molecule .  var Ms : Bag{Molecule} .
---  rl {| M # Ms |} => {| M <| {| Ms |} |} .
---  rl {| M <| {| Ms |} |} => {| M # Ms |} .
endm
```

Figure 3.50: Generic representation of the CHAM in Maude.

However, on our machine (Linux, 2.4GHz, 8GB memory) Maude 2 ran out of memory after several minutes when asked to execute any of the two search commands above.

Figures 3.50, 3.51 and 3.52 give a correct Maude representation of CHAM(IMP), based on the rewrite logic theory $\mathcal{R}_{\text{CHAM(IMP)}}$ in Figure 3.48. Three important observations were the guiding factors of our Maude semantics of CHAM(IMP):

1. While the approach to syntax in Figure 3.49 elegantly allows to include the syntax of evaluation contexts into the syntax of molecules by simply defining $\square$ as a molecular construct, unfortunately it still requires us to redefine the entire syntax of IMP into specific constructs for molecules. This is inconvenient at best. We can do better by simply subsorting to `Molecule` all those syntactic categories that the approach in Figure 3.49 would collapse into `Molecule`. This way, any fragment of IMP code parses to a subsort of `Molecule`, so in particular to `Molecule`. Unfortunately, evaluation contexts are now ill-formed; for example, $\square + 2$ attempts to sum a molecule with an integer, which does not parse. To fix this, we define a new sort for the $\square$ constant, say `Hole`, and declare it as a subsort of each IMP syntactic category that is subsorted to `Molecule`. In particular, `Hole` is a subsort of `AExp`, so $\square + 2$ parses to `AExp`.

2. As discussed above, most of CHAM's heating/cooling rules are reversible and thus, when regarded as rewrite rules, lead to non-termination. To avoid non-termination, we restrict the heating/cooling rules so that heating only applies when the heated subterm has computational contents (i.e., it is not a result) while cooling only applies when the cooled term is completely processed (i.e., it is a result). This way, the heating and the cooling rules are applied in complementary situations, in particular they are not reversible anymore, thus avoiding non-termination. To achieve this, we introduce a subsort `Result` of `Molecule`, together with subsorts of it corresponding to each syntactic category of IMP as well as with an explicit declaration of IMP's results as appropriate terms of corresponding result sort.

3. The CHAM's philosophy is to represent data, in particular program states, using its builtin support for solutions as multisets of molecules, and to use airlock operations to extract pieces of data from such solutions whenever needed. This philosophy is justified both by chemical and by mathematical intuitions, namely that one needs an additional step to observe inside a solution and, respectively, that multiset matching is a complex operation (it is actually an intractable problem) whose complexity

```
mod IMP-HEATING-COOLING-CHAM is including IMP-SYNTAX + CHAM .
  sorts Hole ResultAExp ResultBExp ResultStmt Result .
  subsorts Hole < AExp BExp Stmt < Molecule .
  subsorts ResultAExp ResultBExp ResultStmt < Result < Molecule .
  subsorts Int < ResultAExp < AExp .
  subsorts Bool < ResultBExp < BExp .
  subsorts ResultStmt < Block .
  op {} : -> ResultStmt [ditto] .

  op [] : -> Hole .
  op _~>_ : Molecule Molecule -> Molecule [gather(e E) prec 120] .

  var X : Id .  var C : [Molecule] .  var A A1 A2 : AExp .  var R R1 R2 : Result .
  var B B1 B2 : BExp .  var I I1 I2 : Int .  var S S1 S2 : Stmt .  var Ms : Bag{Molecule} .

 crl {| (A1 + A2 ~> C) # Ms |} => {| (A1 ~> [] + A2 ~> C) # Ms |} if notBool(A1 :: Result) .
  rl {| (R1 ~> [] + A2 ~> C) # Ms |} => {| (R1 + A2 ~> C) # Ms |} .

 crl {| (A1 + A2 ~> C) # Ms |} => {| (A2 ~> A1 + [] ~> C) # Ms |} if notBool(A2 :: Result) .
  rl {| (R2 ~> A1 + [] ~> C) # Ms |} => {| (A1 + R2 ~> C) # Ms |} .

 crl {| (A1 / A2 ~> C) # Ms |} => {| (A1 ~> [] / A2 ~> C) # Ms |} if notBool(A1 :: Result) .
  rl {| (R1 ~> [] / A2 ~> C) # Ms |} => {| (R1 / A2 ~> C) # Ms |} .

 crl {| (A1 / A2 ~> C) # Ms |} => {| (A2 ~> A1 / [] ~> C) # Ms |} if notBool(A2 :: Result) .
  rl {| (R2 ~> A1 / [] ~> C) # Ms |} => {| (A1 / R2 ~> C) # Ms |} .

 crl {| (A1 <= A2 ~> C) # Ms |} => {| (A1 ~> [] <= A2 ~> C) # Ms |} if notBool(A1 :: Result) .
  rl {| (R1 ~> [] <= A2 ~> C) # Ms |} => {| (R1 <= A2 ~> C) # Ms |} .

 crl {| (R1 <= A2 ~> C) # Ms |} => {| (A2 ~> R1 <= [] ~> C) # Ms |} if notBool(A2 :: Result) .
  rl {| (R2 ~> R1 <= [] ~> C) # Ms |} => {| (R1 <= R2 ~> C) # Ms |} .

 crl {| (! B ~> C) # Ms |} => {| (B ~> ! [] ~> C) # Ms |} if notBool(B :: Result) .
  rl {| (R ~> ! [] ~> C) # Ms |} => {| (! R ~> C) # Ms |} .

 crl {| (B1 && B2 ~> C) # Ms |} => {| (B1 ~> [] && B2 ~> C) # Ms |} if notBool(B1 :: Result) .
  rl {| (R1 ~> [] && B2 ~> C) # Ms |} => {| (R1 && B2 ~> C) # Ms |} .

 crl {| (X = A ; ~> C) # Ms |} => {| (A ~> X = [] ; ~> C) # Ms |} if notBool(A :: Result) .
  rl {| (R ~> X = [] ; ~> C) # Ms |} => {| (X = R ; ~> C) # Ms |} .

 crl {| (S1 S2 ~> C) # Ms |} => {| (S1 ~> [] S2 ~> C) # Ms |} if notBool(S1 :: Result) .
  rl {| (R1 ~> [] S2 ~> C) # Ms |} => {| (R1 S2 ~> C) # Ms |} .

 crl {| S # Ms |} => {| (S ~> []) # Ms |} if notBool(S :: Result) .
  rl {| (R ~> []) # Ms |} => {| R # Ms |} .

 crl {| (if (B) S1 else S2 ~> C) # Ms |} => {| (B ~> if ([]) S1 else S2 ~> C) # Ms |}
  if notBool(B :: Result) .
  rl {| (R ~> if ([]) S1 else S2 ~> C) # Ms |} => {| (if (R) S1 else S2 ~> C) # Ms |} .
endm
```

Figure 3.51: Efficient heating-cooling rules for IMP in Maude.

```
mod IMP-SEMANTICS-CHAM is including IMP-HEATING-COOLING-CHAM + STATE .
  subsort Pgm State < Molecule .
  var X : Id .  var Xl : List{Id} .  var C : Molecule .  var Ms : Bag{Molecule} .
  var Sigma : State .  var B B2 : BExp .  var I J I1 I2 : Int .  var S S1 S2 : Stmt .
  rl {| {| X ~> C |} # {| X |-> I & Sigma |} # Ms |}
  => {| {| I ~> C |} # {| X |-> I & Sigma |} # Ms |} .
  rl {| (I1 + I2 ~> C) # Ms |} => {| (I1 +Int I2 ~> C) # Ms |} .
 crl {| (I1 / I2 ~> C) # Ms |} => {| (I1 /Int I2 ~> C) # Ms |} if I2 =/=Bool 0 .
  rl {| (I1 <= I2 ~> C) # Ms |} => {| (I1 <=Int I2 ~> C) # Ms |} .
  rl {| (! true ~> C) # Ms |} => {| (false ~> C) # Ms |} .
  rl {| (! false ~> C) # Ms |} => {| (true ~> C) # Ms |} .
  rl {| (true && B2 ~> C) # Ms |} => {| (B2 ~> C) # Ms |} .
  rl {| (false && B2 ~> C) # Ms |} => {| (false ~> C) # Ms |} .
  rl {| ({S} ~> C) # Ms |} => {| (S ~> C) # Ms |} .
  rl {| {| X = I ; ~> C |} # {| X |-> J & Sigma |} # Ms |}
  => {| {| {} ~> C |} # {| X |-> I & Sigma |} # Ms |} .
  rl {| ({} S2 ~> C) # Ms |} => {| (S2 ~> C) # Ms |} .
  rl {| (if (true) S1 else S2 ~> C) # Ms |} => {| (S1 ~> C) # Ms |} .
  rl {| (if (false) S1 else S2 ~> C) # Ms |} => {| (S2 ~> C) # Ms |} .
  rl {| (while (B) S ~> C) # Ms |} => {| (if (B) {S while (B) S} else {} ~> C) # Ms |} .
  rl {| (int Xl ; S) # Ms |} => {| {| S |} # {| Xl |-> 0 |} # Ms |} .
endm
```

Figure 3.52: The CHAM of IMP in Maude.

cannot be simply "swept under the carpet". While we agree with these justifications for the airlock operator, one should also note that impressive progress has been made in the last two decades, after the proposal of the chemical abstract machine, in terms of multiset matching. For example, languages like Maude build upon very well-engineered multiset matching techniques. We believe that these recent developments justify us, at least in practical language definitions, to replace the expensive airlock operation of the CHAM with the more available and efficient multiset matching of Maude.

Figure 3.50 shows the generic CHAM syntax that we extend in order to define CHAM(IMP). Since we use Maude's multiset matching instead of state airlock and since we cannot use the airlock for evaluation strategies either, there is effectively no need for the airlock in our Maude definition of CHAM(IMP). Moreover, since the airlock rules are reversible, their introduction would yield non-termination. Consequently, we have plenty of reasons to eliminate them, which is reflected in our `CHAM` module in Figure 3.50. The Maude module in Figure 3.51 defines the evaluation strategies of IMP's constructs and should be now clear: in addition to the syntactic details discussed above, it simply gives the Maude representation of the heating/cooling rules in Figure 3.48. Finally, the Maude module in Figure 3.52 implements the semantic rules and the state initialization heating rule in Figure 3.48, replacing the state airlock operation by multiset matching.

To test the heating/cooling rules, one can write Maude commands such as the two search commands below, asking Maude to search for all heatings of a given syntactic molecule (the sort of X, *Id*, is already declared in the last module):

```
search {| X = 3 / (X + 2) ; |} =>! Sol:Solution .
search {| X = (Y:Id + Z:Id) / (X + 2) ; |} =>! Sol:Solution .
```

The former gives only one solution, because heating can only take place on non-result subexpressions

```
Solution 1 (state 4)
states: 5  rewrites: 22 in ... cpu (... real) (... rewrites/second)
Sol:Solution --> {| X ~> [] + 2 ~> 3 / [] ~> X = [] ; ~> [] |}
```

but the latter gives three solutions:

```
Solution 1 (state 5)
states: 8  rewrites: 31 in ... cpu (... real) (... rewrites/second)
Sol:Solution --> {| Y:Id ~> [] + Z:Id ~> [] / (X + 2) ~> X = [] ; ~> [] |}

Solution 2 (state 6)
states: 8  rewrites: 31 in ... cpu (... real) (... rewrites/second)
Sol:Solution --> {| Z:Id ~> Y:Id + [] ~> [] / (X + 2) ~> X = [] ; ~> [] |}

Solution 3 (state 7)
states: 8  rewrites: 31 in ... cpu (... real) (~ rewrites/second)
Sol:Solution --> {| X ~> [] + 2 ~> (Y:Id + Z:Id) / [] ~> X = [] ; ~> [] |}
```

Each of the solutions represents a completely heated term (we used `=>!` in the search commands) and corresponds to a particular order of evaluation of the subexpression in question. The three solutions of the second search command above reflect all possible orders of evaluation allowed by our Maude semantics of CHAM(IMP). Interestingly, there is no order in which X is looked up in between Y and Z. This is not a problem for IMP, but it may result in loss of behaviors for IMP++ programs. Indeed, it may be that other threads modify the values of X, Y, and Z while the expression above is evaluated by another thread in such a way that behaviors are lost if X is not allowed to be looked up between Y and Z. Consequently, our orientation of the heating/cooling rules came at a price: we lost the fully non-deterministic order of evaluation of the arguments of strict operators; what we obtained is a *non-deterministic choice* evaluation strategy (an order of evaluation is non-deterministically chosen and cannot be changed during the evaluation of the expression—this is discussed in more depth in Section 3.5).

Maude can now act as an execution engine for CHAM(IMP). For example, the Maude command

```
rewrite {| sumPgm |} .
```

where `sumPgm` is the first program defined in the module `IMP-PROGRAMS` in Figure 3.4, produces a result of the form:

```
rewrite in TEST : {| sumPgm |} .
rewrites: 7543 in ... cpu (... real) (... rewrites/second)
result Solution: {| {| {} |} # {| n |-> 0 & s |-> 5050 |} |}
```

Like in the previous Maude semantics, one can also search for all possible behaviors of a program using `search` commands such as

```
search {| sumPgm |} =>! Sol:Solution .
```

Like before, only one behavior will be discovered (IMP is deterministic so far). However, an unexpectedly large number of states is generated, 4119 versus the 1709 states generated by the previous small-step semantics), mainly due to the multiple ways to apply the heating/cooling rules:

```
Solution 1 (state 4118)
states: 4119  rewrites: 12658 in ... cpu (... real) (... rewrites/second)
Sol:Solution --> {| {| {} |} # {| n |-> 0 & s |-> 5050 |} |}
```

3.8.4 Notes

The chemical abstract machine, abbreviated CHAM, was introduced by Berry and Boudol in 1990 [8, 9]. In spite of its operational feel, the CHAM should not be mistakenly taken for a variant of (small-step) SOS. In

fact, Berry and Boudol presented the CHAM as an *alternative* to SOS, to address a number of limitations inherent to SOS, particularly its lack of true concurrency and what they called SOS' "rigidity to syntax". The basic metaphor giving its name to the CHAM was inspired by Banâtre and Le Mètayer's GAMMA language [4, 5, 6], which was the first to view a distributed state as a solution in which many molecules float, and the first to understand concurrent transitions as reactions that can occur simultaneously in many points of the solution. GAMMA was proposed as a highly-parallel programming language, together with a stepwise program derivation approach that allows to develop provably correct GAMMA programs. However, following the stepwise derivation approach to writing GAMMA programs is not as straightforward as writing CHAM rules. Moreover, CHAM's nesting of solutions allows for structurally more elaborate encodings of data and in particular for more computational locality than GAMMA. Also, the CHAM appears to be more suitable as a framework for defining semantics of programming languages than the GAMMA language; the latter was mainly conceived as a programming language itself, suitable for executing parallel programs on parallel machines [3], rather than as a semantic framework.

The distinction between heating, cooling and reaction rules in CHAM is in general left to the user. There are no well-accepted criteria and/or principles stating when a rule should be in one category or another. For example, should a rule that cleans up a solution by removing residue molecules be a heating or a colling rule? We prefer to think of it as a cooling rule, because it falls under the broad category of rules which "structurally rearrange the solution after reactions take place" which we methodologically decided to call cooling rules. However, the authors of the CHAM prefer to consider such rules to be heating rules, with the intuition that "the residue molecules evaporate when heated" [9]. While the distinction between heating and cooling rules may have a flavor of subjectivity, the distinction between actual reaction rules and heating/cooling rules is more important because it gives the computational granularity of one's CHAM. Indeed, it is common to abstract away the heating/cooling steps in a CHAM rewriting sequence as internal steps and then define various relations of interest on the remaining reaction steps possibly relating different CHAMS, such as behavioral equivalence, simulation and/or bisimulation relations [8, 9].

The technique we used in this section to reversibly and possibly non-deterministically sequentialize the program syntax by heating/cooling it into a list of computational tasks was borrowed from the K framework [68, 66, 45, 64] (also Section 3.12). This mechanism is also reminiscent of Danvy and Nielsen's refocusing technique [19, 20], used to execute reduction semantics with evaluation contexts by decomposing evaluation contexts into stacks and then only incrementally modifying these stacks during the reduction process. Our representation of the CHAM into rewrite logic was inspired from related representations by Șerbănuță *et al.* [74] and by Meseguer[12] [42]. However, our representation differs in that it enforces the CHAM rewriting to only take place in solutions, this way being completely faithful to the intended meaning of the CHAM reactions, while the representations in [74, 42] are slightly more permissive, allowing rewrites to take place everywhere rules match; as explained, this is relevant only when the left-hand-side of the rule contains precisely one molecule.

We conclude this section with a note on the concurrency of CHAM rewriting. As already explained, we are not aware of any formal definition of a CHAM rewriting relation that captures the truly concurrent computation advocated by the CHAM. This is unfortunate, because its concurrency potential is one of the most appealing aspects of the CHAM. Moreover, as already discussed (after Theorem 21), our rewrite logic representation of the CHAM is faithful only with respect to one non-concurrent step and interleaves steps taking place within the same solutions no matter whether they involve common molecules or not, so we cannot borrow rewrite logic's concurrency to obtain a concurrent semantics for the CHAM. What can be done,

[12]Meseguer [42] was published in the same volume of the Journal of Theoretical Computer Science as the CHAM extended paper [9] (the first CHAM paper [8] was published two years before, in 1990, same as the first papers on rewrite logic [40, 39, 41]).

however, is to attempt a different representation of the CHAM into rewrite logic based on ideas proposed by Meseguer in [43] to capture (a limited form of) graph rewriting by means of equational encodings. The encoding in [43] is theoretically important, but, unfortunately, yields rewrite logic theories which are not feasible in practice using the current implementation of Maude.

3.8.5 Exercises

Exercise 168. *For any CHAM, any molecules $mol_1, \ldots, mol_k$, and any $1 \leq i \leq k$, the sequents*

$$\text{CHAM} \vdash \{\!|mol_1 \ \ldots \ mol_k|\!\} \leftrightarrow \{\!|mol_1 \rhd \{\!|mol_2 \ \ldots mol_i|\!\} \ mol_{i+1} \ldots mol_k|\!\}$$

are derivable, where if $i = 1$ then the bag $mol_2 \ \ldots mol_i$ is the empty bag.

Exercise 169. *Modify the CHAM semantics of* IMP *in Figure 3.45 to use a state data-structure as we did in the previous semantics instead of representing the state as a solution of binding molecules.*

Exercise 170. *Add a cleanup (cooling) rule to the CHAM semantics of* IMP *in Figure 3.45 to remove the useless syntactic subsolution when the computation is terminated. The resulting solution should only contain a state (i.e., it should have the form $\{\!|\sigma|\!\}$, and not $\{\!|\{\!|\sigma|\!\}|\!\}$ or $\{\!|\{\!|\cdot|\!\} \ \{\!|\sigma|\!\}|\!\}$).*

Exercise 171. *Modify the CHAM semantics of* IMP *in Figures 3.44 and 3.45 so that / short-circuits when the numerator evaluates to* 0.
Hint: One may need to make one of the heating/cooling rules for / conditional.

Exercise 172. *Modify the CHAM semantics of* IMP *in Figures 3.44 and 3.45 so that conjunction is non-deterministically strict in both its arguments.*

Exercise 173. *Same as Exercise 84, but for CHAM instead of big-step SOS: add variable increment to* IMP*, like in Section 3.8.2*
★ *Like for the Maude definition of big-step SOS and unlike for the Maude definitions of small-step SOS, MSOS and reduction semantics with evaluation contexts above, the resulting Maude definition can only exhibit three behaviors (instead of five) of the program* `nondet++Pgm` *in Exercise 84. This limitation is due to our decision (in Section 3.8) to only heat on non-results and cool on results when implementing CHAMs into Maude. This way, the resulting Maude specifications are executable at the expense at losing some of the behaviors due to non-deterministic evaluation strategies.*

Exercise 174. *Same as Exercise 88, but for the CHAM instead of big-step SOS: add input/output to* IMP*, like in Section 3.8.2.*

Exercise 175. *Same as Exercise 92, but for the CHAM instead of big-step SOS: add abrupt termination to* IMP*, like in Section 3.8.2.*

Exercise 176. *Same as Exercise 104, but for the CHAM instead of small-step SOS: add dynamic threads to* IMP*, like in Section 3.8.2.*

Exercise⋆ 177. *This exercise asks to define* IMP++ *in CHAM, in various ways. Specifically, redo Exercises 114, 115, 116, 117, and 118, but for the CHAM of* IMP++ *discussed in Section 3.8.2 instead of its small-step SOS in Section 3.5.6.*

Add more citations here from others who used CHAMS to give semantics to different calculi or languages. Also, check whether the graph rewriting community has given the CHAM its desired concurrent semantics, using graph rewriting techniques. Ask Berry and Boudol. Also check with Corradinni and Gadducci.

Add a new (sub)section on CHAM with env/store, like in the Maude code

Modify the CHAM definitions to dissolve statements, as we do in K

Add different types of molecules, and bags of them: for syntax, for state, for environments, location, etc.

3.9 Advanced Denotational Semantics: Continuations, Monads and Powerdomains

this should actually also discuss other advanced denotational semantics features, such as continuations and powerdomains

3.10 Discussion: Advantages, Disadvantages and Comparisons

don't forget to say that big-step cannot even define a non-deterministic plus

have a discussion that syntactic purity may lead to annoying "encodings", such as the one in Exercise 130.

discuss call/cc; we refer to it several times

One could argue that the changes above were necessary because of poor configuration management: instead of defining configurations as triples, one could have defined them as nested configurations of the form $\langle\langle s, \sigma\rangle, \omega\rangle$, and then only add the evaluation contexts "$\mathit{Context} ::= \ldots \mid \langle \mathit{Context}, \mathbf{List}\{\mathit{Int}\}\rangle \mid \texttt{output(}\mathit{Context}\texttt{)}$" and the output rule $\langle c, \omega\rangle[\texttt{output(}i\texttt{)}] \to \langle c, (\omega, i)\rangle[\{\}]$ and change nothing else. Indeed, one would need to change nothing else because all the other rules would apply unchanged in contexts of the form $\langle \square, \omega\rangle$.

3.11 Existing Approaches and Their Limitations for Programming Language Semantics

taken from JLAP'10 submission; needs to redistributed in sections above, as well as in Chapter on K

Since K was motivated by the need for a semantic framework satisfying the requirements discussed in Section 1, it is important to understand the limitations of the existing approaches as semantic frameworks for programming languages. In this section we group the relevant existing approaches into two categories: structural operational semantics frameworks and rewriting frameworks. For each approach we list its main characteristics, its limitations, and its relationship with K.

3.11.1 Structural Operational Semantics (SOS) Frameworks

We here discuss the most common SOS approaches, namely big-step and small-step SOS, their modular variant MSOS, and reduction semantics with evaluation contexts.

Big-Step SOS Introduced as natural semantics in [34], also named relational semantics[49], or evaluation semantics, big-step semantics is "the most denotational" of the operational semantics. One can view big-step definitions as definitions of functions interpreting each language construct in an appropriate domain. Big-step operational semantics can be easily and efficiently interpreted in any recursive, functional or logical framework. It is particularly useful for defining type systems.
Limitations: Due to its monolithic, single-step evaluation, it is hard to debug or trace big-step semantic definitions. If the program is wrong, no information is given about where the failure occurred. Divergence is not observable in the evaluation relation. It may be hard or impossible to model concurrent features. It is not modular, e.g., to add side effects to expressions, one must redefine the rules to allow expressions to evaluate to pairs (value-store). It is inconvenient (and non-modular) to define complex control statements.
Relationship to K: None.

Small-step SOS Introduced by Plotkin in [60, 61], also called transition semantics or reduction semantics, small-step semantics captures the notion of one computational step. Therefore, it stops at errors, pointing them out and it is easy to trace and debug. It gives interleaving semantics for concurrency.
Limitations: Like big-step, it is non-modular. It does not give a "true concurrency" semantics, that is, one has to choose a certain interleaving (no two rules can be applied on the same term at the same time), mainly because reduction is forced to occur only at the top. It is still hard to deal with control—one has to add corner cases (additional rules) to each statement to propagate control changing information. Each small step traverses the entire program to find the next redex; since the size of the program can grow unbounded, each small step may take unbounded resources in the worst case.
Relationship to K: None.

Modular SOS Mosses introduced *modular SOS (MSOS)* [51, 52, 53] to deal with the non-modularity of small-step and big-step SOS. The MSOS solution involves moving the non-syntactic state components as labels on transitions, plus a discipline of only selecting needed attributes from the states.
Limitations: Same as the ones above, except for modularity.
Relationship to K: Both MSOS and K make use of labeled information to achieve modularity. MSOS uses labels as record fields on the transition relation, while K uses labels as cell names in the configuration. In both MSOS and K one can use the labels in semantic rules to only refer to those items of interest.

Reduction semantics with evaluation contexts Introduced by Felleisen and colleagues (see, e.g., [25, 88]), the evaluation contexts style improves over small-step SOS in two ways: (1) it gives a more compact semantics to context-sensitive reduction, by using parsing to find the next redex rather than small-step rules; and (2) it provides the possibility of also modifying the context in which a reduction occurs, making it much easier to deal with control-intensive features. Additionally, one can also incorporate the configuration as part of the evaluation context, and thus have full access to semantic information "by need".
Limitations: It still only allows "interleaving semantics" for concurrency. It is too "rigid to syntax", in that it is hard or impossible to define semantics in which values are more than plain syntax; for example, one cannot give a syntactic semantics to a functional language based on closures for functions (instead of substitution), because one needs special, non-syntactic means to handle and recover environments (as we do in K, see Sections FIXTHIS). Although context-sensitive rewriting might seem to be easily implementable by rewriting, in fact one has to perform an amount of "parsing" work linear in the size of the program for each computational step, like in small-step SOS. However, one might obtain efficient implementations for restricted forms of context-reduction definitions by applying refocusing techniques [19].
Relationship to K: Both reduction semantics and K make use of evaluation contexts and can store or modify them. Reduction semantics relies on a mechanism to "split/plug" syntax into contexts, while K relies on a mechanism to "heat/cool" syntax into computations. The former is achieved by an implicit "advanced" parsing of syntax into a context and a redex, while the latter is achieved using rewrite rules.

3.11.2 Rewriting Frameworks

We here discuss some rewriting approaches which have been used or have the potential to be used for programming language semantics.

Term Rewriting and Rewrite Logic Rewrite logic was proposed by Meseguer [42] as a logical formalism generalizing both equational logic and term rewriting, so it more suitable for language semantics than plain

term rewriting. The rewrite logic semantics (RLS) project [46, 48] is an initiative aiming at using rewrite logic for various semantic aspects of programming languages; a substantial body of work on RLS, both of theoretical and practical nature, is discussed in [46, 48], which we do not repeat here. It is worth mentioning that all the various SOS semantic approaches in Section 3.11.1 (and others not discussed here) can be framed as particular definitional styles in rewriting logic and then executed using Maude [74].
Limitations: Rewrite logic is a general-purpose computational framework, which provides the infrastructure but *no hints on how* to define a language semantics as a rewrite system. Definitional styles need to be represented within rewriting as shown in [74], such as those in Section 3.11.1. Additionally, rule instances cannot overlap, which results in an enforcement of interleaving in some situations where one would like to have true concurrency. For example, two threads reading from a shared store have to interleave the store operations even if they act on different locations, because the corresponding rule instances overlap on the store; some hints are given in [43] on how to use equations in an ingenious way to still achieve theoretical true concurrency in the presence of sharing, but that approach is impractical for language semantics.
Relationship to K: Like rewrite logic, K is a computational framework that aims at maximizing the amount of concurrency that can be achieved by means of term rewriting. However, one K concurrent rewrite step may take several interleaved rewrite logic steps.

Graph Rewriting Graph rewriting [22] extends the intuitions of term rewriting to arbitrary graphs, by developing a match-and-apply mechanism which can work directly on graphs.
Limitations: Graph rewriting is rather complex and we are not aware of works using it for programming language semantics. Part of the reason could be the notorious difficulty of graph rewriting to deal with structure copying and equality testing, operations which are crucial for programming language semantics and relatively easy to provide by term rewriting systems. Finally, in spite of decades of research, at our knowledge there are still no graph rewriting engines comparable in performance to term rewrite engines.
Relationship to K: The K rules, like those in graph-rewriting, have both read-only components, which could be shared among rule instances to maximize concurrency, and write-only components.

The Chemical Abstract Machine Berry and Boudol's chemical abstract machine [9], or CHAM, is both a rewrite-based model of concurrency and a specific style of giving language semantics. The CHAM views a distributed state as a "solution" in which many "molecules" float, and understands concurrent transitions as "reactions" that can occur simultaneously in many points of the solution.
Limitations: While chemistry as a model of computation sounds attractive, technically speaking the CHAM is in fact a restricted case of rewrite logic, as noticed in [42]. Moreover, some of the chemical "intuitions", such as the airlock operation which imposes a particular chemical "discipline" to access molecules inside a solution, inhibit the potential for the now well-understood and efficient matching and rewriting *modulo associativity and commutativity*. In fact, at our knowledge, there is no competitive implementation of the CHAM. Although this solution-molecule paradigm seems to work for languages with simple state structure, it is not clear how one could represent the state for complex languages with threads, locks, environments, etc. Finally, CHAMs provide no mechanism to freeze the current molecular structure as a "value", and then to store or retrieve it, as we would need to define language features like call/cc. It would therefore seem hard to define complex control-intensive language features in CHAM.
Relationship to K: Like the CHAM, K also organizes the configuration of the program or system as a potentially nested structure of molecules (called cells and potentially labeled in K). Like in CHAM, the configuration is also rewritten until it "stabilizes". Unlike in CHAM, the K rules can match and write inside and across multiple cells in one parallel step. Also, unlike in CHAM (and in rewrite logic), K rewrite rules

can apply concurrently even in cases when they overlap (read-only structure).

P-Systems Păun's membrane systems (or P-systems) [58] are computing devices abstracted from the structure and the functioning of the living cell. In classical *transition P-systems*, the main ingredients of such a system are the *membrane structure*, in the compartments of which *multisets* of symbol-objects evolve according to given *evolution rules*. The rules are localized, associated with the membranes (hence, with the compartments), and they are used in a *nondeterministic maximally parallel* manner.
Limitations: When looked at from a programming language semantics perspective, the P-systems have a series of limitations that have been addressed in K, such as: (1) Lack of structure for non-membrane terms, which are plain constants; and (2) Strict membrane locality (even stricter than in the CHAM), which prevents rules to match and rewrite within multiple membranes at the same time. Regarding (1), programming languages often handle complex data-structures or make use of complex semantic structures, such as environments mapping variables to values, or closures holding code as well as environments, etc., which would be hard or impossible to properly encode using just constants. Regarding (2), strict membrane locality would require one to write many low-level rules and introduce and implement by local rules artificial "cross-membrane communication protocols". For example, the semantics of variable lookup involves acquiring the location of the variable from the environment cell (which holds a map), then the value at that location in the store cell (which holds another map), and finally the rewrite of the variable in the code cell into that value. All these require the introduction of encoding constants and rules in several membranes and many computation steps; compare that to the one step K rule in Section FIXTHIS.
Relationship to K: K-systems share with P-systems the ideas of cell structures and the aim to maximize concurrency. However, K rules can span across multiple cells at a time, and the objects contained in a cell can have a rich algebraic structure (as opposed to being constants, as in P-systems). [75] presents a in-depth comparison between the two formalisms, showing how one can use K to model P-systems.

3.11.3 *old limitations stuff about big-step and small-step SOS*

Big-Step vs. Small-Step: Advantages/Disadvantages

The theorem above tells us that the two styles of semantics achieve eventually the same objective, at least for the simple language that we considered: they tell formally when a program p evaluates to an integer i. So when do we choose one versus the other when we define a programming language? To answer this question, one needs to reflect on and to understand the advantages and the disadvantages of each of the two definitional styles.

Advantages of Big-Step SOS.

- When it can be given to a language, it is *easy to understand* since it relates syntactic entities directly to their expected results.

- Because of its recursive nature, big-step SOS gives a strong intuition on how to *implement an interpreter* for a language. Many proponents of big-step SOS regard it "almost as an interpreter" for the defined language.

- *More abstract*, more mathematical/denotational; therefore, one can more easily define and *prove properties* about programs; of course, because of the equivalence of the two semantics, one can also use the transitive closure of the small-step SOS transition relation, but this may be less natural than using the big-step semantics. For example, try to prove the following using both big-step and small-step semantics:

Exercise 178. *Formulate using big-step and small-step SOS, respectively, what it means for the statements* while b (while b s) *and* while b s *to be "equivalent", and then prove it.*

- Particularly useful when defining *type systems* of programming languages; there, values are replaced by their types.

Disadvantages of Big-Step SOS.

- As given, big-step SOS is *not executable*: one always needs to provide a result value or state to each language construct and then use the SOS rules to "check" whether that result value or state is indeed correct. Even though in practice one can often implement an interpreter relatively easily by disseminating the big-step SOS rules of a language, they are in fact intended to give a formal description of what is possible in a language, not to be executable "per se".

- Because of the above, big-step SOS *avoids or hides non-termination* of programs by avoiding or hiding entirely the means by which one can state that a program does not terminate. Non-termination appears as "lack of proof" in the SOS system, but so does a program that performs a division by zero (or, in more complex languages, one that produces any runtime error); it is fair to say that, in principle, "runtime errors" can be handled in big-step SOS by generating and propagating "error values".

- Due to its monolithic, single-step evaluation, it is hard to debug or trace big-step semantic definitions. If the program is wrong, no information is given about where the failure occurred.

- Most importantly, it is widely accepted that it is *very inconvenient or impossible to define nontrivial concurrent languages* using big-step SOS. It is fair to say that big-step SOS provides some limited support for handling non-determinism. For example, one can add a Boolean constant randBool that non-deterministically evaluates to either true or false as follows:

$$\frac{\cdot}{\langle \mathsf{randBool}, \sigma\rangle \Downarrow \langle \mathsf{true}\rangle} \tag{3.6}$$

$$\frac{\cdot}{\langle \mathsf{randBool}, \sigma\rangle \Downarrow \langle \mathsf{false}\rangle} \tag{3.7}$$

The big-step SOS definition of our language extended with the rules above can now relate programs with any of the integer values that they non-deterministically evaluate to.

Unfortunately, the situation is more complex when the non-determinism is a consequence of parallelism. Consider, for example, the addition of a language construct $s_1 \| s_2$ for evaluating statements s_1 and s_2 in parallel. If one naively adds rules like

$$\frac{\langle s_1, \sigma\rangle \Downarrow \langle \sigma''\rangle,\ \langle s_2, \sigma''\rangle \Downarrow \langle \sigma'\rangle}{\langle s_1 \| s_2, \sigma\rangle \Downarrow \langle \sigma'\rangle} \tag{3.8}$$

$$\frac{\langle s_2, \sigma\rangle \Downarrow \langle \sigma''\rangle,\ \langle s_1, \sigma''\rangle \Downarrow \langle \sigma'\rangle}{\langle s_1 \| s_2, \sigma\rangle \Downarrow \langle \sigma'\rangle} \tag{3.9}$$

then one fails to capture the intended non-determinism in our parallel language because one simply sequentializes the executions of the two statements instead of executing them in parallel.

Exercise 179. *Give an example of a parallel program using the parallel statement construct above for which a big-step SOS definition like above would miss some possible results.*

To correctly and completely capture the desired non-determinism of this parallel language, one needs to monolithically detect the first actions that the parallel program can potentially take, then, for a given such action, to perform it collecting the new global state after its execution, and then to generate a big-step configuration comprising a new parallel program. Complications are mostly due to the fact that a big-step semantics for this concurrent language, in case any can be given by some hard dying proponent of big-step SOS, would not be compositional. For example, there is no way to fully characterize the derivability of

$\langle (s_1; s_1')\|s_2, \sigma\rangle \Downarrow \langle \sigma'\rangle$ in terms of the derivability of any group of transitions involving only the statements s_1, s_1' and s_2, such as $\langle s_1\|s_2, \sigma\rangle \Downarrow \langle \sigma''\rangle$, etc.

Advantages of Small-Step SOS.

- It is *executable*. Indeed, one can start with a program to evaluate and keep applying SOS rules whose left configuration *matches*; to apply a small-step rule, one typically needs to call recursively the "execution" procedure on smaller fragments (the ones above the lines). Nevertheless, one can log all the successful applications of rules obtained this way and thus get an execution of the original program. Therefore, it is *easy to trace and debug*. It is relatively straightforward to implement an interpreter for a language by just following, almost blindly, a small-step SOS definition of that language.

- Thanks to the above, non-termination of programs results in non-termination of searching for a proof; however, programs that perform division by zero (or runtime errors in general) can still be evaluated step-by-step until the actual error takes place; then one can be given a *meaningful error message*, including the entire stuck program.

- It *supports definitions of non-deterministic and/or parallel languages*, obeying the interleaving semantics approach to concurrency. In fact, our small-step SOS definition of the language above was non-deterministic; it just "happened" that the final results of evaluating expressions or programs were deterministic (but one needs to prove it). Considering the parallel statement construct $s_1\|s_2$ discussed in the context of big-step SOS above, one can easily give it a small-step SOS semantics as follows:

$$\frac{\langle s_1, \sigma\rangle \rightarrow \langle s_1', \sigma'\rangle}{\langle s_1\|s_2, \sigma\rangle \rightarrow \langle s_1'\|s_2, \sigma'\rangle} \tag{3.10}$$

$$\frac{\langle s_2, \sigma\rangle \rightarrow \langle s_2', \sigma'\rangle}{\langle s_1\|s_2, \sigma\rangle \rightarrow \langle s_1\|s_2', \sigma'\rangle} \tag{3.11}$$

Disadvantages of Small-Step SOS.

- It is somehow *too low level and explicit*, which results into relatively large small-step language definitions; in particular, one needs to explicitly give all the "congruence" rules for ordinary operators such as addition, multiplication, etc.

- Small-step SOS gives only an *interleaving semantics* to a concurrent language, that is, it regards executions or behaviors of concurrent programs as linear lists of actions.

Small-step SOS is inherently limited with regards to giving concurrent systems a *true concurrency semantics*, because it enforces reduction to occur only at the top, thus disallowing concurrent applications of rules. However, on parallel architectures, executions of concurrent programs are *not* interleaved; for example, statements that have only local effect in different processes can be and typically are safely executed concurrently.

- *Less suitable for proving properties* about programs; however, if one can also give a big-step semantics of the same language and prove it equivalent to the small-step semantics, then one can perform proofs using the big-step one.

Disadvantages of both Big-Step and Small-Step SOS

(1) They are *not modular*, in the sense that the addition of new statements to a language may require one to change many other rules corresponding to definitions of unrelated statements.

Exercise 180. ***(20 points)*** *Let us extend our sequential language with two common features, namely* variable increment*, which increments the variable (right away) and returns its new value in context, and* halt*, which*

takes an arithmetic expression, evaluates it to an integer and then stops the program with that value returned as final result:

$$\begin{array}{rcl} AExp & ::= & \ldots \mid ++Var \\ Stmt & ::= & \ldots \mid \mathsf{halt}\ AExp \end{array}$$

1. ***(4 points)*** *Add variable increment to the big-step SOS: list all the new rules, as well as the new versions of all the already existing rules that need to change;*

2. ***(2 points)*** *Do the same for small-step SOS;*

3. ***(4 points)*** *Add halt to the original big-step SOS definition (forget the increment for the time being): list all the new rules, as well as the new versions of all the already existing rules that need to change;*

4. ***(4 points)*** *Do the same for small-step SOS;*

5. ***(4 points)*** *Add both variable increment and halt to the original big-step and small-step SOS definitions: for each of them list all the new rules, as well as the new versions of all the already existing rules that need to change;*

For small-step SOS, pick one of the two variants discussed in the lecture notes.

(2 points) *Comment on why we had to make the various changes and say which of big-step and small-step SOS semantics appears to you to be more modular and why.*

(2) *Big-step and small-step SOS do not properly capture the intended computational granularity as deduction steps.* The original motivation of SOS semantics, both big-step and small-step, was to offer a mathematical, logical framework in which the otherwise purely intuitive notion of "computation" in a programming languages becomes logical deduction, or inference, in the formal SOS definition of the corresponding programming language. While one can argue that SOS proofs properly capture the final result of program computations, they do not properly capture the intended computational granularity of the defined language features.

For example, the intended computational granularity of a "halt a" statement is "evaluate a and then stop the execution of the program with that value"; compare that with "evaluate a, then propagate its value all the way back to the top over the syntax of the executing program, and then stop the program with that value", which is what happens in both big-step and small-step SOS.

A less important violation of computation granularity appears also in the small-step SOS rule for loops: a while loop transitions in one-step into a conditional statement. That wasted step was not originally intended; it is just an artifact of the fact that everything in SOS needs to be a rule. What one would like to say is that that unrolling of while loops is nothing but a structural identity, or an unobservable transition, with no intended computational content.

(3) Somehow related to the previous two disadvantages, it is in general *hard or impossible to define control-intensive language features* in SOS. For example, if one adds functions to our language together with a return statement, then one either needs to traverse back the syntax when a value is returned until the corresponding function call is encountered, thus violating again the desired computational granularity of return, or otherwise needs to add a function stack to the configurations. In the latter case, not only that all the configurations will change having to include the stack, so all the existing rules will have to change thus breaking the modularity of the definition, but more importantly, one needs to grab the current "evaluation context" when a function is invoked in order to push it in the stack, so one can recover it when the function returns. Unfortunately, "evaluation contexts" are captured by "proof contexts" in standard SOS (we will

shortly see that Felleisen's reduction semantics with evaluation contexts will make contexts explicit), and proof contexts are not first class objects in SOS derivations that can be stored and retrieved at will. The list of "SOS-inconvenient" features that can be added to the language can of course continue: exceptions, break/continue in loops, etc.

(4) *Neither big-step nor small-step SOS provides an appropriate semantical foundation for concurrent languages*. Big-step SOS can hardly be used to define any meaningful concurrent language, while small-step SOS gives only an interleaving semantics.

(5) They are both operational and syntax-driven, so they tell us close to *nothing about models* of languages. Models, which in SOS are considered "something else", are related to both denotational semantics and realizations of languages, including implementations for them.

3.11.4 *old stuff about MSOS*

Advantages of MSOS

1) *MSOS adds modularity to SOS* definitions by allowing one to refer only to configuration items of interest in each rule and using "..." for the remaining ones. This way, adding rules for new language features that need additional configuration infrastructure can be done without having to modify any of the rules for unrelated existing features.

Exercise 181. ***(10 points)*** *Add rules for* halt *a (this new statement was explained in the previous homework exercise) to the MSOS definition above. How many rules did you need? How many existing rules did you have to change?*

(Hint: you may need to add one more field in labels to store the "halting signal"; feel free to also change the syntax of the language however you wish, in particular to add one more top-level program construct if you need to "catch" the halting signal).

2) By allowing unobservable transitions, MSOS provide better support for allowing the language designer tune the computation granularity of defined language features.

3.11.5 *old stuff about context reduction*

Advantages of Context Reduction

1) *Evaluation contexts are explicit first-class entities* in context reduction, so they can be modified, passed, stored and retrieved. This allows for elegant definitions of control intensive language features such as abrupt termination of programs using halt-like constructs (just dissolve the current evaluation context), functions with return (stack the evaluation contexts at function calls and restore them at return), exceptions, and even callcc (store the current evaluation context as a value in the store, retrieve it when applied). Moreover, these operations on contexts can be done in such a way that they have the expected computation granularity.

Exercise 182. ***(10 points)*** *Add context reduction definitions for increment and halt (same language constructs as in the first homework exercise). Give definitions using both context reduction styles described above.*

2) Since reductions can only happen in proper evaluation contexts and propagation of computation is done once and for all using the context reduction rule, one needs not worry about propagating local computations up-wards along the syntax; this way, definitions in context reduction become *significantly more compact* than SOS definitions.

Disadvantages of Context Reduction

1) Context reduction is based on an *implicit mechanism to split a program* or a fragment of program p into an evaluation context c and a fragment of program, also called *redex*, e, so that $p = c[e]$. This split operation is somehow assumed atomic and non-computational, so it elegantly captures and hides all the small-step propagation rules of SOS. However, this decomposition is not obvious and may require significant computational resources, sometimes linear in the size of the program. Moreover, since a new evaluation context needs to be recalculated at almost any reduction step, it can easily become the major bottleneck in the efficiency of an interpreter implemented following the context reduction approach. While such an interpreter would be no worse than one implemented following the small-step SOS approach, it can still be a lot slower than it needs to be. Techniques such as *refocusing* have been proposed to incrementally compute the next evaluation context, but these techniques appear to work only when the decomposition of the program into an evaluation context and a redex is deterministic.

2) It is somehow still awkward to define concurrent languages using context reduction. That is because concurrent processes typically communicate with each other from different places in the configuration, so one may need contexts with more than one hole, preferably with an arbitrary number of them. Computing and matching such such contexts on complex configurations becomes a significantly non-trivial issue.

3) It still does not serve as a solid foundation for concurrency, because, just like SOS, it only captures an interleaving semantics of a defined concurrent system.

4) It still says nothing about models, being essentially a purely syntactic reduction technique.

We are going to shortly discuss a language definitional technique, called K, which has none of the limitations above. Moreover, we'll present an automatic translation from any context reduction definition, say CR, into a K definition, say k(CR), which is faithful to the computational granularity of the original context reduction definition, in the sense that any computational step in CR has a precise one-step correspondent in k(CR) and vice-versa, that is, any computational step in k(CR) has a precise correspondent step in CR. Additionally, k(CR) is as compact as the original CR: it adds one equation per context evaluation production and one K-rule per context reduction rule. Therefore, if one wants to use context reduction within K then one is free to do so, but, of course, one cannot circumvent the limitations of context reduction. As shown in the later classes by a series of languages defined in K, such as imperative, functional, object-oriented and logic programming, as well as concurrent languages, one may be better off using the full strength of K, unlimited by restricted methodological fragments of K, such as context reduction.

Disadvantages of MSOS

While modularity was a major drawback of conventional SOS and MSOS proposes solutions to improve it, MSOS still carries most of the other disadvantages of SOS. For example, it still has an interleaving semantics for concurrency and it still says nothing about models. While providing better support for defining the desired computational granularity of language constructs by allowing certain transitions to be unobservable, MSOS still needs to propagate information through the program syntax in order for that information to reach its destination. For example, in the case of halt, the "halting signal" still had to be propagated from subexpressions to parent expressions in labels; it is true that the program did not need to be changed anymore like in the case of small-step SOS, but the number of rule applications remains pretty much the same as in small-step SOS; therefore, we still cannot model the intended computational granularity of halt a, which is "evaluate a to i and then stop immediately the execution of the entire program with result i. We will shortly see how reduction semantics with evaluation contexts allows us to do that.

Like conventional SOS, MSOS is still rather low-level, mimicking by logical inference both one-step computations and propagation of reductions along the language constructs. That means that computation contexts are captured as proof-contexts and, unfortunately, proof contexts cannot be stored and/or retrieved at

will. That means that complex control-intensive language constructs like callcc may be impossible to define in MSOS.

A disadvantage of MSOS compared to SOS is that it is even more syntactic in its nature than SOS: every transition must contain pieces of syntax in both sides. Recall our first small-step SOS variant, which allowed us to derive transitions of the form $\langle s, \sigma \rangle \to \langle \sigma' \rangle$ when the statement s carried no further computation. To write that in MSOS, one would need to first generate an artificial "statement value" and then discard it with another rule. One could find that natural to do, especially in the context of a functional programming language, but on the other hand one can argue that one should not be enforced to produce an artificial value for things that are commonly used exclusively for their side effects.

3.11.6 *other old stuff about context reduction*

The rule for halt in Version 1 could have also been "$c[\mathsf{halt}\ i] \to i$", though it would yield some perhaps undesired top-level nondeterminism: a configuration parsed as $\langle c[\mathsf{halt}\ i], \sigma \rangle$ could indeed reduce to $\langle i, \sigma \rangle$ by applying the characteristic rule of context reduction in the context $\langle \square, \sigma \rangle$, but it could also reduce directly to i because an alternative parsing of it is "$\langle c, \sigma \rangle[\mathsf{halt}\ i]$", thus allowing top-level reductions between terms of different syntactic categories.

Strengths. Context reduction improves over SOS in at least two ways:

1. Since reductions can only happen in proper evaluation contexts and propagation of computation is done once and for all using the characteristic rule of context reduction, one needs not worry about propagating local computations up-wards along the syntax; this way, definitions in context reduction become *more compact* than SOS definitions. In simple terms, context reduction yields more compact definitions than ordinary SOS because it uses parsing to find the next redex rather than towered small-step rules.

2. It provides the possibility of also *altering the evaluation context* in which a reduction occurs. More precisely, unlike SOS, context reduction allows the possibility to handle evaluation contexts like any other entities in the language, in particular to pass them to other contexts or to store them. This way one can easily and elegantly define the semantics of control-intensive language features such as return of functions, break/continue of loops, exceptions, and even callcc.

As shown shortly, context reduction can be faithfully translated (i.e., nothing is lost in translation) into rewrite logic and even in a small fragment of K, replacing each context reduction rule by a corresponding rewrite logic or K rule. That implies that language definitions in K are provably at least as powerful and compact as those using context reduction. However, when using the unleashed computation handling capabilities of K by disobeying what was called in [9] "the rigidity of syntax", one can often write even more compact, and in our (admittedly subjective) view more natural, language definitions.

Weaknesses. The advantages of context reduction over SOS-like approaches, namely the compactness of definitions and the explicit control over evaluation contexts, come at a price. We next discuss the drawbacks of context reduction, first by comparison with SOS and then in general.

The discussion above may imply that context reduction is strictly superior to SOS or other syntactic approaches. That is, unfortunately, not true. SOS' conditional rules have an important advantage, which cannot be recovered in context reduction: they easily and naturally create execution environments just for the subexpressions (or other fragments of program) to be processed as part of the condition, without having to

worry about recovering the execution environment when the processing of the subexpression is finished. For example, recall the MSOS definition (one can obtain an equivalent SOS definition by desugaring MSOS into SOS [53]) of let in Figures **??** and **??**,

$$\frac{e_1 \longrightarrow e_1'}{\mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 \longrightarrow \mathsf{let}\ x = e_1'\ \mathsf{in}\ e_2}$$

$$\frac{e_2 \xrightarrow{\{\mathsf{env}=\rho[v_1/x],\ldots\}} e_2'}{\mathsf{let}\ x = v_1\ \mathsf{in}\ e_2 \xrightarrow{\{\mathsf{env}=\rho,\ldots\}} \mathsf{let}\ x = v_1\ \mathsf{in}\ e_2'}$$

$$\mathsf{let}\ x = v_1\ \mathsf{in}\ v_2 \longrightarrow v_2$$

These rules say that the binding expression e_1 must be first evaluated to a value v_1, which is then bound to x in the environment while the body of the let, e_2, is being reduced; the value v_2 that e_2 hereby eventually reduces to is the result of the evaluation of the let. The important thing to notice here is the second rule: e_2 is reduced in one step to e_2' in a modified environment ($\rho[v_1/x]$), while let $x = v_1$ in e_2 reduces to let $x = v_1$ in e_2' under the original environment (ρ). When using SOS or MSOS, the language designer is therefore not concerned with "recovering" the environment after the reduction of e_2. A similar scenario happens when giving an SOS semantics of function invocation in an environment-based definition. Unfortunately, neither of these is possible in context reduction. Even though the third rule above could be used as is in a hypothetical context reduction definition and the first rule can be replaced by an evaluation context production giving evaluation access to the binding expression,

$$Cxt ::= \ldots \mid \mathsf{let}\ Id = Cxt\ \mathsf{in}\ \mathcal{E},$$

there is no way to find an equivalent for the second rule in context reduction. What one would like to say is, using an unconditional rule, that in an expression of the form let $x = v_1$ in e_2 evaluation access is given to e_2, *but* in the modified environment $\rho[v_1/x]$; then, once e_2 is completely reduced, the environment changes back to ρ. This is simply not possible in context reduction, unless one modifies the syntax in some rather aggressive way (introducing, e.g., explicit environment restore statements), which would lack generality and be against the "syntactic" spirit of context reduction.

Also, one should not be fooled thinking that context reduction is a generalization of SOS, so one can use conditional SOS-like rules as an escape whenever a particular task cannot be accomplished using unconditional context reduction rules. For example, one cannot use a rule like the second one above, simply because one "loses the context" in the condition; indeed, if e_2 reduces to a call/cc during the processing of the condition, then one cannot provide it with the whole evaluation context. How does K solve these problems, considering that it is also an unconditional-rule framework? The answer is that K, by its nature, explicitly disobeys the purity of syntax; in the case of let, one uses the computation structure to perform the desired tasks in the desired order (that is the precise role of the computation structure): bind x to v_1, schedule e_2 for processing, then recover the original environment. Similar environment recovering steps need to be taken when defining control-changing statements such as return of functions, exceptions, or call/cc. Neither of these can be defined in context reduction following an environment-based definitional style.

The designated approach in context reduction to avoid the complications discussed in the paragraph above is to discard the environment-based style all together and follow instead a substitution-based style. Substitution is the basis of λ-calculus' β-reduction rule and can be used to define other common language constructs, such as let and letrec. However, some language constructs that have straightforward environment-based definitions, such as references, objects, threads, etc., require quite intricate and tricky substitution-based

definitions, if any can be found. While substitution-based semantic techniques are interesting in their own way and are worth considering in some practical cases, we believe that a language designer should simply not be forced by the underlying framework to follow a substitution-based style. Moreover, an executable definitional framework should not disregard efficiency, and substitution is expensive. An executable language definition in which each computational step has a complexity linear in the size of the program is simply impractical (wrt executability); not to mention that the size of the program to reduce can grow unbounded in a substitution-based framework. As seen in Section **??**, K also allows substitution-based definitions which are as simple and natural, if not simpler and more natural, than their corresponding context reduction definitions. However, K does not enforce, encourage or discourage substitution-based definitions. We believe that that should be the choice of the language designer and not imposed by the framework.

We next discuss several other weaknesses of context reduction, not necessarily by comparison with SOS.

- Context reduction is based on an *implicit mechanism to split a program* or a fragment of program p into an evaluation context c and a fragment of program, also called *redex*, e, so that $p = c[e]$. This split operation is somehow assumed atomic and non-computational, so it elegantly captures and hides all the small-step propagation rules of SOS. However, this decomposition is not obvious and may require significant computational resources, sometimes linear in the size of the program or worse. Moreover, since a new evaluation context needs to be recalculated at almost any reduction step, it can easily become the major bottleneck in the efficiency of an interpreter implemented following the context reduction approach. While such an interpreter would be no worse than one implemented following the small-step SOS approach, it can still be a lot slower than it needs to be, thus making context reduction unattractive as an executable language definitional framework (see the "ideal framework" requirements in Section 1). Techniques such as *refocusing* [19] have been proposed to incrementally compute the next evaluation context, but these techniques appear to work only when the decomposition of the program into an evaluation context and a redex is deterministic.

---

have a subsection on refocusing?

- It is inconvenient to define concurrent languages using context reduction. That is because concurrent processes typically communicate with each other from different places in the configuration, so one may need contexts with more than one hole, preferably with an arbitrary number of them. While some attempts to define and handle multi-contexts have been proposed (see, e.g., [FIXTHIS]), computing and matching such contexts on complex configurations becomes a significantly non-trivial issue with potentially serious impact on performance.

- Context reduction is still far from serving as a solid foundation for concurrency, because, just like SOS, it only captures an interleaving semantics of a defined concurrent system. In particular, context reduction can only be used to give an interleaving semantics of CCS (middle column in Figure **??**), but not a truly concurrent one (right column in Figure **??**).

- It still says nothing about models, being essentially a purely syntactic reduction technique.

---

this is true for almost all the other operational approaches; I should probably have a discussion on this?

K has all the strengths of context reduction, at the same time avoiding its limitations. Indeed, K's matching is comparatively cheap and rather standard in term rewriting; existing advanced indexing techniques

in term rewriting can be and are being used (by running K in Maude, for example) to efficiently execute K definitions. K is particularly useful to define concurrent languages, because, if used properly, can give a truly concurrent semantics to the defined languages at the same time also allowing an interleaving semantics—K's approach is to let the language designer, rather than the limitations of the framework, decide what semantics for concurrency is desired. Also, K has a model-theoretical semantics, for the time being borrowed via its desugaring into rewrite logic.

---

add a section on the semantics of K; say that the borrowed semantics from rewrite logic is inappropriate because it gives an interleaving semantics where it should not

---

---

some other old stuff

---

in our experiments with embeddings of context reduction into rewrite logic (see Section 3.7.3), this was the factor that influenced the performance of the resulting reduction engine the most; in our experiments, for example, context reduction was most effective when a depth-first-search strategy for contextual representations was employed.

reduction semantics with evaluation contexts style improves over small-step SOS definitional styles in at least three ways:

1. It gives a *more compact semantics to context-sensitive reduction*, by using parsing to find the next redex rather than towered small-step rules;

2. It provides the possibility of also *modifying the context* in which a reduction occurs, making it much easier to control the execution of programs. For example, defining halt is done now using only one rule, $\langle c[\mathsf{halt}i], \ldots\rangle \to \langle i\rangle$, where c is the current evaluation context, preserving the desired computational granularity;

3. It allows the possibility to handle evaluation contexts like any other values in the language, in particular to pass them to other contexts or to store them. This way one can more easily define the semantics of control-intensive language features such as return of functions, exceptions, even callcc(call-with-current-continuation, such as, e.g., in Scheme).

3.11.7 *older stuff about cham*

---

limitation of CHAM: one cannot say that a rewrite rule applies only if the current solution contains precisely the contents described by the LHS of the rule, because of the chemical law

---

When Cham definitions follow the style in Table FIXTHIS, i.e., taking a contex-reduction-like approach, one could use as evaluation strategies *heating only on redexes* and *cooling only on values*, which would lead to a deterministic abstract-machine. Moreover, one can notice that airlock rules were introduced to select elements from a set without specifying the rest of the set, abstracted by a molecule. Efficient implementations should probably do exactly the opposite, that is, matching in the sets. To do that in our rewrite framework, one would orient the airlock rules in the sense of inserting back the "airlocked" molecules into their original solution and to apply them on the terms of the existing rules, to make the definition executable. The only

rules changing in the definition above are those involving the store; for example, the assignment rule will is transformed into:

$$\{\!\{X := I \rhd C\}\!\}, \{\!\{(X, I'), \sigma\}\!\} \rightarrow \{\!\{\{\} \rhd C\}\!\}, \{\!\{(X, I), \sigma\}\!\}$$

One should notice that the specification obtained by these transformations is equivalent to the initial one, since it does not change the equivalence classes and the transitions. The main advantage of the newly obtained specification is that it is also executable in a deterministic fashion, that is, there is no need to search for a solution anymore.

Strengths. Being a special case of rewrite logic, it inherits many of benefits of rewrite logic, being specially well-suited for describing truly concurrent computations and concurrent calculi.

Weaknesses. Heating/cooling rules are hard to implement efficiently in general—an implementation allowing them to be bidirectional in an uncontrolled manner would have to *search* for solutions, possibly leading to a combinatorial explosion. Rewriting strategies such as those in [11, 85] [FIXTHIS] can be of help for solving particular instances of this problem. Although this solution-molecule paradigm seems to work pretty well for languages in which the structure of the state is simple enough, it is not clear how one could represent the state for complex languages, with threads, locks, environments, an so on. Finally, Chams provide no mechanism to freeze the current molecular structure as a ”value”, and then to store or retrieve it, as we would need in order to define language features like callcc. Even though it was easy to define halt because we simply discarded the entire solution, it is hard or impossible to define more complex control-intensive language features in Cham.

3.12 K

3.13 Notes

This language is reminiscent of the Imp language defined by Winskel in his book

> G. Winskel. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press, 1993.

Chapter 4

Idealized Programming Languages, Features, and Models of Computation

---

integrate this:

In this part of the course we discuss two important and closely related mathematical theories:

- *Lambda calculus*, written also *λ-calculus*, is a pure calculus of functional abstraction and function application, with many applications in logic and computer science;
- *Combinatory logic* shows that bound variables can be entirely eliminated without loss of expressiveness. It has applications both in the foundations of mathematics and in the implementation of functional programming languages.

A good reference for these subjects is the book *"The Lambda Calculus: Its Syntax and Semantics" by H.P. Barendregt (Second Edition, North Holland 1984)*. This book also contains a great discussion on the history and motivations of these theories.

4.1 Combinatory Logic

Combinatory logic was proposed by Mosses Schönfinkel in 1920, as a means to reduce the number of primitive notions of logic, in particular to eliminate the need of variables in functions and in predicates. Looking at it through programming language lenses, combinatory logic is a very simple language in which programs can be described as a combination of a small number of primitive elements, called *combinators*, and of *free variables*. Even though combinatory logic defines no explicit functions, each combinator can be thought of as a closed function, that is, a function which declares all the variables that it uses, and their combination can be thought of as function application[1].

Combinatory logic can capture the meaning of any arithmetic or logical statement (and by extension, any non-interactive computer program), making it a Turing-complete computational model. It is often described and/or implemented in terms of rewriting (of graphs, trees, or terms).

Combinatory logic was proposed by Mosses Sch́onfinkel in 1920.

- who proposed it and why

- what is the idea

- formally what it is: the equational theory

- define other combinators: X, J, I, Ω, Y, Composition. Prove SKK=I under eta, etc. SII = Ω. Prove that if F = S (K f) (S I I) then F F is a fixed point for f. Then

result Exp: S (S (S (K S) (S (K K) (S K K))) (S (S (K S) (S (S (K S) (K K)) (K K))) (S (S (K S) (K K)) (K K)))) (S (S (K S) (S (K K) (S K K))) (S (S (K S) (S (S (K S) (K K)) (K K))) (S (S (K S) (K K)) (K K))))

is a fixed point combinator.

More precisely, we show that any closed λ-expression can be systematically transformed into a λ-expression build over only the combinators $K := \lambda xy.x$ and $S := \lambda xyz.xz(yz)$, together with the λ-application operator. For example, the identity λ-abstraction $\lambda x.x$ is going to be SKK; indeed,

$SKK \equiv_\beta \lambda z.Kz(Kz) = \lambda z.(\lambda xy.x)z(Kz) \equiv_\beta \lambda z.z \equiv_\alpha \lambda x.x$.

Interestingly, once such a transformation is applied, one will not need the machinery of λ-calculus and β-reduction anymore. All we'll need to do is to capture the contextual behavior of K and S, which can be defined equationally very elegantly: $KXY = X$ and $SXYZ = XZ(YZ)$, for any other KS-expressions X, Y, Z.

Before we do that, we need to first discuss two other important aspects of λ-calculus: η-equivalence and extensionality.

---

new info

Even though λ-calculus can be defined equationally and is a relatively intuitive framework, as we have noticed several times by now, substitution makes it non-trivial to implement effectively. There are several approaches in the literature addressing the subtle problem of automating substitution to avoid variable capture, all with their advantages and disadvanteges. We here take a different approach. We show how λ-expressions

[1] We use the notion of a function informally here. Functions, function application, and the notions of free and bound variables are formally discussed in Section 4.5, where we present the λ-calculus.

can be automatically translated into expressions over combinators, in such a way that substitution will not even be needed anymore.

A question addressed by many researchers several decades ago, still interesting today and investigated by many, is whether there is any *simple* equational theory that is entirely equivalent to λ-calculus. Since λ-calculus is Turing complete, such a simple theory may provide a strong foundation for computing.

Combinatory logic was invented by Moses Shönfinkel in 1920. The work was published in 1924 in a paper entitled *"On the building blocks of mathematical logic"*. Combinatory logic is a simple equational theory over two sorts, *Var* and *Exp* with *Var* < *Exp*, a potentially infinite set x, y, etc., of constants of sort *Var* written using lower-case letters, two constants K and S of sort *Exp*, one application operation with the same syntax and left-associativity parsing convention as in λ-calculus, together with the two equations

$$KXY = X,$$
$$SXYZ = XZ(YZ),$$

quantified universally over X, Y, Z of sort *Exp*. The constants K and S are defined equationally in such a way to capture the intuition that they denote the combinators $\lambda xy.x$ and $\lambda xyz.xz(yz)$, respectively. The terms of the language, each of which denoting a function, are formed from variables and constants K and S by a single construction, function application. For example, $S(SxKS)yS(SKxK)z$ is a well-formed term in combinatory logic, denoting some function of free variables x, y, and z.

Let CL be the equational theory of combinatory logic above. Note that a function FV returning the free variables that occur in a term in combinatory logic can be defined in a trivial manner, because there are no bound variables in CL. Also, note that the extensionality principle from λ-calculus translates unchanged to CL:

(ext) If $Ex = E'x$ for some $x \notin FV(EE')$, then $E = E'$.

Let CL + ext be CL enriched with the principle of extensionality. The following is a landmark result:

Theorem 22. λ + ext *is equivalent to* CL + ext.

Proof. Let us recall what one needs to show in order for two mathematical theories to be equivalent: (1) how the syntax of one translates into the syntax of the other; and (2) that all the axioms of each of the two theories can be proved from the axioms of the other, along the corresponding translation of syntax.

Let us consider first the easy part: λ + ext implies CL + ext. We first need to show how the syntax of CL + ext translates into that of λ + ext. This is easy and it was already mentioned before: let K be the combinator $\lambda xy.x$ and let S be the combinator $\lambda xyz.xz(yz)$. We then need to show that the two equational axioms of CL + ext hold under this translation: they can be immediately proved by β-equivalence. We also need to show that the extensionality in CL + ext holds under the above translation: this is obvious, because it is exactly the same as the extensionality in λ + ext.

Let us now consider the other, more difficult, implication. So we start with CL + ext, where K and S have no particular meaning in λ-calculus, and we need to define some map that takes any λ-expression and translates it into an expression in CL.

To perform such a transformation, let us add syntax for λ-abstractions to CL, but without any of the equations of λ-calculus. This way one can write and parse λ-expressions, but still have no meaning for those. The following ingenious *bracket abstraction* rewriting system transforms any uninterpreted λ-expression into an expression using only K, S, and the free variables of the original λ-expression:

1. $\lambda x.\rho \Rightarrow [x]\rho$
2. $[x]y \Rightarrow \begin{cases} SKK & \text{if } x = y \\ Ky & \text{if } x \neq y \end{cases}$
3. $[x](\rho\rho') \Rightarrow S([x]\rho)([x]\rho')$
4. $[x]K \Rightarrow KK$
5. $[x]S \Rightarrow KS$

The first rule removes all the λ-bindings, replacing them by corresponding bracket expressions. Here ρ and ρ' can be any expressions over K, S, variables, and the application operator, but also over the λ-abstraction operator $\lambda_._ : Var \to Exp$. However, note that rules 2-5 systematically elliminate all the brackets. Therefore, the *bracket abstraction* rules above eventually transform any λ-expression into an expression over only K, S, variables, and the application operator.

The correctness of the translation of λ `+ ext` into `CL + ext` via the bracket abstraction technique is rather technical: one needs to show that the translated versions of equations in λ can be proved (by structureal induction) using the machinery of `CL + ext`.

Exercise 183. ***(Technical)*** *Prove the correctness of the translation of λ `+ ext` into `CL + ext` above.*

We do not need to understand the details of the proof of correctness in the exercise above in order to have a good intuition on why the bracket abstraction translation works. To see that, just think of the bracket abstraction as a means to associate equivalent λ-expressions to other λ-abstractions, within the framework of λ-calculus, where K and S are their corresponding λ-expressions. As seen above, it eventually reduces any λ-expression to one over only combinators and variables, containing no explicit λ-abstractions except those that define the combinators K and S. To see that the bracket abstraction is correct, we can think of each bracket term $[x]E$ as the λ-expression that it was generated from, $\lambda x.E$, and then show that each rule in the bracket abstraction transformation is sound within λ-calculus. For example, rule 3 can be shown by extensionality: $(\lambda x.\rho\rho')z \equiv_\beta (\rho[z/x])(\rho'[z/x]) \equiv_\beta ((\lambda x.\rho)z)((\lambda x.\rho')z)) \equiv_\beta (\lambda xyz.xz(yz))(\lambda x.\rho)(\lambda x.\rho')z = S(\lambda x.\rho)(\lambda x.\rho')z$, so by extensionality, $\lambda x.\rho\rho' \equiv_{\beta\text{ext}} S(\lambda x.\rho)(\lambda x.\rho')$.

This way, one can prove the soundness of each of the rules in the bracket abstraction translation. As one may expect, the tricky part is to show the completeness of the translation, that is, that everything one can do with λ-claculus and `ext` can also do with with its subcalculus `CL + ext`. This is not hard, but rather technical.

Exercise 184. *Define the bracket abstraction translation above formally in Maude. To do it, first define `CL`, then add syntax for λ-abstraction and bracket to `CL`, and then add the bracket abstraction rules as equations (which are interpreted as rewrite rules by Maude). Convince yourself that substitution is not a problem in `CL`, by giving an example of a λ-expression which would not be reducible with the definition of λ-calculus in Exercise* **??***, but whose translation in `CL` can be reduced with the two equations in `CL`.*

---

some additional notes:

In the pure system there is a small set of constants all of which are themselves pure combinators, usually the primitive combinators are named S, K and I, though I is in fact definable in terms of S and K.

The intended meaning of these constants can be described simply by showing a corresponding transformation or reduction which the constant may be thought of as effecting.

The transformations are as follows: I x = x K x y = x S f g x = (f x)(g x) This set of combinators has the property of combinatory completeness which means that for any function definition of the form: f x = combinatory expression involving x

There exists an expression E in S,K and I such that: f = E

This property establishes the close relationship between Pure Combinatory Logic and the Pure lambda Calculus, in which a special notation for functional abstraction is available. It shows that the notation for functional abstraction, though a great convenience, adds no expressiveness to the language.

As well as having combinatorial completeness, Pure Combinatory Logic is able to express all effectively computable functions over natural numbers appropriately represented as combinators.

[Barenderegt84] Barendregt, H.P.; The Lambda Calculus - Its Syntax and Semantics; Second Edition, North Holland 1984 purchase from amazon purchase from ibs

This is the first published work in the field which came later to be known as combinatory logic.

Schnfinkel shows how the use of bound variables in logic can be dispensed with. The use of higher order functions makes possible the reduction of logic to a language consisting of one constructor (the application of a function to an argument) and three primitive constants U, C (now usually called K) and S. A function is termed ”higher order” if it will accept a function as an argument, or return one as its result. U, C, and S are all higher order functions.

The meaning of these primitives may be understood through the following informal definitions:

Cxy = (C(x))(y) = x C is a function which given any value x, returns the constant x valued function. Sfgx = ((S(f))(g))(x) = (f(x))(g(x)) S is a function which combines two functions, say f and g, supplied as successive arguments. The resulting function, given a value x, returns the value obtained by applying the value of f at x to the value of g at x. UPQ = (U(P))(Q) = forall x. not(P(x) and Q(x)) U is a generalised scheffer stroke. It should be thought of as applying to two predicates and returns the universal quantification of the negated conjunction of the two predicates.

These combinators are sufficient to enable arbitrary first order predicates to be expressed without the use of bound variables, which appear in first order logic whenever a quantifier is used. This can be demonstrated most straightforwardly using a simple algorithm which converts lambda-expressions to combinators. They are not limited to first-order predicates, but without some constraints (equivalent to those found in first order logic, or more liberally and appropriately to those in Church’s Simple Theory of Types [Church40]) the logic which results from the use of these combinators is at risk of proving inconsistent. Combinatory logic has traditionally tried to avoid type constraints and has therefore been dogged with difficulties in achieving strength and consistency.

Schnfinkel’s paper remains an accessible introduction to combinatory logic which makes clear the original motivation for this innovation. This original motivation was vigorously pursued later by H.B.Curry and his collaborators. The results of Curry’s programme were published in Combinatory Logic, Vols 1 & 2.

While Schfinkel was successful in showing that combinators could provide plausible notation, the semantic and proof theoretic aspects were more difficult to put in place. The difficulty is more pronounced if it hoped to use combinatory logic as a foundation for mathematics, than it is for the use of combinators in less demanding roles (e.g. for programming languages).

The Pure combinatory logic may be thought of as Schnfinkels system with the logical operator ”U” omitted (though the presentation may vary). Illative combinatory logics are formed by adding additional logical primitives to the pure combinators. There are many illative combinatory logics, which vary among themselves in substance as well as in presentation.

Pure combinatory logic is so closely related to Church's lambda-calculus that it is best studied alongside the lambda-calculus, for which the most comprehensive modern text is probably The Lambda Calculus. A very popular and entertaining introduction to the pure combinators may be found in To Mock a Mockingbird.

Research on illative combinatory logics has been continued by Curry's students, Roger Hindley and Jonathan Seldin, and by Martin Bunder. Hindley, Lercher and Seldin published An Introduction to Combinatory Logic, an excellent and readable short introduction to the subject, now superseded by the more comprehensive An Introduction to Combinators and the Lambda-Calculus

Recently Randall Holmes has developed logical systems based on combinatory logic and the lambda calculus using methods derived from Quine's formulations of set theory.

The attempt to make a foundation for mathematics using the illative combinatory logic were blighted by Curry's paradox and semantic opacity. While systems avoiding this paradox have been devised, they are typically weak, and lack the kind of convincing semantic intuition found in first order set theory.

[Curry58] Curry, Haskell B., Feys, Robert; Combinatory Logic, Volume I; North Holland 1958; [Curry72] Curry, Haskell B., Hindley, J.Roger, Seldin, Jonathan P.; Combinatory Logic, Volume II; North Holland 1972; ISBN 0 7204 2208 6

4.2 Turing Machines

4.2.1 Notes

Encodings of Turing computability into equational deduction and/or rewriting are well-known. For example, Bergstra and Tucker [7] (1995) and Baader and Nipkow [1] (1998) give encodings that have properties similar to ours in this section, in the sense that their resulting equational specifications, if regarded as term rewrite systems, are confluent and terminate whenever the original computation terminates. It seems, however, that due to the particular two-infinite-end-tape variant of Turing machine chosen in this chapter, our encoding is somewhat simpler. The ideas underlying our encodings presented in this section ware first published by the author in 2006 [65].

As mentioned at the beginning of this section, our purpose here was not to address the wonderful theoretical implications of Turing machines and computability, but rather to regard the Turing machine through the programming language lens and to show how we can give it a rewriting semantics just like for all the languages discussed in this book.

Instructions are labeled tales, such as:

$$1: \begin{array}{|c|}\hline \circ \\ \hline \bullet\circ\circ \\ \hline\end{array} \quad 2: \begin{array}{|c|}\hline \circ\bullet \\ \hline \circ\circ \\ \hline\end{array} \quad 3: \begin{array}{|c|}\hline \bullet\bullet\circ \\ \hline \bullet\bullet \\ \hline\end{array}$$

Programs are label sequences, such as:

3 2 3 1

Figure 4.1: Post correspondence problem as a programming language

4.3 Post Correspondence

Turing machines allow us to formally state and prove that certain problems are *undecidable*, that is, that they cannot be solved by computers, no matter what programming language is being used. The canonical undecidable problem in the context of Turing machines is the *halting problem*: given a turing machine $\mathcal{M}$ and an input i, does $\mathcal{M}$ halt on i? Because of its reflective nature, the halting problem is somehow complicated to use in practice. In this section we discuss another canonical undecidable problem, the *Post correspondence problem*, showing how it can be reduced to a step-for-step equivalent rewrite theory. The reason for which we discuss this problem in this chapter dedicated to idealized programming languages, is because we can indeed regard it as an overly simplified programming language, whose basic instructions are two-dimentional tales and whose programs are sequences of such tales, as shown in Figure **??**.

...

4.4 Counter Machines

4.5 Lambda Calculus

Lambda calculus was introduced in 1930s, as a mathematical theory together with a proof calculus aiming at capturing foundationally the important notions of function and function application. Those years were marked by several paradoxes in mathematics and logics. The original motivation of λ-calculus was to provide a foundation of logics and mathematics. Whether λ-calculus indeed provides a strong foundation of mathematics is still open; what is certain is that λ-calculus was and still is a quite successful *theory of computation*. Today, more than 70 years after its birth, λ-calculus and its afferent subjects still fascinates computer scientists, logicians, mathematicians and, certainly, philosophers.

λ-Calculus is a convenient framework to describe and explain many programming language concepts. It formalizes the informal notion of "expression that can be evaluated" as a *λ-term*, or *λ-expression*. More precisely, λ-calculus consists of:

- *Syntax* - used to express λ-terms, or λ-expressions;
- *Proof system* - used to prove λ-expressions equal;
- *Reduction* - used to reduce λ-expressions to equivalent ones.

We will show how λ-calculus can be formalized as an *equational theory* in equational logic (Section 2.4), provided that an important mechanism of *substitution* is available. That means that its syntax can be defined as an algebraic signature; its proof system becomes a special case of equational deduction; and its reduction becomes a special case of rewriting (when certain equations are regarded as rewrite rules). These results support the overall theme underlying this book, that equational logic and rewriting are a strong foundation for describing and explaining programming language concepts. Even though λ-calculus is a special equational theory, it has the merit that it is powerful enough to express most programming language concepts quite naturally. Equational logic gives us, in some sense, "too much freedom" in how to define concepts; its constraints and intuitions are not restrictive enough to impose an immediate mapping of programming language concepts into it. As extensively illustrated in Chapter 3, due to their generality, equational and rewrite logics need *appropriate methodologies* to define languages, while λ-calculus, due to its particularity, gives an immediate means to syntactically encode programming languages and hereby give them a semantics.

As discussed in Section 4.5.6, many programming language concepts, and even entire programming languages, translate relatively naturally into λ-calculus concepts or into λ-expressions. That includes the various types of values and numbers, data structures, recursion, as well as complex language constructs.

4.5.1 Syntax

Syntactically, λ-calculus consists of two syntactic constructs, λ-abstraction and λ-application, and a given infinite set of variables. Let us assume an infinite set of variables, or names, *Var*. Then the syntax of *λ-expressions* is given by the following grammar (in BNF notation):

Exp	::=	*Var*	— variables are λ-expressions
	|	*λVar.Exp*	— *λ-abstraction,* or *nameless function*
	|	*Exp Exp*	— *λ-application*

We will use lower letters x, y, z, etc., to refer to variables, and capital letters E, E', E_1, E_2, etc., to refer to λ-expressions. The following are therefore examples of λ-expressions:

$\lambda x.x$

$\lambda x.xx$

$\lambda x.(fx)(gx)$

$(\lambda x.fx)x$

λ-Expressions of the form $\lambda x.E$ are called *λ-abstractions*, and those of the form E_1E_2 are called *λ-applications*. The former captures the intuition of nameless functions, while the latter that of function applications. To avoid parentheses, we assume that λ-application is left associative and binds tighter than λ-abstraction; for example, $\lambda x.\lambda y.\lambda z.xyz$ is the same as $\lambda x.\lambda y.\lambda z.((xy)z)$.

4.5.2 Free and Bound Variables

Variable occurrences in λ-expressions can be either *free* or *bound*. Given a λ-abstraction $\lambda x.E$, also called a *binding*, then the variable x is said to be *declared* by the λ-abstraction, or that *$\lambda x.E$ binds x in E*; also, E is called the *scope* of the binding. Formally, we define the set $FV(E)$ of *free variables of E* as follows:

- $FV(x) = \{x\}$,
- $FV(E_1E_2) = FV(E_1) \cup FV(E_2)$, and
- $FV(\lambda x.E) = FV(E) \setminus \{x\}$.

Consider the three underlined occurrences of x in the λ-expression $(\lambda \underline{x}.\lambda y.y\underline{x}y)\underline{x}$. The first is called a *binding occurrence of x*, the second a *bound occurrence of x* (this occurrence of x is bound to the binding occurrence of x), and the third a *free occurrence of x*. Expressions E with $FV(E) = \emptyset$ are called *closed* or *combinators*.

4.5.3 Substitution

Evaluation of λ-expressions is *by substitution*. That means that the λ-expression that is passed to a λ-abstraction is *copied as is* at all the bound occurrences of the binding variable. This will be formally defined later. Let us now formalize and discuss the important notion of *substitution*. Intuitively, $E[E'/x]$ represents the λ-expression obtained from E by replacing each free occurrence of x by E'. For notational simplicity, we assume that, syntactically, substitution binds tighter than any λ-calculus construct; as usual, parentheses can be used to enforce desired grouping. Formally, substitution can be defined as follows:

- $y[E'/x] = \begin{cases} E' & \text{if } y = x \\ y & \text{if } y \neq x \end{cases}$
- $(E_1E_2)[E'/x] = E_1[E'/x]\ E_2[E'/x]$
- $(\lambda x.E)[E'/x] = \lambda x.E$

The tricky part is to define substitution on λ-abstractions of the form $(\lambda y.E)[E'/x]$, where $y \neq x$. That is because E' may contain free occurrences of y; these occurrences of y would become bound by the binding variable y if one simply defined this substitution as $\lambda y.(E[E'/x])$ (and if E had any free occurrences of x), thus violating the intuitive meaning of binding. This phenomenon is called *variable capturing*. Consider, for example, the substitution $(\lambda y.x)[yy/x]$; if one applies the substitution blindly then one gets $\lambda y.yy$, which is most likely *not* what one meant (since $\lambda y.x$ is by all means equivalent to $\lambda z.x$—-this equivalence will be formalized shortly—while $\lambda y.yy$ is not equivalent to $\lambda z.yy$). There are at least three approaches in the literature to deal with this delicate issue:

1. Define $(\lambda y.E)[E'/x]$ as $\lambda y.(E[E'/x])$, but pay special attention whenever substitution is used to add sufficient conditions to assure that y is not free in E'. This approach simplifies the definition of substitution, but complicates the presentation of λ-calculus by having to mention obvious additional hypotheses all the time a substitution is invoked;

2. Define substitution as a *partial operation*: $(\lambda y.E)[E'/x]$ is defined and equal to $\lambda y.(E[E'/x])$ if and only if $y \notin FV(E')$. This may seem like the right approach, but unfortunately is also problematic, because the entire equational definition of λ-calculus would then become *partial*, which has serious technical implications w.r.t. mechanizing equational deduction (or the process of proving λ-expressions equivalent) and rewriting (or reduction);

3. Define substitution as a *total operation*, but apply a renaming of y in $\lambda y.E$ to some variable that does not occur in E or E' in case $y \in FV(E')$ (this renaming is called *α-conversion* and will be defined formally shortly). This approach slightly complicates the definition of substitution, but simplifies the presentation of many results later on. It is also useful when one wants to mechanize λ-calculus, because it provides an algorithmic way to avoid variable capturing:

$$(\lambda y.E)[E'/x] = \begin{cases} \lambda y.(E[E'/x]) & \text{if} \quad y \notin FV(E') \\ \lambda z.((E[z/y])[E'/x]) & \text{if} \quad y \in FV(E') \end{cases}$$

   where z is a new variable that does not occur in E or E'. Note that the the requirement "z does not occur in E or E'" is stronger than necessary, but easier to state that way.

All three approaches above have their advantages and disadvantages, with no absolute best approach. Sections 4.5.9 and 4.5.10 discuss alternative approaches to define λ-calculus, where the explicit lambda bindings and the implicit variable capturing problem are eliminated by design.

4.5.4 Alpha Conversion

In mathematics, functions that differ only in the name of their variables are equal. For example, the functions f and g defined (on the same domain) as $f(x) = x$ and $g(y) = y$ are considered identical. However, with the machinery developed so far, there is no way to show that the λ-expressions $\lambda x.x$ and $\lambda y.y$ are equal. It is common in the development of mathematical theories to add desirable but unprovable properties as axioms. The following is the first meaningful equational axiom of λ-calculus, known under the name of *α-conversion*:

$$\lambda x.E = \lambda z.(E[z/x]), \text{ for any variable } z \text{ that does not occur in } E \qquad (\alpha)$$

The requirement on z not occurring in E is again stronger than necessary, but it is easier to state.

Using the equation above, one has now the possibility to formally prove λ-expressions equivalent. To capture this provability relation formally, we let $E \equiv_\alpha E'$ denote the fact that the equation $E = E'$ can be proved using standard equational deduction from the equational axioms above ((α) plus those for substitution).

4.5.5 Beta Equivalence and Beta Reduction

We now define another important equation of λ-calculus, known under the name of *β-equivalence*:

$$(\lambda x.E)E' = E[E'/x] \qquad (\beta)$$

The equation (β) tells us how λ-abstractions are *applied*. Essentially, the argument λ-expression passed to a λ-abstraction is copied at every free occurrence of the variable bound by the λ-abstraction within its scope.

We let $E \equiv_\beta E'$ denote the fact that the equation $E = E'$ can be proved using standard equational deduction from the equational axioms above: (α), (β), plus those for substitution. For example $(\lambda f.fx)(\lambda y.y) \equiv_\beta x$, because one can first deduce that $(\lambda f.fx)(\lambda y.y) = (\lambda y.y)x$ by (β) and then that $(\lambda y.y)x = x$ also by (β); the rest follows by the transitivity rule of equational deduction.

When the equation (β) is applied only from left to write, that is, as a rewrite rule, it is called *β-reduction*. We let $\Rightarrow_\beta$ denote the corresponding rewriting relation on λ-expressions. To be more precise, the relation $\Rightarrow_\beta$ is defined on α-equivalence classes of λ-expressions; in other words, $\Rightarrow_\beta$ applies *modulo α-equivalence*.

Given a λ-expression E, one can always apply α-conversion on E to rename its binding variables so that all these variables have different names and they do not occur in $FV(E)$. If that is the case, then note that variable capturing cannot occur when applying a β-reduction step. In particular, that means that one can follow the first, i.e., the simplest approach of the three discussed previously in Section 4.5.3 to define or implement substitution. In other words, if one renames the binding variables each time before applying a β-reduction, then one does not need to rename binding variables during substitution. This is so convenient in the theoretical developments of λ-calculus, that most works on this subject adopt the following convention: *all the binding variables occurring in any given λ-expression at any given moment are assumed to be different; moreover, it is assumed that a variable cannot occur both free and bound in any λ-expression.* If a λ-expression does not satisfy this, then one can apply a certain number of α-conversions and eventually transform it into an α-equivalent one that does satisfy it. Clearly, this process of renaming potentially all the binding variables before applying any β-reduction step may be computationally quite expensive. In a more familiar setting, it is like traversing and changing the names of all the variables in a program at each execution step! There are techniques aiming at minimizing the amount of work to be performed in order to avoid variable capturing. All these techniques, however, incur certain overheads.

One should not get tricked by thinking that one renaming of the binding variables, at the beginning of the reduction process, should be sufficient. It is sufficient for just one step of β-reduction, but not for more. Consider, for example, the closed λ-expression, or the combinator, $(\lambda z.zz)(\lambda x.\lambda y.xy)$. It has three binding variables, all distinct, so their renaming makes no difference. However, if one applies substitution in β-reductions blindly then one quickly ends up capturing the variable y:

$$\underline{(\lambda z.zz)(\lambda x.\lambda y.xy)} \Rightarrow_\beta \underline{(\lambda x.\lambda y.xy)(\lambda x.\lambda y.xy)} \Rightarrow_\beta \lambda y.\underline{(\lambda x.\lambda y.xy)\boxed{y}} \Rightarrow_\beta \lambda y.\lambda y.\boxed{y}y$$

Section 4.5.9 will discuss an ingenious technique to transform λ-calculus into combinatory logic which, surprisingly, eliminates the need for substitutions entirely.

Consider the λ-expression $(\lambda f.(\lambda x.fx)y)g$. There are two different ways to apply β-reduction on it:

1. $(\lambda f.\underline{(\lambda x.fx)y})g \Rightarrow_\beta (\lambda f.fy)g$, and
2. $\underline{(\lambda f.(\lambda x.fx)y)g} \Rightarrow_\beta (\lambda x.gx)y$.

Nevertheless, both resulting λ-expressions above can be further reduced to gy by applying β-reduction. This brings us to one of the most notorious results in λ-calculus ($\Rightarrow_\beta^*$ *is the reflexive and transitive closure of* $\Rightarrow_\beta$):

Theorem 23. *$\Rightarrow_\beta$ is confluent. That means that for any λ-expression E, if $E \Rightarrow_\beta^* E_1$ and $E \Rightarrow_\beta^* E_2$ then there is some λ-expression E' such that $E_1 \Rightarrow_\beta^* E'$ and $E_2 \Rightarrow_\beta^* E'$. All this is, of course, modulo α-conversion.*

The confluence theorem above says that it essentially does not matter how the β-reductions are applied on a given λ-expression. A λ-expression is called a *β-normal form* if no β-reduction can be applied on it.

A λ-expression E is said to *admit a β-normal form* if and only if there is some β-normal form E' such that $E \Rightarrow_\beta^* E'$. The confluence theorem implies that if a λ-expression admits a β-normal form then that β-normal form is *unique modulo α-conversion*.

Note, however, that there are λ-expressions which admit no β-normal form. Consider, for example, the λ-expression $(\lambda x.\, x\ x)\ (\lambda x.\, x\ x)$, known as the *divergent combinator* `omega`. It is easy to see that `omega` $\Rightarrow_\beta$ `omega` and that's the only β-reduction that can apply on `omega`, so it has no β-normal form.

4.5.6 Lambda Calculus as a Programming Language

In contrast to Turing machines, which are rather low-level computational devices, λ-calculus is closer in spirit to higher-level programming languages, particularly to functional programming languages. In this section we show how several common programming language features, including particular language constructs, data-types, and recursion, can be naturally represented as λ-expressions. We say that such language features are *derived* (from λ-calculus), or *syntactic sugar* (for equivalent but harder to read λ-expressions). All these suggest that λ-calculus is rich and intuitive enough to safely be regarded as a programming language itself.

Anonymous Functions and Let Binders

Most functional programming languages, and also some other languages which are not necessarily considered functional, provide *anonymous*, or *nameless functions*, which are functions that are not bound to any particular name. For example, the anonymous successor function is written as `fn x => x+1` in the Standard ML functional language. Anonymous functions can still be applied to arguments, for example `(fn x => x+1) 2` evaluates to 3 in Standard ML, and they can typically be passed as arguments to or returned as results by other functions. Almost all languages supporting anonymous functions have a different syntax for them. Nevertheless, anonymous functions are nothing but λ-abstractions. For example, the successor function above can be regarded as the λ-abstraction λ`x.(x+1)`, where `1` and `+` are either builtins or can be also encoded as λ-expressions (these are discussed in the sequel).

Another common construct in programming languages is the `let` binder, which allows to bind expressions to names and then use those names as replacements for their corresponding expressions. For example,

```
let x1 = E1 and x2 = E2 and...and xn = En in E
```

first binds the expressions `E1`, `E2`, ..., `En` to the names `x1`, `x2`, ..., `xn`, respectively, and then `E` is evaluated. It is not hard to see that this construct is equivalent to the λ-expression (λ`x1`.λ`x2` ... λ`xn`.`E`) `E1 E2` ... `En`.

Currying

Recall from Section 2.1.1 that there is a bijection between $[A \times B \to C]$ and $[A \to [B \to C]]$, where $[X \to Y]$ represents the set of functions of domain X and codomain Y. Indeed, any function $f : A \times B \to C$ can be regarded as a function $g : A \to [B \to C]$, where for any $a \in A$, $g(a)$ is defined as the function $h_a : B \to C$ with $h_a(b) = c$ if and only if $f(a, b) = c$. Similarly, any function $g : A \to [B \to C]$ can be regarded as a function $f : A \times B \to C$, where $f(a, b) = g(a)(b)$.

This observation led to the important concept called *currying*, which allows us to eliminate functions with multiple arguments from the core of a language, replacing them systematically by functions admitting only one argument as above. Thus, functions with multiple arguments are just syntactic sugar.

From now on we may write λ-expressions of the form $\lambda xyz \cdots .E$ as shorthands for their *uncurried versions* $\lambda x.\lambda y.\lambda z. \cdots .E$. With this convention, λ-calculus therefore admits multiple-argument λ-abstractions.

Note, however, that unlike in many familiar languages, curried functions can be applied on fewer arguments. For example, $(\lambda xyz.E)E'$ β-reduces to $\lambda yz.(E[E'/x])$. Also, since λ-application was defined to be left-associative, $(\lambda xyz.E)E_1E_2$ β-reduces to $\lambda z.((E[E_1/x])[E_2/y])$.

Most functional languages today support curried functions. The advantage of currying is that one only needs to focus on defining the meaning or on implementing effectively functions of one argument.

Church Booleans

Booleans are perhaps the simplest data-type that one would like to have in a programming language. λ-calculus so far provides no explicit support for Booleans or conditionals. We next show that λ-calculus provides *implicit* support for Booleans. In other words, the machinery of λ-calculus is powerful enough to simulate Booleans and what one would normally want to do with them in a programming language. What we discuss next is therefore a *methodology* to program with Booleans in λ-calculus.

The idea is to regard a Boolean through a behavioral prism: with a Boolean, one can always choose one of any two objects—if true then the first, if false then the second. In other words, one can identify a Boolean b with a universally quantified conditional "for any `x` and `y`, `if` b `then x else y`". With this behavior of Booleans in mind, one can now relatively easily translate Booleans and Boolean operations in λ-calculus:

$$\texttt{true} := \lambda xy.x$$

$$\texttt{false} := \lambda xy.y$$

$$\texttt{if-then-else} := \lambda xyz.xyz$$

$$\texttt{and} := \lambda xy.(x\, y\, \texttt{false})$$

Other Boolean operations can be defined in a similar style (see Exercise 198).

This encoding for Booleans is known under the name of *Church Booleans*. We can use β-reduction to show, for example, that `and true false` $\Rightarrow_\beta$ `false`, that is, `and true false` $\equiv_\beta$ `false`. We can show relatively easily that the Church Booleans have all the desired properties of Booleans. Let us, for example, show the associativity of `and`. First, note the obvious equivalences:

$$\texttt{and (and x y) z} \equiv_\beta \texttt{x y false z false}$$

$$\texttt{and x(and y z)} \equiv_\beta \texttt{x (y z false) false}$$

These equivalences tell us that we cannot expect the properties of Booleans to hold for any λ-expressions. Therefore, in order to complete the proof of associativity of `and`, we need to make further assumptions regarding the Booleanity of `x`, `y`, `z`. If `x` is `true`, that is $\lambda xy.x$, then both right-hand-side λ-expressions above reduce to `y z false`. If `x` is `false`, that is $\lambda xy.y$, then the first reduces to `false z false` which further reduces to `false`, while the second reduces to `false` in one step. We can similarly prove the desired properties of all the Boolean operators (Exercise 199), relying only on the generic β-equivalence of λ-calculus.

We may often introduce definitions such as the above for the Church Booleans, using the symbol `:=`. Note that this is not a meta binding constructor on top of λ calculus. It is just a way for us to avoid repeating certain frequent λ-expressions; one can therefore regard them as macros. Anyway, they admit a simple translation into standard λ-calculus, using the usual convention for translating bindings. Therefore, one can regard the λ-expression "`and true false`" as syntactic sugar for

$$(\lambda\texttt{and}.\lambda\texttt{true}.\lambda\texttt{false}.\,\texttt{and true false})\ ((\lambda\texttt{false}.\lambda xy.\ x\, y\, \texttt{false})\ (\lambda xy.y))\ (\lambda xy.x)\ (\lambda xy.y)$$

Pairs

λ-calculus can also naturally encode data-structures of interest in most programming languages. The idea is that λ-abstractions, by their structure, can store useful information. Let us, for example, consider pairs as special cases of records. Like Booleans, pairs can also be regarded behaviorally: a pair is a black-box that can store any two expressions and then allow one to retrieve those through appropriate projections. Formally, we would like to define λ-expressions `pair`, `1st` and `2nd` in such a way that for any other λ-expressions E and E', it is the case that `1st` (`pair` E E') and `2nd` (`pair` E E') are β-equivalent to E and E', respectively. Fortunately, these can be defined quite easily:

$$\texttt{pair} := \lambda xyb.\, bxy,$$

$$\texttt{1st} := \lambda p.\, p\,\texttt{true}, \text{ and}$$

$$\texttt{2nd} := \lambda p.\, p\,\texttt{false}.$$

The idea is therefore that `pair` E E' β-reduces to the λ-expression $\lambda b.\, b\, E\, E'$, which freezes E and E' inside a λ-abstraction, together with a handle, b, which is expected to be a Church Boolean, to unfreeze them later. Indeed, the first projection, `1st`, takes a pair and applies it to `true` hereby unfreezing its first component, while the second projection applies it to `false` to unfreeze its second component.

Church Numerals

Numbers and the usual operations on them can also be defined as λ-expressions. The basic idea is to regard a natural number n as a λ-expression that has the potential to apply a given operation n times on a given starting λ-expression. Therefore, λ-numerals, also called *Church numerals*, take two arguments, *what to do* and *what to start with*, and apply the first as many times as the intended numeral on the second. Intuitively, if the action was *successor* and the starting expression was *zero*, then one would get the usual numerals. Formally, we define numerals as follows:

$$0_\lambda := \lambda sz.\, z$$

$$1_\lambda := \lambda sz.\, s\, z$$

$$2_\lambda := \lambda sz.\, s\,(s\, z)$$

$$3_\lambda := \lambda sz.\, s\,(s\,(s\, z))$$

$$\ldots$$

With this intuition for numerals in mind, we can now easily define a successor operation on numerals:

$$\texttt{succ} := \lambda nsz.\, n\, s\,(s\, z)$$

The above says that for a given numeral n, its successor `succ` n is the numeral that applies the given operation s for n times starting with $s\ z$. There may be several equivalent ways to define the same intended meaning. For example, one can also define the successor operation by applying the operation s only once, but on the expression $n\ s\ z$; therefore, we can define

$$\texttt{succ'} := \lambda nsz.\, s\,(n\, s\, z)$$

One may, of course, want to show that `succ` and `succ'` are equal. However, they are *not* equal as λ-expressions. To see it, one can apply both of them on the λ-expression $\lambda xy.\, x$: one gets after β-reduction $\lambda sz.\, s$ and, respectively, $\lambda sz.\, s\, s$. However, they are equal when applied on Church numerals (see Exercise 200).

One can also define addition as a λ-abstraction, e.g., as follows:

$$\texttt{plus} := \lambda mnsz.\, m\, s\, (n\, s\, z)$$

One of the most natural questions that one can and should ask when one is exposed to a new model of natural numbers, is whether it satisfies the Peano axioms. In our case, this translates to whether the following properties hold for all Church numerals n_λ and m_λ (see Exercise 201):

$$\texttt{plus}\ 0_\lambda\ m_\lambda\ \equiv_\beta\ m_\lambda, \text{ and}$$

$$\texttt{plus}\ (\texttt{succ}\ n_\lambda)\ m_\lambda\ \equiv_\beta\ \texttt{succ}\ (\texttt{plus}\ n_\lambda\ m_\lambda).$$

Church numerals in combination with pairs allow us to define certain recursive behaviors. Let us next define a more interesting function on Church numerals, namely one that calculates Fibonacci numbers. Specifically, we want to define a λ-expression `fibo` with the property that `fibo` n_λ β-reduces to the nth Fibonacci number. Recall that Fibonacci numbers are defined recursively as $f_0 = 0$, $f_1 = 1$, and $f_n = f_{n-1} + f_{n-2}$ for all $n \geq 2$. The idea is to define a two-number window that slides through the sequence of Fibonacci numbers until it reaches the desired number. The window is defined as a pair and the sliding by moving the second element in the pair on the first position and placing the next Fibonacci number as the second element. The shifting operation needs to be applied as many times as the index of the desired Fibonacci number:

$$\texttt{start} := \texttt{pair}\ 0_\lambda\ 1_\lambda,$$

$$\texttt{step} := \lambda p.\, \texttt{pair}\ (\texttt{2nd}\ p)\ (\texttt{plus}\ (\texttt{1st}\ p)\ (\texttt{2nd}\ p)),$$

$$\texttt{fibo} := \lambda n.\, \texttt{1st}\ (n\ \texttt{step}\ \texttt{start}).$$

We will shortly discuss a technique to support recursive definitions of functions in a general way, not only on Church numerals.

Another use of the technique above is in defining the predecessor operation on Church numerals:

$$\texttt{start} := \texttt{pair}\ 0_\lambda\ 0_\lambda,$$

$$\texttt{step} := \lambda p.\, \texttt{pair}\ (\texttt{2nd} p)\ (\texttt{plus}\ 1_\lambda\ (\texttt{2nd} p)),$$

$$\texttt{pred} := \lambda n.\, \texttt{1st}\ (n\ \texttt{step}\ \texttt{start}).$$

Note that $\texttt{pred}\ 0_\lambda\ \equiv_\beta\ 0_\lambda$, which is a slight violation of the usual properties of the predecessor operation on integers. The above definition of predecessor is computationally very inefficient. Unfortunately, there does not seem to be any better way to define this operation on Church numerals.

Subtraction can now be defined easily:

$$\texttt{sub} := \lambda mn.\, n\, \texttt{pred}\, m.$$

Note, again, that negative numbers are collapsed to 0_λ.

Let us next see how relational operators can be defined on Church numerals. These are useful to write many meaningful programs. We first define a helping operation, to test whether a number is zero:

$\texttt{zero?} := \lambda n.\, n\ (\texttt{and}\ \texttt{false})\ \texttt{true}.$

Now the "less than or equal to" (`leq`), the "larger than or equal to" (`geq`), and the "equal to" (`equal`) can be defined as follows:

$\texttt{leq} := \lambda mn.\, \texttt{zero?}\ (\texttt{sub}\ m\ n),$

$\texttt{geq} := \lambda mn.\, \texttt{zero?}\ (\texttt{sub}\ n\ m),$

$\texttt{equal} := \lambda mn.\, \texttt{and}\ (\texttt{leq}\ m\ n)\ (\texttt{geq}\ m\ n).$

Adding Built-ins

As shown above, λ-calculus is powerful enough to encode many data-structures and data-types of interest. However, as it is the case with many other *pure* programming paradigms, in order for λ-calculus to be usable as a reasonably efficient programming language, it needs to provide built-ins comprising efficient implementations for common data-types and operations on them.

We here only discuss the addition of *built-in integers* to λ-calculus; the addition of other data-types is similar. We say that the new λ-calculus obtained this way is *enriched*. Depending on how the underlying substitution operation and how the builtin operations are defined, enriching λ-calculus with builtin integers can take very little effort: syntactically, we may only need to add integers as λ-expressions, and semantically, we may only need to add the rules defining the builtin operations to the language, in addition to the already existing β-reduction rule. Specifically, assume a substitution operation that works over any syntactic extension of λ-calculus, essentially applying homomorphically through all non-binding language constructs, like in Exercise 196. Also, assume a syntactic category *Int* for integer values, e.g., 5, 7, etc., and operations on them, e.g., +, together with a rewrite relation capable of reducing *Int* expressions, e.g., $5+7 \Rightarrow 12$.

There are at least two ways to enrich λ-calculus with integers: the simplest is to collapse the *Int* and *Exp* syntactic categories; another is to make *Int* a syntactic subcategory of *Exp*, but then all the *Int* operations need to be overloaded to take and yield *Exp*. In both cases, the integer values become particular expressions, and the integer reduction rules harmoniously co-exist with β-reduction. For example, 5+7 still reduces to 12. But what is more interesting is that $\lambda x.7+5$ also makes sense and reduces to $\lambda x.12$ (without applying any β-reduction step, but only the reduction that *Int* provides). Moreover, $(\lambda yx.7+y)\ 5$ first β-reduces to $\lambda x.7+5$ and then *Int*-reduces to $\lambda x.12$. The above β-reduction works because integer values are regarded as constants, that is, operations with no arguments, so the generic substitution (homomorphically) applies on them, too, keeping them unchanged, as expected: $I[E'/x] = I$, for any integer I.

Note that one can now write λ-expressions that are not well formed, such as the λ application of one integer to another: 7 5. It would be the task of a type checker to catch such kind of errors. We here focus only on the evaluation, or reduction, mechanism of the enriched calculus, assuming that programs are well-formed. Ill-formed λ-expressions may get stuck when β-reduced.

4.5.7 Fixed-Points and Recursion

Recursion almost always turns out to be a subtle topic in foundational approaches to programming languages. We have already seen the divergent combinator

$\texttt{omega} := (\lambda x.\, x\ x)\ (\lambda x.\, x\ x),$

which has the property that $\texttt{omega} \Rightarrow_\beta \texttt{omega}$, that is, it leads to an infinite recursion. While `omega` has a recursive behavior, it does not give us a principled way to define recursion in λ-calculus.

But what is *recursion*? Or to be more precise, what is a *recursive function*? Let us examine the definition of a factorial function, in some conventional programming language, that we would like to be recursive:

```
function f(x) {
   if x == 0 then 1 else x * f(x - 1)
}
```

In a functional language that is closer in spirit to λ-calculus the definition of factorial would be:

```
letrec
   f(x) = if x == 0 then 1 else x * f(x - 1)
in f(3) .
```

Note that the `letrec` binding is necessary in the above definition. If we used `let` instead, then according to the syntactic sugar transformation of functional bindings into λ-calculus, the above would be equivalent to

```
(λ f . f 3)
 (λ x . if x == 0 then 1 else x * f(x - 1)) ,
```

so the underlined `f` is free rather than bound to λ `f`, as expected. A functional programming language would report an error here.

The foundational question regarding recursion in λ-calculus is therefore the following: how can we define a λ-abstraction

```
f := <begin-exp ... f ... end-exp>,
```

that is, one in which the λ-expression refers to itself in its scope? Let us put the problem in a different light. Consider instead the well-formed well-behaved λ-expression

```
F := λ f .  <begin-exp ... f ... end-exp>,
```

that is, one which takes any λ-expression, in particular a λ-abstraction, and plugs it at the right place into the scope of the λ-expression that we want to define recursively. The question now becomes the following one: can we find a *fixed point* `f` of `F`, that is, a λ-expression `f` with the property that

$$\texttt{F f} \equiv_\beta \texttt{f}\,?$$

Interestingly, λ-calculus has the following notorious and perhaps surprising result:

Theorem 24. ***(Fixed-point theorem of untyped λ-calculus)*** *For any λ-expression F, there is some λ-expression X such that $FX \equiv_\beta X$.*

Proof. One such X is the λ-expression $(\lambda x.\, F\,(x\,x))\,(\lambda x.\, F\,(x\,x))$. Indeed,

$$\begin{array}{rcl} X & = & (\lambda x.\, F\,(x\,x))\,(\lambda x.\, F\,(x\,x)) \\ & \equiv_\beta & F\,((\lambda x.\, F\,(x\,x))\,(\lambda x.\, F\,(x\,x))) \\ & = & F\,X. \end{array}$$

Therefore, X is a fixed-point of F. □

The proof of the fixed-point theorem above suggests defining the following famous *fixed-point combinator*:

$$Y \;:=\; \lambda F.\,(\lambda x.\, F\,(x\,x))\,(\lambda x.\, F\,(x\,x)).$$

With this, for any λ-expression F, the λ-application $Y\,F$ becomes the fix-point of F; therefore, $F\,(Y\,F) \equiv_\beta Y\,F$. Thus, we have a constructive way to build fixed-points for any λ-expression F. Note that F does not need to be a λ-abstraction.

Let us now return to the recursive definition of factorial in λ-calculus enriched with integers. For this particular definition, let us define the λ-expression:

```
F := λ f . λ x . (if x == 0 then 1 else x * f(x - 1))
```

The recursive definition of factorial is therefore a fixed-point of `F`, namely, Y`F`. It is such a fixed-point λ-expression that the `letrec` functional language construct in the definition of factorial refers to!

Let us experiment with this λ-calculus definition of factorial, by calculating factorial of 3:

```
(Y F) 3 ≡β
F (Y F) 3 =
(λ f . λ x . (if x == 0 then 1 else x * f(x - 1))) (Y F) 3 ⇒β
if 3 == 0 then 1 else 3 * (Y F)(3 - 1) ⇒
3 * ((Y F) 2) ≡β
...
6 * ((Y F) 0) ≡β
6 * (F (Y F) 0) =
6 * ((λ f . λ x . (if x == 0 then 1 else x * f(x - 1))) (Y F) 0) ⇒β
6 * if 0 == 0 then 1 else 0 * (Y F)(0 - 1) ⇒
6 * 1 ⇒
6
```

Therefore, λ-calculus can be regarded as a simple programming language, providing support for functions, numbers, data-structures, and recursion. It can be shown that any computable function can be expressed in λ-calculus in such a way that its computation can be performed by β-reduction. This means that λ-calculus is a *Turing-complete model of computation*.

There are two aspects of λ-calculus that lead to complications when one wants to implement it.

One is, of course, the substitution: efficiency and correctness are two opposing tensions that one needs to address in any direct implementation of λ-calculus.

The other relates to the strategies of applying β-reductions: so far we used what is called *full β-reduction*, but other strategies include *normal evaluation*, *call-by-name*, *call-by-value*, etc. There are λ-expressions whose β-reduction does not terminate under one strategy but terminates under another. Moreover, depending upon the strategy of evaluation employed, other fixed-point combinators may be more appropriate.

Like β-reduction, the evaluation of expressions is confluent in many pure functional languages. However, once a language allows side effects, strategies of evaluation start playing a crucial role; to avoid any confusion, most programming languages hardwire a particular evaluation strategy, most frequently *call-by-value*.

We do not discuss strategies of evaluation here. Instead, we approach the other delicate operational aspect of λ-calculus, namely the substitution. In fact, we show that it can be completely eliminated if one applies a systematic transformation of λ-expressions into expressions over a reduced set of combinators.

More precisely, we show that any closed λ-expression can be systematically transformed into a λ-expression build over only the combinators $K := \lambda xy.x$ and $S := \lambda xyz.xz(yz)$, together with the λ-application operator. For example, the identity λ-abstraction $\lambda x.x$ is going to be $S\,K\,K$; indeed,

$S\,K\,K \equiv_\beta \lambda z.Kz(Kz) = \lambda z.(\lambda xy.x)z(Kz) \equiv_\beta \lambda z.z \equiv_\alpha \lambda x.x.$

Interestingly, once such a transformation is applied, one will not need the machinery of λ-calculus and β-reduction anymore. All we'll need to do is to capture the contextual behavior of K and S, which can be defined equationally very elegantly: $KXY = X$ and $SXYZ = XZ(YZ)$, for any other KS-expressions X, Y, Z.

Before we do that, we need to first discuss two other important aspects of λ-calculus: η-equivalence and extensionality.

4.5.8 Eta Equivalence and Extensionality

Here we discuss two common but equivalent extensions of λ-calculus, both concerned with additional proof support for equality of λ-expressions (besides α-conversion and β-equivalence). The former is called *η-equivalence* and allows us to state that expressions $\lambda x.\, E\, x$ and E are always equivalent, provided that x does not occur free in E. The latter is called *extensionality* and allows us to derive that E and E' are equal provided that $E\, x$ and $E'\, x$ are equal for some variable x that does not occur free in any of E and E'.

Eta Equivalence

Let us consider the λ-expression $\lambda x.Ex$, where E is some λ-expression that does not contain x free. Intuitively, $\lambda x.Ex$ does nothing but wraps E: when called, it passes its argument to E and then passes back E's result. When applied on some λ-expression, say E', note that $\lambda x.Ex$ and E *behave the same*. Indeed, since E does not contain any free occurrence of x, one can show that $(\lambda x.Ex)E' \equiv_\beta EE'$. Moreover, if E is a λ-abstraction, say $\lambda y.F$, then $\lambda x.Ex = \lambda x.(\lambda y.F)x \equiv_\beta \lambda x.F[x/y]$. The latter is α-equivalent to $\lambda y.F$, so it follows that in this case $\lambda x.Ex$ is β-equivalent to E.

Even though $\lambda x.Ex$ and E have similar behaviors in applicational contexts and they can even be shown β-equivalent when E is a λ-abstraction, there is nothing to allow us to use their equality as an axiom in our equational inferences. In particular, there is no way to show that the combinator $\lambda x.\lambda y.xy$ is equivalent to $\lambda x.x$. To increase the proving capability of λ-calculus, still without jeopardizing its basic intuitions and applications, we consider its extension with the following equation:

$$(\eta) \qquad \lambda x.Ex = E\,,$$

for any $x \notin FV(E)$. We let $E \equiv_{\beta\eta} E'$ denote the fact that the equation $E = E'$ can be proved using standard equational deduction from all the equational axioms above: (α), (β), (η), plus those for substitution. The relation $\equiv_{\beta\eta}$ is also called *$\beta\eta$-equivalence*. The λ-calculus enriched with the rule (η) is also written $\lambda + \eta$.

Extensionality

Extensionality is a deduction rule encountered in several branches of mathematics and computer science. It intuitively says that in order to prove two objects equal, one may first extend them in some rigorous way. The effectiveness of extensionality comes from the fact that it may often be the case that the extended versions of the two objects are easier to prove equivalent.

Extensionality was probably first considered as a proof principle in set theory. In naive set theory, sets are built in a similar fashion to Peano numbers, that is, using some simple constructors (together with several constraints), such as the empty set $\emptyset$ and the list constructor $\{x_1, \ldots, x_n\}$. Thus, $\{\emptyset, \{\emptyset, \{\emptyset\}\}\}$ is a well-formed set. With this way of constructing sets, there may be the case that two sets with the same elements have totally different representations. Consequently, it is almost impossible to prove any meaningful property on sets, such as distributivity of union and intersection, etc., by just taking into account how sets are constructed. In particular, proofs by structural induction are close to useless.

Extensionality is often listed as the first axiom in any axiomatization of set theory. In that context, it basically says that two sets are equal iff they have the same elements. Formally, sets S and S' are equal whenever we can prove the statement "for any element x, $x \in S$ iff $x \in S'$". Therefore, in order to show sets S and S' equal, we can first extend them (regarded as syntactic terms) by applying them the membership operator. In most cases the new task is easier to prove.

Similarly, the principle of extensionality applies to functions: two functions are equal whenever they evaluate to the same values for the same elements. Formally, functions f and g are equal whenever we can prove the statement "for any element x, $f(x) = g(x)$". We again have to prove a syntactically extended task, but like for sets it is often the case that the extended task is simpler to prove.

In λ-calculus, extensionality takes the following shape:

(ext) `If` $Ex = E'x$ `for some` $x \notin FV(EE')$, `then` $E = E'$.

Therefore, two λ-abstractions are equal if they are equal when applied on some variable that does not occur free in any of them. Note that "for some x" can be replaced by "for any x" in `ext`. We let $E \equiv_{\beta\texttt{ext}} E'$ denote the fact that the equation $E = E'$ can be proved using standard equational deduction using (α) and (β), together with `ext`. The λ-calculus extended with `ext` is also called $\lambda + \texttt{ext}$.

The following important result says the extensions of λ-calculus with (η) and with `ext` are equivalent:

Theorem 25. *$\lambda + \eta$ is equivalent to $\lambda + \texttt{ext}$.*

Proof. In order to show that two mathematical theories are equivalent, one needs to show two things: (1) how the syntax of one translates into the syntax of the other, or in other words to show how one can mechanically translate assertions in one into assertions in the other, and (2) that all the axioms of each of the two theories can be proved from the axioms of the other, along the corresponding translation of syntax. In our particular case of $\lambda + \eta$ and $\lambda + \texttt{ext}$, syntax remains unchanged when moving from one logic to another, so (1) above is straightforward. We will shortly see another equivalence of logics, where (1) is rather involved. Regarding (2), all we need to show is that under the usual λ-calculus with (α) and (β), the equation (η) and the principle of extensionality are equivalent.

Let us first show that (η) implies `ext`. For that, let us assume that $Ex \equiv_{\beta\eta} E'x$ for some λ-expressions E and E' and for some variable $x \notin FV(EE')$. We need to show that $E \equiv_{\beta\eta} E'$:

$$E \equiv_{\beta\eta} \lambda x.Ex \equiv_{\beta\eta} \lambda x.E'x \equiv_{\beta\eta} E'.$$

Note the use of $\equiv_{\beta\eta}$ in the equivalences above, rather than just $\equiv_\beta$. That is because, in order to prove the axioms of the target theory, $\lambda + \texttt{ext}$ in our case, one can use the entire calculus machinery available available in the source theory, $\lambda + \eta$ in our case.

Let us now prove the other implication, namely that `ext` implies (η). We need to prove that $\lambda x.Ex \equiv_{\beta\texttt{ext}} E$ for any λ-expression E and any $x \notin FV(E)$. By extensionality, it suffices to show that $(\lambda x.Ex)x \equiv_{\beta\texttt{ext}} Ex$, which follows immediately by β-equivalence because x is not free in E. □

4.5.9 Combinatory Logic

Even though λ-calculus can be defined equationally and is a relatively intuitive framework, as we have noticed several times by now, substitution makes it non-trivial to implement effectively. There are several approaches in the literature addressing the subtle problem of automating substitution to avoid variable capture, all with their advantages and disadvanteges. We here take a different approach. We show how λ-expressions can be

automatically translated into expressions over combinators, in such a way that substitution will not even be needed anymore.

A question addressed by many researchers several decades ago, still interesting today and investigated by many, is whether there is any *simple* equational theory that is entirely equivalent to λ-calculus. Since λ-calculus is Turing complete, such a simple theory may provide a strong foundation for computing.

Combinatory logic was invented by Moses Shönfinkel in 1920. The work was published in 1924 in a paper entitled *"On the building blocks of mathematical logic"*. Combinatory logic is a simple equational theory over two sorts, *Var* and *Exp* with $Var < Exp$, a potentially infinite set x, y, etc., of constants of sort *Var* written using lower-case letters, two constants K and S of sort *Exp*, one application operation with the same syntax and left-associativity parsing convention as in λ-calculus, together with the two equations

$$KXY = X,$$
$$SXYZ = XZ(YZ),$$

quantified universally over X, Y, Z of sort *Exp*. The constants K and S are defined equationally in such a way to capture the intuition that they denote the combinators $\lambda xy.x$ and $\lambda xyz.xz(yz)$, respectively. The terms of the language, each of which denoting a function, are formed from variables and constants K and S by a single construction, function application. For example, $S(SxKS)yS(SKxK)z$ is a well-formed term in combinatory logic, denoting some function of free variables x, y, and z.

Let CL be the equational theory of combinatory logic above. Note that a function *FV* returning the free variables that occur in a term in combinatory logic can be defined in a trivial manner, because there are no bound variables in CL. Also, note that the extensionality principle from λ-calculus translates unchanged to CL:

(ext) If $Ex = E'x$ for some $x \notin FV(EE')$, then $E = E'$.

Let CL + ext be CL enriched with the principle of extensionality. The following is a landmark result:

Theorem 26. λ + ext *is equivalent to* CL + ext.

Proof. Let us recall what one needs to show in order for two mathematical theories to be equivalent: (1) how the syntax of one translates into the syntax of the other; and (2) that all the axioms of each of the two theories can be proved from the axioms of the other, along the corresponding translation of syntax.

Let us consider first the easy part: λ + ext implies CL + ext. We first need to show how the syntax of CL + ext translates into that of λ + ext. This is easy and it was already mentioned before: let K be the combinator $\lambda xy.x$ and let S be the combinator $\lambda xyz.xz(yz)$. We then need to show that the two equational axioms of CL + ext hold under this translation: they can be immediately proved by β-equivalence. We also need to show that the extensionality in CL + ext holds under the above translation: this is obvious, because it is exactly the same as the extensionality in λ + ext.

Let us now consider the other, more difficult, implication. So we start with CL + ext, where K and S have no particular meaning in λ-calculus, and we need to define some map that takes any λ-expression and translates it into an expression in CL.

To perform such a transformation, let us add syntax for λ-abstractions to CL, but without any of the equations of λ-calculus. This way one can write and parse λ-expressions, but still have no meaning for those. The following ingenious *bracket abstraction* rewriting system transforms any uninterpreted λ-expression into an expression using only K, S, and the free variables of the original λ-expression:

1. $\lambda x.\rho \Rightarrow [x]\rho$
2. $[x]y \Rightarrow \begin{cases} SKK & \text{if } x = y \\ Ky & \text{if } x \neq y \end{cases}$
3. $[x](\rho\rho') \Rightarrow S([x]\rho)([x]\rho')$
4. $[x]K \Rightarrow KK$
5. $[x]S \Rightarrow KS$

The first rule removes all the λ-bindings, replacing them by corresponding bracket expressions. Here ρ and ρ' can be any expressions over K, S, variables, and the application operator, but also over the λ-abstraction operator $\lambda_._ : Var \to Exp$. However, note that rules 2-5 systematically elliminate all the brackets. Therefore, the *bracket abstraction* rules above eventually transform any λ-expression into an expression over only K, S, variables, and the application operator.

The correctness of the translation of λ `+ ext` into `CL + ext` via the bracket abstraction technique is rather technical: one needs to show that the translated versions of equations in λ can be proved (by structureal induction) using the machinery of `CL + ext`.

Exercise 185. ***(Technical)*** *Prove the correctness of the translation of λ `+ ext` into `CL + ext` above.*

We do not need to understand the details of the proof of correctness in the exercise above in order to have a good intuition on why the bracket abstraction translation works. To see that, just think of the bracket abstraction as a means to associate equivalent λ-expressions to other λ-abstractions, within the framework of λ-calculus, where K and S are their corresponding λ-expressions. As seen above, it eventually reduces any λ-expression to one over only combinators and variables, containing no explicit λ-abstractions except those that define the combinators K and S. To see that the bracket abstraction is correct, we can think of each bracket term $[x]E$ as the λ-expression that it was generated from, $\lambda x.E$, and then show that each rule in the bracket abstraction transformation is sound within λ-calculus. For example, rule 3 can be shown by extensionality: $(\lambda x.\rho\rho')z \equiv_\beta (\rho[z/x])(\rho'[z/x]) \equiv_\beta ((\lambda x.\rho)z)((\lambda x.\rho')z)) \equiv_\beta (\lambda xyz.xz(yz))(\lambda x.\rho)(\lambda x.\rho')z = S(\lambda x.\rho)(\lambda x.\rho')z$, so by extensionality, $\lambda x.\rho\rho' \equiv_{\beta\text{ext}} S(\lambda x.\rho)(\lambda x.\rho')$.

This way, one can prove the soundness of each of the rules in the bracket abstraction translation. As one may expect, the tricky part is to show the completeness of the translation, that is, that everything one can do with λ-claculus and `ext` can also do with with its subcalculus `CL + ext`. This is not hard, but rather technical.

Exercise 186. *Define the bracket abstraction translation above formally in Maude. To do it, first define `CL`, then add syntax for λ-abstraction and bracket to `CL`, and then add the bracket abstraction rules as equations (which are interpreted as rewrite rules by Maude). Convince yourself that substitution is not a problem in `CL`, by giving an example of a λ-expression which would not be reducible with the definition of λ-calculus in Exercise* **??***, but whose translation in `CL` can be reduced with the two equations in `CL`.*

---

some additional notes:

In the pure system there is a small set of constants all of which are themselves pure combinators, usually the primitive combinators are named S, K and I, though I is in fact definable in terms of S and K.

The intended meaning of these constants can be described simply by showing a corresponding transformation or reduction which the constant may be thought of as effecting.

The transformations are as follows: I x = x K x y = x S f g x = (f x)(g x) This set of combinators has the property of combinatory completeness which means that for any function definition of the form: f x = combinatory expression involving x

There exists an expression E in S,K and I such that: f = E

This property establishes the close relationship between Pure Combinatory Logic and the Pure lambda Calculus, in which a special notation for functional abstraction is available. It shows that the notation for functional abstraction, though a great convenience, adds no expressiveness to the language.

As well as having combinatorial completeness, Pure Combinatory Logic is able to express all effectively computable functions over natural numbers appropriately represented as combinators.

[Barenderegt84] Barendregt, H.P.; The Lambda Calculus - Its Syntax and Semantics; Second Edition, North Holland 1984 purchase from amazon purchase from ibs

This is the first published work in the field which came later to be known as combinatory logic.

Schnfinkel shows how the use of bound variables in logic can be dispensed with. The use of higher order functions makes possible the reduction of logic to a language consisting of one constructor (the application of a function to an argument) and three primitive constants U, C (now usually called K) and S. A function is termed ”higher order” if it will accept a function as an argument, or return one as its result. U, C, and S are all higher order functions.

The meaning of these primitives may be understood through the following informal definitions:

Cxy = (C(x))(y) = x C is a function which given any value x, returns the constant x valued function. Sfgx = ((S(f))(g))(x) = (f(x))(g(x)) S is a function which combines two functions, say f and g, supplied as successive arguments. The resulting function, given a value x, returns the value obtained by applying the value of f at x to the value of g at x. UPQ = (U(P))(Q) = forall x. not(P(x) and Q(x)) U is a generalised scheffer stroke. It should be thought of as applying to two predicates and returns the universal quantification of the negated conjunction of the two predicates.

These combinators are sufficient to enable arbitrary first order predicates to be expressed without the use of bound variables, which appear in first order logic whenever a quantifier is used. This can be demonstrated most straightforwardly using a simple algorithm which converts lambda-expressions to combinators. They are not limited to first-order predicates, but without some constraints (equivalent to those found in first order logic, or more liberally and appropriately to those in Church’s Simple Theory of Types [Church40]) the logic which results from the use of these combinators is at risk of proving inconsistent. Combinatory logic has traditionally tried to avoid type constraints and has therefore been dogged with difficulties in achieving strength and consistency.

Schnfinkel’s paper remains an accessible introduction to combinatory logic which makes clear the original motivation for this innovation. This original motivation was vigorously pursued later by H.B.Curry and his collaborators. The results of Curry’s programme were published in Combinatory Logic, Vols 1 & 2.

While Schfinkel was successful in showing that combinators could provide plausible notation, the semantic and proof theoretic aspects were more difficult to put in place. The difficulty is more pronounced if it hoped to use combinatory logic as a foundation for mathematics, than it is for the use of combinators in less demanding roles (e.g. for programming languages).

The Pure combinatory logic may be thought of as Schnfinkels system with the logical operator ”U” omitted (though the presentation may vary). Illative combinatory logics are formed by adding additional logical primitives to the pure combinators. There are many illative combinatory logics, which vary among themselves in substance as well as in presentation.

Pure combinatory logic is so closely related to Church's lambda-calculus that it is best studied alongside the lambda-calculus, for which the most comprehensive modern text is probably The Lambda Calculus. A very popular and entertaining introduction to the pure combinators may be found in To Mock a Mockingbird.

Research on illative combinatory logics has been continued by Curry's students, Roger Hindley and Jonathan Seldin, and by Martin Bunder. Hindley, Lercher and Seldin published An Introduction to Combinatory Logic, an excellent and readable short introduction to the subject, now superseded by the more comprehensive An Introduction to Combinators and the Lambda-Calculus

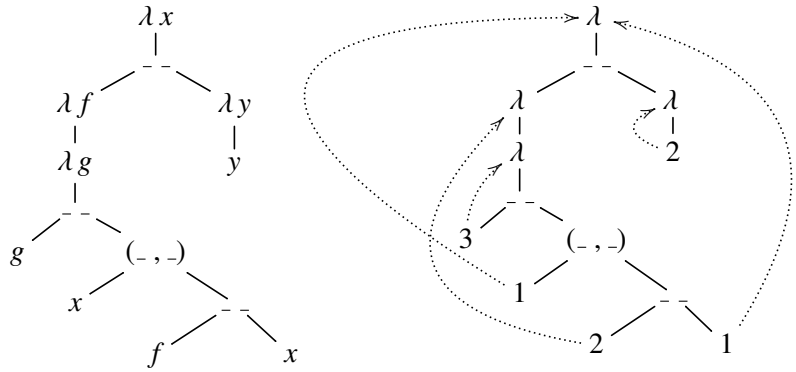
Recently Randall Holmes has developed logical systems based on combinatory logic and the lambda calculus using methods derived from Quine's formulations of set theory.

The attempt to make a foundation for mathematics using the illative combinatory logic were blighted by Curry's paradox and semantic opacity. While systems avoiding this paradox have been devised, they are typically weak, and lack the kind of convincing semantic intuition found in first order set theory.

[Curry58] Curry, Haskell B., Feys, Robert; Combinatory Logic, Volume I; North Holland 1958; [Curry72] Curry, Haskell B., Hindley, J.Roger, Seldin, Jonathan P.; Combinatory Logic, Volume II; North Holland 1972; ISBN 0 7204 2208 6

4.5.10 De Bruijn Nameless Representation

Interestingly, note that the morphism obtained above contains no references to the variables that occur in the original λ-expression. It can be shown that the interpretation of λ-expressions into a CCC is *invariant to α-conversion*. To see that, let us draw the morphism above as a tree, where we write λ instead of *curry*, $_\ _$ instead of $\langle _\,,_ \rangle$; $app^{[\![s]\!],[\![t]\!]}$, $(_\,,_)$ instead of the remaining $\langle _\,,_ \rangle$ and i instead of π_i (and omit the types):



The (right) tree above suggests a representation of λ-expressions that is invariant to α-conversion: each binding variable is replaced by a natural number, representing the number of λs occurring on the path to it; that number then replaces consistently all the bound occurrences of the variable. The corresponding lambda expression without variables obtained using this transformation is $\lambda\,(\lambda\,\lambda\,(3\ (1, (2\ 1)))\ \lambda\,2)$.

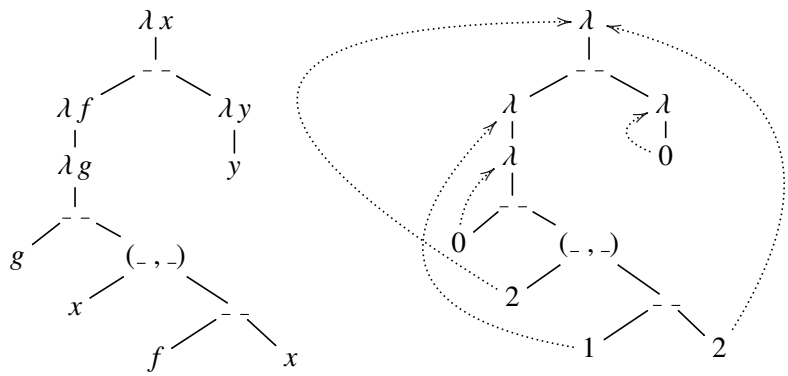
Exercise 187. *Explain why this representation is invariant to α-conversion.*

The representation of α-expressions above was explicitly proposed as a means to implement λ-calculus in 1971 by Nicholas de Bruijn.

In the same paper, de Bruijn proposed another encoding which became more popular. We do not know whether de Bruijn was influenced by the CCC interpretation of λ-expressions or not, but we discuss his other representation technique here.

de Bruijn Nameless Representation of λ-expression

The second and more popular representation technique of λ-expressions proposed by Nicholas de Bruijn in 1971 is a bottom-up version of the above representation. For the above example the tree representing the encoding is (we omit the types):



In this encoding, each variable is replaced by the number of lambda abstractions on the path from it to the lambda abstraction binding it. The encoding for the given example is $\lambda\,(\lambda\,\lambda\,(0\ (2, (1\ 2)))\ \lambda\,0)$.

One can easily define application for the above de Bruijn encoding:

$(\lambda\, E\ E') \Rightarrow E[E'/0]$
$(E\ E')[E''/i] \Rightarrow (E[E''/i])\ (E'[E''/i])$ $(\lambda\, E)[E'/i] \Rightarrow \lambda\,(E[\uparrow E'/(i1 +_{Int})])$ $j[E/i] \Rightarrow$ if $j == i$ then E else (if $j > i$ then $j - 1$ else j fi) fi
$\uparrow E' \Rightarrow \uparrow^0 E$
$\uparrow^i (E\ E') \Rightarrow (\uparrow^i E)\ (\uparrow^i E')$ $\uparrow^i (\lambda\, E) \Rightarrow \lambda\,(\uparrow^{i+1} E)$ $\uparrow^i j \Rightarrow$ if $j >= i$ then $(j + 1)$ else j fi

Exercise 188. *Define the transformation above formally in Maude.*

Exercise 189. *Define application for the first de Bruijn encoding, in a similar style to the one above.*

4.5.11 Notes

4.5.12 Exercises

Exercise 190. ★ *Define the syntax of λ-calculus in a* `Maude` *module using mix-fix notation. Then parse the various λ-expressions that were discussed in this section.*

Exercise 191. ★ *Extend the Maude definition of λ-calculus in Exercise 190 with a definition of free variables. You should define an operation* `fv` *taking an expression and returning a set of variables.*

Exercise 192. *Prove the following equivalences of λ-expressions:*

- $\lambda x.x \equiv_\alpha \lambda y.y$,
- $\lambda x.x(\lambda y.y) \equiv_\alpha \lambda y.y(\lambda x.x)$,
- $\lambda x.x(\lambda y.y) \equiv_\alpha \lambda y.y(\lambda y.y)$.

Exercise 193. *Show that* $(\lambda x.(\lambda y.x))yx \equiv_\beta y$.

Exercise 194. ★ *Define substitution in Maude, as a partial operation. As discussed in Section 4.5.3, a partial substitution applies only if it does not lead to a variable capture; otherwise the substitution operation is undefined. In other words, when applying a partial substitution we do not need to perform any α-conversions.*
Hint. *Define $(\lambda y.E)[E'/x]$ when $y \neq x$ as a conditional equation, with condition $y \notin FV(E')$.*

Exercise 195. ★ *Define unrestricted substitution in Maude, which takes into account and avoids variable capture. Use the core-level Maude capabilities for this exercise (see also Exercise 196).*
Hint. *You need a mechanism to generate fresh variables. For example, you may define an operation* `var` *from Int to Var, where* `var`(1), `var`(2), *etc., represent fresh variables. A simple but wasteful solution when calculating $(\lambda y.E)[E'/x]$ is to always replace y in $\lambda y.E$ by a fresh variable before applying the substitution; a better solution is to only do it when $y \in FV(E')$.*

Exercise⋆ 196. ★ *Define unrestricted substitution in Maude, using the meta-level capabilities of Maude (see also Exercise 195). You should traverse the meta-term and propagate the substitution operation through any operation label which is not a λ-binding. For λ-bindings you should apply the same conceptual operation as in Exercise 195. The advantage of the meta-level approach is that it is more modular that the other approaches. We will be able to add new constructs to λ-calculus, for example builtins, without having to change the definition of the substitution.*

Exercise 197. ★ *Define λ-calculus formally in Maude, using each of the substitution approaches in Exercises 194, 195, and 196. For the former approach (partial substitution), show that there are λ-expressions that cannot be β-reduced automatically with your definition of λ-calculus to a normal form, even though they are closed (or combinators) and all the binding variables are initially distinct from each other.*

Exercise 198. *Define the other Boolean operations which were not already defined in Section 4.5.6 (including at least* `or`, `not`, `implies`, `iff`, *and* `xor`*) as λ-expressions.*

Exercise 199. *Prove that the Church Boolean operators defined in Exercise 198 have all their desired properties (a set of such desired properties can be found in the Maude BOOL module; use the command "*`show module BOOL`*" lists them).*

Exercise 200. *With the notions and notations in Section 4.5.6, show that for any Church numeral n_λ, both* `succ` *n_λ and* `succ'` *n_λ represent the same numeral, namely $(n+1)_\lambda$.*
Hint. *Induction on the structure of n_λ.*

Exercise 201. *Prove that Church numerals form indeed a model of natural numbers, by showing the two properties derived from Peano's axioms (see Section 4.5.6).*

Exercise 202. *Define multiplication on Church numerals and prove its Peano properties.*
Hint. *Multiplication can be defined several different interesting ways.*

Exercise 203. *Define the power operator (raising a number to the power of another) using Peano-like axioms. Then define power on Church numerals and show that it satisfies its Peano axioms.*

Exercise 204. *Add builtin integers to λ-calculus in Maude, extending Exercise 197 based on substitution as defined in Exercise 196.*
Hint. *The only change needed should be to subsort* `Int` *to* `Exp`*. Since Maude's parsing is at the level of kinds, instead of sorts, we should not need to overload the* `Int` *operations to take and return* `Exp` *sorts.*

4.6 Simply-Typed Lambda Calculs

We now discuss a non-trivial extension of λ-calculus with *types*. The idea is that each variable binding is assigned a type, which will allow one to calculate a unique type for each well-formed λ-expression or λ-term.

As we know from our experience with programming languages, the addition of types will allow one to reject programs that are not well-typed, with the intuition that those programs are most likely wrong (with respect to what the programmer meant).

Typing comes at a price: sometimes correct programs are rejected. One can, of course, argue that those programs are not correct (by definition, because they do not type). All in all, practice has shown that typing is overall useful in programming languages. Simply typed λ-calculus is perhaps the simplest typed language.

4.6.1 Syntax

The BNF syntax of simply-typed λ-calculus is

$$Var ::= x \mid y \mid \ldots$$

$$Type ::= \bullet \mid Type \to Type$$

$$Exp ::= Var \mid \lambda Var{:}Type.Exp \mid ExpExp.$$

To keep the presentation simple, for the time being we assume only one constant type, $\bullet$, and only one type constructor, $\to$. Thus $(\bullet \to \bullet) \to (\bullet \to (\bullet \to \bullet))$ is a well-formed type. To simplify writing, we assume that $\to$ is *right-associative*; the type above can then be written $(\bullet \to \bullet) \to \bullet \to \bullet \to \bullet$. As in the case of untyped λ-calculus, the λ-application is still assumed to be *left-associative*.

Exercise 205. *Define the syntax above in a Maude module, using the alternative mix-fix notation.*

4.6.2 Terms and Typye Checking

Using the syntax above, one can naturally generate simply-typed terms or program, such as, for example, $\lambda x{:}\bullet.\lambda f{:}\bullet \to \bullet.fx$. The intuition for this λ-abstraction is that it takes some λ-expressions of types $\bullet$ and $\bullet \to \bullet$, respectively, and applies the latter on the former. Naturally, the type of the result is expected to be $\bullet$.

Unlike in the case of untyped λ-calculus, the BNF (or, equivalently, the mix-fix notation) is not powerful enough to express all the intended well-formed, or better say well-typed, terms. Indeed, for example the term $\lambda x{:}\bullet.xx$ does not make any sense, because x, in order to be applied on an expression of type $\bullet$, in particular on itself, must have the type $\bullet \to s$ for some type s; however, x is declared of type $\bullet$. Moreover, one can formally show that $\lambda x{:}s.xx$ is not well-formed for any type s.

Even more, it can be shown that there is *no context free grammar* (CFG) whose language consists of all well-typed λ-expressions. This is perhaps the simplest language supporting the folklore claim that "well-formed programs do not form a context-free language".

Now the natural question is how to characterize the well-formed simply-typed λ-expressions. There are three equivalent approaches to do this, all being easily adaptable to other typed frameworks. Let us first introduce some important notation.

A *type assignment* is a finite set $X = \{x_1 : s_1, \ldots, x_n : s_n\}$ of pairs $x{:}s$, where x is a variable and s a type, with the property that each variable occurs at most once in X: it is not possible to have $x{:}s_1, x{:}s_2 \in X$ for

different types s_1, s_2. One of the main reasons for this limitation is that well-formed λ-expressions, including those which are just variables, are desired to have unique types. Then if x occurs in X, written (admittedly ambiguously) $x \in X$, we may let $X(x)$ denote the type s such that $x : s \in X$. Often the curly brackets "{" and "}" are omitted from the notation of X and its elements are permuted conveniently; thus, if $x \notin X$, then $X, x : s$ is a type assignment containing $x : s$. We let *TypeAssignment* denote the set of type assignments.

For the time being, let us introduce the notation $X \triangleright E : t$, typically called a *type judgment*, or a *well-typed λ-expression* or *λ-term* (sometimes "well-typed" may be dropped if understood), with the intuition that under the type assignment X, the λ-expression E is well-typed and has the type t. For example, one can write $x : \bullet, f : \bullet \to \bullet \triangleright f x : \bullet$. We will shortly see three different ways to define this intuition precisely.

Alternative notations for $X \triangleright E : t$ could be $(\forall X)E : t$ or $(E : t).X$, or just $E : t$ when X is understood from context, or even simply E if both X and t are understood. Let us next discuss the three different (but related) formal approaches to define this.

1. Proof system

We can define e *proof system* that can derive precisely the well-typed λ-expressions. The following three rules do this:

$$X, x : s \triangleright x : s \qquad \text{for any type assignment } X, x : s;$$

$$\frac{X, x : s \triangleright E : t}{X \triangleright \lambda x : s.E : s \to t} \qquad \text{for any type assignment } X, x : s, \text{ any } Exp\text{-term } E, \text{ and any type } t;$$

$$\frac{X \triangleright E : s \to t \qquad X \triangleright E' : s}{X \triangleright EE' : t} \qquad \text{for any type assignment } X, Exp\text{-terms } E, E', \text{ and types } s, t.$$

As usual, $X \triangleright E : t$ is called *derivable* if there is some sequence $X_1 \triangleright E_1 : t_1, \ldots, X_n \triangleright E_n : t_n$ such that $X_n \triangleright E_n : t_n$ is $X \triangleright E : t$ and each $X_i \triangleright E_i : t_i$ follow by one of the three rules above from previously ($< i$) derived well-typed terms. We may write $\vdash X \triangleright E : t$, or sometimes even just $X \triangleright E : t$, whenever $X \triangleright E : t$ is derivable.

Exercise 206. *Derive $x : \bullet \triangleright \lambda f : \bullet \to \bullet . f x : \bullet$. Also, derive $\emptyset \triangleright \lambda x : \bullet . \lambda f : \bullet \to \bullet \to \bullet \to \bullet . f x x : \bullet \to \bullet$*

When X is empty, we write $E : t$ instead of $\emptyset \triangleright E : t$. We are not going to (re)define the notions of *free variable*, with the corresponding operator FV, and *substitution*. They have precisely the same meaning as in untyped λ-calculus.

Properties about well-typed λ-expressions are typically proved by induction on the length of derivation.

Proposition 14. *The following hold:*

- *If $X \triangleright E : t$ then $FV(E) \subseteq X$;*
- *If $X \triangleright \lambda x : s.E : s \to t$ then $X, y : s \triangleright (E[y/x]) : t$ for any $y \notin X$;*
- *If $X \triangleright E : s$ and $X \triangleright E : t$ then $s = t$;*
- *If $X, x : s \triangleright E : t$ and $X \triangleright E' : s$ then $X \triangleright E[E'/x] : t$;*
- *If X, X' are type assignments and E is a λ-expression such that for all $x \in FV(E)$, $x : s \in X$ iff $x : s \in X'$, then $X \triangleright E : t$ iff $X \triangleright E' : t$;*
- *$\lambda x : s.xx$ does not type.*

2. Typing Algorithm

We can also define a relatively trivial *typing algorithm* that takes a type assignment X together with a λ-expression E, and tries to calculate a type t for E. The algorithm traverses E recursively:

Algorithm $\mathcal{A}(X, E)$

- if E is x and $x : s \in X$ then return s;
- if E is $\lambda\, x : s.E'$ and $x \notin X$ and $\mathcal{A}((X, x : s), E')$ returns t then return $t \to s$;
- if E is $E_1 E_2$ and $\mathcal{A}(X, E_1)$ returns $s \to t$ and $\mathcal{A}(X, E_2)$ returns s then return t;
- otherwise return error

Exercise 207. *Prove that $X \triangleright E : t$ is derivable if and only if $\mathcal{A}(X, E)$ returns t. (Hint: By structural induction on E.)*

3. Sets of Terms

Let us next give another characterization of the well-typed λ-terms. We define the family of sets $\{\mathcal{T}_t(X)\}_{X \in TypeAssignment, t \in Type}$ as the (componentwise) smallest set $\{W_{X,t}\}_{X \in TypeAssignment, t \in Type}$ of words in the (CF) language of λ-calculus such that:

- $x \in W_{X,s}$ if $x : s \in X$
- $\lambda\, x : s.E \in W_{X, s \to t}$ if $E \in W_{(X, x:s), t}$
- $E_1 E_2 \in W_{X,t}$ if $E_1 \in W_{X, s \to t}$ and $E_2 \in W_{X,s}$ for some $s \in Type$.

Exercise 208. *Prove that $X \triangleright E : t$ is derivable iff $E \in \mathcal{T}_t(X)$.*
(Hint: By structural induction on E.)

4.6.3 Equational Rules

We have discussed so far techniques to check that λ-expressions are well-typed. From now on we assume that all the λ-expressions that occur in any context are well-typed. More precisely, whenever we write $X \triangleright E : t$, we assume that E is well-typed under the type assignment X and that it has the type t. We now focus on equational properties of simply-typed λ-calculus. These equations play a dual role: on the one hand they give means to show programs equivalent, while on the other hand underlay the infrastructure necessary to define a canonical model of λ-calculus.

An *equation* is a 4-tuple consisting of a type assignment X, two λ-expressions E and E', and a type t, such that $X \triangleright E : t$ and $X \triangleright E' : t$. To simplify notation, we write such equations as $(\forall X)\ E =_t E'$, with the intuition that for any interpretation of the variables in X (i.e., any assignment of values of corresponding type to variables in X), the expressions E and E' evaluate to the same value, which has the expected type t.

A set of equations $\mathcal{E}$ is also called an *equational theory* (in λ-calculus). Given an equational theory $\mathcal{E}$ and an equation e, we call the syntactic construct $\mathcal{E} \vdash e$ an *equational judgment*. We next give a set of derivation rules for equational judgments:

$$\text{(axiom)} \qquad \mathcal{E} \vdash (\forall X)\ E =_t E' \quad \text{if } (\forall X)\ E =_t E' \text{ is in } \mathcal{E}$$

$$\text{(add)} \qquad \frac{\mathcal{E} \vdash (\forall X)\ E =_t E'}{\mathcal{E} \vdash (\forall X, x : s)\ E =_t E'} \quad \text{if } x \notin X$$

$$\text{(reflexivity)} \qquad \mathcal{E} \vdash (\forall X)\ E =_t E \quad \text{if } X \triangleright E : t$$

$$\text{(symmetry)} \qquad \frac{\mathcal{E} \vdash (\forall X)\ E =_t E'}{\mathcal{E} \vdash (\forall X)\ E' =_t E}$$

$$\text{(transitivity)} \qquad \frac{\mathcal{E} \vdash (\forall X)\ E =_t E' \quad \mathcal{E} \vdash (\forall X)\ E' =_t E''}{\mathcal{E} \vdash (\forall X)\ E =_t E''}$$

$$\text{(application)} \qquad \frac{\mathcal{E} \vdash (\forall X)\ E_1 =_{s \to t} E_1' \qquad \mathcal{E} \vdash (\forall X)\ E_2 =_s E_2'}{\mathcal{E} \vdash (\forall X)\ E_1 E_2 =_t E_1' E_2'}$$

$$(\xi) \qquad \frac{\mathcal{E} \vdash (\forall X, x : s)\ E =_t E'}{\mathcal{E} \vdash (\forall X)\ \lambda x : s.E =_{s \to t} \lambda x : s.E'}$$

$$(\beta) \qquad \mathcal{E} \vdash (\forall X)\ (\lambda x : s.E)E' =_t E[E'/x] \quad \begin{array}{l} \text{if } X, x : s \triangleright E : t \text{ and} \\ X \triangleright E' : s \end{array}$$

$$(\eta) \qquad \mathcal{E} \vdash (\forall X)\ \lambda x : s.Ex =_{s \to t} E \quad \text{if } x \notin FV(E)$$

The rule (axiom) says that any equation already existing in $\mathcal{E}$ is derivable from $\mathcal{E}$. The rule (add) allows one to add typed variables to the type assignment of a derived equation; this is necessary for several technical reasons, such as, for example, to bring the two equations to the same type assignment in order to apply the (transitivity) or the (application) rule. The next four rules, (reflexivity), (symmetry), (transitivity), and (application), are self explanatory and are instances of general equational reasoning in algebraic specifications to the signature of λ-calculus; in particular, (application) is an instance of the congruence deduction rule. The rule (ξ) is almost an instance of the equational congruence rule to the λ-abstraction construct; note, however, that the type assignment needs to be changed appropriately. The last two rules, (β) and (η) are nothing but the equational rule and typed versions of the (β) and (η) equations from untyped λ-calculus.

Proposition 15. *If $\mathcal{E} \vdash (\forall X)\ E =_t E'$ then $X \triangleright E : t$ and $X \triangleright E' : t$.*

Proposition 16. *If E, E' are two λ-expressions and $x \notin X$ then*
$\mathcal{E} \vdash (\forall X)\ E =_t E'$ iff $\mathcal{E} \vdash (\forall X, x : s)\ E =_t E'$.

Corollary 8. *If E, E' are λ-expressions and X, X' are type assignments such that $x : s \in X$ iff $x : s \in X'$ for any $x \in FV(EE')$, then $\mathcal{E} \vdash (\forall X)\ E =_t E'$ iff $\mathcal{E} \vdash (\forall X')\ E =_t E'$.*

Proposition 17. *If $\mathcal{E} \vdash (\forall X)\ E =_s E'$ and $Y, y : s \triangleright F : t$ such that $X \cup Y$ is a proper type assignment, then $\mathcal{E} \vdash (\forall X \cup Y)\ F[E/y] =_t F[E'/y]$.*
(Proof hint: Eliminate the substitution by applying the rule (β) twice backwards.)

Comment. In the next iteration of the class, allow well-typed terms of the form $\lambda x : s.\lambda x : t.E$ and define λ-calculus as an instance of equational reasoning (including substitution) enriched with just (β) and (η).

4.6.4 Models

1. A *Type-indexed set* $M = \{M_t\}_{t \in Type}$ is an infinite collection of sets, one for each type; there is no relationship required among the sets M_s, M_t, and $M_{s \to t}$ for any types s and t. Note that type assignments can be regarded as *Type*-indexed sets with only a finite number of sets non-empty. For example, the type assignment $\{x : s, y : s, z : t\}$ can be regarded as the *Type*-indexed set whose s-component is $\{x, y\}$, whose t-component is $\{z\}$, and whose other components are all empty.

For a given $X \in TypeAssignment$, we let $\mathcal{T}(X)$ denote the *Type*-indexed set $\{\mathcal{T}_t(X)\}_{t \in Type}$.

2. Given *Type*-indexed sets $M = \{M_t\}_{t \in Type}$ and $N = \{N_t\}_{t \in Type}$, a *Type-indexed function* $h : M \to N$ is a collection of functions $\{h_t : M_t \to N_t\}_{t \in Type}$ defined on the corresponding components of the *Type*-indexed sets, one for each type t. If X is a type assignment and M is a *Type*-indexed set, then we call the *Type*-indexed functions $\rho : X \to M$ *M-environments*. As usual, we let $[X \to M]$ denote the set of all M-environments over the assignment X. If $x : s \in X$ and $v \in M_s$, then we let $\rho[v/x] : X \to M$ denote the M-environment ρ' with $\rho'(y) = \rho(y)$ for all $y \neq x$ and $\rho'(x) = v$.

A *pre-frame* or *pre-model* is a pair $(\{M_t\}_{t \in Type}, \{M^{s,t} : M_{s \to t} \times M_s \to M_t\}_{s,t \in Type})$ consisting of a *Type*-indexed set and a (*Type*×*Type*)-indexed collection of functions, such that $M^{s,t}$ is *extensional* for any s, t: for any $f, g \in M_{s \to t}$, if $M^{s,t}(f, v) = M^{s,t}(g, v)$ for all $v \in M_s$, then $f = g$.

A pre-frame or pre-model $\mathcal{M} = (\{M_t\}_{t \in Type}, \{M^{s,t} : M_{s \to t} \times M_s \to M_t\}_{s,t \in Type})$ is called a *frame* or *model* of simply-typed λ-calculus iff there is a *Type*-indexed mapping, say $M_{_}$, taking well-typed λ-expressions $X \triangleright E : t$ to mappings $M_{X \triangleright E:t} : [X \to M] \to M_t$ with the following properties for any M-environment $\rho : X \to M$:

1) $M_{X \triangleright x:s}(\rho) = \rho(x : s) \in M_s$;

2) $M^{s,t}(M_{X \triangleright \lambda x:s.E:s \to t}(\rho), v) = M_{X,x:s \triangleright E:t}(\rho[v/x])$ for any $v \in M_s$;

3) $M_{X \triangleright E_1 E_2:t}(\rho) = M^{s,t}(M_{X \triangleright E_1:s \to t}(\rho), M_{X \triangleright E_2:s}(\rho))$.

When such a mapping exists, we say, by a slight language abuse, that the model $\mathcal{M}$ *extends* the pre-model $(\{M_t\}_{t \in Type}, \{M^{s,t} : M_{s \to t} \times M_s \to M_t\}_{s,t \in Type})$.

Exercise 209. *Show that there is at most one extension of any pre-model to a model.*
(Hint: by induction on λ-expressions, using extensionality.)

Therefore, if a pre-frame can be extended to a frame, than that extension is unique. Given a model $\mathcal{M}$ and an M-environment $\rho : X \to M$, we let $\rho^{\#} : \mathcal{T}(X) \to M$ denote the *Type*-indexed map defined as $\rho^{\#}(X \triangleright E : t) = M_{X \triangleright E:t}(\rho)$.

Definition 24. *A model $\mathcal{M}$ satisfies an equation $(\forall X)\ E =_t E'$, written $\mathcal{M} \models (\forall X)\ E =_t E'$, iff $\rho^{\#}(X \triangleright E : t) = \rho^{\#}(X \triangleright E' : t)$ for any $\rho : X \to M$.*

Given a set of equations $\mathcal{E}$ and an equation e, we extend our satisfaction relation to $\mathcal{M} \models \mathcal{E}$ iff $\mathcal{M}$ satisfies all equations in $\mathcal{E}$, and $\mathcal{E} \models e$ iff for any model $\mathcal{M}$, if $\mathcal{M} \models \mathcal{E}$ then $\mathcal{M} \models e$.

Theorem 27. *(Soundness) If $\mathcal{E} \vdash e$ then $\mathcal{E} \models e$.*

Proof Sketch. By induction on the length of the derivation. All one needs to prove is that each derivation rule is sound. For example, in the case of the (ξ) rule, we should show that if $\mathcal{E} \models (\forall X, x : s)\ E =_t E'$ then $\mathcal{E} \models (\forall X)\ \lambda x : s.E =_{s \to t} \lambda x : s.E'$. Let $\mathcal{M}$ be a model such that $\mathcal{M} \models \mathcal{E}$, and let $\rho : X \to M$ be an M-environment. Then note that

$$\begin{aligned} M^{s,t}(\rho^{\#}(X \triangleright \lambda x : s.E : s \to t), v) &= M_{X,x:s \triangleright E:t}(\rho[v/x]) \\ &= M_{X,x:s \triangleright E':t}(\rho[v/x]) \\ &= M^{s,t}(\rho^{\#}(X \triangleright \lambda x : s.E' : s \to t), v). \end{aligned}$$

Then by extensionality we get

$$\rho^{\#}(X \triangleright \lambda x{:}s.E{:}s \to t) = \rho^{\#}(X \triangleright \lambda x{:}s.E'{:}s \to t).$$

Exercise 210. *Show that all the equational inference rules of simply-typed λ-calculus are sound.*

4.6.5 Full Type Frame

We next define a special (but very important) type frame, or model, of λ-calculus, called the *full type frame*. It consists of the most intuitive interpretation of λ-expressions, namely as values and functions.

Let us fix a set T and let us define inductively the following set $\mathcal{HO}$ of sets of high-order functions starting with T:

- $T \in \mathcal{HO}$;
- $[A \to B] \in \mathcal{HO}$ whenever $A, B \in \mathcal{HO}$.

Recall that for any two sets A and B, $[A \to B]$ is the *set* of all functions of domain A and codomain B. In other words, $\mathcal{HO}$ is defined as the smallest set of sets that is closed under the operations above; that is, it contains T and whenever it contains the sets A, B, it also contains the set of functions between them.

We can now define a unique function $[\![_]\!] : \mathit{Type} \to \mathcal{HO}$ with the property that $[\![\bullet]\!] = T$ and $[\![s \to t]\!] = [[\![s]\!] \to [\![t]\!]]$ for any $s, t \in \mathit{Type}$. Note that this function actually organizes $\mathcal{HO}$ as a *Type*-indexed set: $\mathcal{HO} = \{[\![t]\!]\}_{t \in \mathit{Type}}$.

From now on we regard $\mathcal{HO}$ as a *Type*-indexed set and organize it into a model of simply-typed λ-calculus. To make it a pre-model, let us define $\mathcal{HO}^{s,t} : [\![s \to t]\!] \times [\![s]\!] \to [\![t]\!]$ as expected: $\mathcal{HO}^{s,t}(f, x) = f(x)$ for any $s, t \in \mathit{Type}$ and any $f : [\![s]\!] \to [\![t]\!]$ and $x \in [\![s]\!]$; note that x can be itself a function if s is a function type. One can immediately see that $\mathcal{HO}^{s,t}$ are extensional: indeed, if $f(x) = g(x)$ for any x then $f = g$ (by the definition of function equality). Therefore, $\mathcal{HO}$ is a pre-model.

To make $\mathcal{HO}$ a model, we need to define appropriate interpretations of well-typed λ-expressions. For simplicity, we use the same notation $[\![_]\!]$ as for the interpretation of types. For a given $X \triangleright E{:}t$, we define $[\![X \triangleright E{:}t]\!] : [X \to \mathcal{HO}] \to [\![t]\!]$ by induction as follows:

- $[\![X, x{:}s \triangleright x{:}s]\!](\rho) \stackrel{def}{=} \rho(x{:}s) \in [\![s]\!]$ for any $\mathcal{HO}$-environment $\rho : X \to \mathcal{HO}$;
- $[\![X \triangleright \lambda x{:}s.E{:}s \to t]\!](\rho)(v) \stackrel{def}{=} [\![X, x{:}s \triangleright E{:}t]\!](\rho[v/x])$ for any $\rho : X \to \mathcal{HO}$ and $v \in [\![s]\!]$;
- $[\![X \triangleright EE'{:}t]\!] \stackrel{def}{=} ([\![X \triangleright E{:}s \to t]\!](\rho))([\![X \triangleright EE'{:}s]\!](\rho))$ for any $\mathcal{HO}$-environment ρ.

Exercise 211. *Prove that $\mathcal{HO}$ defined above is a model of λ-calculus.*

$\mathcal{HO}$ is perhaps the most natural model of simply-typed λ-calculus, in which types are interpreted as sets of their corresponding values, λ-abstractions as functions on appropriate domains and co-domains, and λ-applications as function applications.

Term model

Let us now fix a *Type*-indexed set $\mathcal{X} = \{\mathcal{X}_t\}_{t \in Type}$ such that $\mathcal{X}_t$ is *infinite* for any $t \in Type$ and $\mathcal{X}_s \cap \mathcal{X}_t = \emptyset$ for $s, t \in Type$. From now on we consider only (well-typed) λ-experiments over variables in $\mathcal{X}$, i.e., of the form $X \triangleright E : t$ with $X \subseteq \mathcal{X}$.

Technically speaking, since $\mathcal{X}$ is a partition of $\bigcup_{t \in Type} \mathcal{X}_t$, each variable is now tagged automatically with its type. This means that one can simply ignore the type assignment X when writing well-typed terms $X \triangleright E : t$. However, for uniformity in notation, we prefer to keep the assignments in the notation of terms; we can think of them as the variables over which the corresponding λ-expression was *intended* to be defined. For example, the right-hand-side in the equation $(\forall a : s, b : s)\ (\lambda x : s.\lambda y : s.x)ab =_s a$ was intended to be $(a : s, b : s) \triangleright a : s$ in order for the equation to make sense, even though $b : s$ is not necessary in the type assignment.

Given a set of equations $\mathcal{E}$, we define the *$\mathcal{E}$-equivalence class* of the a λ-expression $X \triangleright E : t$ as the set

$$[X \triangleright E : t]_{\mathcal{E}} \stackrel{def}{=} \{X' \triangleright E' : t \mid \text{there is some } Y \text{ such that } \mathcal{E} \vdash (\forall Y)\ E =_t E'\}.$$

Proposition 18. $[X \triangleright E : t]_{\mathcal{E}} = [X' \triangleright E' : t]_{\mathcal{E}}$ *iff there is some Y such that $\mathcal{E} \vdash (\forall Y)\ E =_t E'$.*

We can now define a *Type*-indexed set $\mathcal{T}_{\mathcal{E}} = \{\mathcal{T}_{\mathcal{E},t}\}_{t \in Type}$, by letting $\mathcal{T}_{\mathcal{E},t}$ be the set $\{[X \triangleright E : t]_{\mathcal{E}} \mid X \subseteq \mathcal{X}\}$ for any $t \in Type$.

Further, we can extend $\mathcal{T}_{\mathcal{E}}$ to a pre-model, by defining functions $\mathcal{T}_{\mathcal{E}}^{s,t} : \mathcal{T}_{\mathcal{E},s \to t} \times \mathcal{T}_{\mathcal{E},s} \to \mathcal{T}_{\mathcal{E},t}$ for any types $s, t \in Type$ as follows:

$$\mathcal{T}_{\mathcal{E}}^{s,t}([X \triangleright E : s \to t]_{\mathcal{E}}, [Y \triangleright F : s]_{\mathcal{E}}) \stackrel{def}{=} [X \cup Y \triangleright EF : t]_{\mathcal{E}}\ .$$

Proposition 19. *$\mathcal{T}_{\mathcal{E}}$ is a pre-model.*

Proof. All we need to show is that $\mathcal{T}_{\mathcal{E}}^{s,t}$ is well-defined and extensional.

For well-definedness, we need to prove that if $[X' \triangleright E' : s \to t]_{\mathcal{E}} = [X \triangleright E : s \to t]_{\mathcal{E}}$ and $[Y' \triangleright F' : s]_{\mathcal{E}} = [Y \triangleright F : s]_{\mathcal{E}}$ then $[X' \cup Y' \triangleright E'F' : t]_{\mathcal{E}} = [X \cup Y \triangleright EF : t]_{\mathcal{E}}$. Since there are some $\bar{X}$ and $\bar{Y}$ such that $\mathcal{E} \vdash (\forall \bar{X})\ E =_{s \to t} E'$ and $\mathcal{E} \vdash (\forall \bar{Y})\ F =_s F'$, by using the rule (add) a finite number of times we can derive $\mathcal{E} \vdash (\forall \bar{X} \cup \bar{Y})\ E =_{s \to t} E'$ and $\mathcal{E} \vdash (\forall \bar{X} \cup \bar{Y})\ F =_s F'$; then by (application) we can derive $\mathcal{E} \vdash (\forall \bar{X} \cup \bar{Y})\ EF =_t E'F'$. By Proposition 18, it follows that $[X' \cup Y' \triangleright E'F' : t]_{\mathcal{E}} = [X \cup Y \triangleright EF : t]_{\mathcal{E}}$.

For extensionality, we need to show that given $X \triangleright E : s \to t$ and $X' \triangleright E' : s \to t$ such that $[X \cup Y \triangleright EF : t]_{\mathcal{E}} = [X' \cup Y \triangleright E'F : t]_{\mathcal{E}}$ for any $Y \triangleright F : s$, it is indeed the case that $[X \triangleright E : t]_{\mathcal{E}} = [X' \triangleright E' : t]_{\mathcal{E}}$. Let us pick $Y = \{y : s\} \nsubseteq X \cup X'$ and $F = y$. Then $[X, y : s \triangleright Ey : t]_{\mathcal{E}} = [X', y : s \triangleright E'y : t]_{\mathcal{E}}$, so by Proposition 18, $\mathcal{E} \vdash (\forall Z)\ Ey =_t E'y$ for some $Z \subseteq \mathcal{X}$. Note that, in order for $Z \triangleright Ey : t$ and $Z \triangleright E'y : t$ to be well-typed, Z must contain the variable $y : s$. Let Z be $W, y : s$. By rule (ξ) we then derive $\mathcal{E} \vdash (\forall W)\ \lambda y : s.Ey =_{s \to t} \lambda y : s.E'y$. Finally, by applying the rule (η) twice we can derive $\mathcal{E} \vdash (\forall W)\ E =_{s \to t} E'$, which concludes our proof that $[X \triangleright E : t]_{\mathcal{E}} = [X' \triangleright E' : t]_{\mathcal{E}}$. Therefore, $\mathcal{T}_{\mathcal{E}}$ is a pre-model. □

Our goal next is to organize $\mathcal{T}_{\mathcal{E}}$ as a model. To do it, we first need to define mappings $\mathcal{T}_{\mathcal{E},X \triangleright E:t} : [X \to \mathcal{T}_{\mathcal{E}}] \to \mathcal{T}_{\mathcal{E},t}$ for all $X \triangleright E : t$. Note that $\mathcal{T}_{\mathcal{E}}$-environments map variables to $\mathcal{E}$-equivalence classes of λ-expressions. If $X = \{x_1 : s_1, \ldots x_n : s_n\}$ and $\rho : X \to \mathcal{T}_{\mathcal{E}}$ is a $\mathcal{T}_{\mathcal{E}}$-environment taking x_i to, say $[X_i \triangleright E_i : s_i]_{\mathcal{E}}$, then we let $\mathcal{T}_{\mathcal{E},X \triangleright E:t}(\rho)$ be defined as $[\bigcup_{i=1}^n X_i \triangleright E[E_1, \ldots, E_n/x_1, \ldots, x_n]]_{\mathcal{E}}$, where $E[E_1, \ldots, E_n/x_1, \ldots, x_n]$ is the term obtained by substituting $E_1, \ldots, E_n$ for $x_1, \ldots, x_n$ in E *in parallel*. One way to achieve this is to choose some fresh variables $z_1 : s_1, \ldots, z_n : s_n$ in $\mathcal{X} \setminus (X \cup \bigcup_{i=1}^n X_i)$ and to let $E[E_1, \ldots, E_n/x_1, \ldots, x_n]$ be defined as $E[z_1/x_1] \ldots [z_n/x_n][E_1/z_1] \ldots [E_n/z_n]$.

Exercise 212. *Why would it not be correct to define parallel substitution as $E[E_1/x_1]\ldots[E_n/x_n]$?*

Propositions 18 and 17 tell us that the maps $\mathcal{T}_{\mathcal{E},X \triangleright E:t} : [X \to \mathcal{T}_{\mathcal{E}}] \to \mathcal{T}_{\mathcal{E},t}$ are indeed well defined.

Proposition 20. *$\mathcal{T}_{\mathcal{E}}$ is a model of simply-typed λ-calculus.*

Proof. We need to prove that the three conditions in the definition of a model hold.

1) Let $X, x : s$ be a type assignment and let $\rho : X, x : s \to \mathcal{T}_{\mathcal{E}}$ be a $\mathcal{T}_{\mathcal{E}}$-environment where $\rho(x_i : s_i) = [X_i \triangleright E_i : s_i]_{\mathcal{E}}$ for all $x_i : s_i \in X$ and $\rho(x : s) = [Y \triangleright F : s]$. Then $\mathcal{T}_{\mathcal{E},(X,x:s \triangleright x:s)}(\rho)$ is by definition $[Y \cup \bigcup_{i=1}^n X_i \triangleright F : s]_{\mathcal{E}}$, which is equal to $\rho(x : s)$ by Proposition 18, noting that $\mathcal{E} \vdash (\forall Y \cup \bigcup_{i=1}^n X_i)\ F =_s F$ by (reflexivity).

2) Let $X \triangleright \lambda x : s.E : s \to t$ be a well formed λ-expression, let $\rho : X \to \mathcal{T}_{\mathcal{E}}$ be a $\mathcal{T}_{\mathcal{E}}$-environment, and let $[Y \triangleright F : s]_{\mathcal{E}}$ be an $\mathcal{E}$-equivalence class in $\mathcal{T}_{\mathcal{E},s}$. We need to show that $\mathcal{T}_{\mathcal{E}}^{s,t}(\mathcal{T}_{\mathcal{E},(X \triangleright \lambda x:s.E:s\to t)}(\rho), [Y \triangleright F : s]_{\mathcal{E}}) = \mathcal{T}_{\mathcal{E},(X,x:s \triangleright E:t)}(\rho[x \leftarrow [Y \triangleright F : s]_{\mathcal{E}}])$.

If $\rho(x_i : s_i) = [X_i \triangleright E_i : s_i]_{\mathcal{E}}$ for each $x_i : s_i \in X$, then

$$\mathcal{T}_{\mathcal{E},(X \triangleright \lambda x:s.E:s\to t)}(\rho) = \rho^{\#}(X \triangleright \lambda x : s.E : s \to t) =$$

$$= [\bigcup_{i=1}^n X_i \triangleright \lambda x : s.E[E_1, \ldots, E_n/x_1, \ldots, x_n] : s \to t]_{\mathcal{E}} ,$$

so the left-hand-side of the equality becomes, after an application of (β), $[Y \cup \bigcup_{i=1}^n X_i \triangleright E[E_1, \ldots, E_n, F/x_1, \ldots, x_n, x] : t]_{\mathcal{E}}$, which is by definition equal to $[\mathcal{T}_{\mathcal{E},(X,x:s \triangleright E:t)}(\rho[[Y \triangleright F : s]_{\mathcal{E}}/x])]$.

3) Easy.

Exercise 213. *Prove 3) above, thus completing the proof that $\mathcal{T}_{\mathcal{E}}$ is a model.*

4.6.6 Completeness

We are now ready to prove one of the most important results of simply-typed λ-calculus, namely the completeness of the equational deduction rules. In other words, we show that the equational rules are sufficient to derive any equation that is true in all models of λ-calculus.

Let us first investigate the satisfaction in $\mathcal{T}_{\mathcal{E}}$. By definition, $\mathcal{T}_{\mathcal{E}} \models (\forall X)\ E =_t E'$ iff for any $\mathcal{T}_{\mathcal{E}}$-environment $\rho : X \to \mathcal{T}_{\mathcal{E}}$, it is the case that $\rho^{\#}(X \triangleright E : t) = \rho^{\#}(X \triangleright E' : t)$. If $\rho(x_i : s_i) = [X_i \triangleright E_i : s_i]_{\mathcal{E}}$ for any $x_i : s_i \in X$, then the above says that $[\bigcup_{i=1}^n X_i \triangleright E[E_1, \ldots, E_n/x_1, \ldots, x_n] : t]_{\mathcal{E}} = [\bigcup_{i=1}^n X_i \triangleright E'[E_1, \ldots, E_n/x_1, \ldots, x_n] : t]_{\mathcal{E}}$, or by Proposition 18 that there is some $Y \subseteq \mathcal{X}$ such that $\mathcal{E} \vdash (\forall Y)\ E[E_1, \ldots, E_n/x_1, \ldots, x_n] =_t E'[E_1, \ldots, E_n/x_1, \ldots, x_n]$. Taking ρ to be the identity $\mathcal{T}_{\mathcal{E}}$-environment, that is, $\rho(x_i : s_i) = [x_i : s_i \triangleright x_i : s_i]$, we obtain that $\mathcal{T}_{\mathcal{E}} \models (\forall X) E =_t E'$ implies $\mathcal{E} \vdash (\forall Y) E =_t E'$ for some $Y \subseteq \mathcal{X}$. By Proposition 16, we then get the following important result:

Proposition 21. *$\mathcal{T}_{\mathcal{E}} \models (\forall X)\ E =_t E'$ iff $\mathcal{E} \vdash (\forall X)\ E =_t E'$.*

Corollary 9. *$\mathcal{T}_{\mathcal{E}} \models \mathcal{E}$.*

Theorem 28. *(Completeness) If $\mathcal{E} \models e$ then $\mathcal{E} \vdash e$.*

Proof. If $\mathcal{E} \models e$ then by Corollary 9 we get that $\mathcal{T}_{\mathcal{E}} \models e$, so by Proposition 21 we obtain that $\mathcal{E} \vdash e$. □

4.6.7 Cartesian Closed Categories as Simply-Typed Lambda-Calculus Models

We next focus on showing that CCCs can be organized as models of simply-typed λ-calculus. The idea is to interpret types as objects and λ-expressions as morphisms. As expected, exponentiation is going to play a crucial role. Indeed, if A and B are objects types interpreting s and t, respectively, then B^A will interpret the type $s \to t$.

To make our categorical framework and modeling more interesting, in the sequel we consider a common extended variant of simply typed λ-calculus, namely one with a *set* of type constants (as opposed to just one type constant, $\circ$, like before), a product type constructor, and a set of typed λ-constants.

Formally, let S be a set of *type constants*, also possibly called *basic* or *primitive types*. The extended λ-calculus that we consider from here on has types built over the following grammar:

$$Type \quad ::= \quad S \quad | \quad Type \to Type \quad | \quad Type \times Type$$

As expected, types $s \times t$ stand for pairs of values, one of type s and the other of type t.

Let us also consider a set Σ of *typed λ-constants*, that is, pairs $\sigma : t$, where t is some type. Like for type assignments, we assume that Σ does not contain constants with multiple types and let $\Sigma(\sigma)$ denote the type t such that $\sigma : t \in \Sigma$. We also take the liberty to write $\sigma \in \Sigma$, without mentioning the type of σ. Then the formal CFG of (possibly not well-typed) λ-expressions is:

$$\begin{array}{rcl} Exp & ::= & Var \mid \Sigma \mid \lambda x{:}s.Exp \mid ExpExp \mid \\ & & \mid (Exp, Exp) \mid proj_1 Exp \mid proj_2 Exp \end{array}$$

The set of constant λ-expressions Σ is also called a *signature*. Besides the constants in the signature, our extended simply-typed λ-calculus also includes the pairing construct, as well as the two corresponding projections. The typing rules for the new constructs are as follows:

$$X \triangleright \sigma : s \qquad \text{for any } X \text{ and } \sigma{:}s \in \Sigma;$$

$$\frac{X \triangleright E_1{:}t_1 \qquad X \triangleright E_2{:}t_2}{X \triangleright (E_1, E_2){:}t_1 \times t_2} \qquad \text{for any } X, \text{ any } Exp\text{-terms } E_1, E_2, \text{ and any types } t_1, t_2;$$

$$\frac{X \triangleright E{:}t_1 \times t_2}{X \triangleright proj_1 E{:}t_1} \qquad \text{for any type assignment } X, Exp\text{-term } E, \text{ and types } t_1, t_2;$$

$$\frac{X \triangleright E{:}t_1 \times t_2}{X \triangleright proj_2 E{:}t_2} \qquad \text{for any type assignment } X, Exp\text{-term } E, \text{ and types } t_1, t_2.$$

Two equations need to be added, to capture the relationship between pairing and projections:

$$(\mathsf{Proj})_1 \quad \mathcal{E} \vdash (\forall X)\, proj_1(E_1, E_2) =_{t_1} E_1 \quad \text{if } X \triangleright E_1{:}t_1 \text{ and } X \triangleright E_2{:}t_2;$$

$$(\mathsf{Proj})_2 \quad \mathcal{E} \vdash (\forall X)\, proj_2(E_1, E_2) =_{t_2} E_2 \quad \text{if } X \triangleright E_1{:}t_1 \text{ and } X \triangleright E_2{:}t_2.$$

Exercise 214. *Show that it is not necessarily the case that $\mathcal{E} \vdash (\forall X)(proj_1 E, proj_2 E) =_{t_1 \times t_2} E$ whenever $X \triangleright E{:}t_1 \times t_2$. What if the types s associated to constants $\sigma : s$ in Σ are such that they contain no product type construct?*

Simply-Typed λ-Calculus
Captures Algebraic Specifications

The extension of simply-typed λ-calculus defined above captures algebraic signatures quite elegantly. Given a many-sorted algebraic signature (S, Σ), i.e., a set of sorts S and a set of operations $\sigma : s_1 \times \ldots s_n \to s$ (we define algebraic signatures in Maude, for example), all one needs to do is to declare the sorts in S as basic types and the operations in Σ as λ-constants, noting that $s_1 \times \ldots s_n$ are types that can be constructed in λ-calculus (assume right or left associativity for the product of types).

One can show that the simply-typed λ-calculus obtained this way is a *conservative extension* of the original equational theory, in the sense that it can derive precisely all the equational consequences of the equational theory.

(S, Σ)**-CCC**

The simply-typed λ-calculus defined above is therefore parameterized by the pair (S, Σ) of basic types and constant λ-expressions. We next show that any CCC providing interpretations for just the basic types in S and for the λ-constants in Σ can be extended to a model of (simply-typed) λ-calculus, providing interpretations for all the types and for all the λ-expressions. This is possible because the CCCs provide elegant corresponding categorical machinery to the syntactic constructs of types and λ-expressions.

An (S, Σ)-*CCC* is a CCC *Cxt* together with

- a mapping $[\![_]\!] : S \to |Cxt|$, associating some object $[\![s]\!] \in |Cxt|$ to any $s \in S$;
- some morphism $[\![\sigma]\!] : \star \to [\![t]\!]$ for any $\sigma : t \in \Sigma$, where $[\![_]\!] : Type \to |Cxt|$ extends the map above as follows:
  - $[\![s \to t]\!] \stackrel{def}{=} [\![t]\!]^{[\![s]\!]}$, and
  - $[\![s \times t]\!] \stackrel{def}{=} [\![s]\!] \times [\![t]\!]$.

Recall that $\star$ is the final object of *Cxt*.

We show that the operator $[\![_]\!] : \Sigma \to Cxt$ above can be extended to any λ-expressions. Our goal is therefore to define for each well-typed λ-expression $X \triangleright E : t$, where $X = x_1 : s_1, \ldots, x_n : s_n$, a morphism $[\![X \triangleright E : t]\!] : [\![s_1]\!] \times \ldots [\![s_n]\!] \to [\![t]\!]$ in *Cxt*.

Before that, let us first discuss how such morphisms would relate to the more natural interpretation of well-typed λ-expressions in Henkin models, namely as functions $M_{X \triangleright E:t} : [X \to M] \to M_t$ on *environments*. The key observation here is the set of M-environments $[X \to M]$ is in a one-to-one correspondence with the set $M_{s_1} \times \ldots \times M_{s_n}$. Indeed, an M-environment is nothing but a choice of an element in each of the sets $M_{s_1}, \ldots, M_{s_n}$, which is nothing but an element in $M_{s_1} \times \ldots \times M_{s_n}$. Therefore, we regard the functions $M_{X \triangleright Et}$ that appear in a Henkin model as functions $M_{s_1} \times \ldots \times M_{s_n} \to M_t$, which now look closely related to our morphisms $[\![X \triangleright E : t]\!] : [\![s_1]\!] \times \ldots \times [\![s_n]\!] \to [\![t]\!]$ defined next. We will actually see later on that there is a very tight relationship between the CCC models defined next and Henkin models.

Recall that $A \times B$ and $B \times A$ are isomorphic in *Cxt*. Therefore, the objects $[\![s_1]\!], \ldots, [\![s_n]\!]$ can be permuted in any order in the product $[\![s_1]\!], \ldots, [\![s_n]\!]$. However, in what follows, we prefer to work with type assignments as lists rather than sets; this way we can generate fixed interpretations rather than the possibly confusing up-to-isomorphism ones. To accomplish this, we need to slightly modify the inference rules to account for lists rather than sets. Fortunately, this is quite easy. We only need to replace each "$X, x : s$" by "$X_1, x : s, X_2$" correspondingly in the premise of rules, where X_1 and X_2 are any type assignments. For example, the rule (ξ)

becomes

$$\frac{\mathcal{E} \vdash (\forall X_1, x\!:\!s, X_2)\ E =_t E'}{\mathcal{E} \vdash (\forall X_1, X_2)\ \lambda\, x\!:\!s.E =_{s\to t} \lambda\, x\!:\!s.E'}$$

Now we can extend $[\![_]\!] : \mathit{Type} \to |\mathit{Cxt}|$ to type assignments, i.e., $[\![_]\!] : \mathit{TypeAssignment} \to |\mathit{Cxt}|$, as follows:

$$[\![x_1\!:\!s_1, \ldots, x_n\!:\!s_n]\!] \stackrel{def}{=} [\![s_1]\!] \times \ldots \times [\![s_n]\!]\ .$$

We are now ready to define the morphisms $[\![X \triangleright E\!:\!t]\!] : [\![X]\!] \to [\![t]\!]$ in Cxt, inductively on the structure of well-typed λ-expressions:

- $[\![X_1, x_i\!:\!s_i, X_2 \triangleright x_i\!:\!s_i]\!] \stackrel{def}{=} \pi_i$, the i-th projection given by the definition of the product, i.e., $\pi_i : [\![X_1]\!] \times [\![s_i]\!] \times [\![X_2]\!] \to [\![s_i]\!]$;

- $[\![X \triangleright \sigma\!:\!t]\!] \stackrel{def}{=} !_{[\![X]\!]}; [\![\sigma]\!]$ for any $\sigma\!:\!t \in \Sigma$, where $!_{[\![X]\!]} : [\![X]\!] \to \star$ is the unique morphism from $[\![X]\!]$ to the final object $\star$, and $[\![\sigma]\!] : \star \to [\![t]\!]$ is the interpretation of σ given as part of the definition of the (S, Σ)-CCC Cxt. Intuitively, the morphism $[\![X \triangleright \sigma\!:\!t]\!]$ forget its input and recalls the built in σ;

- $[\![X \triangleright \lambda\, x\!:\!s.E\!:\!s \to t]\!] \stackrel{def}{=} \mathit{curry}([\![X, x\!:\!s \triangleright E\!:\!t]\!])$. Note that, indeed, this morphism is well-defined: this is because $[\![X, x\!:\!s \triangleright E\!:\!t]\!]$ is a morphism $[\![X]\!] \times [\![s]\!] \to [\![t]\!]$ and curry takes morphisms in $\mathit{Cxt}([\![X]\!] \times [\![s]\!], [\![t]\!])$ to morphisms in $\mathit{Cxt}([\![X]\!], [\![t]\!]^{[\![s]\!]})$. Therefore, $\mathit{curry}([\![X, x\!:\!s \triangleright E\!:\!t]\!])$ is a morphism $[\![X]\!] \to [\![s \to t]\!]$, as expected. Note the elegance of the definition of this CCC interpretation, in contrast to the corresponding definition in Henkin models;

- $[\![X \triangleright E_1 E_2\!:\!t]\!] \stackrel{def}{=} \langle [\![X \triangleright E_1\!:\!s \to t]\!], [\![X \triangleright E_2\!:\!s]\!] \rangle; \mathit{app}^{[\![s]\!],[\![t]\!]}$. This definition needs some explanation. Note that $[\![X \triangleright E_1\!:\!s \to t]\!]$ is a morphism $[\![X]\!] \to [\![t]\!]^{[\![s]\!]}$, while $[\![X \triangleright E_2\!:\!s]\!]$ is a morphism $[\![X]\!] \to [\![s]\!]$, so $\langle [\![X \triangleright E_1\!:\!s \to t]\!], [\![X \triangleright E_2\!:\!s]\!] \rangle$ is a morphism $[\![X]\!] \to [\![t]\!]^{[\![s]\!]} \times [\![s]\!]$. This morphism can be indeed composed with $\mathit{app}^{[\![s]\!],[\![t]\!]} : [\![t]\!]^{[\![s]\!]} \times [\![s]\!] \to [\![t]\!]$, thus giving a morphism $[\![X]\!] \to [\![t]\!]$, as desired. All these morphisms are depicted in the following figure:

$$\begin{tikzcd}
 & & {[\![t]\!]^{[\![s]\!]}} & \\
{[\![X]\!]} \arrow[urr, bend left, "{[\![X \triangleright E_1 : s\to t]\!]}"] \arrow[r, "{\langle [\![X \triangleright E_1 : s\to t]\!], [\![X \triangleright E_2 : s]\!] \rangle}"] \arrow[drr, bend right, "{[\![X \triangleright E_2 : s]\!]}"'] & {[\![t]\!]^{[\![s]\!]} \times [\![s]\!]} \arrow[ur, "\pi_1"] \arrow[r, "{\mathit{app}^{[\![s]\!],[\![t]\!]}}"] \arrow[dr, "\pi_2"'] & {[\![t]\!]} & \\
 & & {[\![s]\!]} &
\end{tikzcd}$$

Like in the diagram above, it is often the case that constructions and proofs in category theory are driven almost automatically by diagrams; note that there are no other simpler ways to put together the morphisms $[\![X]\!] \to [\![t]\!]^{[\![s]\!]}$ and $[\![X]\!] \to [\![s]\!]$ into a morphism $[\![X]\!] \to [\![t]\!]$.

The remaining definitions are straightforward:

- $[\![X \triangleright (E_1, E_2)\!:\!t_1 \times t_2]\!] \stackrel{def}{=} \langle [\![X \triangleright E_1\!:\!t_1]\!], [\![X \triangleright E_2\!:\!t_2]\!] \rangle$;

- $[\![X \triangleright \mathit{proj}_1 E\!:\!t_1]\!] \stackrel{def}{=} [\![X \triangleright E\!:\!t_1 \times t_2]\!]; \pi_1$;

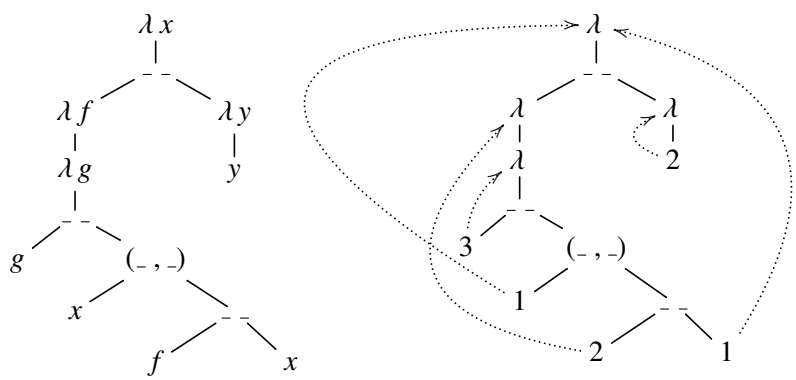
- $[\![X \triangleright proj_2 E : t_2]\!] \stackrel{def}{=} [\![X \triangleright E : t_1 \times t_2]\!]; \pi_2$.

We can associate a morphism $[\![X \triangleright E : t]\!] : [\![X]\!] \to [\![t]\!]$ to any well-typed λ-expression $X \triangleright E : t$. Let us construct such a morphism for a concrete λ-expression:

$$
\begin{array}{l}
[\![\emptyset \triangleright \lambda x{:}s.(\lambda f{:}s \to s.\lambda g{:}s \times s \to t.g(x, fx))(\lambda y{:}s.y){:}s \to (s \times s \to t) \to t]\!] = \\
curry([\![x{:}s \triangleright (\lambda f{:}s \to s.\lambda g{:}s \times s \to t.g(x, fx))(\lambda y{:}s.y){:}(s \times s \to t) \to t]\!]) = \\
curry(\langle [\![x{:}s \triangleright \lambda f{:}s \to s.\lambda g{:}s \times s \to t.g(x, fx){:}u]\!], [\![x{:}s \triangleright \lambda y{:}s.y{:}v]\!]\rangle; app^{[\![u]\!],[\![v]\!]}) = \\
curry(\langle curry(curry([\![x{:}s, f{:}s \to s, g{:}s \times s \to t \triangleright g(x, fx){:}t]\!])), curry([\![x{:}s, y{:}s \triangleright y{:}s]\!])\rangle; app^{[\![u]\!],[\![v]\!]}) = \\
curry(\langle curry(curry(\langle [\![X \triangleright g{:}s \times s \to s]\!], [\![X \triangleright (x, fx){:}s \times s]\!]\rangle; app^{[\![s \times s]\!],[\![s]\!]})), curry(\pi_2)\rangle; app^{[\![u]\!],[\![v]\!]}) = \\
curry(\langle curry(curry(\langle \pi_3, \langle \pi_1, \langle \pi_2, \pi_1\rangle; app^{[\![s]\!],[\![s]\!]}\rangle\rangle; app^{[\![s \times s]\!],[\![s]\!]})), curry(\pi_2)\rangle; app^{[\![u]\!],[\![v]\!]}),
\end{array}
$$

where $u \stackrel{def}{=} (s \to s) \to (s \times s \to t) \to t$, $v \stackrel{def}{=} s \to s$, and $X \stackrel{def}{=} x{:}s, f{:}s \to s, g{:}s \times s \to s$.

Interestingly, note that the morphism obtained above contains no references to the variables that occur in the original λ-expression. It can be shown that the interpretation of λ-expressions into a CCC is *invariant to α-conversion*. To see that, let us draw the morphism above as a tree, where we write λ instead of *curry*, $_\ _$ instead of $\langle _\,,_\rangle; app^{[\![s]\!],[\![t]\!]}$, $(_\,,_)$ instead of the remaining $\langle _\,,_\rangle$ and i instead of π_i (and omit the types):



The (right) tree above suggests a representation of λ-expressions that is invariant to α-conversion: each binding variable is replaced by a natural number, representing the number of λs occurring on the path to it; that number then replaces consistently all the bound occurrences of the variable. The corresponding lambda expression without variables obtained using this transformation is $\lambda\,(\lambda\,\lambda\,(3\ (1, (2\ 1)))\ \lambda\,2)$.

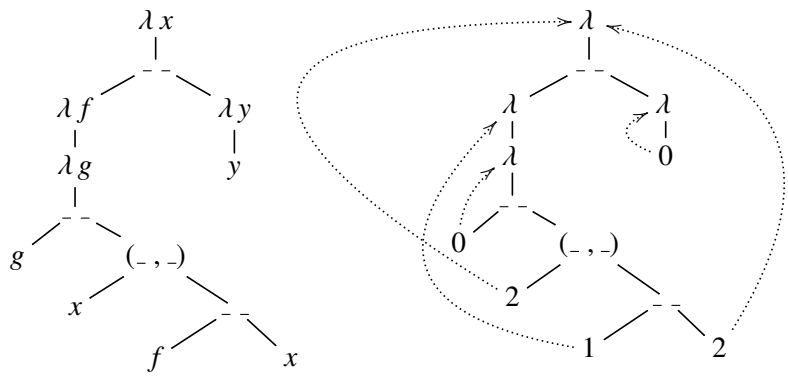
Exercise 215. *Explain why this representation is invariant to α-conversion.*

The representation of α-expressions above was explicitly proposed as a means to implement λ-calculus in 1971 by Nicholas de Bruijn.

In the same paper, de Bruijn proposed another encoding which became more popular. We do not know whether de Bruijn was influenced by the CCC interpretation of λ-expressions or not, but we discuss his other representation technique here.

de Bruijn Nameless Representation of λ-expression

The second and more popular representation technique of λ-expressions proposed by Nicholas de Bruijn in 1971 is a bottom-up version of the above representation. For the above example the tree representing the encoding is (we omit the types):



In this encoding, each variable is replaced by the number of lambda abstractions on the path from it to the lambda abstraction binding it. The encoding for the given example is $\lambda\,(\lambda\,\lambda\,(0\ (2, (1\ 2)))\ \lambda\,0)$.

One can easily define application for the above de Bruijn encoding:

$(\lambda\, E\, E') \Rightarrow E[E'/0]$
$(E\ E')[E''/i] \Rightarrow (E[E''/i])\ (E'[E''/i])$ $(\lambda\, E)[E'/i] \Rightarrow \lambda\,(E[\uparrow E'/(i1 +_{Int})])$ $j[E/i] \Rightarrow$ if $j == i$ then E else (if $j > i$ then $j - 1$ else j fi) fi
$\uparrow E' \Rightarrow \uparrow^0 E$
$\uparrow^i (E\ E') \Rightarrow (\uparrow^i E)\ (\uparrow^i E')$ $\uparrow^i (\lambda\, E) \Rightarrow \lambda\,(\uparrow^{i+1} E)$ $\uparrow^i j \Rightarrow$ if $j >= i$ then $(j + 1)$ else j fi

Exercise 216. *Define the transformation above formally in Maude.*

Exercise 217. *Define application for the first de Bruijn encoding, in a similar style to the one above.*

Extending Henkin models

We can extend the definition of Henkin models to (S, Σ)-*Henkin* models. Thus, an (S, Σ)-*preframe* or *premodel (Henkin)* consists of the following:

- a set M_t for each type t;
- a (*Type*-indexed) mapping $M_{_} : \Sigma \to M$;
- for any $s, t \in Type$
  - a function $M^{s,t} : M_{s\to t} \times M_s \to M_t$ which is *extensional*, i.e., for any $f, g \in M_{s\to t}$, if $M^{s,t}(f, u) = M^{s,t}(g, u)$ for all $u \in M_s$ then $f = g$;
  - two functions $M_s^{s\times t} : M_{s\times t} \to M_s$ and $M_t^{s\times t} : M_{s\times t} \to M_t$ which form an *extensional pair*, i.e., for any $p, q : M_{s\times t}$, if $M_s^{s\times t}(p) = M_s^{s\times t}(q)$ and $M_t^{s\times t}(p) = M_t^{s\times t}(q)$ then $p = q$;

Given $u \in M_s$ and $v \in M_t$, when it exists we let (u, v) denote the *unique* element in $M_{s\times t}$ such that $M_s^{s\times t}(p) = u$ and $M_t^{s\times t}(p) = v$.

A well-typed λ-expression $X \triangleright E : t$ is interpreted by a mapping $M_{X \triangleright E:t} : [X \to M] \to M_t$ such that:

- $M_{X \triangleright \sigma:s}(\rho) = M_\sigma \in M_s$;
- $M_{X \triangleright x:s}(\rho) = \rho(x : s) \in M_s$;
- $M^{s,t}(M_{X \triangleright \lambda x:s.E:s\to t}(\rho), v) = M_{X,x:s \triangleright E:t}(\rho[v/x])$ for any $v \in M_s$;

- $M_{X \triangleright E_1 E_2 : t}(\rho) = M^{s,t}(M_{X \triangleright E_1 : s \to t}(\rho), M_{X \triangleright E_2 : s}(\rho))$;

- $M_s^{s \times t}(M_{X \triangleright (E,E') : s \times t}(\rho)) = M_{X \triangleright E : s}(\rho)$ and $M_t^{s \times t}(M_{X \triangleright (E,E') : s \times t}(\rho)) = M_{X \triangleright E' : t}(\rho)$;

- $M_{X \triangleright proj_1 E : s}(\rho) = M_s^{s \times t}(M_{X \triangleright E : s \times t}(\rho))$;

- $M_{X \triangleright proj_2 E : t}(\rho) = M_t^{s \times t}(M_{X \triangleright E : s \times t}(\rho))$.

Henkin models are CCCs
Given an (S, Σ)-Henkin model $\mathcal{M}$, one can define a (S, Σ)-CCC $\mathbb{C}$:

- the objects of $\mathbb{C}$: M_t for each $t \in Type$;

- the set of morphisms $\mathbb{C}(M_s, M_t)$ is $M_{s \to t}$;

- the identity morphism 1_{M_s} is $M_{\emptyset \triangleright \lambda x:s.x:s \to s}$;

- given $f \in \mathbb{C}(M_s, M_t)$ and $g \in \mathbb{C}(M_t, M_{t'})$, define $f; g$ as $M_{u:s \to t, v:t \to t' \triangleright \lambda x:s.v(ux):s \to t'}(\rho)$ where $\rho(u) = f$ and $\rho(v) = g$;

- $M_s \times M_t = M_{s \times t}$ and the projections are $M_s^{s \times t}$ and $M_t^{s \times t}$;

- the exponentiation object $M_t^{M_s}$ is $M_{s \to t}$, and the application morphism app^{M_s, M_t} is $M_{\emptyset \triangleright \lambda x:(s \to t) \times s.proj_1(x)\ proj_2(x):(s \to t) \times s \to t}$;

- $[\![t]\!] = M_t$ and $[\![\sigma]\!] = M_\sigma$.

Exercise 218. *Prove that $\mathbb{C}$ is indeed a (S, Σ)-CCC.*

Some CCCs are Henkin models Let $\mathbb{C}$ be an (S, Σ)-CCC such that for each object A, $\mathbb{C}(\star, A)$ is a *homomorphic family*, that is, for any object B and any two morphisms $f, g : A \to B$, if $h; f = h; g$ for each $h : \star \to A$ then $f = g$; such a category is also called *well-pointed*. Then we can define a Henkin model $\mathcal{M}$ for (S, Σ) as follows:

- $M_t = \mathbb{C}(\star, [\![t]\!])$ for each type t;

- $M^{s,t} : M_{s \to t} \times M_s \to M_t$ is given by: for any $f : \star \to [\![s \to t]\!] = [\![t]\!]^{[\![s]\!]}$ and any $x : \star \to [\![s]\!]$, $M^{s,t}(f, x) = \langle f, x \rangle; app^{[\![s]\!], [\![t]\!]}$;

- given $X = x_1 : s_1, \ldots, x_n : s_n$ let $\prod_X = [\![s_1]\!] \times \cdots \times [\![s_n]\!]$. Each morphism $h : \star \to \prod_X$ is equivalent with the tuple $\langle h; \pi_{[\![s_1]\!]}, \ldots, h; \pi_{[\![s_n]\!]} \rangle$. Thus environments $\rho : X \to M$ and morphisms $\star \to \prod_X$ are in bijection. Let $\overline{\rho} : \star \to \prod_X$ denote the image of ρ through this bijection;

- For each well-typed λ-expression $X \triangleright E : s$, and each assignment ρ, let $M_{X \triangleright E : s}(\rho) = \overline{\rho}; [\![X \triangleright E : s]\!]$.

Exercise 219. *Show that $\mathcal{M}$ is indeed an (S, Σ)-Henkin model.*

4.7 Recursion and PCF

this should be split in various sections

We are already familiar with various facets of recursion (in languages that we encountered, in rewriting, in lambda-calculus, etc.). The following is an interesting observation that deserves some further investigation:

> *The simpler the programming or specification language paradigm, the simpler the treatment of recursion.*

Indeed, recursion is not even noticeable as a special feature in a rewrite engine (for example Maude) or in an imperative programming language (like C), but it requires special language support, including special syntax, for more complex languages (such as `ML`).

Recursion in Term Rewriting

Rewriting supports recursion naturally, without any particular technical or theoretical infrastructure. It is just us, humans, who understand a certain rewriting definition as recursive; the rewrite engine does not even need to be aware of that. Consider, for example, a rewriting implementation of the length operator on lists using Maude notation:

```
eq length(nil) = 0 .
eq length(M ; L) = 1 + length(L) .
```

From the perspective of rewriting, there is nothing special going on here. A term `length(L)` for some list `L` is iteratively rewritten using the rules above as well as the rules of integer numbers, until it is eventually reduced to a natural number. Even though we may perceive the rewriting definition of `length` as recursive, the rewrite engine does nothing but *match-and-apply* rules until a normal form is reached, which we can interpret as the "value" obtained after "evaluating" the original expression.

However, note that the process of rewriting itself is recursive. Indeed, we can regard the process of rewriting a term t, say *rewrite(t)*, as a procedure defined recursively as follows:

rewrite(t)

1. find a rule $l \to r$ such that l matches some subterm u of t; if no such rule exists then return t;
2. let θ be a substitution with $\theta(l) = u$;
3. replace u by $\theta(r)$ in t and obtain a new term t';
4. *rewrite(t')*

In order for rewriting definitions to be practical, one needs to ensure that they *terminate*. To achieve that, one typically needs to ensure that a term is always reduced to a *simpler* term. This reduces to showing that any instance of the right-hand side of a rule is somehow simpler than the corresponding instance of the left-hand-side. What "simpler" means is dependent on the particular definition. In the case of `length`, simpler means the operator is applied on a strictly smaller list in the right-hand-side term. Similar well-founded techniques are needed to prove termination of recursive programs in any programming language; the point here is, again, that rewriting definitions do not need to treat recursive definitions any differently.

It is interesting to note that the concept of "simpler term" is a semantical one - the term does not have to be simpler as a tree or w.r.t. some other immediate syntactic criterion, as shown by the following rewriting definition of bubble-sort:

```
eq bubbleSort(L)
 = if process(L) == L then L else bubble(process(L)) fi .

eq process(N ; M ; L)
 = if N <= M then N ; process(M ; L)  else M ; process(N ; L) fi .
eq process(N) = N .
eq process(nil) = nil .
```

Here, the computation of `bubble(L)` eventually reducces to the computation of `bubble(L')`, where the latter is simpler than the former in the sense that `L'` is *more sorted* than `L`.

Note, however, that one can speculate the recursive nature of rewriting and define bubble sort by just one simple and elegant conditional rewriting rule:

```
ceq M ; N = N ; M if N <= M .
```

Now there is no explicit term in the right-hand-side of the rule that one can show smaller that a corresponding term in the left-hand-side. Nevertheless, one can show that any application of the rule above would decrease the "weight" of any list that it applies on.

Besides termination, *confluence* is another important factor that needs to be considered when designing recursive rewriting definitions. Confluence generalizes the Church-Rosser result (for variants of λ-calculus) to any rewrite system: a rewrite system is confluent if and only if for any two reductions of a term t to t_1 and t_2, there is some term t' such that t_1 and t_2 reduce to t'. A rewrite system that is both confluent and terminating is called *canonical*. Note that terms have *unique normal forms* when reduced using canonical rewrite systems. In our context, a recursive rewriting definitions that is a canonical rewrite system can and should be regarded as a *well-behaved definition*.

Consider, for example, the following (deliberatly more complicated than needed) definition of addition of natural numbers:

```
eq M + 0 = M .
eq 0 + N = N .
eq s(M) + N = s(M + N) .
eq M + s(N) = s(M + N) .
```

The above simulates a non-deterministic recursive definition. According to this definition, there are many ways to compute the sum of m and n, since each time there is no unique way to reduce a term containing + to a simpler one. One can relatively easilly show that this definition is well-behaved, because each rule simplifies the term to reduce in some way (first two elliminate a + operator, while the other two push an `s` operator higher).

We can therefore see that canonical rewrite systems support in a simple and natural way arbitrarily complex recursive definitions. Moreover, since rewrite rules can be applied in parallel when their applicaltions do not overlap, canonical rewrite systems can also be regarded as potentially very efficient implementations of recursive definitions. In particular, devide-and-concur recursive definitions can be regarded operationally as starting a new thread solving the subproblems; on a parallel implementation of a rewrite engine, one can achieve a computational speed close to as many times as processors available at no additional programming effort.

One should not underestimate the difficulty of showing rewrite systems canonical. Since rewriting is Turing-complete, not even termination is decidable. One can show that in general neither termination nor

confluence of a rewrite system is decidable. To have a better feel for the difficulty of the confluence problem, consider the following problem.

Suppose that a group of children have a large collection of black and white balls and decide to play a game. They put a bunch of balls in a bag and then each of them, at any moment they wish and potentially in parallel, extracts two balls from the bag and puts one back as follows: if the two balls have the same color they put a black ball back in the bag, otherwise they put a white one. If the bag is declared as an associative binary operator __, then this game can be easily defined as a four rule ground rewrite system:

$$\bullet\,\bullet \to \bullet\,,\ \circ\,\circ \to \bullet\,,\ \bullet\,\circ \to \circ\,,\ \circ\,\bullet \to \circ.$$

Using variables, one can define it as just one rule:

```
eq X Y = if X == Y then black else white fi .
```

Exercise 220. *Prove that the rewrite system above is canonical.*

Recursion in Simple Imperative Languages

Recall the implementation of the Hanoi tower problem in a simple C-like imperative language:

```
function h(x, y, z, n) {
  if n >= 1 then {
    h(x, z, y, n - 1) ;
    print(x) ;
    print(z) ;
    h(y, x, z, n - 1)
  }
}
```

In this recursive function, `h` refers to itself directly and transparently. This is possible because of the default design conventions of the imperative language:

- all functions are declared at the top level of the program (more precisely at the begining);
- no nested function declarations allowed;
- function declarations are assumed visible anywhere in the program, including in the body of the functions themselves.

Therefore, languages can support recursion naturally if they are constrained by design appropriately. Nevertheless, many programmers may not be happy with such language constraints just for the sake of supporting recursion transparently.

Recursion in Higher Order Languages

Recursion can be supported in a more flexible (but also more intricate) way when we climb to higher order languages, where functions, as first-order volatile citizens of the program, start raising non-trivial scoping and visibility issues. For instance, in the recursive definition of factorial

```
letrec f n = if n eq 0 then 1 else n * f(n - 1)  in  f 7
```

one has to *explicitly* state, using `letrec` instead of `let`, that the expression defining `f` is not a usual, but a recursive binding. Any other trick of simulating recursion would be unnatural and statically untypeable, like the classical

```
let f n g = if n eq 0 then 1 else n * g (n - 1) g  in  f 7 f
```

Recursion in Untyped λ-Calculus

The latter (untypeable) definition of factorial, which did not have to use the explicit recursion construct `letrec`, was given in the style of untyped λ-calculus, by passing a function to itself as an argument. As we have already seen, this technique can be applied very generally in the context of untyped λ-calculus, taking advantage of the *fixed point theorem*:

Theorem. If Y denotes $\lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$, then $YF \equiv_\beta F(YF)$ for any λ-expression F.

A λ-expression Y with the property above, namely that $YF \equiv_\beta F(YF)$ for any λ-expression F, is called a *fixed point operator*. The particular Y above is also known as the *call-by-name* fixed point operator (for reasons that will become clear shortly). As expected, there can are many other fixed point operators, including the following:

Exercise 221. *Show that the following λ-expressions are also fixed point operators:*

- $(\lambda x.\lambda f.f(xxf))(\lambda x.\lambda f.f(xxf))$*, also known as the Turing fixed point operator;*
- $\lambda f.(\lambda x.f(\lambda y.xxy))(\lambda x.f(\lambda y.xxy))$*, also known as the call-by-value fixed point operator.*

We have already discussed how the fixed-point operator Y in the fixed-point theorem can be used to prove equational properties about recursive programs. For example, if we let F be the usual factorial λ-expression

$$\lambda f.\lambda n.\ \texttt{if}\ n\ \texttt{eq}\ 0\ \texttt{then}\ 1\ \texttt{else}\ n * f(\text{pred}\ n)\ ,$$

where we assume `if-then-else` and pred , the latter taking any strictly positive number n into $n-1$ and 0 into 0, as already defined syntactic sugar (note that `if-then-else` has no associated evaluation strategy.), then one can easily show by using the equational rules of λ-calculus properties like $(YF)3 \equiv_\beta 6$.

But how does (recursive) *computation* actually take place in λ-calculus? Of course, our computational mechanism must be based on β-reduction here. We have no need, and no intuitive computational meaning, for the η rule; the latter makes sense only for expressing and proving *equivalence of programs*.

However, one cannot blindly apply β-reduction whenever a possibility is found inside the current term, since this way one might miss oportunities of termination; although it is true that, by the Church-Rosser theorem, no β-reduction is an irreparable mistake, nevertheless a persistent chain of wrong β-reductions might indefinitely continue a virtually terminating computation. For example, one can easily see that in the case of our factorial, the following can take place

$$(YF)3 \Rightarrow_\beta Y_F3 \Rightarrow_\beta FY_F3 \Rightarrow_\beta F(FY_F)3 \Rightarrow_\beta F(F(FY_F))3 \ldots ,$$

where Y_F is the λ-expression $Y[F/f]$.

Exercise 222. *If for any λ-expression F we let Y_F denote the λ-expression $(\lambda x.F(xx))(\lambda x.F(xx))$, show that $Y_F \Rightarrow_\beta FY_F$. The fixed-point theorem tells us that $YF \equiv_\beta F(YF)$ anyway; is it also true that $YF \Rightarrow_\beta F(YF)$? How about the other two fixed-point operators in Exercise 221?*

Thanks to the Church-Rosser theorem, we know that if a normal form exists then it is unique. In particular, 6 is the only normal form of $(YF)3$ despite the fact that the reduction of $(YF)3$ may not terminate. Needless to mention that, in practical implementations of programming languages, one would like to devise reduction or evaluation strategies that would lead to normal forms whenever they exists.

One simplistic way to achieve the desired computation would be by breadth-first search in the space of computations, until a β-normal form is eventually reached. Clearly, such a computation strategy would be unacceptably inefficient in practice. For that reason, strategies that are not based on search are desirable. Many reduction strategies have been investigated in the literature and implemented in various languages.

We next discuss one interesting reduction strategy, called *leftmost-outermost β-reduction*, also known as *call-by-name* or *normal-order*: each time, β-reduction is applied outside in and left-to right; this corresponds to β-reducing the first suitable subterm found by a prefix traversal of the term to reduce. The application of a β-reduction step can enable another reduction which occurs earlier in the prefix traversal, so each time a reduction is applied, the entire term to rewrite is assumed to be retraversed using the prefix strategy (better algorithms are used in practical implementations). Notice that argument expressions passed to λ-abstractions are not evaluated at *call time*, but instead at *need time*.

The first programming language providing a call-by-name evaluation strategy was Algol-60. An important variation of call-by-name, supported by several programming languages including most notoriusly Haskel, is *call-by-need*. Under call-by-need, λ-expressions are evaluated at most once: any subsequent use of a λ-expression just uses the previously calculated normal form. As we know from CS422, in the context of side effects call-by-name and call-by-need can lead to different normal forms.

> ***Theorem.*** If a λ -expression has a β-normal form, then it can be reached by leftmost-outermost β-reductions.

An intuitive argument for the above result is the following: if one applies β-reductions as close to the top as possible, then one can prevent any unnecessary non-termination obtained by infinitely rewriting a term which is supposed to dissapear anyway by an upper β-reduction.

Exercise 223. *Modify your Maude implementation of λ-calculus (either the one based on the De Bruijn transformation or the one based on combinators) to support the call-by-name reduction strategy.*
Hint. *You only need to add appropriate rewriting strategies to certain operations in the signature.*

Another very important, if not the most important, reduction strategy is the one called *call-by-value*. Under call-by-value, λ-expressions are always evaluated *before* they are passed to λ-abstractions. In order for call-by-value to work, one needs to inhibit reductions under certain language constructs, such as within the body of λ-abstractions and within the two choices of the conditional. Also, an appropriate fixed-point operator needs to be chosen.

Exercise 224. *Reduce* $(YF)3$ *using a call-by-value strategy. Can you just use the call-by-need fixed-point operator? How about the other two fixed-point operators in Exercise 221?*

Mu Simply-Typed Lambda Calculus

Once one decides that types are a useful feature of one's language and one decides to declare and/or check them statically, the next important thing one should ask oneself is how such a decision affects recursive definitions.

Typed functional languages do not allow functions to pass themselves to themselves, so in order to support recursion, one needs to introduce a new language construct (`letrec`). The goal of this topic is to discuss in more depth the relationship between types and recursive definitions of functions. To focus on the

essence of the problem, we start with the simplest typed language discussed so far, namely simply-typed λ-calculus.

Let us first understand why the presented fixed-point technique to support recursive definitions in untyped λ-calculus does not work in the context of types. Consider the fixed-point combinator Y suggested by the fixed-point theorem, $\lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$. To make Y a well-typed λ-expression in simply typed λ-calculus, we need to come up with some types t_f, t_x^1 and t_x^2, such that the λ-expression $\lambda f\!:\!t_f.(\lambda x\!:\!t_x^1.f(xx))(\lambda x\!:\!t_x^2.f(xx))$ can be typed. The reason for that is the subexpression xx: there is no type that one can assign to x such that xx can be calculated a type.

Let us assume though, for the sake of the discussion, that fixed-point combinators Y existed in simply-typed λ-calculus and let us try to understand what their type should be. Recall that such combinators are intended to apply on λ-expressions F encoding the one-step unrolling of a recursive definition.

For example, the unrolling λ-expression associated to the definition of the (typed) factorial function of type nat $\to$ nat would be

$$F := \lambda f\!:\!\text{nat} \to \text{nat}.\lambda n\!:\!\text{nat}.\ \texttt{if}\ n\ \texttt{eq}\ 0\ \texttt{then}\ 1\ \texttt{else}\ n * f(\text{pred}\ n)$$

having the type (nat $\to$ nat) $\to$ nat $\to$ nat.

Any recursive function, of any type, has such a one-step unrolling λ-expression associated to it. For example, a function of type nat $\to$ `bool` returning the parity of a natural number is associated the following λ-expression of type (nat $\to$ `bool`) $\to$ nat $\to$ `bool`:

$$F := \lambda f\!:\!\text{nat} \to \texttt{bool}.\lambda n\!:\!\text{nat}.\ \texttt{if}\ n\ \texttt{eq}\ 0\ \texttt{then}\ \text{true}\ \texttt{else}\ \texttt{not}\ (f(\text{pred}\ n)).$$

It is interesting to note that recursion can mean more than just a recursive function. For example, in lazy functional programming it is customary to define infinite data structures, such as infinite lists or infinite trees, by defining their actual behavior when asked to produce values. For example, an infinite list of zeros should be able to produce a zero whenever asked for its first element (by passing it to `car`). The unrolling λ-expression for such a recursive data-structure would be

$$F := \lambda l\!:\!\texttt{list}\ \text{nat}.\ \texttt{cons}\ 0\ l,$$

which has the type `list` nat $\to$ `list` nat.

Finally, one may want to compactly define error values for any type t as recursive expressions referring just to themselves, having therefore associated the trivial unrolling λ-expression $F := \lambda x\!:\!t.x$ of type $t \to t$.

Consequently, for any type t, it is reasonable to assume that any recursively defined λ-expression E of type t has an unrolling λ-expression F_E of type $t \to t$. Since one expects that $E \equiv YF_E$, one can intuitively claim that a fixed-point combinator Y, if it exists, must have the type $(t \to t) \to t$. More precisely, one would expect for each type t some corresponding fixed-point combinator Y_t of type $(t \to t) \to t$.

In fact, one can prove that there exist no such magic fixed-point combinators Y_t in the simply-typed λ-calculus language. It is relatively usual in computer science foundational works to enrich a certain formalism with desired features *by definition*, or *by axiom*, when those features cannot be explained or simulated by the existing framework. Since the concept of fixed-point seems to very naturally capture the essence of recursion, the solution that has been adopted by scientists to allow recursion in typed programming environments is to extend the language by simply assuming a Y_t for each type t.

Thus, for each type t we assume the existence of a new constant $Y_t\!:\!(t \to t) \to t$, called for simplicity the *fixed-point operator for type t*. Whether Y_t can or not be explained from more basic principles does not concern us here. Instead, we axiomatize its behavior by requiring that

$Y_t F \equiv F(Y_t F)$ for any λ-expression F of type $t \to t$.

Alternatively and more compactly, noticing that the considered expression F always has the form $\lambda x{:}t.E$, we can define a new recursive binding operator μ to bind x in E and replace the functionality of Y_t as follows:

$$Y_t F = Y_t(\lambda x{:}t.E) = \mu x{:}t.E.$$

More precisely, we are going to have a new binding operator μ with the same syntax as λ and a new equational rule saying that

$$\mu x{:}t.E \equiv E[(\mu x{:}t.E)/x],$$

with the intuition that arbitrarily many copies of $\mu x : t.E$ can be generated, typically by need, nesting the original expression in all places where x occures free. An even more intuitive equivalent form of this rule is

$$\mu x{:}t.E \equiv (\lambda x{:}t.E)(\mu x{:}t.E),$$

which says that $\mu x{:}t.E$ is a fixed point of the function λ-abstracting the expression E in the variable x.

So far, we only justified the μ-extension of simply-typed λ-calculus by examples. A rigorously formulated reason for extending simply-typed λ-calculus is that it is not expressive enough as a model of computation: it can only encode very simple programs which do not have any recursive behavior. In particular, simply-typed λ-calculus is *decidable*; this follows from two facts:

- like for untyped λ-calculus, the Church-Rosser theorem holds;
- unlike for untyped λ-calculus, all computations eventually terminate, thus leading to the reach of unique normal forms.

Hence one can decide if two terms are provably equal by simply reducing them into their normal forms and then syntactically comparing the results.

Programming Language for Computable Functions

As one might expect, simply typed λ-calculus extended with the recursion operator μ has full computational power. We are going to present a language called PCF (abbreviating Programming language for Computable Functions), a μ-extension of simply typed λ-calculus with a particular but important signature (S, Σ) together with corresponding equations.

The BNF syntax for types and expressions in PCF is the following

$$Var ::= x \mid y \mid \ldots$$

$$Type ::= \boxed{S} \mid Type \to Type$$

$$Exp ::= \boxed{\Sigma} \mid Var \mid ExpExp \mid \lambda Var{:}Type.Exp \mid \boxed{\mu Var{:}Type.Exp}.$$

where $S = \{\mathrm{nat}, \mathtt{bool}\}$ is the set of basic types. and $\Sigma = \{0 : \mathrm{nat}, \mathrm{succ} : \mathrm{nat} \to \mathrm{nat}, \mathrm{pred} : \mathrm{nat} \to \mathrm{nat}, \mathrm{true} : \mathtt{bool}, \mathrm{false} : \mathtt{bool}, \mathrm{zero?} : \mathrm{nat} \to \mathtt{bool}\} \cup \{\mathrm{cond}_t : bool \to t \to t \to t \mid \forall t \in Type\}$ is the signature.

When discussing typing and equations, we build on top of the already defined (S, Σ)-simply-typed λ-calculus.

We only add one typing rule:

$$\frac{X, x{:}t \vdash E{:}t}{X \vdash \mu x{:}t.E{:}t} \qquad \text{for any type assignment } X, x{:}t, \ \lambda\text{-expression } E \text{ of type } t;$$

Exercise 225. *Type the following expressions:*

- $+ := \mu f\!:\!\text{nat} \to \text{nat} \to \text{nat}.\lambda x\!:\!\text{nat}.\lambda y\!:\!\text{nat}.$
  $\text{cond}_{\text{nat}}\ (\text{zero?}\ x)\ y\ (\text{succ}\ f(\text{pred}\ x)\ y),$

- $* := \mu f\!:\!\text{nat} \to \text{nat} \to \text{nat}.\lambda x\!:\!\text{nat}.\lambda y\!:\!\text{nat}.$
  $\text{cond}_{\text{nat}}\ (\text{zero?}\ x)\ 0\ (+\ (f(\text{pred}\ x)\ y)\ y).$

We add the folowing equational rules, describing the desired behavior of the Σ-constants and of the recursion operator μ; by a language abuse, we let n denote succ succ $\ldots$ succ 0 (n times):

$\mathcal{E} \vdash (\forall\emptyset)\ \text{pred}\ 0 =_{\text{nat}} 0$

$\mathcal{E} \vdash (\forall X)\ \text{pred succ}\ n =_{\text{nat}} n$

$\mathcal{E} \vdash (\forall\emptyset)\ \text{zero?}\ 0 =_{\texttt{bool}} \text{true}$

$\mathcal{E} \vdash (\forall\emptyset)\ \text{zero? succ}\ n =_{\texttt{bool}} \text{false}$

$\mathcal{E} \vdash (\forall X)\ \text{cond}_t\ \text{true}\ E\ E' =_t E \quad \text{if}\ X \triangleright E\!:\!t,\ X \triangleright E'\!:\!t$

$\mathcal{E} \vdash (\forall X)\ \text{cond}_t\ \text{false}\ E\ E' =_t E' \quad \text{if}\ X \triangleright E\!:\!t,\ X \triangleright E'\!:\!t$

$$(\xi_\mu) \qquad \frac{\mathcal{E} \vdash (\forall X, x\!:\!t)\ E =_t E'}{\mathcal{E} \vdash (\forall X)\ \mu x\!:\!t.E =_t \mu x\!:\!t.E'}$$

$$(\mu) \qquad \mathcal{E} \vdash (\forall X)\ \mu x\!:\!t.E =_t E[(\mu x\!:\!t.E)/x]$$

$$(\mathsf{Con}_\sigma) \qquad \frac{\mathcal{E} \vdash (\forall X)\ E =_{\text{nat}} E'}{\mathcal{E} \vdash (\forall X)\ \sigma E =_{\text{nat}} \sigma E'} \quad \text{, where } \sigma \in \{\text{succ}, \text{pred}\}$$

$$(\mathsf{Con}_{\text{zero?}}) \qquad \frac{\mathcal{E} \vdash (\forall X)\ E =_{\text{nat}} E'}{\mathcal{E} \vdash (\forall X)\ \text{zero?}\ E =_{\texttt{bool}} \text{zero?}\ E'}$$

$$(\mathsf{Con}_{\text{cond}}) \qquad \frac{\mathcal{E} \vdash (\forall X)\ C =_{\texttt{bool}} D,\ \mathcal{E} \vdash (\forall X)\ E =_t F,\ \mathcal{E} \vdash (\forall X)\ E' =_t F'}{\mathcal{E} \vdash (\forall X)\ \text{cond}_t\ CEE' =_t \text{cond}_t\ DFF'}$$

Exercise 226. *Derive the following identities:*

- $(\forall\emptyset)\ +\ (\text{succ}\ 0)\ (\text{succ}\ (\text{succ}\ 0)) =_{\text{nat}} \text{succ}\ (\text{succ}\ (\text{succ}\ 0)),$

- $(\forall\emptyset)\ \text{zero?}\ (\text{succ}\ 0) =_{\texttt{bool}} \text{false}$.

We have already discussed (in a rather informal context) a possible computational semantics of untyped λ-calculus, based on leftmost-outermost β-reduction.

Now we shall give the PCF language two operational semantics, a transitional or small-step one, and a natural or big-step one.

4.7.1 Transitional semantics

The transitional semantics will be given as a *restricted* rewriting relation on λ-terms ("restricted" in the sense that it cannot be applied inside any arbitrary context). This relation will tell us how to *evaluate* programs (that is, terms) to *values* (that is, terms which are normal forms). But what are the values in our framework? They are just the usual natural and Boolean values, plus the λ-abstractions. Here is the BNF description of values:

$$\mathit{Value} ::= 0 \mid \text{succ } \mathit{Value} \mid \text{true} \mid \text{false} \mid \lambda\, \mathit{Var}\!:\!\mathit{Type}.\mathit{Exp}$$

The fact that we take *all* λ-abstractions as values, even though it might be possible to further evaluate their body, might seem strange at a first glance; however, this is natural if we consider the body of a λ-abstraction as just the text describing a function, which will be evaluated only if the function is to be applied (remember *closures* from functional languages). We shall soon come back to this discussion.

Like in any functional language, we have to make a choice between several evaluation strategies when we define semantics. We choose *call-by-name* below, since this is immediately related to the intuition of β-reduction, but one can easily adapt it to consider other evaluation startegies.

Exercise 227. *In a similar manner to the call-by-name transitional semantics below, give a transitional semantics to PCF following the call-by-value evaluation strategy.*

We define the *one-step-transition* binary relation $\to$ on λ-expressions as the smallest relation closed under the following rules, where we assume that all (outermost) expressions that appear in the left-hand-side of the rules are well-typed (e.g., we assume that FE in the application rule is well-typed, hence we also implicitly assume that E and F are typeable to some types $t \to s$ and t, respectively):

$$\begin{array}{l}
\text{pred } 0 \to 0 \\
\text{pred succ } V \to V \qquad \text{, if } V \text{ is a value} \\
\text{zero? } 0 \to \text{true} \\
\text{zero? succ } V \to \text{false} \qquad \text{, if } V \text{ is a value} \\
\text{cond}_t \text{ true } E\ E' \to E \\
\text{cond}_t \text{ false } E\ E' \to E'
\end{array}$$

$$\frac{E \to E'}{\sigma E \to \sigma E'} \quad \text{, if } \sigma \in \{\text{succ}, \text{pred}, \text{zero?}\}$$

$$\frac{C \to C'}{\text{cond}_t\ CEE' \to \text{cond}_t\ C'EE'}$$

$$\frac{F \to F'}{FE \to F'E}$$

$$(\lambda\, x\!:\!t.E)E' \to E[E'/x]$$

$$(\mu x\!:\!t.E) \to E[(\mu x\!:\!t.E)/x]$$

Note that, unlike in the case of general term rewriting, the above rules cannot be applied inside *any* context (subterm), but actually apply on top Some incursions into the to-be-rewritten term context are possible, but only those obeying some restrictions:

- never evaluate the passed argument (call-by-name),
- never evaluate the second and third arguments of a condition,
- never apply β-reduction or μ-unwinding in the body of another λ or μ-abstraction, etc.

And all the above negative facts are just implicit in the rules, which do not allow arbitrary congruence.

A property that one wants to be satisfied by the one-step-transition relation is that the latter is *sound*, in the sense that it rewrites a program into an equivalent program.

Proposition 22. *(Soundness of the one-step transition relation) Suppose $X \triangleright E : t$.*
If $E \to E'$, then $\vdash (\forall X) E =_t E'$.

(Proof hint: Define the relation R on terms as $E \simeq E'$ iff there exists a type assignment X such that $\vdash (\forall X) E =_t E'$, and show that $\simeq$ is closed under all the transition rules.)

An important consequence is the fact that types are preserved by transitions, which allows one to type-check the program only once, at the begining, and then not worry about types during the execution:

Corollary 10. *If $X \triangleright E : t$ and $E \to E'$, then $X \triangleright E' : t$.*

Another fact to notice is that we were able to define our values (i.e. normal forms) *a priori*, and then guide our notion of computation towards them. And indeed, what we initially called "values" are precisely those closed terms that cannot be further reduced:

Proposition 23. *The following are equivalent for any closed (i.e., without free variables) expression E:*

- *E is a value;*
- *there is no E' such that $E \to E'$.*

(Proof hint: Show that, on the one hand, a value cannot be rewritten and, on the other hand, that any closed non-value can be rewritten.)

Notice that there are non-closed expressions, like the variables or applications between two variables, that are neither values nor can transition in one step into anything. Moreover, there are closed expressions that are neither values nor can transition in one step into any expression *other than themselves*, like, for instance, $\mu x : t.x$.

Exercise 228. *Characterize the non-closed expressions satisfying the former property. Is there any other expression, not α-equivalent to $\mu x : t.x$, satisfying the latter property?*

Define $\to^*$, the *transition relation*, to be the reflexive and transitive closure of $\to$. By the soundness of the one-step-transition relation, one can immediately infer

Theorem*(Soundness of the transition relation)* Suppose $X \triangleright E : t$. If $E \to^* E'$, then $\vdash (\forall X)\ E =_t E'$.

Corollary 11. *(Subject reduction) If $X \triangleright E : t$ and $E \to^* E'$, then $X \triangleright E' : t$.*

Since our language is *deterministic*, any expression can evaluate to *at most* one value:

Proposition 24. *If E is an expression and V, V' are values such that $E \to^* V$ and $E \to^* V'$, then $V = V'$.*

(Proof hint: First show that, for each expression E, there exists *at most one* expression E' such that $E \to E'$.)

If $E \to^* V$, then we say that E *evaluates to* V. There are expressions (like $\mu x\!:\!t.x$) that do not evaluate to any value; this is natural, since not all programs terminate (on all inputs). But if an expression E evaluates to some value V, then this unique value is considered to be the result of the computation. Thus we have a *partial function* mapping expressions to values, which could be regarded as the final product of the transitional semantics.

What about completeness?

We saw that the transition relation is sound w.r.t. equivalence of programs. But how complete is it? That is, what can we infer about equivalence of two programs by evaluating them transitionally? Well, according to this restricted and pragmatic form of evaluation, not much. And here are two reasons for that (can you find more reasons?):

- η-equivalence is not in the scope of our transitional semantics; for instance, $\lambda f\!:\!\text{nat} \to \text{nat}.\lambda y\!:\!\text{nat}.fy$ and $\lambda f\!:\!\text{nat} \to \text{nat}.f$ are different values;

- even without the η equation, two different values can be proved equivalent, if they are λ-abstractions; for instance, $\lambda x\!:\!\text{nat}.(\lambda y\!:\!\text{nat}.y)x$ and $\lambda x\!:\!\text{nat}.x$; this is actually a *semantical decision*, since we do not want to look into the body of functions, unless we have to apply them.

However, one can prove a restricted form of completeness:

Theorem*(Basic-type-completeness of the transition)* If $\vdash (\forall X)\ E =_t E'$ and $t \in \{\text{nat}, \texttt{bool}\}$, then, for each value V, E evaluates to V if and only if E' evaluates to V.
(Proof hint: First show that, if V is a value of type $t \in \{\text{nat}, \texttt{bool}\}$ and $\vdash (\forall X)\ E =_t V$, then $E \to^* V$.)

Hence, if we restrict our class to programs of basic types only, two programs that are provably equivalent either both do not terminate, or they both terminate and return the same value. Note that the restriction of returning basic typed values does not inhibit higher-order programing: diverse functions can be passed as arguments and returned as values in the process of calculating an integer value! Note that the restriction to basic types is crucial.

4.7.2 Natural Semantics

The two-step process of first defining a transition on expressions, and then extracting from it a partial map from expressions to values, can be performed in only one step. The *natural semantics* captures directly the idea of evaluation $E \to^* V$, by giving rules in terms of pairs (Expression,Value). The binary relation $\Downarrow$ between expressions and values is defined as the smallest closed under the following rules (below, V denotes a value, and E, E_1, E_2 expressions; obviously, succ V is then also a value; we assume that all expressions below are well-typed:

$$V \Downarrow V \qquad \frac{E \Downarrow V}{\text{succ } E \Downarrow \text{succ } V}$$

$$\frac{E \Downarrow 0}{\text{pred } E \Downarrow 0} \qquad \frac{E \Downarrow \text{succ } V}{\text{pred } E \Downarrow V}$$

$$\frac{E \Downarrow 0}{\text{zero? } E \Downarrow \text{true}} \qquad \frac{E \Downarrow \text{succ } V}{\text{zero? } E \Downarrow \text{false}}$$

$$\frac{C \Downarrow \text{true}\ ,\ E_1 \Downarrow V}{\text{cond}_t\ C\ E_1\ E_2 \Downarrow V} \qquad \frac{C \Downarrow \text{false}\ ,\ E_2 \Downarrow V}{\text{cond}_t\ C\ E_1\ E_2 \Downarrow V}$$

$$\frac{E_1 \Downarrow \lambda x{:}s.E\ ,\ E[E_2/x] \Downarrow V}{E_1 E_2 \Downarrow V} \qquad \frac{E[x \leftarrow (\mu x{:}t.E)] \Downarrow V}{\mu x{:}t.E \Downarrow V}$$

The above rules describe the evaluation process in a bottom-up fashion, starting from the values. One can show that transitional and natural semantics are equivalent:

Proposition 25. *For each expression E and value V, $E \Downarrow V$ iff $E \to^* V$.*

The next corollary, expressing the determism of evaluation, might be instructive to be proved directly about natural semantics:

Corollary 12. *If E is an expression and V, V' are values such that $E \Downarrow V$ and $E \Downarrow V'$, then $V = V'$.*

4.7.3 Denotational Semantics of PCF

We are now ready to define a canonical model of PCF. Unlike the canonical functional models of simply-typed λ-calculus, our model should be able to capture non-termination of programs; non-termination will be handled by the interpretation of the recursion operator μ. In our semantical framework,

- basic types will be interpreted as certain CPOs with bottom element $\perp$, where $\perp$ will stand for "undefined";
- types will be interpreted as continuous functions between the interpretations of their component types;
- environments will be, as usual, (*Type*-indexed) mappings from type assignments into the model;
- well-typed expressions will be interpreted as mappings from environments into the interpretation of their types.

In what follows, we consider *bottomed CPOs* (CPOs for short), i.e., structures of the form $(P, \sqsubseteq, \perp)$, where $(P, \leq)$ is a CPO and $\perp$ is its bottom element. Note that $Cont((P, \leq, \perp_P), (Q, \leq, \perp_Q))$, the set of continuous functions from P to Q, can be naturally endowed with a CPO structure, if we let $f \leq g$ iff $f(p) \leq g(p)$ for all $p \in P$; the bottom element will be the function $\perp$ defined by $\perp(p) = \perp_Q$ for all $p \in P$. In what follows, this will be the CPO structure that we will implicitly assume on $Cont((P, \leq, \perp_P), (Q, \leq, \perp_Q))$.

Exercise 229. *Prove that $Cont((P, \leq, \perp_P), (Q, \leq, \perp_Q))$, with the indicated structure, is indeed a CPO.*

Exercise 230. *Prove that CPOs, together with continuous functions between them, form a category.*

Recall that *Nat* and *Bool* denote the sets of natural numbers and Booleans respectively. Let $\perp \notin Nat \cup Bool$, and let $Nat_\perp$ and $Bool_\perp$ be the sets $Nat \cup \{\perp\}$ and $Bool \cup \{\perp\}$ respectively, endowed with the partial orders $\{(\perp, i) \mid i \in Nat\} \cup \{(i, i) \mid i \in Nat \cup \{\perp\}\}$ and $\{(\perp, \text{false}), (\perp, \text{true}), (\perp, \perp), (\text{true}, \text{true}), (\text{false}, \text{false})\}$ respectively.

Exercise 231. *Find a natural correspondence between the set of partial functions from X to Y and the set of continuous functions from $X_\perp$ to $Y_\perp$, where $X, Y \in \{Nat, Bool\}$. Is this correspondence bijective?*

Interpreting types
We are now ready to define $[\![_]\!] : Type \to$ [The class of CPOs]:

- $[\![\text{nat}]\!] = Nat_\perp$, $[\![\texttt{bool}]\!] = Bool_\perp$;
- $[\![s \to t]\!] = Cont([\![s]\!], [\![t]\!])$.

We let $\mathcal{HO}_\perp$ denote the *Type*-indexed set $\{[\![s]\!]\}_{s \in Type}$.
Interpreting constants

- $[\![\text{succ}]\!], [\![\text{pred}]\!] \in [\![\text{nat} \to \text{nat}]\!] = Cont(Nat_\perp, Nat_\perp)$, $[\![\text{succ}]\!](v) = \begin{cases} v+1 & \text{, if } n \in Nat \\ \perp & \text{, if } v = \perp \end{cases}$ $[\![\text{pred}]\!](v) = \begin{cases} v-1 & \text{, if } n \in Nat - \{0\} \\ 0 & \text{, if } n = 0 \\ \perp & \text{, if } v = \perp \end{cases}$
- $[\![\text{true}]\!], [\![\text{false}]\!] \in [\![\texttt{bool}]\!]$, $[\![\text{true}]\!] = \text{true}$, $[\![\text{false}]\!] = \text{false}$
- $[\![\text{cond}_t]\!] \in [\![\texttt{bool} \to t \to t \to t]\!] = Cont(Bool_\perp, Cont([\![t]\!], Cont([\![t]\!], [\![t]\!])))$, $[\![\text{cond}_t]\!](b)(v_1)(v_2) = \begin{cases} v_1 & \text{, if } b = \text{true} \\ v_2 & \text{, if } b = \text{false} \\ \perp & \text{, if } b = \perp \end{cases}$

Exercise 232. *Show that all the above are correct interpretations, in the sense that* $[\![\text{succ}]\!]$, $[\![\text{pred}]\!]$, $[\![\text{cond}_t]\!]$, $[\![\text{cond}_t]\!](b)$, $[\![\text{cond}_t]\!](b)(v)$ *are indeed continuous functions.*

Interpreting well-typed terms
We define $[\![X \triangleright E : t]\!] : [X \to \mathcal{HO}_\perp] \to [\![t]\!]$ recursively on the structure of $X \triangleright E : t$.

- $[\![X \triangleright x : t]\!](\rho) = \rho(x)$ if x is a variable;
- $[\![X \triangleright \sigma : t]\!](\rho) = [\![\sigma]\!] \in [\![t]\!]$ if σ is a constant;
- $[\![X \triangleright E_1 E_2 : t]\!](\rho) = ([\![X \triangleright E_1 : s \to t]\!](\rho))([\![X \triangleright E_2 : s]\!](\rho))$,
  if $X \triangleright E_1 : s \to t$ and $X \triangleright E_2 : s$;
- $[\![X \triangleright \lambda x : s.E : s \to t]\!](\rho)(v) = [\![X, x : s \triangleright E : t]\!](\rho[v/x])$
  for each $v \in [\![s]\!]$;
- $[\![X \triangleright \mu x : t.E : t]\!](\rho) = fix([\![\lambda x : t.E : t \to t]\!](\rho))$.

Exercise 233. *Show that the above mapping is correctly defined, in the sense that the returned values indeed belong to the specified codomains. (Hint: The proof will proceed, as usually, by induction on E; note that there is nothing to prove for the case of a μ-operator on top.)*

Let us try to explain the intuition behind the definition of $[\![X \triangleright \mu x{:}t.E{:}t]\!](\rho)$. To parallel the syntactic intuition of μ (given formally in the equational proof system which acts like a guide for all our semantic frameworks), the desired denotation would be a fixed point of the function whose law is expressed by E, that is, a fixed point of $[\![\lambda\, x{:}t.E{:}t \to t]\!](\rho)$ (since we want to be able to unwind $\mu x{:}t.E$ into $E[(\mu x{:}t.E)/x]$, i.e., into $(\lambda\, x{:}t.E)\mu x{:}t.E$). But why *the least fixed point*? Intuitively, we do not want $[\![X \triangleright \mu x{:}t.E{:}t]\!](\rho)$ to possess more information than the one provided by iterated unwindings.

Exercise 234. *The denotation of $X \triangleright \mu x{:}t.E{:}t$ could be equivalently expressed as $fix(g)$, where $g : [\![t]\!] \to [\![t]\!]$ is defined by $g(v) = [\![X, x{:}t \triangleright E{:}t]\!](\rho[v/x])$ for all $v \in [\![t]\!]$.*

As usually, we say that $\mathcal{HO}_\perp$ satisfies an equation $(\forall X)E =_t E'$, denoted $\mathcal{HO}_\perp \models (\forall X)E =_t E'$, iff $[\![X \triangleright E{:}t]\!] = [\![X \triangleright E'{:}t]\!]$.

Theorem $\mathcal{HO}_\perp$ is a model of PCF, in the sense that it satisfies all the PCF rules.

Corollary 13. *If PCF $\vdash (\forall X)E =_t E'$, then $[\![X \triangleright E{:}t]\!] = [\![X \triangleright E'{:}t]\!]$.*

Corollary 14. *If $E \to E'$, $X \triangleright E{:}t$ and $X \triangleright E'{:}t$, then $[\![X \triangleright E{:}t]\!] = [\![X \triangleright E'{:}t]\!]$.*

Corollary 15. *If $E \Downarrow V$, $X \triangleright E{:}t$ and $X \triangleright V{:}t$, then $[\![X \triangleright E{:}t]\!] = [\![X \triangleright V{:}t]\!]$.*

Proposition 26. *If V is a value and $X \triangleright V{:}t$, where $t \in \{\text{nat}, \texttt{bool}\}$, then $[\![X \triangleright V{:}t]\!] \neq \perp$.*

Theorem If $X \triangleright E{:}t$, $X \triangleright V{:}t$ and $[\![X \triangleright E{:}t]\!] = [\![X \triangleright V{:}t]\!]$, then $E \Downarrow V$.

4.8 Polymorphims

Types of Polymorphism

The word *polymorphism* comes from the Greek language, where it means "having many forms". In the study of programming languages, people use polymorphism to state that the same expression or operation or segment of code can be used with different types. Generally, there are three kinds of polymorphism in programming languages:

Overloading or coercion. For example, we can use + on related entities, such as integers, natural numbers, and/or real numbers, as well as on unrelated ones, such as strings and/or Booleans. However, typically the underlying algorithms are entirely different; think, for example, of addition on integers versus addition on float number. Many algebraic specification languages, including Maude, support operator overloading. In combination with subsorting, as most of you are already aware, operator overloading can lead to quite non-trivial theoretical and practical issues. We do not intend to discuss this kind of polymorphism in this class.

Parametric/universal polymorphism. This kind of polymorphism, also encountered under the name of "generics" in the literature, will be discussed in depth shortly. The idea underlying parametric, or universal, polymorphism is that types need not be concrete, but rather have parameters which can be *instantiated* by need. While parametric types can be quite handy in many situations, they may lead to difficult technical problems, especially in the context of type inference.

Subtype polymorphism. This is mostly found and considered indispensable in object-oriented languages, because of the intrinsic need of these languages to support inheritance. We will also discuss subtype polymorphism in this course. As it is customary in discussions on subtyping, we will set up a relatively simple formal framework, extending the simply typed λ-calculus with records.

Parametric/Universal Polymorphism

To capture the essence of parametric polymorphism in a general and uniform way, we next consider an extension of typed λ-calculus with *universal types*. This extension is known in the literature under the names *polymorphic λ-calculus*, *second-order λ-calculus*, or *system F*, and represents the foundation for many works in type theory.

Interestingly, this calculus was invented independently by two famous scientists, the logician Jean-Yves Girard and the computer scientist John Reynolds.

The extra ingredients in this calculus are *type variables*, which can be universally quantified, and *type instantiations*.

The BNF syntax for types and expressions is extended as follows (as usual, we use the color *red* for the new features):

$$\mathit{TypeVar} ::= s \mid t \mid \ldots$$

$$\mathit{Type} ::= S \mid \mathit{Type} \rightarrow \mathit{Type} \mid \mathit{TypeVar} \mid (\forall \mathit{TypeVar})\mathit{Type}$$

(where S is some set of basic (constant) types)

$$\mathit{Var} ::= x \mid y \mid \ldots$$

$$\mathit{Exp} ::= \mathit{Var} \mid \mathit{ExpExp} \mid \lambda\, \mathit{Var}{:}\mathit{Type}.\mathit{Exp} \mid \lambda\, \mathit{TypeVar}.\mathit{Exp} \mid \mathit{Exp}[\mathit{Type}]$$

Type variables s, t, etc., will be used essentially as parameters for universal types. These should not be confused with the basic types S (such as `bool`, `int`, etc.).

A universal type is quantified universally by a type variable, with the intuition that it represents a collection of types, one for each particular instance of the parameter. For example, $(\forall t)\ t \rightarrow t$ represents the universal type of the (polymorphic) identity function: it can be regarded as a collection of functions, one for each instance of the type t.

Besides the usual λ-expressions, we now allow type-parametric expressions together with a means to instantiate them. Precisely, a λ-expression $\lambda\, t.E$ represents the λ-expression E parameterized by the type t; that means that E can freely use the type t just like any other type constant (those in S), but, however, when required by an instantiation, say $(\lambda\, t.E)[T]$ where T is any type, one should be able to replace each free occurrence of t in E by T. The meaning of parametric types will be formally given as usual with equations.

In this enriched context, type assignments need to be extended appropriately to consist of not only typed variables of the form $x : T$, but also of type variables of the form t:

$$\begin{aligned}\mathit{TypeAssignment} ::= \emptyset \mid\ & \mathit{Var}{:}\mathit{Type}, \mathit{TypeAssignment} \mid \\ & \mathit{TypeVar}, \mathit{TypeAssignment}\end{aligned}$$

In what follows, we let $E, E', \ldots$ denote expressions, $T, T', \ldots$ types and $X, X', X_1, X_2 \ldots$ type assignments.

Typing Rules

For deriving the well-formed expressions $X \triangleright E{:}t$, we consider all the previous typing rules, plus:

$X, x{:}T \triangleright x{:}T$ if all the free type variables in T are in X

$$\frac{X, t \triangleright E{:}T}{X \triangleright \lambda\, t.E{:}(\forall t)T}$$

$$\frac{X \triangleright E{:}(\forall t)T}{X \triangleright E[T']{:}T[T'/t]}$$ if all the free type variables in T' are in X

It is intuitively clear what the "free type variables in a type" are—those that are not bound by any universal quantifier; also $T'[T/t]$ is the type obtained from T' by replacing each free occurrence of t in T' with T. Note that, like in the case of substitution in λ-calculus, some renamings of bound variables might be necessary in order to avoid type variable captures. All these can be formally expressed:

Exercise 235. *Define formally $Free(T)$ and $T[T'/t]$.*

Exercise 236. *Define a type checker for System F equationally, in Maude. Your type checker should take a closed term E as input and return a type T if and only if $\emptyset \triangleright E : T$ (otherwise it can return any error type—the "core dump" thing).*

Equational Rules

We consider all the previous equational rules that we defined for λ-calculus, plus the following three rules giving the expected meaning to the new features. The first two rules are congruence- or (ξ)-like rules for the new syntax, while the third gives meaning to type instantiations:

$$\frac{(\forall X, t)\ E =_T E'}{(\forall X)\ \lambda t.E =_{(\forall t)T} \lambda t.E'}$$

$$\frac{(\forall X)\ E =_{(\forall t)T} E'}{(\forall X)\ E[T'] =_{T[T'/t]} E'[T']} \text{ if } Free(T') \subseteq X$$

$$(\forall X)\ (\lambda t.E)[T'] =_{T[T'/t]} E[T'/t] \text{ if } X, t \triangleright E : T \text{ and } Free(T') \subseteq X$$

All the equations that can be derived using the rules above are well-typed:

Proposition 27. *If $(\forall X)\ E =_T E'$ is derivable with the rules above then $X \triangleright E : T$ and $X \triangleright E' : T$.*

Carrying and checking the type of equalities can be quite inconvenient in efforts to efficiently automate the applications of the equational rules above (this would be more or less equivalent to dynamic type checking). A common practice in formal definitions of typed languages is, whenever possible, to drop the subscript types of equalities and to derive instead untyped equations. The three rules above would then be:

$$\frac{(\forall X, t)\ E = E'}{(\forall X)\ \lambda t.E = \lambda t.E'}$$

$$\frac{(\forall X)\ E = E'}{(\forall X)\ E[T'] = E'[T']} \text{ if } Free(T') \subseteq X$$

$$(\forall X)\ (\lambda t.E)[T'] = E[T'/t] \text{ if } Free(T') \subseteq X$$

When type subscripts are dropped from equations, a natural question arises: are the equations consistent with the types? While the execution of the rules above becomes much simplified, the problem with dropping the types is that one could be able to derive equalities containing expressions which are not well-typed:

Exercise 237. *Give two examples of such meaningless equalities that can be derived with the untyped equational rules above. The two examples should reflect two different problems of the resulting deduction system (more precisely, of the last two rules above).*

Fortunately, the untyped equations *preserve* the well-typed-ness, which is the main result supporting and justifying type-checking:

Proposition 28. ***(Type preservation)*** *If* $(\forall X)\ E = E'$ *is derivable with the untyped equational rules and* $X \triangleright E : T$ *for some type* T*, then* $X \triangleright E' : T$*.*

In practical implementations of programming languages, the result above says that if one wants to correctly execute a program E, all one needs to do is to type-check E before execution and then ignore the types during the execution as far as the equational rules above are not violated. This allows more efficient implementations, which is precisely why most compilers have built-in type-checkers as front-ends.

Different transitional semantics can be now given by orienting and restricting the applications of the equations above accordingly. Since the obtained transition relations are included in the equational derivation relation, that is, $(\forall X)\ E \to E'$ implies $(\forall X)\ E = E'$, the type preservation property also holds for the various transitional semantics.

Notice that polymorphic λ-calculus is, in some sense, an untyped (w.r.t. type variables) λ-calculus over a typed (w.r.t. usual variables) λ-calculus. For that reason, polymorphic λ-calculus is also often called *second-order typed λ-calculus*. Instead of going through the theoretical intricacies of this calculus, we shall just give some examples showing how it can be used in parameterized programming.

Some Examples

Polymorphic conditional.

Recall that in simply typed λ-calculus we had a conditional constant for any type t, namely cond_t : $\texttt{bool} \to t \to t \to t$. In polymorphic λ-calculus we can instead define just one constant of polymorphic type, namely cond : $(\forall t)\ \texttt{bool} \to t \to t \to t$. Given a type T, it follows by the typing rules that cond$[T]$ has the type $\texttt{bool} \to T \to T \to T$. To capture the expected meaning of the conditional, two equational rules need to be added (we only consider the untyped equations here):

$(\forall X)$ cond$[T]$ true $E\ E' = E$ if $Free(T) \subseteq X$

$(\forall X)$ cond$[T]$ false $E\ E' = E'$ if $Free(T) \subseteq X$

Exercise 238. *Does the type preservation property still hold when the polymorphic conditional is added to the language? If yes, prove it. If no, give a counter-example.*

The drawback of polymorphic conditionals in particular, and of expressions of polymorphic type in general, is that one needs to instantiate them explicitly whenever one wants to apply them.

It is interesting to note that the conditional in Maude is also polymorphic (type "`show module TRUTH .`"), but that one does not need to instantiate it explicitly.

Exercise 239. *Would it be possible to change the definition of polymorphic λ-calculus so that one does not need to instantiate polymorphic expressions explicitly, that is, so that polymorphic expressions are instantiated automatically depending on the context in which they are used? Comment on the advantages and the disadvantages of such a language.*

Identity function.

Recall the `FUN` example that we used to motivate the concept of *let-polymorphism*:

```
let i = fun x -> x
in if (i true) then (i 7) else 0
```

Without let-polymorphism the program above will not type, that is, there is no way to find a type - by type inference or otherwise - to `i`, making useless the polymorphic declarations of functions.

However, in polymorphic λ-calculus, one can define the identity function explicitly polymorphic (or universal) as the λ-expression $\lambda t.\lambda x{:}t.x$ of universal type $(\forall t)\, t \to t$. Then the `FUN` program above can be given as the following λ-expression which is well-typed (has the type `nat`) in polymorphic λ-calculus:

$$(\lambda i{:}(\forall t)\, t \to t\,.\,\mathrm{cond}[\mathrm{nat}]\ (i[\texttt{bool}]\ \mathrm{true})\ (i[\mathrm{nat}]\ 7)\ 0)\ (\lambda t.\lambda x{:}t.x)$$

Function composition.

Function composition is a standard operation in functional programming languages. One would like to generically compose two functions $f : t_1 \to t_2$ and $g : t_2 \to t_3$ to obtain a function $t_1 \to t_3$, for *any* types t_1, t_2 and t_3. The corresponding λ-expression in polymorphic λ-calculus is

$$\lambda t_1.\lambda t_2.\lambda t_3.\lambda f{:}t_1 \to t_2.\lambda g{:}t_2 \to t_3.\lambda x{:}t_1.g(fx)$$

and will type to $(\forall t_1)(\forall t_2)(\forall t_3)(t_1 \to t_2) \to (t_2 \to t_3) \to t_1 \to t_3$.

Exercise 240. *Derive the type of the function composition expression formally, using the typing rules.*

Exercise 241. *Would it make any sense to introduce the parameter types by need, that is, to define function composition as*

$$\lambda t_1.\lambda t_2.\lambda f{:}t_1 \to t_2.\lambda t_3.\lambda g{:}t_2 \to t_3.\lambda x{:}t_1.g(fx)$$

Type this expression and comment on its advantages and disadvantages in contrast to the standard polymorphic function composition above.

On Recursion

Recall that in simply-typed λ-calculus there was no way to type an expression of the form $\lambda x{:}T.xx$, for any type T. Let us try to represent this expression within System F.

First, note that $\lambda x{:}(\forall t)\, t \to t.xx$ is not a good choice, because it would not type. Indeed, trying to type it, we need to find some type T such that $x{:}(\forall t)\, t \to t \triangleright xx : T$; then the type of the entire expression would be $((\forall t)\, t \to t) \to T$. Trying to type the expression xx, one gets the type constraint $((\forall t)\, t \to t) \to T = (\forall t)\, t \to t$, which, unfortunately, does not admit a solution even in the enriched context of polymorphic types.

The solution is to use a type instantiation in order to change the type of the first x to one which can be applied on the second x: the expression $\lambda x : (\forall t)\ t \to t\ .\ x[(\forall t)\ t \to t]\ x$, say E, will type to $((\forall t)\, t \to t) \to ((\forall t)\, t \to t)$.

Note, however, that we cannot type anything like similar to the untyped ω combinator, namely $(\lambda x.xx)(\lambda x.xx)$, or to the untyped fixed-point operators. As a matter of fact, we cannot simulate recursion in System F without μ.

Exercise 242. *Try, and fail, to simulate recursion in system F. You should see that, despite the fact that one may use instantiation to type expressions previously untypable in simply typed λ-calculus, one actually cannot use polymorphism for dealing with recursion.*

For that reason, like we did in simply typed λ-calculus, to deal with recursion we extend our calculus with the usual μ-abstraction. Assume the same usual typing/equational rules for μ.

More Examples

Polymorphic lists.

Recall that one of the major problems in typing `FUN` programs in CS422 was the fact that lists were *not* polymorphic. Because of that reason, we were not able to regard, for example, the empty list as both a list of integers and as a list of Booleans.

Polymorphic λ-calculus, fortunately, supports polymorphic lists very naturally. In what follows we add the list type construct, give the signature of the usual list operators, and show how polymorphic lists can be used in practice. Here we are not concerned with how lists are represented, nor with how the list operators are defined: we just assume them given. We will later discuss Church lists, showing that all the list operators can be defined from more basic principles.

Let us first add the type construct for lists:

$$\mathit{Type} = \ldots \mid \text{list } \mathit{Type}$$

The usual operators on lists can be now given as just constants of universal types:

$$\begin{aligned}
&\text{nil} : (\forall t)\ \text{list } t \\
&\text{cons} : (\forall t)\ t \to \text{list } t \to \text{list } t \\
&\text{nil?} : (\forall t)\ \text{list } t \to \texttt{bool} \\
&\text{car} : (\forall t)\ \text{list } t \to t \\
&\text{cdr} : (\forall t)\ \text{list } t \to \text{list } t
\end{aligned}$$

Supposing these list operations already defined, let us define a polymorphic *map* operator, which takes as arguments a list and a function and applies that function on each element of the list:

$$\begin{aligned}
&\lambda t_1 . \lambda t_2 . \lambda f : t_1 \to t_2. \\
&\quad \mu\, m : \text{list } t_1 \to \text{list } t_2 . \\
&\qquad \lambda\, l{:}\text{list } t_1. \\
&\qquad\quad \text{cond}[\text{list } t_2]\ (\text{nil?}[t_1]\ l) \\
&\qquad\qquad\qquad (\text{nil}[t_2]) \\
&\qquad\qquad\qquad (\text{cons } [t_2]\ (f\ (\text{car}[t_1]\ l))\ (m\ (\text{cdr}[t_1]\ l)))
\end{aligned}$$

Exercise 243. *Derive the type of the map expression above using the formal type inference rules.*

Exercise 244. *Define and type a reverse function on polymorphic lists.*

Church Booleans.

Recall the Church Booleans from untyped λ-calculus. They already had a polymorphic intuitive meaning: true and false were the first and the second projection functions, respectively, expecting two arguments of the same type. With the machinery of polymorphic λ-calculus, we can take the Boolean type to be the following universal type:

$$\texttt{bool}_\lambda := (\forall t)\ t \to t \to t$$

and the two Boolean constants to be:

$$\begin{aligned}
&\text{true} := \lambda t.\lambda x{:}t.\lambda y{:}t.x \\
&\text{false} := \lambda t.\lambda x{:}t.\lambda y{:}t.y
\end{aligned}$$

Note that the two expressions above have indeed the type $\texttt{bool}_\lambda$. The logical operators can now be naturally defined. For instance, `not` can be defined as $\lambda\, b{:}\texttt{bool}_\lambda.\lambda\, t.\lambda\, x{:}t.\lambda\, y{:}t.b[t]\, y\, x$.

Exercise 245. *Define and type the other Church Boolean operators.*

Church numerals.

Recall that, in untyped λ-calculus, Church numerals were characterized by their latent application to a *successor* function s and a *zero* element z, which would result in applying s to z a certain number of times. Therefore, the expected type of s would be $t \to t$, where t is the type of z. Thus we can define the polymorphic type of Church numerals as

$$\text{nat}_\lambda := (\forall t)\ (t \to t) \to t \to t$$

Number n can then be encoded as the expression of type nat_λ

$$n_\lambda\ := \lambda\, t\ .\ \lambda\, s{:}t \to t\ .\ \lambda\, z{:}t\ .\ s(s \ldots (s\ z) \ldots),$$

with n applications of s. All the usual arithmetical operations on numbers can now be defined. For instance,

$$\text{succ}_\lambda\ := \lambda\, n{:}\text{nat}_\lambda\ .\ \lambda\, t\ .\ \lambda\, s{:}t \to t\ .\ \lambda\, z{:}t\ .\ s\ (n[t]\ s\ z)$$
$$+_\lambda\ := \lambda\, n{:}\text{nat}_\lambda\ .\ \lambda\, m{:}\text{nat}_\lambda\ .\ \lambda\, t\ .\ \lambda\, s{:}t \to t\ .\ \lambda\, z{:}t\ .\ n[t]\ s\ (m[t]\ s\ z).$$

Exercise 246. *Define the polymorphic multiplication and power.*

Church lists.

Following a similar idea as for Church numerals, one can define Church lists. A list is regarded through its latent application to a "binary operation" f and an "initial value" v, which would result in iteratively applying the binary operation as follows: first to the last element of the list and v, then to the last but one element and the previous result, and so on, until the list is exhausted. For instance, the list $[a, b, c]$ is encoded as $\lambda\, f\ .\ \lambda\, v\ .\ f\ a\ (f\ b\ (f\ c\ v))$.

For each type t, let us define the polymorphic type

$$\text{list}\ t := (\forall p)\ (t \to p \to p) \to p \to p$$

Note, however, that we want the list operations to be themselves polymorphic. For example, we want the type of cons to be $(\forall t)\ t \to \text{list}\ t \to \text{list}\ t$. We can now define the usual list operators quite naturally:

$$\text{nil} := \lambda\, t\ .\ \lambda\, p\ .\ \lambda\, f{:}t \to p \to p\ .\ \lambda\, v{:}p\ .\ v$$
(the type of nil is $(\forall t)\ \text{list}\ t$),

$$\begin{aligned}\text{cons}\ := \lambda\, t\ .\ \lambda\, \text{head}{:}t\ .\ \lambda\, \text{tail}{:}\text{list}\ t\ .\\ \lambda\, p\ .\ \lambda\, f{:}t \to p \to p\ .\ \lambda\, v{:}p\ .\\ f\ \text{head}\ (\text{tail}[p]\ f\ v)\end{aligned}$$
(the type of cons is $(\forall t)\ t \to \text{list}\ t \to \text{list}\ t$),

$$\text{nil?} := \lambda\, t\ .\ \lambda\, l{:}\text{list}\ t\ .\ l[\texttt{bool}_\lambda]\ (\lambda\, x{:}t\ .\ \lambda\, y{:}\texttt{bool}_\lambda\ .\ \text{false}\ )\ \text{true}$$
(the type of nil? is $(\forall t)\ \text{list}\ t \to \texttt{bool}_\lambda$),

$$\text{car} := \lambda\, t\ .\ \lambda\, l{:}\text{list}\ t\ .\ l[t]\ (\lambda\, x{:}t\ .\ \lambda\, y{:}t\ .\ x)\ (\text{error}[t])$$
(the type of car is $(\forall t)\ \text{list}\ t \to t$)

Note that car is intended to be a *partial* function, undefined on the empty list; hence we introduced a polymorphic constant error, of type $(\forall t)t$. This constant is also useful for any other partial function definitions. Some prefer to define "error for type t" as an infinite recursion, for example $\mu x : t.x$; in other words, they either replace each instance error[t] by $\mu x : t \,.\, x$, or otherwise add an equation error $= \lambda t \,.\, \mu x : t \,.\, x$. The drawback of this adhoc convention is that the evaluation of expressions applying car *will not terminate* under call-by-value evaluation strategy.

Exercise 247. *Define and formally type the* cdr *operator.*

Giving System F an Executable Semantics

Let us now focus on the original formulation of System F, namely the one without recursion (μ). Let us orient all the equations left-to-right, thus obtaining a transitional semantics ($\rightarrow$) of System F. One of the most famous results of System F is the following:

> ***Theorem (Girard).*** *(very hard) In System F without μ, the transition relation $\rightarrow$ is confluent and terminates.*

Exercise 248. *Define System F equationally in Maude. Your definition should take a polymorphic λ-expression and evaluate it to its unique normal form.*
Hint. *You may need to use the de Bruijn technique to properly avoid variable captures; note that variable captures can now appear both at the λ-expression level and at the type level. Do not worry whether your normal form contains de Bruijn indexes.*

Type inference/reconstruction The problem of *type inference* can be stated in the polymorphic λ-calculus framework as follows.

> *Given any untyped λ-expression E, is it the case that E is typable in System F?*

In other words, is there any polymorphic λ-expression E_F in System F such that erase(E_F) = E? Here, the operator erase just loses any type information and can be formally defined as follows:

$$
\begin{aligned}
&\text{erase}(x) = x,\\
&\text{erase}(\lambda\, x{:}T.E) = \lambda\, x.\text{erase}(E),\\
&\text{erase}(E_1 E_2) = \text{erase}(E_1)\text{erase}(E_2),\\
&\text{erase}(\lambda\, t.E) = \text{erase}(E),\\
&\text{erase}(E[T]) = \text{erase}(E)
\end{aligned}
$$

This problem, open for more than 20 years, was finally proven to be *undecidable*. Algorithms were developed to partially solve this problem, some of them based on Huet's partial algorithms for *higher-order unification* (another undecidable problem).

Subtype Polymorphism

We next focus on the other major approach to polymorphism, namely *subtype polymorphism*, which is mostly encountered in the context of modern object-oriented programming languages.

To stay focused on the major aspects of subtype polymorphism, we here introduce a very simply λ-calculus language extended with various features that reflect most of the interesting issues related to subtype polymorphism encountered in other languages or formalisms.

Simply Typed λ-Calculus with Records

We extend simply typed λ-Calculus by adding *records*. Let $Field$ be a countably infinite set, disjoint from *Var*, and let us extend types and expressions as follows:

$Type ::= S \mid Type \rightarrow Type \mid \{Field{:}Type, \ldots, Field{:}Type\}.$

$Exp ::= Var \mid Exp\ Exp \mid \lambda Var{:}Type.Exp \mid \{Field = Exp, \ldots, Field = Exp\} \mid Exp.Field$

Thus a record type is a set of typed attributes, each represented as a (*Field*, *Type*) pair. For example, we can declare a record type, say *person*, as $\{name : string, age : nat, height : nat, \ldots\}$. Two kinds of expressions are introduced for dealing with records:

- *record expressions* (written like sets of equalities, assigning expressions to the fields of a record), and
- *field accessing* expressions.

The first one creates a record by assigning a value to every attribute, while the second fetches the value of an attribute of a record. For instance, the expression

$\{name = \text{'John Smith'}, age = 27, height = 180, \ldots\}$

defines a record variable, say *john*. We can get the values of its fields with expressions like *john.name*, *john.age*, etc.

Records inherently bring the issue of *subtyping*. Intuitively, if a context requires a record R of a certain kind, it should be the case that a record R' having *more information* than needed be still suitable for that context. Thus we would like to be able to write

$(\lambda\, p : \{age : nat\}\ .\ (p.age))$

$\{name=\text{'John Smith'}, age=27, height=180, \ldots\}$

However, with the previous typing system, this is not allowed since the argument type is not *exactly the same* as the parameter type. To avoid this kind of unnecessary constraints, but still maintain a rigorous typing discipline, we introduce the following important notion of subtyping, first intuitively and then rigorously:

> *Intuitive definition of subtyping:* Given two types t_1 and t_2, we say that t_1 *is a subtype of* t_2, written $t_1 \leq t_2$, iff t_1 *has at least the same information as* t_2, or, in other words, a value of type t_1 can be used wherever a value of type t_2 is expected; one also may say that t_1 *is more concrete than* t_2, or that t_2 *is more general than* t_1.

Subtyping Rules

Now we set up the rules that will allow us to formally derive subtype relationships of the form $t_1 \leq t_2$, which will be further needed to define the typing system:

(≤-reflexivity) $\quad t \leq t$

(≤-transitivity) $\quad \dfrac{t_1 \leq t \quad t \leq t_2}{t_1 \leq t_2}$

(≤-arrow) $\quad \dfrac{t_2 \leq t_1 \quad t'_1 \leq t'_2}{t_1 \rightarrow t'_1 \leq t_2 \rightarrow t'_2}$

(≤-record) $\quad \dfrac{t_1 \leq t'_1 \quad \ldots \quad t_m \leq t'_m}{\{l_1{:}t_1, \ldots, l_n{:}t_n\} \leq \{l_1 : t'_1, \ldots, l_m{:}t'_m\}}$ if $m \leq n$

The first two rules are clear. To understand the (≤-arrow) rule, let us suppose a context which expects a value of type $t_2 \rightarrow t'_2$ but actually receives a value V of type $t_1 \rightarrow t'_1$. In a presumptive later use, V might

be applied to an argument W of type t_2. To assure that V can handle W, one needs that every inquiry that V makes to its argument (expected to be of type t_1) be answered by W, i.e., that W provides at least as much information as a value of type t_1; thus $t_2 \leq t_1$. Furthermore, the result of applying V to W is a value of type t'_1, while a value of type t'_2 would be expected; thus $t'_1 \leq t'_2$.

The rule (≤-record) says two important things: first, that the subtype record must include the fields of the supertype record, and second, that the types of those fields in the subtype record are subtypes of the types of those fields in the supertype record. Both these facts are needed in order for a value of subtype record to be used in contexts where values of supertype record are expected.

The next two properties of the subtype relation say that one can only derive meaningful subtypings. These properties may be used later to prove important properties of type systems supporting subtyping.

Exercise 249. *If $t \leq t_2 \to t'_2$ then t has the form $t_1 \to t'_1$ such that $t_2 \leq t_1$ and $t'_1 \leq t'_2$.*

Exercise 250. *If $t \leq \{l_1 : t'_1, \ldots, l_m : t'_m\}$ then t has the form $\{l_1 : t_1, \ldots, l_n : t_n\}$ with $m \leq n$ and $t_i \leq t'_i$, $i \in \{1, \ldots, m\}$.*

Type System

Building upon the subtype relation formally defined above, we can now give subtype-flexible rules for deriving types:

$$\text{(subsumption)} \quad \frac{X \triangleright E : t_1}{X \triangleright E : t_2} \quad \text{if } t_1 \leq t_2$$

$$\text{(record)} \quad \frac{X \triangleright E_1 : t_1 \quad \ldots \quad X \triangleright E_n : t_n}{X \triangleright \{l_1 = E_1, \ldots, l_n = E_n\} : \{l_1 : t_1, \ldots, l_n : t_n\}}$$

$$\text{(field access)} \quad \frac{X \triangleright E : \{l_1 : t_1, \ldots, l_n : t_n\}}{X \triangleright E.l_j : t_j} \quad \text{if } j \in \{1, \ldots, n\}$$

(subsumption) allows us to lift the type of an expression to any supertype, thus formally justifying our informal claim that "expressions of type t_1 can be used in any context where expressions of supertype t_2 are expected".

(record) allows us to derive a type of a record from the names and the types of its fields, while (field access) allows us to derive the type of a field once the type of the entire record is known.

Proposition 29. *If $X \triangleright (\lambda x : t.E) : t_1 \to t_2$ then $t_1 \leq t$ and $X, x : t \triangleright E : t_2$.*

Proposition 30. ***(Substitution.)*** *If $X, x : s \triangleright E : t$ and $X \triangleright F : s$ then $X \triangleright E[x \leftarrow F] : t$.*

Equational Rules

The following natural equational rules are added to those of simply-typed λ-calculus. We here assume the untyped variants of equational rules, which, as usual, will rise the question of type preservation:

$$(\forall X)\ \{l_1 = E_1, \ldots, l_n = E_n\}.l_i = E_i \text{ for all } i \in \overline{1, n}$$

$$\frac{(\forall X)\ E = E'}{(\forall X)\ E.l = E'.l} \text{ for any field } l$$

$$\frac{(\forall X)\ E_n = E'_n}{(\forall X)\ \{(l_i = E_i)_{i=\overline{1,n-1}}, l_n = E_n\} = \{(l_i = E_i)_{i=\overline{1,n-1}}, l_n = E'_n\}}$$

Exercise 251. *Using the rules for typing and for equation derivation above, show formally that the expression*

$$(\lambda\, p{:}\{age{:}\mathrm{nat}\}\,.\,(p.age))$$
$$\{name = \text{`John Smith'},\ age = 27,\ height = 180\ldots\}$$

types to nat *and is equal to 27.*

Proposition 31. ***(Type preservation.)*** *If* $(\forall X)\ E = E'$ *is derivable and* $X \triangleright E{:}t$ *then* $X \triangleright E'{:}t$.

As an operational consequence of the property above, we obtain the so-called "subject reduction" property for the transition relation $\rightarrow$ obtained by orienting the equations:

Corollary 16. *If* $X \triangleright E{:}t$ *and* $E \rightarrow E'$ *then* $X \triangleright E'{:}t$.

Subtyping and Other Language Features

Subtyping occurs many places in computer science. Therefore, it is important to understand how subtyping interacts with different other features that also appear often in practice.

We next investiage the relationship between subtyping and several other interesting and important programming language features, including built-in types, lists, references, arrays, type casting and a bit with objects. The list of features can continue. In fact, it is customary that programming language designers analyze the effectiveness of new conceptual developments by studying their interaction with subtyping.

Built-in Types

Basic subtyping may be given by many other features besides records. As you know, programming languages usually provide *built-in*, also called *basic*, types, such as bool, int, real, etc.

Programmers find it very convenient to assume subtyping on some basic types, e.g., bool $\leq$ int. In fact, in many languages, the constant *true* is represented as integer 1 and $false$ as 0. This way, one can use a bool expression whenever an integer expression is expected. For example, with a Boolean variable b, one can write an expression: $scale * b$, which evaluating to wither 0 or *scale*, depending on whether b is false or true. To support this feature, we need that bool $\leq$ int. Other common subtype relationships are nat $\leq$ int, or int $\leq$ real.

Lists

We can also introduce list types into our type system:

$$Type ::= \ldots \mid list\ Type$$

The subtyping rule for lists is

$$\frac{t_1 \leq t_2}{list\ t_1 \leq list\ t_2}$$

Although this rule seems straightforward and intuitive, it only works when we build and use lists without applying any change on them. As we will next see in the discussion of references and arrays, some restrictions are raised by changing values at given locations in the list.

References

Most languages allow assignments of new values to existing, i.e., already declared, names. In many languages this is supported by introducing the notion of *reference* (also called *location* or *cell index*). We next extend our simply typed λ-calculus to support references and assignments:

$$Type ::= \ldots \mid Ref\ Type \mid Unit$$
$$Exp ::= \ldots \mid ref\ Exp \mid !Exp \mid Exp := Exp$$

Therefore, we allow explicit types for references. For example, *Ref* (nat → nat) is a type for references to, or locations storing, functions nat → nat. Values of reference types are just like any other values in the languages, that is, they can be passed to and returned as results of functions.

Together with refereces and assignments, *side effects* are unavoidable. *Unit* is the type of expressions, such as assignments, that are intended to be used just for their side effects; these are not supposed to evaluate to any particular value. In other words, one may read *Unit* as "no type".

Three new expression constructors are introduced.

ref E evaluates to a new location, say L, where the expression E is also stored; this is equivalent to the "new" construct in object oriented languages, or to "malloc" in C. Depending upon the particular evaluation strategy desired in one's language, the expression E stored at L may be already evaluated to some value. The language construct *ref* is also called *referencing*.

$!E$, which expects E to evaluate to a location, say L, returns the expression/value stored at L. The language construct ! is also called *dereferencing*.

$E := F$ first evaluates E to a location, say L, and then stores F at location L. Again, depending upon the desired evaluation strategy, one may first evaluate F to some value V and then store V at L.

To give a full equational semantics of our λ-calculus language extended with references, we would need to follow the same approach as in `FUN`, that is, to introduce infrastructure for *stores*, to evaluate λ-abstractions to closures, etc. We do not do this here, because it would essentially duplicate what we've already done in the definition of `FUN`. Instead, we just focus on aspects related to typing in the context of references.

Typing references.

The typing rules for references are straighforward:

$$\frac{X \triangleright E : t}{X \triangleright ref\ E : Ref\ t}$$

$$\frac{X \triangleright E : Ref\ t}{X \triangleright !E : t}$$

$$\frac{X \triangleright E : Ref\ t \quad X \triangleright E' : t}{X \triangleright E := E' : Unit}$$

The subtleties of typing in the context of references come from their interaction with subtyping.

Subtyping and references.

The question to be asked here is how, and under what conditions, can one derive subtyping relations of the form $Ref\ t_1 \leq Ref\ t_2$? In other words, when can a reference to an expression/value of type t_1 be safely used in a context where a reference to an expression/value of type t_2 is expected?

There are two concerns regarding the use of a reference expression/value R of type $Ref\ t_1$ when a reference expression/value of type $Ref\ t_2$ is expected:

1. If R is dereferenced (read) in a context, such as, for example, in $3+!R$, the expression/value stored at R should be safely usable in that context, where an expression/value of type t_2 is expected; therefore, $t_1 \leq t_2$.

2. If R is assigned (written) in a contex, for example using an assignment of the form $R := E$ with E of type t_2, then since there can be other places in the program expecting (by dereferencing R) the expression/value at location R to have the declared type t_1, one deduces that $t_2 \leq t_1$.

Therefore, the only safe possibility to have $Ref\ t_1 \leq Ref\ t_2$ is that $t_1 = t_2$, which obviously implies $Ref\ t_1 = Ref\ t_2$. We conclude from here that reference types admit no proper subtypes.

While this is an elegant conclusion in what regards the implementation of a type system, because one basically needs to do *absolutely nothing* to support it, it is very important to understand the deep motivations underlying it.

> *Don't speak unless you can improve on the silence*
>
> Spanish Proverb

While this may seem rather straightforward, when references and types live together in a language almost nothing is simple enough to avoid misunderstandings or subtle problems.

Arrays

Arrays can be very easilly added to a language, at least in what regards their typing. For example, in our language we can extend both the types and the expressions as follows:

$$Type ::= \ldots \mid \text{Array } Type$$

$$Exp ::= \ldots \mid Exp[Exp] \mid Exp[Exp] := Exp$$

Essentially, arrays are similar to references. In fact, in many languages, arrays are nothing but references pointing to the first location of a block of memory. Consequently, by an analysis of subtyping similar to that for references, we can infer that the only reasonable way to have Array $t_1 \leq$ Array t_2 is that $t_1 = t_2$, maning that array types have no proper subtypes. This conclusion can similarly be applied to lists when one is allowed to write at specific locations in lists (with statements of the form car $E := E'$, car $(cdr\ E) := E'$, etc.).

Some programming language designers, while still adopting type systems for their languages, find some of the above (static) subtyping restrictions too strong, arguing that they limit the use of references, arrays, lists, etc. In fact, designers of programming languages tend to easily become "religious"; for example, those designing untyped or dynamically typed languages think of static typing as a serious impediment the programmers have to deal with in one way or another.

Some languages split the task of type checking into a static component and a dynamic one. For example, Java only takes the deferencing (reading) into account during its static type checking, and checks every write at runtime to maintain the type safety. But this is also considered by some researchers as a design flaw ...

Type Casting

Type casting allows one to assign to terms types that type checkers may not be able to find statically. One can regard casting as a type annotation which helps the type checker analyze the program. At some extent, unless a language admits some form of dynamic type checking that can lead to different executions when expressions are evaluated to values of different types (such as instanceOf checks), one can regard all the type declarations as just annotations to help a particular static program analysis tool, the type checker, analyze the program.

The corresponding syntax for type casting in our language can be:

$$Exp := \ldots \mid \langle Type \rangle\ Exp$$

We do not discuss the formal (equational) semantics of casting here, but, intuitively, $\langle t \rangle E$ is simply equal to E when E can be shown of type t. In some cases one may be able to show statically that E has type t or that E cannot have type t, in which case the type casting may be simply dropped or a static error reported, but

in general the problem is undecidable. In practice, a dynamic checker, or a monitor, is inserted to ensure that the type of E is indeed as claimed; if not, an error or an exception is generated. The benefit of this dynamic check is that the static type checker can then assume that E has indeed the claimed type t and can therefore continue unperturbed the type checking process on the rest of the program. The typing rule of casting is then simply as follows:

$$\frac{X \triangleright E : t_2}{X \triangleright \langle t_1 \rangle E : t_1}$$

Therefore, as far as an expression type checks to any type, it can also be cast to any other type, but a dynamic check still needs to be performed. For example, $(\langle\{age : nat\}\rangle x).age$ is assumed of type *nat* for static type checking purposes, but it may cause a runtime error if the type t of x is not a record type containing a field age of type nat.

Exercise 252. *Give a simple program containing the expression* $(\langle\{age : nat\}\rangle x).age$ *which types and executes correctly, but which would not type if one replaced the expression above by just* $x.age$.

To facilitate casting, many languages have a top type, like the *Object* class in Java.

Syntax-Directed Subtyping

The subtyping rules, together with the typing rules in the context of subtypes, gave us a logical means to entail type judgements of the form $X \triangleright E : t$. However, note that the type system is bound to the fact that all three components of the type judgement, namely X, E, and t, need to be available.

For some typed frameworks, for example simply typed λ-calculus, as we know it is relatively trivial to translate the typing rules into a *typing algorithm*, calculating for a given expression E in a given type environment X a type t, if the expression is indeed well-typed, and reporting an error otherwise.

In the context of subtyping, one needs some additional work to obtain an effective typing algorithm.

Let us first understand what are the complications that appear when one tries to type a program within a language with subtyping.

Unlike typing in the context of our previous, non-subtyped languages, in the context of subtypes the application of the typing rules is *not deterministic*, i.e., *not syntax-directed*. Consider for example the λ-expression, say *mkYounger*,

$$\lambda\, x : \{age : nat, height : nat\}\ .$$
$$\{age = x.age - 20,\ height = x.height + 3\}$$

and suppose that we would like to derive that it can have the type $\{age : nat, height : nat\} \rightarrow \{age : nat\}$, say t. There are two different ways to do it:

1) One can first apply typing rules for as long as possible, and then "ask for the help" the subtyping rules:

(1) $\dfrac{\dots}{\emptyset \triangleright mkYounger{:}\{age{:}nat,height{:}nat\}\rightarrow\{age{:}nat,height{:}nat\}}$

(2) $\dfrac{\dots}{\{age{:}nat,height{:}nat\}\rightarrow\{age{:}nat,height{:}nat\}\leq\{age{:}nat,height{:}nat\}\rightarrow\{age{:}nat\}}$

(3) $\dfrac{(1)\quad(2)}{\emptyset \triangleright mkYounger{:}t}$

2) Alternatively, a subtyping rule can be applied earlier, before typing the outermost expression:

(1) $\dfrac{\dots}{x{:}\{age{:}nat, height{:}nat\} \triangleright \{age{=}x.age-1, height{=}x.height-10\}{:}\{age{:}nat, height{:}nat\}}$

(2) $\dfrac{}{\{age{:}nat, height{:}nat\} \leq \{age{:}nat\}}$

(3) $\dfrac{(1) \quad (2)}{x{:}\{age{:}nat, height{:}nat\} \triangleright \{age{=}x.age-1, height{=}x.height-10\}{:}\{age{:}nat\}}$

(4) $\dfrac{(3)}{\emptyset \triangleright mkYounger : t}$

In order for the entailment system to immediately provide a typing algorithm, one would like to have a well-defined, deterministic way to apply the typing rules by just examining the syntax of the program without any search. This is also called *syntax-directed* typing.

Of course, one may argue that, in the above example, chosing one of the two possible computations is not problematic, thanks to their confluence; hence one could pick, for instance, the lexicographically lower computation. But there are some rules in our context of subtyping that are really non-syntax-directed, in the sense that one has no clue where to find the catalyzers needed to continue computation.

Undeterministic subtyping rules.

Consider for instance the rules:

(≤-subtyping) $\dfrac{t_1 \leq t \quad t \leq t_2}{t_1 \leq t_2}$ How to pick a t when applying this rule?

(subsumption) $\dfrac{X \triangleright E : t_1 \quad t_1 \leq t_2}{X \triangleright E : t_2}$ How to pick a t_1? Structure/syntax of E is *not* taken into account.

Can one change the rules so that typing becomes syntax-directed and mechanical? The answer is yes.

Eleminating the bad rules.

A first interesting observation is that the (≤-transitivity) rule is *not needed*. The intuitive reason underlying this claim is that the subtyping relation derived using the other subtyping rules, excluding (≤-transitivity), *is already transitive*.

Exercise 253. *Prove formally the claim above.*

A second important observation is that we can also eliminate the other problematic rule, (subsumption), by carefully inspecting its usage. This rule really only needs to be used to type check function applications: $(\lambda\, x{:}t\ .\ E)E'$ requires that the type of E' is a subtype of t. Then one can simply eliminate the problematic (subsumption) rule and instead modify the λ-abstraction application rule into the following rule. To reflect the fact that the newly obtained type system is different from the original one, we use a different but closely related syntax, namely $X \trianglerighteq E : t$, to denote the type judgements derived with the latter type system:

(≤-application) $\dfrac{X \trianglerighteq E_1 : t_1 \to t_1',\ X \trianglerighteq E_2 : t_2,\ t_2 \leq t_1}{X \trianglerighteq E_1 E_2 : t_1'}$

Note that the (≤-application) rule above is syntax-driven, in the sense that the syntax of the goal type judgment (an application), tells us precisely what to do next: calculate the type of the two expressions involved and then derive the corresponding subtype relation.

Proposition 32. *Prove that the resulting typing system above has the following properties:*

- *For each expression E and type assignment X, one can derive $X \trianglerighteq E : t$ for a at most one type t;*

- ***(Soundness)*** *$X \trianglerighteq E : t$ implies $X \triangleright E : t$;*
- ***(Completeness)*** *$X \triangleright E : t$ implies $X \trianglerighteq E : t'$, where $t' \leq t$.*

Therefore, in terms of the original type system, the new type system derives *the most concrete type* of an expression.

Syntax-directed subtyping algorithm.

The important proposition above immediately provides an algorithm to decide whether, under a type assignment X, an expression E has a type t:

> *First derive $X \trianglerighteq E : t'$ and then check whether $t' \leq t$; if any of the two fails than E cannot have the type t in the type environment X.*

Exercise 254. *Define the syntax-directed typing algorithm above in Maude. You need to define both the subtyping relation and the typing rules equationally. Your definition should be executable.*

Typing Conditionals

In simply typed λ-calculus we had to define one conditional constant $cond_t : t \to t \to \texttt{bool}$ for each type t. With subtyping, like with universal polymorphism, we only need to define one generic conditional. Precisely, we can add a Top constant type defined to be the *most general* type together with a subtyping rule

$$\text{(top)} \quad \frac{-}{t \leq \text{Top}}$$

and then define just one constant conditional expression, cond : $\texttt{bool} \to \text{Top} \to \text{Top} \to \text{Top}$. (This actually follows a general technique to simluate universal polymorphism with subtyping).

Previously, the rule for typing conditionals was the following:

$$\frac{X \triangleright C : \texttt{bool}, \quad X \triangleright E_1 : t, \quad X \triangleright E_2 : t}{X \triangleright \text{cond}\, C\, E_1\, E_2 : t}$$

This rule still works in the context of subtyping, but note, however, that one is expected to use subsumption to lift the possibly more concrete types of E_1 and E_2 to some *common* supertype. In fact, the conditional expression can be derived any type that is a supertype of the most concrete types of E_1 and E_2. For example, consider the following expression ("keep one's height if one is older than 10, otherwise keep one's weight"):

> cond $(x.age > 10)$
>
> $\{name = x.name,\ age = x.age,\ height = x.height\}$ $\{name = x.name,\ age = x.age,\ weight = x.weight\}$

One cannot apply directly the typing rule of the conditional, because the two branches have different types. But by subsumption one can first calculate some common type to both branches, such as $\{name : String,\ age : nat\}$, or $\{name : String\}$, or $\{age : nat\}$, or even $\{\}$, and then apply the typing rule for conditional.

The limitations of the typing rule for conditional above becomes clear in the context of syntax-directed typing, where one calculates the most concrete types of the two branches and then one wants to calculate a type for the conditional.

First, note that under syntax-directed typing, the typing rule for conditional above is close to useless, because the most concrete types of the branches may be different.

Second, what should the type of the conditional actualy be, knowing the types t_1 and t_2 of its branches? Since we want to calculate the type of the conditional *statically*, unless using sophisticated theorem provers

(which typically do not to scale), we cannot know which of the two branches would be taken during an actual execution.

One possibility would be two consider both branches separately and ensure that the program would type regardless of which branch is taken. Unfortunately, the number of possibilities to analyze doubles with each conditional in the program. Therefore, despite its precision in analysis, this exhaustive approach would hardly have any practical use.

An extreme possibility would be to say that the type of the conditional is Top, because we do not know which of the two branches is taken. The problem with this aggressive approach is that all the type information about the two branches is lost, so one may need casting to explicitly concretize the type of the conditional to the expectations of a particular context.

The practical solution here is to accept loosing some precision but not all of it. Any common supertype of t_1 and t_2 is clearly better than Top. This suggests that we should actually pick the *least common supertype* of t_1 and t_2, written $t_1 \vee t_2$, as the type of the conditional; since the subtype relation is a partial order, the least common supertype of t_1 and t_2, also called their *join* type, is nothing but the least upper bound of t_1 and t_2 with respect to the subtyping relation. With this, the syntax-driven typing rule of the conditional is the following:

$$\frac{X \trianglerighteq C : \texttt{bool} \quad X \trianglerighteq E_1 : t_1, \quad X \trianglerighteq E_2 : t_2}{X \trianglerighteq \text{cond}\, C\, E_1\, E_2 : t_1 \vee t_2}$$

Thus our sample conditional expression discussed above types to $\{name : String,\ age : nat\}$.

Exercise 255. *(continuation of Exercise 254) Define (also in Maude) the join operation on types and add the universal conditional to the language together with its syntax-driven typing.*

Subtypes and Objects

Objects and the object-oriented (OO) paradigm form an important player in today's theory and practice of programming languages. Objects are considered by many software engineers crucial in the process of software development, because they improve the modulality and reusability of code.

Essentially, an object *encapsulates* a state and provides the outside world an *interface* to partially manipulate its state, like for example to access or modify parts of it. We next show how objects and some OO concepts can be supported, without any additional machinery, by typed λ-calculus with subtyping and references.

That should not mean that we are actually claiming that typed high-order languages should replace OO languages. Similarly, the fact that all programming language paradigms can be supported by rewriting does not mean that rewriting can replace all these.

Syntax

The syntax is left almost unchanged:

$$Type ::= \ldots \mid Ref\, Type \mid Unit$$

$$Exp ::= \ldots \mid ref\, Exp \mid !\, Exp \mid Exp := Exp \mid ()$$

The only increment here is () that is used for calling functions without paramenters. This feature is not required, since we can pass a dummy argument that is ignored in the function, but it is more natural and concise for programming. The type of () is *Unit*, as expected:

$X \triangleright () : Unit$

Representing objects

Let us consider a very simple example, that of a counter object which contains a private integer value and provides two functions to the outside world, namely *get*() to get the value of the integer and *inc*() to increment it. Once such a counter object is created, say *c*, one would like to be be able to write expressions like *c.get*() and *c.inc*(), with the obvious meaning.

In our framework, it is then natural to represent objects as records, containing a field for each operation allowed to be visible, or part of the interface, to the outside world. In our particular case, an already created counter object can be represented as a record $\{get = \ldots, inc = \ldots\}$. The concrete integer value is not intended to be accessed directly from the outside, so it does not have a corresponding field in the record; neverthless, one still wants to access is indirectly, using the field *get*.

For the time being, let us assume that the state of an object has already been created. In our case of the simple counter, since the state consists of just one integer, let us assume that is has already been created in an outer scope and that a reference *x* to it is available. Then the representation of our already created object is:

$$\{get = \lambda_ : Unit\ .\ !x,\ \ inc = \lambda_ : Unit\ .\ x := (!x + 1)\}$$

We will later see that the state of an object can be actually kept in a record (of references), which is made visible to the functions. As we already know by now, the evaluation strategies play a much more important role in the design of a language in the presence of side effects. Since OO programming is all about side effects, we impose a call-by-value evaluation strategy in our language. Also, function bodies are evaluated only when functions are called; othewise, the body of the function *inc* can be evaluated indefinitely and therefore the counter incremented in an uncontrolled manner.

Creating objects

One needs to be able to create an object before one can use it. Intuitively, creation of an object comes to intializing the state of the object. Considering again our example, we need to create an integer reference with some value and give that reference to the object. To achieve this, we can use a macro, say *createCounter*, defined as follows:

$$(\lambda\, x : Ref\ \mathrm{nat}\ .$$
$$\{get = \lambda_ : Unit\ .\ !x,\ inc = \lambda_ : Unit\ .\ x := succ(!x)\})$$
$$(ref\ 1)$$

Because of the assumed call-by-value evaluation strategy, each time the macro above is evaluated the following happen (recall also the definition of `FUN`): (1) a location is created and the integer value is stored at that location; (2) that location is bound to *x* in an environment, say *Env*, in which the record is evaluated; (3) the two functions in the record are evaluated to corresponding *closures*, each freezing the environment *Env* (further calls of these functions will therefore see the same location of *x*).

In short, this macro creates a counter object with the initial value of 1, which types as expected:

$$\emptyset \triangleright createCounter : \{get : Unit \to \mathrm{nat},\ \ inc : Unit \to Unit\}$$

One would like to have the possibility to create many objects with a similar functionality, in our case many counters, without having to write the creation code over an over again. In a similar fashion to the *new* construct in object oriented programming, one can define a *newCounter* macro as the λ-abstraction $\lambda_ : Unit\ .\ createCounter$. One can now create counter objects in any context, by simply invoking *newCounter*(). Note that a new state is indeed created with each invocation of *newCounter*.

Exercise 256. *Define another macro for creating counter objects, namely one which takes as argument an intenger and creates a counter object initialized to that integer. Therefore, newCounter(7) should create a counter object initialized to 7.*

Subtyping objects

We know from OO that objects that are instances of subclasses can also be regarded as instances of superclasses. Without yet discussing classes and sub/superclasses, let us first see how an object having more information than another one, is actually typed to a subtype of the later object in our framework.

Consider an enriched counter, which, besides the usual *get* and *inc* methods, has a method *reset* which resets the counter to 0. The new enriched counter objects have therefore the type:

$$\{get : Unit \to \text{nat},\ \ inc : Unit \to Unit,\ \ reset : Unit \to Unit\}$$

Therefore, the type of the enriched counters is a subtype of the simpler counters, which is consistent with the OO intuitions for objects and their types/classes. One can easily write a λ-abstraction for creating reset counter objects, say *newResetCounter*:

$$\begin{array}{l}
(\lambda_ : Unit\ . \\
\quad (\lambda\, x : Ref\,\text{nat}\ . \\
\quad\quad \{get = \lambda_ : Unit\ .\ !x, \\
\quad\quad\ \ inc = \lambda_ : Unit\ .\ x := (!x + 1), \\
\quad\quad\ \ reset = \lambda_ : Unit\ .\ x := 0\} \\
\quad)\ (ref\ 1) \\
)
\end{array}$$

Let us next understand how classes can be encoded in our typed high-order framework.

Classes

In OO programming languages, classes typically are regarded as the types of objects, while objects are regarded as instances of classes. As it turns out, these words can mean almost everything, depending upon context, who says them and who listens to them.

It is customary though to consider that classes contain a certain kind of functionality and structure, while an object of a class contains, or encapsulates, a concrete state together with a handle to the class that it is an instances of. By a state, one typically means a type-consistent assignment of values to fields.

With this model in mind, we can refine our definition of an object as follows.

1. First introduce a record type containing all the intended fields of a class; since the values of these fields may be changed during the life-time of an object, the types of the fields must be reference types. In the case of our counter we have only one field, so let us introduce the record type $\{x : Ref\,\text{nat}\}$ and call it *CounterRef*. Whenever an object is created as an instance of a class, one should first create a record allocating concrete values to the fields;

2. Next define the class itself as a λ-abstraction taking a value record type as above and adding methods to it to taste. In our case, we can define a class *CounterClass* as follows:

$$\begin{array}{l}
\lambda\, r : CounterRef\ . \\
\quad\quad \{get = \lambda_ : Unit\ .\ (r.x), \\
\quad\quad\ \ inc = \lambda_ : Unit\ .\ (r.x) := (r.x + 1)\}
\end{array}$$

Classes therefore evaluate to closures and type to function types taking field record types to method record type. For example, the class *CounterClass* types to

$$CounterRef \to \{get : Unit \to \text{nat},\ \ inc : Unit \to Unit\}$$

3. Create objects by passing desired field records to classes. In our case, a counter with integer value 1 is created by simply evaluating $Counter(\{x = ref\ 1\})$. Also, one can define *newCounter* as $\lambda_ : Unit\ .\ Counter(\{x = ref\ 1\})$, and so on.

This way, we therefore have a means to devise classes and then to create objects as instances of them. The type system of our calculus maintains a certain discipline in how objects are created and used, but it may still allow one to develop programs which one would rather want to reject.

Subclasses/Inheritance

Inheritance, or the process of extending existing classes with functionality, is an important, if not the most important, feature of the OO paradigm. The extended classes, despite the fact that they add functionality to the extended classes, are actually called *subclasses*. This terminology is consistent with our terminology for subtypes. Let us next see, again via an example, how we can simulate inheritance within our framework in a consistent way, in the sense that the type system will rank subclasses as subtypes.

Let us define a class *ResetCounterClass* which extends *CounterClass* by adding a reset method:

$$\begin{array}{l}
\lambda\, r : CounterRef\ . \\
\quad (\lambda\, super : ?\ . \\
\qquad \{get = super.get, \\
\qquad\ \ inc = super.inc, \\
\qquad\ \ reset = \lambda_ : Unit\ .\ (r.x = 0)\}) \\
\quad (CounterClass\ r)
\end{array}$$

Exercise 257. *What is the type of super above (the red question mark)? Type the ResetCounterClass above and show that it is a subtype of CounterClass.*

The use of *super* above is not required, but it helps to highlight the relationship between the subclass and the superclass.

However, although we would like to see a one-to-one relationship between the notions of subclass and corresponding subtype in our framework, the simulation of the OO paradigm above is so flexible that it actually allows one to also remove methods from the superclass, thus breaking the subtyping relationship, disallowing the use of the subclass where the superclass is expected, etc. To prevent that, we may need to provide more powerful mechanisms or primitives in the language, e.g., extends/inherits, together with appropriate typing rules/policies.

Exercise 258. *Comment on the encoding of OO programming in high-order typed languages presented above. Tell straight your opinion and think of desirable OO features that could not be supported this way. Also, show how self-reference can also be supported by allowing recursion (μ) in the language. How about dynamic versus static method dispatch?*

Bibliography

[1] Franz Baader and Tobias Nipkow. *Term rewriting and all that*. Cambridge University Press, New York, NY, USA, 1998.

[2] Leo Bachmair, Ta Chen, I. V. Ramakrishnan, Siva Anantharaman, and Jacques Chabin. Experiments with associative-commutative discrimination nets. In *IJCAI*, pages 348–355, 1995.

[3] J.-P. Banâtre, A. Coutant, and D. Le Métayer. A parallel machine for multiset transformation and its programming style. *Future Generation Computer Systems*, 4(2):133 – 144, 1988.

[4] Jean-Pierre Banâtre and Daniel Le Métayer. A new computational model and its discipline of programming. Technical Report INRIA-RR–566, Institut National de Recherche en Informatique et en Automatique (INRIA), 35 - Rennes (France), 1986.

[5] Jean-Pierre Banâtre and Daniel Le Métayer. Chemical reaction as a computational model. In *Functional Programming*, Workshops in Computing, pages 103–117. Springer, 1989.

[6] Jean-Pierre Banâtre and Daniel Le Métayer. The gamma model and its discipline of programming. *Sci. Comput. Program.*, 15(1):55–77, 1990.

[7] Jan Bergstra and J. V. Tucker. Equational specifications, complete term rewriting systems, and computable and semicomputable algebras. *Journal of the Association for Computing Machinery*, 42(6):1194–1230, 1995.

[8] Gérard Berry and Gérard Boudol. The chemical abstract machine. In *POPL*, pages 81–94, 1990.

[9] Gérard Berry and Gérard Boudol. The chemical abstract machine. *Theoretical Computer Science*, 96(1):217–248, 1992.

[10] Peter Borovanský, Horatiu Cirstea, Hubert Dubois, Claude Kirchner, Hélène Kirchner, Pierre-Etienne Moreau, Christophe Ringeissen, and Marian Vittek. ELAN *V 3.4 User Manual*. LORIA, Nancy (France), fourth edition, January 2000.

[11] Peter Borovanský, Claude Kirchner, Hélène Kirchner, and Pierre-Etienne Moreau. ELAN from a rewriting logic point of view. *Theoretical Computer Science*, 285(2):155–185, 2002.

[12] Peter Borovanský, Claude Kirchner, Hélène Kirchner, Pierre-Etienne Moreau, and Christophe Ringeissen. An overview of ELAN. *ENTCS*, 15, 1998.

[13] Christiano Braga. *Rewriting Logic as a Semantic Framework for Modular Structural Operational Semantics*. PhD thesis, Departamento de Informática, Pontificia Universidade Católica de Rio de Janeiro, Brasil, 2001.

[14] Christiano Braga and José Meseguer. Modular rewriting semantics in practice. *Electr. Notes Theor. Comput. Sci.*, 117:393–416, 2005.

[15] Christiano de O. Braga, E. Hermann Hæusler, José Meseguer, and Peter D. Mosses. Mapping modular sos to rewriting logic. In *LOPSTR'02: Proceedings of the 12th international conference on Logic based program synthesis and transformation*, pages 262–277, Berlin, Heidelberg, 2003. Springer-Verlag.

[16] Fabricio Chalub and Christiano Braga. Maude MSOS tool. In Grit Denker and Carolyn Talcott, editors, *Proceedings of the Sixth International Workshop on Rewriting Logic and its Applications (WRLA 2006)*, volume 176(4) of *Electronic Notes in Theoretical Computer Science*, pages 133–146. Elsevier, 2007.

[17] M. Clavel, F. Durán, S. Eker, J. Meseguer, P. Lincoln, N. Martí-Oliet, and C. Talcott. *All About Maude, A High-Performance Logical Framework*, volume 4350 of *Lecture Notes in Computer Science*. Springer, 2007.

[18] Manuel Clavel, Francisco Durán, Steven Eker, Patrick Lincoln, Narciso Martí-Oliet, José Meseguer, and Jose F. Quesada. Maude: specification and programming in rewriting logic. *Theoretical Computer Science*, 285(2):187–243, 2002.

[19] Oliver Danvy and Lasse R. Nielsen. Refocusing in reduction semantics. Technical Report BRICS RS-04-26, University of Aarhus, November 2004.

[20] Olivier Danvy and Lasse R. Nielsen. Syntactic theories in practice. In *Second International Workshop on Rule-Based Programming (RULE 2001)*, volume 59(4) of *ENTCS*, pages 358–374, 2001.

[21] Răzvan Diaconescu and Kokichi Futatsugi. *CafeOBJ Report. The Language, Proof Techniques, and Methodologies for Object-Oriented Algebraic Specification*, volume 6 of *AMAST Series in Computing*. World Scientific, 1998.

[22] Hartmut Ehrig. Introduction to the algebraic theory of graph grammars (a survey). In *International Workshop on Graph-Grammars and Their Application to Computer Science and Biology*, volume 73 of *Lecture Notes in Computer Science*, pages 1–69. Springer, 1979.

[23] Azadeh Farzan, Feng Chen, José Meseguer, and Grigore Rosu. Formal analysis of Java programs in JavaFAN. In Rajeev Alur and Doron Peled, editors, *Computer Aided Verification, 16th International Conference, CAV 2004, Boston, MA, USA, July 13-17, 2004, Proceedings*, volume 3114 of *Lecture Notes in Computer Science*, pages 501–505. Springer, 2004.

[24] Matthias Felleisen, Robert Bruce Findler, and Matthew Flatt. *Semantics Engineering with PLT Redex*. MIT Press, 2009.

[25] Matthias Felleisen and Robert Hieb. A revised report on the syntactic theories of sequential control and state. *Theoretical Computer Science*, 103(2):235–271, 1992.

[26] J. A. Goguen, J. W. Thatcher, E. G. Wagner, and J. B. Wright. Initial algebra semantics and continuous algebras. *J. ACM*, 24:68–95, January 1977.

[27] Joseph Goguen and José Meseguer. Completeness of many-sorted equational logic. *Houston Journal of Mathematics*, 11(3):307–334, 1985. Preliminary versions have appeared in: *SIGPLAN Notices*, July 1981, Volume 16, Number 7, pages 24–37; SRI Computer Science Lab, Report CSL-135, May

1982; and Report CSLI-84-15, Center for the Study of Language and Information, Stanford University, September 1984.

[28] Joseph Goguen, Timothy Winkler, José Meseguer, Kokichi Futatsugi, and Jean-Pierre Jouannaud. Introducing OBJ. In *Software Engineering with OBJ: algebraic specification in action*. Kluwer, 2000.

[29] Joseph A. Goguen and Grant Malcolm. *Algebraic Semantics of Imperative Programs*. Foundations of Computing. The MIT Press, May 1996.

[30] Carl A. Gunter and Dana S. Scott. Semantic domains. In *Handbook of Theoretical Computer Science, Volume B: Formal Models and Sematics (B)*, pages 633–674. MIT Press / Elsevier, 1990.

[31] Yuri Gurevich. Evolving algebras 1993: Lipari guide. In *Specification and validation methods*, pages 9–36. Oxford University Press, Inc., New York, NY, USA, 1995.

[32] Matthew Hennessy. *The Semantics of Programming Languages: an Elementary Introduction using Structural Operational Semantics*. John Wiley and Sons, New York, N.Y., 1990.

[33] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation (3rd Edition)*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2006.

[34] Gilles Kahn. Natural semantics. In Franz-Josef Brandenburg, Guy Vidal-Naquet, and Martin Wirsing, editors, *STACS 87, 4th Annual Symposium on Theoretical Aspects of Computer Science, Passau, Germany, February 19-21, 1987, Proceedings*, volume 247 of *Lecture Notes in Computer Science*, pages 22–39. Springer, 1987.

[35] Narciso Martí-Oliet and José Meseguer. Rewriting logic as a logical and semantic framework. In D. Gabbay and F. Guenthner, editors, *Handbook of Philosophical Logic, 2nd. Edition*, pages 1–87. Kluwer Academic Publishers, 2002. First published as SRI Tech. Report SRI-CSL-93-05, August 1993. Second published in Electronic Notes in Theoretical Computer Science, Volume 4, 1996.

[36] Narciso Martí-Oliet and José Meseguer. Rewriting logic: roadmap and bibliography. *Theoretical Computer Science*, 285:121–154, 2002.

[37] Jacob Matthews, Robert Bruce Findler, Matthew Flatt, and Matthias Felleisen. A visual environment for developing context-sensitive term rewriting systems. In *Proceedings of 15th International Conference on Rewriting Techniques and Applications, (RTA'04)*, volume 3091 of *Lecture Notes in Computer Science*, pages 301–311, 2004.

[38] Jacob Matthews, Robert Bruce Findler, Matthew Flatt, and Matthias Felleisen. A visual environment for developing context-sensitive term rewriting systems. In Vincent van Oostrom, editor, *RTA*, volume 3091 of *Lecture Notes in Computer Science*, pages 301–311. Springer, 2004.

[39] José Meseguer. Conditional rewriting logic: Deduction, models and concurrency. In *Conditional and Typed Rewriting Systems (CTRS'90)*, volume 516 of *Lecture Notes in Computer Science*, pages 64–91. Springer, 1990.

[40] José Meseguer. A logical theory of concurrent objects. In *OOPSLA/ECOOP*, pages 101–115, 1990.

[41] José Meseguer. Rewriting as a unified model of concurrency. In *Theories of Concurrency: Unification and Extension (CONCUR'90)*, volume 458 of *Lecture Notes in Computer Science*, pages 384–400. Springer, 1990.

[42] José Meseguer. Conditional rewriting logic as a unified model of concurrency. *Theoretical Computer Science*, 96(1):73–155, 1992.

[43] Josè Meseguer. Rewriting logic as a semantic framework for concurrency: a progress report. In Ugo Montanari and Vladimiro Sassone, editors, *CONCUR '96: Concurrency Theory*, volume 1119 of *Lecture Notes in Computer Science*, pages 331–372. Springer Berlin / Heidelberg, 1996.

[44] José Meseguer and Christiano Braga. Modular rewriting semantics of programming languages. In Charles Rattray, Savi Maharaj, and Carron Shankland, editors, *Algebraic Methodology and Software Technology, 10th International Conference, AMAST 2004, Stirling, Scotland, UK, July 12-16, 2004, Proceedings*, volume 3116 of *Lecture Notes in Computer Science*, pages 364–378. Springer, 2004.

[45] José Meseguer and Grigore Rosu. Rewriting logic semantics: From language specifications to formal analysis tools. In *Proceedings of the 2nd International Joint Conference on Automated Reasoning (IJCAR'04)*, volume 3097 of *Lecture Notes in Computer Science*, pages 1–44. Springer, 2004.

[46] José Meseguer and Grigore Rosu. Rewriting logic semantics: From language specifications to formal analysis tools. In David A. Basin and Michaël Rusinowitch, editors, *Automated Reasoning - Second International Joint Conference, IJCAR 2004, Cork, Ireland, July 4-8, 2004, Proceedings*, volume 3097 of *Lecture Notes in Computer Science*, pages 1–44. Springer, 2004.

[47] José Meseguer and Grigore Rosu. The rewriting logic semantics project. *J. TCS*, 373(3):213–237, 2007. Also appeared in *SOS '05*, volume 156(1) of *ENTCS*, pages 27–56, 2006.

[48] José Meseguer and Grigore Rosu. The rewriting logic semantics project. *Theoretical Computer Science*, 373(3):213–237, 2007.

[49] Robin Milner, Mads Tofte, Robert Harper, and David Macqueen. *The Definition of Standard ML (Revised)*. MIT Press, Cambridge, MA, USA, 1997.

[50] Peter D. Mosses. Denotational semantics. In *Handbook of Theoretical Computer Science, Volume B: Formal Models and Sematics (B)*, pages 575–631. MIT Press / Elsevier, 1990.

[51] Peter D. Mosses. Foundations of modular sos. In Miroslaw Kutylowski, Leszek Pacholski, and Tomasz Wierzbicki, editors, *MFCS*, volume 1672 of *Lecture Notes in Computer Science*, pages 70–80. Springer, 1999.

[52] Peter D. Mosses. Pragmatics of modular SOS. In Hélène Kirchner and Christophe Ringeissen, editors, *Algebraic Methodology and Software Technology, 9th International Conference, AMAST 2002, Saint-Gilles-les-Bains, Reunion Island, France, September 9-13, 2002, Proceedings*, volume 2422 of *Lecture Notes in Computer Science*, pages 21–40. Springer, 2002.

[53] Peter D. Mosses. Modular structural operational semantics. *Journal of Logic and Algebraic Programming*, 60-61:195–228, 2004.

[54] Peter D. Mosses and Mark J. New. Implicit propagation in structural operational semantics. *Electronic Notes in Theoretical Computer Science*, 229(4):49–66, 2009.

[55] Tobias Nipkow. Winskel is (almost) right: Towards a mechanized semantics. *Formal Asp. Comput.*, 10(2):171–186, 1998.

[56] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL - A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer, 2002.

[57] Nikolaos S. Papaspyrou. Denotational semantics of ANSI C. *Computer Standards and Interfaces*, 23(3):169–185, 2001.

[58] Gheorghe Paun. Computing with membranes. *Journal of Computer and System Sciences*, 61:108–143, 2000.

[59] G. D. Plotkin. A powerdomain construction. *SIAM J. of Computing*, 5(3):452–487, September 1976.

[60] Gordon D. Plotkin. A structural approach to operational semantics. Technical Report DAIMI FN-19, University of Aarhus, 1981. Republished in Journal of Logic and Algebraic Programming, Volume 60-61, 2004.

[61] Gordon D. Plotkin. A structural approach to operational semantics. *Journal of Logic and Algebraic Programming*, 60-61:17–139, 2004.

[62] Emil L. Post. Finite combinatory processes—formulation 1. *The Journal of Symbolic Logic*, 1(3):pp. 103–105, 1936.

[63] John C. Reynolds. The discoveries of continuations. *Lisp Symbolic Computation*, 6:233–248, November 1993.

[64] Grigore Rosu. Cs322, fall 2003 - programming language design: Lecture notes. Technical Report UIUCDCS-R-2003-2897, University of Illinois at Urbana-Champaign, Department of Computer Science, December 2003. Lecture notes of a course taught at UIUC.

[65] Grigore Rosu. Equality of streams is a pi_2^0-complete problem. In *Proceedings of the 11th ACM SIGPLAN International Conference on Functional Programming (ICFP'06)*. ACM, 2006.

[66] Grigore Rosu. K: a Rewrite-based Framework for Modular Language Design, Semantics, Analysis and Implementation. Technical Report UIUCDCS-R-2006-2802, Computer Science Department, University of Illinois at Urbana-Champaign, 2006. A previous version of this work has been published as technical report UIUCDCS-R-2005-2672 in 2005. K was first introduced in 2003, in the technical report UIUCDCS-R-2003-2897: lecture notes of CS322 (programming language design).

[67] Hartley Rogers Jr. *Theory of Recursive Functions and Effective Computability*. MIT press, Cambridge, MA, 1987.

[68] Grigore Rosu and Traian Florin Serbănuță. An overview of the K semantic framework. *Journal of Logic and Algebraic Programming*, 79(6):397–434, 2010.

[69] David A. Schmidt. *Denotational semantics: a methodology for language development*. William C. Brown Publishers, Dubuque, IA, USA, 1986.

[70] Dana Scott and Christopher Strachey. Toward a mathematical semantics for computer languages. Programming Research Group Technical Monograph PRG-6, Oxford University Computing Laboratory, 1971.

[71] Dana S. Scott. Outline of a mathematical theory of computation. Technical Monograph PRG–2, Oxford University Computing Laboratory, Oxford, England, November 1970.

[72] R. C. Sekar, R. Ramesh, and I. V. Ramakrishnan. Adaptive pattern matching. In Werner Kuich, editor, *ICALP*, volume 623 of *Lecture Notes in Computer Science*, pages 247–260. Springer, 1992.

[73] R. C. Sekar, R. Ramesh, and I. V. Ramakrishnan. Adaptive pattern matching. *SIAM J. Comput.*, 24(6):1207–1234, 1995.

[74] Traian Florin Serbănută, Grigore Rosu, and José Meseguer. A rewriting logic approach to operational semantics. *Information and Computation*, 207:305–340, 2009.

[75] Traian Florin Serbănută, Gheorghe Stefănescu, and Grigore Rosu. Defining and executing P systems with structured data in K. In David W. Corne, Pierluigi Frisco, Gheorghe Paun, Grzegorz Rozenberg, and Arto Salomaa, editors, *Workshop on Membrane Computing (WMC'08)*, volume 5391 of *Lecture Notes in Computer Science*, pages 374–393. Springer, 2009.

[76] Michael Sipser. *Introduction to the Theory of Computation*. International Thomson Publishing, 1996.

[77] Christopher Strachey. Towards a formal semantics. In *Proceedings of IFIP TC2 Working Conference on Formal Language Description Languages for Computer Programming*, pages 198–220. North Holland, Amsterdam, 1966.

[78] Christopher Strachey. Fundamental concepts in programming languages. *Higher-Order and Symbolic Computation*, 13:11–49, 2000. Lecture Notes for a 1967 NATO International Summer School in Computer Programming, Copenhagen; also available from Programming Research Group, University of Oxford, August 1967.

[79] Christopher Strachey and Christopher P. Wadsworth. Continuations: A mathematical semantics for handling full jumps. *Higher-Order and Symbolic Computation*, 13:135–152, 2000. Reprinted version of 1974 Programming Research Group Technical Monograph PRG-11, Oxford University Computing Laboratory.

[80] Alan M. Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, 2(42):230–265, 1937.

[81] Mark van den Brand, Jan Heering, Paul Klint, and Pieter A. Olivier. Compiling language definitions: the asf+sdf compiler. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 24(4):334–368, 2002.

[82] Mark G. J. van den Brand, J. Heering, P. Klint, and P. A. Olivier. Compiling language definitions: the ASF+SDF compiler. *ACM TOPLAS*, 24(4):334–368, 2002.

[83] Alberto Verdejo and Narciso Martí-Oliet. Executable structural operational semantics in Maude. Technical Report 134-03, Departamento de Sistemas Informàticos y Programaciòn, Universidad Complutense de Madrid, 2003.

[84] Alberto Verdejo and Narciso Martí-Oliet. Executable structural operational semantics in maude. *J. Log. Algebr. Program.*, 67(1-2):226–293, 2006.

[85] Eelco Visser. Program Transf. with Stratego/XT: Rules, Strategies, Tools, and Systems. In *Domain-Specific Program Generation*, pages 216–238, 2003.

[86] Philip Wadler. The essence of functional programming. In *"Proceedings of the 19th ACM SIGPLAN-SIGACT symposium on Principles of Programming Languages"*, ACM, pages 1–14, 1992.

[87] Glynn Winskel. *The formal semantics of programming languages: an introduction*. MIT Press, Cambridge, MA, USA, 1993.

[88] Andrew K. Wright and Matthias Felleisen. A syntactic approach to type soundness. *Information and Computation*, 115(1):38–94, 1994.

[89] Yong Xiao, Zena M. Ariola, and Michael Mauny. From syntactic theories to interpreters: A specification language and its compilation. In *First International Workshop on Rule-Based Programming (RULE 2000)*, 2000.

[90] Yong Xiao, Amr Sabry, and Zena M. Ariola. From syntactic theories to interpreters: Automating the proof of unique decomposition. *Higher Order and Symbolic Computation*, 14(4):387–409, 2001.

discuss defining complete languages versus constructs, separated in modules

new approaches versus traditional approaches; after all, RL is only 10 years younger than SOS . . .

model-based semantics in addition to an operational one; discuss the difference, because apparently the initial model is the transition system advocated by SOS . . .

additional notation brings additional benefits; one is not forced to use, but one may find advantages in using it

sometimes an order of application of rules may be desired; add strategy language?

Appendix A

Maude Stuff

A.1 Defining "Builtin" Modules for Maude Language Definitions

add a section on how to do the definition in Figure 2.12 in Maude, including the code for it; make sure that, for executability reasons, we add strategies to the function construct and the conditional.

This section needs to be completely rewritten, to be in synch with the new builtins in Figure A.1.

When defining language semantics in Maude, for example using the operational semantics styles discussed in Chapter 3, one may naturally want to use the existing Maude data-types and libraries, particularly because they are well-engineered and thus can lead to efficient interpreters and tools for the defined languages. However, one needs to exercise care when importing Maude builtin modules, because it is very difficult, if not impossible, to reuse the names of the builtin operations as programming language constructs. The reason is that there is a very high chance that one wants to give those language constructs different attributes from those of the builtin operations, but Maude, for good reasons that go beyond our purpose in this book, does not allow it. For example, one may want to include an addition operation `_+_` in one's language, but one may not want it to be associative, commutative and have `0` as identity, like the `_+_` operation provided by the builtin Maude module `INT`.

In this section we give some Maude 2.4 specific hints on how one can appropriately redefine the Maude builtin modules to work smoothly with the Maude language semantics in Chapter 3. We advise the reader to run Maude without including its prelude, that is, with the command:

```
maude -no-prelude  <language-semantics>.maude
```

This way, no warnings will be issued when we redefine the builtin modules `INT` and `BOOL`. If for some reason one really needs to include the Maude prelude at start, then one should at least make sure that the `BOOL` module is not automatically included in other modules using the Maude command:

FIXTHIS

Figure A.1: The Maude builtins used in the IMP semantics in Chapter 3.

```
set include BOOL off .
 --- not needed if one runs maude with flag -no-prelude
```

We can now define our own and safe BOOL module as follows:

```
mod BOOL is
  sort Bool .
  op true : -> Bool [ctor special (id-hook SystemTrue)] .
  op false : -> Bool [ctor special (id-hook SystemFalse)] .
  op if_then_else_fi : Bool Universal Universal -> Universal [poly (2 3 0)
    special (id-hook BranchSymbol  term-hook 1 (true)  term-hook 2 (false))] .
  op _==_ : Universal Universal -> Bool [poly (1 2) prec 51 special (
    id-hook EqualitySymbol  term-hook equalTerm (true)  term-hook notEqualTerm (false))] .
  op _=/=_ : Universal Universal -> Bool [poly (1 2) prec 51 special (
    id-hook EqualitySymbol  term-hook equalTerm (false)  term-hook notEqualTerm (true))] .
endm
```

The module above includes only a few components from the actual Maude builtin BOOL module. More precisely, it discards all the specific Boolean operators for conjunction, negation, implication, etc., because one may want to define these as language constructs in one's language, possibly having a different semantics. For example, the conjunction _and_ of IMP in Chapter 3 is short-circuited, so in particular it is *not* commutative (while the Maude BOOL builtin _and_ is commutative). If one really needs to include the original Maude BOOL builtin operations, then we recommend that one change their names, like we do within the next module.

Like the module BOOL above, the module INT below defines only a portion of the Maude INT builtin module of integers. Unlike in BOOL, this time we also include some library operations on integers as part of the new builtin INT, but, however, we change their names by appending the word "Int" to their original names, e.g., _+Int_, _<=Int_, etc.:

```
mod INT is including BOOL .
  sort Int .
  op 0 : -> Int [ctor] .
  op s_ : Int -> Int [iter ctor special (id-hook SuccSymbol  term-hook zeroTerm (0))] .
  op -Int_ : Int -> Int [ctor special (id-hook MinusSymbol
    op-hook succSymbol (s_ : Int ~> Int)  op-hook minusSymbol (-Int_ : Int ~> Int))] .
  op _+Int_ : Int Int -> Int [assoc comm prec 33 special (id-hook ACU_NumberOpSymbol (+)
    op-hook succSymbol (s_ : Int ~> Int)  op-hook minusSymbol (-Int_ : Int ~> Int))] .
  op _/Int_ : Int Int -> Int [prec 31 gather (E e) special (id-hook NumberOpSymbol (quo)
    op-hook succSymbol (s_ : Int ~> Int))] .
  op _<=Int_ : Int Int -> Bool [prec 37 special (id-hook NumberOpSymbol (<=)
    op-hook succSymbol (s_ : Int ~> Int)  op-hook minusSymbol (-Int_ : Int ~> Int)
    term-hook trueTerm (true)  term-hook falseTerm (false))] .
endm
```

This way, one can still have full access to Maude's builtin libraries, but one elegantly avoids name clashes with homonymous operations that one may want to define as part of one's language syntax.

Finally, we define the VAR module. As discussed in Section 3.1.1, we want it to provide a sort Id and at least each letter in the alphabet as a constant of sort Id to be used in writing readable programs. We additionally include all the Maude builtin quoted identifiers (e.g., 'abc, 'abc123, etc.) as variables, so that one can use them as well:

```
mod VAR is
  sort Id .
  ops a b c d e f g h i j k l m n o p q r s t u v w x y z : -> Id [format (g o)] .
  op <Qids> : -> Id [special (id-hook QuotedIdentifierSymbol)] .
```

```
  sort List{Id} .
  subsort Id < List{Id} .
  op .List{Id} : -> List{Id} .
  op _,_ : List{Id} List{Id} -> List{Id} [assoc id: .List{Id}] .
endm
```

Note that the module above also includes the Maude definition of lists of variables; these may be needed for variable declarations, as we did in Section 3.1.1 for IMP.